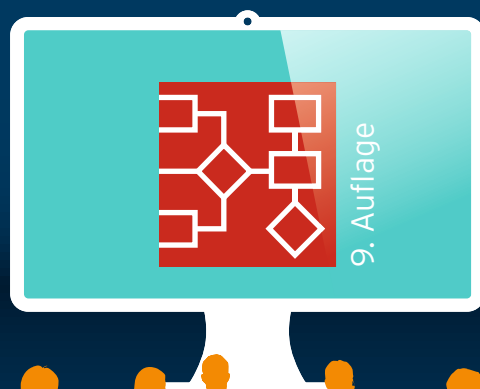




Effektive Software- architekturen

gernot STARKE

Ein praktischer Leitfaden



Ideal zur Vorbereitung auf die
iSAQB-Zertifizierung

HANSER

INOQ

Starke

Effektive Softwarearchitekturen



Bleiben Sie auf dem Laufenden!

Unser **Computerbuch-Newsletter** informiert Sie monatlich über neue Bücher und Termine. Profitieren Sie auch von Gewinnspielen und exklusiven Leseproben. Gleich anmelden unter:

www.hanser-fachbuch.de/newsletter



Gernot Starke

Effektive Softwarearchitekturen

Ein praktischer Leitfaden

9., überarbeitete Auflage

HANSER

Der Autor:

Dr. Gernot Starke, Köln

INNOQ Fellow

www.gernotstarke.de

Alle in diesem Buch enthaltenen Informationen, Verfahren und Darstellungen wurden nach bestem Wissen zusammengestellt und mit Sorgfalt getestet. Dennoch sind Fehler nicht ganz auszuschließen. Aus diesem Grund sind die im vorliegenden Buch enthaltenen Informationen mit keiner Verpflichtung oder Garantie irgendeiner Art verbunden. Autor und Verlag übernehmen infolgedessen keine juristische Verantwortung und werden keine daraus folgende oder sonstige Haftung übernehmen, die auf irgendeine Art aus der Benutzung dieser Informationen – oder Teilen davon – entsteht.

Ebenso übernehmen Autor und Verlag keine Gewähr dafür, dass beschriebene Verfahren usw. frei von Schutzrechten Dritter sind. Die Wiedergabe von Gebrauchsnamen, Handelsnamen, Warenbezeichnungen usw. in diesem Buch berechtigt deshalb auch ohne besondere Kennzeichnung nicht zu der Annahme, dass solche Namen im Sinne der Warenzeichen- und Markenschutz-Gesetzgebung als frei zu betrachten wären und daher von jedermann benutzt werden dürften.



Bibliografische Information der Deutschen Nationalbibliothek:

Die Deutsche Nationalbibliothek verzeichnet diese Publikation in der Deutschen Nationalbibliografie; detaillierte bibliografische Daten sind im Internet über <http://dnb.d-nb.de> abrufbar.

Dieses Werk ist urheberrechtlich geschützt.

Alle Rechte, auch die der Übersetzung, des Nachdruckes und der Vervielfältigung des Buches, oder Teilen daraus, vorbehalten. Kein Teil des Werkes darf ohne schriftliche Genehmigung des Verlages in irgendeiner Form (Fotokopie, Mikrofilm oder ein anderes Verfahren) – auch nicht für Zwecke der Unterrichtsgestaltung – reproduziert oder unter Verwendung elektronischer Systeme verarbeitet, vervielfältigt oder verbreitet werden.

© 2020 Carl Hanser Verlag München, www.hanser-fachbuch.de

Lektorat: Brigitte Bauer-Schiewek

Copy editing: Petra Kienle, Fürstenfeldbruck

Umschlagdesign: Marc Müller-Bremer, www.rebranding.de, München

Umschlagrealisation: Max Kostopoulos, unter Verwendung von Grafiken für das Titelmotiv:

© shutterstock.com/Rawpixel.com und Vector VA

Layout: Manuela Treindl, Fürth

Gesamtherstellung: Kösel, Krugzell

Ausstattung patentrechtlich geschützt. Kösel FD 351, Patent-Nr. 0748702

Printed in Germany

Print-ISBN: 978-3-446-46376-9

E-Book-ISBN: 978-3-446-46589-3

E-Pub-ISBN: 978-3-446-46690-6

Inhalt

Vorwort	XIII
Vorwort zur neunten Auflage	XIV
1 Einleitung	1
1.1 Softwarearchitekt(inn)en	5
1.2 Effektiv, agil und pragmatisch	6
1.3 Wer sollte dieses Buch lesen?	9
1.4 Wegweiser durch das Buch	10
1.5 Webseite zum Buch	12
1.6 Weiterführende Literatur	12
1.7 Danksagung	13
2 Architektur und Architekten	15
2.1 Was ist Softwarearchitektur?	16
2.2 Die Aufgaben von Softwarearchitekten	21
2.3 Wie entstehen Architekturen?	26
2.4 In welchem Kontext steht Architektur?	29
2.5 Weiterführende Literatur	32
3 Vorgehen bei der Architekturentwicklung	33
3.1 Informationen sammeln	37
3.2 Anforderungen klären	38
3.2.1 Was ist die Kernaufgabe des Systems?	38
3.2.2 Welche Kategorie von System?	39
3.2.3 Wesentliche Qualitätsanforderungen ermitteln	39
3.2.4 Relevante Stakeholder ermitteln	44
3.2.5 Fachlichen und technischen Kontext ermitteln	45
3.3 Einflussfaktoren und Randbedingungen ermitteln	47
3.4 Entwerfen und kommunizieren	53
3.5 Umsetzung begleiten	54
3.6 Lösungsstrategien entwickeln	55
3.7 Weiterführende Literatur	57

4	Entwurf: Grundlagen, Methoden und Muster	59
4.1	Grundlagen	61
4.1.1	Grundsätze des Entwurfs (Maxime)	62
4.1.2	Prinzipien	65
4.1.3	SOLID-Prinzipien des objektorientierten Entwurfs	71
4.1.3.1	Offen-Geschlossen-Prinzip	71
4.1.3.2	Liskov-Substitutionsprinzip (LSP)	73
4.1.3.3	Interface Segregation Principle (ISP)	74
4.1.3.4	Dependency Inversion Principle (DIP)	76
4.2	Heuristiken	79
4.3	Entwurfsmethoden	84
4.3.1	Domain-Driven Design (Entwurf nach Fachlichkeit)	84
4.3.2	Quality-Driven Software Architecture	89
4.3.3	Top-down und Bottom-up	97
4.4	Schnittstellen entwerfen	98
4.4.1	Anforderungen an Schnittstellen	99
4.4.2	Worauf müssen Sie achten?	100
4.4.3	Tipps zum Entwurf von Schnittstellen	101
4.5	Architekturstile und -muster	102
4.5.1	Datenflussarchitekturstil	103
4.5.1.1	Architekturstil Batch-Sequentiell	103
4.5.1.2	Architekturstil Pipes und Filter	104
4.5.2	Datenzentrierter Architekturstil	107
4.5.2.1	Repository	107
4.5.2.2	Blackboard	108
4.5.3	Hierarchische Architekturstile	109
4.5.3.1	Master-Slave	109
4.5.3.2	Schichten (Layer)	110
4.5.3.3	Architekturstil Ports-und-Adapter	113
4.5.4	Architekturstile verteilter Systeme	116
4.5.4.1	Client-Server	117
4.5.4.2	Command Query Responsibility Segregation	117
4.5.4.3	Broker	119
4.5.4.4	Peer-to-Peer	120
4.5.5	Ereignisbasierte Systeme – Event Systems	121
4.5.5.1	Ungepufferte Event-Kommunikation	122
4.5.5.2	Message- oder Event-Queue-Architekturen	122
4.5.5.3	Message-Service-Architekturen	123
4.5.6	Interaktionsorientierte Systeme	124
4.5.6.1	Model-View-Controller	124
4.5.6.2	Presentation Model	125
4.5.7	Weitere Architekturstile und -muster	128
4.6	Entwurfsmuster	130
4.6.1	Entwurf mit Mustern	130

4.6.2	Adapter	131
4.6.3	Beobachter (Observer)	132
4.6.4	Dekorierer (Decorator)	133
4.6.5	Stellvertreter (Proxy)	134
4.6.6	Fassade	135
4.6.7	Zustand (State)	136
4.7	Weiterführende Literatur	137
5	Kommunikation und Dokumentation von Architekturen	139
5.1	Architekten müssen kommunizieren und dokumentieren	140
5.2	Effektive Architekturdokumentation	142
5.2.1	Anforderungen an Architekturdokumentation	142
5.2.2	Regeln für gute Architekturdokumentation	144
5.3	Typische Architekturdokumente	147
5.3.1	Zentrale Architekturbeschreibung	148
5.3.2	Architekturüberblick	151
5.3.3	Dokumentationsübersicht	151
5.3.4	Übersichtspräsentation der Architektur	151
5.3.5	Architekturtapete	152
5.4	Sichten	152
5.4.1	Sichten in der Softwarearchitektur	153
5.4.2	Vier Arten von Sichten	155
5.4.3	Entwurf der Sichten	157
5.5	Kontextabgrenzung	159
5.5.1	Elemente der Kontextabgrenzung	159
5.5.2	Notation der Kontextabgrenzung	160
5.5.3	Entwurf der Kontextabgrenzung	160
5.6	Bausteinsicht	161
5.6.1	Elemente der Bausteinsicht	165
5.6.2	Notation der Bausteinsicht	166
5.6.3	Entwurf der Bausteinsicht	167
5.7	Laufzeitsicht	168
5.7.1	Elemente der Laufzeitsicht	169
5.7.2	Notation der Laufzeitsicht	170
5.7.3	Entwurf der Laufzeitsicht	171
5.8	Verteilungssicht	171
5.8.1	Elemente der Verteilungssicht	172
5.8.2	Notation der Verteilungssicht	172
5.8.3	Entwurf der Verteilungssicht	173
5.9	Dokumentation von Schnittstellen	174
5.10	Dokumentation technischer Konzepte	177
5.11	Werkzeuge zur Dokumentation	177
5.12	TOGAF zur Architekturdokumentation	179
5.13	Weiterführende Literatur	181

6	Modellierung für Softwarearchitekten	183
6.1	Modelle als Arbeitsmittel	183
6.1.1	Grafische oder textuelle Modellierung	185
6.2	UML 2 für Softwarearchitekten	186
6.2.1	Die Diagrammarten der UML 2	187
6.2.2	Die Bausteine von Architekturen	189
6.2.3	Schnittstellen	190
6.2.4	Die Bausteinsicht	191
6.2.5	Die Verteilungssicht	193
6.2.6	Die Laufzeitsicht	195
6.2.7	Darum UML	198
6.2.8	Darum nicht UML	199
6.3	Tipps zur Modellierung	199
6.4	Weiterführende Literatur	200
7	Technische Konzepte und typische Architekturasspekte	201
7.1	Persistenz	205
7.1.1	Motivation	205
7.1.2	Einflussfaktoren und Entscheidungskriterien	208
7.1.2.1	Art der zu speichernden Daten	209
7.1.2.2	Konsistenz und Verfügbarkeit (ACID, BASE oder CAP)	210
7.1.2.3	Zugriff und Navigation	212
7.1.2.4	Deployment und Betrieb	212
7.1.3	Lösungsmuster	213
7.1.3.1	Persistenzschicht	213
7.1.3.2	DAO: Eine Miniatur-Persistenzschicht	217
7.1.4	Bekannte Risiken und Probleme	218
7.1.5	Weitere Themen zu Persistenz	219
7.1.6	Zusammenhang zu anderen Aspekten	223
7.1.7	Praktische Vertiefung	224
7.1.8	Weiterführende Literatur	225
7.2	Geschäftsregeln	226
7.2.1	Motivation	226
7.2.2	Funktionsweise von Regelmaschinen	229
7.2.3	Kriterien pro & kontra Regelmaschinen	231
7.2.4	Mögliche Probleme	232
7.2.5	Weiterführende Literatur	233
7.3	Integration	233
7.3.1	Motivation	233
7.3.2	Typische Probleme	235
7.3.3	Lösungskonzepte	236
7.3.4	Entwurfsmuster zur Integration	240
7.3.5	Zusammenhang mit anderen Aspekten	241
7.3.6	Weiterführende Literatur	242

7.4	Verteilung	242
7.4.1	Motivation	242
7.4.2	Typische Probleme	243
7.4.3	Lösungskonzept	243
7.4.4	Konsequenzen und Risiken	245
7.4.5	Zusammenhang mit anderen Aspekten	245
7.4.6	Weiterführende Literatur	245
7.5	Kommunikation	246
7.5.1	Motivation	246
7.5.2	Entscheidungsalternativen	246
7.5.3	Grundbegriffe der Kommunikation	246
7.5.4	Weiterführende Literatur	250
7.6	Grafische Oberflächen (GUI)	251
7.6.1	Motivation	251
7.6.2	Einflussfaktoren und Entscheidungskriterien	251
7.6.3	GUI-relevante Architekturmuster	253
7.6.4	Struktur und Ergonomie von Benutzeroberflächen	254
7.6.5	Bekannte Risiken und Probleme	255
7.6.6	Zusammenhang zu anderen Aspekten	257
7.7	Geschäftsprozess-Management: Ablaufsteuerung im Großen	258
7.7.1	Workflow-Sprachen	260
7.7.2	Vorhersagbarkeit	263
7.7.3	Zweck der Ablaufsteuerung	264
7.7.4	Lösungsansätze	266
7.7.5	Integration von Workflow-Systemen	269
7.7.6	Mächtigkeit von WfMS	270
7.7.7	Weiterführende Literatur	270
7.8	Sicherheit	271
7.8.1	Motivation – Was ist IT-Sicherheit?	271
7.8.2	Sicherheitsziele	272
7.8.3	Lösungskonzepte	274
7.8.4	Security Engineering mit Patterns	281
7.8.5	Weiterführende Literatur	282
7.9	Protokollierung	283
7.9.1	Typische Probleme	283
7.9.2	Lösungskonzept	284
7.9.3	Zusammenhang mit anderen Aspekten	285
7.9.4	Weiterführende Literatur	285
7.10	Ausnahme- und Fehlerbehandlung	286
7.10.1	Motivation	286
7.10.2	Fehlerkategorien schaffen Klarheit	288
7.10.3	Muster zur Fehlerbehandlung	290
7.10.4	Mögliche Probleme	291
7.10.5	Zusammenhang mit anderen Aspekten	292

7.10.6	Weiterführende Literatur	293
7.11	Skalierbarkeit	293
7.11.1	Was bedeutet Skalierbarkeit?	293
7.11.2	Skalierungsstrategien	294
7.11.3	Elastizität	294
7.11.4	Scale-Up-Strategie	294
7.11.5	Vertikale Scale-Out-Strategie	295
7.11.6	Horizontale Scale-Out-Strategie	295
7.11.7	Der Strategiemix	295
7.11.8	Allgemeine Daumenregeln	296
7.11.9	CPU-Power	297
7.11.10	GPU-Power	297
7.11.11	RAIDs, SANs und andere Speichersysteme	297
7.11.12	Bussysteme für die Speicheranbindung	298
7.11.13	Geringere Bandbreite im Netz	298
7.12	Blockchain und dezentralisierte Architektur	299
7.12.1	Definition und Abgrenzung	299
7.12.2	Technische Grundlagen von Blockchains	301
7.12.3	Smart Contracts	302
7.12.4	Blockchains in nichtöffentlichen Kontexten	302
7.12.5	Architekturabwägungen	304
7.12.6	Architekturmuster	305
8	Analyse und Bewertung von Softwarearchitekturen	307
8.1	Qualitative Architekturbewertung	310
8.2	Quantitative Bewertung durch Metriken	317
8.3	Werkzeuge zur Bewertung	319
8.4	Weiterführende Literatur	320
9	Systematische Verbesserung und Evolution	321
9.1	Wege in den Abgrund	323
9.2	Systematisch verbessern	324
9.3	Bewährte Praktiken und Muster	327
9.4	Analyse: Probleme identifizieren	329
9.5	Evaluate: Probleme und Maßnahmen bewerten	331
9.6	Improve: Verbesserungsmaßnahmen planen und durchführen	332
9.6.1	Maxime für Verbesserungsprojekte	332
9.6.2	Kategorien von Verbesserungsmaßnahmen	332
9.7	Crosscutting: phasenübergreifende Praktiken	336
9.8	Mehr zu AIM ⁴²	337
9.9	Weiterführende Literatur	337

10	Microservices	339
10.1	Was sind Microservices?	340
10.2	Warum Microservices?	340
10.3	Eigenschaften von Microservices	341
10.4	Microservices und die Organisation	343
10.5	Für welche Systeme eignen sich Microservices?	344
10.6	Herausforderungen bei Microservices	344
10.6.1	Überblick über viele Services behalten	345
10.6.2	Microservices effektiv entwickeln	345
10.6.3	Service Discovery	346
10.6.4	UI-Integration	347
10.6.5	Dezentralisierte Daten	347
10.6.6	Versionierung von Microservices	348
10.6.7	Laufzeitumgebungen und Infrastruktur verwalten	349
10.7	Beispiele für Microservices	349
10.8	Weiterführende Literatur	349
11	Enterprise-IT-Architektur	351
11.1	Wozu Architekturebenen?	352
11.2	Aufgaben von Enterprise-Architekten	353
11.2.1	Management der Infrastrukturkosten	353
11.2.2	Management des IS-Portfolios	354
11.2.3	Definition von Referenzarchitekturen	355
11.2.4	Weitere Aufgaben	357
11.3	Weiterführende Literatur	359
12	Beispiele von Softwarearchitekturen	361
12.1	Beispiel: Datenmigration im Finanzwesen	362
12.2	Beispiel: Kampagnenmanagement im CRM	379
13	Werkzeuge für Softwarearchitekten	409
13.1	Kategorien von Werkzeugen	409
13.2	Typische Auswahlkriterien	413
14	iSAQB Curriculum	415
14.1	Standardisierte Lehrpläne für Softwarearchitekten	416
14.1.1	Grundlagenausbildung und Zertifizierung Foundation-Level	416
14.1.2	Fortgeschrittene Aus- und Weiterbildung (Advanced-Level)	417
14.2	iSAQB-Foundation-Level-Lehrplan	418
14.2.1	Können, Wissen und Verstehen	418
14.2.2	Voraussetzungen und Abgrenzungen	419
14.2.3	Lernziele des iSAQB Foundation Level	420
14.2.4	Zertifizierung gemäß iSAQB	422
14.3	Beispielfragen zur Foundation Level Prüfung	423

15	Nachwort: Architektonien	429
15.1	In sechs Stationen um die (IT-)Welt	429
15.2	Ratschläge aus dem architektonischen Manifest.	432
16	Literatur.....	437
	Stichwortverzeichnis.....	441

Vorwort

*Haben Sie jemals einen dummen Fehler zweimal begangen?
– Willkommen in der realen Welt.
Haben Sie diesen Fehler hundertmal hintereinander gemacht?
– Willkommen in der Software-Entwicklung.*

Tom DeMarco,
in: „Warum ist Software so teuer?“

Wenn Sie sich für Baukunst interessieren, dann erkennen Sie sicherlich die „Handschrift“ berühmter Architekten wie Frank Lloyd Wright, Le Corbusier oder Mies van der Rohe immer wieder, egal, wo auf der Welt Sie auf Bauwerke dieser Meister stoßen. Die Funktionalität des Guggenheim Museums in New York oder des Opernhauses in Sydney gepaart mit deren Schönheit und Ästhetik sind unvergessliche Eindrücke. Das erwarten wir heute auch von unseren IT-Systemen: Funktionalität gepaart mit Stil!

Seit mehr als zwanzig Jahren versuche ich, Systementwicklern die Kunst des Architektur-entwurfs nahezubringen. Die Erfahrung hat mich gelehrt, dass man jede Person, die mit gesundem Menschenverstand ausgestattet ist, zu einem guten Systemanalytiker ausbilden kann. Softwarearchitekten auszubilden, ist wesentlich schwieriger.

Früher waren viele unserer Systeme so einfach, dass einzelne Personen die Struktur leicht im Kopf behalten konnten. Heutzutage gehört mehr dazu, um die Struktur eines Systems zu beherrschen, die Auswirkungen von Technologieentscheidungen vorausszusehen und die Vielzahl von Hilfsmitteln wie Generatoren, Frameworks, Libraries und Entwicklungswerkzeuge kosteneffizient und zielführend einzusetzen.

Viele Jahre war ich davon überzeugt, dass nur Erfahrung in der Erstellung großer Systeme und selbst gemachte Fehler gute Architekten hervorbringen. Wir wussten einfach zu wenig über Wirkungen und Folgewirkungen von Designentscheidungen. In den letzten Jahren ist die Entwicklung von Architekturen mehr und mehr zur Ingenieursdisziplin herangereift.

Gernot Starke ist es gelungen, die Essenz dieser Disziplin auf den Punkt zu bringen. Die Tipps und Tricks, die er in diesem Buch zusammengetragen hat, vermitteln Ihnen eine Fülle von Praxiserfahrungen. Selbst wenn Sie zu den Veteranen der Branche gehören, werden Sie neben vielen Déjà-vu-Erlebnissen sicherlich noch die eine oder andere Perle entdecken. Wenn Sie gerade Ihre ersten Sporen als Architekt(in) verdienen, dann können Sie sich mit den Empfehlungen den einen oder anderen Holzweg ersparen.

Trotz aller Fortschritte in der IT bleiben Konstruktion und Ausgestaltung von Architekturen dauerhaft eine Domäne für kreative Gestaltungsarbeit von Menschen und Teams. Softwarearchitekt ist daher ein Beruf mit sicherer Zukunft!

Aachen, im September 2013

Peter Hruschka

■ Vorwort zur neunten Auflage

Seit der ersten Auflage 2002 sind 18 Jahre vergangen – in denen „Softwarearchitektur“ aus den Kinderschuhen zu einer volljährigen Disziplin der Softwareentwicklung reifen konnte.

Unsere Branche hat viel gelernt, Agilität und iterativ-inkrementelle Entwicklung sind endlich im Mainstream angekommen, schnelles Feedback eher Normalität denn Ausnahme, beispielsweise durch automatisiertes Testen.

Architekturaufgaben und technische Entscheidungen liegen oftmals beim Entwicklungsteam oder werden zumindest von mehreren Personen nach Rücksprache mit den Teams getroffen – und nicht mehr von Einzelpersonen „diktiert“.

Dafür sind allerdings die Anforderungen an unsere Systeme signifikant gestiegen: Benutzerzahlen von Online-Systemen gehen oft in die Millionen, Datendurchsatz oder -volumen messen wir in Tera oder Giga, nicht mehr in Kilo ... Benutzungsschnittstellen müssen hochgradig ergonomisch sein und auf multiplen Endgeräten laufen – *zero-downtime* gehört (fast) zur Selbstverständlichkeit. Embedded- und Informationssysteme müssen oftmals sicherheitskritische Entscheidungen treffen. Ach ja – Entwicklungskosten für solche komplexen IT-Systeme sollen tendenziell sinken und neue Features am besten in kürzester Zeit zur Verfügung stehen. Für solche (krassen!) Herausforderungen benötigen Entwicklungsteams fundierte Architekturkompetenz – und genau dafür habe ich dieses Buch geschrieben.

Zur neunten Auflage: Seit der achten Auflage (2017) haben viele gründliche Leserinnen und Leser einige Dutzend kleine Fehler gefunden, die in der vorliegenden Auflage (hoffentlich) behoben sind. Ich danke allen, die mich netterweise auf diese Dinge hingewiesen haben.

Die Aktualisierungen des iSAQB-CPSA-F-Lehrplans sind komplett eingeflossen, und das Kapitel über Microservices ist von den Altlasten der SOA befreit ☺.

Meinen Kollegen Dr. Lars Hupel konnte ich dazu gewinnen, einen kleinen (aber spannenden) Exkurs zum Thema „Blockchain“ beizusteuern.

Zum Buch gibt es eine (*responsive design*) Website, die ich mittels Github¹, Jekyll und Docker pflege (<http://esabuch.de>).

Möge dieses Buch helfen, bessere Software zu entwickeln!

Köln, Juni 2020

Gernot Starke

¹ Für Insider oder falls Sie Verbesserungsvorschläge haben: <https://github.com/gernotstarke/esabuch.de-site>

1

Einleitung

*Wir bauen Software wie Kathedralen:
Zuerst bauen wir – dann beten wir.*

Gerhard Chroust

Bitte erlauben Sie mir, Sie mit einer etwas bössartigen kleinen Geschichte zur weiteren Lektüre dieses Buchs zu motivieren.

Eine erfolgreiche Unternehmerin möchte sich ein Domizil errichten lassen. Enge Freunde raten ihr, ein Architekturbüro mit dem Entwurf zu betrauen und die Erstellung begleiten zu lassen. Nur so ließen sich die legendären Probleme beim Hausbau (ungeeignete Entwürfe, mangelnde Koordination, schlechte Ausführung, Pfusch bei Details, Kostenexplosion und Terminüberschreitung) vermeiden.

Um die für ihr Vorhaben geeigneten Architekten zu finden, beschließt sie, einigen namhaften Büros kleinere Testaufträge für Einfamilienhäuser zu erteilen. Natürlich verrät sie keinem der Kandidaten, dass diese Aufträge eigentlich Tests für das endgültige Unterfangen sind.

Nach einer entsprechenden Ausschreibung in einigen überregionalen Tageszeitungen trifft unsere Bauherrin folgende Vorauswahl:

- Wasserfall-Architektur KG, Spezialisten für Gebäude und Unterfangen aller Art,
- V&V Architektur GmbH & Co. KG, Spezialisten für Regierungs-, Prunk- und Profanbauten,
- Extremarchitekten AG.

Alle Büros erhalten identische Vorgaben: Ihre Aufgabe besteht in Entwurf und Erstellung eines Einfamilienhauses (EFH). Weil unsere Unternehmerin jedoch sehr häufig, manchmal fast sprunghaft, ihre Wünsche und Anforderungen ändert, beschließt sie, die Flexibilität der Kandidaten auch in dieser Hinsicht zu testen.

Wasserfall-Architektur KG

Die Firma residiert im 35. Stock eines noblen Bürogebäudes. Dicke Teppiche und holzvertäfelte Wände zeugen vom veritablen Wohlstand der Firmeneigner.

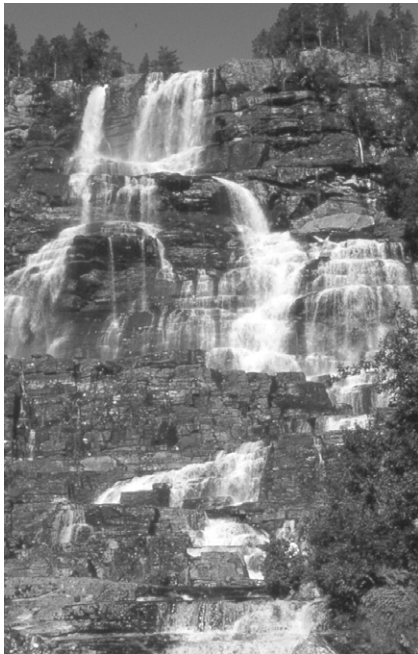


Foto von Wolfgang Korn

„Wir entwerfen auch komplexe technische Systeme“, erklärt ein graumeliertes Mittfünfziger der Bauherrin bei ihrem ersten Treffen. Sein Titel „Bürovorsteher“ prädestiniert ihn wohl für den Erstkontakt zu dem vermeintlich kleinen Fisch. Von ihm und einer deutlich jüngeren Assistentin wurde sie ausgiebig nach ihren Wünschen hinsichtlich des geplanten Hauses befragt.

Als sie die Frage nach den Türgriffen des Badezimmer-schranks im Obergeschoss nicht spontan beantworten kann, händigt man ihr ein Formblatt aus, das ausführlich ein Change-Management-Verfahren beschreibt.

Das Team der Wasserfall-Architektur KG legte nach wenigen Wochen einen überaus detaillierten Projektplan vor. Gantt-Charts, Work-Breakdown-Struktur, Meilensteine, alles dabei. Die nächsten Monate verbrachte das Team mit der Dokumentation der Anforderungsanalyse sowie dem Entwurf.

Pünktlich zum Ende dieser Phase erhielt die Unternehmerin einen Ordner (zweifach) mit fast 400 Seiten Beschreibung eines Hauses. Nicht ganz das von ihr Gewünschte, weil das Entwicklungsteam aus Effizienzgründen und

um Zeit zu sparen einige (der Bauherrin nur wenig zusagende) Annahmen über die Größe mancher Räume und die Farbe einiger Tapeten getroffen hatte. Man habe zwar überall groben Sand als Bodenbelag geplant, könne das aber später erweitern. Mit etwas Zement und Wasser vermischt, stünden den Hausbewohnern später alle Möglichkeiten offen. Im Rahmen der hierbei erwarteten Änderungen habe das Team vorsorglich die Treppen als Rampe ohne Stufen geplant, um Arbeitern mit Schubkarren den Weg in die oberen Etagen zu erleichtern. Das Begehren unserer Unternehmerin, doch eine normale Treppe einzubauen, wurde dem Change-Management übergeben.

Die nun folgende Erstellungsphase (die Firma verwendete hierfür den Begriff „Implementierungsphase“) beendete das Team in 13 statt der geplanten acht Monate. Die fünf Monate Zeitverzug seien durch widrige Umstände hervorgerufen, wie ein Firmensprecher auf Nachfrage erklärte. In Wirklichkeit hatte ein Junior-Planning-Consultant es versäumt, einen Zufahrtsweg für Baufahrzeuge zu planen – das bereits fertiggestellte Gartenhaus musste wieder abgerissen werden, um eine passende Baustraße anlegen zu können.

Ansonsten hatte das Implementierungsteam einige kleine Schwächen des Entwurfs optimiert. So hatte das Haus statt Treppe nun einen Lastenaufzug, weil sich die ursprünglich geplante Rampe für Schubkarren als zu steil erwies. Das Change-Management verkündete stolz, man habe bereits erste Schritte zur Anpassung des Sandbodens unternommen: Im ganzen Haus seien auf den Sand Teppiche gelegt worden. Leider hatte ein Mitglied des Wartungsteams über den Teppich dann, in sklavischer Befolgung der Planungsvorgaben, Zement und Wasser

aufgebracht und mit Hilfe ausgeklügelte brachialer Methoden zu einer rotgrauen zähen Paste vermischt. Man werde sich in der Wartungsphase darum kümmern, hieß es seitens der Firma. Die zu diesem Zeitpunkt von den Wasserfall-Architekten ausgestellte Vorabrechnung belief sich auf das Doppelte der ursprünglich angebotenen Bausumme. Diese Kostensteigerung habe die Bauherrin durch ihre verspätet artikulierten Zusatzwünsche ausschließlich selbst zu verantworten.

V&V Architektur GmbH & Co. KG

Die V&V Architektur GmbH & Co. KG (nachfolgend kurz V&V) hatte sich in den vergangenen Jahren auf Regierungs-, Prunk- und Profanbauten spezialisiert. Mit dem unternehmenseigenen Verfahren, so wird versichert, könne man garantiert jedes Projekt abwickeln. Der von V&V ernannte Projektleiter überraschte unsere Unternehmerin in den ersten Projektwochen mit langen Fragebögen – ohne jeglichen Bezug zum geplanten Haus. Man müsse unbedingt zuerst das Tailoring des Vorgehensmodells durchführen, das Modell exakt dem geplanten Projekt anpassen. Am Ende dieser Phase erhielt sie, in zweifacher Ausfertigung, mehrere Hundert Seiten Dokumentation des geplanten Vorgehens.

Dass ihr Einfamilienhaus darin nicht erwähnt wurde, sei völlig normal, unterrichtete sie der Projektleiter. Erst jetzt, in der zweiten Phase, würde das konkrete Objekt geplant, spezifiziert, realisiert, qualitätsgesichert und konfigurationsverwaltet.

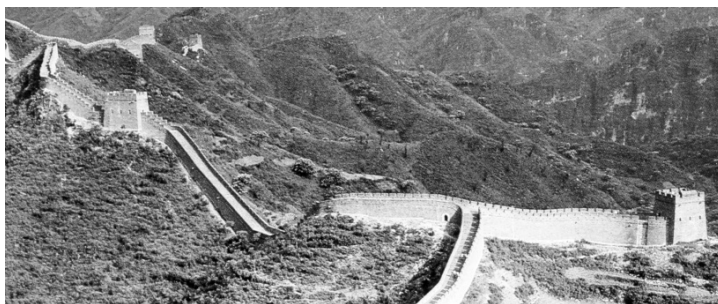


Foto von Ralf Harder

Der Auftraggeberin wurde zu diesem Zeitpunkt auch das „Direktorat EDV“ der Firma V&V vorgestellt. Nein, diese Abteilung befasste sich nicht mit Datenverarbeitung – die Abkürzung stand für „*Einhaltung Des Vorgehensmodells*“.

Nach einigen Monaten Projektlaufzeit stellte unsere Bauherrin im bereits teilweise fertiggestellten Haus störende signalrote Inschriften auf sämtlichen verbauten Teilen fest. Das sei urkundenechte Spezialtinte, die sich garantiert nicht durch Farbe oder Tapete verdecken ließe, erklärte V&V stolz. Für die Qualitätssicherung und das Konfigurationsmanagement seien diese Kennzeichen unbedingt notwendig. Ästhetische Einwände, solche auffälligen Markierungen nicht in Augenhöhe auf Fenster, Türen und Wänden anzubringen, verwarf die Projektleitung mit Hinweis auf Seite 354, Aktivität PL 3.42, Paragraph 9 Absatz 2 des Vorgehensmodells, in dem Größe, Format, Schrifttyp und Layout dieser Kennzeichen verbindlich definiert seien. Die Bauherrin hätte bereits beim Tailoring widersprechen müssen, nun sei es wirklich zu spät.

Extrem-Architekten AG

Die Extrem-Architekten laden unsere Unternehmerin zu Projektbeginn zu einem Planungsspiel ein. Jeden Raum ihres geplanten EFH soll sie dabei der Wichtigkeit nach mit Gummi-

bärchen bewerten. Die immer nur paarweise auftretenden Architekten versprechen ihr eine erste funktionsfähige Version des Hauses nach nur sechs Wochen. Auf Planungsunterlagen würde man im Zuge der schnellen Entwicklung verzichten.



„Gummibär-Tango“ von Klaus Terjung

Zu Beginn der Arbeiten wurde das Team in einer Art Ritual auf die gemeinsame Vision des Hauses eingeschworen. Wie ein Mantra murmelten alle Teammitglieder ständig mit seltsam gutturaler Betonung die Silben „Einfamilien-Haus“, was sich nach einiger Zeit zu „Ei-Mi-Ha“ abschliff. Mehrere Außenstehende wollen gehört haben, das Team baue einen bewohnbaren Eimer. Sie stellten eine überdimensionale Tafel am Rande des Baugeländes auf. Jeder durfte darauf Verbesserungsvorschläge oder Änderungen eintragen. Dies gehöre zu einem Grundprinzip der Firma: „Kollektives geistiges Eigentum: Planung und Entwurf gehören allen.“

Nach exakt sechs Wochen laden die Extrem-Architekten die Unternehmerin zur Besichtigung der ersten funktionsfähigen Version ein. Wieder treten ihr zwei Architekten entgegen, jedoch erkennt sie nur einen davon aus dem Planungsspiel

wieder. Der andere arbeitet jetzt bei den Gärtnern. Der ursprüngliche andere Gärtner hilft dem Elektriker, ein Heizungsbauer entwickelt dafür die Statik mit. Auf diese Weise verbreite sich das Projektwissen im Team, erläutern beide Architekten eifrig.

Man präsentiert ihr einen Wohnwagen. Ihren Hinweis auf fehlende Küche, Keller und Dachgeschoss nehmen die Extrem-Architekten mit großem Interesse auf (ohne ihn jedoch schriftlich zu fixieren).

Weitere sechs Wochen später hat das Team eine riesige Grube als Keller ausgehoben und den Wohnwagen auf Holzbohlen provisorisch darüber befestigt. Das Kellerfundament haben ein Zimmermann und ein Statiker gegossen. Leider blieb der Beton zu flüssig. Geeignete Tests seien aber bereits entwickelt, dieser Fehler käme garantiert nie wieder vor.

Mehrere weitere 6-Wochen-Zyklen gehen ins Land. Bevor unsere Unternehmerin das Projekt (vorzeitig) für beendet erklärt, findet sie zwar die von ihr gewünschte Küche, leider jedoch im Keller. Ein Refactoring dieses Problems sei nicht effektiv, erklärte man ihr. Dafür habe man im Dach einen Teil der Wohnwagenküche verbaut, sodass insgesamt die Zahl der Küchen-Gummibären erreicht worden sei.

Das immer noch flüssige Kellerfundament hat eines der Teams bewogen, auf die Seitenwände des Hauses auf Dauer zu verzichten, um die Lüftung des Kellers sicherzustellen. Im Übrigen besitzt das Haus nur ein Geschoss, das aktuelle Statik-Team (bestehend aus Zimmermann und Gärtner) hat dafür die Garage in drei Kinderzimmer unterteilt.

Weil das Team nach eigenen Aussagen auf die lästige und schwergewichtige Dokumentation verzichtet hatte, waren auch keine Aufzeichnungen der ursprünglichen Planung mehr erhalten.

Im Nachhinein beriefen sich alle Projektteams auf ihren Erfolg. Niemand hatte bemerkt, dass die Bauherrin keines der „implementierten“ Häuser wirklich akzeptierte.

Chaos nur am Bau?

Keineswegs! Ähnlichkeiten mit bekannten Vorgehensweisen bei der Softwareentwicklung sind ausdrücklich gewollt, denn nicht nur beim Hausbau herrscht Chaos.¹ Auch andere Ingenieurdisziplinen erleben *turbulente Situationen*, obwohl der Maschinenbau über mehr als 200 Jahre Erfahrung verfügt. In der Softwarebranche geht es mindestens ebenso schlimm zu.

Der regelmäßige Chaos-Report der Standish-Group zeigt eine seit Jahren gleichbleibende Tendenz: Über 30 % aller Softwareprojekte werden (erfolglos) vorzeitig beendet, in über 50 % aller Softwareprojekte kommt es zu drastischen Kosten- oder Terminüberschreitungen.²

■ 1.1 Softwarearchitekt(inn)en



Liebe Softwarearchitektinnen – ich meine überall auch Sie – obwohl ich im weiteren Buch durchgängig die maskuline (weil kürzere) Form „Softwarearchitekt“ verwende. Ich meine grundsätzlich Softwarearchitektinnen und Softwarearchitekten – verzichte aber auf künstliche Konstrukte wie Softwarearchitekt*innen. Danke für Ihr Verständnis!

Softwarearchitekten allein können diese Probleme nicht lösen. Stakeholder mit klaren Zielvorstellungen, ein motiviertes Entwicklungsteam und ein effektives, flexibles und (hoffentlich) agiles Management bilden wichtige Voraussetzungen für erfolgreiche Softwareentwicklung³. Die Rolle „Softwarearchitektur“ kommt in Softwareprojekten besondere Bedeutung zu:



Softwarearchitekten bilden die Schnittstelle zwischen Analyse, Entwurf, Implementierung, Management und Betrieb von Software.

Diese verantwortungsvolle Schlüsselrolle bleibt in vielen Projekten oft unbesetzt oder wird nicht angemessen ausgefüllt. Architekten sollten als „Anwälte der Kunden“ arbeiten. Sie sollen sicherstellen, dass die Anforderungen der Kunden einerseits umsetzbar sind und andererseits auch umgesetzt werden.

Architekten als Anwälte der Kunden

Softwarearchitekten denken langfristig – auf die gesamte Lebensdauer von IT-Systemen bezogen. Sie ermöglichen kurzfristige Änderungen, sichern gleichzeitig die Langlebigkeit und Nachhaltigkeit von Software.

Langfristigkeit

¹ Viele Grüße nach Berlin. Ähnlichkeiten mit gescheiterten Flughafenprojekten sind rein zufällig.

² Quelle: The Standish Group Chaos Report. Erhältlich unter www.standishgroup.com

³ Eine gute Voraussetzung für Projekterfolg ist es, im Team die Eigenschaften **Kompetenz, Energie und Verantwortung** zu bündeln – danke an Dierk König (@mittie) für diese Formulierung.

Konzeptionelle Integrität

Softwarearchitekten verfolgen konzeptionelle Integrität (auch genannt *Konsistenz*): Die gesamte Konstruktion von Software sollte einem einheitlichen Stil folgen. Insbesondere sollten *ähnliche Aufgabenstellungen in Systemen ähnlich gelöst* werden. Dies erleichtert das Verständnis und die langfristige Weiterentwicklung. Das Buch gibt aktiven und angehenden Softwarearchitekten praktische Ratschläge und Hilfsmittel, diese komplexe Aufgabe effektiver zu erfüllen. Es unterbreitet konkrete Vorschläge, wie Sie als Softwarearchitekt in der Praxis vorgehen sollten. Auch wenn Sie in anderen Funktionen in Softwareprojekten arbeiten, kann dieses Buch Ihnen helfen. Sie werden verstehen, welche Bedeutung Architekturen besitzen und wo die Probleme beim Entwurf von Architekturen liegen.

■ 1.2 Effektiv, agil und pragmatisch

Effektivität, Agilität und Pragmatismus prägen die Grundhaltung erfolgreicher Softwarearchitekten.

Agilität ist notwendig

Software wird in vielen Projekten immer noch als starres, unveränderliches Produkt betrachtet, obwohl Anwender und Auftraggeber laut nach hochgradig flexiblen Lösungen rufen. In der Praxis ähnelt die Softwareentwicklung leider oftmals eher dem Brückenbau: Eine Rheinbrücke bleibt auch in den kommenden Jahren eine Rheinbrücke. Weder verändert sich der Flusslauf noch wird aus einer Eisenbahnbrücke eine Startbahn für Passagierflugzeuge. Für Software stellt sich die Lage ganz anders dar: Hier kann aus einem abteilungsinternen Informationssystem schnell eine global genutzte Internet-E-Business-Lösung entstehen.

Langjährige Untersuchungen ergeben, dass sich 10 bis 25 % der Anforderungen an Software pro Jahr ändern (Quelle: Peter Hruschka). Management und Architekten von Softwareprojekten müssen sich durch flexible und bedarfsgerechte Vorgehensweisen darauf einstellen. Das Schlüsselwort lautet „Agilität“.

Agilität und flexibles Vorgehen wird in Softwareprojekten an vielen Stellen dringend benötigt:

- Requirements Manager müssen in Projekten flexibel mit Änderungen von Anforderungen umgehen.
- Softwarearchitekturen müssen stabile Grundgerüste bereitstellen, die die Umsetzung neuer und geänderter Anforderungen ermöglichen. In der heutigen Marktsituation müssen solche Änderungen schnell und effektiv erfolgen – oder sie sind wirkungslos.
- Projekt- und Produktmanager müssen in der Lage sein, während der Erstellung eines Systems flexibel auf neue Anforderungen, neue Technologien oder aktualisierte Produkte zu reagieren. Hier bietet agiles Vorgehen und risikobasiertes Projektmanagement viele Vorteile gegenüber den strikt am Vorgehensmodell orientierten konventionellen Methoden.
- Dokumentation muss sich an spezifischen Projektbedürfnissen orientieren statt an fix vorgegebenen Ergebnistypen. Inhalt ist in flexiblen Projekten wichtiger als Form.

- Agilität erfordert allerdings auch hohe Qualifikation und Professionalität der Beteiligten. Wenn Sie in einem agilen Projekt arbeiten, müssen Sie in allen Belangen mitdenken und Verantwortung übernehmen.

Insgesamt zählt in einem Projekt nur das Ergebnis. Selten kümmern sich Anwender oder Auftraggeber im Nachhinein um die Einhaltung starrer Vorgehensmodelle.

Softwarearchitekten müssen in ihrer Funktion als Schnittstelle zwischen den Projektbeteiligten diesen Ansatz der Agilität aufnehmen und in der Praxis umsetzen.



Handeln Sie agil!

Von Peter Hruschka



Agil heißt beweglich und flexibel sein. Mitdenken statt „Dienst nach Vorschrift“ und Dogma.

Kein Vorgehensmodell passt für alle Arten von Projekten. Eine agile Vorgehensweise beurteilt das jeweilige Risiko der unterschiedlichen Aufgaben bei der Softwareentwicklung und wählt dann die geeigneten Maßnahmen. Folgende Schwerpunkte werden dabei gesetzt:

- offen für Änderungen statt Festhalten an alten Plänen,
- eher ergebnisorientiert als prozessorientiert,
- „miteinander darüber reden“ statt „gegeneinander schreiben“,
- eher Vertrauen als Kontrolle,
- Bottom-up „Best Practices“ austauschen und etablieren statt Top-down-Vorgaben diktieren.

Trotzdem heißt „Agilität“ nicht, Anarchie zuzulassen:

- Agile Vorgehensmodelle haben Ergebnisse, nur unterscheiden sich diese für unterschiedliche Projekte in Anzahl, Tiefgang und Formalismus.
- Agile Entwicklung kennt Prozesse, nur lassen diese mehr Spielraum für Alternativen und Kreativität.
- Agile Methoden setzen auf Verantwortung; es werden nur notwendige Rollen besetzt.
- Agile Methoden basieren auf Feedback und Iterationen. Fordern und geben (*push und pull*) Sie Rückmeldung zu Ergebnissen – nur so können Teams und Ergebnisse besser werden.

Das Risiko entscheidet: Jeder Projektleiter, Systemanalytiker, Architekt, Tester und Entwickler überprüft ständig Risiken und entscheidet in seinem Umfeld über notwendige Maßnahmen, damit aus Risiken keine Probleme werden.

Dr. Peter Hruschka (hruschka@b-agile.de) ist unabhängiger Trainer und Methodenberater. Er ist Prinzipal der Atlantic Systems Guild, eines internationalen Think Tank von Methodengurus, deren Bücher den State of the Art wesentlich mitgestaltet haben (www.systemsguild.com).

Effektiv = Ziele erreichen

Weil das Begriffspaar „effektiv und effizient“ immer wieder für Missverständnisse sorgt, möchte ich die Bedeutung beider Wörter hier kurz gegenüberstellen.

Effizient =
hoher Wirkungsgrad

Eine Lexikondefinition des Begriffs „Effizienz“ lautet „Wirkungsgrad“, also das Verhältnis von Aufwand zu Ertrag. Wenn Sie Aufgaben effizient erledigen, dann arbeiten Sie also mit hohem Wirkungsgrad.

Sie investieren für den gewünschten Ertrag einen minimalen Aufwand. Spitzensportler etwa vermeiden in ihren Disziplinen überflüssige Bewegungen oder Aktionen, was in hochgradig effizientem Ausführen der jeweiligen Sportart resultiert.

Prägnant ausgedrückt, bedeutet das:

Effizient = Dinge richtig machen

Effektiv = zielorientiert

„Effektiv“ bedeutet zielorientiert. Sie arbeiten effektiv, wenn Sie Dinge erledigen, die zur Erreichung Ihrer konkreten Ziele notwendig sind. Auch für diesen Begriff wieder eine prägnante Definition:

Effektiv = die richtigen Dinge machen

Es ist viel wichtiger, die richtigen Dinge zu erledigen, als irgendwelche Dinge besonders effizient zu tun.

Softwarearchitekten müssen in hohem Maße effektiv arbeiten. Kunden und Auftraggeber bestimmen Ziele, Architekten müssen sicherstellen, dass diese Ziele auch erreicht werden.

Effektiv = agil und angemessen

Effektiv bedeutet auch, angemessen und bedarfsgerecht zu agieren. Auf die Entwicklung von Software angewandt, heißt das, sich permanent an den Bedürfnissen der Kunden und Auftraggeber zu orientieren (und nicht starr an den Buchstaben eines formalen Vorgehensmodells). Ich plädiere in diesem Buch für Agilität in diesem Sinne.

Effektive Softwarearchitektur bedeutet, das (für Kunden und Auftraggeber) richtige System zu konstruieren und zu bauen – mit technisch und organisatorisch angemessenen Mitteln.

Effektiv = pragmatisch

Projekterfolg wird grundsätzlich vom Auftraggeber beurteilt, nicht vom Architekten. Auftraggeber wollen in erster Linie ein produktives System erhalten und nicht die Einhaltung eines starren Vorgehensmodells erzwingen. Architekten müssen daher den Zweck des Systems im Auge behalten.

Pragmatisches Vorgehen bedeutet:

- auf Dogmen und starre Vorschriften zu verzichten, wo sie nicht angemessen sind,
- zielorientiert (= effektiv) Lösungen im Sinne der Kunden zu entwickeln,
- auf Perfektionismus zu verzichten. 80 % des Ertrags erreicht man mit 20 % des Aufwands.

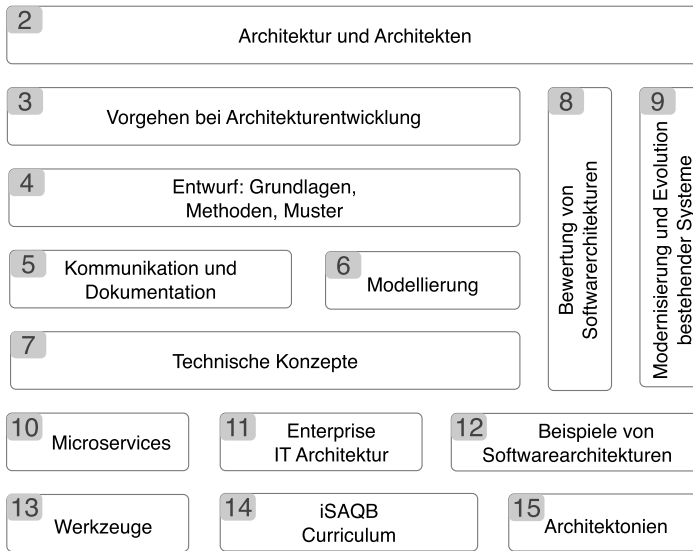
■ 1.3 Wer sollte dieses Buch lesen?

Grundsätzlich können alle Stakeholder⁴ von diesem Buch profitieren und erhalten Antworten auf zentrale Fragen.

- Softwarearchitekten
 - Was sind die Methoden und Werkzeuge unserer Zunft?
 - Wie gehen Sie beim Entwurf von Architekturen sinnvoll vor?
 - Welche praktisch erprobten Heuristiken und Ratschläge gibt es?
 - Wie meistern Sie externe Einflussfaktoren, die den Entwurf von Architekturen erschweren?
 - Welche Sichten auf Architekturen benötigen Sie in der Praxis?
- Softwareentwickler(innen)
 - Wie hängen Architektur und Implementierung zusammen?
 - Wie kann das gesamte Entwicklungsteam bei Entwurf und Pflege der Architektur konstruktiv mitwirken?
 - Welche allgemeinen Prinzipien von Softwarearchitektur und -entwurf sollten Sie bei der Implementierung unbedingt befolgen?
- Alle Personen, die sich auf die Prüfung zum „Certified Professional for Software Architecture – Foundation Level“ (CPSA-F) des iSAQB e. V. vorbereiten
- Technische Manager(innen)
 - Warum sollen Sie für Architektur Geld ausgeben? Warum ist Softwarearchitektur wichtig?
- Product-Owner, Scrum-Master, Projektleiter(innen)
 - Was genau bewirkt Architektur in Entwicklungsprojekten?
 - Welche Aufgaben erfüllen Softwarearchitekten?
 - Welche Bedeutung hat die Dokumentation von Architekturen („Was bedeuten all diese Symbole?“)?
 - Welche grundlegenden Lösungsansätze gibt es für Architekturen?
- Business-Analysten und Requirements-Engineers
 - Was bedeuten all diese Begriffe, die das Entwicklungsteam und die Architekten ständig verwenden?
 - Wie können Sie bei der Architekturentwicklung konstruktiv beitragen?

⁴ Im Verlaufe des Buchs benutze ich den Begriff Stakeholder für Personen oder Organisationen, die bei der Erstellung des Systems mitwirken, es beeinflussen oder am entwickelten System ein fachliches, technisches oder kommerzielles Interesse haben.

■ 1.4 Wegweiser durch das Buch



Wegweiser

Kapitel 2 erläutert die Begriffe „Architektur und Architekt“. Es beantwortet die Fragen nach dem Was, Warum und Wer von Softwarearchitekturen.

Kapitel 3 legt den Grundstock für eine flexible und systematische Vorgehensweise zum Entwurf von Softwarearchitekturen. Es beschreibt die wesentlichen Aktivitäten bei der Architekturentwicklung, das Vorgehen „im Großen“. Insbesondere erfahren Sie, wie Sie aus den Faktoren, die eine Architektur beeinflussen, angemessene Lösungsstrategien ableiten können.

Kapitel 4 widmet sich dem Entwurf von Systemen. Sie lernen hier die Grundlagen des Entwurfs, geltende Prinzipien und erprobte Heuristiken. Sie finden neben Methoden für die Strukturierung Ihrer Systeme und Bausteine sowie der Gestaltung von Schnittstellen auch die wichtigsten Architektur- und Entwurfsmuster, die Sie in unterschiedlichen Bereichen der Softwareentwicklung anwenden können. Dieses Kapitel gibt Ihnen die methodischen Werkzeuge für zielgerichtete Entwurfsentscheidungen an die Hand.

arc42 Template

Kapitel 5 beschreibt, wie Sie mit praxisorientierten Sichten Ihre Softwarearchitekturen kommunizieren und dokumentieren. Jede Sicht beschreibt das System aus einer anderen Perspektive. Außerdem finden Sie hier einige Hinweise für gute Architekturdokumentation sowie das bekannte Architekturtemplate arc42. Passend dazu fasst **Kapitel 6** die Grundlagen der Modellierung für Softwarearchitekten zusammen. Sie finden u. a. einige Basiskonstrukte der UML, abgestimmt auf die Bedürfnisse von Softwarearchitekten.

(technische) Konzepte

Kapitel 7 enthält einen Katalog häufig benötigter technischer Konzepte. Hierzu zählen Persistenz (Datenspeicherung), Integration, Verteilung, Kommunikation, Sicherheit, grafische Benutzeroberflächen, übergreifende

Ablaufsteuerung (Workflow Management), Ausnahme- und Fehlerbehandlung sowie Management von Geschäftsregeln.

Kapitel 8 erklärt Ihnen die qualitative und szenariobasierte Bewertung von Softwarearchitekturen.

Kapitel 9 beleuchtet Änderung, Modernisierung und Evolution von Software – womit Softwareentwickler vermutlich 80 % ihrer beruflichen Zeit verbringen.

Evolution

In **Kapitel 10** stelle ich Ihnen die Grundlagen von Microservices vor, sowohl von der technischen wie auch organisatorischen Seite. Sie finden hier einige Hinweise auf praktische Beispiele von Microservice-Systemen.

Microservices

Schließlich hebt **Kapitel 11** Ihren Blick über den Tellerrand reiner Software- und Systemarchitekturen hinaus und führt Sie in die Unternehmens-IT-Architektur ein – neudeutsch Enterprise-IT-Architektur.

Kapitel 12 enthält Beispiele von Softwarearchitekturen, beschrieben nach der Strukturvorlage arc42 (die Sie in Kapitel 5 kennengelernt haben).

Beispiele von Architekturen

Kapitel 13 stellt einige Kategorien von nützlichen Werkzeugen vor sowie mögliche Kriterien für deren Auswahl.

Kapitel 14 erläutert Ihnen, wie der standardisierte Lehrplan des *International Software Architecture Qualification Board* (ISAQB) inhaltlich mit diesem Buch zusammenhängt. Dieses Kapitel hilft Ihnen bei der Vorbereitung zur CPSA-F-Prüfung.⁵

Vorbereitung auf CPSA-F

In **Kapitel 15** finden Sie eine etwas *andere* Zusammenfassung in Form eines Reiseberichts durch die IT-Welt.

Jedes Kapitel bietet kommentierte Literaturverweise, die Ihnen Hinweise und Empfehlungen für die Vertiefung des jeweiligen Themas geben.

Tipps, Ratschläge, Heuristiken und Regeln



Dieses Buch enthält viele praktische Tipps, Ratschläge, Heuristiken und Regeln. Damit Sie diese Regeln leichter auffinden können, sind sie typografisch durch graue Schattierung hervorgehoben.

⁵ CPSA-F: Certified Professional for Software Architecture

■ 1.5 Webseite zum Buch

Website: www.esabuch.de

Auf der Website www.esabuch.de finden Sie Informationen, die aus Platzgründen ins Internet weichen mussten. Dazu gehören aktuelle Literaturhinweise und Links sowie sonstige Hilfsmittel für Softwarearchitekten zum Download.

<https://arc42.de>

Sie können unter www.arc42.de den aktuellen Stand des praxisnahen und umfassend erprobten Architekturtemplates herunterladen – hilfreich für Entwurf, Entwicklung und Dokumentation von Softwarearchitekturen.

■ 1.6 Weiterführende Literatur



[DeMarco+07] beschreiben (äußerst humorvoll und gnadenlos wahr) mehr als 80 hilfreiche „Verhaltensmuster“ aus IT-Projekten – die meiner Meinung nach alle ITler kennen sollten.

Ebenfalls in Form informeller Muster nehmen Peter Hruschka und ich im [Knigge] diverse typische positive und negative Verhaltensweisen rund um Softwarearchitektur ins Visier.

Falls Sie Softwarearchitektur an realen Beispielen „erleben“ möchten (und Ihnen Kapitel 12 dieses Buchs nicht genügt) – unter <https://leanpub.com/arc42byexample> finden Sie sechs solcher Beispiele (u. a. eine Schach-Engine und ein Portal für Radsport).

Da die Ursache der meisten Krankheiten in Softwareprojekten in menschlichen statt technischen Problemen liegt, lege ich Ihnen eine eingehende Beschäftigung mit „Soft-Skills“ nahe – was besser durch Praxis als durch Literaturstudium funktioniert.

■ 1.7 Danksagung

Danke an meine Traumfrau, Cheffe Uli, für Engelsgeduld, Motivation und tolle gemeinsame Zeit auf Bergen, Rädern, Yogamatten und Sofas sowie seit einigen Monaten auch an Hanteln. Du sorgst für mein glückliches Leben!

Danke an meine Diskussionspartner und Reviewer, dank deren Kommentaren und Ergänzungen das Buch Form (und Inhalt) annehmen konnte: Julia Conradt, Stefan Densow, Khalid Dermoumi, Karl Eilebrecht, Ralf Engelschall, Wolfgang Keller (alias Mr. Review-Torture), Klaus Kiehne, Holger Kreißl, Jürgen Krey, Christian (CK) Küster, Andreas Krüger, Dr. Carola Lilienthal, Michael Mahlberg, Prof. Erwin Mathis, Thekla Müller-Schenker, Alexander Nachtigall, Axel Noellchen, Michael Perlin, Robert Reimann, Lothar Piepmeyer, Peter Roßbach, Jan Schenk, Till Schulte-Coerne, Simon Sippert, Manuel Stahl, Thomas Walz, Stefan Zörner, Marco Zurbriggen.

Danke an Martin Bartonitz, Phillip Ghadir, Tobias Hahn, Alexander Heusingfeld, Lars Hupel, Wolfgang Korn, Tammo van Lessen, Stefan Tilkov und Oliver Wolf für ihre Beiträge.

Danke an das großartige Team der INNOQ – ihr seid die Besten!

Ich habe ungemein von den Diskussionen und Arbeiten mit Peter Hruschka rund um arc42 profitiert. Peter hat mich ursprünglich motiviert, dieses Buch überhaupt zu schreiben.

Lynn und Per, für die wirklichen Prioritäten.

Zu guter Letzt vielen Dank an meine Kunden, dass ich in Ihren/euren Projekten so viel über Softwarearchitekturen erfahren und erleben durfte.

2

Architektur und Architekten

Architecture is about people.

Norman Foster



Fragen, die dieses Kapitel beantwortet:

- Was ist Softwarearchitektur?
- Warum ist Architektur wichtig?
- Welche Aufgaben erfüllen Softwarearchitekten?
- In welchem Kontext steht Architektur?
- Wie entstehen Architekturen?

Dieses Kapitel beantwortet die Frage „Was ist Softwarearchitektur?“. Es beschreibt Architekturen im Kontext der gesamten Softwareentwicklung. Eine zentrale Stellung kommt der vielseitigen Rolle des Softwarearchitekten zu.

Softwaresysteme werden zunehmend komplexer und umfangreicher. Damit nimmt die Bedeutung von Entwurf und Beschreibung grundlegender Systemstrukturen zu, während die Bedeutung der Auswahl von Programmiersprachen, Algorithmen und Datenstrukturen zurückgeht.

■ 2.1 Was ist Softwarearchitektur?

Softwarearchitektur beschäftigt sich mit Abstraktion, mit Zerlegung und Zusammenbau, mit Stil und Ästhetik.

[Kruchten95]

Von den vielen Definitionen der einschlägigen Literatur möchte ich Ihnen diejenige des IEEE-Standards¹ 1471 vorstellen:

Softwarearchitektur: Die grundsätzliche Organisation eines Systems, verkörpert durch dessen Komponenten, deren Beziehung zueinander und zur Umgebung sowie die Prinzipien, die für seinen Entwurf und seine Evolution gelten.

Dazu helfen einige Erläuterungen weiter:

Architektur enthält Strukturen

Strukturen, Zerlegung, Schnittstellen, Beziehungen

Die Architektur eines Softwaresystems besteht aus seinen Strukturen,² der Zerlegung in Komponenten,³ deren Schnittstellen und Beziehungen untereinander.

Architektur definiert demnach die Bausteine eines Systems, beschreibt deren wesentliche (extern sichtbare) Merkmale und charakterisiert deren Beziehungen untereinander.

Damit enthält Architektur sowohl statische als auch dynamische Aspekte. Sie erfüllt sowohl die Aufgabe eines *Bauplans* als auch die eines *Ablaufplans* für Software.

Architektur beschreibt eine Lösung

Beschreibung einer Lösung

Die Architektur eines Systems umfasst Strukturen und Konzepte eines Systems im Sinne von Konstruktionszeichnungen, Bauanleitungen, übergreifend geltenden Regeln und grundlegenden Entwurfsentscheidungen.

Für Softwaresysteme bestehen Architekturen aus (dokumentierten) Komponenten, Schnittstellen und übergreifenden Konzepten (in der obigen Definition genannt „Prinzipien“). Erst durch eine konkrete Implementierung werden daraus reale Systeme.

Architektur basiert auf Entwurfsentscheidungen

Entwurfsentscheidungen

Die Architektur eines Softwaresystems resultiert aus einer Vielzahl von Entscheidungen. Einige davon bilden die Grundlage für den Entwurf der Komponenten, andere bestimmen über die Auswahl der eingesetzten Technologie.

¹ http://en.wikipedia.org/wiki/IEEE_1471

² Sie lesen richtig: Es sind mehrere unterschiedliche. Die weiter unten beschriebenen „Sichten“ beschreiben dasselbe System, aber unterschiedliche Strukturen. Im weiteren Verlauf des Buchs benutze ich aus Gründen der Lesbarkeit trotzdem meistens den Singular.

³ Komponenten bezeichne ich nachfolgend meistens als Bausteine – und erkläre sie in Abschnitt 4.4 genauer.

Seien Sie sich als Softwarearchitekt der Tatsache bewusst, dass Sie die Konsequenz mancher Ihrer Entscheidungen erst viel später bewerten können. So können Sie beispielsweise erst nach einigen Änderungen und Anpassungen eines Systems beurteilen, ob es die gewünschte Änderbarkeit oder Flexibilität auch wirklich erreicht hat.

Entscheidungen von besonderer Tragweite sollten Sie systematisch vorbereiten, treffen und dokumentieren – dazu gibt Ihnen Kapitel 3 weitere Hinweise.

Architektur unterstützt den Übergang von der Analyse zur Realisierung

Softwarearchitektur unterstützt den schwierigen Übergang von der Problemanalyse zur konkreten technischen Realisierung. Sie schlägt die Brücke zwischen der Fachdomäne und deren Umsetzung in Software, indem sie die geforderten Funktionen und Leistungsmerkmale auf eine Struktur von Softwarekomponenten und deren Beziehungen untereinander abbildet. Welchen Freiheitsgrad Sie als Architekt dabei haben, hängt von der Situation ab, in der Sie die Architektur entwickeln:

- Wenn Sie „auf der grünen Wiese“ entwickeln, sind Sie relativ frei in der Wahl von Komponenten und der Abbildung der Fachdomäne auf Komponenten.
- Wenn Sie ein Teilsystem innerhalb einer existierenden Systemlandschaft entwickeln, unterliegen Sie starken Restriktionen (die Sie teilweise erst ermitteln müssen).
- Wenn Sie nach einer vorgegebenen domänenspezifischen Softwarearchitektur eines von vielen ähnlichen Systemen entwickeln, reduziert sich die Architektur darauf, Fachlichkeit in eine bestehende Architektur zu gießen.

Architektur kann Systeme aus verschiedenen Sichten zeigen

Gebäudearchitekten erstellen eine Vielzahl unterschiedlicher Pläne und Sichten für Gebäude. Einige Beispiele dafür sind Grundriss, Statik, Elektro- und Heizungsplan, 3D-Sicht. Jede dieser einzelnen Sichten dokumentiert einzelne Aspekte des Gesamtsystems. Den verschiedenen Projektbeteiligten wird damit eine spezifische und ihren jeweiligen Belangen angemessene Sicht vermittelt. Jede Sicht ist für bestimmte Stakeholder nützlich.

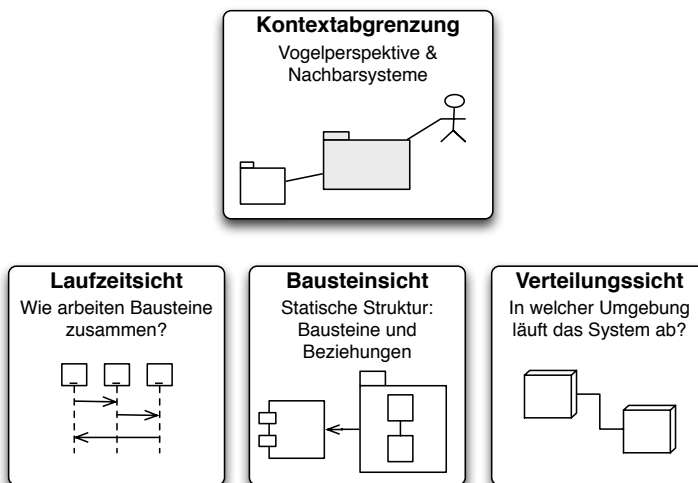


Bild 2.1
Vier Sichten
auf Architektur

Unterschiedliche Sichten

Für Softwarearchitekturen gilt das analog: Sie sollten Architekturen mit Hilfe unterschiedlicher Sichten beschreiben. Die wichtigsten Sichten für Softwarearchitekturen sind die Bausteinsicht, die Laufzeitsicht sowie die Verteilungssicht, die eine Abbildung von Laufzeitbausteinen auf technische Infrastruktur (Hardware) beschreibt. Kapitel 4 beschäftigt sich ausführlich mit den Sichten, ihrer Notation und Entwicklung. In Bild 2.1 sehen Sie die Sichten im Überblick.

Architektur schafft Verständlichkeit**Architektur schafft Ordnung**

Softwarearchitekturen machen Komplexität von Systemen beherrschbar und verständlich, indem sie (komplexe) Anforderungen in (geordnete) Strukturen und Konzepte übersetzen und diese übersichtlich dokumentieren.

Architektur ermöglicht Überblick

Für alle Projektbeteiligten dokumentieren sie angemessen und problembezogen die Struktur und das Zusammenwirken der einzelnen Komponenten. Architektur ermöglicht damit allen Beteiligten, die

Lösung zu verstehen:

- Für das Management bilden sie die erste Möglichkeit der Verifikation, ob die Anforderungen erfüllbar sind und erfüllt werden.
- Für neue Mitglieder des Entwicklungsteams bildet die Architektur die Grundlage der Einarbeitung und eine ideale Möglichkeit, sich mit der Struktur des Systems sowie den übergreifenden Konzepten vertraut zu machen.
- Architekturen ermöglichen es Wartungsteams, die betroffenen Bestandteile leichter zu finden und die Folgen von Änderungen abzuschätzen (*impact analysis*).
- Für Betreiber von Softwaresystemen dokumentieren Softwarearchitekturen, welche Softwarekomponenten auf welchen physischen Systemteilen ablaufen. Sie stellen damit die Basis integrierter Systemarchitekturen dar, die neben Software auch Hardware und Organisationsstrukturen umfassen.

Architektur bildet den Rahmen für flexible Systeme**Architektur stellt Erweiterbarkeit sicher**

Softwarearchitektur ist nach Tom DeMarco ein „*framework for change*“. Sie bildet also den Rahmen, der Flexibilität und Erweiterbarkeit von Software sicherstellt.

Softwarearchitekturen berücksichtigen dabei die Auswirkungen externer Einflussfaktoren organisatorischer und technischer Art auf das System. Diese Faktoren sind häufig Auslöser neuer Anforderungen an das System. Sie bestimmen, in welcher Hinsicht das System Flexibilität besitzen muss.

Architektur basiert auf Abstraktionen**Architektur lässt nicht benötigte Informationen gezielt weg**

Im ersten Moment mag es Ihnen unglaublich vorkommen: Eine essenzielle Leistung von Architekten besteht darin, die für eine bestimmte Aufgabe nicht benötigten Informationen gezielt wegzulassen, zu abstrahieren. Diese Abstraktion besitzt eine Analogie in der Gebäu-

dearchitektur: Eine Grundrisszeichnung enthält beispielsweise keine Informationen über Elektro- und Wasserleitungen. Hier wird bewusst Information gefiltert, um eine spezifische Darstellung der Architektur lesbar und verständlich zu halten.

Architektur schafft Qualität

Die Qualität eines Systems bezeichnet ganz allgemein die Summe seiner gewünschten Eigenschaften. Hierzu zählen Aspekte wie Performance, Verständlichkeit, Flexibilität, Robustheit, Sicherheit und natürlich auch Funktionalität.

Anforderungen an Qualitätseigenschaften, manchmal ungenau als *nichtfunktionale Anforderungen* (NFA) bezeichnet, stellen häufig die wirklich schwierigen Aufgaben für Softwarearchitekten dar. Einen bestimmten Algorithmus ohne jegliche Qualitätsanforderungen zu programmieren ist erheblich leichter, als dieses Programm gleichzeitig auch noch verständlich, erweiterbar und performant für viele parallele Benutzer zu entwickeln.⁴

Die Architektur eines Systems bildet die Basis für dessen Qualität. Sie ist auch die Grundlage, um spezifische Qualitätseigenschaften bewerten zu können (Kapitel 8 erklärt Ihnen die Bewertung von Softwarearchitekturen).

In Abschnitt 3.6 (Lösungsstrategien entwickeln) finden Sie Tipps zur Erfüllung typischer Qualitätsanforderungen.

Architektur orientiert sich an Werten⁵

Architekturentscheidungen sollen sich primär an den Wertmaßstäben der maßgeblichen Stakeholder orientieren. Falls time-to-market besonders wichtig ist, sollten Entscheidungen mit Fokus auf Entwicklungsproduktivität getroffen werden. Als Softwarearchitekt sollten Sie Ihr Handeln grundsätzlich an den Werten und Qualitätsmaßstäben der relevanten Stakeholder ausrichten – niemals primär an Ihren eigenen Zielen.

Orientiert an Stakeholder-Werten

Architektur entsteht begleitend

Ich vertrete die Auffassung, dass Softwarearchitektur begleitend zur Entwicklung/Implementierung entstehen sollte. Bei jeglicher Änderung, Erweiterung oder Anpassung von Systemen müssen Sie irgendwelche Entwurfsentscheidungen treffen – also Architektur praktizieren. Jeder Softwareentwickler sollte über ein Mindestmaß an Architekturfähigkeiten verfügen.

Entsteht begleitend

Architektur ist Design⁶, Design ist Architektur

„Architektur“ und „Design“ beschäftigen sich mit dem Entwurf von Systemen. Ich vertrete die Auffassung, dass die Grenze zwischen Softwarearchitektur und Softwaredesign fließend ist, die Begriffe Design und Architektur eng zusammen liegen. Beides umfasst konkrete Entwurfsentscheidungen.

Manchmal gehört der Entwurf einer konkreten Klassenstruktur zur Aufgabe von Architekten – in anderen Fällen bleibt Architektur auf höherem Abstraktionsniveau. Gehen Sie mit diesen Begriffen pragmatisch um und suchen Sie nicht nach einer „formalen“ Definition oder Abgrenzung.

⁴ Das ist auch ein Grund, wieso sich Übungsbeispiele aus Programmierkursen fast nie in die Wirklichkeit übertragen lassen: Anspruchsvolle Qualitätsanforderungen bleiben in solchen Beispielen in der Regel außen vor – machen in der Praxis allerdings das Leben schwer!

⁵ Danke an Andreas Krüger für diese Formulierung.

⁶ Mit Design ist hier der Entwurf „kleiner“ Softwarebausteine gemeint, nicht aber Grafik- oder UI Design.

Architektur kann sowohl Soll- wie auch Ist-Zustand beschreiben

Sie können die Architektur eines bestehenden Systems beschreiben, mit Strukturen, Konzepten und grundlegenden Architekturentscheidungen. Damit haben Sie den Ist-Zustand dokumentiert.

Soll- und Ist-Zustand

Andererseits können Sie Architektur zur Diskussion oder Planung zukünftiger Zustände („Soll-Zustand“) verwenden – indem Sie *mögliche* Strukturen, Konzepte oder Entscheidungen verwenden.

Diese beiden zeitlichen Dimensionen können während der Entwicklung von Systemen parallel zueinander existieren.

Der Smoketest für Softwarearchitekturen



Wenn Ihnen jemand die Dokumentation einer Architektur vorlegt, stellen Sie folgende Fragen (die eine gute Architekturdokumentation immer beantworten sollte):

- Welche Grundsätze, Konzepte oder Entscheidungen liegen der Lösung zugrunde und warum?
- Welche Verantwortlichkeiten (*responsibilities*) trägt jedes der Kästchen und Verbindungslinien in Diagrammen?
- Wie erfüllt die Architektur die an das System gestellten Qualitätsanforderungen?

Schon mit diesen einfachen Fragen können Sie Entwicklungsteams anregen, ihre Architekturdokumentation grundlegend zu verbessern.

Darum Softwarearchitektur!

Ich möchte den Nutzen und die Ziele von Softwarearchitektur nochmals zusammenfassen:

- Qualitätsanforderungen und -ziele erfüllen: Explizite konstruktive und technische Konzepte, Strategien, Maßnahmen, Taktiken oder Praktiken stellen die Erreichung der notwendigen Qualität sicher.
- Konzeptionelle Integrität (Konsistenz): Ähnliche Aufgaben werden im System durchgängig ähnlich gelöst – Ausnahmen sind begründet.
- Verständlichkeit des Systems und seiner grundlegenden Entwurfsentscheidungen, Strukturen und Konzepte/Prinzipien: Angemessene Verwendung von Abstraktionen, Modellen, Dokumentation zur Erläuterung von Entscheidungen, Strukturen, Konzepten und Quellcode.
- Starker Fokus auf Langfristigkeit – daher Betonung von (langfristigen) Architekturzielen gegenüber (meist kurzfristigen) Projektzielen.
- Unterstützung des gesamten Lebenszyklus: Von der Problem- und Anforderungsanalyse über Konstruktion, Entwicklung, Implementierung bis zum praktischen Einsatz und Betrieb von Systemen, sowohl bei Neuentwicklung wie auch Wartung/Änderung von Systemen.
- Wir erreichen mit Architektur die wesentlichen Ziele unserer maßgeblichen Stakeholder.

■ 2.2 Die Aufgaben von Softwarearchitekten

Das Leben von Softwarearchitekten besteht aus einer langen und schnellen Abfolge suboptimaler Entwurfsentscheidungen, die meist im Dunkeln getroffen werden.

[Kruchten2001]

Softwarearchitekten müssen deutlich mehr leisten, als „nur“ Softwarearchitekturen zu entwerfen: Sie sind „Anwälte der Auftraggeber“ und Berater für Manager und für das Realisierungsteam.

Architekten konstruieren, entwerfen und implementieren

Softwarearchitekten (gemeinsam mit Entwicklungsteams) entwerfen und konstruieren alle Bestandteile, die für Entwicklung, Betrieb und Wartung eines Softwaresystems notwendig sind:

- Bausteine:⁷ Architekten und Entwickler konstruieren Systeme aus Bausteinen, die ihrerseits wieder komplexe Subsysteme sein können. Bausteine besitzen klare *Verantwortlichkeiten*.
- Schnittstellen: Über Schnittstellen kommuniziert ein System mit der Außenwelt. Schnittstellen bilden die Grundlage der „Verträge“, auf deren Basis die Bausteine miteinander arbeiten (*design by contract*). Erst die Zusammenarbeit der Bausteine über Schnittstellen ermöglicht es dem System, seine Aufgaben zu erfüllen.
- Strukturen: Durch Bausteine und deren Zusammenwirken entwerfen Architekten sowohl statische als auch dynamische Strukturen.

Ich bin der Meinung, dass Architekten ihre Entwurfs-, Struktur- und Technologieentscheidungen am besten in einem (kleinen) Team treffen – nur in Ausnahmefällen alleine.

Architekten entscheiden

Dem bekannten Software- und Systemarchitekten Philippe Kruchten verdanken wir das Zitat zu Beginn dieses Abschnitts 2.2. Er bringt darin einige wesentliche Aspekte von Softwarearchitekten zum Ausdruck, die ich aus meiner persönlichen Erfahrung vollauf bestätigen kann:

- „... *schnelle Folge suboptimaler Entwurfsentscheidungen*“: Architekten müssen viele Entscheidungen treffen: Welche Bausteine, welche Schnittstellen, welche Abläufe? Welche technischen Frameworks? Selbst implementieren, kaufen oder einen Mittelweg davon? Welches Teilteam entwickelt welche Komponenten? Wie sollen die Bausteine der Architektur heißen?⁸
- „... *die meist im Dunkeln getroffen werden*“: Architekten wissen oftmals wenig über die langfristigen Konsequenzen ihrer Entscheidungen. Teilweise zeigt sich erst Monate (oder Jahre!) später, ob eine Architekturentscheidung vernünftig und angemessen war.

⁷ Bausteine sind Komponenten, Subsysteme, Klassen, Funktionen, Module oder andere Abstraktionen von Quellcode. Siehe Abschnitt 5.6.

⁸ Sinnvolle Benennung von „Dingen“ trägt erheblich zur Qualität von Softwarearchitekturen und deren Implementierung bei. Vergeben Sie aussagekräftige Namen – für Schnittstellen, Subsysteme, Pakete, Komponenten, Klassen, Methoden oder sonstige Bezeichner! Benennen Sie um, wenn der Name eines Architekturbausteins nicht mehr seinen Zweck widerspiegelt. Geben Sie speziellen Bezeichnungen Vorrang vor allgemeinen.

Darüber hinaus arbeiten Softwarearchitekten oftmals mit innovativen technischen Frameworks, Betriebssystemen oder sonstigen Dingen, deren genaues Verhalten sie gar nicht kennen können. In solcher Unsicherheit hilft iteratives Vorgehen weiter (siehe Abschnitt 2.3). Entscheidungen großer Tragweite sollten Sie angemessen dokumentieren. In Abschnitt 5.2 finden Sie dazu einen Vorschlag.

Architekten garantieren die Erfüllung von Anforderungen

Architekten belegen Machbarkeit

Softwarearchitekten stellen die *Machbarkeit* und *Erfüllung* von Anforderungen sicher. Sie gewährleisten, dass Anforderungen einerseits erfüllbar sind und andererseits auch erfüllt werden. Dies bezieht sich sowohl auf die funktionalen Anforderungen (*required capabilities*) als auch auf die nichtfunktionalen Anforderungen und Randbedingungen (*required constraints*).

Architekten belegen die Machbarkeit des Systems durch Prototypen.

Nicht zuletzt sorgen Architekten auch dafür, dass Systeme mit angemessenen Kosten realisiert werden können!

Architekten beraten

Softwarearchitekten beraten andere Stakeholder in architekturrelevanten Fragestellungen. Sie beraten:

- Management und Auftraggeber bei der Projektplanung und -organisation.
- Auftraggeber und Analyseteams hinsichtlich der Machbarkeit von Anforderungen. Dazu unterstützen sie bei der Bewertung von Kosten und Nutzen von Anforderungen. Sie klären die Auswirkungen von Anforderungen auf die Struktur, die Realisierung und den Betrieb von Systemen.
- Das Management bezüglich (technischer) Risiken. Architekten sollten zusätzlich auch organisatorische Risiken kennen und berücksichtigen.
- Implementierungsteams bezüglich der Umsetzung der Architektur in Software. Dazu müssen Architekten das Team von der Architektur überzeugen und bei Bedarf ausbilden und unterstützen.
- Hardwarearchitekten und Betreiber des Systems hinsichtlich der Anforderungen, die das System an die zugrunde liegende Hardware stellt.
- Qualitätssicherung und Test hinsichtlich der Kritikalität und Testbarkeit von Systembestandteilen.

Architekten dokumentieren

Damit Entwicklungsprojekte agil, flexibel und kurzfristig wandlungsfähig bleiben, müssen Architekten in hohem Maße angemessen und bedarfsgerecht arbeiten.

Angemessen dokumentieren

Im Wesentlichen kommt es auf angemessene Dokumentation und Kommunikation an. Sowohl Art als auch Umfang und Detaillierung müssen sich an den Bedürfnissen der jeweiligen Adressaten orientieren. Manchmal genügt eine kurze Skizze („auf der Rückseite eines gebrauchten Briefumschlags“), manchmal ein detailliertes UML-Diagramm und in anderen Fällen treffen umfangreiche Dokumente die Bedürfnisse der Adressaten.

Kapitel 4 zeigt Ihnen, wie Sie bedarfsgerechte Dokumentationen von Architekturen erstellen, konkrete Beispiele dafür finden Sie in Kapitel 12.

Architekten sind Diplomaten und Akrobaten

Als Diplomaten schließen Architekten Kompromisse zwischen widersprüchlichen oder konkurrierenden Forderungen.

Als Akrobaten balancieren Architekten mit einer Vielzahl von Faktoren, die sich gegenseitig beeinflussen. Tabelle 2.1 zeigt einige solcher Faktoren (nach [Rechtin2000]), die im Wettbewerb miteinander stehen (und für Architekten die Arbeit interessant machen). Bild 2.2 zeigt eine Lösung in Abhängigkeit von der Stärke solcher Forderungen und Einflüsse.

Architektur enthält Kompromisse

Tabelle 2.1 Konkurrierende Faktoren, die Kompromisse und Balance benötigen

Form	↔	Funktion
Strikte Kontrolle	↔	Agile Entscheidungen
Kosten & Termine	↔	Leistung und Qualität
Komplexität	↔	Einfachheit bzw. Flexibilität
Technische Innovation	↔	Etablierte Technologien
Top-down-Planung	↔	Bottom-up-Realisierung
Kontinuierliche Verbesserung	↔	Produktstabilität
Minimale Schnittstellen	↔	Enge Integration

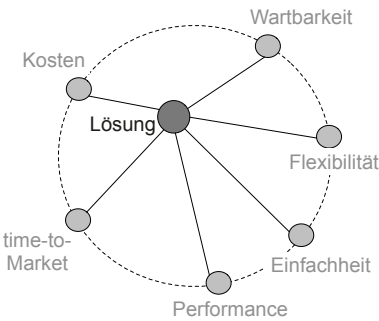


Bild 2.2 „Gummiband-Diagramm“: Lösung als Kompromiss

Architekten vereinfachen

Eine wichtige Regel für Architekten lautet: Vereinfache! Vereinfache! Vereinfache! Eine andere Lesart: Weniger ist (manchmal) mehr!⁹

Entwurfsprinzipien: Kapitel 4

Einfache Strukturen sind leichter und günstiger realisierbar, einfacher verständlich, weniger fehleranfällig. Die zuverlässigste, preiswerteste und robusteste Komponente eines Systems ist diejenige, die erst gar nicht realisiert werden muss!

Einfacher bedeutet manchmal „kleiner“ – dabei könnte die Einführung von Microservices helfen, siehe Kapitel 10. Allerdings benötigt dieser Architekturansatz erhebliche organisatorische Unterstützung, beispielsweise müssen Sie dazu cross-funktionale Teams einführen.

⁹ nach [Rechtin2000], einem Kultbuch von Systemarchitekten.

Architekten kommunizieren

Architekten müssen den anderen Stakeholdern die Architektur vermitteln und sie von den Architekturentscheidungen überzeugen. Dazu gehören sowohl der Wille als auch die Fähigkeit, technische Sachverhalte für unterschiedliche Stakeholder angemessen aufzubereiten (siehe Kapitel 5), zu präsentieren und zu diskutieren.

Gute Softwarearchitekten zeichnen sich dadurch aus, dass sie ihre Ideen, Entwürfe und Entscheidungen aktiv an die übrigen Stakeholder kommunizieren können.

- Architekten erläutern und argumentieren ihre Entscheidungen. Wenn nötig, verteidigen sie diese auch gegen Angriffe. Hierfür benötigen sie diplomatisches Geschick – davon wird im folgenden Abschnitt die Rede sein.
- Architekten überzeugen das Entwicklungsteam von Strukturen und Schnittstellen, indem sie Vor- und Nachteile transparent darstellen und unter den Rahmenbedingungen und Einflüssen des konkreten Systems gegeneinander abwägen.
- Architekten präsentieren und vermarkten die Architektur, sodass möglichst alle Beteiligten sie als DIE Architektur akzeptieren und in technischer Hinsicht an einem Strang ziehen.
- Sie unterrichten und coachen: treten als Berater, Trainer oder Lehrer auf, um das benötigte Know-how im Team zu verbreiten.

Für diese Aufgaben benötigen Softwarearchitekten starke kommunikative Fähigkeiten, sie sollten motivieren, präsentieren und argumentieren können. Fundierte technische Kenntnisse stellen lediglich eine *notwendige* Voraussetzung für diesen Teil der Architekturarbeit dar.

Architekten bewerten

Zielerreichung bewerten

Architekten müssen die Güte der Architekturen bewerten, um jederzeit den Grad der Zielerreichung zu kennen. Sie müssen wissen, ob und an welchen Stellen der Systeme nichtfunktionale Anforderungen (wie etwa Performance) riskant oder kritisch sind. Aus dieser objektiven Bewertung heraus können Architekten Maßnahmen zur Optimierung oder Risikominderung ableiten – in der Regel gemeinsam mit anderen Verantwortlichen.

Mehr zur Bewertung von Architekturen finden Sie in Kapitel 8.

Architekten explizieren

Architekten sollten wichtige Annahmen und Voraussetzungen explizit machen, um die möglichen Probleme *impliziter Annahmen* zu adressieren. Sie vermeiden dadurch mögliche Missverständnisse zwischen Beteiligten.

Architekten brauchen Mut

Unter Unsicherheit entscheiden

Am Ende eines erfolgreichen Projekts ist es einfach, Entwurfsentscheidungen zu kritisieren. Zu diesem Zeitpunkt wissen alle Beteiligten über Technologien, Produkte und auch Anforderungen genau Bescheid. Architekten mitten im Stress der Entwicklung können solche Dinge teilweise nur vermuten. Architekten verfügen aus Zeitgründen oftmals nicht über genügend Informationen, um optimale Entscheidungen zu treffen. Damit die Entwicklung weiterlaufen kann, müssen Architekten (und Entwicklungsteams) in solchen Fällen Mut zu (möglicherweise) suboptimalen Entscheidungen unter Unsicherheit aufbringen.

**Beachten Sie den Unterschied zwischen Mut und Waghalsigkeit:**

Mutig bedeutet, manche Risiken bewusst einzugehen, auch gegen den Willen anderer. Architekten benötigen Mut zu unbequemen Entscheidungen, zu potenziellen Konflikten mit anderen Projektbeteiligten, zu frühzeitigem Eingeständnis von früheren Fehlentscheidungen.

Waghalsigkeit hingegen nenne ich schnelle Entscheidungen ohne bewusste Risikoabwägung, ohne Beachtung von Konsequenzen, ohne Feedback von anderen Beteiligten oder ohne Prüfung möglicher Alternativen.

Architekten können die Schuld nicht auf diejenigen schieben, die die Anforderungen gestellt haben. Allerdings sollten Architekten die übrigen Beteiligten möglichst frühzeitig auf mögliche Konsequenzen bestimmter (kritischer) Anforderungen hinweisen (und damit das Risikomanagement ihres Managements unterstützen!).

Die Werkzeuge von Architekten:

- Modelle, also vereinfachte Abbildungen und Abstraktionen der Wirklichkeit.
- Angemessene Dokumentation dient Architekten als Grundlage effektiver Kommunikation mit anderen Beteiligten. Dokumentation enthält neben Modellen noch weitere Informationen. Siehe dazu Kapitel 5.
- Heuristiken (griechisch: *heuriskein*, einen Weg finden, führen): Erfahrungen, Regeln, Tipps. Siehe dazu insbesondere Kapitel 4.
- Muster (*patterns*): einfach anwendbare Vorlagen oder Schablonen für elegante Lösungen zu spezifischen Entwurfsproblemen.¹⁰ Durch Muster können Architekten Wiederverwendung auf hohem Niveau betreiben. Beispiele für Entwurfs- und Architekturmuster finden Sie in Abschnitt 4.5 und 4.6.
- Zerlegung (Partitionierung): Architekten beherrschen Komplexität durch Zerlegung in Teilprobleme. Sie finden mehr über Zerlegung in Kapitel 4.
- Zusammensetzung (Aggregation): das Gegenstück zur Zerlegung. Architekten setzen Einzelteile zu Softwaresystemen zusammen.
- Iteration: Softwareentwicklung benötigt iterative Prozesse und schnelles Feedback. Daher sollten auch Architekten in Iterationen und Zyklen vorgehen. Anders ausgedrückt: Bei der Vielzahl von Einflussfaktoren und Anforderungen an Software ist es häufig schwierig, schon im „ersten Wurf“ das Ziel exakt zu treffen.
- Compiler, Debugger und Prototypen: Diese Hilfsmittel benötigen Architekten, wenn es gilt, das Team bei der Implementierung zu unterstützen, die Machbarkeit von Implementierungsentscheidungen zu verifizieren oder technische Risiken zu überprüfen (*architect also implements*, nach [Coplien]).

¹⁰ In der Pattern Community sind auch Muster für organisatorische Probleme, Prozessprobleme sowie Implementierungsprobleme beschrieben. Siehe <http://www.hillside.net/patterns>

Fazit: Darum kümmern sich Softwarearchitekten!

Sie haben in den letzten Abschnitten das Aufgabenspektrum von Softwarearchitekten kennengelernt. Bild 2.3 zeigt, worum sich Softwarearchitekten kümmern sollten.

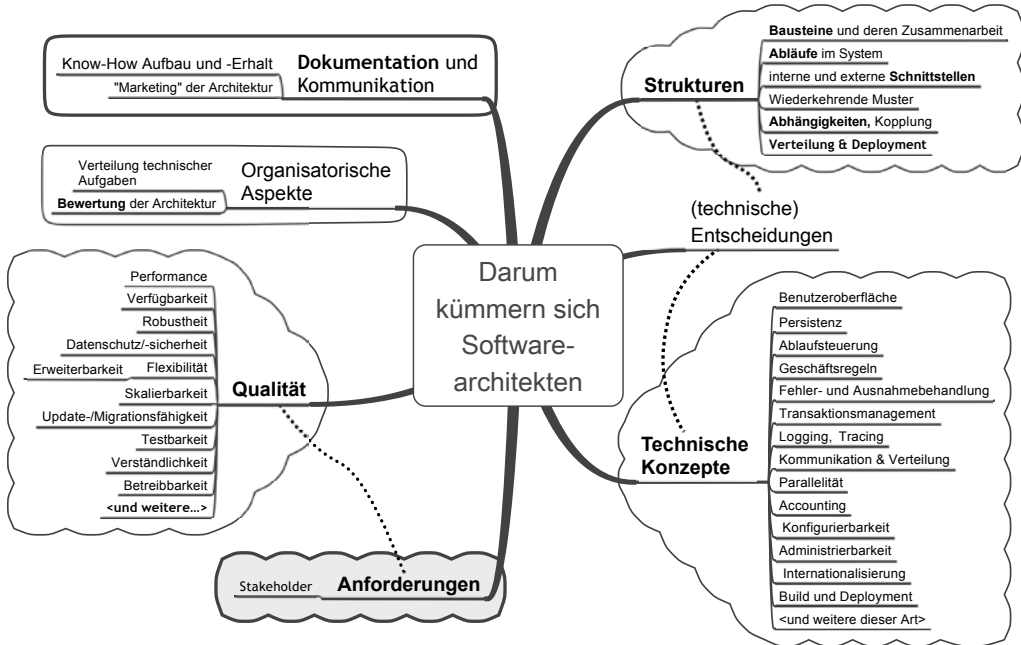


Bild 2.3 Zusammenfassung der Aufgaben von Softwarearchitekten

Der nächste Abschnitt gibt Ihnen einige Anhaltspunkte, wie Sie die Fülle dieser Aufgaben angehen können: iterativ und in kleinen Teams.

Kapitel 3 vertieft dann, wie Sie diese Aufgaben praktisch umsetzen können.

2.3 Wie entstehen Architekturen?

In Zyklen und Iterationen

Iterativ und inkrementell
vorgehen

Die Anforderungen an Software, ihre Randbedingungen und Einflussfaktoren ändern sich. Am Ende einer Implementierung sieht die tatsächliche Lösung immer etwas anders aus, als zu Beginn geplant.

Die Entwicklung von Softwaresystemen ähnelt der Verfolgung von beweglichen Zielen (*moving targets*). Bild 2.4 illustriert, wie sich Zwischenlösungen und das bewegliche Ziel im Verlauf der Entwicklung durch Iterationen immer besser einander annähern.

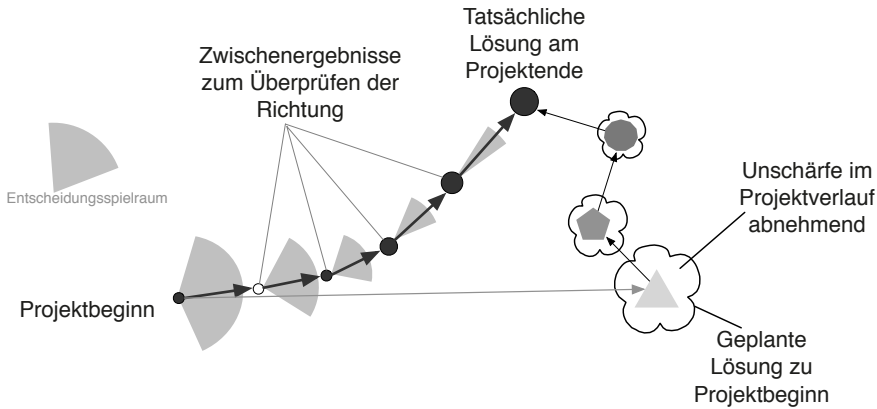


Bild 2.4 Verfolgung des „Moving Target“ im iterativen Entwicklungsprozess (mit freundlicher Genehmigung von Klaus Eberhardt, www.iteratec.de)

Gute Architekturen entstehen in Zyklen und Iterationen. Entwurfsentscheidungen können Anforderungen und damit organisatorische Abläufe beeinflussen (siehe Bild 2.5). [Bass+03] nennen diesen Zyklus den „*Architecture Business Cycle*“.

Architecture Business Cycle

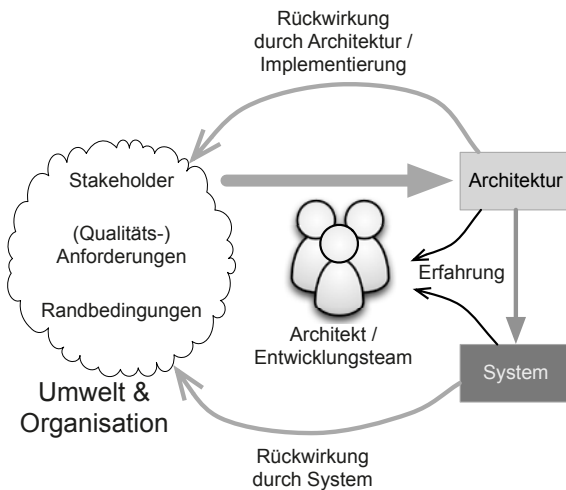


Bild 2.5 Rückwirkungen zwischen Architektur, Organisation und System (nach [Bass+03])

Die Abbildung zeigt, dass sowohl die Architektur als auch das fertige System Rückwirkungen auf die Organisation beinhalten. Architekten gewinnen durch diese Rückkopplungen Erfahrung, indem sie die Auswirkungen der von ihnen gestalteten Systeme auf Organisationen erleben. Beachten Sie in diesem Zusammenhang auch die zeitlose Regel von Conway:¹¹

¹¹ Er hat sie schon 1968 formuliert. Eine kurze Erläuterung von Open-Source-Guru Eric Raymond finden Sie unter <http://catb.org/~esr/jargon/html/C/Conways-Law.html>



Die Strukturen von Organisationen und der von ihnen entwickelten Architekturen sind isomorph.

Architekturen, Organisationen und Systeme beeinflussen sich gegenseitig. Insbesondere kommt das bei Microservices zum Tragen – siehe Kapitel 10.

In kleinen Teams

Gute Architekturen sind das Ergebnis eines kleinen Teams mit dem gemeinsamen Ziel, die Anforderungen der Auftraggeber zu erfüllen.

Gemischte Erfahrung:
Software und Fachdomäne

Zum Erfolg braucht dieses Team eine gute Mischung aus Erfahrung im Softwareengineering und in der jeweiligen Fachdomäne. Es benötigt Flexibilität hinsichtlich seiner internen Struktur und Arbeitsweise. Die einen werden vielleicht Architekten von Subsystemen, andere fachliche und organisatorische Berater des Auftraggebers.

Wichtig ist jedoch eine klar identifizierte Person¹² mit folgenden Stärken:

- In Konflikten vermitteln und sie lösen helfen
- Mutige Entscheidungen treffen. „Mutig“ bedeutet hierbei „notfalls unter Unsicherheit“.
- Motivieren
- Kommunizieren
- Strukturieren

Vorgehen: Kapitel 3 und 4

Als Antwort auf die Frage nach dem „Wie“ von Softwarearchitekturen stelle ich Ihnen in Kapitel 3 und 4 flexible *methodische Werkzeuge* vor, mit denen Sie Architekturen entwerfen können.

Wie Architekturen nicht entstehen sollten

- Im Architekturkomitee, entkoppelt von der Realität echter Systeme.
- Im Elfenbeinturm, losgelöst von Kunden, Auftraggebern und dem Realisierungsteam.
- Nur auf bunten Marketing-Folien.¹³
- Als „Wir machen jetzt <Name-der-bevorzugten-Technologie>“. Die Datenbank ist nicht die Architektur, ebenso wenig das Netzwerk, der Transaktionsmonitor, REST, die 4-GL-Sprache oder ein beliebiger Standard. Diese Begriffe sind Teile oder Aspekte der Architektur, aber die Architektur kann nicht nur aus einem isolierten Aspekt bestehen [Kruchten2001].

¹² Sie finden für diese Rolle manchmal andere Bezeichnungen, etwa: Chefarchitekt, Chief Technical Officer (CTO) oder Technischer Projektleiter.

¹³ Diese Art von Architekturdokumentation, in der Regel stark abstrahiert, vereinfacht und für nichttechnische Beteiligte ausgelegt, kann ein wichtiges Instrument zur Kommunikation der Architektur, beispielsweise an Manager oder Auftraggeber, sein.

■ 2.4 In welchem Kontext steht Architektur?

In Softwareprojekten nimmt Architektur eine der zentralen Rollen ein. Sie fungiert als Brücke zwischen Analyse und Implementierung.

Zentrale Rolle

Gleichzeitig dient die Architektur fast allen Projektbeteiligten als Leitbild oder Referenz.

Bild 2.6 zeigt, wie Softwarearchitektur mit anderen Entwicklungsaktivitäten zusammenhängt. In allen Fällen beeinflussen sich die Aktivitäten gegenseitig.

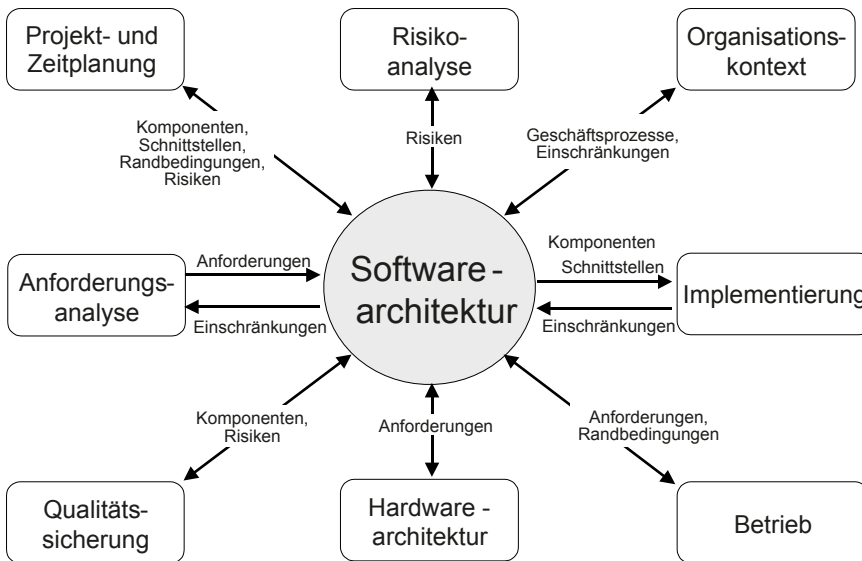


Bild 2.6 Architektur und andere Entwicklungsaufgaben

Das stellt ein weiteres Argument für die iterative Entwicklung dar. Durch Iterationen können Architekten die teilweise konkurrierenden Einflüsse und Anforderungen ausbalancieren.

Architektur und Anforderungsanalyse

In der Anforderungsanalyse werden die fachlichen und technischen Anforderungen an das System formuliert. Dazu gehören:

- funktionale Anforderungen (Required Capabilities);
- nichtfunktionale (Qualitäts-)Anforderungen (Required Constraints).

Auf Basis dieser Anforderungen und der externen Einflussfaktoren entsteht die Softwarearchitektur. Architekten prüfen die genannten Anforderungen hinsichtlich ihrer Umsetzbarkeit und stimmen gegebenenfalls Änderungen mit den Analyseteams ab.

Hierbei kommt es häufig vor, dass die Architektur weitreichende Rückwirkungen auf die Anforderungen hat. Architekten entscheiden häufig über die technische Machbarkeit fachlicher Anforderungen.

Architektur beeinflusst Anforderungen

Ein Auftraggeber wird eventuell auf eine „teure“ Anforderung verzichten, um den Erfolg des Gesamtprojekts zu sichern oder ein vernünftiges Kosten-Nutzen-Verhältnis einzuhalten. Es gehört zur Aufgabe des Architekten, die Anforderungen zu präzisieren, sie zu vervollständigen, ihre Auswirkungen zu prüfen und ihre Machbarkeit sicherzustellen!

Ein Architekt, der zu Beginn seiner Arbeit vollständige und konsistente Anforderungen benötigt, mag ein brillanter Entwickler sein – aber er ist kein Architekt. [Rechtin2000]

Anforderungsanalyse =
das Richtige entwickeln

entwickelt.

Die Anforderungsanalyse eines Systems bestimmt dessen fachlichen Umfang (*scope*). Sie stellt sicher, dass das „richtige“ System entwickelt wird. Entwurf stellt sicher, dass man das System „richtig“

Entwurf = richtig entwickeln

Der Scope eines Systems oder Projekts gehört zu den wichtigsten Entscheidungen. Ein unklar definierter und bei den Entscheidern umstrittener Umfang gehört zu den größten Risiken!

Architektur und Entwicklungsplanung

Aufgabenplanung

Bei agilem Vorgehen können Architekten mit dem Entwicklungsteam dem Scrum-Master und Product-Owner helfen, User-Stories, Features und deren Risiken und technische Konsequenzen zu durchdringen. Damit kommen Teams zu besseren Sprint-Planungen.

Falls Ihre Organisation (immer noch) kein agiles, iteratives Entwicklungsmodell eingeführt hat und mit klassischen Planungsmodellen arbeitet, besitzt Architektur großen Einfluss auf die Projektplanung. Auf Basis der zu realisierenden Komponenten und Schnittstellen kann die Projektleiterin eine detaillierte *Work Breakdown Structure* und einen entsprechenden Projektplan erstellen.¹⁴ Architektur bietet dem Management die Möglichkeit, eine bessere Aufgabenplanung vorzunehmen.

Architektur und Implementierung

Architekturen bilden den abstrakten Rahmen, der durch die Implementierung zum laufenden System wird. Architekten konstruieren (oft gemeinsam mit dem Entwicklungsteam) die zu implementierenden Bausteine, entwickeln Prototypen oder Musterlösungen als *Referenzarchitekturen*. Technische Details der Implementierung wiederum können die Architektur beeinflussen, indem sie Randbedingungen vorgeben.

Architekten wirken als Dienstleister des Entwicklungsteams, sie beraten in technischen oder strukturellen Fragen, helfen bei der Definition und Implementierung zentraler Bausteine und Schnittstellen. Oft erleichtern sie dem Team die ungeliebten Dokumentations- und Kommunikationsaufgaben. Außerdem treffen Architekten, in Abstimmung mit dem Team, wesentliche Architekturentscheidungen.

¹⁴ Leider zeigt die Praxis oftmals ein anderes Bild: Projektleiter planen ohne Kenntnis von Architekturen. Dies führt im weiteren Projektverlauf zu Problemen, weil es zwei unterschiedliche Planungsstrukturen gibt: eine gemäß der Projektleitung und eine im Sinne der Architektur.

Architektur und Risikoanalyse

Architekten können (und sollten) zur Risikoanalyse und zum Risikomanagement von Projekten erheblich beitragen. Sie können technische und organisatorische Risiken erkennen und bewerten und darauf abgestimmte, angemessene Lösungsstrategien zusammen mit dem Management entwickeln.

Risikomanagement



Arbeiten Sie als Architekt beim Risikomanagement eng mit Management, Kunden und Auftraggebern zusammen. Das gibt Ihnen die Möglichkeit, Risiken und passende Maßnahmen im Sinne von Lösungsstrategien aktiv zu steuern.

Mehr über Risikomanagement, Einflussfaktoren und Lösungsstrategien finden Sie in Kapitel 3.

Architektur und Organisationskontext

Softwaresysteme entstehen in einem organisatorischen Kontext, der ihre Entwicklung und damit verbunden auch die resultierende Architektur maßgeblich beeinflusst. Zu den organisatorischen Faktoren zählen etwa:

Organisatorische Faktoren:
Kapitel 3

- Entwicklungsprozess
- Motivation und Erfahrung aller Beteiligten
- Termin- und Kostendruck

Manche Organisationen benutzen Vorgehensmodelle (wie z. B. das V-Modell oder den Rational Unified Process [Kruchten2001]), die den gesamten Prozess der Systementwicklung mehr oder minder detailliert beschreiben. Solche Vorgehensmodelle prägen durch ihre teilweise detaillierten Vorgaben die Erstellung von Architekturen.

Umgekehrt kann die Architektur auch organisatorische Abläufe bestimmen. Betriebswirtschaftliche Ablauf- oder Prozessoptimierung kann sich an Vorgaben oder Gegebenheiten von Systemarchitekturen orientieren.

Architektur und Betrieb

Die Architektur bestimmt die Art der Laufzeitkomponenten und beeinflusst damit den Betrieb eines Systems. Die für den Betrieb verantwortlichen Stakeholder müssen eine der Architektur entsprechende Laufzeitumgebung für das System bereitstellen. Diese umfasst beispielsweise Prozessoren, Speicher, Netzwerke, Betriebssysteme, Datenbanken und sonstige Softwareservices.

(Software-)Architektur und Hardwarearchitektur

Bei der Erstellung komplexer Systeme müssen Software- und Hardwarearchitekten Hand in Hand arbeiten, weil sich beide gegenseitig beeinflussen. Insbesondere betrifft dies Qualitätsanforderungen, wie etwa Performanz, Hochverfügbarkeit oder Sicherheit.

Besonders kritisch stellt sich die Zusammenarbeit zwischen Software- und Hardwarearchitekten bei den sogenannten eingebetteten Systemen (*Embedded Systems*) dar, weil dort essenzielle Systemfunktionen sowohl von Soft- als auch von Hardware wahrgenommen werden können.

Architektur und Qualitätssicherung

Auf Basis der Softwarearchitektur kann die Qualitätssicherung eines Projekts gemeinsam mit dem Architekturteam die gestellten Anforderungen hinsichtlich ihrer Test- und Prüfbarkeit bewerten.

Daneben sollte die Qualitätssicherung auf Basis der Architektur die Testplanung vornehmen. Die sogenannten Blackbox-Tests können auch ohne Kenntnis der Architektur geplant werden, doch besitzen in vielen Fällen auch Whitebox-Tests große Bedeutung für die Qualität von Systemen. Diese Tests beziehen sich explizit auf einzelne Systemkomponenten.

■ 2.5 Weiterführende Literatur



[Coplien] beschreibt eine Vielzahl von „Organizational Patterns“, die für Architekten große Bedeutung besitzen. Dazu gehören unter anderem: „Architekt steuert das Produkt“, „Architekt implementiert“, „Entwurf zu zweit“, „Belohne Erfolg“ und einige andere. Diese Muster sind für Architekten wie für Management und Auftraggeber gleichermaßen wertvoll.

[Hofmeister2000] erläutert, wie Einflussfaktoren und Randbedingungen die Erstellung von Softwarearchitekturen prägen.

[Martin08] zeigt die Bedeutung guten Quellcodes auf. Architekten können daraus viel über Verständlichkeit und Dokumentation lernen.

[Rechtin2000] ist eine ausführliche und verständliche Einführung in „Systemarchitektur“. Die Autoren zeigen viele Gemeinsamkeiten zwischen Architekturen in unterschiedlichen Fachgebieten (etwa: Flugzeugbau, Gebäudearchitektur, Softwarearchitektur). Sie motivieren die Aufgabe der Architekten als „Anwälte der Kunden“. Eine Kernthese lautet: Architekten müssen auf Heuristiken (als kodifiziertes Erfahrungswissen) zurückgreifen. Das Buch enthält mehr als hundert solcher „Ratschläge“, die Sie auf jede Art von Architektur anwenden können.

[SEI2001] vergleicht verschiedene Definitionen des Begriffs „Softwarearchitektur“ miteinander. Der Artikel zeigt eindrucksvoll, wie unterschiedlich und vielseitig die Fachwelt diesen Begriff interpretiert.

3

Vorgehen bei der Architekturentwicklung

*Erfahrung ist die härteste Lehrerin.
Sie gibt Dir zuerst den Test und anschließend den Unterricht.*

Susan Ruth, 1993



Fragen, die dieses Kapitel beantwortet:

- Wie sollten Sie bei Architekturarbeit vorgehen?
- Was sind die typischen Aufgaben von Softwarearchitekten?
- Wie entwickeln Sie eine „erste Vorstellung“ vom System?
- Wie finden Sie die relevanten Einflussfaktoren?
- Wie berücksichtigen Sie diese Einflüsse beim Entwurf von Architekturen?
- Wie formulieren und erreichen Sie Qualitätsanforderungen?
- Welche typischen Risiken drohen bei Softwarearchitekturen?
- Welche Lösungsstrategien adressieren diese Risiken?

Wie sollen Architekten vorgehen?

Die schlechte Nachricht zuerst: Es gibt kein deterministisches Verfahren, das in jedem Fall zu guten Softwarearchitekturen führt.¹ Sie werden sich praktisch immer iterativ-inkrementell an die Lösung heranarbeiten müssen.

Und jetzt die gute Nachricht: Bild 3.1 zeigt einige grundlegende Aktivitäten, die Ihnen beim effektiven Entwurf von Architekturen helfen.

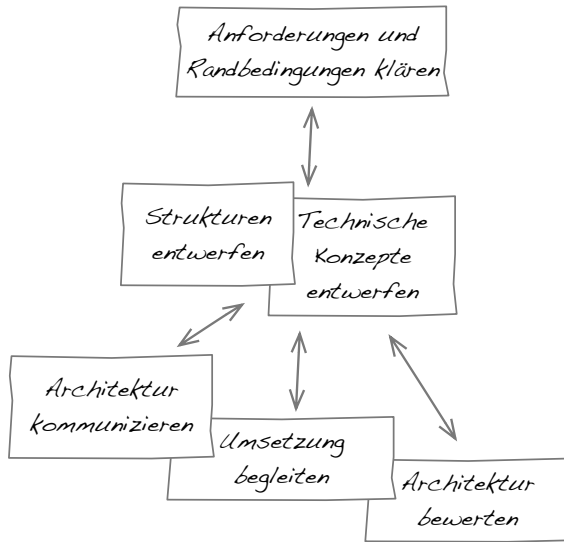
Diese Aktivitäten beziehen sich sowohl auf die Neuentwicklung als auch auf die Änderung (= Wartung, Evolution, Sanierung) von Systemen. Auf einige Besonderheiten von „Änderungs- oder Wartungsfällen“ gehe ich in Abschnitt 3.6 bzw. Kapitel 9 ein.

Sie sollten sich in jedem Fall über mögliche Wiederverwendung informieren: Wer hat eine ähnliche Aufgabe vor Ihnen gelöst und wie? Sammeln Sie Lösungsideen, anstatt immer neu zu entwerfen.²

Konzepte und Ideen
wiederverwenden

¹ Architekturen und Systeme würden dann automatisch generiert und Sie hätten dieses Buch nicht gekauft.

² Bei der Wiederverwendung helfen Ihnen Architekturmuster und -stile (finden Sie in Kapitel 4) sowie technische Konzepte (Kapitel 7).

**Bild 3.1**

Systematischer Entwurf
von Softwarearchitekturen

- Klären Sie Anforderungen mit den *maßgeblichen* Beteiligten. Im Idealfall liegen Ihnen dazu aktuelle, inhaltlich korrekte und präzise Beschreibungen aus dem *Requirements Engineering* oder von Ihren Produktverantwortlichen vor. Siehe Abschnitt 3.2.
- Identifizieren Sie spezifische Randbedingungen und Einflussfaktoren (= Einschränkungen). Diese bilden *Leitplanken* Ihrer Entwurfsentscheidungen. Aus Anforderungen und Einflussfaktoren leiten Sie Risiken ab, die den Erfolg Ihrer Architektur- und Entwicklungsarbeit bedrohen. Siehe Abschnitt 3.3.
- Nun treffen Sie (am besten in einem kleinen Team) Entwurfsentscheidungen über Strukturen und Konzepte. Entwickeln Sie Lösungsideen und -ansätze auf Basis der Anforderungen und Randbedingungen. Validieren Sie Entwürfe und Entscheidungen möglichst frühzeitig durch konkrete Implementierung. Hierzu finden Sie weitere Informationen in den Kapiteln 4 und 7.
 - Entwerfen Sie Strukturen: Welche Bausteine gibt es, wie arbeiten sie zusammen, wie und wo laufen sie ab? Diese Strukturentscheidungen folgen grundlegenden Entwurfsprinzipien, „Best Practices“ und Heuristiken. Außerdem sollten Sie Architekturstile und -muster, Referenzarchitekturen oder ähnliche Vorlagen zurate ziehen. Häufig beginnen Sie mit fachlichen Strukturen und erweitern diese dann sukzessive um technische Details. Genauer lernen Sie in Kapitel 4 (Strukturentwurf) kennen.
 - Treffen Sie Technologieentscheidungen und entwerfen Sie übergreifende (querschnittliche, oftmals technische) Konzepte wie Persistenz, Benutzeroberfläche, Verteilung, Sicherheit u. a. Das gesamte Kapitel 7 dieses Buchs unterstützt Sie dabei. Das geschieht in der Regel parallel zum Entwurf der Strukturen – daher stehen diese beiden in Bild 3.1 zusammen.
- Kommunizieren Sie Ihre Softwarearchitekturen bedarfsgerecht und angemessen. In Kapitel 5 stelle ich Ihnen dazu einige Sichten vor, die spezifische Interessen der verschiedenen Projektbeteiligten berücksichtigen. Kommunizieren Sie sowohl schriftlich als auch mündlich!

- Während der Implementierung sollten Sie kontinuierlich Rückmeldungen Ihres Entwicklungsteams einholen. Überprüfen Sie, ob Entscheidungen angemessen umgesetzt werden beziehungsweise die von Ihnen gewünschten Auswirkungen zeigen. Ein paar Tipps dazu finden Sie in Abschnitt 3.5.
- Schließlich sollten Sie sich regelmäßig fragen, ob das System und dessen Architektur den Anforderungen aller Stakeholder genügen – sowohl in funktionaler wie in qualitativer Hinsicht. Kapitel 8 zeigt Ihnen den Weg zu dieser systematischen Analyse und Bewertung. Auch daraus leiten Sie Risiken ab – die Ihnen wiederum Anregungen für konkrete Entwurfsentscheidungen geben.

Architekturen entstehen iterativ und inkrementell

Einige Aktivitäten aus Bild 3.1 können parallel stattfinden – in Ihrem Kopf oder auf verschiedene Personen verteilt. In jedem Fall klären Sie regelmäßig, welche Anforderungen und Einflussfaktoren sich geändert haben oder welche neuen Erkenntnisse das Entwicklungsteam gewonnen hat – daraufhin müssen Sie eventuell Ihre Entscheidungen überarbeiten und die Architektur Ihres Systems anpassen.

Architektur entsteht iterativ

Gehen Sie inkrementell vor, d. h., treffen Sie neue, zusätzliche Entscheidungen auf der Grundlage früherer Erfahrungen und vorhandener Strukturen, Konzepte und Implementierung. Auf diese Weise kann Ihr System auf der Basis von Feedback und konkreter Erfahrung *wachsen*.

Ich möchte Ihnen das iterative Vorgehen bei der Architekturentwicklung noch aus einer weiteren Perspektive verdeutlichen (als ergänzende Erklärung zu Bild 3.1):

Systeme müssen sowohl fachliche Anforderungen als auch Qualitätsanforderungen erfüllen. Sie können sich als Architekt oder Entwicklungsteam diesem Problem aus unterschiedlichen Richtungen annähern:

- top-down: vom Abstrakten zum Detaillierten,
- bottom-up: von Details oder feingranularen Teilen zum Abstrakten,
- outside-in: ausgehend von externen Schnittstellen das Innere des Systems gestalten.

Bild 3.2 zeigt eine solche Perspektive auf iteratives Vorgehen bei der Architekturentwicklung:

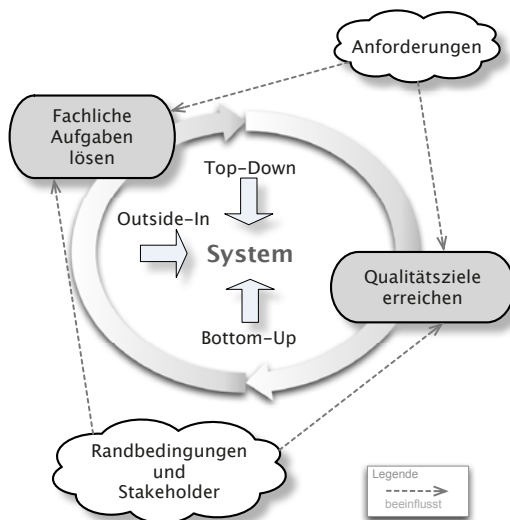


Bild 3.2

Fachliche und Qualitätsziele iterativ erreichen

Architekturen entstehen (meistens) im Team

Softwarearchitekten sollten meiner Ansicht nach mit (überschaubaren) Teams entscheiden, um einerseits das Wissen dieser Teams zu nutzen, andererseits die Akzeptanz von Entscheidungen frühzeitig zu sichern.



Verabschieden Sie sich von dem Gedanken, Softwarearchitekturen „ein für alle Mal“ zu entwickeln. Arbeiten Sie iterativ. Entwickeln Sie Ihre Entwürfe in Zyklen. Bleiben Sie agil und nehmen Sie Rückkopplungen der Projektbeteiligten und der Organisation in Ihre Entwürfe auf. So entstehen Architekturen, die an den wirklichen Bedürfnissen und Anforderungen orientiert sind!

Bild 3.3 zeigt schematisch, dass sowohl Ihre Kunden wie auch andere Projektbeteiligte Anforderungen und Einflussfaktoren ändern. Aufgrund dieser Änderungen müssen Sie Ihren Architekturentwurf und Ihre Architekturentscheidungen iterativ den veränderten Gegebenheiten anpassen.

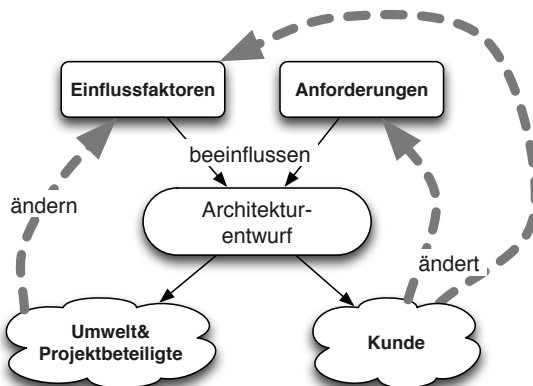


Bild 3.3

Iterativer Entwurf aufgrund veränderter Anforderungen

Im vorigen Kapitel haben Sie unter dem Begriff „Architecture Business Cycle“ bereits eine andere Art der *Iteration* von Architekturen kennengelernt.

Treffen Sie Architektur- und Entwurfsentscheidungen bewusst, angemessen und systematisch

Machen Sie sich vor und während Ihrer Architekturentwicklung bewusst, welche architekturrelevanten Entscheidungen Sie oder andere Stakeholder treffen müssen. Stimmen Sie mit betroffenen Stakeholdern ab, ob und wann diese Entscheidung notwendig wird – es kann sowohl für schnelle wie auch für verzögerte Entscheidungen gute Gründe geben.

Entscheidungen dokumentieren

Wenn Sie eine Entscheidung für besonders wichtig oder bemerkenswert halten, dann sollten Sie diese dokumentieren: Halten Sie dazu den genauen Entscheidungsumfang („Was ist das Problem?“) sowie die Auslöser für diese Entscheidung („Warum müssen wir das entscheiden?“) fest sowie

die Rahmenbedingungen, Ihre getroffenen Annahmen, Risiken und mögliche (verworfen) Alternativen.³ In jedem Fall sollten Sie Ihre Entscheidung begründen.

Entwickeln Sie Mut zu Entscheidungen! Akzeptieren Sie, dass Sie so manche unsichere Entscheidung treffen müssen (und erinnern Sie sich bisweilen an das Zitat von Philippe Kruchten aus Abschnitt 2.2 ...).

■ 3.1 Informationen sammeln

Noch bevor Sie eigene Lösungsideen entwickeln, sollten Sie recherchieren, ob und wie andere vor Ihnen ähnliche Probleme gelöst haben:

Quellen für Wiederverwendung finden



- Beginnen Sie mit Ihrer eigenen Erfahrung – Ihre Kunden erwarten sowohl Domänenwissen als auch technisches Wissen von Ihnen.
- Prüfen Sie die Systeme innerhalb Ihrer Organisation, ob sich Ergebnisse wiederverwenden lassen. Seien Sie dabei pragmatisch!
- Suchen Sie im Internet nach Beschreibungen ähnlicher Systeme. Vielleicht können Sie passende Komponenten dazukaufen.
- Suchen Sie in der technischen Literatur und durchforsten Sie die Entwurfsmuster auf Ihrem Gebiet.

In vielen Fällen bekommen Sie dadurch Beispiele und Muster und müssen nur noch kleine Teile völlig neu entwerfen.



- Sie sollten ein tiefes Misstrauen gegenüber solchen Lösungen hegen, die sich in den Grundzügen nicht auf bekannte Dinge zurückführen lassen. Nur die wenigsten, die solche (angeblich innovativen) Lösungen präsentieren, sind wirklich geniale Erfinder – die meisten haben nur schlecht recherchiert.

³ Stefan Zörner schlägt vor, bei wichtigen Entscheidungen auch die Namen der Entscheider festzuhalten.

■ 3.2 Anforderungen klären

Als Grundlage Ihrer Entwurfstätigkeiten sollten Sie eine Vorstellung wichtiger Systemeigenschaften und -anforderungen besitzen und mindestens folgende Fragen über das System beantwortet haben:

- Was ist die Kernaufgabe des Systems (= funktionale Anforderungen)? Welches sind die wichtigsten Elemente der Fachdomäne?
- Um welche Kategorie von System handelt es sich?
- Welche Qualitätsanforderungen muss das System erfüllen (= Architekturziele)?
- Welche Personen oder Gruppen haben Interessen am System (= Stakeholder)?
- Was sind die (fachlichen und technischen) Nachbarsysteme (= externe Schnittstellen, Kontextabgrenzung)?

Die Antworten auf diese Fragen werden Ihnen (und dem Entwicklungsteam) eine Orientierung für erste Lösungsansätze geben.

3.2.1 Was ist die Kernaufgabe des Systems?



- Beschreiben Sie die Kernaufgabe und Verantwortlichkeit des Systems in zwei bis drei Sätzen. Formulieren Sie positiv und benutzen Sie die Kernbegriffe der Fachdomäne.
- Fügen Sie die wichtigsten Begriffe oder Aspekte der Fachdomäne hinzu; wenige Begriffe (fünf bis zehn) genügen häufig.
- Stimmen Sie diese Formulierung mit Kunden und Auftraggebern ab!



Beispiel: Das System unterstützt Call-Agents bei der Erfassung und Bearbeitung von Schadensmeldungen von Privatkunden.

Die wichtigsten Aspekte der Fachdomäne sind Vorfall, Schaden, Partner und Vertrag.

Eine solche kurze Formulierung erleichtert die Kommunikation über das System. Sie definiert für alle Beteiligten das wichtigste Ziel (= das System). Gleichzeitig schafft sie einen begrifflichen Kontext, an dem sich alle Beteiligten orientieren können.

Manchmal können Sie diese Aussagen aus der Anforderungsanalyse übernehmen. Andernfalls müssen Sie die Kernaufgabe mit Ihren Auftraggebern oder Kunden selbst formulieren.

3.2.2 Welche Kategorie von System?

Meist geben die Kernaufgaben eines Systems bereits Aufschluss, zu welcher der folgenden Kategorien es gehört:

- **Interaktive Online-Systeme:** Auch als operationale Systeme bezeichnet, arbeiten diese Systeme als Teil der normalen Geschäftsprozesse in Unternehmen. In den meisten Fällen enthalten sie Operationen auf Daten (Transaktionen, Einfüge-, Änderungs- und Löschoperationen), die vom Ablauf her in die Benutzeroberfläche eingebettet sind. Die Art der Transaktionen ist festgelegt. Die Systeme operieren auf möglichst aktuellen Datenbeständen. Sie erfordern ein hohes Maß an Systemverfügbarkeit und Performance.
- **Mobile Systeme:** Eine (moderne) Variante von Online-Systemen – durch die starke Verbreitung von Smartphones und Tablet-Computern sehr wichtig geworden. Starker Fokus auf Benutzeroberfläche und Benutzbarkeit, oftmals per Internet an Backend-Systeme angebunden.
- **Entscheidungsunterstützungssysteme (*decision support system*):** arbeiten oftmals auf Kopien der aktuellen Unternehmensdaten (*data warehouse*) und enthalten hauptsächlich lesende Datenzugriffe. Die Art der Anfragen an die Daten ist flexibel. Benutzer können neue Anfragen (*queries*) formulieren. Daher ist die Laufzeit von Anfragen im Vorfeld kaum abschätzbar. Solche Systeme tolerieren höhere Ausfallzeiten und geringere Performance.
- **Hintergrundsysteme (Offline-Systeme, Batch-Systeme):** dienen hauptsächlich der Datenmanipulation, oftmals zur Vor- oder Nachverarbeitung vorhandener Datenbestände; werden zur Interaktion mit anderen Systemen eingesetzt. In Kapitel 7 finden Sie mehr zum Thema Integration.
- **Eingebettete Systeme (*embedded systems*):** arbeiten eng verzahnt mit spezieller Hardware. Ein Beispiel für eingebettete Systeme sind Mobiltelefone (viele haben auch Echtzeitanforderungen).
- **Systeme mit Echtzeitanforderungen (*real-time systems*):** Operationen werden innerhalb garantierter Zeiten fertiggestellt. Beispiel: Produktionssteuerung (Fließbänder mit festen Taktzeiten), Herzschrittmacher. Solche Systeme werden im Folgenden nicht mehr weiter ausführen – sie sind eine Wissenschaft für sich. ☹



Wenn das System zu mehreren der genannten Kategorien gehört, sollte die Architektur aus entsprechenden Teilsystemen bestehen, die jeweils zu einer Kategorie gehören.

3.2.3 Wesentliche Qualitätsanforderungen ermitteln

Qualität, laut Duden definiert als „Beschaffenheit, Güte, Wert“, stellt ein wichtiges Ziel für Softwarearchitekten dar. Bei Qualität handelt es sich allerdings um ein vielschichtiges Konzept, das mit einer Reihe gravierender Probleme behaftet ist:

- **Qualität ist nur indirekt messbar:** Es gibt kein absolutes Maß für die Qualität eines Produkts, höchstens für einzelne Eigenschaften (etwa: Zeit- oder Ressourceneffizienz).

- Qualität ist relativ: Verschiedene Stakeholder haben unterschiedliche Qualitätsbegriffe und -anforderungen.
 - Management fordert oft Kosteneffizienz, Flexibilität und Wartbarkeit.
 - Endanwender fordern hohe Performance und einfache Benutzbarkeit.
 - Entwicklungsverantwortliche fordern oft Parallelisierbarkeit der Implementierung.
 - Betreiber fordern Administrierbarkeit und Sicherheit.
- Qualität der Architektur korreliert nicht notwendigerweise mit der Qualität des Endprodukts: Gute Architekturen können schlecht implementiert sein und dadurch die Qualität des Gesamtsystems mindern. Aus hervorragendem Quellcode kann jedoch nicht auf die Qualität der Architektur geschlossen werden. Insgesamt gilt daher: Architektur ist für die Qualität eines Systems notwendig, aber nicht hinreichend.
- Die Erfüllung sämtlicher funktionaler Anforderungen lässt kaum Aussagen über die Erreichung der Qualitätsanforderungen zu. Betrachten Sie das Beispiel eines einfachen Sortierverfahrens: Die (triviale) Anforderung, eine Menge von Variablen gemäß einem vorgegebenen Sortierkriterium aufsteigend zu ordnen, ist eine beliebte Programmieraufgabe für Einsteiger. Nun denken Sie an einige zusätzliche Qualitätsanforderungen, etwa:
 - Sortierung großer Datenmengen (Terabyte), die nicht mehr zeitgleich im Hauptspeicher gehalten werden können,
 - Sortierung robust gegenüber unterschiedlichen Sortierkriterien (Umlaute, akzentuierte Zeichen, Phoneme, Ähnlichkeitsmaße und anderes),
 - Sortierung für viele parallele Benutzer,
 - Sortierung unterbrechbar für lang laufende Sortiervorgänge,
 - Erweiterbarkeit um weitere Algorithmen, beispielsweise für ressourcenintensive Vergleichsoperationen,
 - Entwickelbarkeit im räumlich verteilten Team.

Diese Qualitätsanforderungen können von naiven Implementierungen nicht erfüllt werden – dazu bedarf es grundlegender architektonischer Maßnahmen!

Viele Publikationen⁴ über Softwarearchitektur ignorieren das Thema der Qualitätsanforderungen völlig. Es scheint fast, als fielen Verständlichkeit, Wartbarkeit und Performance von Systemen als Nebenprodukte ab, wenn eifrige Entwickler nur in ausreichender Menge Design- und Architekturmuster anwenden⁵. Das Gegenteil ist der Fall:



Qualitätsanforderungen müssen explizite Entwurfsziele sein. Treffen Sie Entscheidungen zur Erreichung solcher Ziele bewusst und frühzeitig. Qualität müssen Sie mit Ihren Entwicklungsteams explizit konstruieren – sie entsteht nicht von selbst!

⁴ Asche auf mein Haupt – in der ersten Auflage dieses Buchs habe ich die gleiche Unterlassung begangen. Sie halten zum Glück eine neuere Auflage in der Hand. ☺

⁵ Leider helfen selbst Ansätze wie „clean-code“ nur begrenzt: Auch lupenreiner „clean-code“ kann elementare Qualitätsanforderungen wie Datensicherheit, Benutzbarkeit oder Betriebbarkeit verletzen!

Typische Qualitätsanforderungen

Die Qualität von Softwaresystemen wird immer bezogen auf einzelne Eigenschaften oder Merkmale. Beispiele für solche Merkmale sind Effizienz (Performance), Verfügbarkeit, Änderbarkeit und Verständlichkeit.

Es gibt eine ganze Reihe unterschiedlicher Definitionen von Qualitätsmodellen und Qualitätsmerkmalen. Alle definieren einige zentrale Qualitätseigenschaften (beispielsweise Zuverlässigkeit, Effizienz, Wartbarkeit, Portabilität etc.) und verfeinern diese Eigenschaften durch eine Hierarchie weiterer Merkmale.

Egal, welches dieser Modelle Sie verwenden: Achten Sie innerhalb Ihrer Systeme oder Projekte auf eine einheitliche Terminologie für Qualitätsanforderungen.

Tabelle 3.1 zeigt Ihnen die Qualitätsmerkmale gemäß DIN/25010 in verkürzter Form. Diese Norm enthält die wesentlichen Begriffe rund um Softwarequalität.

Tabelle 3.1 Qualitätsmerkmale nach DIN/ISO 25010

Funktionale Eignung	Vorhandensein von Funktionen, die explizite oder implizite Anforderungen erfüllen
▪ Angemessenheit	Liefern der richtigen oder vereinbarten Ergebnisse oder Wirkungen, z. B. die benötigte Genauigkeit von berechneten Werten
▪ Genauigkeit	Liefert das System Ergebnisse mit dem erforderlichen Grad an Präzision?
Zuverlässigkeit	Fähigkeit, das Leistungsniveau unter festgelegten Bedingungen über einen festgelegten Zeitraum aufrechtzuerhalten
▪ Verfügbarkeit	Grad, in dem eine Softwarekomponente einsatzbereit und bei Bedarf verfügbar ist
▪ Fehlertoleranz	Fähigkeit, ein spezifiziertes Leistungsniveau bei Softwarefehlern oder Nichteinhaltung einer spezifizierten Schnittstelle zu bewahren
▪ Wiederherstellbarkeit	Fähigkeit, bei einem Versagen das Leistungsniveau wiederherzustellen und direkt betroffene Daten wiederzugewinnen
Effizienz (engl.: performance efficiency)	Verhältnis zwischen dem Leistungsniveau und dem Umfang der eingesetzten Ressourcen (etwa: Speicher, CPU)
▪ Zeitverhalten	Antwort- und Verarbeitungszeiten sowie Durchsatz bei der Funktionsausführung
▪ Verbrauchsverhalten	Anzahl und Dauer der benötigten Betriebsmittel für die Erfüllung der Funktionen
Betreibbarkeit	Grad, in dem das System verstanden, gelernt, benutzt und für Benutzer attraktiv ist
▪ Erkennbarkeit der Angemessenheit	Grad, in dem das System es Benutzern ermöglicht zu erkennen, ob es für ihre Bedürfnisse geeignet ist
▪ Erlernbarkeit	Aufwand für Benutzer, die Anwendung des Systems zu erlernen

Tabelle 3.1 (Fortsetzung) Qualitätsmerkmale nach DIN/ISO 25010

▪ Einfachheit der Benutzung	Grad, in dem das Softwareprodukt es den Benutzern leicht macht, es zu bedienen und zu kontrollieren
▪ Hilfsbereitschaft	Grad, in dem das System Hilfe bereitstellt, wenn nötig
▪ Attraktivität	Grad, in dem das System attraktiv für Benutzer ist
▪ Zugänglichkeit	Grad der Bedienbarkeit für Benutzer mit bestimmten Einschränkungen
Sicherheit	Schutz vor versehentlichem oder böswilligem Zugriff, Nutzung, Änderung, Zerstörung oder Offenlegung
▪ Vertraulichkeit	Grad, in dem das System Schutz vor unbefugter Offenlegung von Daten oder Informationen bietet
▪ Integrität	Grad, in dem Genauigkeit und Vollständigkeit von Daten gewährleistet ist
▪ Nicht-Abstreitbarkeit	Grad, in dem Handlungen nachweislich stattgefunden haben, so dass sie später nicht zurückgewiesen werden können
▪ Verantwortlichkeit	Grad, in dem die Handlungen einer Entität (Person oder System) eindeutig auf diese Entität zurückgeführt werden können
▪ Authentizität (Glaubwürdigkeit)	Grad, in dem die Identität eines Subjekts oder einer Ressource als die beanspruchte nachgewiesen werden kann
Kompatibilität	Grad, zu dem ein System Informationen mit anderen Systemen austauschen kann und/oder seine erforderlichen Funktionen ausführen kann, während es dieselbe Hardware- oder Softwareumgebung teilt
▪ Ersetzbarkeit	Fähigkeit, das System anstelle eines anderen für den gleichen Zweck in der gleichen Umgebung zu verwenden
▪ Koexistenz	Fähigkeit, mit anderer Software in einer gemeinsamen Umgebung ohne nachteilige Auswirkungen zu koexistieren
▪ Interoperabilität	Fähigkeit, mit vorgegebenen Systemen zusammenzuwirken. Hierunter fällt auch die Einbettung in die Betriebsinfrastruktur.
Wartbarkeit	Aufwand, der zur Durchführung vorgegebener Änderungen notwendig ist Änderungen: Korrekturen, Verbesserungen oder Anpassungen an Änderungen der Umgebung, der Anforderungen und der funktionalen Spezifikationen
▪ Modularität	Grad, zu dem ein System oder Computerprogramm aus einzelnen Komponenten besteht, so dass eine Änderung einer Komponente nur minimalen Einfluss auf andere Komponenten hat
▪ Wiederverwendbarkeit	Grad, zu dem Systemteile für mehr als ein System oder zur Entwicklung weiterer Systeme genutzt werden können
▪ Analysierbarkeit	Aufwand, um Mängel oder Ursachen von Versagen zu diagnostizieren oder um änderungsbedürftige Teile zu bestimmen
▪ Modifizierbarkeit (Änderbarkeit)	Aufwand zur Ausführung von Verbesserungen, zur Fehlerbeseitigung oder Anpassung an Umgebungsänderungen

Tabelle 3.1 (Fortsetzung) Qualitätsmerkmale nach DIN/ISO 25010

▪ Stabilität	Wahrscheinlichkeit des Auftretens unerwarteter Wirkungen von Änderungen
▪ Prüfbarkeit	Aufwand, der zur Prüfung der geänderten Software notwendig ist
Übertragbarkeit	Eignung der Software, von einer Umgebung in eine andere übertragen zu werden. Umgebung kann organisatorische Umgebung, Hardware- oder Softwareumgebung einschließen.
▪ Anpassbarkeit	Software an verschiedene festgelegte Umgebungen anpassen
▪ Installierbarkeit	Grad, in dem das System in einer bestimmten Umgebung erfolgreich installiert und deinstalliert werden kann

Qualitätsanforderungen über Szenarien konkretisieren

Jetzt kennen Sie zwar die wichtigsten Qualitätsmerkmale, Sie benötigen aber noch ein Mittel, um diese Merkmale praxisgerecht für Ihre Systeme zu konkretisieren. Hierzu eignen sich Szenarien (nach [Bass+12]). Szenarien beschreiben, was beim Eintreffen eines Stimulus auf ein System in bestimmten Situationen geschieht.

Ich möchte Ihnen die Anwendung von Szenarien zur Konkretisierung von Qualitätsanforderungen anhand einiger Beispiele⁶ verdeutlichen.

- *Anwendungsszenarien (beschreiben das Verhalten des Systems bei seiner Nutzung):*
 - Das System muss bei den Benutzern nach einer Angebotsanfrage im Regelbetrieb in weniger als fünf Sekunden eine Antwort anzeigen. Im Betrieb unter Hochlast (Jahresendgeschäft) darf eine Antwort bis zu 15 Sekunden dauern; in diesem Fall ist vorher ein entsprechender Hinweis darzustellen.
 - Ein Benutzer ohne Vorkenntnisse muss bei der erstmaligen Verwendung des Systems innerhalb von 15 Minuten in der Lage sein, die gewünschte Funktionalität zu lokalisieren und zu verwenden.
 - Bei Eingabe illegaler oder fehlerhafter Daten in die Eingabefelder muss das System entsprechende spezifische Hinweistexte ausgeben und anschließend im Normalbetrieb weiterarbeiten.
- *Änderungsszenarien (beschreiben gewünschte Reaktionen bei Änderungen am System oder der Umgebung):*
 - Die Programmierung neuer Versicherungstarife muss in weniger als 30 Personentagen möglich sein.
 - Die Unterstützung einer neuen Browser- oder Client-JDK-Version muss in weniger als 30 Personentagen programmiert und getestet werden können.
 - Bei Ausfall einer CPU muss das Ersatzsystem (Hot-Spare) im Normalbetrieb innerhalb von 15 Minuten online sein.
 - Die Anbindung eines neuen CRM-Systems⁷ muss innerhalb von 60 Tagen möglich sein.

⁶ Zahlreiche Beispiele von Szenarien für Qualitätsanforderungen finden Sie unter <https://github.com/arc42/quality-requirements>

⁷ CRM (Customer-Relationship-Management, deutsch Verwaltung von Kundenbeziehungen) hat das Ziel, Kundenbeziehungen in einem Unternehmen zu organisieren und somit die Kundenzufriedenheit und -bindung sowie die Profitabilität zu erhöhen (nach: [Wikipedia]).

Bestandteile von Szenarien

Nach diesen Beispielen können Sie sicherlich etwas Methodik vertragen: Szenarien bestehen aus folgenden wesentlichen Teilen (siehe Bild 3.4):

- Bestimmte Ereignisse (Stimuli) werden von Stakeholdern oder anderen Systemen ausgelöst. Beispiele: Ein Benutzer ruft eine Funktion auf, ein Entwickler programmiert eine Erweiterung, ein Sensor liefert einen Messwert.
- Die Antwort (*Reaktion*) beschreibt, wie das System auf den Auslöser reagieren soll. Wird die vom Benutzer aufgerufene Funktion ausgeführt? Wie lange benötigt der Entwickler zur Programmierung? Welche Systemteile sind von Installation/Konfiguration betroffen?
- Metrik beschreibt einen Messwert oder eine Größe dieser Antwort. Beispiele: Ausfallzeit in Stunden, Korrektheit Ja/Nein, Änderungszeit in Personentagen, Reaktionszeit in Sekunden.

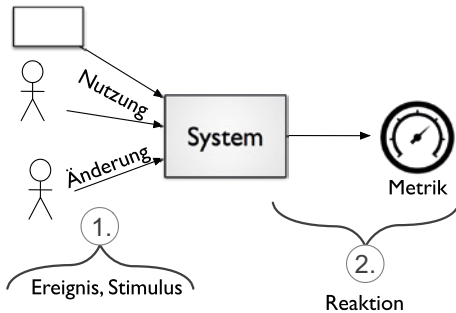


Bild 3.4
Schematische Darstellung
von Szenarien (nach [Bass+12])



Qualitätsanforderungen explizit über Szenarien zu beschreiben ist einfach und deswegen nützlich!

Zusätzlich dienen diese Szenarien als wesentliche Grundlage der systematischen Architekturbewertung, die Sie in Kapitel 8 näher kennenlernen.

3.2.4 Relevante Stakeholder ermitteln

Stakeholder

Welche Rollen oder Personen (= Stakeholder) innerhalb oder außerhalb der Organisation haben ein Interesse am System? Beispiele sind Benutzer der Kernfunktionalität („Anwender“), Administratoren und Betreiber, Benutzer mit Sonderfunktionen (Prüfer, Auditoren oder Ähnliche).

Welche Stakeholder stehen dem (neuen) System negativ gegenüber? Die jeweilige Einstellung der Stakeholder prägt die Art der Informationen, die sie Ihnen über das System geben werden.⁸ Insbesondere müssen Sie wissen, welche Erwartungshaltung die Stakeholder gegenüber dem Architektur- und Entwicklungsteam haben, welche Artefakte oder Dokumente Sie liefern und pflegen müssen.

⁸ Softwaresysteme sind soziale Systeme. Architekten werden daher häufig mit subjektiven Eindrücken statt Tatsachen konfrontiert, mit Meinungen statt Fakten, mit unrealistischen Erwartungen statt begründeter Anforderungen. Stellen Sie sich darauf ein!

Ich empfehle Ihnen, die Stakeholder Ihres Systems als Tabelle in die Architekturdokumentation aufzunehmen – wobei Sie folgende Informationen festhalten sollten:

Tabelle 3.2 Aufbau der Stakeholder-Tabelle

Rolle	Welche Aufgabe/Verantwortung tragen diese Stakeholder bezüglich des Systems?
Beschreibung	
Ziel/Intention	Welche konkreten Ziele verfolgen die Stakeholder bezüglich des Systems?
Erwartungshaltung	Welche konkreten Ergebnisse, Dokumente, Artefakte erwarten diese Stakeholder von Ihnen, der Architektur oder dem Entwicklungsteam?
Kontakt ⁹	Sie benötigen für viele Ihrer Aufgaben Feedback der Stakeholder, daher sollten Sie deren Kontaktinformationen (E-Mail, Telefon) kennen.
Relevanz für Abnahme	Können diese Stakeholder die Abnahme oder den Einsatz des Systems erlauben, fördern oder behindern?

Beispiele für Stakeholder

Nachfolgend nenne ich Ihnen einige Beispiele für Stakeholder, die in meinen eigenen Projekten relevant waren (nicht alle auf einmal ...):



Management, Auftraggeber, Projektsteuerkreis, sonstige Projektgremien, PMO, Projektmanager, Produktmanager, Fachbereich, Unternehmens-/Enterprise-Architekt, Architekturabteilung, Methodenabteilung, QS-Abteilung, IT-Strategie, (interner) Softwarearchitekt, Softwaredesigner, Entwickler, Tester, Konfigurationsmanager, Build-Manager, Release-Manager, Wartungsteam, externe Dienstleister, Zulieferer, Hardwaredesigner, Rollout-Manager, Infrastrukturplaner, Sicherheitsbeauftragter, Behörde, Aufsichtsgremium, Auditor, Mitbewerber/Konkurrent, Endanwender, Fachpresse, Fachadministrator, Systemadministrator, Operator, Hotline, Betriebsrat, Lieferant von Standardsoftware, verbundene Projekte, Normierungsgremium, Gesetzgeber ...

3.2.5 Fachlichen und technischen Kontext ermitteln

Sie sollten immer eine präzise Vorstellung der (fachlichen wie technischen) Umgebung eines Systems besitzen (= Kontext), d. h. welche Nachbarsysteme oder Benutzergruppen mit dem System interagieren.

Klären Sie möglichst frühzeitig:

- Welche Schnittstellen gibt es zu externen Systemen? Hierzu zählen:
 - Schnittstellen anderer genutzter Systeme,
 - Schnittstellen, die für andere Systeme bereitgestellt werden.

⁹ Das mag formalistisch klingen, ist im Entwicklungsalltag jedoch praktisch zu wissen!

- Sind diese externen Schnittstellen stabil und zuverlässig oder volatil? Das bedeutet: Sind die Fremdsysteme bezüglich ihrer Schnittstellen stabil?
- Wie fehlertolerant muss das neue System gegenüber den fremden Systemen sein?
- Handelt es sich um funktionale Schnittstellen oder Datenschnittstellen? Können Sie über diese Schnittstellen Funktionen oder Methoden des anderen Systems aufrufen oder funktioniert die Schnittstelle nur über den Austausch von Daten?
- Kommuniziert das System über die Schnittstellen synchron oder asynchron?
- Wie performant sind die anderen Systeme? Wie beeinflusst das Verhalten der Schnittstellen die Performance des Gesamtsystems?
- Gibt es die Möglichkeit, die Schnittstelle auf Seiten des anderen Systems zu ändern? Liegt der Quellcode der Schnittstelle vor?



- Dokumentieren Sie, welche Schnittstellen Sie bis jetzt identifiziert haben.
- Wenn das System Schnittstellen zu Fremdsystemen besitzt, sollten Sie über einen funktionalen Prototyp die technische Machbarkeit „end-to-end“ verifizieren. Insbesondere kritische oder riskante Schnittstellen sollten Sie möglichst frühzeitig auf diese Weise überprüfen.

Kontext als Diagramm – Erläuterung als Tabelle

Ich rate Ihnen, den Kontext Ihres Systems grafisch zu dokumentieren – und dieses Diagramm dann mit einer Tabelle zu erläutern (Bild 3.5).

Tabellarisch erklären Sie dazu die genauen Bezeichnungen aller Schnittstellen, die Richtung (ob sie aus der Sicht Ihres Systems benötigte (*required*) oder angebotene (*provided*) Schnittstellen sind) und die fachliche Bedeutung in Stichworten.

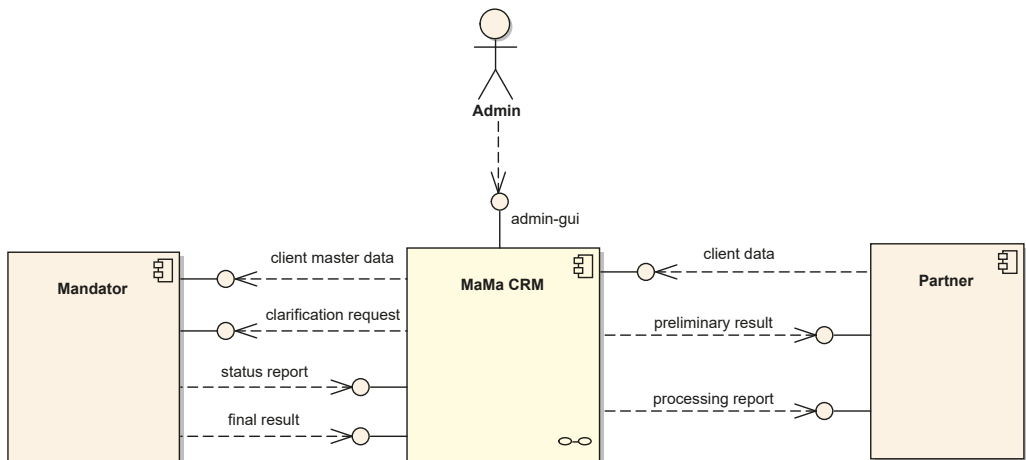


Bild 3.5 Beispiel einer Kontextabgrenzung

■ 3.3 Einflussfaktoren und Randbedingungen ermitteln

Um als Softwarearchitekt anwendungs- und problembezogene Entwurfsentscheidungen zu treffen, müssen Sie die Faktoren kennen, die Ihre Architekturen beeinflussen oder einschränken werden (siehe dazu Bild 3.6). Diese Faktoren schränken Sie in Ihren Entscheidungen ein und besitzen daher einen prägenden Einfluss auf das Gesamtsystem.

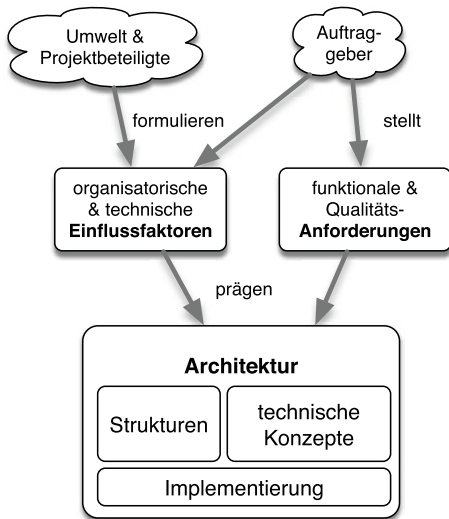


Bild 3.6

Anforderungen und Einflussfaktoren prägen den Architekturentwurf.

Aus den Systemanforderungen sowie den Anforderungen der Systemumwelt ermitteln Sie architekturrelevante Einflussfaktoren. Im Folgenden stelle ich Ihnen einige typische Einflussfaktoren vor.

Randbedingungen und Einflussfaktoren fallen in folgende Kategorien:

- Organisatorische und politische Faktoren. Manche solcher Faktoren wirken auf ein bestimmtes System ein. Andere beeinflussen sämtliche Projekte innerhalb einer Organisation. [Rechtin2000] charakterisiert diese Faktoren als *facts of life*.
- Technische Faktoren. Sie prägen das technische Umfeld des Systems und seiner Entwicklung.

Meiner Erfahrung nach tendieren Softwarearchitekten dazu, die politischen und organisatorischen Faktoren zu unterschätzen und zu vernachlässigen. Das kann im Extremfall dazu führen, dass an sich lauffähige Systeme nicht zum Einsatz kommen.

Das hat mehrere Gründe: Erstens bestimmen die technischen Faktoren ein System viel offensichtlicher. Zweitens sind Softwarearchitekten in technischen Themen oftmals erfahrener als in organisatorischen oder gar politischen. Drittens herrscht in Softwareprojekten manchmal ein technischer Zweckoptimismus, getreu dem Motto: Mit der neuen X-Technik und dem Y-Werkzeug haben wir unsere Probleme im Griff.

Wie [Rechtin2000] es treffend formuliert: *Es gibt keine rein technischen Probleme*. Sie werden schnell zu organisatorischen oder politischen Schwierigkeiten und damit entgleiten sie auch der Kontrolle rein technisch orientierter Projektbeteiligter.



Die Einflussfaktoren und Randbindungen auf eine konkrete Softwarearchitektur sollten Sie in einer Tabelle (als Bestandteil der Architekturdokumentation) festhalten.

Organisatorische Faktoren

Diese Faktoren beziehen sich im weitesten Sinne auf die Umgebung, in der das System erstellt wird. Sie prägen das System indirekt. Hierzu zählen Aspekte wie Termin- und Budgetplanung, vorhandene Mitarbeiter, technische Ressourcen, Entwicklungsprozesse, Vorgaben bezüglich Werkzeuge und Ähnliches. Tabelle 3.3 gibt einen Überblick über typische organisatorische Faktoren.

Einige Tipps zur Identifikation von organisatorischen Faktoren und Risiken Ihrer Projekte:

- Denken Sie negativ. Murphys Regel besagt: Wenn etwas schiefgehen kann, wird es irgendwann schiefgehen. Und es geschieht immer im denkbar schlechtesten Moment.
- Politik und Organisation, nicht Technik, setzen die Grenzen dafür, was Entwicklungsteams erreichen können und dürfen. Bringen Sie die „politischen“ Stakeholder auf Ihre Seite. Stellen Sie sicher, dass Sie die Intention der „Politiker“ innerhalb Ihrer Organisation richtig verstanden haben.
- Eine fundamentale Gleichung lautet: „Geld = Politik“ ([Rechtin2000]). Die Politik gibt Entwicklung die Kostenregeln vor. Diese Kostenregeln besitzen prägenden Einfluss auf Architekturen.
- Finden Sie heraus, welche Ziele die einzelnen Stakeholder mit dem System verfolgen. Oftmals unterscheiden oder widersprechen sich Ziele verschiedener Stakeholder.
- Die besten technischen Lösungen sind nicht unbedingt die besten politischen Lösungen. Im Regelfall sind „politische“ Stakeholder die Auftraggeber und Eigentümer von Projekten (nach [Rechtin2000]). Sie entscheiden meist nach anderen Kriterien als „technische“ Stakeholder.
- Die Erfahrung der beteiligten Entscheider spielt meistens eine wichtige Rolle. So schränkt eine negative Erfahrung des Auftraggebers mit einer bestimmten Technologie Ihre Entwurfsalternativen möglicherweise ein.¹⁰
- Werfen Sie einen Blick auf das Risikomanagement Ihres Managements. Eventuell finden Sie dort neue und für die Architektur wichtige Faktoren.

¹⁰ Edward Yourdon rät in seinem Buch „Death March: Surviving Mission Impossible Projects“ dazu, bei einer nicht akzeptablen Fülle solcher Einschränkungen das Projekt zu verlassen oder zu kündigen, um das eigene (berufliche) Überleben zu sichern.

Tabelle 3.3 Typische organisatorische Einflussfaktoren

Faktor	Erläuterung
Organisation und Struktur	
Organisationsstruktur beim Auftraggeber	▪ Droht Änderung von Verantwortlichkeiten? ▪ Änderung von Ansprechpartnern
Organisationsstruktur des Entwicklungsteams	▪ Mit/ohne Unterauftragnehmer ▪ Entscheidungsbefugnis des Managements
Entscheidungsträger	▪ Erfahrung mit ähnlichen Projekten ▪ Risiko-/Innovationsfreude
Bestehende Partnerschaften oder Kooperationen	▪ Hat die Organisation bestehende Kooperationen mit bestimmten Softwareherstellern? ▪ Solche Partnerschaften geben oftmals (unabhängig von Systemanforderungen) Produktentscheidungen vor.
Eigenentwicklung oder externe Vergabe	▪ Selbst entwickeln oder an externe Dienstleister vergeben?
Entwicklung als Produkt oder zur eigenen Nutzung?	▪ Bedingt andere Prozesse bei Anforderungsanalyse und Entscheidungen. Im Fall der Produktentwicklung: ▪ Neues Produkt für neuen Markt? ▪ Verbessertes Produkt für bestehenden Markt? ▪ Vermarktung eines bestehenden (eigenen) Systems ▪ Entwicklung ausschließlich zur eigenen Nutzung?
Ressourcen (Budget, Zeit, Personal)	
Festpreis oder Zeit/Aufwand?	Festpreis oder Abrechnung nach Zeit und Aufwand?
Zeitplan	Wie flexibel ist der Zeitplan? Gibt es einen festen Endtermin? Welche Stakeholder bestimmen den Endtermin?
Zeitplan und Funktionsumfang ¹¹	Was ist höher priorisiert: der Termin oder der Funktionsumfang?
Release-Plan	Zu welchen Zeitpunkten soll welcher Funktionsumfang in Releases/Versionen zur Verfügung stehen?
Projektbudget	Fest oder variabel? In welcher Höhe verfügbar?
Budget für technische Ressourcen	Kauf oder Miete von Entwicklungswerkzeugen (Hardware und Software)?
Team	Anzahl der Mitarbeiter und deren Qualifikation, Motivation und kontinuierliche Verfügbarkeit

¹¹ Tom DeMarco beschreibt die Probleme mit Zeitplan und Funktionsumfang in seinem amüsanten und lehrreichen Roman über Projektmanagement („Der Termin“, 1998 bei Hanser erschienen).

Tabelle 3.3 (Fortsetzung) Typische organisatorische Einflussfaktoren

Faktor	Erläuterung
Organisatorische Standards	
Vorgehensmodell	Vorgaben bezüglich Vorgehensmodell? Hierzu gehören auch interne Standards zu Modellierung, Dokumentation und Implementierung.
Qualitätsstandards	Fällt die Organisation oder das System in den Geltungsbereich von Qualitätsnormen (wie ISO 9000)?
Entwicklungswerkzeuge	Vorgaben bezüglich der Entwicklungswerkzeuge (etwa: CASE-Tool, Datenbank, Integrierte Entwicklungsumgebung, Kommunikationssoftware, Middleware, Transaktionsmonitor)
Konfigurations- und Versionsverwaltung	Vorgaben bezüglich Prozessen und Werkzeugen
Testwerkzeuge und -prozesse	Vorgaben bezüglich Prozessen und Werkzeugen
Abnahme- und Freigabeprozesse	Datenmodellierung und Datenbankdesign Benutzeroberflächen Geschäftsprozesse (Workflow) Nutzung externer Systeme (etwa: schreibender Zugriff bei externen Datenbanken)
Service Level Agreements	Gibt es Vorgaben oder Standards hinsichtlich Verfügbarkeiten oder einzuhaltender Service-Levels?
Juristische Faktoren	
Haftungsfragen	Hat die Nutzung oder der Betrieb des Systems mögliche rechtliche Konsequenzen? Kann das System Auswirkungen auf Menschenleben oder Gesundheit haben? Kann das System Auswirkungen auf Funktionsfähigkeit externer Systeme oder Unternehmen haben?
Datenschutz	Speichert oder bearbeitet das System „schutzwürdige“ Daten?
Nachweispflichten	Bestehen für bestimmte Systemaspekte juristische Nachweispflichten?
Internationale Rechtsfragen	Wird das System international eingesetzt? Gelten in anderen Ländern eventuell andere juristische Rahmenbedingungen für den Einsatz (Beispiel: Nutzung von Verschlüsselungsverfahren)?

Zu den organisatorischen Einflussfaktoren ein kleines Beispiel:



Beispiel: Die Informatik-Tochterfirma eines Konzerns erhielt den internen Auftrag, ein internetbasiertes Informationssystem für Finanznachrichten und Geschäftsberichte zu entwickeln und zu vermarkten.

Das Entwicklungsteam schien anfangs frei von organisatorischen und technischen Einflüssen zu sein. Im Laufe der Entwicklung stellte sich heraus, dass die Firma mit Marktpartnern und Kunden projektübergreifende „Service Level Agreements“ (SLAs) bezüglich der Verfügbarkeit von Software vertraglich vereinbart hatte. Als Konsequenz für die Architektur ergab sich die Notwendigkeit, eine (anfänglich nicht geplante) Überwachungskomponente zum verbindlichen Nachweis der Systemverfügbarkeit zu entwickeln. Dies machte eine kosten- und zeitintensive Änderung des gesamten Persistenzkonzepts notwendig.

Der zuständigen Projektleiterin gelang es, mit den betroffenen Endkunden des Systems eine Änderung der Nachweispflicht und der SLAs zu vereinbaren und damit zumindest den Termindruck der notwendigen Änderungen zu entschärfen.

Technische Faktoren

Technische Faktoren mit Relevanz für die Softwarearchitektur betreffen einerseits die technische Infrastruktur, also die Ablaufumgebung des Systems. Andererseits umfassen sie auch technische Vorgaben für die Entwicklung, einzusetzende Fremdsoftware und vorhandene Systeme.

Einige Tipps bei der Suche nach technischen Faktoren:

- Analysieren Sie andere Systeme oder Entwicklungsprojekte innerhalb Ihrer Organisation. Befragen Sie deren Architekten und Management.
- Betrachten Sie andere Systeme innerhalb der Organisation. Wie sieht die technische Umgebung dieser Systeme aus? Wie werden diese Systeme betrieben?
- Betrachten Sie die vorhandene Infrastruktur hinsichtlich Hardware und Software.
- Das Qualitätsmanagement der Organisation kann Hinweise auf weitere Einflussfaktoren geben.
- Gibt es Methoden, Standards oder Vorlagen für Softwareprojekte?
- Analog zum Tipp bei den organisatorischen Faktoren: Betrachten Sie eventuelle systemübergreifende Abnahme- und Freigabeprozesse.

Tabelle 3.4 zeigt einige typische technische Einflussfaktoren.

Tabelle 3.4 Typische technische Einflussfaktoren

Faktor	Erläuterung
Hardwareinfrastruktur	Prozessoren, Speicher, Netzwerke, Firewalls und andere relevante Elemente der Hardwareinfrastruktur
Softwareinfrastruktur	Betriebssysteme, Datenbanksysteme, Middleware, Kommunikationssysteme, Transaktionsmonitor, Webserver, Verzeichnisdienste
Systembetrieb	Batch- oder Online-Betrieb des Systems oder notwendiger externer Systeme? Cloud oder nicht?
Verfügbarkeit der Laufzeitumgebung	Rechenzentrum mit 7 x 24 h Betriebszeit? Gibt es Wartungs- oder Backupzeiten mit eingeschränkter Verfügbarkeit des Systems oder wichtiger Systemteile?
Grafische Oberfläche	Existieren Vorgaben hinsichtlich grafischer Oberfläche (Style Guide)?
Bibliotheken, Frameworks und Komponenten	Sollen bestimmte „Softwarefertigteile“ eingesetzt werden?
Programmiersprachen	Objektorientierte, strukturierte, deklarative oder Regelsprachen, kompilierte oder interpretierte Sprachen?
Referenzarchitekturen	Gibt es in der Organisation vergleichbare oder übertragbare Systeme als Referenz?
Datenstrukturen	Vorgaben für bestimmte Datenstrukturen, Schnittstellen zu bestehenden Datenbanken oder Dateien
Programmierschnittstellen	Schnittstellen zu bestehenden Programmen
Programmiervorgaben	Programmierkonventionen, fester Programmaufbau
Technische Kommunikation	Synchron oder asynchron, Protokolle

Das alles sollte die Systemanalyse geliefert haben

In einer idealen Situation hat die Fachseite, das Requirements Engineering oder Ihr Product-Owner, die bis hier aufgeführten Informationen (iterativ-inkrementell) ermittelt und passend aufbereitet.

Nach meiner Erfahrung betreiben jedoch nur wenige Organisationen eine methodische Systemanalyse. In den übrigen Fällen müssen Entwicklungsteams (vermutlich Sie als Architekten) Teile dieser Analyse nachholen, insbesondere die notwendigen Qualitätsanforderungen ermitteln und beschreiben.

■ 3.4 Entwerfen und kommunizieren

Bis hierhin haben Sie die Voraussetzungen geschaffen, um angemessen entwerfen (und implementieren) zu können. Fast der gesamte restliche Teil des Buchs ist der Erklärung und Vertiefung dieser Aufgabe gewidmet (siehe Bild 3.7).

- **Strukturen entwerfen:** Hier geht es primär um den Aufbau des Systems aus Quellcode, die Struktur der Implementierung. Kapitel 4 hilft Ihnen weiter.
- **Konzepte entwerfen:** Hier geht es um die übergreifenden, meist technischen Belange. Kapitel 7 kümmert sich ausführlich darum.
- Sie sollten als verantwortungsbewusster Softwarearchitekt die Umsetzung (d. h. Implementierung und Test) des Systems im Detail begleiten. Dazu finden Sie einige Tipps am Ende dieses Abschnitts.
- Meistens arbeiten wir in Teams – dazu müssen Sie über Architektur, Entwürfe, Entscheidungen, Strukturen und Konzepte miteinander kommunizieren. Kapitel 5 gibt Ihnen hierzu einige Hinweise.
- Explizit zu bewerten, ob ein System und dessen Architektur die geltenden Anforderungen erfüllen können, gehört ebenfalls zu Ihren Aufgaben. Kapitel 8 stellt einen systematischen Ansatz dazu vor.

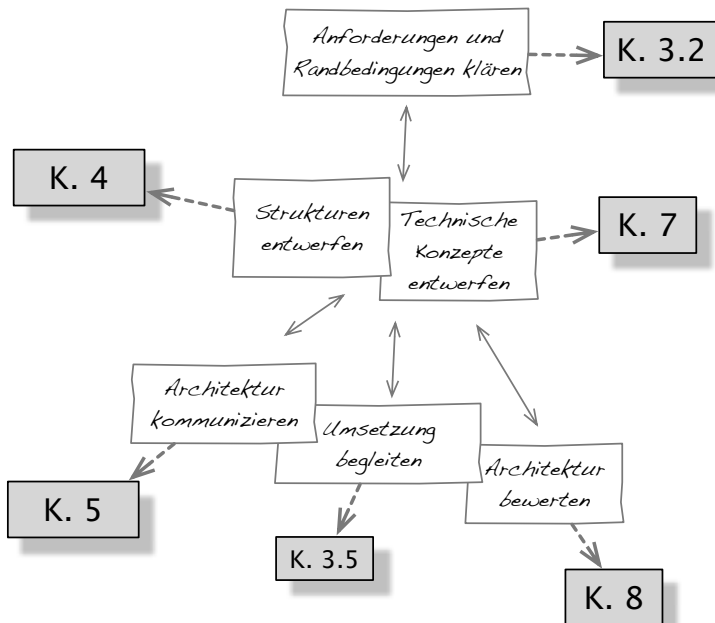


Bild 3.7 Hier finden Sie die weiteren Architekturaufgaben erklärt.

■ 3.5 Umsetzung begleiten

Goldstücke suchen

Oftmals haben in meinen Projekten einzelne Entwickler hervorragende Verbesserungen gefunden und umgesetzt. Ihre Lösungen waren besser, einfacher, schneller oder günstiger als diejenigen des übrigen Teams oder der Architekturgruppe. Solche Lösungsansätze sollten Sie in jedem Fall in die Gesamtarchitektur übernehmen.¹² In anderen Industriezweigen nennt man dieses dauernde Suchen nach Verbesserungsmöglichkeiten „Kai-Zen“ – ich nenne es gerne „Goldstücke suchen“.



Bei der Entwicklung eines (Java-)Systems mit hohen Anforderungen an Flexibilität (genauer: Flexibilität von Datenim- und -exporten) hatten wir diese Herausforderung durch (XML-basierte) Konfiguration gelöst. Das hat gut funktioniert – allerdings war die Erstellung von Testfällen aufgrund der abstrusen Zahl spitzer Klammern eine wahre Strafarbeit für Entwickler und Tester ...

Knapp ein Dutzend Entwickler fand über lange Zeit täglich neue Gründe, keine Testfälle für diese Konfiguration zu schreiben. Die Menge unserer Testfälle blieb sträflich gering ...

Eines Tages hatte ein einzelner Entwickler plötzlich eine deutlich höhere Testrate. Auf Nachfragen stellte sich heraus, dass er unsere (bisher manuell erstellte) XML-Konfiguration sehr geschickt durch ein Sprachkonstrukt von Groovy¹³ ergänzt – und damit den für eine komplexe Konfiguration nötigen Aufwand von bisher über 100 Zeilen XML-Konfiguration auf zehn bis 20 Zeilen Groovy-Quellcode gesenkt hatte. Und das auch noch mit großartiger IDE-Unterstützung.

Ein echtes „Goldstück“ für das gesamte Team, das diese Idee begeistert aufgenommen hat – und damit ein wesentliches Qualitätsziel des Systems deutlich verbessern konnte.

Missverständnisse aufdecken

Auf der anderen Seite könnten Entwickler auch Entwurfsentscheidungen oder Konzepte missverstehen oder schlecht umsetzen. In solchen Fällen sollten Sie als Architekt coachen und den Betroffenen die Entscheidungen oder Konzepte erläutern oder sie bei deren Implementierung unterstützen.

Quellcode lesen

In beiden Fällen müssen Sie in den Quellcode Ihrer Systeme schauen (= Code-Reviews) und die Ist-Implementierung mit dem gewünschten Soll vergleichen. Erst durch diese (Detail-)Arbeit können Sie wirklich erkennen, ob und wie Architekturentscheidungen letztlich umgesetzt werden.

¹² Eine Prüfung vorausgesetzt sowie die Eignung für die jeweilige Problemstellung.

¹³ Groovy ist eine dynamische Sprache auf der JVM und hier ging es um die Groovy-MarkupBuilder (<http://groovy-lang.org>).

Peer-Reviews und Delegation

Gerade in größeren Teams kann eine einzelne Person nicht mehr den gesamten erstellten oder geänderten Quellcode überprüfen. Codeanalyse-Werkzeuge können helfen, aber in wesentlichen Fällen (kritische oder wichtige Stellen im System) sollten Menschen solche Reviews durchführen (nur sie können nach Gründen fragen und diese situativ einschätzen).

Oft genügt es auch, kleine Teile des Quellcodes zu inspizieren, um den Aufwand dieser Aufgabe in angemessenem Rahmen zu halten.

■ 3.6 Lösungsstrategien entwickeln

Nach der Klärung von Anforderungen, Einflussfaktoren und Risiken, insbesondere der geforderten Qualitätsmerkmale und Risiken, kennen Sie also die Schwierigkeiten, die beim Entwurf des Systems auf Sie zukommen. Nun gilt es, passende Lösungsstrategien und -alternativen zu entwickeln.

Diese Strategien entwickeln Sie parallel zum Entwurf von Sichten und Konzepten (zu denen Sie in den folgenden Kapiteln noch viel mehr lesen).

Ich möchte mich in diesem Abschnitt auf Ratschläge und Anregungen zu organisatorischen Problemen beschränken, primär den Mangel an Zeit, Budget oder Erfahrung.

In Abschnitt 4.3.2 zeige ich Ihnen den methodischen Ansatz zur Erreichung von Qualitätsanforderungen (Quality-Driven Software Architecture) und gebe dort Beispiele zu den Themen Performance, Flexibilität und Erweiterbarkeit.

Sollten einige Ihrer spezifischen Probleme in eher technischen Bereichen angesiedelt sein, kann Ihnen sicherlich der Katalog typischer Konzepte (Kapitel 7) weiterhelfen.

Strategien gegen organisatorische Risiken

Meiner Erfahrung nach leiden die meisten Entwicklungsprojekte unter mindestens einem der folgenden (organisatorischen) Risiken:

- *Zu wenig Zeit:* Die Zeit bis zum Endtermin ist zu knapp bemessen, um die Anforderungen mit dem zugehörigen Entwicklungsteam zu erfüllen.
- *Zu wenig Budget:* Es mangelt dem Projekt an Geld. Aus Projektsicht notwendige Investitionen in Hardware, Software oder Wissen (in Form von Schulungen oder weiteren Mitarbeitern) unterbleiben.
- *Zu wenig Wissen und Erfahrung:* Es mangelt dem Team an Wissen und Erfahrung in einigen Bereichen der Entwicklung. Beispielsweise wird ein neues, dem Team unbekanntes Entwicklungswerkzeug eingesetzt.

Für die in der Praxis häufig vorkommenden „normal kritischen“ Situationen gebe ich Ihnen bezüglich der Softwarearchitektur folgende Ratschläge:



- Versuchen Sie, Risiken durch iteratives Vorgehen möglichst frühzeitig zu erkennen und zu adressieren. Iteration und Feedback verschafft Ihnen (hoffentlich) mehr Zeit, die garantiert auftretenden Probleme zu erkennen und zu lösen.
- Arbeiten Sie bei organisatorischen Problemen eng mit dem Management zusammen.
- Softwarearchitekten können dem Management wertvolle Hinweise zu technischen Risiken liefern.
- Sie können gemeinsam mit dem Management die Auswirkung kritischer Anforderungen mit dem Auftraggeber diskutieren (und hier ebenfalls eine Entspannung der Situation erreichen).
- Falls sich die organisatorischen Probleme direkt auf technische Aspekte des Systems oder einzelne Bestandteile auswirken: Nutzen Sie diese Sachverhalte für Verhandlungen mit Management, Auftraggebern und Kunden.
- Prüfen Sie mit dem Auftraggeber, ob eine Revision der *Make-or-buy*-Entscheidung die Situation entschärfen kann. Gegebenenfalls kann auch der Zukauf einzelner Komponenten oder Systembestandteile helfen.
- Überarbeiten Sie mit dem Auftraggeber die Anforderungen an das System. Verhandeln Sie über andere Prioritäten der Anforderungen, um die Einschränkungen durch einige Einflussfaktoren zu lockern. Besonders kritisch und aufwendig sind (oftmals überzogene) Anforderungen an Verfügbarkeit und Performance.

Bedenken Sie bei organisatorischen Problemen einige wichtige Erfahrungen:



- Niemand arbeitet unter hohem Druck besonders produktiv. Und keiner denkt unter Druck schneller. Im Gegenteil: Hoher Druck erhöht die Fehlerquote und die Bereitschaft zu „schmutzigen Tricks“.
- Einem verspäteten Projekt mehr Mitarbeiter zu geben, verzögert das Projekt zusätzlich ([Brooks95]).
- Weitere Probleme tauchen von allein auf. Bauen Sie daher in alle Schätzungen Sicherheitszuschläge ein. Unterschätzen Sie sich lieber.
- Wenn die Politik nicht mitspielt, wird das System niemals laufen.

Weiter mit Entwurf, Sichten und Dokumentation!

Im folgenden Kapitel 4 lernen Sie Grundlagen und Methoden des Entwurfs kennen – Sie finden die wesentlichen Entwurfsprinzipien und viele nützliche Heuristiken („Faustregeln“) für Ihre Arbeit.

Im Kapitel 5 stelle ich Ihnen die wichtigsten *Sichten* auf Softwarearchitekturen vor. Die sollten Sie kennen, wenn Sie Ihre eigenen Entwürfe kommunizieren und dokumentieren wollen.

■ 3.7 Weiterführende Literatur

Die hier vorgestellten Ansätze, Einflussfaktoren zu gruppieren, zu priorisieren und mit spezifischen Strategien zu adressieren, werden in einigen anderen Disziplinen seit langem erfolgreich angewandt, etwa im Risikomanagement von Projekten oder im *System Engineering* (siehe [Rechtin2000]). [Bass+12] diskutieren Qualitätsmerkmale und Möglichkeiten zur Umsetzung als sogenannte *Architectural Tactics*.



[Hofmeister2000] beschreibt eine „globale Analyse“, die auch auf die Darstellung der Einflussfaktoren sowie die Entwicklung von Strategien abzielt.

Das pragmatische und überaus lesenswerte Buch von Stefan Toth über Vorgehensmuster [Toth-19] gibt konkrete Anregungen, wie Sie Ihre „Pferdestärken auf die Straße“ bringen können – insbesondere im agilen Umfeld. Unbedingte Leseempfehlung!

[Martin08] zeigt den durchweg positiven Effekt von *Clean Code* auf: Verständlicher Quellcode hilft auch beim Verständnis von Architekturen.

[Rechtin2000] ist eine Fundgrube für Heuristiken und Vorgehensweisen beim Entwurf von Architekturen.

4

Entwurf: Grundlagen, Methoden und Muster

*Der Schlüssel zur Maximierung der Wiederverwendung
liegt im Voraussehen von neuen Anforderungen ...
sowie im Entwurf Ihrer Systeme derart,
dass sie sich entsprechend entwickeln können.*

[Gamma95] (Übersetzung Dirk Riehle)



Fragen, die dieses Kapitel beantwortet:

- Wie können Sie Entwurfsentscheidungen treffen?
- Welche wichtigen Grundlagen und Prinzipien gelten für den Entwurf von Software?
- Welche Heuristiken helfen beim Entwurf?
- Wie können Sie Bausteine Ihrer Systeme entwerfen?
- Was bedeutet „Domain-Driven Design“?
- Welche Architekturmuster helfen beim „Entwurf im Großen“?
- Welche Entwurfsmuster reduzieren Abhängigkeiten?

Struktur von Software degeneriert mit der Zeit

Das Entwicklungsteam und die Architekten haben eine saubere und flexible Struktur für ein Softwaresystem entworfen und implementiert. In der ersten Version der Software ist diese Struktur klar erkennbar, übergreifende Konzepte sind konsistent und durchgängig umgesetzt. Dann kommt die Software zum praktischen Einsatz und es geschieht das Unvermeidliche: Mit der Zeit ändern sich Anforderungen und technische Voraussetzungen. Die Software wird angepasst, verändert, erweitert, gepflegt. Manchmal übersteht die ursprüngliche Struktur diese Änderungen. In den meisten Fällen entsteht über die Zeit jedoch ein schwer verständlicher, unwartbarer Moloch ohne erkennbare Struktur. Änderungen sind schwierig und führen zu Problemen an völlig anderen Stellen. Das System wird labil, Erweiterungen werden immer riskanter. Robert C. Martin bezeichnet in [Martin2000] diesen Zustand als „verfaultes Design“.

Viele Abhängigkeiten lassen
Design verfaulen

Symptome von verfaultem Design

Es sind im Wesentlichen drei Eigenschaften von Software, die auf ein verfaultes Design hinweisen:

- *Starrheit*: Selbst einfache Änderungen an der Software sind schwierig und bedingen Modifikationen in einer Vielzahl abhängiger Komponenten.
- *Zerbrechlichkeit*: Änderungen an einer Stelle des Programms führen zu Fehlern an ganz anderen Stellen.
- *Schlechte Wiederverwendbarkeit*: Selbst innerhalb eines Systems können Komponenten kaum wiederverwendet werden, weil sie zu viele Abhängigkeiten von anderen Programmteilen enthalten.

Abhängigkeiten verwalten

Unsere Herausforderung liegt darin, die gegenseitigen Abhängigkeiten aller Bestandteile von Software (Subsysteme, Komponenten, Pakete, Klassen, Funktionen) in einer vernünftigen Weise zu verwalten. Manche Abhängigkeiten zwischen Bausteinen lassen sich nicht vermeiden oder sind sogar erwünscht (etwa: einer anderen Klasse muss eine bestimmte Nachricht geschickt werden, eine bestimmte Funktion eines anderen Subsystems muss aufgerufen werden).



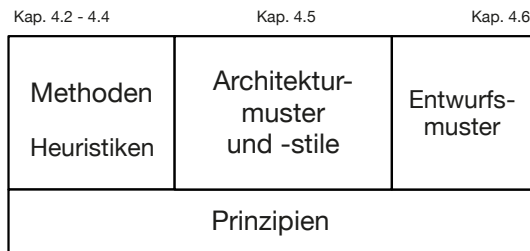
Wir müssen in Entwürfen die Abhängigkeiten zwischen Komponenten derart konstruieren, dass künftige Änderungen möglichst wenige neue Abhängigkeiten erzeugen.

In diesem Kapitel stelle ich Ihnen einige Ansätze zum Entwurf von Strukturen vor, zur Zerlegung eines Systems in Bausteine. Das ist die *klassische* Entwurfsaufgabe, zu der es keine algorithmische Lösung gibt. Wichtige Werkzeuge für angemessene (d. h. effektive) Entwürfe sind (und bleiben!) Erfahrung, Iterationen und Feedback, gepaart mit persönlichem Geschmack.

In Abschnitt 4.1 und Abschnitt 4.2 finden Sie einige fundamentale Aussagen. Die sollten Sie in jedem Fall „quer“ lesen – und bei Entwürfen grundsätzlich berücksichtigen.

In Abschnitt 4.3 erkläre ich einige Methoden, die Sie zu Strukturentwürfen (d. h. Festlegung von Bausteinen) Ihrer Systeme führen können, beispielsweise das Top-down-Vorgehen oder Domain-Driven Design.

Abschnitt 4.4 kümmert sich um Schnittstellen – eines der zentralen Themen beim Entwurf: Wie arbeiten Bausteine konkret zusammen und welche Anforderungen gelten für Schnittstellen?



Kap. 4.1

Bild 4.1
Aufbau des Kapitels

In Abschnitt 4.5 finden Sie eine umfangreiche Sammlung wichtiger Architekturmuster und -stile, die Sie als Beispiele oder Vorlage für eigene Systeme verwenden können. Architekturmuster beziehen sich auf Struktur-im-Großen.

Abschnitt 4.6 schließlich fasst für Sie einige der bekannten Entwurfsmuster zusammen und gibt damit Hinweise für Strukturen-im-Kleinen.

*Wenn unsere Entwürfe aufgrund der permanent wechselnden Anforderungen versagen,
so ist das der Fehler unserer Entwürfe!
Wir müssen einen Weg finden, unsere Entwürfe unverwüstlich gegen solche Änderungen
zu machen und sie vor Verfaulen zu schützen.*

[Martin2000, p. 4]

■ 4.1 Grundlagen

Ich möchte Ihnen die Grundlagen, sozusagen das kleine Einmaleins des Entwurfs, in drei Stufen vorstellen, die ich Grundsätze, Prinzipien und Heuristiken nenne¹.

Über unsere Grundsätze (die *Maxime*), sollten wir nicht diskutieren, sondern sie schlichtweg als allgemeingültige Regeln akzeptieren. Sie klingen abstrakt und werden durch Prinzipien, Heuristiken oder Methoden konkretisiert.

Als *Prinzipien* bezeichne ich konkrete und etablierte Regeln des methodischen Software-engineering.

Heuristiken schließlich sind Ratschläge oder Hinweise, wie Sie in komplexen Entwurfssituationen vorgehen könnten (Konjunktiv!). Heuristiken haben den Charakter von „Daumenregeln“. In Bild 4.1 stehen sie zusammen mit den Methoden. Eine exakte gegenseitige Abgrenzung von Heuristiken und Methoden fällt mir persönlich schwer (und ist für die Praxis auch irrelevant). Nach meiner Auslegung sind die Heuristiken eher „klein“, während die Methoden eher umfassend oder „groß“ sind. Heuristiken finden Sie in Abschnitt 4.2.

¹ Herzlichen Dank an Ralf Engelschall für diese Namensvorschläge und die konstruktive Diskussion zum Thema. Ralf verwendet diese Begriffe schon lange in seinen Trainings.

4.1.1 Grundsätze des Entwurfs (Maxime)

Maxime: größte, oberste Aussage, Grundsatz des Wollens und Handelns

(nach: Wikipedia)

Stakeholder beurteilen Erfolg von Lösungen

Ob das von Ihnen gebaute System erfolgreich ist, beurteilen Kunden, Auftraggeber, Benutzer und andere maßgebliche Stakeholder. Ihre eigene Meinung zählt nur in Ausnahmefällen.

„Business Drives Technology“ nennen das die IT-Strategen und diesem Ratschlag möchte ich mich gerne anschließen: Wir bauen Systeme, um einen kommerziellen Zweck unserer Auftraggeber oder Kunden zu verfolgen. Systeme aus rein altruistischen Motiven, wie etwa Open-Source, sind eher die Ausnahme.

Ihre Stakeholder haben aber sowohl kurz- wie auch langfristige Interessen, wobei die kurzfristigen oft höher priorisiert werden. Als verantwortungsbewusster Architekt wissen Sie um die Notwendigkeit wart- und änderbarer Systeme, um die Wichtigkeit auch der langfristigen Ziele wie Wartbarkeit und Verständlichkeit. In Ihren Entwürfen und deren Implementierung müssen Sie daher kurz- und langfristige Anforderungen ausbalancieren.

Zur Vorbereitung auf diesen Teil unserer Grundsätze hatte ich Ihnen in Abschnitt 3.2.4 bereits vorgeschlagen, relevante Stakeholder zu ermitteln, inklusive deren Erwartungshaltung.

Einfachheit gewinnt (Keep It Small and Simple, KISS)

Perfektion ist nicht dann erreicht, wenn es nichts mehr hinzuzufügen gibt, sondern wenn man nichts mehr weglassen kann.

Antoine de Saint-Exupéry

Einfachheit oder Schlichtheit sind wünschenswerte Eigenschaften eines jeglichen Softwareentwurfs. (Es sei denn, Sie möchten verhindern, dass andere Menschen diesen Entwurf verstehen.)



Entwerfen Sie nach dem Prinzip „So einfach wie möglich“. Fragen Sie bei jeder, aber wirklich jeder Entwurfsentscheidung immer wieder: Wie geht das einfacher? Dieser Ratschlag gilt für alle Ebenen des Entwurfs, für Geschäftsmodelle, Architekturen und Klassendiagramme.

Möglichst einfach!

Schlichtheit und Einfachheit erleichtern allen Personen, die Ihre Entwürfe später lesen, das Verständnis. Sie vermeiden, dass potenzielle Probleme durch überflüssige Komplexität verdeckt werden. Einfachheit hilft, Probleme bereits im Vorfeld zu erkennen.

Einfache Lösungen sind leichter wartbarer. Sie können einfache Lösungen leichter implementieren und der Quellcode wird kürzer und verständlicher sein.

Vermeiden Sie unnötige (= accidental) Komplexität in Ihren Lösungen und Entwürfen. Stellen Sie besonders komplexe, trickreiche oder überraschende Lösungen in Frage.

Natürlich können Sie komplexe oder schwierige fachliche Sachverhalte (Daten, Prozesse) nicht beliebig vereinfachen.



Welche Komponenten Ihres Systems werden garantiert die wenigsten Fehler enthalten? Diejenigen, die Sie durch sinnvolle Vereinfachung weglassen konnten.

Entscheiden Sie spezifisch

Diesen Grundsatz finden Sie auch unter dem Titel „No Silver Bullet“²: Es gibt kein Patentrezept oder gar Algorithmus zur Lösung beliebiger Entwurfsprobleme in der Softwareentwicklung. Sie benötigen system- oder problemspezifische Entscheidungen, Abwägungen und Festlegungen, um eine für Ihre Anforderungen und Ziele angemessene Lösung zu erarbeiten.

Explizit statt implizit

Sie sollten Voraussetzungen, Annahmen und auch Entscheidungen möglichst *explizit* machen – um die möglichen Probleme *impliziter* Annahmen zu vermeiden.

Für Anforderungen wie auch Lösungsvorschläge gibt es meist mehrere mögliche Interpretationen – und im Zweifel interpretiert jeder Beteiligte nach persönlichen Vorlieben oder aktuellem Kenntnisstand. Schwierig wird das, wenn innerhalb eines Entwicklungsteams mehrere verschiedene Interpretationen (Auslegungen) ein- und desselben Sachverhalts existieren. Dies können Sie durch *Explizieren* vermeiden – und damit für Klarheit in Bezug auf relevante Themen sorgen.

Das haben Sie bereits in Abschnitt 2.2 gelesen – als eine der wichtigen Aufgaben von Softwarearchitekten.

Zu Beispielen solcher expliziten Aussagen gehören etwa:

- fachliche oder Domänenmodelle, um fachliche Zusammenhänge explizit darzustellen,
- klar benannte Verantwortlichkeiten von Bausteinen, um deren Aufgabe im System explizit darzustellen,
- ausdrücklich formulierte Qualitätsanforderungen (siehe auch Abschnitt 3.2.3).

Erwarte Änderungen

Mit der Zeit ändert sich vieles: Menschen, Organisationen, Anforderungen, Probleme, Lösungsansätze, Technologien – sprich: sämtliche Fundamente und Entscheidungen unserer Entwürfe können von Änderungen betroffen sein.

Richten Sie sich bei Ihren Entwürfen auf Änderungen ein – die werden in jedem Fall kommen. Klären Sie, ob Änderungen im Problem- oder Lösungsraum in der Vergangenheit aufgetreten sind oder wo die wesentlichen Stakeholder solche erwarten.

Heutzutage berücksichtigen insbesondere die agilen Entwicklungsmethoden Scrum & Co. diesen Teil unseres „Grundgesetzes“ bereits von Hause aus: Durch Iterationen und systematisches Feedback helfen sie, mit Änderungen besser umgehen zu können.

Zwar schlägt die „agile Szene“ vor, keine zukünftigen Anforderungen vorwegzunehmen und heute bereits zu implementieren. Ich warne allerdings vor dem Optimismus, alles „einfach durch Refactoring“ anpassen zu können.

² Aus dem Klassiker von Fred Brooks: „No Silver Bullet – Essence and Accident in Software Engineering“. *Proceedings of the IFIP Tenth World Computing Conference* (1986): 1069–1076.



Gehen Sie insbesondere davon aus, dass die ursprünglichen Anforderungen sich ändern werden. Anforderungen werden durch Implementierung und Architektur (mit-)geprägt! Hinterfragen Sie Anforderungen, weisen Sie auf Konsequenzen und Alternativen hin. Helfen Sie Auftraggebern und Management dabei, Aufwand und Nutzen der Anforderungen abzuwägen.

Eine Warnung: Änderbarkeit können Sie leicht übertreiben. Beispielsweise ist „Konfigurierbarkeit“ eine mögliche Antwort auf erwartete Änderungen. Hochgradig konfigurierbare Systeme sind jedoch viel schwerer testbar und oftmals weniger robust als ihre eher starren Pendants.

Erwarte Fehler

Sie kennen Murphys Gesetz: *„Anything that can go wrong, will go wrong“*. Alles, was schiefgehen kann, wird irgendwann schiefgehen.

Stellen Sie sich auf Fehler, Defekte oder Ausfälle ein. Es gibt (praktisch) niemals nur Optimal-, Sonnenschein- oder Gutfälle. Ihre Systeme müssen (praktisch) immer auf irgendwelche Fehlersituationen reagieren.

Stellen Sie daher in Entwurfssituationen immer Fragen der folgenden Form:

- Was könnte ausfallen, überlaufen, abstürzen oder einfach aufhören zu existieren?
- Was könnte anders sein als vereinbart oder abgesprochen, gegen Regeln verstoßen, von Normen abweichen?
- Sorgen Sie dafür, dass Fehler leicht gefunden werden können. Dazu hilft Ihnen Einfachheit (siehe oben).
- Halten Sie Auswirkungen von Fehlern möglichst lokal. Vermeiden Sie Störungen des Gesamtsystems nach dem Eintritt von Fehlersituationen.

Abschnitt 7.10 gibt Ihnen dazu noch konkrete Ratschläge.

Halte Qualitätsanforderungen immer ein

Ein prägnanter Name für diesen Paragraphen unseres Grundgesetzes lautet „Continuous Compliance“³. Sie sollten *jederzeit* während der Entwicklung die Einhaltung der wichtigsten Qualitätsanforderungen und -ziele überprüfen und sicherstellen. In Abschnitt 3.2.3 habe ich Ihnen Szenarien empfohlen, um diese wichtigen Anforderungen konkret zu formulieren. Nun gehen wir einen Schritt weiter und stellen kontinuierlich sicher, dass wir die Qualitätsziele auch erreichen.

Nutzen Sie hierfür interne Reviews (etwa die qualitative Bewertung nach ATAM, siehe Kapitel 8) oder auch quantitative Methoden, Performance-Tests oder ähnliche Ansätze. Somit merken Sie frühzeitig, wenn Qualitätsanforderungen und Ihre Lösung voneinander abweichen.

Strategien oder Maßnahmen für einige Qualitätsmerkmale finden Sie in Abschnitt 4.3 zum Thema „Quality Driven Software Architecture“.

Soweit die Maxime des Entwurfs, das Grundgesetz. Als Nächstes möchte ich Ihnen die übergreifenden Regeln des Entwurfs vorstellen, die *Prinzipien*.

³ Diese Bezeichnung habe ich von Ralf Engelschall gelernt.

4.1.2 Prinzipien

Prinzip: Gesetzmäßigkeit, übergreifende Regel

Die Anwendung von Prinzipien müssen Sie abwägen: Ähnlich wie Zutaten beim Kochen können Sie mehr oder weniger davon verwenden. Sie werden für die Erfüllung von Anforderungen bezahlt, für die Lösung eines Problems, nicht aber für die perfekte Einhaltung von Prinzipien.

Diese Empfehlung formuliere ich direkt als das erste Prinzip:

Wäge Prinzipien gegen Anforderungen ab

Sie sollten *konkrete Anforderungen* höher gewichten als *Prinzipien*.

Andererseits basieren die Prinzipien auf jahrelanger Erfahrung – daher sollten Sie sich nicht leichtfertig über diese Regeln hinwegsetzen.

Ein Beispiel dazu: Das Prinzip „Lose Kopplung“ (siehe folgender Absatz) unterstützt Verständlichkeit, Wartbarkeit und Modularität – es könnte in manchen Fällen die Performance (= Laufzeiteffizienz) negativ beeinflussen. Insofern müssen Sie die Anwendung des Prinzips „Lose Kopplung“ gegen Laufzeitanforderungen abwägen.

Strebe nach loser (geringer) Kopplung

Kopplung ist ein Maß für die zwischen Bausteinen bestehenden Abhängigkeiten. Lose (geringe) Kopplung fördert Einfachheit und ist eine wichtige Voraussetzung für Flexibilität und Verständlichkeit von Entwürfen. Hohe (enge) Kopplung bedeutet gleichzeitig hohe Komplexität und führt zu Starrheit von Entwürfen.

Erst Beziehungen (= Kopplung) zwischen Bausteinen ermöglichen deren Zusammenarbeit, bilden also die Grundsubstanz kooperierender Strukturen. Andererseits bedeuten Beziehungen auch *Abhängigkeiten* zwischen Bausteinen. Diese Abhängigkeiten können jedoch zu einem gravierenden Problem führen:

Änderungen an einem Baustein führen zu Konsequenzen an anderen Stellen: Ändern Sie beispielsweise eine Schnittstelle, die ein anderer Baustein verwendet, so müssen Sie den nutzenden Baustein möglicherweise ebenfalls ändern.

Falls Sie die Forderung nach loser Kopplung, also geringen Abhängigkeiten, allzu wörtlich nehmen und zwischen Ihren Bausteinen überhaupt keine Abhängigkeiten bestehen, verhindern Sie damit deren Zusammenarbeit ... Das wäre nicht im Sinne des Erfinders – gefragt ist also ein *gesundes* Maß an Abhängigkeit oder Kopplung.

Folgende Arten der Kopplung werden Ihnen in der Praxis oft begegnen:

- **Kopplung durch Aufruf:** Ein Baustein benutzt direkt einen anderen Baustein, ruft beispielsweise eine Funktion oder Methode des benutzten Bausteins auf. Der Aufgerufene liefert dem Aufrufer in der Regel Ergebnisse zurück.
- **Kopplung durch Benachrichtigung:** Ein Baustein benachrichtigt andere Bausteine über bestimmte Sachverhalte. Wer diese Nachrichten empfängt, verarbeitet oder gar darauf reagiert, interessiert den Sender in der Regel nicht.
- **Kopplung durch Erzeugung:** Ein Baustein erzeugt einen anderen (oder eine Instanz einer Klasse von anderen Bausteinen). Diese Art der Abhängigkeit können Sie durch Fabrik-

Muster (*Factory-Pattern*, siehe [GoF] und [Eilebrecht+19]) verringern, in modernen Systemen auch durch *Dependency Injection*⁴.

- Kopplung durch Daten oder Datenstrukturen: Das kann beispielsweise ein Parameter eines Funktions- oder Methodenaufrufs sein oder aber eine Datenbankstruktur. In serviceorientierten Architekturen trifft diese Art der Kopplung auch auf sogenannte Dokumentstrukturen zu.
- Kopplung über Hardware oder Laufzeitumgebung trifft zu, wenn Bausteine beispielsweise im gleichen Adressraum oder innerhalb der gleichen virtuellen Maschine oder des gleichen Netzwerksegments ablaufen müssen.
- Kopplung über die Zeit: Falls für Aktivitäten verschiedener Bausteine eine notwendige Reihenfolge besteht, beispielsweise eine Aktion eines Bausteins immer zeitlich *vor* einer anderen Aktion eines anderen Bausteins erfolgen muss.

Wenn es Ihnen gelingt, die Kopplung von Bausteinen lose zu gestalten, können Sie diese Bausteine unabhängig voneinander variieren – Ihr Gesamtsystem bleibt wandlungsfähig und flexibel.

Setzen Sie Abhängigkeiten zwischen Bausteinen bewusst und vorsichtig ein. Erinnern Sie sich stets daran, dass ein Übermaß an Abhängigkeiten die Flexibilität Ihres Systems drastisch reduziert und zu *verfaultem Design* führen kann.



Kopplung von Systemen können Sie ausgezeichnet auf Quellcodeebene kontrollieren. Viele Werkzeuge⁵ können entsprechende Metriken ermitteln. Setzen Sie solche Metriken ein und beobachten Sie kontinuierlich die Kopplung Ihrer Systembausteine. Falls die Kopplung während der Entwicklung signifikant ansteigt, sollten Sie durch Zerlegung und Umstrukturierung gegenzusteuern.

Möglichst lose koppeln!

Als zusätzliche Hilfsmittel, mit denen Sie Abhängigkeiten in Ihren Entwürfen und Architekturen systematisch reduzieren können, haben sich einige Entwurfsmuster⁶ (*design patterns*) erwiesen. Einige davon lernen Sie in Abschnitt 4.6 kennen.

Eine Spezialisierung dieses Prinzips ist die Vermeidung zyklischer Abhängigkeiten – die finden Sie in den Bildern 4.3 und 4.4.

Strebe nach hoher Kohäsion

Möglichst hohe Kohäsion

Kohäsion, zu Deutsch „Zusammenhangskraft“, ist ein Maß für den inneren Zusammenhalt von Elementen (Bausteinen, Funktionen). Sie zeigt, wie eng die Verantwortlichkeiten eines Bausteins inhaltlich zusammengehören. Jeder Baustein Ihrer Entwürfe sollte Verantwortlichkeiten, Konzepte oder Abstraktionen enthalten, die inhaltlich eng zusammengehören (= hohe Kohäsion besitzen).

⁴ Perfektioniert beispielsweise in Frameworks wie Spring (www.springframework.org). Eine einführende Motivation und Erklärung hat Martin Fowler online beschrieben: <http://martinfowler.com/articles/injection.html>

⁵ Suchen Sie im Internet nach „Dependency Analyzer“.

⁶ Entwurfsmuster sind größtenteils für objektorientierte Systeme publiziert. Das Verständnis dieser Muster übt jedoch unserer Erfahrung nach einen positiven Einfluss auf beliebige Softwaresysteme aus (siehe insbesondere [Gamma95] und [Buschmann+96]).

Eine Schwierigkeit bei Kohäsion ist die unklare Definition, was inhaltlich denn zusammengehört. Bild 4.2 zeigt ein Beispiel: Vier verschiedene „Komponenten“ (Menschen, Möbelfabrik, Wald und ein See) sollen gruppiert werden. Dazu können zum Beispiel folgende Kriterien herangezogen werden:

1. „Arbeit“: Menschen arbeiten in der Fabrik.
2. „Wanderung“: Menschen wandern im Wald.
3. „Natur“: Wald und See sind beides natürliche Habitate.
4. „Kühlung“: Die Möbelfabrik benötigt Wasser zur Kühlung.
5. „Schwimmen“: Menschen schwimmen im See.
6. „Produktion“: Die Fabrik benötigt Holz zur Produktion.

Das Beispiel soll illustrieren, dass für Kohäsion unbedingt explizite Kriterien erforderlich sind.

Bild 4.2 zeigt Beispiele hoher und niedriger Kohäsion:

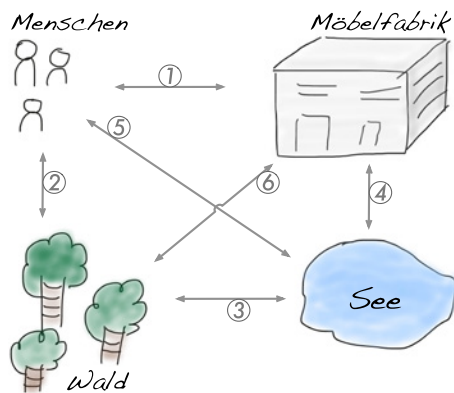


Bild 4.2
Was gehört zusammen?

Trenne Verantwortlichkeiten/Belange

(engl: *Separation of Concern*)

Geben Sie den Bausteinen Ihrer Systeme jeweils *spezifische* Verantwortlichkeiten, Zuständigkeiten oder Aufgaben. Nennen Sie diese Verantwortlichkeiten beim Namen! Beschreiben Sie sie in kurzen, präzisen Sätzen. Kommen darin mehrere „und“ vor, könnten Sie versehentlich mehrere Belange oder Verantwortlichkeiten vermischt haben.

Trenne Verantwortlichkeiten

Verantwortlichkeiten sollten Sie auf allen Ebenen des Entwurfs beachten, für einzelne Klassen in objektorientierten Entwürfen genauso wie für ganze Systeme und Subsysteme.

Verantwortlichkeiten gibt es für „Wissen“ (*knowing*) und „Handeln“ (*doing*):

- **Wissen:** Eine Komponente ist für eine bestimmte Menge an Information verantwortlich.
- **Handeln:** Eine Komponente ist für bestimmte Aktivitäten verantwortlich (etwa: Steuern, Kontrollieren, Berechnen, Erzeugen).

Wenn Sie Verantwortlichkeiten an Komponenten delegieren, müssen Sie die Auswirkungen dieser Verantwortung berücksichtigen. Insbesondere haben Verantwortlichkeiten große Auswirkungen auf die Kopplung und Kohäsion von Bausteinen untereinander.

Einige mögliche Ausprägungen dieses Prinzips möchte ich Ihnen vorstellen:

- Trennen Sie in Ihren Systemen fachliche von technischen Bestandteilen. Diese fundamentale Regel bekommen Sie weiter unten als eigenes Prinzip vorgestellt. Verschiedene Entwurfsmethoden konkretisieren dies (siehe Abschnitt 4.3).
- Unterteilen Sie komplexe Diagramme in mehrere einfache, die jeweils einen einzelnen Aspekt modellieren. Setzen Sie gegebenenfalls Hierarchien oder zusätzliche Abstraktionen ein, um einzelne Sachverhalte darzustellen.
- Die meisten Architekturmuster aus Abschnitt 4.5 trennen Belange systematisch voneinander – beispielsweise Pipes & Filter (trennt Transport von Bearbeitung).
- Trennen Sie schreibende von lesenden Bausteinen – eine Ausprägung davon ist das Architekturmuster CQRS (siehe Abschnitt 4.5.4.2).

Zerlege Systeme in Module

Die Modularität eines Systems besagt, ob und wie es in eine Menge jeweils in sich geschlossener Bausteine (Module) zerlegt wurde. Jedes Modul sollte bestimmte Verantwortlichkeiten kapseln, die über wohldefinierte Schnittstellen zugänglich sind. Die Bausteine eines modularen Systems sollten Blackboxes sein, deren Innenleben möglichst verborgen bleibt: Es genügt, die Schnittstelle von Bausteinen zu kennen.

Modularität hat eine Reihe bestechender Vorteile: Modulare Bausteine

- können getrennt voneinander entwickelt,
- unabhängig voneinander geändert und
- ohne Nebenwirkungen durch andere Bausteine mit identischer Schnittstelle ausgetauscht werden.

Die Forderung nach Modularität ist eine Spezialisierung des Prinzips der getrennten Verantwortlichkeiten.

Abstraktion, Kapselung und das Geheimnisprinzip

The private statement is your friend!

Trennen Sie „Was“ vom „Wie“ bei der Implementierung von Bausteinen.

Verbergen (kapseln) Sie das Innenleben Ihrer Bausteine vor der Außenwelt. Definieren Sie für Ihre Bausteine eine öffentliche Schnittstelle, die beschreibt, „was“ der Baustein leistet. Interne Entwurfs- oder Implementierungsentscheidungen, das „Wie“, halten Sie vor äußerem Zugriff verborgen.

Ohne Kapselung wird alles schwieriger

Wenn Sie die Kapselung oder das Geheimnisprinzip missachten, entstehen ungewollte und schwierige Abhängigkeiten zwischen den einzelnen Systemteilen (Subsystemen, Komponenten), die insgesamt zu einer höheren Komplexität führen. Ohne Kapselung erschwert eine Zerlegung des Systems das Problem, statt es zu vereinfachen (denn: was bekannt ist, wird auch ausgenutzt!). Dieses Prinzip ist meiner Ansicht nach einer der Klassiker schlechthin – zuerst 1972 (!) von David Parnas⁷ beschrieben. Es ist auch bekannt als *Information Hiding*. Im Grunde ist

⁷ <https://www.cs.umd.edu/class/spring2003/cmsc838p/Design/criteria.pdf>

es eine Spezialisierung von „Trenne Belange/Verantwortlichkeiten“: Die Verantwortung für die Implementierung (das „Wie“) wird getrennt von der Beschreibung der Schnittstelle „Was“.

Konsistenz

Der Begriff *Konsistenz* bedeutet in unserem Kontext Stimmigkeit, Geschlossenheit oder Zusammenhalt.

Konzeptionelle Integrität (conceptual integrity⁸) ist eine wesentliche Eigenschaft guter Entwürfe: Ähnliche Probleme sollten ähnlich gelöst werden, ein System soll eine kleine Menge von Entwurfs- oder Lösungsansätzen konsistent verfolgen. Entwurfsentscheidungen sollten innerhalb des Systems *stimmig* getroffen werden – zueinander passend.

Das C2Wiki⁹ nennt einige interessante Beispiele konzeptioneller Integrität:

- Unix: Das gesamte System basiert durchgängig auf der zentralen Abstraktion „File“: Directories, Devices, Sockets repräsentieren so etwas wie Files.
- SQL: Alle Daten liegen in Tabellen – mit Schlüsselbeziehungen untereinander.
- Lisp: Alles ist eine Liste.

Für mich zählen zu diesem Prinzip kleine wie große Entscheidungen. Unter anderem finde ich konsistenten Programmierstil relevant, aber auch der konsistente Einsatz von Bibliotheken, die konsistente Anwendung von Entwurfsmustern oder Lösungskonzepten.



Ein Anti-Beispiel: In einem mittelgroßen System kamen zur XML-Verarbeitung sieben (!) unterschiedliche XML-Parser zum Einsatz. Das hat neben technischen Problemen für eine Menge Verwirrung bei den Entwicklern gesorgt sowie für unnötige Komplexität.

Vermeide Redundanz (Don't Repeat Yourself, DRY)

Vermeiden Sie Redundanz, halten Sie alle Informationen in oder über Ihr System möglichst an einer einzigen Stelle. Dieses Prinzip ist auch bekannt als „*Single Source of Truth*“.

DRY fordert Sie zur Wiederverwendung auf: Faktorisieren Sie mehrfach benötigte Codestellen heraus, definieren oder beschreiben Sie Dinge an nur einer Stelle.

Wenn Sie dieses Prinzip durchgängig anwenden, müssen Sie bei Änderungen oder Fehlerkorrekturen nur an dieser einen Stelle ändern – theoretisch zumindest. In der Realität stößt DRY schnell an Grenzen – wenn Sie beispielsweise in Ihrer Dokumentation eine Bausteinsicht beschreiben – aber später die Bezeichner der Bausteine im Quellcode ändern. Sofern Sie nicht Ihren Code aus dem Modell heraus generieren, müssen Sie in solchen Fällen an mehreren Stellen Änderungen vornehmen.

Vorsicht¹⁰: Ein klein wenig Redundanz kann den Komfort oder die Verständlichkeit erhöhen. Sie möchten als Leser einer (Architektur-)Dokumentation lieber einen in sich geschlossenen Text lesen, statt dauernd in referenzierten Dokumenten nachblättern zu müssen.

⁸ Das hat bereits Fred Brooks 1975 gefordert und in [Brooks-95] bekräftigt.

⁹ <http://c2.com/cgi/wiki?ConceptualIntegrity>

¹⁰ Einen Kontrapunkt zu DRY setzt Joel Abrahamsson: <http://joelabrahamsson.com/the-dry-obsession/>

Und direkt noch eine Einschränkung: Wiederverwendung von Code kann auch dazu führen, dass Aufrufer die bestehenden Restriktionen bzw. Schnittstellen der aufgerufenen Codestellen einhalten müssen – obwohl sie das gar nicht wollen ...

Vermeiden Sie zyklische Abhängigkeiten

Zyklische Abhängigkeiten sind für alle Arten von Strukturen problematisch, weil sie die Wartbarkeit und Änderbarkeit von Systemen erschweren. Bild 4.3 zeigt eine solche Struktur. Drei Module hängen zyklisch voneinander ab. Stellen Sie sich nun vor, Sie hätten die Implementierung von Modul A verbessert. Bevor Sie eine neue Version des Moduls freigeben können, müssen Sie nicht nur Modul A testen, sondern aufgrund der zyklischen Abhängigkeiten auch noch die Module B und C.

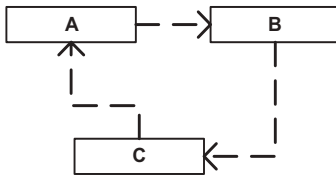


Bild 4.3
Zyklische Abhängigkeit

Praktisch werden Sie zyklische Abhängigkeiten nicht immer vermeiden können. Ihnen stehen jedoch einige Mittel zur Abhilfe zur Verfügung:

- Erzeugen Sie ein neues Modul oder Paket, je nachdem, welche Abstraktionsstufe eines Entwurfs Sie bearbeiten. Es enthält diejenigen Bestandteile aus A, die von C genutzt werden.
- Wenden Sie das Prinzip der Abhängigkeit von Schnittstellen an, wie in Bild 4.4 gezeigt: Modellieren Sie eine Schnittstelle für die von C genutzten Bestandteile des Moduls A und platzieren Sie diese in Modul C. Dadurch ersetzen Sie die benutzt-Beziehung zwischen den Modulen durch eine Vererbungsbeziehung.

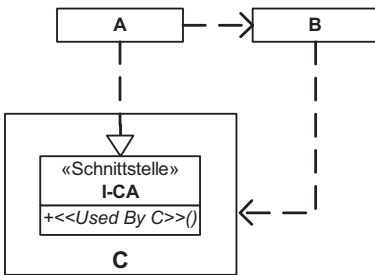


Bild 4.4
Auflösung zyklischer Abhängigkeiten

Erkläre angemessen

Stellen Sie Ihren Stakeholdern so viele Informationen (etwa in Form von Dokumentation) bereit, wie diese benötigen, in der für diese Stakeholder passenden Form: Dazu müssen Sie die Erwartungshaltung von Stakeholdern klären (was wir bereits in Abschnitt 3.2.4 diskutiert haben).

Sie sollten dabei das Gebot der Angemessenheit bezüglich Erklärung, Kommunikation und Dokumentation beachten: *So viel wie nötig, aber so wenig wie möglich*. Anders ausgedrückt: Arbeiten Sie bezüglich Dokumentation systematisch sparsam.

Das gesamte Kapitel 5, insbesondere die hierarchischen Sichten (siehe Abschnitt 5.6) wie auch das Prinzip der Abstraktion (siehe oben), kann Ihnen dabei helfen.

4.1.3 SOLID-Prinzipien des objektorientierten Entwurfs

Da objektorientierte Entwicklung immer noch zu den wesentlichen Programmierparadigmen zählt, möchte ich einige bekannte Prinzipien für Sie zusammenfassen.

In der Literatur sind diese als SOLID-Prinzipien bekannt und wurden schon in grauer Vorzeit von Robert Martin als Grundlagen des objektorientierten Entwurfs publiziert¹¹. Seine Formulierungen beziehen sich auf den Entwurf einzelner Klassen, lassen sich aber in guten Teilen auch auf größere Bausteine übertragen.

Tabelle 4.1 Die SOLID-Prinzipien (nach Robert Martin)

S O L I D	SRP: Single Responsibility Principle: Das erkennen Sie wieder – wir haben das weiter vorne bereits als „Trenne Verantwortlichkeiten“ (Separation of Concerns) kennengelernt. Eine Klasse soll nur genau einen Grund haben, geändert zu werden.
	OCP: Open-Close Principle: Sie sollten Klassen erweitern können, ohne bestehenden Code zu modifizieren (siehe Abschnitt 4.1.3.1).
	LSP: Liskov Substitution Principle: Unterklassen müssen grundsätzlich für ihre Oberklassen eingesetzt werden können: wenn schon Vererbung, dann richtig (siehe Abschnitt 4.1.3.2).
	ISP: Interface Segregation Principle: Verwenden Sie feingranulare, client-spezifische Schnittstellen statt ein großes Universalinterface (siehe Abschnitt 4.1.3.3).
	DIP: Dependency Inversion Principle: Erlaube Abhängigkeiten von Abstraktionen, nicht von konkreten Implementierungen (siehe Abschnitt 4.1.3.4).

4.1.3.1 Offen-Geschlossen-Prinzip

(engl: *Open-Close Principle, OCP*)



Komponenten sollen offen für Erweiterung sein, aber geschlossen für Änderungen (Offen-Geschlossen-Prinzip).¹²

Komponenten sollten grundsätzlich so beschaffen sein, dass künftige Erweiterungen ohne Änderungen von bestehendem Quellcode möglich sind. Egal, ob Sie Klassen, Pakete oder andere Strukturen entwerfen: Wenn Komponenten dem Offen-Geschlossen-Prinzip (OGP) genügen, können Sie neue und zusätzliche Eigenschaften hinzufügen, ohne bestehenden Code modifizieren zu müssen. OGP stellt sicher, dass künftige Erweiterungen ohne Nebenwirkungen funktionieren, weil sich Änderungen nicht im bestehenden und funktionierenden Code fortpflanzen können!

OGP minimiert
Änderungsrisiken!

¹¹ <http://butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod> sowie http://www.objectmentor.com/resources/articles/Principles_and_Patterns.pdf

¹² Bei der Benennung dieses Prinzips hat Robert C. Martin Pate gestanden. Ich verwende hier die englischen Abkürzungen.

Wenn Sie funktionierenden Code ohne Änderung beibehalten, werden alle Komponenten, die diesen Code nutzen, auch weiterhin so arbeiten wie bisher!

Betrachten Sie das folgende (schlechte) Codefragment:

```
...
void draw(Form f) {
    if (f.type == circle)
        drawCircle( f );
    else if (f.type == square)
        drawSquare( f );
    ...
}
```

Hier soll in einer Methode `draw()` eine geometrische Form gezeichnet werden. Je nach deren Typ (Kreis oder Quadrat) wird über eine geschachtelte Abfrage die eigentliche Zeichensmethode aufgerufen.

Falls dieses Programm um weitere Formen, etwa Ellipse oder Dreieck, erweitert werden soll, müsste jedes Mal die oben gezeigte `draw()`-Methode entsprechend erweitert und im Quelltext angepasst werden.

Bild 4.5 zeigt eine sinnvolle Modellierung dieses Sachverhalts. Das Zeichnen geometrischer Objekte wurde in einer Schnittstelle (*interface*) modelliert. Die geometrischen Objekte selbst implementieren diese Schnittstelle. Das aufrufende System ist nur von der Schnittstelle abhängig und kennt die Klassen der beteiligten Objekte nicht mehr.

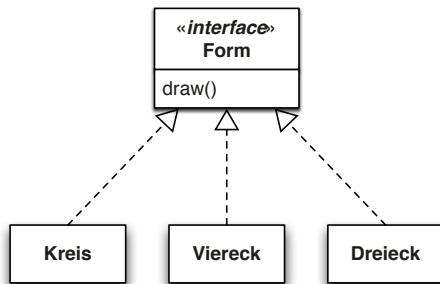


Bild 4.5

(Besseres) polymorphes Modell geometrischer Formen, genügt dem OGP

Polymorphismus

Diese Art der Modellierung nutzt eine der grundlegenden Eigenschaften¹³ objektorientierter Sprachen, den Polymorphismus. Der Begriff stammt aus dem Griechischen und bedeutet *Vielgestaltigkeit*. Kurz gesagt, entkoppelt Polymorphismus die Schnittstelle von der Implementierung, das „Was“ vom „Wie“.

In den meisten objektorientierten Sprachen funktioniert Polymorphismus auf Basis von dynamischem Binden. Hierbei wird die Zuordnung zwischen aufrufendem und aufgerufenem Objekt erst zur Laufzeit getroffen. Prozedurale Sprachen legen diese Zuordnung bereits zur Übersetzungszeit statisch fest.

¹³ Die anderen beiden zentralen Eigenschaften sind Datenabstraktion und Vererbung.

OGP für nichtobjektorientierte Systeme

Das Sprachmittel der Polymorphie steht in der Regel nur bei objektorientierten Programmiersprachen zur Verfügung. Dennoch können Sie in allen Fällen vom OGP profitieren. Selbst wenn nur die Modelle Ihrer Systeme dem Offen-Geschlossen-Prinzip genügen und die Implementierung ohne den Komfort von Polymorphismus auskommen muss – die konzeptionelle Auflösung von Abhängigkeiten verbessert die Qualität der Entwürfe.

OGP verbessert Entwürfe

4.1.3.2 Liskov-Substitutionsprinzip (LSP)

Das Substitutionsprinzip wurde von Barbara Liskov schon 1988 das erste Mal erwähnt. Es besagt, dass jede Basisklasse immer durch ihre abgeleiteten Klassen (Unterklassen) ersetzbar sein soll. Eine Methode, die ein Objekt vom Typ der Basisklasse erwartet, soll auch korrekt funktionieren, wenn ein Objekt der Unterklasse U übergeben wird.

So selbstverständlich dies klingt – die Technik der Vererbung erlaubt auch anderes Verhalten.

Betrachten Sie das Beispiel von Kreis und Ellipse (nach [Martin2000]): Die Schulweisheit, wonach ein Kreis lediglich eine spezielle Ellipse darstellt, könnte dazu verleiten, die Vererbungsbeziehung wie in Bild 4.6 zu modellieren. Rein technisch ist solche Vererbung kompilierbar, doch birgt die Implementierung versteckte Risiken.

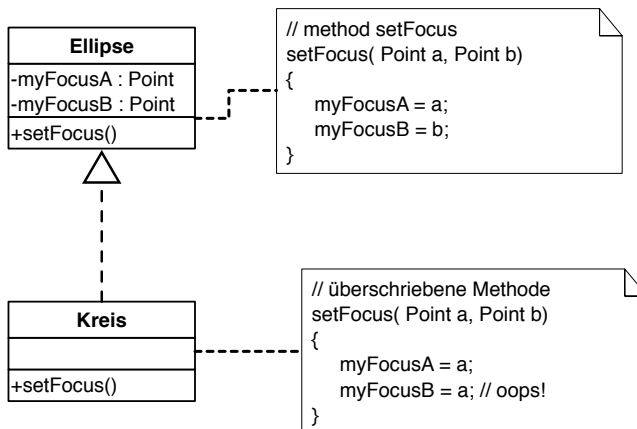


Bild 4.6
(Falsche) Vererbung
von Kreis und Ellipse

So besitzt eine Ellipse zwei Brennpunkte, ein Kreis jedoch nur einen, nämlich seinen Mittelpunkt. Die Implementierung der setFocus()-Methode der Ellipse wird in der Unterklasse Kreis überschrieben. Die Klasse Kreis stellt damit sicher, dass ihre Objekte nur einen einzigen Mittelpunkt haben. Das ist mathematisch korrekt, schafft jedoch ein Problem:

Ein Benutzer einer Ellipse darf erwarten, dass folgendes Stück Code erfolgreich funktioniert:

```

void f( Ellipse e) {
    Point p1 = new Point( 1,1);
    Point p2 = new Point( 2,2);
    e.setFocus( p1, p2 );
    assert ( e.getFocusB() == p2); // oh je!
}
  
```

Die Funktion setzt zwei Brennpunkte einer Ellipse und prüft anschließend, ob diese korrekt übernommen wurden.

Für Ellipsen funktioniert obiger Code problemlos. Sie können der Funktion jedoch auch eine Instanz von Kreis übergeben. In diesem Fall arbeitet die obige Methode jedoch fehlerhaft: Die überschriebene `setFocus()`-Methode der Klasse `Kreis` verhält sich anders als erwartet!

Substituierbarkeit gewährleistet Einhaltung von Verträgen

Wenn Sie das Substitutionsprinzip durchgängig berücksichtigen, erfüllen sämtliche Unterklassen die impliziten „Verträge“ ihrer Oberklassen. Überschriebene Methoden besitzen keine stärkeren Vorbedingungen als die der Oberklasse und keine schwächeren Nachbedingungen. Robert Martin formuliert es prägnant:



Unterklassen sollen nicht mehr erwarten und nicht weniger liefern als ihre Oberklassen.

4.1.3.3 Interface Segregation Principle (ISP)



Mehrere spezifische Schnittstellen sind oftmals besser als eine große Universalschnittstelle.

Wenn eine Komponente von mehreren anderen Komponenten eines Systems benutzt wird, dann entwerfen Sie mehrere nutzerspezifische Schnittstellen.



Modellieren Sie die verschiedenen Verantwortungsbereiche einzelner Komponenten in jeweils einer eigenen Schnittstelle.

Spezifische Schnittstellen

Jede dieser spezifischen Schnittstellen modelliert ausschließlich Aufgaben einer Komponente. So vermeiden Sie umfangreiche Universalschnittstellen. Das hält die einzelnen Schnittstellen einfacher und verringert somit Komplexität.

Was geschieht, wenn Sie dieses Prinzip nicht anwenden, sehen Sie in Bild 4.7. Eine Klasse wird von zwei Komponenten benutzt (im Bild als Client A und Client B bezeichnet).

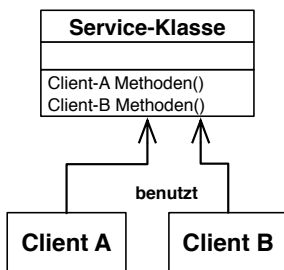
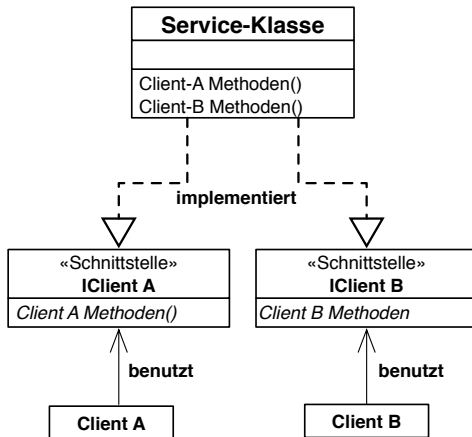


Bild 4.7
(Schlechte) Universalschnittstelle

**Bild 4.8**

(Bessere) separierte Schnittstellen

Wird eine der ausschließlich von Client A genutzten Methoden geändert, muss auch der Code von Client B neu übersetzt, getestet und ausgeliefert werden.

In Bild 4.8 sehen Sie eine bessere Modellierung dieses Systems. Die spezifischen Schnittstellen für Client A und Client B sind explizit als solche modelliert. Eine Änderung in den spezifischen Methoden für Client A zieht keine Änderung an Client B nach sich. Insgesamt ist das System flexibler und wartungsärmer geworden.



Aus einem etwas anderen Blickwinkel betrachtet: Wählen Sie Schnittstellen mit einem (einzigen) semantischen Zusammenhang, beispielsweise „sortierbar“. Klassen können solche einfachen Schnittstellen leichter implementieren als große Schnittstellenmonolithen wie etwa „sortierDruckUndSpeicherbar“.

Schnittstellen angemessen abtrennen

Die Einführung neuer Schnittstellen zur Entkopplung von Benutzern und Anbietern verbessert in vielen Fällen die Erweiterbarkeit und Flexibilität von Entwürfen. Gerade beim Entwurf von Klassenstrukturen kann die Abtrennung von Schnittstellen allerdings schnell unübersichtlich werden, weil die Anzahl der Schnittstellen stark wächst.

Beachten Sie daher das Prinzip der Schlichtheit: Wenn Ihre Entwürfe nach der Abtrennung von Schnittstellen unübersichtlich werden, dann:

Gruppieren oder kategorisieren Sie die verschiedenen Clienten.

Modellieren Sie getrennte Schnittstellen für diese Kategorien und nicht für jeden einzelnen Client.

4.1.3.4 Dependency Inversion Principle (DIP)



Erlauben Sie Abhängigkeiten nur von Abstraktionen, nicht von konkreten Implementierungen.

Die Abhängigkeit von Abstraktionen¹⁴ anstelle von konkreten Implementierungen ist eine effektive Option zur Steigerung von Flexibilität und Erweiterbarkeit.

Klassische Entwürfe für prozedurale Sprachen zeigen eine sehr charakteristische Struktur von Abhängigkeiten. Wie Bild 4.9 illustriert, beginnen diese Abhängigkeiten meist an einer zentralen Stelle, etwa dem Hauptprogramm oder dem zentralen Modul einer Anwendung. Die Module auf einer so hohen Ebene implementieren oft abstrakte Prozesse. Sie sind aber direkt von der Implementierung der konkreten Einheiten, nämlich den einzelnen Funktionen, abhängig.

Abhängigkeit schafft Probleme

Diese hierarchischen Abhängigkeiten schaffen in der Praxis oft Probleme. Sie bewirken nämlich, dass bei Änderungen von Low-Level-Funktionen oft auch die Implementierung der aufrufenden

Teile angepasst werden müssen.

Es tritt genau das Gegenteil davon ein, was Sie sich für flexible Architekturen wünschen. Möchten Sie beispielsweise die (im Bild 4.9 grau gefärbte) Funktion 1-2 ändern, müssen Sie möglicherweise Funktion-1 und Funktion-2 anpassen, ggfs. sogar ihr Hauptprogramm.

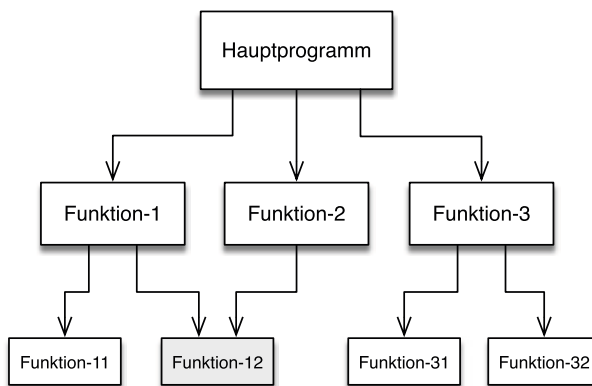


Bild 4.9
(Schlechte) Abhängigkeiten
in prozeduralen Systemen

Beschränken Sie Abhängigkeiten weitgehend auf Abstraktion, resultiert das in flexibleren und wartbareren Entwürfen. Betrachten Sie Bild 4.10. Dort hängt der zentrale Geschäftsprozess ausschließlich von (abstrakten) Klassen oder Schnittstellen ab. Konkrete Implementierungen hängen durch die Vererbungsbeziehung ebenfalls von Schnittstellen oder abstrakten Klassen ab. Auf hoher Ebene gibt es dabei keine Abhängigkeit von konkreten Implementierungen.

¹⁴ In der Literatur, etwa [Martin2000], finden Sie für dieses Prinzip oft die Bezeichnung „Prinzip der Umkehrung von Abhängigkeiten“ (*dependency inversion principle*).

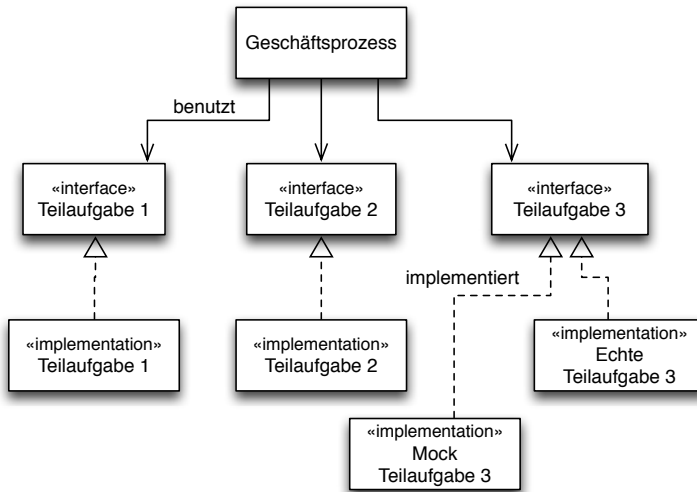


Bild 4.10
Gute Abhängigkeitsstruktur

Abhängigkeitsstrukturen dieser Art erlauben es, Implementierungskomponenten austauschbar zu halten.

Dependency Injection

Bei der Nutzung von Schnittstellen im objektorientierten Entwurf besteht immer das Problem, zu einer (abstrakten) Schnittstelle zur Laufzeit eine (konkrete) Instanz zu beschaffen: Wer übernimmt die Verantwortlichkeit, den Lebenszyklus der genutzten Instanzen zu verwalten? Wer entscheidet, welche konkrete Klasse zur Laufzeit letztlich instanziiert werden soll? Bild 4.11 zeigt diesen Fall: Ein Client der Klasse A modelliert eine benutzt-Beziehung zu einer Schnittstelle ISrv. Zur Laufzeit muss eine konkrete Instanz von ISrv erzeugt werden, auf die A zugreifen kann. Wer soll das bewerkstelligen? Wenn A dafür selbst verantwortlich ist, so besteht eine zu enge Kopplung zwischen A und dem Typ der konkreten Klasse (Srv).

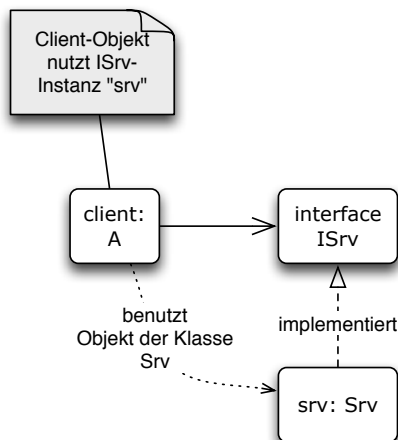


Bild 4.11
Woher kommt die Instanz bei Entkopplung über Schnittstellen?

Das „Entkopplungsmuster“ *Dependency Injection* stellt dafür einen eigenständigen Baustein bereit (den *Assembler* oder *Dependency Injector*), der zur Laufzeit über die oben genannten Fragen entscheidet. Der Assembler übergibt die Referenzen auf konkrete Instanzen an die abhängigen Objekte. Im Beispiel (siehe Bild 4.12) erzeugt er in Schritt 1 eine Instanz (namens *srv*) und übergibt diese im zweiten Schritt an den Client A.

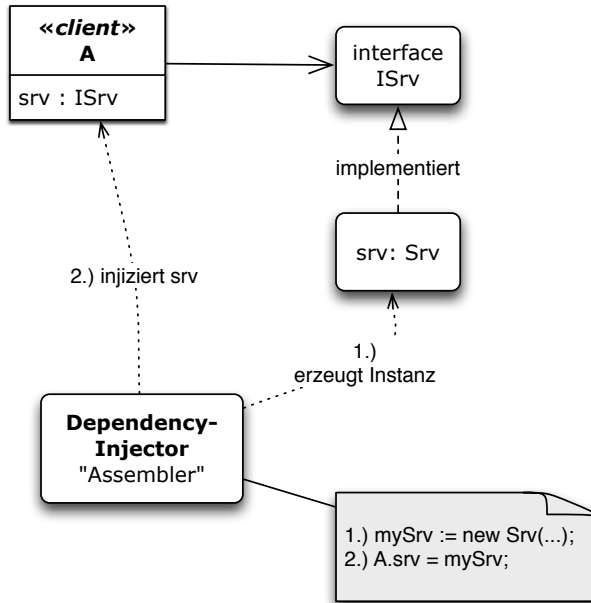


Bild 4.12
Dependency Injection

Martin Fowler diskutiert in seinem Einführungsartikel¹⁵ die sogenannte „Setter-Injection“, bei der Abhängigkeiten über set-Methoden injiziert werden, und die „Constructor Injection“, bei der die notwendigen Instanzen bereits beim Konstruktoraufwurf mitgegeben werden. Viele Frameworks (wie Spring¹⁶ für Java und .NET, PicoContainer oder Guice) implementieren dieses Prinzip und ermöglichen es somit, Abhängigkeiten „per Konfiguration“ zu verwalten.



Sollten Ihre Entwürfe unter einer großen Zahl von Abhängigkeiten (im Quellcode) leiden, sollten Sie den Einsatz eines Dependency-Injection-Frameworks in Erwägung ziehen. Allerdings kurieren diese Frameworks nur Abhängigkeiten auf Codeebene. Abhängigkeiten „im Großen“, zwischen Paketen und Subsystemen, müssen Sie weiterhin selbst verwalten!

¹⁵ <http://martinfowler.com/articles/injection.html>

¹⁶ www.springframework.org und www.springframework.net

■ 4.2 Heuristiken

Heuristik: die Kunst (nicht Wissenschaft, nicht Prozess), mit unvollständigen Informationen zu angemessenen Lösungen zu kommen.

Heuristiken sind Regeln zur Behandlung komplexer und wenig strukturierter (= schwieriger) Probleme (für die es meist beliebig viele Lösungsalternativen gibt).

Auf dem Weg in den sicheren Hafen (des fertigen Systems) bieten Heuristiken Orientierung im Sinne von Wegweisern, Straßenmarkierungen und Warnschildern. Aber Vorsicht: Sie geben lediglich Hinweise und garantieren nichts. Es bleibt in Ihrer Verantwortung, die passenden Heuristiken für eine bestimmte Situation auszuwählen:

Heuristiken geben
Orientierung, keine Garantie

*Die Kunst der Architektur liegt nicht in der Weisheit der Heuristiken,
sondern in der Weisheit,
a priori die passenden Heuristiken für das aktuelle Projekt auszuwählen.*

[Rechtin2000]

Recherchiere gründlich

Lernen Sie aus unterschiedlichen Quellen möglichst viel über das Problem und das darüber bekannte Wissen. Erarbeiten Sie die wichtigsten Grundbegriffe des Problemraums sowie deren Zusammenhänge. Suchen Sie bekannte Lösungsoptionen und -alternativen und verstehen Sie deren Stärken und Schwächen.

Beantworten Sie die folgenden Fragen:

- Gibt es Beispiele für erfolgreiche Lösungen? Falls ja, können Sie diese wiederverwenden? Welche grundlegenden Konzepte oder Ideen stecken hinter diesen Lösungen? Welche Kernabstraktionen verwenden sie?
- Gibt es Beispiele für gescheiterte Lösungsversuche? Was können Sie aus deren Problemen oder Fehlern lernen?
- Finden Sie heraus, wer Ihnen helfen kann. Verfügt Ihre Organisation über Experten auf diesem oder verwandten Sachgebieten?

Ich persönlich habe gute Erfahrungen damit gemacht, mit mindestens zwei verschiedenen Personen über Probleme und Anforderungen zu sprechen – und damit implizite Annahmen oder vergessene Anforderungen zu identifizieren.

Verfeinere schrittweise

*Wenn Du es nicht in fünf Minuten erklären kannst,
hast Du es entweder selbst nicht verstanden, oder es funktioniert nicht.*

[Rechtin2000]

Ein klassischer und systematischer Ansatz der Beherrschung von Komplexität lautet „teile und herrsche“ (*divide et impera*), auch genannt „*Divide and Conquer*“. Zerlegen Sie Ihr Problem in immer kleinere Teile, bis diese Teilprobleme eine überschaubare Größe annehmen.

Komplexität durch Zerlegung
beherrschen

Trial-and-Error

„We don't know what works until we try it“

(Tom Gilb)

Die Einflüsse und Anforderungen an Systeme sind in der Regel so komplex und vielseitig, dass a priori Entwurf (auch genannt *upfront-design*) oftmals nur eingeschränkt funktioniert. Eine mögliche Abhilfe heißt „ausprobieren“ – also über das wechselseitige Versuchen und Bewerten mehr über ein komplexes Thema lernen.

Prototypen oder Piloten zu entwickeln, ist eine Form von Trial-and-Error und die Philosophie iterativer Prozesse basiert ebenfalls stark darauf.

Wichtig sind dabei Bewertung und Feedback zu den unternommenen Versuchen: Wenn Sie etwas ausprobiert haben, sollten Sie anschließend explizit bewerten (analysieren, untersuchen, testen, messen), um es im nächsten Versuch dann wirklich besser machen zu können.

Ansätze wie „Test-Driven Development“ (TDD) oder „Behavior-Driven Development“ (BDD) gehen diese Heuristik systematisch an: zuerst eine Erwartungshaltung (Test) explizit aufschreiben und dann in Versuchen erarbeiten, wie dieses konkrete Detailproblem gelöst werden könnte.



Testen Sie so früh wie möglich. Durch Testen lernen Sie viel über das Problem und mögliche Lösungen.

Implementieren Sie (auch in Ihrer Rolle als Architekt!) automatisierte Testfälle mit den Testframeworks Ihrer Wahl. Das gibt Ihnen sehr kurzfristig Feedback zu Entwurfsentscheidungen.

Generalisiere

Suchen Sie Verallgemeinerungen, um von (vielen) Spezialfällen zu abstrahieren. Durch Generalisieren können Sie Mengenkomplexität auflösen.

Suchen Sie systematisch nach Dingen, die Sie anhand gemeinsamer Kriterien zusammenfassen können. Dabei dürfen Sie bei der Wahl dieser Kriterien sehr kreativ sein. Einige Beispiele:

- In Ihrer Kontextabgrenzung könnten Sie externe Systeme/Schnittstellen anhand ähnlicher fachlicher Inhalte zusammenfassen: Zeigen Sie im Kontext nur ein einziges externes (generalisiertes) System zur Kreditkartenprüfung – obwohl Sie in der Verfeinerung (Bausteinsicht Level 1, siehe Kapitel 5) mehrere unterschiedliche Kreditkartenunternehmen und -dienstleister anbinden¹⁷.
- Bausteine ähnlicher Implementierungstechnologie bieten sich für Zusammenfassung an.
- Ähnlich aufgebaute Abläufe bieten eine Menge Möglichkeiten der Generalisierung: Im Beispiel von Kapitel 12 haben wir in einem CRM-System ganz unterschiedliche Abläufe (aus Versicherung, Energieversorgung und Telekommunikation) durch Generalisierung viel einfacher implementieren können.

¹⁷ Mit dieser einfachen Generalisierung habe ich die Kontextabgrenzung eines größeren Systems von DIN-A2-Größe auf normales Schreibpapier verkleinern können – zur Freude aller Beteiligten.

- Vererbungsbeziehungen in der objektorientierten Entwicklung stellen offensichtliche Generalisierungen dar. Im Quellcode nur die Oberklasse bzw. ein Interface zu benutzen, schafft Flexibilität und entkoppelt Sie von den konkreten Implementierungen.

Spezialisieren

Betrachten Sie zuerst Spezialfälle, um einen Eindruck von der Lösung des allgemeinen Problems zu bekommen. Unter Spezialfällen verstehe ich hier beispielsweise Gutfälle, Normalfälle oder Fälle reduzierter Komplexität.

Betrachten Sie mehrere solche „einfachen“ Fälle, um wiederkehrende Lösungsmuster oder auch typische Probleme erkennen zu können.



Wir Softwareentwickler neigen dazu, generische Lösungen für möglichst alle Fälle zu finden. Manchmal jedoch genügt die Lösung eines Spezialfalls, um das aktuell bestehende Problem zu lösen.

Beispiele:

- Klären Sie bei komplexen Anforderungen grundsätzlich zuerst den „Sonnenschein“-Fall, in dem alles gut läuft und keinerlei Fehler auftreten. Das ist die einfachste Ausprägung der Spezialisierung.
- Können Sie in diesem einfachen Fall weiter vereinfachen? Eine Klasse von Eingabedaten auf feste Werte reduzieren? Aus einem „n“ eine „1“ machen, aus einer Liste möglicher Werte einen einzigen festlegen? Suchen Sie ganz bewusst nach einer möglichst trivialen Spezialisierung.
- Diskutieren Sie danach ein ganz einfaches Fehlerszenario, in dem nur ein unkritischer Fehler auftritt.
- Schließlich betrachten Sie ein Worst-Case-Szenario: Ein Teil Ihrer Hardware ist defekt, das System hat keinen Zugriff mehr auf das Dateisystem oder die Datenbank.

Wechseln Sie die Perspektive

Im realen Leben können wir kritische Situationen manchmal besser bewältigen, wenn wir uns in andere Personen hineinversetzen und deren Perspektive einnehmen.

Diesen Ansatz des systematischen Perspektivenwechsels halte ich für eine wirkungsvolle Methode beim Entwurf – Sie können sich damit selbst konstruktive Rückmeldungen zu Ihren Entscheidungen geben und die Konsequenzen Ihrer Entwurfsentscheidungen besser einschätzen.

Einige Beispiele für solche Perspektiven:

- Die klassischen Architektursichten (Baustein-, Laufzeit- und Verteilungssicht), die Sie in Kapitel 5 ausgiebig erläutert finden.
- Die Kontextabgrenzung, die das gesamte System als eine einzige Blackbox ansieht (und damit von Lösungsdetails abstrahiert).
- Technische Perspektiven, beispielsweise Datenspeicherung (Persistenz), Ablaufsteuerung und andere. Diese finden Sie unter dem Oberbegriff „Technische Konzepte“ in Kapitel 7.

- Die fachliche Perspektive, auch als Fachdomäne oder fachliche Struktur bekannt. Einen Aspekt davon, das Domain-Driven Design, stelle ich Ihnen in Abschnitt 4.3.1 vor. Einen weiteren finden Sie gleich im nächsten Abschnitt über fachliche und technische Teile.
- In Systemen mit einem hohen Maß an Benutzerinteraktion hilft die dedizierte Betrachtung der grafischen Oberfläche: grafische Gestaltung (Layout, Farbgebung), Typografie, optischer Fokus, Detailabläufe und die sogenannte „User Experience“.
- Beobachtungen des Systems zur Laufzeit, beispielsweise Messungen von Speicherbedarf, Datenvolumina, Durchlaufzeiten, Start- oder Verarbeitungszeiten, Benutzerzahlen und -verhalten.

Sie sehen – es gibt eine Menge Perspektiven, aus denen Sie Ihr System betrachten können ☺

Trenne Fachlichkeit und Technik

Trennen Sie technische und fachliche (d. h. anwendungsspezifische) Teile Ihres Systems. Halten Sie diese beiden Aspekte in Ihren Entwürfen getrennt, um später sowohl Technik als auch Fachlichkeit getrennt voneinander weiterentwickeln zu können.

Die strikte Trennung dieser unterschiedlichen Aspekte erfüllt die Prinzipien geringer Kopplung, hoher Kohäsion und getrennter Verantwortlichkeiten (siehe oben). Sie erleichtert die Arbeitsaufteilung, weil sich Teams dadurch auf bestimmte Aspekte konzentrieren und bei Bedarf spezialisieren können.

Eine methodische Ausprägung dieser Regel ist das Domain-Driven Design, das ich Ihnen in Abschnitt 4.3.1 vorstelle.

Achte auf Schnittstellen

Von technischen Details abgesehen, stecken die wichtigsten Aspekte für Architekten in den Schnittstellen und Beziehungen zwischen den Bausteinen ([Rechtin2000]). Das hat mehrere Gründe:

- Die einzelnen Bausteine oder Teilsysteme kommunizieren und kooperieren über Schnittstellen miteinander. Diese Zusammenarbeit bildet die Basis des Gesamtsystems – ohne Schnittstellen funktioniert nichts!
- Einzelne Bausteine konzentrieren sich auf ihre (lokalen) Probleme. Sie betrachten die übrigen Systeme oder Bausteine als „Peripherie“. Sie als Architekt halten die Fäden des gesamten Systems zusammen – eben über die Schnittstellen.
- Über Schnittstellen findet die Kommunikation mit der Außenwelt statt – ohne die jedes System nutzlos bleibt.
- Schließlich betreffen Fehler an Schnittstellen immer mindestens zwei Bausteine, nämlich sowohl Anbieter als auch Nutzer der Schnittstelle. Solche Probleme haben dadurch größere Konsequenzen – es sind nämlich mehr Leute betroffen, als wenn „nur“ ein einzelner Baustein defekt ist.

Der gesamte Abschnitt 4.4 widmet sich dem Thema „Schnittstellen“.

Kenne die Risiken

Je nach Kontext kann es nützlich sein, entweder mit den einfachen oder auch den riskanten Teilen eines Systems zu beginnen. Das klingt wie ein Widerspruch – aber beide Situationen können notwendig und sinnvoll sein:

Manche Teams brauchen möglichst schnell „schon mal etwas zum Vorzeigen“ – und für den schnellen Erfolg bieten sich einfache, risikoarme Teile des Systems an. Vorteil: Der entstehende Prototyp sieht für Endbenutzer oder Auftraggeber „schick“ aus, das Team gewinnt Vertrauen und erhält schnelle Rückmeldung der Stakeholder. Nachteil: Solche Prototypen enthalten aus architektonischer oder technischer Sicht Risiken und Unwägbarkeiten – die schwierigen Aufgaben hat das Team noch vor sich.

Falls Sie anders vorgehen und schwierige oder riskante Teile Ihrer Systeme oder Entwürfe möglichst frühzeitig angehen, haben Sie dafür

Riskante Teile zuerst

erstens länger Zeit und zweitens stellen Sie große Probleme früher fest. Möglicherweise dauert es länger, bis Sie „vorzeigbare“ Resultate vorweisen können – dafür haben Sie die großen Risiken bzw. Probleme dann auch schon gelöst. Dieses „riskante-Teile-zuerst“-Vorgehen sollte der Normalfall für Entwicklungsteams sein.

Für beide Varianten ist es unabdingbar, dass Sie mögliche Risiken kennen: Klären Sie mit dem Entwicklungsteam, wo Schwierigkeiten, Probleme, Unwägbarkeiten lauern. Dann können Sie bewusst entscheiden, wie Sie mit diesen Risiken umgehen.



Legen Sie Risiken offen: Führen Sie eine explizite Risikoliste, in der Sie Auswirkungen und mögliche Abhilfen aufzeigen.

Falls möglich klären Sie die großen Risiken zuerst – beginnen Sie mit den schwierigen oder kritischen Teilen des Systems.

Beachte die Geselligkeit von Fehlern

Die Anzahl der in einem Baustein noch *nicht entdeckten* Fehler ist proportional zur Anzahl der in diesem Baustein bereits *entdeckten* Fehler. Anders formuliert: Fehler clustern. Wenn Sie in einem Baustein bereits viele Fehler gefunden haben, werden Sie dort noch weitere Fehler finden. Konzentrieren Sie sich bei Entwurf und Test auf diese (kritischen) Bestandteile Ihres Systems!

Fehler sind gesellig

- Implementieren Sie (auch in Ihrer Rolle als Architekt!) automatisierte Testfälle mit den Testframeworks Ihrer Wahl. Das gibt Ihnen konkretes und kurzfristiges Feedback zu Entwurfsentscheidungen.
- Prüfen Sie Codequalität im Rahmen Ihrer Continuous-Integration oder automatisierten Builds. Falls Sie so etwas noch nicht haben, wird's höchste Zeit!
- Fehler-Cluster können Indikator für überhöhte Komplexität sein: Versuchen Sie, die Bausteine mit hoher Fehlerrate zu verkleinern oder anderweitig zu vereinfachen.

■ 4.3 Entwurfsmethoden

Jetzt kennen Sie eine Reihe wesentlicher Grundlagen für den Entwurf – nun wird es Zeit für einige konkrete Methodiken, die ich gerne „methodische Werkzeuge“ nenne.



Mein zentraler Ratschlag: Verwenden Sie diese methodischen Werkzeuge wechselseitig. Mindestens den fachlich orientierten Ansatz (DDD), siehe Abschnitt 4.3.1) sollten Sie mit regelmäßigen Ausflügen in QDSA (siehe Abschnitt 4.3.2) kombinieren.

Sorgen Sie durch Top-down- und Bottom-up-Vorgehen (siehe Abschnitt 4.3.3) für wechselnde „Flughöhen“ – dadurch behalten Sie sowohl den Überblick als auch die wichtigen Details im Blick.

4.3.1 Domain-Driven Design (Entwurf nach Fachlichkeit)

Beginnen Sie Ihren Entwurf grundsätzlich mit der Strukturierung der jeweiligen Fachdomäne. Dieser Ratschlag stellte bereits die Grundlage der (ehrwürdigen) *strukturierten Analyse* der frühen 1980er-Jahre dar, gilt aber unverändert weiter. In den letzten Jahren hat Eric Evans dafür den Begriff *Domain-Driven Design* (DDD)¹⁸ geprägt und die zugehörigen Konzepte präzisiert. Ich gebe Ihnen nachfolgend eine (zu) kurze Zusammenfassung, lege Ihnen daher zur Vertiefung weitere Literatur ans Herz, etwa [Evans04] (sehr ausführlich) oder [Vernon-16] (kompakter Einstieg).

Modellieren Sie die Sprache der Fachdomäne

Die Basis jeder *guten* Software ist ein profundes Verständnis der jeweiligen Fachdomäne. Zu Beginn jeder Softwareentwicklung sollten Sie daher Ihr Domänenverständnis vertiefen. Modellieren Sie eine Abstraktion der Domäne, diskutieren Sie diese Abstraktion intensiv mit Fachexperten, aber auch mit Softwareentwicklern. Halten Sie dieses Domänenmodell frei von technischen Aspekten, beschränken Sie sich ausschließlich auf fachliche Themen!

Ein stabiles und akzeptiertes Domänenmodell bietet Ihnen (mindestens) folgende Vorteile:

- Über das Modell verbessern Sie die Kommunikation zwischen Fachexperten und Entwicklungsteam.
- Fachexperten können ihre Anforderungen präziser ausdrücken.
- Wenn Sie das Domänenmodell direkt in Software abbilden, können Sie es leicht testen – ein unschätzbarer Vorteil.

Auf der Basis des gemeinsam abgestimmten Domänenmodells entwickeln Sie mit dem Team eine gemeinsame Terminologie der Domäne; [Evans04] nennt sie *Ubiquitous Language*. Das wird Ihr Systemjargon, Ihre domänenspezifische Sprache (deren Elemente Sie bei Bedarf in ein Glossar aufnehmen sollten).

¹⁸ Nicht zu verwechseln mit dem ehrwürdigen Debugger-Frontend gleichen Namens, zu finden unter <http://www.gnu.org/software/ddd/>

Clustern Sie im Großen („Strategisches Design“)

Im „strategischen Design“ (siehe Bild 4.13) finden Sie Fachgebiete oder Themenbereiche, die inhaltlich eng zusammengehören, genannt *Bounded Context* (BC). Innerhalb einer solchen fachlichen Einheit besitzen Begriffe eine konsistente Bedeutung. Im Normalfall gehören zur fachlichen Grundlage eines Systems (oder insbesondere einer Firma oder Organisation) mehrere solche Bounded Contexts. Jeder BC kann sein eigenes und eigenständiges Modell haben, in dem *Entities*, *Aggregate* und *Services* jeweils BC-spezifische Bedeutung besitzen.

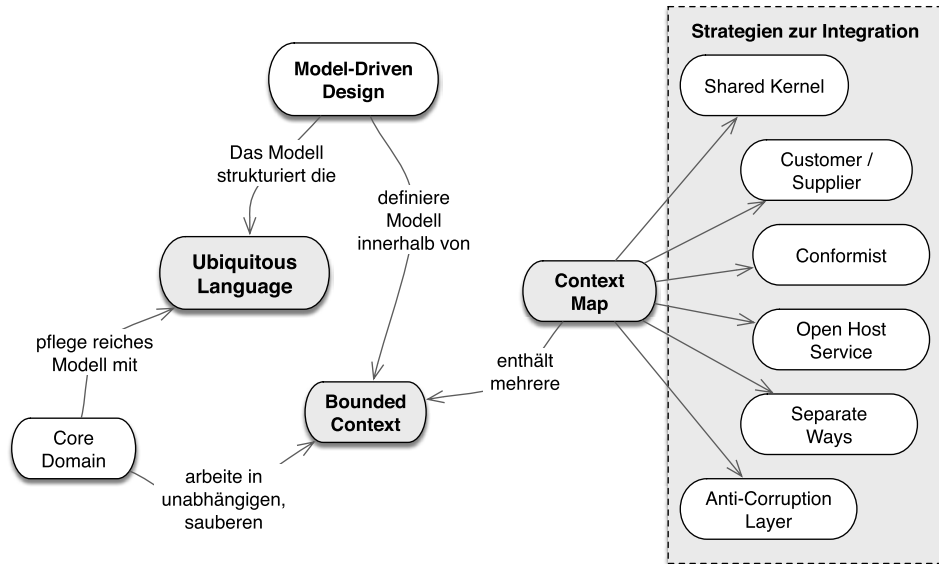


Bild 4.13 DDD-Muster zum „Strategischen Design“

Modellieren Sie den Zusammenhang mehrerer *Bounded Contexts* in einer *Context Map*: DDD macht mehrere Vorschläge (siehe rechte Seite von Bild 4.13), wie Sie diese integrieren können. Details dazu finden Sie in [Evans-04] und [Vernon-16].

Ein triviales Beispiel zeigt Bild 4.14 – die stark vereinfachte Domäne eines Telefonanbieters¹⁹: Darin finden Sie mehrere fachliche Bereiche (Bounded Context):

- **Telefonie (Netzbetrieb)**: Das Herzstück des Telefonanbieters – hier werden Telefongespräche vermittelt und die sogenannten Call-Data-Records (CDR) gesammelt – detaillierte Protokolle jedes einzelnen Telefonats (wer hat mit wem zu welcher Zeit über welche Leitungs- oder Funkstrecke telefoniert, waren Roaming-Partner involviert etc.)
- **Rechnungswesen („Billing“)**: Das zweite Herzstück – ohne Rechnungen bekommt das Unternehmen kein Geld. Hier werden die angefallenen Gebühren für Gespräche, Pauschalen, Sonderdienste usw. gesammelt und den Kunden in Rechnung gestellt.
- **Marketing**: Hier werden über verschiedene Kanäle und Aktionen Kunden geworben und die Bekanntheit des Unternehmens gesteigert.

¹⁹ Alle Telefonie-Experten mögen mir die drastische Simplifizierung verzeihen – Telefonie, deren monetäre Verrechnung sowie deren Vermarktung sind um Größenordnungen komplizierter, als ich das hier darstelle.

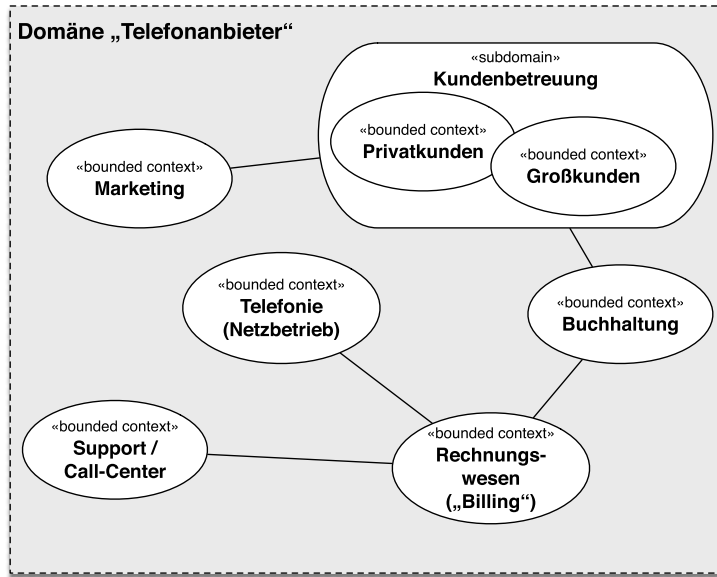


Bild 4.14 DDD-Beispiel „Telefonunternehmen“

- **Buchhaltung:** Verwaltung der Ein- und Ausgaben, Buchführung, Kontierung und ähnliche (langweilige) Dinge – die nichts mit Telefonie zu tun haben, aber für ein funktionierendes Unternehmen dennoch wichtig sind.
- **Support/Call-Center:** Kundenbetreuung, Bearbeitung aller Kontakt- und Supportanfragen, sowohl durch persönliche Betreuung wie auch Online-Kanäle.

Strukturieren Sie im Kleinen („Taktisches Design“)

Innerhalb eines einzelnen Bounded Context schlägt DDD vor, dass Sie feingranulare Basisbausteine modellieren (und implementieren!), [Evans04] nennt sie Internal Building Blocks (oder auch: *Tactical Patterns*, siehe Bild 4.15):

- **Entities** verfügen innerhalb der Domäne über eine unveränderliche Identität (einen Schlüssel) und einen klar definierten Lebenszyklus. Entities sind praktisch immer persistent. Sie stellen die Kernobjekte einer Fachdomäne dar.
- **Value-Objects** besitzen keine eigene Identität und beschreiben den Zustand anderer Objekte. Sie können aus anderen Value-Objects bestehen, niemals aber aus Entities.
- **Domain-Events** repräsentieren (vergangene) Ereignisse innerhalb der Domäne (etwa: Kunde-wurde-angelegt, Telefonat-wurde-begonnen, Telefonat-wurde-beendet). Oft implizieren Events Zustandsänderungen anderer Domänenobjekte (Entities, Aggregate). Umgekehrt lässt sich der Zustand von Entities oder Aggregaten aus den Events der Vergangenheit immer vollständig rekonstruieren.
- **Services** stellen Abläufe oder Prozesse der Domäne dar, die nicht von Entities wahrgenommen werden. Es handelt sich dabei um Operationen, die in der Regel nicht über einen eigenen Zustand verfügen. Parameter und Ergebnisse dieser Operationen sind Domänenobjekte (Events, Aggregate, Entities oder Value-Objects).

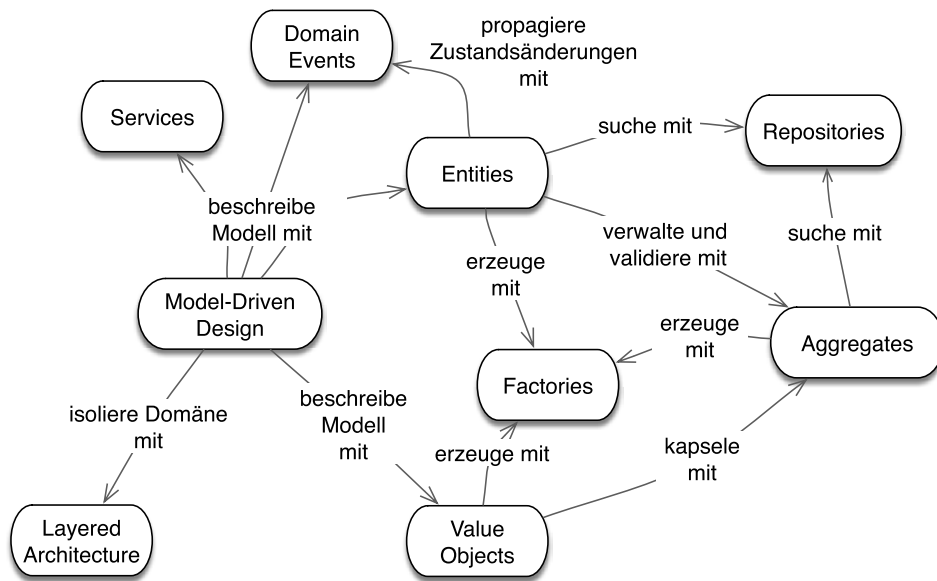


Bild 4.15 DDD-Muster zum „Taktischen Entwurf“ (nach [Evans04])

Für das kleine Beispiel des Telefonunternehmens beachten Sie beispielsweise, dass der Begriff „Kunde“ als Entität mit unterschiedlichen Bedeutungen in den folgenden Bounded Contexts vorkommt:

- Im Marketing sind Kunden primär solche Personen oder Organisationen, die zukünftig eine Geschäftsbeziehung mit dem Unternehmen eingehen könnten. Wichtig ist für das Marketing, welches Budget dieser Kunde für Telefonie zu investieren bereit ist, zu welcher Einkommens-, Interessens- oder Altersgruppe er/sie gehört. Adressinformationen sind relevant, aber konkrete Vertragsbeziehungen zu diesen Kunden existieren noch nicht.
- In der Kundenbetreuung sind Vertragsbeziehungen und Tarife wichtig, eventuell auch Beziehungen zu Familienmitgliedern. Kunden haben eine eindeutige ID, unter der durch- aus mehrere Verträge laufen können. Zu Kunden gehören hier ebenfalls Adressen und Kontakt- und Geburtsdaten sowie die zu Verträgen gehörenden SIM-Karten.
- In der Telefonie ist ein Kunde primär durch eine oder mehrere SIM-Karten gekennzeichnet, über die wirklich Telefongespräche geführt werden. Gesprächsprotokolle werden über die IDs dieser SIM-Karten in sogenannten Call-Date-Records aufgezeichnet. Adressen beispielsweise spielen hier keine Rolle.

Sie sehen – je nach Kontext sind für „Kunden“ völlig unterschiedliche Informationen relevant. DDD schlägt vor, dass Bounded Contexts die jeweils benötigten Daten selbst speichern und es kein kontextübergreifendes „globales“ Datenmodell oder Daten-Repository geben sollte – sondern besser kontextspezifische Datenspeicher.

Isolieren Sie die Domäne in der Architektur

Das Fachmodell dient als Grundlage des weiteren Entwurfs. Damit auch langfristig die Struktur und Konzepte der Fachdomäne in der Architektur repräsentiert sind, sollten Sie

die Implementierung der Domäne (etwa Aggregate, Entitäten, Services und Value-Objekte) in eine eigene Schicht der Architektur (*layered architecture*) verlagern.

DDD propagiert folgende Schichten:

User Interface (Presentation Layer)	Darstellung und Informationsanzeige, nimmt Eingaben und Kommandos von Benutzern entgegen.
Application Layer	Beschreibt oder koordiniert Geschäftsprozesse, delegiert an den Domain Layer oder den Infrastructure Layer.
Domain Layer (Fachmodellschicht)	Der Kern von DDD! Diese Schicht repräsentiert die Fachdomäne. Hier „lebt das Modell“ ([Evans04]) mit seinem aktuellen Zustand. Die Persistenz seiner Entitäten delegiert diese Schicht an den Infrastructure Layer.
Infrastructure Layer	Allgemeine technische Services wie beispielsweise Persistenz, Kommunikation mit anderen Systemen.

Als eine Variante der Implementierung von DDD bieten sich Microservices (siehe Kapitel 10) oder Self-Contained Systems (siehe <http://scs-architecture.org>) an.

Verwalten Sie Domänenobjekte systematisch

Weiterhin schlägt [Evans04] folgende DDD-Patterns vor, mit deren Hilfe Sie Ihre Domänenobjekte systematischer verwalten können:

- **Aggregate:** Sie kapseln vernetzte (d. h. miteinander assoziierte) Domänenobjekte. Ein Aggregat hat grundsätzlich eine einzige Entität als Wurzelobjekt. Diese Wurzel ist der einzige „Einstiegspunkt“ in das Aggregat, sämtliche mit der Wurzel verbundenen Domänenobjekte sind *lokal*. Objekte von außen dürfen nur Referenzen auf die Wurzelentität halten. Die Wurzel des Aggregats steuert den Lebenszyklus des Aggregats und der darin enthaltenen Elemente.
- **Factories:** Entities und insbesondere Aggregate können komplexe Strukturen vernetzter Objekte bilden, die Sie nicht über triviale Konstruktoraufrufe erzeugen wollen. Verwenden Sie Factories, um die Erzeugung von Aggregaten und Entitäten zu kapseln. Factory-Objekte arbeiten ausschließlich innerhalb der Domäne und haben keinen Zugriff auf den Infrastrukt-Layer.
- **Repositories:** Alle Arten von Objekten (sowohl aus dem Domain Layer wie auch dem Application Layer) benötigen eine Möglichkeit, die Objektreferenzen anderer Objekte zu erhalten. *Repositories* kapseln die technischen Details der Infrastrukturschicht gegenüber den Domänenobjekten. Dadurch bleibt das Domänenmodell auch in dieser Hinsicht „technologiefrei“. Repositories beschaffen beispielsweise Objektreferenzen von Entitäten, die aus Datenbanken gelesen werden müssen. Konkrete Persistenztechnologie, wie etwa Datenbank- und/oder Caching-Systeme, gehören zu diesem Repository²⁰.

Sie können DDD beispielsweise mit Hilfe event-basierter Architekturen umsetzen oder es mit CQRS (siehe Abschnitt 4.5.4.2) implementieren.

Siehe dazu auch <https://www.infoq.com/presentations/microservices-docker-cqrs>.

²⁰ Falls Sie DDD mit Hilfe von Microservices oder Self-Contained Systems (siehe Kapitel 10) umsetzen: Jeder Microservice hat seine eigene Datenhaltung.

Lesen Sie weiter ...

Für eine tiefergehende Abhandlung des (wichtigen!) Themas „Domain-Driven Design“ habe ich in diesem Buch zu wenig Platz. Falls DDD für Sie interessant klingt und/oder Ihre Fachdomäne halbwegs kompliziert oder umfangreich ist, sollten Sie an anderen Stellen weiterlesen oder -gucken:

Michael Ploed (Twitter: @bitboss) veröffentlicht ein großartiges Beispiel als eBook (DDD-by-Example, <https://leanpub.com/ddd-by-example>). Darin sehen Sie, wie DDD in der realen Welt (Finanzbranche) aussehen kann. Michaels Vorträge über DDD (zu finden bei YouTube oder Speakerdeck) finde ich ebenfalls hilfreich als Ergänzung.

[Evans 04] Die Originalquelle zu DDD, mit 700 Seiten allerdings sehr schwergewichtig und meiner Ansicht nach auch recht schwer zugänglich. Enthält wenige Beispiele, dafür umso mehr fundierten Inhalt. Meiner Ansicht nach höchstens als Referenz nützlich, nicht für die praktische Arbeit.

Viele DDD-Praktiker schwören auf Event Storming – eine Methodik zur grundlegenden Erarbeitung von Domänenwissen auf Basis fachlicher Ereignisse (Domain Events). Eine kurze Einführung gibt der Blog <http://ziobrando.blogspot.de/2013/11/introducing-event-storming.html>, ausführlicher finden Sie Event Storming in [Brandolini-17] erklärt.

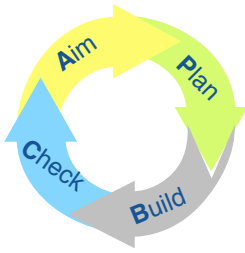
4.3.2 Quality-Driven Software Architecture

Bei der Quality-Driven Software Architecture (QDSA) legen Sie den Schwerpunkt der Betrachtung auf Ihre konkreten Qualitätsziele und die zu deren Erreichung notwendigen Maßnahmen.

Als Softwarearchitekt müssen Sie bei der Konstruktion und Implementierung Ihrer Systeme also von Beginn an großen Wert auf die Erreichung der wichtigen Qualitätsmerkmale legen. Zum Glück ist das in der Praxis mit etwas Systematik auch ohne neue Werkzeuge gut machbar. Sie sollten Qualität zu Ihrem wichtigsten Entwurfsziel erheben und ihr dementsprechendes Augenmerk widmen. Folgen Sie dann einem klassischen Planen-Handeln-Prüfen-Zyklus mit den folgenden Schritten (siehe Bild 4.16):

1. **Aim**, zielen: Konkretisieren und priorisieren Sie die notwendigen Qualitätsanforderungen. Für Sie ein alter Hut – darüber haben Sie in Abschnitt 3.2.3 bereits gelesen.
2. **Plan**, Maßnahmen planen: Definieren Sie (gemeinsam mit Ihrem Team) Maßnahmenpakete für die jeweiligen Qualitätsanforderungen. Priorisieren Sie auf Basis fachlicher und technischer Gegebenheiten.
3. **Build**, bauen: Setzen Sie die hoch priorisierten Maßnahmen um.
4. **Check**, prüfen: Überprüfen Sie die Wirksamkeit Ihrer Maßnahmen. Hierzu müssen Sie möglichst nah am laufenden System oder Teilen davon messen und testen. Bei Bedarf können Sie nun nachsteuern. Fahren Sie dazu bei Schritt 1 (Aim) fort.

Dieses zyklische oder iterative Vorgehen erhebt die systematische Überprüfung der Zielerreichung zum System. Sie stellen dadurch sicher, dass hoch priorisierte Qualitätsanforderungen rechtzeitig in Konstruktion, Implementierung und Änderung von Systemen berücksichtigt werden.

**Bild 4.16**

Phasen der Quality-Driven Software Architecture

Aim: Qualitätsziele konkretisieren

In der idealen Welt bekommen Sie als Softwarearchitekt bereits aus der Anforderungsanalyse konkrete und spezifische Qualitätsanforderungen als Vorgabe. Allerdings habe ich in langen Jahren in der IT-Branche selten ideale Zustände angetroffen – regelmäßig musste ich die Qualitätsanforderungen an Systeme nacharbeiten. Das hat den einfachen Grund, dass die Funktionen und Abläufe eines Systems sich meist recht einfach beschreiben lassen, sich Auftraggeber und andere Stakeholder jedoch mit der konkreten Beschreibung von Wartbarkeit, Flexibilität, Robustheit, Ergonomie und Effizienz schwer tun.

Dabei gibt es doch seit langer Zeit Abhilfe gegen dieses Problem: Qualitätsszenarien (siehe Abschnitt 3.2.3) helfen, Qualitätsanforderungen und -ziele pragmatisch zu beschreiben. Viele Qualitätsmerkmale lassen sich mit deren Hilfe konkretisieren, beispielsweise Effizienz, Performance, Flexibilität, Erweiterbarkeit, Zuverlässigkeit.

Ich möchte Ihnen hier Szenarien nochmals am Beispiel vorstellen. Für ein hypothetisches Beispiel (*Foto-App*) wünschen sich die Auftraggeber die in Bild 4.17 als Qualitätsbaum dargestellten Qualitätseigenschaften – in der Abbildung lediglich als Begriffe formuliert.

Die dazu passenden Szenarien finden Sie in Tabelle 4.2.

Ich selbst habe gute Erfahrung damit gemacht, die Szenarien in Form einer Mindmap statt eines Baums zu gruppieren, weil die Zuordnung von Szenarien zu Qualitätsmerkmalen oftmals mehrdeutig ist. Mindmaps bieten attraktive Layoutmöglichkeiten und eignen sich hervorragend für interaktives Brainstorming, in dem Sie Qualitätsziele mit verschiedenen Stakeholdern erarbeiten können. Letztlich ist die Anordnung aber reine Geschmackssache.

Bitte beachten Sie in der obigen Tabelle die Spalte „Priorität“: Damit bringen wir die Qualitätsanforderungen in eine aus fachlicher Sicht angemessene Reihenfolge. Das fertige System wird sämtliche Qualitätsziele erreichen, die Priorität hilft während der Entwicklung bei Entwurfs- und Implementierungsentscheidungen.

Nun haben wir die Qualitätsziele und -anforderungen geklärt und konkretisiert – jetzt geht es an den nächsten Schritt, Strategie und Taktik zur Erreichung dieser konkreten Ziele.

Plan: Maßnahmen definieren

Dem Ingeniör ist nichts zu schwör – das gilt im Falle der jetzt konkretisierten Qualitätsziele auch für Softwarearchitekten: Kennen wir unsere Ziele (präzisiert durch die Szenarien), definieren wir spezifische Maßnahmen oder Konzepte zu deren Erreichung. Dabei können wir uns neben den *Best Practices* des Softwareengineering auch auf gängige Strategien und Taktiken ([Bass+12] hat dafür den Begriff *Quality-Tactics* eingeführt) für einzelne Qualitätsmerkmale stützen – wir brauchen also keinesfalls immer gleich Räder neu zu erfinden.

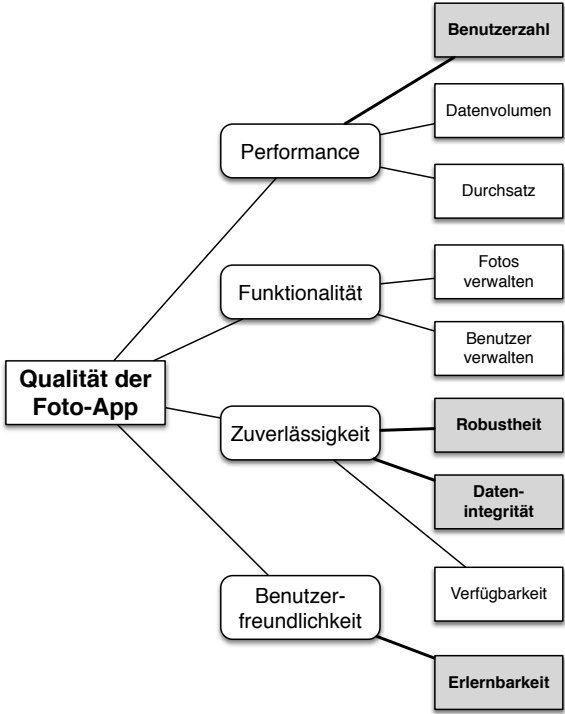


Bild 4.17
Qualitätsbaum einer
(hypothetischen) Fotoapplikation

Tabelle 4.2 Beispiele für priorisierte Qualitätsszenarien

Merkmal	Untermerkmal	Szenario	Prio
Zuverlässigkeit	Robustheit	Beim Upload eines korruptierten jpg-Fotos gibt das System einen aussagekräftigen Hinweis ohne Absturz.	B
Zuverlässigkeit	Robustheit	Beim Upload eines unbekannten Dateityps (etwa: pdf, svg, tiff) gibt das System eine entsprechende Meldung und speichert die Daten ohne Absturz.	A
Zuverlässigkeit	Datenintegrität	Das System wird unter keinen Umständen die von Benutzern hochgeladenen Fotos modifizieren oder beschädigen.	A
Performance	Benutzerzahl	Das System unterstützt bis zu 15 Millionen Benutzer, die jeweils bis zu 50 GByte Speicher für Fotos und Metadaten nutzen dürfen.	B
Benutzerfreundlichkeit	Erlernbarkeit	Gelegenheitsnutzer sollten in weniger als zwei Minuten in der Lage sein, ihr erstes Foto zu speichern.	B
Benutzerfreundlichkeit	Erlernbarkeit	Benutzer sollten auch komplexere Suchanfragen ohne Zuhilfenahme der Dokumentation erstellen können.	C

Bis dato fehlt zwar eine geschlossene Sammlung solcher Qualitätstaktiken, dennoch liefert Ihnen die gängige Literatur einen großen Fundus: Die seit einigen Jahren so verbreiteten *Patterns* helfen beispielsweise für die Qualitätsmerkmale Flexibilität, Robustheit, Performance und Sicherheit etc. schon gut weiter – sofern Sie selbige mit dem Fokus auf qualitätssteigernde Maßnahmen betrachten.

Bei der *Quality-Driven Software Architecture* verwenden Sie die vorher definierten Qualitätsziele, um für jedes bekannte Qualitätsszenario oder -merkmal eine Menge möglicher Maßnahmen zu deren Erreichung zu sammeln. Sammeln Sie diese Maßnahmen mit Ihrem Team und halten Sie sie schriftlich fest. Eine einfache Tabelle oder ein Flipchart genügt – Sie brauchen keine besonderen Werkzeuge dafür.

Das Brainstorming, Sammeln und systematische Analysieren von Maßnahmenvorschlägen bilden den Kern der *Quality-Driven Software Architecture*.

In diesem zentralen Schritt gilt es allerdings, an einer Stelle Vorsicht walten zu lassen: Einzelne Maßnahmen können leicht auf mehrere Qualitätsmerkmale Einfluss nehmen. Eine positive Auswirkung auf ein Merkmal, etwa Performance, kann leicht negative Auswirkungen auf andere Merkmale haben, etwa Speicherbedarf, Robustheit oder Einfachheit. Sie sollten vorab eine *Konsequenzanalyse* durchführen, ehe Sie mit der Umsetzung Ihrer Qualitätsmaßnahmen beginnen. Es hat sich nach meiner Erfahrung bewährt, solche potenziellen oder bekannten Konsequenzen direkt mit den Qualitätsmaßnahmen zu den jeweiligen Szenarien oder Qualitätszielen zu dokumentieren, d. h. sie auf jeden Fall schriftlich festzuhalten.



There is no free lunch – es gibt nichts geschenkt!

Bei Qualitätsmerkmalen gilt diese Lebensweisheit ganz besonders, weil viele Merkmale oftmals negativ miteinander korrelieren:

Verbessern Sie ein Q-Merkmal eines Systems, werden Sie dabei fast zwangsläufig andere Merkmale verschlechtern.

Klassische Beispiele dafür sind folgende:

- Speicherplatz und Performance: Ihre schöne Performance-Optimierung geht oftmals auf Kosten von zusätzlich benötigtem Speicherplatz.
- Performance und Verständlichkeit: Durch den Cache und die DB-Denormalisierung sowie Ihr neues parallel verarbeitendes Hashverfahren ist zwar alles schneller geworden – die Implementierung ist aber auch deutlich komplexer – und damit weniger verständlich als vorher.
- Sicherheit und Benutzerfreundlichkeit: Benutzer von Software sind fast immer über Sicherheitsmaßnahmen genervt: Ablaufende Sessions, Nutzung von Einmalkennwörtern, 2-Faktor-Authentisierung machen Systeme sicherer, aber auf Kosten der Benutzerfreundlichkeit.
- Flexible, hochgradig konfigurierbare Anwendungen sind meistens weniger robust.

Wenige Pärchen von Qualitätsmerkmalen jedoch beeinflussen sich gegenseitig positiv: Senken Sie die Komplexität (gut!), wird sicher die Verständlichkeit steigen (auch gut!).

Daher rate ich Ihnen grundsätzlich, die Konsequenzen Ihrer QDSA-Maßnahmen im Team zu diskutieren.

Was jetzt noch fehlt, ist klassisches Projektgeschäft, nämlich die Maßnahmen im Alltag einzuplanen und anschließend umzusetzen.

In den folgenden Abschnitten möchte ich Ihnen exemplarisch²¹ einige Maßnahmen vorschlagen, mit denen Sie Performance, Anpassbarkeit, Flexibilität oder Verfügbarkeit verbessern können.

Strategien für hohe Performance

In meinen Entwicklungsprojekten habe ich gelernt, dass praktisch jeder Auftraggeber eine hohe Performance²² fordert. In vielen Fällen wird diese Forderung sehr allgemein formuliert („kurze Antwortzeiten“).



Beispiel: Für eine Callcenter-Software forderte der Auftraggeber eine maximale Antwortzeit von einer Sekunde für sämtliche Operationen. Diese Anforderung wurde sehr hoch priorisiert und als formales Abnahmekriterium festgeschrieben. Gleichzeitig schrieb der Auftraggeber vor, sämtliche Daten des Systems mittels bereits vorhandener Mainframe-Systeme zu bearbeiten. Sekundäre oder redundante Datenspeicher zur Performance-Steigerung waren nicht erlaubt.

Durch einen funktionalen Prototypen konnte das Entwicklungsteam nachweisen, dass viele der Zugriffe auf diese Mainframe-Systeme bereits deutlich mehr als eine Sekunde benötigten.

Die Anforderung bezüglich der Antwortzeiten wurde daraufhin neu verhandelt. Der Auftraggeber verringerte die Priorität und formulierte sie wie folgt um:

„Die Antwortzeiten des Systems dürfen maximal eine Sekunde plus die kumulierten Antwortzeiten der beteiligten Mainframe-Systeme betragen.“

Einige grundlegende Hinweise zur Bewältigung hoher Performance-Anforderungen:



- Benutzen Sie einen Profiler und führen Sie Lasttests durch. Lastkurven helfen in vielen Fällen, die Performance-Engpässe zu finden.
- Lösen Sie das Problem durch zusätzliche Hardware. Hardware ist (manchmal) preiswerter als Gehirnschmalz. Zusätzliche Prozessoren, mehr Speicher, schnellere Netzwerke oder Server können in manchen Fällen für ausreichende Performance sorgen. Einige Probleme bleiben mit diesem Ansatz jedoch bestehen:
 - Er erfordert teilweise erhebliche (Anfangs-)Investitionen.²³
 - Sie müssen im Vorfeld den „Beweis“ erbringen, dass die zusätzliche Hardware die Performance-Anforderungen erfüllt.

²¹ Vorschläge für qualitätssteigernde Maßnahmen wären sicher ein eigenes Buch wert – aus Platzgründen muss ich mich hier stark einschränken.

²² Ich bleibe hier beim Begriff „Performance“, obwohl die DIN/ISO-Norm 25010 von „Effizienz“ spricht.

²³ Oder Sie denken über „Cloud-Computing“ nach, bei dem Sie kaum eigene Hardware benötigen, sondern Rechenleistung im Internet mieten.

- Versuchen Sie, mit Ihren Auftraggebern die gewünschte Performance des Systems neu zu verhandeln.
- Verringern Sie die Kommunikation der Systemkomponenten. Nutzen Sie Caching oder Proxies, um Kommunikation und Datentransfer zu sparen.
- Verringern Sie die Flexibilität des Systems. Verzichten Sie auf zusätzliche Abstraktionsschichten.
- Verzichten Sie auf Verteilung. Verlagern Sie performancekritische Teile der Software zusammen auf einen Rechner oder Knoten, statt sie auf unterschiedliche Knoten zu verteilen.
- Führen Sie Redundanzen ein: Sie können Kopien kritischer Daten im Speicher halten, statt sie aus einer Datenbank zu lesen.
- Wenn Performance wirklich wichtig ist, können Sie den Abstraktionsgrad der Programmiersprache verringern. Setzen Sie systemnahe Sprachen oder Assembler statt objektorientierter Sprachen ein.



Beispiel: Analysen sogenannter „Protein-Faltung“ hilft der Wissenschaft bei der Konzeption und Entwicklung neuer Medikamente. So startete Anfang 2020 beispielsweise im Projekt „Folding At Home“²⁴ (FAH) eine umfangreiche Analyse des Covid-19-SARS-Virus, um Ansatzpunkte für die Entwicklung eines Impfstoffs gegen das Corona-Virus zu finden. Hierzu sind immens aufwendige Berechnungen auf Molekülketten notwendig, die über viele Computer verteilt ablaufen.

Bei diesen Algorithmen ist Performance immens wichtig: Die FAH-Anwendung nutzt daher eine eventuell vorhandene Grafikkarte („GPU“), einer Hardwarekomponente mit hoher Rechenleistung für parallele Prozesse: Je nach Hersteller werkeln auf einer GPU viele Cores (je nach Modell mehrere Tausend!) gleichzeitig. Damit verliert FAH sicherlich an Verständlichkeit des Quellcodes im Tausch gegen signifikant höhere Performance.

Ähnliche sind Implementierungen für Deep-Learning oder die Erzeugung von Kryptowährungen wie Bitcoin (das sogenannte *Crypto-Mining*) aufgebaut.

Strategien für Anpassbarkeit und Flexibilität

Flexibilität eines Systems umfasst Analysierbarkeit (änderungsbedürftige Teile identifizieren), Modifizierbarkeit (Anpassung) und Stabilität (keine unerwarteten Auswirkungen nach Änderungen). In praktisch allen Fällen kollidieren Anpassbarkeit und Flexibilität mit anderen Eigenschaften. Durch mehr Flexibilität:

- Steigt die Komplexität (und sinkt insofern die Verständlichkeit), weil Konfigurationsparameter und Indirektionen häufig über zusätzlichen Quellcode erkauft werden;
- sinkt die Robustheit, weil sich beispielsweise unverträgliche Konfigurationseinstellungen nicht mehr zur Compile- oder Installationszeit überprüfen lassen;

²⁴ Siehe <https://foldingathome.org/covid19/> und <https://en.wikipedia.org/wiki/Folding@home>

- sinkt die Performance, weil manche Entscheidungen erst zur Laufzeit getroffen werden können.



Beispiel: In einem unserer Projekte stand aufgrund der hohen erwarteten Lebensdauer des Systems (mehr als 15 Jahre!) die Flexibilität der Anwendung im Vordergrund aller Anforderungen. Sämtliche Details der eingesetzten Hard- und Softwareplattform sollten durch Architekturmittel gekapselt werden, CORBA war „politisch“ verpönt. Direkte Methodenaufrufe zwischen verschiedenen Komponenten wurden vom Auftraggeber untersagt und ausschließlich über das Entwurfsmuster „Command“ erlaubt. Die Softwarearchitekten konnten diese Anforderungen durch die Einführung von Abstraktionsschichten erreichen, die jeweils technische Details der darunterliegenden Schichten kapselten.

Gleichzeitig forderte der Auftraggeber allerdings eine sehr hohe Performance und kurze Antwortzeiten. Diese Forderung kollidiert mit der Schichtenbildung, die für die gewünschte Flexibilität notwendig war.

In diesem Fall zeigte ein ausführlicher technischer Prototyp die Unvereinbarkeit beider Anforderungen. Nach langen Verhandlungen willigte der Kunde ein, die (aus meiner Sicht übertriebenen) Forderungen hinsichtlich der Flexibilität zu lockern.

Wenn Flexibilität zu den Risiken (oder wichtigen Anforderungen) Ihres Systems gehört, dann können Ihnen folgende Ratschläge helfen:



Finden Sie gemeinsam mit Auftraggeber und Anwendern des Systems heraus, in welcher Hinsicht das System flexibel sein muss:

- Funktionalität (Hinzufügen neuer oder Modifizieren bestehender Funktionen),
- Datenstrukturen oder Datenmodell,
- eingesetzte Fremdsoftware (Datenbanksystem, GUI-Bibliothek, Middleware oder andere),
- Schnittstellen zu anderen Systemen (neue Schnittstellen, Modifikation bestehender Schnittstellen),
- Benutzerschnittstelle (veränderte Gestaltung oder Inhalte),
- Ziel- oder Betriebsplattform (Portabilität auf andere Betriebssysteme, Datenbanken oder Kommunikationssysteme).

Diese Informationen können den Bereich der notwendigen Flexibilität einschränken und Ihnen beim Entwurf des Systems mehr Freiheiten verschaffen. Sie können auf Basis dieser Informationen „Was-wäre-wenn“-Szenarien entwickeln, um verschiedene Architekturalternativen auf ihre Eignung zu prüfen.



- Sichern Sie die häufig geänderten Teile Ihres Systems durch automatische Tests (Unit- und Akzeptanztests) ab.
- Betreiben Sie möglichst ausgiebig „Information Hiding“.
- Verbergen Sie die internen Details einer Komponente vor anderen Komponenten.
- Führen Sie interne Abstraktionsschichten ein.
- Verringern Sie Abhängigkeiten zwischen Komponenten. Dabei können Ihnen Abschnitt 4.1 und Abschnitt 4.2 über Entwurfsprinzipien helfen.
- Halten Sie die Änderungen möglichst lokal: Die notwendigen oder erwarteten Änderungen sollten auf möglichst wenige Bausteine des Gesamtsystems beschränkt bleiben.
- Reduzieren Sie die Kopplung von Systembestandteilen untereinander:
 - Lassen Sie Bausteine immer über Schnittstellen miteinander kommunizieren. Verwenden Sie niemals Implementierungsdetails der benutzten Bausteine (das gewährleistet die Austauschbarkeit von Bausteinen).
 - Verwenden Sie Adapter, Fassaden oder Proxies (siehe Abschnitt 4.6 und [GoF]), um Bausteine loser zu koppeln.
- Sorgen Sie für verständlichen Quellcode – der unterstützt auch den Umbau im Großen. In Robert Martins empfehlenswertem Buch *Clean Code* ([Martin08]) finden Sie mehr dazu.

Falls Sie auch Flexibilität zur Laufzeit Ihrer Systeme benötigen, können Ihnen die folgenden Vorschläge helfen. Ihr Tenor lautet: Entscheiden Sie möglichst spät.



- Verwenden Sie Konfigurationsparameter, um Installations- oder Laufzeitparameter möglichst lange flexibel zu halten.
- Lagern Sie Geschäftsregeln oder -prozesse in externe Repräsentationen aus (Stichwort: Process- oder Rule-Engines). Techniken wie BPEL, BPMN oder deklarative Regel-Engines unterstützen Sie dabei.
- Für objektorientierte Systeme:
Verwenden Sie Polymorphismus²⁵, um die Identifikation konkreter Typen erst zur Laufzeit vornehmen zu lassen (leider funktioniert das nur mit objektorientierten Programmiersprachen).

Strategien für hohe Verfügbarkeit

Sie können die Verfügbarkeit Ihrer Systeme durch folgende grundsätzliche Maßnahmen verbessern: Fehlererkennung (*fault detection*), Fehlerbehebung (*fault recovery*) und Fehlerverhütung (*fault prevention*). Zu allen dreien gebe ich Ihnen einige (kurze) Ratschläge, die Sie für Ihre konkreten Anforderungen spezialisieren müssen:

²⁵ Falls Sie diese Bezeichnung nicht mögen – Sie können alternativ „Runtime-Type-Identification“ sagen.



- **Zur Fehlererkennung:**
Prüfen Sie frühzeitig auf Situationen, die zu Ausfällen oder Beeinträchtigungen führen können. Spendieren Sie beispielsweise Ihren Systemen eine robuste Ausnahmebehandlung und/oder realisieren Sie Mechanismen, die periodisch (über *ping*- oder *echo*-Mechanismen) oder kontinuierlich (*heartbeat*) die Verfügbarkeit Ihrer Systeme prüfen.
- **Zur Fehlerbehebung:**
 - Lassen Sie mehrere redundante Bausteine über die Richtigkeit von Ergebnissen abstimmen (*voting*). Dies ist ein beliebtes, aber aufwendiges Mittel aus der Hochsicherheitstechnik.
 - Halten Sie Systembestandteile mehrfach redundant verfügbar (*hot spare*) und ersetzen Sie im Fehlerfall den defekten Systembestandteil durch ein solches Ersatzteil. Das gilt für Hardware wie auch für Software. Beispiel hierfür ist die Auslegung großer Rechenzentren, in denen die komplette Hard- und Software mehrfach und voneinander räumlich getrennt vorgehalten wird.²⁶
- **Zur Fehlerverhütung:**
 - Verwenden Sie Transaktionen, indem Sie logisch zusammengehörige Operationen klammern. Falls innerhalb einer solchen Transaktion ein Fehler auftritt, setzen Sie das System durch ein Rollback auf den Zustand vor der Transaktion zurück.
 - Speichern Sie periodisch den gesamten Systemzustand in sogenannten *Checkpoints*. Im Fehlerfall können Sie auf den letzten Checkpoint zurücksetzen.

4.3.3 Top-down und Bottom-up

In Bild 3.2 haben Sie bereits den Perspektivenwechsel kennengelernt, der Ihnen beim Entwerfen fast immer helfen wird.

„Top-down geht vom Abstrakten, Allgemeinen, Übergeordneten schrittweise hin zum Konkreten, Speziellen, Untergeordneten. Bottom-up bezeichnet die umgekehrte Richtung. (Zwei grundsätzlich verschiedene Denkrichtungen, um komplexe Sachverhalte zu verstehen, zu beschreiben, darzustellen.)“

(aus: Wikipedia)

Bei einer Top-down-Betrachtung nehmen Sie bezüglich eines Systems eine Vogelperspektive ein: Blenden Sie Detailfragen weitgehend aus und kümmern Sie sich beispielsweise um Schnittstellen, die Top-Level-Struktur oder übergreifende Konzepte/Themen. Ihr Fokus liegt dabei auf dem „großen Ganzen“ – Einzelheiten sollten Sie in dieser Perspektive ruhig ausblenden.

²⁶ Für diese Strategie finden Sie auch den Begriff des Failover Cluster, wobei zwischen Hot-Standby- (Ersatzteil übernimmt ausschließlich im Fehlerfall) und Active-Active-Clustern (alle Teile laufen ständig mit und im Fehlerfall wird der defekte Teil ausgeschaltet) differenziert wird. Eine gute Erläuterung dazu finden Sie in [Wikipedia].

Sie können Top-down-Betrachtungen beispielsweise verwenden, um eine fachliche Zerlegung vorzubereiten: Teilen Sie das betreffende System top-down erst einmal in disjunkte Sachgebiete auf – deren Details sie dann einzeln weiterbearbeiten.

Bottom-up-Betrachtung ist entwicklerfreundlich: Sie können sehr schnell Detailprobleme lösen, kleine Aspekte des Gesamtproblems implementieren. Sie lernen dadurch einzelne fachliche oder technische Details kennen – die Sie anschließend als Baustein oder Konzept kapseln und in den Gesamtentwurf einbringen können.



Wechseln Sie Ihre Flughöhe: Achten Sie auf (manche) Details und klären Sie bottom-up deren Lösung.

Überprüfen Sie Konsistenz von Entscheidungen und Lösungen aus der Vogelperspektive (top-down).

■ 4.4 Schnittstellen entwerfen

It's Always the Goddamned Interfaces

Muster #43 aus „Adrenalin Junkies & Formular Zombies“²⁷

Schnittstellen erlauben die Zusammenarbeit verschiedener Teile von Systemen und bilden damit die Grundlage für modulare Systeme. Leider passieren erschreckend viele Fehler an Schnittstellen – sowohl zur Lauf- wie auch zur Entwicklungszeit.

In diesem Abschnitt möchte ich Ihnen einige Aspekte des Schnittstellenentwurfs vorstellen, ohne Ihnen jedoch eine abschließende Antwort auf die Frage liefern zu können: „Wie komme ich zu einer guten Schnittstelle?“

Meiner Ansicht nach hilft es in der Praxis beträchtlich weiter, sich die Anforderungen an Schnittstellen explizit vor Augen zu führen (siehe Abschnitt 4.4.1) und stets auf die wesentlichen Herausforderungen bei Schnittstellen zu achten (siehe Abschnitt 4.4.2). Danach bekommen Sie einige Optionen zum Entwurf von Schnittstellen aufgezeigt.

Zunächst einige Begriffe rund um Schnittstellen – illustriert an einem simplen Beispiel einiger Bausteine (Bild 4.18).

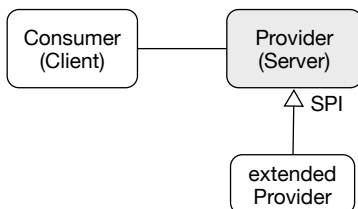


Bild 4.18

Grundbegriffe von Schnittstellen

²⁷ [DeMarco+07]

Consumer (auch genannt Dienstnehmer oder Client) und Provider (Anbieter, Server) können in unterschiedlichen Varianten zusammenarbeiten:

1. Consumer ruft einen Dienst (Funktion, Methode, Service) des Providers und erwartet synchron eine Antwort (sync-call-return).
2. Consumer ruft asynchron einen Dienst des Providers und erwartet irgendwann später eine Antwort.
3. Einer benachrichtigt den anderen – eine Antwort oder Bestätigung (*acknowledge*) ist nicht erforderlich.
4. Einer schickt eine Nachricht in die Welt hinaus, ohne sich darum zu kümmern, ob sich für diese Nachricht überhaupt jemand interessiert (*publish*).

Bei diesen unterschiedlichen Arten der Zusammenarbeit müssen wir jetzt *jeweils* unterscheiden, ob dem Consumer der jeweilige Provider zur Compile-Zeit bekannt ist oder ob Consumer und Provider erst zur Laufzeit miteinander verknüpft werden. Im letzten Fall könnte es also sein, dass es mehrere unterschiedliche Provider gibt, und zur Laufzeit wird entschieden, welcher konkrete Provider für die Zusammenarbeit ausgewählt wird.

Die Fälle eins bis drei sind typische Vertreter sogenannter API-(*application programming interface*)-Schnittstellen – bei denen ein Consumer durch Zusammenarbeit mit einem Provider ein spezifisches Ziel erreichen möchte.

Zwischen Consumer und Provider können über diese Schnittstelle beliebige Daten ausgetauscht werden: einzelne oder komplexe Attribute, Dokumente oder Dateien. Die Kommunikation kann dabei über beliebige technische Protokolle erfolgen, in-Memory oder auch über Netzwerke.

Etwas anders gelagert ist die Zusammenarbeit zwischen Provider und extended Provider, dem sogenannten *Service Programming Interface* (SPI). Dabei handelt es sich um Schnittstellen, die einen Baustein (hier: extended Provider) implementieren oder erweitern:

- Entweder der Provider bekommt über das SPI die Möglichkeit, erweiterte Leistungen in Anspruch zu nehmen (Beispiel: der Provider selbst verfügt nur über eingeschränkte Fähigkeiten zur Verschlüsselung, der extended Provider könnte erweiterte Schlüssellängen oder stärkere kryptografische Verfahren implementieren).
- Oder: Ein extended Provider bietet im Sinne eines Plug-ins überhaupt erst die Möglichkeit einer bestimmten Leistung.

Ein SPI gibt einem extended Provider eine Schnittstelle vor, die dieser zu implementieren hat, aber nicht selber aufruft.

Schnittstellen können innerhalb von Systemen gelten, aber auch nach außen (*public-API*).

4.4.1 Anforderungen an Schnittstellen

Eine Schnittstelle soll folgende Mindestanforderungen erfüllen (bitte schmunzeln Sie nicht bei dieser Aufzählung – ich habe genügend Beispiele gesehen, in denen nur wenige der folgenden Punkte galten):

- die Anforderungen der Nutzer (i. d. R. Consumer) erfüllen. Das ist insbesondere für öffentliche Schnittstellen (public API) *praktisch* schwierig, weil Provider ihre Consumer möglicherweise gar nicht kennen.

- einfach zu erlernen und benutzen. Entwickler möchten bei der Nutzung einer API möglichst schnell und einfach ihr Ziel erreichen – die Schnittstelle sollte das ermöglichen. Ein geschickter Weg zu diesem Ziel sind Consumer-spezifische Schnittstellen (siehe Abschnitt 4.1.3.3). Sie können als Provider auch automatisierte Tests (Unit-Tests) ihrer Schnittstellen als Dokumentation verwenden – was Entwicklern den Start erleichtern kann.
- Es sollte schwer sein, die Schnittstelle falsch oder missbräuchlich zu benutzen.
- Der Quellcode von Consumern (Clients, Benutzern der Schnittstelle) sollte verständlich und leicht wartbar sein. Machen Sie sich als Anbieter von Schnittstellen Gedanken, wie der entsprechende Consumer/Client-Code aussehen muss, um Ihre API zu verwenden.
- konsistent zu anderen Schnittstellen im System, also den „Gepflogenheiten“ des Systems entsprechen. Verwendet ein System durchgängig JSON, sollten Sie nicht an Ihrer Schnittstelle (überraschenderweise) XML nutzen²⁸.
- leicht erweiterbar. Consumer und Anwender kommen ständig auf neue Ideen und benötigen an Schnittstellen neue oder erweiterte Funktionalität – darauf sollte eine API vorbereitet sein.

4.4.2 Worauf müssen Sie achten?

Für Softwareentwickler ist sofort klar, dass sie bei der Definition von Schnittstellen auf Datenformate und -typen achten, die Signatur von Methoden oder Funktionen klären und darüber hinaus den Übertragungsweg zwischen Consumer und Provider entscheiden oder sicherstellen müssen.

Zusätzlich sollten Sie an Schnittstellen auf angemessene Fehlerbehandlung achten – siehe Abschnitt 4.1.1 (erwarte Fehler: Murphys Gesetz).

Richtig kompliziert wird es allerdings, wenn wir noch weitere Anforderungen an Schnittstellen hinzunehmen – ich stelle Ihnen eine kleine (schreckliche!) Auswahl vor:

- Sicherheit – Teil 1: Die Nutzung der Schnittstelle ist nur autorisierten oder authentisierten Nutzern (Consumern) erlaubt. Vor der Nutzung der Schnittstelle sind sicherheitsrelevante Vorarbeiten nötig.
- Sicherheit – Teil 2: Die Kommunikation über diese Schnittstelle ist vertraulich. Wir müssen Fragen der Verschlüsselung (Festlegung von Verschlüsselungsverfahren, Schlüssel- oder Zertifikatsbehandlung, Nichtabstreitbarkeit etc.) klären.
- Qualität: Die Schnittstelle soll bestimmte Garantien bezüglich Durchsatz, Performance oder Verfügbarkeit geben. Jetzt müssen Sie erstens diese sogenannten *Qualities of Service* (QoS) beobachten oder messen und zweitens Maßnahmen ergreifen, falls die QoS nicht mehr eingehalten werden.
- Kosten: Die Benutzung der Schnittstelle kostet Geld. Beispiel: pro Aufruf oder pro Datenvolumen. Als Provider möchten Sie ein verlässliches Protokoll, welcher Consumer sie aufgerufen hat, als Consumer möchten Sie gleichermaßen den Provider kontrollieren – und zwar buchhaltungs- oder fälschungssicher.

²⁸ Von dieser Regel dürfen Sie natürlich abweichen, wenn konkrete Bedürfnisse von Consumern das erfordern!

Ich höre hier einfach mal auf: Sie merken, dass bezüglich Schnittstellen sehr schwierige Themen drohen, die beliebig hohen Aufwand verursachen können. Pauschale Lösungen zu diesen Fragen gibt es leider nicht – aber Sie sollten diese Stolperfallen zumindest kennen.



Schnittstellen benötigen große Aufmerksamkeit: Als Softwarearchitekt sollten Sie die wesentlichen Schnittstellen Ihrer Systeme im Blick behalten:

- Bewerten Sie die Risiken sämtlicher Schnittstellen.
- Wo Risiken drohen, sollten Sie im Team klare Verantwortungen für die Schnittstellen vergeben.
- Gehen Sie riskante Schnittstellen frühzeitig an.

4.4.3 Tipps zum Entwurf von Schnittstellen

Einige Ratschläge²⁹ zum Entwurf von Schnittstellen möchte ich Ihnen noch mitgeben – ohne Anspruch auf methodische Vollständigkeit:

- Benutzen Sie Ihre eigenen Schnittstellen, beispielsweise indem Sie Tests gegen diese Schnittstelle implementieren. Das hat gleich zwei Vorteile: Erstens merken Sie, ob sich die Arbeit mit der Schnittstelle aus Sicht eines Consumers *gut anfühlt* – und zweitens prüfen Sie deren Korrektheit. Auch bekannt als „eat-your-own-dogfood“.
- Klären Sie die Anforderungen an Schnittstellen grundsätzlich mit Consumer und Provider, weil die beiden manchmal komplett unterschiedliche Sichtweisen oder Anforderungen haben. Das gilt gleichermaßen für interne wie externe Schnittstellen.
- Verbergen Sie Implementierungsdetails von Bausteinen, beispielsweise intern genutzte Datenstrukturen. Damit beachten Sie das Geheimnisprinzip und können Ihre Bausteine unabhängig optimieren oder weiterentwickeln.
- An Schnittstellen sind sprechende Namen und deren konsistente Benutzung besonders wichtig. Begriffe aus dem Domänenmodell bieten sich an (siehe Abschnitt 4.3.1).
- Minimieren Sie Überraschungen oder Seiteneffekte: Benutzer Ihrer Schnittstelle sollten immer genau (!) das bekommen, was sie erwarten (bzw. was die Namen von Methoden oder Schnittstellen suggerieren). Das ist eine Konkretisierung des Grundprinzips „Separation of Concerns“.
- Stellen Sie an Schnittstellen „atomare“ Informationseinheiten bereit, anstelle Informationen in Zeichenketten zu kombinieren. Nutzer der Schnittstelle sollten einfachen Zugriff auf Informationen erhalten, statt Strings parsen zu müssen.
- Dokumentieren Sie öffentliche Schnittstellen. Denken Sie dabei insbesondere an Sonder- und Fehlerfälle. Als einfache Form der Dokumentation schlage ich Unit-Tests vor ... entwicklerfreundlich, ausführbar und Sie sollten die ohnehin haben. ☺

²⁹ Einige davon habe ich einem großartigen Papier von Josh Bloch entlehnt – „How to Design a Good API“:
<http://lcsd05.cs.tamu.edu/slides/keynote.pdf>

■ 4.5 Architekturstile und -muster

Muster und Stil

Ein Architekturstil³⁰ fasst gemeinsame Merkmale von IT-Systemen zusammen, IT-Systeme eines bestimmten Stils bilden damit eine *Kategorie*. Jeder Stil enthält einige Regeln oder Muster zur Strukturierung eines Systems aus Elementen (Bausteinen) und Verbindungen (Beziehungen).

Architekturmuster können Ihnen bei der Zerlegung Ihres Systems in Bausteine und bei der Verteilung der Verantwortlichkeiten helfen. Sie bilden Vorlagen für Systemstrukturen. Ich verwende hier *Architekturmuster* als eine Konkretisierung oder Ausprägung von *Architekturstilen*³¹.

Tabelle 4.3 Architekturstile und Beispiele

Stil	Intention	Beispiele
Datenflusssysteme	Besteht aus Folge (Sequenz) von Operationen auf Daten	Batch-Sequentiell (Abschnitt 4.5.1.1), Pipes und Filter (Abschnitt 4.5.1.2), Prozesssteuerung
Datenzentrische Systeme	Gemeinsam verwendeter, zentraler Datenbestand	Abschnitt 4.5.2: Repository, Blackboard
Hierarchische Systeme	Bestehen aus Bausteinen in unterschiedlichen Ebenen einer Hierarchie	Haupt- und Unterprogramm, Master-Slave, Schichten (Abschnitt 4.5.5), Ports und Adapter (Abschnitt 4.5.6), virtuelle Maschine
Verteilte Systeme	Bestehen aus Speicher- und Verarbeitungsbausteinen, die über Kommunikationsnetze interagieren	Abschnitt 4.5.4: Client-Server, CQRS, Broker-Architekturen, Peer-to-Peer
Ereignisbasierte Systeme	Bestehen aus voneinander unabhängigen Bausteinen, die über Ereignisse (Events) kommunizieren, sich implizit aufrufen	Abschnitt 4.5.5: Ungepufferte Kommunikation: Publisher-Subscriber Gepufferte Kommunikation: Message-Queue, Message-Service
Interaktionsorientierte Systeme	Systeme mit (grafischer) Bedienoberfläche	Abschnitt 4.5.6: Model-View-Controller, Model-View-Presenter, Presentation-Abstraction-Control, Presentation Model
Heterogene Systeme	Verwenden mehrere Architekturstile parallel	Siehe „Sonstige Architekturstile“, beispielsweise REST

³⁰ Der Begriff Architekturstil stammt aus [Shaw+96].

³¹ Die genaue Abgrenzung zwischen Muster und Stil halte ich für irrelevant – beides sind nützliche Begriffe.

Eine breit angelegte Sammlung von Patterns (mit etwas anderen Kategorien) finden Sie online unter <http://patterns.arc42.org>.

Reale Systeme enthalten oft Merkmale mehrerer Architekturstile gleichzeitig, weil die Definitionen der Stile sich teilweise überschneiden. Das nennen wir dann *heterogene Architekturen*³².

Reale Systeme: heterogen

Im Vergleich zu den Entwurfsmustern („Design Patterns“, siehe Abschnitt 4.5 sowie [GoF]) beschreiben Architekturstile und Architekturmuster die Struktur von Systemen als Ganzes, weniger die Struktur einzelner Klassenverbünde oder Komponenten.

Ich zeige Ihnen Beispiele einiger praktisch bedeutsamer Architekturstile – die Tabelle 4.3 gibt einen ersten Überblick.

Diese Architekturstile bzw. Systemkategorien können Ihnen bei Entwurfsentscheidungen „im Großen“ nützlich sein. Daher sollten Sie für diese Stile folgende Parameter kennen:

- Arten der Bausteine, aus denen ein solches System besteht,
- Arten der Verbindungen zwischen den Bausteinen,
- Vor- und Nachteile dieses Architekturstils.

4.5.1 Datenflussarchitekturstil

4.5.1.1 Architekturstil Batch-Sequentiell

Batchsysteme stammen aus den Zeiten von COBOL und Mainframes. Sie verarbeiten ihre Eingangsdaten in einer Folge von Transformationen, wobei diese meist strikt sequentiell ablaufen, d. h., ein Nachfolger startet erst, wenn sein Vorgänger sämtliche Eingangsdaten komplett verarbeitet hat. Diese Systeme schicken somit Daten in „Stapeln“³³ von einem Verarbeitungsbaustein zum nächsten. Die Verbindungen zwischen diesen Bausteinen bilden oft Dateien, d. h., die Schnittstellen der einzelnen Datentransformationen sind Ein- und Ausgabedateien (siehe Bild 4.19).

Ablaufsteuerung und Koordination müssen durch externe Bausteine erfolgen, beispielsweise durch Skripte.

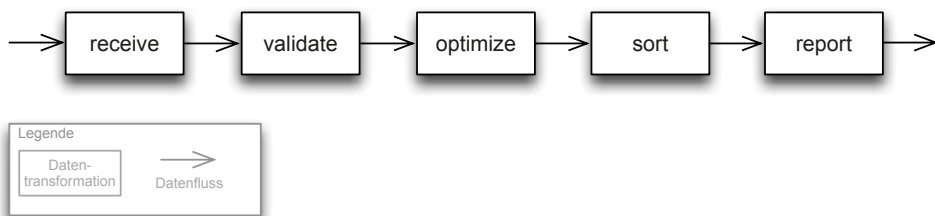


Bild 4.19 Beispiel für Architekturstil „Batch-Sequentiell“

³² Siehe [Qiang+10] für eine ausführliche Darstellung – weitaus moderner und praxisnäher als das mittlerweile sehr alte Original [Shaw+96].

³³ Daher der Name „batch“ (von engl. Batch = Stapel)

Trotz dieser einfachen Struktur und der fehlenden Interaktionsmöglichkeit im laufenden System laufen auch heute noch wesentliche Teile der (Offline-)Datenverarbeitung, etwa bei Banken und Versicherungen, nach diesem grundsätzlichen Muster ab – allerdings erweitert um Ablaufsteuerung, Fehler- und Ausnahmebehandlung sowie auf Basis von Datenbanken anstelle von Dateien.

Kriterien für die Anwendung

- Ergebnisse können durch eine (feste) Kette von Verarbeitungsschritten entstehen.
- Sämtliche Zwischenergebnisse können in ein einheitliches Format („Datei“) gebracht werden.
- Einzelne Verarbeitungsschritte können von mehreren oder allen Eingabeinformationen abhängen (Beispiele: Aggregationen mehrerer oder aller Eingangsdaten, inhaltliche Bezüge von Eingangsdaten zueinander).

Vorteile

- Einfache, funktional motivierte Aufteilung des Systems,
- (Meist) einfache und ähnliche Schnittstellen, z. B. Ein- und Ausgabedateien.

Nachteile

- Ein externer Baustein muss die Steuerung des Systems übernehmen, ebenso die übergreifende Fehlerbehandlung.

Mögliche Erweiterungen

- Parallelisierung einzelner Verarbeitungsschritte, um den Durchsatz zu erhöhen,
- Erweiterung einzelner Bausteine um weitere Ausgangsschnittstellen, um beispielsweise Verzweigungen (Fallunterscheidungen, Verarbeitungsvarianten) zu ermöglichen,
- Flexibilisierung der Schnittstellen zwischen den Bausteinen, Verwendung von Datenbanken oder Message-Queues in Ergänzung zu Dateien.

Implementierung

Das (Open-Source-)Spring-Batch Framework³⁴ stellt eine robuste und flexible Basis für (Java-) Batch-Anwendungen bereit. Seine außergewöhnlich gute Dokumentation³⁵ enthält viele Ratschläge für Entwurf und Implementierung solcher Systeme.

4.5.1.2 Architekturstil Pipes und Filter

Kennen Sie das Pipe-Symbol der Unix-Kommandozeile? Es leitet die Ausgabe einzelner Unix-Kommandos an nachfolgende Programme weiter. Ein Beispiel:

```
ls -l | grep ".pl" | sort
```

³⁴ <http://projects.spring.io/spring-batch/>

³⁵ <http://spring.io/docs>

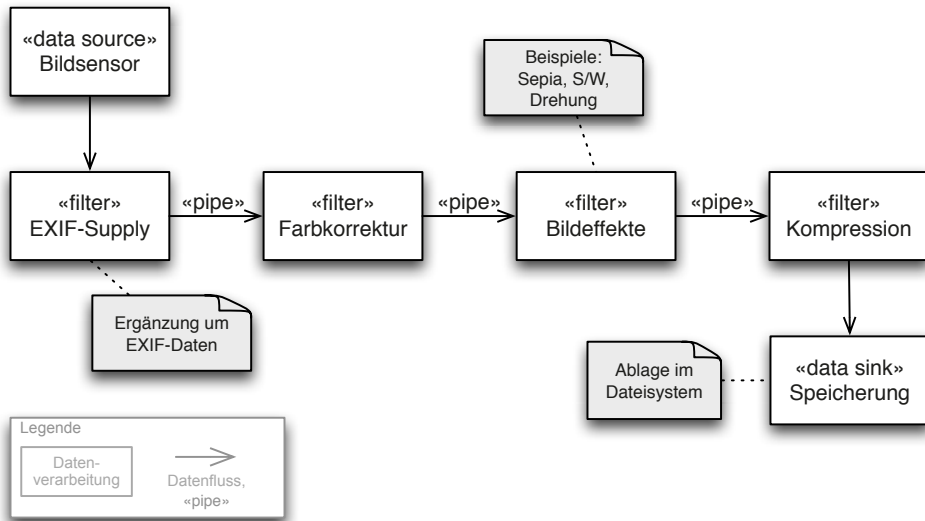


Bild 4.20 Pipe- und Filter-Struktur einer Digitalkamera (vereinfacht)

Diese Befehlsfolge erzeugt zuerst (zeilenweise) eine Liste aller Dateien des aktuellen Verzeichnisses und übergibt das Resultat (durch das Symbol „|“, gesprochen „pipe“) an das grep-Programm zum Filtern aller Zeilen, in denen die Zeichenfolge „.pl“ vorkommt.

Verallgemeinern Sie dieses Verfahren, gelangen Sie zum „Pipes- und Filter“-Architekturmuster: eine Folge von Verarbeitungseinheiten (Filter), miteinander durch Datenkanäle (Pipes) verbunden. Am Beispiel einer Digitalkamera zeigt Bild 4.20 dieses Muster. Jeder Filter gibt seine Ergebnisse über eine Pipe an den nächsten Filter weiter – die Pipes dienen lediglich als Transportmittel für Daten.

Filter verarbeiten Daten

Filter sind aktive Verarbeitungs- oder Transformationsbausteine, die Eingabedaten über Eingangs-Pipe(s) erhalten, prozessieren und über Ausgangs-Pipe(s) weitergeben.

Filter verarbeiten

Filter haben in diesem Muster keinerlei direkte Abhängigkeiten zu anderen Filtern.

Filter können ihre Verarbeitungsergebnisse an eine oder mehrere Ausgangs-Pipes weiterreichen. Sie können damit Verzweigungen innerhalb der Datenflüsse realisieren.

Unterscheiden Sie zwischen aktiven und passiven Filtern:

- Aktive Filter holen sich aktiv ihre Eingangsdaten aus der Eingangs-Pipe und stellen nach der vollständigen Verarbeitung ihre Ergebnisse aktiv der Ausgangspipe zur Verfügung (pull- und push).
- Passive Filter warten, bis sie Eingangsdaten übergeben bekommen und bis ihre Ergebnissdaten abgeholt werden.

Die aktiv-/passiv-Eigenschaft von Filtern kann sich auch auf ihren Ein- bzw. Ausgang beschränken, beispielsweise *aktiv-Input*, *passiv-Output*.

Pipes puffern und transportieren Daten

Pipes puffern und transportieren

Pipes sind Datenkanäle oder Transportmittel – sie können Daten entweder sofort oder auch zeitversetzt weitergeben. Manchmal fungieren sogar Datenbanken als Pipes (ein umfangreiches Beispiel in

Kapitel 11 illustriert das). Pipes können Daten puffern, um unterschiedliche Verarbeitungsgeschwindigkeiten ihrer Ein- und Ausgangsfilter auszugleichen.

In Pipe-und-Filter-Systemen können sämtliche Bausteine gleichzeitig aktiv sein – ganz im Sinne paralleler Prozesse.

Kriterien für die Anwendung

- Ergebnisse können durch eine (feste) Kette von Verarbeitungsschritten entstehen.
- Einzelne Datensätze können völlig unabhängig von anderen bearbeitet werden.
- Das Datenformat auf den Pipes ist stabil.

Vorteile

- Einfache Struktur, leicht zu implementieren,
- lose Kopplung beziehungsweise geringe Abhängigkeiten der Filter voneinander. Filter können i. d. R. einfach durch alternative Implementierungen ausgetauscht werden.

Nachteile

- Fehlerbehandlung: Da Filter einander nicht kennen, ist die Behandlung von „Folgefehlern“ (ohne strukturelle Erweiterung am Muster) zumindest schwierig, teilweise kaum möglich.
- Die Konfiguration oder Initialisierung der gesamten Verarbeitungskette benötigt eine „zentrale Steuerung“. Verlagern Sie diese Aufgabe in die Filter, so erzeugen Sie Abhängigkeiten der Filter voneinander.
- Es gibt keinen „gemeinsamen Zustand“ von Filtern. Alle Verarbeitungsinformationen müssen mit den Daten übertragen (oder einer zentralen Steuerung überlassen) werden.
- Ungeeignet für interaktive Systeme, in denen Benutzer Eingriffsmöglichkeiten in die Verarbeitung benötigen.

Mögliche Erweiterungen

- Pipes können entscheiden, an welche konkrete Instanz eines Filters sie ihre aktuellen Daten weitergeben („Wer hat gerade freie Kapazitäten?“).

Implementierung

In der Realität kommen häufig Kombinationen aus Batch-Sequentiell und Pipes und Filter vor. Beispiel:

1. Eine Eingangsdatei wird zuerst (batch-sequentiell) vollständig gelesen, entkomprimiert und entschlüsselt. Diese drei Operationen müssen für alle Daten gemeinsam durchgeführt werden (genauer: für die gesamte Datei, d. h. für alle darin enthaltenen Datensätze).
2. Anschließend werden (batch-sequentiell) sämtliche Datensätze kategorisiert (d. h. auf kleinere Datenstapel verteilt).

3. Jeder dieser einzelnen Stapel wird gemäß Pipe-und-Filter-Verarbeitung datensatzweise prozessiert.
4. Am Ende werden die Ergebnisse der Verarbeitung wieder (batch-sequentiell) in einer einzelnen Datei gesammelt.

4.5.2 Datenzentrierter Architekturstil

Datenzentrierte Systeme enthalten einen zentralen (passiven) Datenspeicher, der von den übrigen Bausteinen des Systems verwendet wird.

4.5.2.1 Repository

Repository-Systeme gruppieren eine Reihe aktiver Bausteine (in [Quiang+10] *Agenten* genannt) um den passiven Datenspeicher. Die Agenten realisieren Verarbeitung und Ablaufsteuerung innerhalb des Systems.

Bild 4.21 zeigt die grundsätzliche Struktur von Repository-Systemen.

Generell folgen sehr viele Informationssysteme dieser allgemeinen Struktur:

- Betriebliche Informationssysteme auf Basis von Datenbanken: Meist haben die „Funktionalen Bausteine“ hierbei eine komplexe innere Struktur.
- Versionsverwaltungssysteme für Quellcode (*Sourcecode-Repositories*), wie Subversion. Die modernen DVCS (*distributed version control systems*) wie Git oder Mercurial verteilen die Aufgaben des Repository auf mehrere getrennte Instanzen und ermöglichen die Synchronisation dazwischen.
- Data-Warehouse- oder Business-Intelligence-Systeme enthalten meist ein zentrales Repository, das über ETL-Mechanismen (*Extract-Transform-Load*) aus dem operativen Datenbestand befüllt wird.
- Die Registry von Microsoft Windows™.

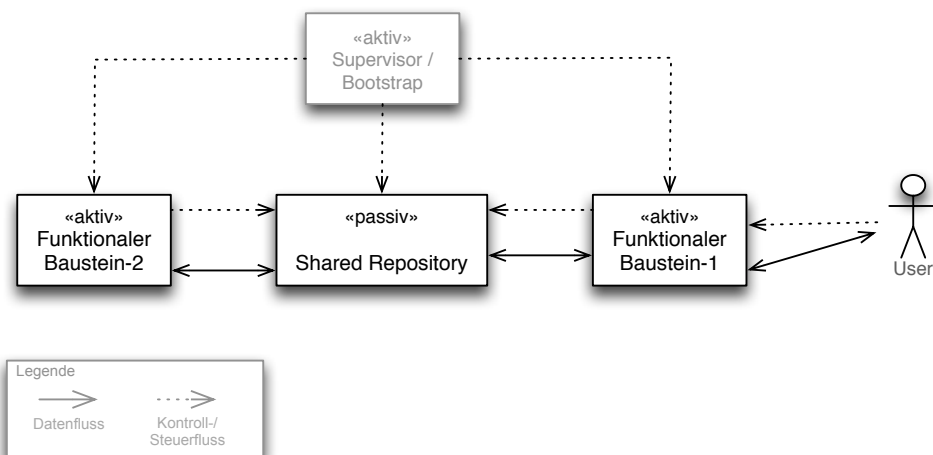


Bild 4.21 Bausteinstruktur (vereinfacht) eines Repository-Systems

4.5.2.2 Blackboard

The Blackboard architectural pattern is useful for problems for which no deterministic solution strategies are known. In Blackboard several specialized subsystems assemble their knowledge to build a possibly partial or approximate solution.

Aus: [Buschmann+96, p71]

Blackboard wurde im Bereich der künstlichen Intelligenz für Expertensysteme eingesetzt, etwa zur Sprach- oder Mustererkennung. [Buschmann+96] nennen drei wesentliche Bestandteile:

- eine oder mehrere Knowledge-Sources, die das Problem aus ihrer spezifischen Sicht untersuchen und Lösungsvorschläge an das Blackboard weitergeben;
- das zentrale „Blackboard“, das die Lösungsansätze oder -bestandteile der Knowledge-Sources verwaltet. Interpretieren Sie das Blackboard als die Sammelstelle für Zwischenergebnisse sowie endgültige Resultate der Verarbeitung;
- eine Kontrollkomponente, die das Blackboard beobachtet und bei Bedarf die Ausführung der Knowledge-Sources steuert.

Oft sind die Knowledge-Sources über einen publish/subscribe-Mechanismus an das Blackboard gekoppelt und erfahren somit, ob das Blackboard neue oder geänderte Informationen enthält.

Kriterien für die Anwendung

- Falls Sie sich mit künstlicher Intelligenz beschäftigen, könnte Blackboard für Sie passen. Systeme mit externalisierten Geschäftsregeln³⁶ (siehe Abschnitt 7.2) setzen im Grunde dieses Muster um: Das Blackboard ist der „Arbeitsspeicher“ der Regelmaschinen, in dem Regeln auf vorhandene Informationen angewendet werden.
- Komplexe Probleme ohne feste Lösungsverfahren, bei denen mehrere unabhängige Bausteine jeweils Teilaspekte einer möglichen Lösung berechnen können.
- Probleme, die sich nicht praktikabel über vollständige Suche oder Aufzählung lösen lassen.
- Probleme, bei denen partielle oder suboptimale Lösungen akzeptabel sind.

Vorteile

- Das Hinzufügen weiterer Agenten (Wissens- oder Informationsverarbeiter) ist einfach.
- Parallelität: Alle beteiligten Agenten können parallel arbeiten.

Nachteile

- Synchronisierung mehrerer Agenten kann aufwendig und schwierig sein.
- Aufgrund der bearbeiteten Probleme kann es schwierig sein, einen günstigen Zeitpunkt für das Ende der Berechnung zu finden.
- Test und Fehlersuche kann schwierig sein.

³⁶ etwa mit Regelengines wie JBoss-Drools, Visual-Rules oder ILog-Rules

4.5.3 Hierarchische Architekturstile

Systeme hierarchischer Architekturstile bestehen aus Elementen, die einander über- bzw. untergeordnet sind. Untergeordnete Bausteine bieten dabei ihren „Oberen“ feste Dienste an. Aufruf- oder Nutzungsbeziehungen verlaufen strikt in Richtung dieser Hierarchie.

Hierarchische Strukturen werden in der Praxis oftmals mit anderen Architekturstilen kombiniert.

Zu diesem Stil gehören die folgenden wichtigen Vertreter:

- Haupt-/Unterprogramm-Struktur, bekannt aus imperativen Programmiersprachen,
- Master-Slave, bei dem mehrere (redundante) Slave-Bausteine identische Dienste einem Master anbieten,
- Schichten (Layer), das in kommerziellen Informationssystemen wohl am meisten verbreitete Architekturmuster,
- Ports und Adapter, eine spezielle Ausprägung des Schichtenmodells,
- virtuelle Maschine, ebenfalls eine Spezialisierung des Schichtenmodells.

Viele Betriebssysteme oder systemnahe Programme prägen eine klare Hierarchie von Bausteinen aus – siehe Bild 4.22:

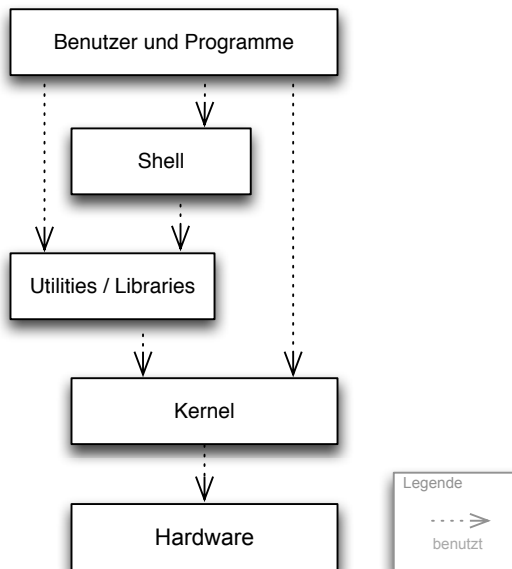


Bild 4.22
 Hierarchischer Aufbau des Betriebssystems Unix (vereinfacht)

4.5.3.1 Master-Slave

Eine wichtige Ausprägung hierarchischer Architekturstile, *Master-Slave*, unterstützt die Qualitätsanforderungen Fehlertoleranz und Verfügbarkeit: Ein Master verwendet mehrere Slaves, (redundante) Anbieter desselben Dienstes.

Die Slaves können in objektorientierter Terminologie Instanzen derselben Klasse sein, aber auch unterschiedliche Implementierungen desselben Interfaces. Der Master könnte im letz-

teren Fall beispielsweise eine Anfrage an mehrere Slaves vergeben und das Gesamtergebnis durch einen Mehrheitsentscheid oder eine Mittelung bestimmen.

Grundsätzlich können sämtliche Slaves parallel arbeiten, bei Bedarf sogar auf jeweils unterschiedlicher Hardware.

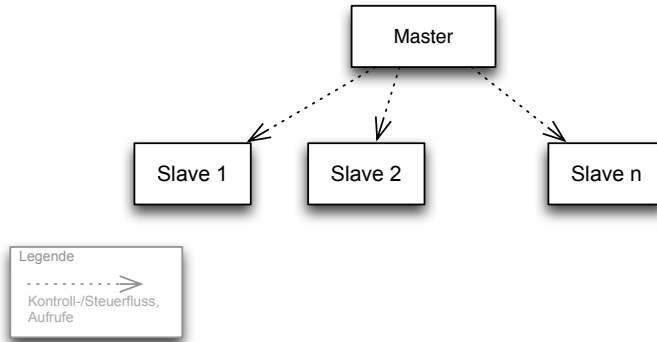


Bild 4.23
Master-Slave

Kriterien für die Anwendung

Aufgrund der möglichen Redundanz von Slaves eignet sich das Muster für Systeme mit hohen Anforderungen an Verfügbarkeit und Zuverlässigkeit.

4.5.3.2 Schichten (Layer)

In Informations- oder Websystemen hat sich die Schichtenbildung (*layering*)³⁷ als ein klassisches Mittel zur Strukturierung etabliert. Eine Schicht bietet den darüberliegenden Schichten bestimmte Dienste (*services*) an. Eine Schicht kapselt die Details ihrer Implementierung gegenüber der Außenwelt. Sie kann dabei ein beliebig komplexes Subsystem darstellen.

Komponenten innerhalb einer Schicht besitzen einen ähnlichen Abstraktionsgrad. So kapseln „niedrige“ Schichten oftmals technische oder sogar physikalische Aspekte von Systemen („Hardware- oder Infrastrukturschicht“).

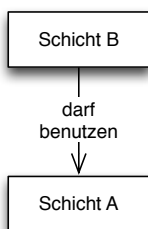


Bild 4.24
Schichten

Im Schichtenmodell liegt die Richtung von benutzt-Beziehungen fest: Nur obere (höhere) Schichten („B“) dürfen Dienste einer darunterliegenden Schicht („A“) benutzen.

³⁷ Layer (englisch): Schicht, Lage, Ebene

Die umgekehrte Nutzung (eine darunterliegende Schicht benutzt eine höhere) sollten Sie unbedingt unterbinden, da sie zu zirkulären Abhängigkeiten zwischen den Schichten führt. Wenn die wechselseitige Nutzung von Komponenten notwendig ist, sollten sich diese Komponenten in ein- und derselben Schicht befinden.

Schichten haben Vorteile ...

- Sie können die Implementierung einer Schicht austauschen, sofern die neue Implementierung die gleichen Dienste anbietet.
- Schichtenbildung kann helfen, Abhängigkeiten zwischen Komponenten verschiedener Schichten zu reduzieren.
- Schichtenbildung ist ein leicht verständliches Strukturkonzept.

Ein Beispiel für (erfolgreiche) Schichtenbildung sind die ISO/OSI-Schichten für Kommunikation. So können Sie jederzeit einen eigenen ftp-Klienten auf Basis der TCP/IP-Schicht schreiben, ohne die Details darunterliegender Schichten zu kennen (siehe Abschnitt 7.4 (Kommunikation)).

... aber auch Nachteile

- Schichtenbildung kann die Performance eines Systems beeinträchtigen, weil Anfragen unter Umständen durch mehrere Schichten durchgereicht werden, bis sie anschließend bearbeitet werden. Abhilfe schafft das sogenannte *Layer-Bridging*. Es erlaubt das „Überspringen“ von Zwischenstufen (siehe dazu Bild 4.25. Ein Aufruf der Schicht C an einen Service, der von Schicht A bereitgestellt wird, kann eine Schicht überspringen. Der Vorteil besserer Performance wird durch die zusätzliche Abhängigkeit (Schicht C hängt von Schicht A ab) erkauft.

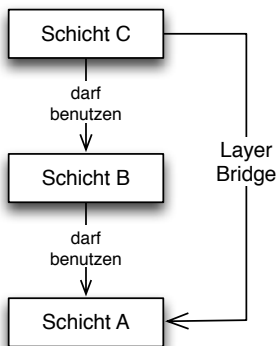


Bild 4.25
Layer Bridge

- Manche Änderungen eines Systems werden von Schichten schlecht unterstützt. Beispiel: Sie fügen einem System ein neues Datenfeld hinzu, das sowohl gespeichert als auch in der grafischen Benutzeroberfläche angezeigt werden soll. Das zieht Änderungen an allen beteiligten Schichten nach sich.

Ein (Standard-)Vorschlag für Schichten

Die folgenden Schichten bieten oftmals einen Startpunkt für eine weitere Zerlegung (siehe auch die Diskussion zu *Domain-Driven Design* in Abschnitt 4.3):



- Präsentation (Benutzeroberfläche),
- (optional): Applikationsschicht, Koordination von Fachobjekten und Delegation an Fachdomäne und Infrastruktur,
- Fachdomäne (*Business Architecture*): Ein solches Subsystem ermöglicht es seinen Entwicklern, sich auf die fachlichen Aspekte des Systems zu konzentrieren.
- Infrastruktur (*Technical Architecture*): kapselt die Komplexität der technischen Infrastruktur gegenüber der Fachdomäne und der Präsentation. Die technische Infrastruktur können Sie bei Bedarf in weitere Komponenten oder Schichten zerlegen. Typische Untergliederungen hierfür sind:
 - Persistenz und Datenhaltung,
 - physikalische Infrastruktur (Schnittstellen zur Hardware),
 - Integration (von Fremdsystemen),
 - Sicherheit,
 - Kommunikation und Verteilung.

Client/Server

Interessant für die Praxis ist häufig die Aufteilung der Präsentation in einen Client- und einen Server-Teil. Der Client-Teil läuft dabei auf den Client-Rechnern ab, die Server-Präsentationsschicht auf dem Server.



- Vermeiden Sie es, Aspekte der Fachdomäne (fachliche Logik) in die Präsentationsschicht zu verlagern. Das führt einerseits zu schwer wartbaren Systemen, andererseits reduziert es die Möglichkeit der Wiederverwendung innerhalb der Fachdomäne. Abhilfe bietet hier der Ports-und-Adapter-Architekturstil aus Abschnitt 4.5.3.3.
- Wenn Sie viele externe Ressourcen (Fremdsysteme) integrieren, kann eine Aufteilung der Infrastrukturschicht in Integrationsschicht und Ressourcenschicht übersichtlich sein.

Nachteile der (Standard-)Schichten

In der Realität infiltriert³⁸ bei den oben genannten „Standard“-Schichten sehr häufig fachliche Logik die Applikations- und Präsentationsschicht. Das führt allerdings bei Systemen mit langer Lebensdauer und/oder hoher Änderungsrate zu einer Reihe gravierender Probleme:

- Die Testbarkeit der Fachlichkeit („Domänenlogik“) sinkt – weil diese Logik jetzt über mehrere Schichten verteilt ist.
- Eine alternative Nutzung des Systems, beispielsweise über eine Batch-Schnittstelle oder ein neues grafisches Interface, wird deutlich erschwert, weil fachliche Logik für diese Schnittstelle erneut implementiert werden muss.
- Die Verständlichkeit des Gesamtsystems sinkt aufgrund der niedrigen Kohäsion der Bausteine innerhalb von Applikations- und Präsentationsschicht.

Aber damit nicht genug: Die Infrastrukturschicht, oft synonym für Datenbank, wird aus der darüberliegenden Fachdomäne aufgerufen: Damit dringen sehr leicht technische Details über die Persistenz in die Fachdomäne, die ja eigentlich komplett technikneutral sein sollte.

Aufgrund dieser praktischen Probleme hat Alistair Cockburn in 2005 eine bedeutsame Präzisierung der ursprünglichen Schichtenidee vorgenommen ([Cockburn05]). Ursprünglich hat er sie als *hexagonale Architektur* veröffentlicht, mittlerweile auch bekannt³⁹ unter dem Namen *Ports-und-Adapter*.

4.5.3.3 Architekturstil Ports-und-Adapter

(In der Literatur auch bekannt als *Hexagonal Architecture*, von Robert Martin als *Clean-Architecture*⁴⁰ bezeichnet)

Die jeweilige Fachdomäne sollte den Kern eines jeden Systems darstellen und keine (!) Abhängigkeiten von jeglicher Infrastruktur enthalten.⁴¹

Keine Abhängigkeit von Infrastruktur

Die ursprüngliche Terminologie (von Alistair Cockburn) der *Ports* und *Adapter* finden Sie in Bild 4.26: Externe Systeme, Implementierungen von Use-Cases oder die Benutzerschnittstelle verwenden Ports der Fachdomäne. Die technische Infrastruktur implementiert diese Ports (im Sinne des Dependency-Inversion-Prinzips, siehe Abschnitt 4.1.3.4).

Bild 4.27, stark angelehnt an Robert Martins Clean-Architecture illustriert dieselbe Idee in einem Schalenmodell: In einem inneren Kreis ist nichts (!) bekannt, was weiter außen definiert wird. Sourcecode-Abhängigkeiten zeigen immer (!) nach innen.

Die **Fachdomäne** kapselt fachliche Entitäten, Regeln, Objekte mit fachlichen Methoden, fachliche Funktionen oder fachliche Datenstrukturen. Innerhalb der Fachdomäne sollten Sie niemals Dinge ändern müssen, nur weil sich irgendwelche Details der Infrastruktur ändern.

Applikations-/Anwendungsregeln oder **Use-Cases** enthalten Regeln, Objekte oder Daten, die sich auf den Anwendungs- oder Systemzustand oder das Anwendungs- oder Systemverhalten beziehen. Sie implementieren die Anwendungsfälle des Systems – unter Zuhilfenahme der fachlichen Entitäten/Regeln der Fachdomäne.

³⁸ Den Begriff „infiltrieren“ habe ich aus [Cockburn05] übernommen – den Sie (online) unbedingt lesen sollten.

³⁹ Die Ports-und-Adapter-Ideen finden Sie beispielsweise in [Vernon13], [Freeman+10] sowie dem Apache-Isis-Framework.

⁴⁰ <http://blog.8thlight.com/uncle-bob/2012/08/13/the-clean-architecture.html>

⁴¹ Siehe hierzu auch Tom Hombergs <https://leanpub.com/get-your-hands-dirty-on-clean-architecture>.

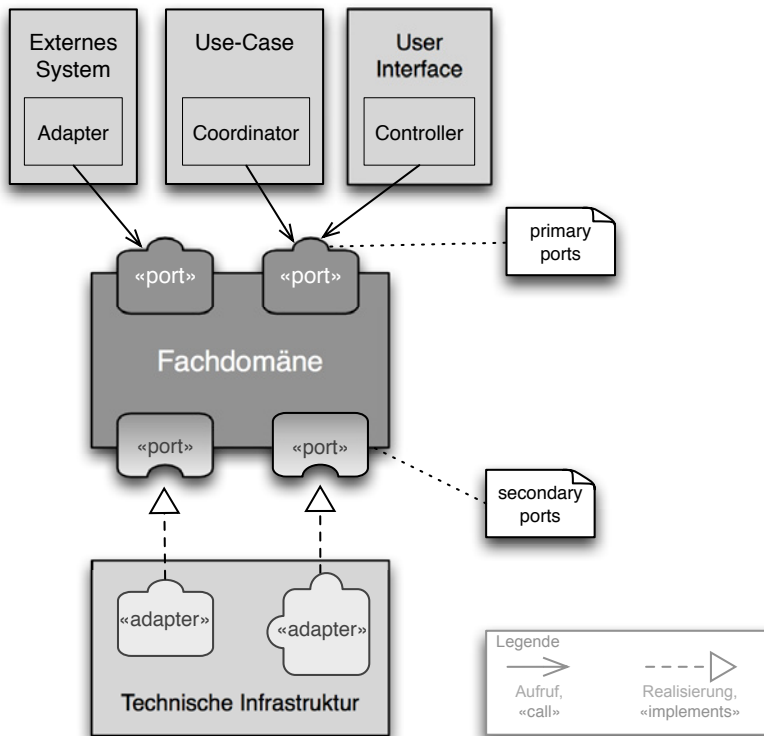


Bild 4.26 Ports und Adapter

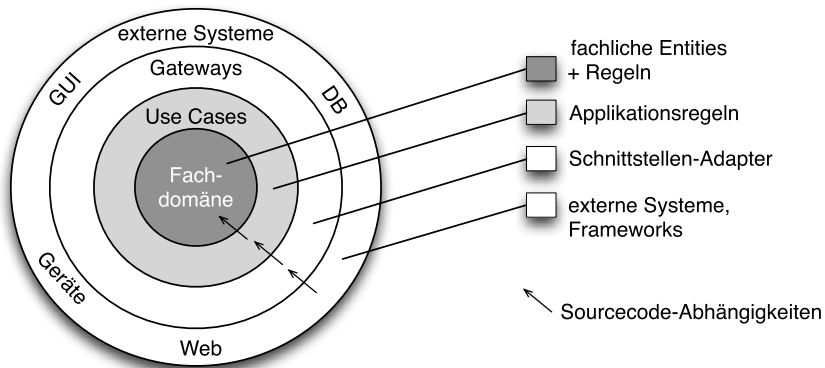


Bild 4.27 Schalen- oder Zwiebelmodell der Clean-Architecture

Die **Schnittstellenadapter** konvertieren Datenformate oder implementieren Schnittstellen der Fachdomäne und kapseln damit ihr gegenüber technische Details der konkreten Infrastruktur. Falls beispielsweise Daten in einer spezifischen NoSQL-Datenbank gespeichert werden müssen, so versteckt ein entsprechender Adapter sämtliche für diese Datenbank nötigen Details. Zu den Schnittstellenadaptern gehören auch Views, Controller oder Presenter einer grafischen Oberfläche.

Den äußersten Ring bilden **Frameworks** oder Softwareprodukte, die Sie im Rahmen einer Systementwicklung praktisch niemals selbst anpassen oder erweitern – außer mit sogenanntem Glue-Code.

Implementierung des (strikten) Schalenmodells

Zur Implementierung dieses Architekturstils bietet sich Dependency-Inversion an: Weiter außen liegende Bausteine implementieren Schnittstellen (Interfaces), die von inneren Bausteinen definiert werden. Die inneren Bausteine (beispielsweise Domain-Entities) benutzen nur die von ihnen selbst definierten Schnittstellen – die passende Implementierung wird zur Laufzeit bereitgestellt, beispielsweise durch Dependency-Injection oder Callbacks (siehe Bild 4.28).

- Primäre Ports stellt die Fachdomäne als Methoden oder Funktionen zur direkten Verwendung (Aufruf) bereit (DomainAPI). In typischen OO-Sprachen werden sie als public-Methoden deklariert und implementiert.
- Sekundäre Ports hingegen sind lediglich Schnittstellen, zu denen andere Bausteine noch Implementierungen bereitstellen müssen. Insbesondere wird die konkrete technische Infrastruktur (etwa: Persistenz) solche sekundären Ports implementieren, aber auch Mock-Objekte für Unit- und Integrationstests.
- Die „adapter“⁴² erfüllen die Aufgabe des gleichnamigen Entwurfsmusters – sie überführen die von Ports angeforderten Schnittstellen in die von der Infrastruktur bereitgestellten.

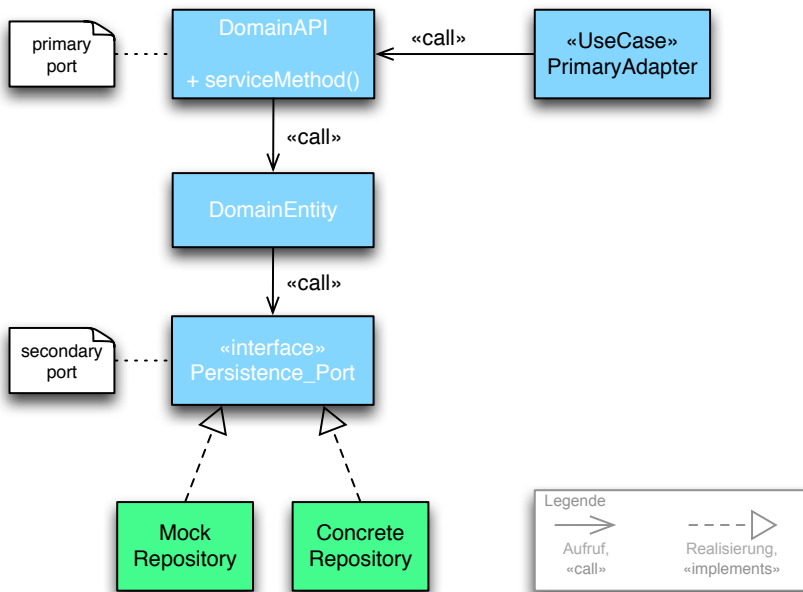


Bild 4.28 Implementierung der Clean-Architecture

⁴² Cockburn unterscheidet genau genommen noch zwischen primären und sekundären Adaptern: Die primären rufen primäre Ports auf, die sekundären Adapter implementieren sekundäre Ports. Ich vereinfache hier – und nenne nur die „sekundären“ Adapter.

Kriterien für die Anwendung

- Langlebige fachliche Strukturen, sowohl Daten wie auch Prozesse. In kommerziellen Informationssystemen ist das bei der Mehrheit der Systeme gegeben.
- Notwendigkeit hoher bis sehr hoher Testabdeckung der Fachdomäne. Durch die Ports können Infrastrukturteile (etwa: Datenbank, Verzeichnisdienste) oder externe Schnittstellen hervorragend durch Mock-Implementierungen ersetzt werden – was die Testbarkeit signifikant steigert.

Vorteile

- Implementierung fachlicher Teile komplett frei von Einflüssen der Infrastruktur,
- hohe Wartbarkeit und einfache Änderbarkeit des gesamten Systems durch hohe Modularisierung,
- leichte Austauschbarkeit der Infrastruktur, weil der fachliche Kern des Systems technologieunabhängig ist.

Nachteile

- Die strikte Einhaltung der Ports- und Adapterstruktur erfordert eine saubere, konsistente Modellierung bzw. Implementierung der Fachdomäne. In bestehenden Systemen fehlt diese klare Trennung meistens.

4.5.4 Architekturstile verteilter Systeme

Ein verteiltes System zeichnet sich durch Zusammenarbeit von Verarbeitungs- und Speicherbausteinen über Kommunikationsnetze hinweg aus. Daten, Prozesse und auch Benutzer können damit räumlich verteilt voneinander arbeiten.

Die Bausteine verteilter Systeme können über unterschiedliche Mechanismen miteinander kommunizieren: einerseits durch direkten und synchronen Aufruf entfernter Bausteine (remote procedure call, remote method invocation), andererseits durch indirekten und asynchronen Austausch von Nachrichten oder Events (siehe Abschnitt 4.5.5).

Die wesentlichen Ausprägungen verteilter Architekturen sind:

- Client-Server,
- Broker,
- serviceorientierte Architekturen (siehe auch Kapitel 10),
- Peer-to-Peer.

Manche der in Abschnitt 4.5.3 erläuterten hierarchischen Systeme (Layer, Ports-und-Adapter) werden in der Praxis als verteilte Systeme implementiert, d. h., die Kommunikation der Schichten untereinander findet über Netzwerke statt.

4.5.4.1 Client-Server

Client-Server-Systeme bestehen aus mindestens zwei unabhängigen, miteinander kommunizierenden Prozessen:

- Clients benötigen bestimmte Dienste oder Services, die sie bei einem ihnen bekannten Server erfragen.
- Server stellen Clients bestimmte Dienste bereit. Auf entsprechende Anfragen hin erfüllen sie diese Aufgaben und schicken über das jeweilige Netzwerk die Ergebnisse an den aufrufenden Client zurück.

Die Kommunikation zwischen Client und Server erfolgt synchron – ansonsten klassifiziert man diese Systeme als ereignis- oder nachrichtenbasiert (siehe Abschnitt 4.5.5).

Den Vorteilen der hohen Flexibilität und klaren Trennung der Verantwortlichkeiten dieses Architekturmodells stehen Nachteile hinsichtlich Sicherheit, höherem Entwicklungsaufwand und vielfältigen Fehlerquellen gegenüber. Dennoch hat sich dieses Modell in vielen Facetten durchgesetzt – insbesondere in den Schichten (siehe Abschnitt 4.5.3.2 sowie Abschnitt 4.5.3.3).

4.5.4.2 Command Query Responsibility Segregation

Von Oliver Wolf

Command Query Responsibility Segregation (CQRS) ist ein Architekturstil, der aktuell ziemlich in Mode ist und der einige der als gegeben angenommenen Muster der Softwarearchitektur ziemlich auf den Kopf stellt. Dabei ist die Idee, die CQRS zugrunde liegt, schon alt: Schon in den 1980er-Jahren hat Bertrand Meyer, einer der Väter des Design-by-Contract-Paradigmas und OO-Guru, vorgeschlagen, Methoden⁴³ danach zu klassifizieren, ob sie den Zustand von Daten verändern (Commands) oder Daten lediglich lesen, ohne dabei Seiteneffekte zu verursachen (Queries).

Diese Unterscheidung ist unter anderem im Hinblick auf parallele Verarbeitung sinnvoll, denn Queries lassen sich beliebig parallelisiert ausführen, ohne sich gegenseitig zu beeinflussen. Außerdem sind Queries idempotent: Im Fehlerfall können sie problemlos erneut ausgeführt werden und liefern stets das gleiche Ergebnis (sofern nicht zwischenzeitlich durch ein Command Daten modifiziert wurden).

Bei CQRS wird das Prinzip von CQS nun nicht nur auf eine einzelne Klasse, sondern auf ein System als Ganzes übertragen – „CQS in the large“ gewissermaßen. Ein erster Schritt dabei ist es, zunächst einmal äußere Schnittstellen wie Web-Service-Interfaces, die klassisch meistens eine Kombination von Queries und Commands enthalten (getCustomer(), updateCustomer(), ...), aufzuteilen in Command- und Query-Schnittstellen. Diese getrennten Schnittstellen können dann technisch auch getrennt realisiert werden – im Extremfall in verschiedenen Programmiersprachen, auf physisch getrennten Systemen oder sogar in getrennten Netzen. Besondere Vorteile bringt eine solche Trennung oft dann, wenn zur Laufzeit des Systems einer verhältnismäßig geringen Menge von (eher komplexen) Commands eine große Anzahl schnell abzuarbeitender Queries gegenübersteht. In einem solchen Fall könnten die Queries zum Beispiel von vielen leichtgewichtigen Query-Prozessoren in der Cloud bearbeitet werden, während einige wenige leistungsfähige Rechner im Rechenzentrum die Commands übernehmen.

⁴³ Im Sinne der Objektorientierung, d. h. Methoden von Klassen und Objekten

Um zusätzlich Rechenaufwand bei der Verarbeitung von Queries einzusparen, ist es oft außerdem sinnvoll, sich von der Vorstellung eines gemeinsamen Datenbankschemas für Commands und Queries zu lösen und ein spezielles Query-Schema mit bereits voraggregierten oder denormalisierten Daten aufzubauen.

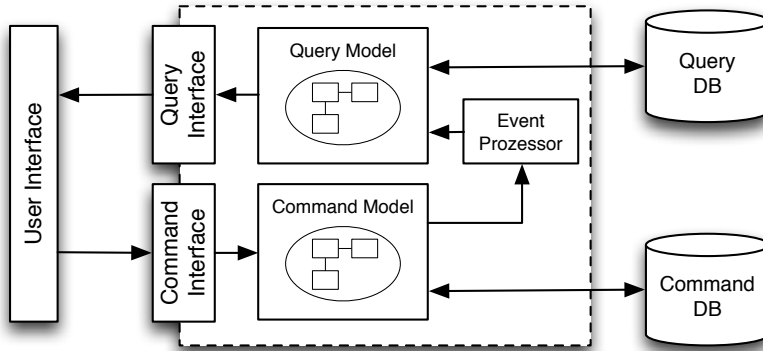


Bild 4.29 CQRS, Command Query Responsibility Segregation

Die Konsistenz der Daten zwischen Query- und Command-Komponenten kann im einfachsten Fall natürlich mit einer gemeinsam genutzten Datenbank (und gegebenenfalls einem Abgleich der Schemas z. B. über Trigger o. Ä.) erreicht werden. Aber auch das ist in vielen Fällen nicht zwingend notwendig, denn oft sind in realen Anwendungssystemen die Anforderungen an die Datenkonsistenz gar nicht so strikt, wie man auf den ersten Blick vermuten würde. Typisches Beispiel: Ein Sachbearbeiter ruft in seiner Anwendung die Maske mit den Kundenstammdaten auf, um eine Adresse zu ändern. Nun klingelt das Telefon und ein Kollege verwickelt ihn in ein Gespräch. Als er endlich anfängt, die neue Adresse einzutragen, sind die Daten auf seinem Bildschirm bereits zehn Minuten alt! In einem solchen System muss bei Schreiboperationen ohnehin immer mit der Möglichkeit von Inkonsistenzen gerechnet und mit Mechanismen wie optimistischem Locking Abhilfe dagegen geschaffen werden. Dementsprechend ist es hier oft auch ausreichend, wenn die Ergebnisse von Commands erst leicht verzögert wirksam und für Queries sichtbar werden („Eventual Consistency“). Für ein CQRS-System hat das eine wichtige Konsequenz – es bedeutet nämlich, dass durchaus auch völlig getrennte und technologisch unterschiedliche Datenbanken für Commands und Queries verwendet werden können. Zur Kommunikation von durch Commands vorgenommenen Datenänderungen dienen dann Events, welche durch einen geeigneten Mechanismus wie z. B. ein Message-Queueing-System vom Command- zum Query-Teil übertragen werden. Die damit mögliche Optimierung der Query-Verarbeitung, z. B. durch Verwendung hochskalierbarer NoSQL-Datenbanken, macht häufig den zusätzlichen Aufwand und die höhere Komplexität mehr als wett.

Ein letzter Schritt, der sich noch stärker von traditionellen Anwendungsarchitekturen entfernt, ist das sogenannte Event Sourcing. Hierbei wird praktisch komplett auf die persistente Speicherung von Zuständen fachlicher Entitäten verzichtet. Stattdessen werden die von Commands erzeugten Events, die von der Erzeugung bis zur eventuellen Löschung einer Entität anfallen, mit Zeitstempeln versehen gespeichert. Der aktuelle Zustand kann dann jederzeit durch Wiedereinspielen der Events ab „Stunde null“ (in der Regel transient) reproduziert werden.

Ein großer Vorteil dabei: Auch der (historische) Zustand zu jedem anderen Zeitpunkt lässt sich so wiederherstellen. Demgegenüber stehen der hohe Speicherbedarf für die Events sowie der Rechenaufwand für die Eventverarbeitung beim Systemanlauf oder beim erstmaligen Zugriff auf eine Entität. Letzterer kann durch regelmäßige Erzeugung von Snapshots (die wiederum als Events aufgefasst werden können) stark reduziert werden.

Oliver Wolf (oliver.wolf@innoq.com) arbeitet als Principal Consultant bei innoQ. Nach langjähriger Erfahrung in verschiedenen Unternehmen als Lead Architect, Technischer Produktmanager, Entwicklungsleiter und Berater für IT-Sicherheit gilt sein aktuelles Interesse der Architektur hochskalierbarer, verteilter Systeme sowie der engen Verzahnung von Entwicklung und Betrieb („DevOps“).

4.5.4.3 Broker

Die klassische CORBA (Common Object Request Broker Architecture)-Architektur hat in den letzten Jahren meiner Einschätzung nach stark an Bedeutung verloren. Trotz ihrer konzeptionellen Klarheit und der potenziell hohen Flexibilität haben sich CORBA-Systeme in der Praxis kaum durchgesetzt. Ich glaube, die Ideen der Ortstransparenz (*location transparency*) und Unabhängigkeit von Programmiersprache und Netzwerktopologie wurden bei CORBA durch „Design-by-Committee“⁴⁴ gründlich sabotiert.

Dennoch möchte ich (und sei es nur aus historischen Gründen, denn mir gefällt die Idee von CORBA ...) die zentralen Begriffe dieses Architekturstils vorstellen (siehe Bild 4.30):

- *Client*: ein Baustein, der einen bestimmten Dienst verwenden möchte, jedoch nur dessen Schnittstellendefinition kennt. Die konkrete Implementierung sowie der Ausführungsort dieses Dienstes sind dem Client unbekannt und egal.
- Ein *Service* kann einen bestimmten Dienst für interessierte Clients erbringen. Technische Details und Ausführungsort der Clients sind dem Service unbekannt und egal.
- Ein Broker koordiniert Aufrufe eines Clients mit angebotenen Diensten eines Service. Er sorgt für die Übergabe aller notwendigen Parameter sowie die Rückgabe der Ergebnisse. Zu seinen Aufgaben gehört es, die konkreten Implementierungen/Ausführungsorte von Services zu finden.
- Stubs (clientseitige Proxies) vermitteln zwischen Client und dem Broker. Stubs verbergen alle technischen Details der Remote-Kommunikation, beispielsweise Serialisierung und Marshalling/Unmarshalling. Im Falle von CORBA übersetzen sie in das Internet-Inter-Orb-Protocol, IIOP.
- Skeletons (serverseitige Proxies) vermitteln zwischen Broker und Service. Wie Stubs kapseln sie Details der technischen Kommunikation. In CORBA übersetzen sie IIOP zurück in die Technologie des jeweiligen Service.
- Bridges sind (optionale) Verbindungen zwischen unterschiedlichen Brokern, die technische Details deren Implementierung überwinden⁴⁵. Sie würden beispielsweise zwischen Brokern unterschiedlicher Anbieter übersetzen oder zwischen .NET-Remoting, Java- und CORBA-Brokern.

⁴⁴ „A camel looks like a horse designed by committee“, erstmals in der Zeitschrift VOGUE im Jahr 1958 beschrieben.

⁴⁵ Diese wundersame Brückenbildung habe ich in der Praxis niemals in realen Systemen erlebt – das scheiterte immer schon an kleinen Beispielen :-). Aber die Idee klingt super, finde ich ...

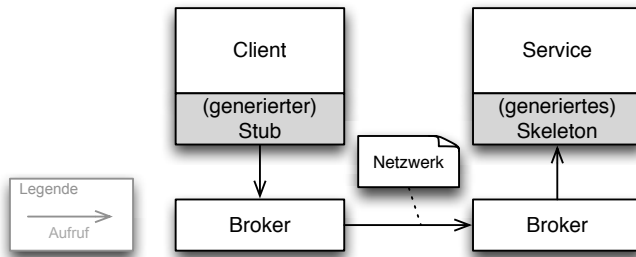


Bild 4.30
(CORBA) Broker-Architekturstil

Der Grundansatz der Broker, das Vermitteln von Nachrichten oder Aufrufen zwischen Aufrufenden und Anbietern, wird in der Praxis oftmals von nachrichten- oder ereignisbasierter Middleware wahrgenommen.

In CORBA-Broker-Architekturen werden die Schnittstellen von Services mit Hilfe einer Interface-Definition-Language (IDL) beschrieben. Aus dieser IDL generiert ein Broker-spezifischer Compiler dann Stubs und Skeletons, die dann jeweils mit Client und Service zusammenarbeiten. Grundsätzlich können mit diesem Ansatz Clients und Services in unterschiedlichen Programmiersprachen entwickelt werden – die notwendigen (technischen) Konvertierungen zwischen Daten- und Objektformaten übernehmen die Stubs und Skeletons.

4.5.4.4 Peer-to-Peer

Peer-to-Peer-Architekturen bestehen aus gleichberechtigten und über Netzwerke verbundenen Komponenten (Peers), die im Netz gleichzeitig die Rolle von Clients und Servern wahrnehmen beziehungsweise sich *Ressourcen* teilen. Ressourcen in diesem Sinne sind etwa CPU-Zeit, Speicherplatz, aber auch Dateien.

Ein wichtiger Einsatzzweck von P2P-Netzen ist die ausfallsichere Datenverteilung,⁴⁶ bekannt durch Netzwerke wie Emule oder Bittorrent. Heute werden auch digitale Telefonate oder Instant-Messaging-Daten über P2P-Netze geleitet (siehe Bild 4.31).

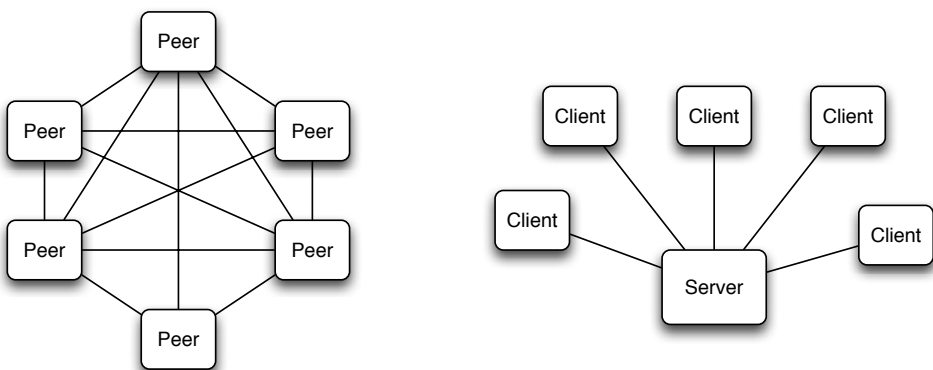


Bild 4.31 Peer-to-Peer versus Client/Server

⁴⁶ So werden beispielsweise umfangreiche Linux-Distributionen häufig durch P2P-Netze weitergegeben.

P2P-Architekturen zeichnen sich durch hohe Ausfallsicherheit aus, weil es keinen *Flaschenhals* für Verfügbarkeit gibt (keinen *single point of failure*): Andererseits sind das Auffinden und Erkennen von Peers in großen Netzen schwierig, ebenso die Fehler- und Ausnahmebehandlung (was geschieht beispielsweise, wenn ein einzelner Peer seine ihm zugewiesene Aufgabe falsch löst?).

Für *kleinere* Architekturen besitzt P2P praktisch keine Bedeutung, sondern es kommt primär im Internet zum Einsatz.

Kriterien für die Anwendung

- Teilnehmer (Peers) können vorhandene Ressourcen (Speicher, Prozessor, Netzbandbreite) teilen – einzelne Teilnehmer müssen daher keine beliebig großen Mengen davon auf Vorrat halten.
- Anbieter umfangreicher Downloads verfügen nicht über ausreichende Serverkapazitäten (und können keine der bekannten Download-Plattformen verwenden).
- Hohe bis höchste Ausfallsicherheit ist gefordert.
- Es besteht keine besondere Anforderung an die Schutzwürdigkeit der verarbeiteten Daten (aufgrund der praktisch nicht kontrollierbaren Laufzeitstruktur wäre deren Sicherheit kaum zu gewährleisten).

Vorteile

- Potenziell höchste Ausfallsicherheit durch hohe Redundanz – sofern eine ausreichend große Zahl von Peers vorhanden ist.
- Peers können sich selbstständig finden – zentrale Administration wird grundsätzlich nicht benötigt. Dieser Vorteil führt jedoch zu einigen Nachteilen – siehe unten.

Nachteile

- Mangels zentraler Administration könnten Daten durch manipulierte Peers verfälscht oder gelöscht werden. Das lässt sich durch (aufwendige) kryptografische Verfahren erschweren.
- Verteilte Kommunikation in Rechnernetzen erfordert Beschäftigung mit den möglichen Fehlerquellen – insbesondere technischen Details der Rechner-Rechner-Kommunikation.

4.5.5 Ereignisbasierte Systeme – Event Systems

Systeme auf Basis von Ereignissen (Events) bestehen grundsätzlich aus zwei Arten von Bausteinen: Ereignisquellen und -senken (*Event-Sources* und *Event-Sinks*). Quellen und Senken kommunizieren in diesem Stil über Ereignisse anstatt im Quellcode verankerte Abhängigkeiten miteinander. Die Kommunikationsbeziehung arbeitet unidirektional von Ereignisquelle zu -senke – diese Beziehungen werden in der Regel erst zur Laufzeit hergestellt. Weil so keine direkten Aufrufe implementiert werden, heißt dieser Stil auch *implicit invocation style*.

4.5.5.1 Ungepufferte Event-Kommunikation

In der *broadcast*-Variante ereignisbasierter Systeme publiziert eine Event-Source Ereignisse auf einem lokalen Netzwerk. Alle verbundenen Empfänger prüfen jedes Ereignis, ob es für den jeweiligen Empfänger interessant ist: Falls ja, wird das Ereignis bearbeitet, falls nein, wird es ignoriert⁴⁷.

Alternativ können sich Empfänger von Ereignissen (*Subscriber*) bei den Sendern (*Publisher*) registrieren. Sender übermitteln Ereignisse in diesem Fall direkt an Subscriber (siehe Bild 4.32).

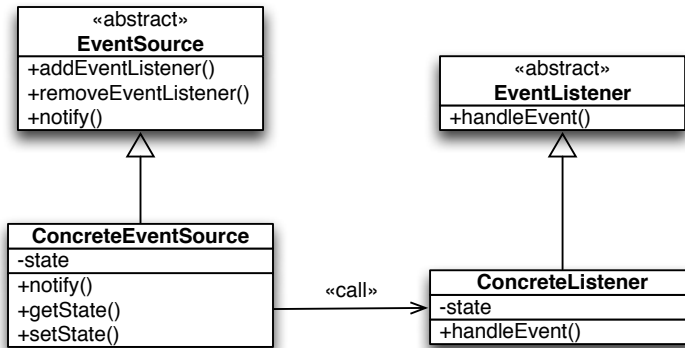


Bild 4.32
Bausteine ungepufferter Event-Kommunikation

4.5.5.2 Message- oder Event-Queue-Architekturen

Die direkte Verbindung zwischen Sendern und Empfängern von Events wird im Architekturstil der „Message- oder Event-Queue-Systeme“ um Puffer (Warteschlangen) ergänzt (siehe Bild 4.33).

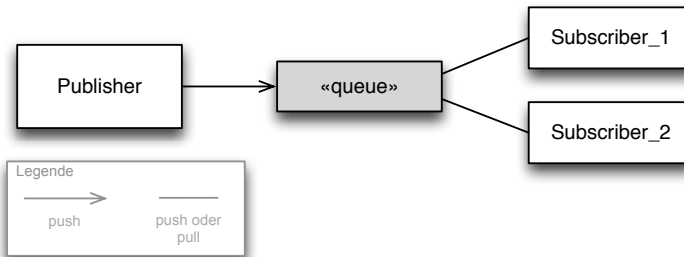


Bild 4.33
Message-Queue:
Puffer für Nachrichten und Events

Ein Erzeuger von Nachrichten schickt diese an die Queue, aus der Empfänger sie entweder synchron oder asynchron abholen. Diese Struktur wird manchmal als *fire-and-forget* bezeichnet, weil Erzeuger keine Bestätigung über Erhalt oder Verarbeitung ihrer Nachrichten erhalten. Insbesondere können Erzeuger keine Annahmen darüber treffen, ob ihre Nachrichten überhaupt bearbeitet werden beziehungsweise in welcher Reihenfolge.

Sind diese Informationen für die Erzeuger wichtig oder müssen die Empfänger gar explizite Antworten auf ihre jeweiligen Nachrichten geben, so bedarf das Message-Queue-Modell einiger Erweiterungen, etwa über explizite Konversationen oder Korrelations-IDs⁴⁸.

⁴⁷ [Hohpe+04] nennt das „publish-subscribe with reactive filtering“.

⁴⁸ Ich bemühe schon wieder [Hohpe+04]: Er bezeichnet diese Techniken als *Conversations* und *Correlations*.

Damit können auch mehrere Nachrichten miteinander in Zusammenhang gebracht werden, so dass der ursprüngliche Erzeuger alle Reaktionen auf eine Nachricht nachvollziehen kann.

4.5.5.3 Message-Service-Architekturen

Wir können die Aufgaben der Warteschlange des vorigen Abschnitts noch um allgemeine Message-Services erweitern – beispielsweise um folgende:

- Routing von Nachrichten an bestimmte Empfänger, etwa *content-based routing*,
- Ändern der Reihenfolge von Nachrichten – solche mit höherer Priorität werden bevorzugt zugestellt,
- Vorverarbeitung von Nachrichten – etwa Archivierung, Verschlüsselung, Protokollierung, statistische oder sonstige Auswertung,
- Zusammenfassung mehrerer Nachrichten oder Aufteilung von Nachrichten,
- Überwachung hinsichtlich bestimmter Qualitätseigenschaften – etwa: Werden die Nachrichten vom Empfänger in der gewünschten Zeit verarbeitet? Ist ein bestimmter Empfänger überlastet und muss eine zusätzliche Instanz gestartet werden?
- Zuverlässige Speicherung von Nachrichten, bis die jeweiligen Empfänger die vollständige Verarbeitung bestätigt haben (*reliable messaging*).

Willkommen in der vielseitigen Welt der *Enterprise Application Integration*, die im Wesentlichen auf dieser Art der entkoppelten Kommunikation basiert. Gregor Hohpe hat mit [Hohpe+04] einen Klassiker zu diesem Thema geschaffen, mit vielen auch heute noch gültigen Patterns.

Kriterien für die Anwendung

- Erzeuger (Sender) von Nachrichten benötigt für die weitere Verarbeitung keine synchrone Antwort (*fire-and-forget*).
- Integrationsszenarien: Es müssen mehrere unterschiedliche Systeme zusammenarbeiten.
- Sender erzeugt Nachrichten erheblich schneller, als sie von Empfängern verarbeitet werden können. Eine Message-Queue als Puffer kann hier ausgleichen.

Vorteile

- Sender und Empfänger von Nachrichten können in völlig unterschiedlichen Technologien und Programmiersprachen erstellt werden.
- Message-Queues, insbesondere solche mit zuverlässiger Zustellung (*reliable messaging*), können die Verfügbarkeit und Robustheit von Systemen erheblich steigern. Aus diesem Grund werden MQ-Systeme insbesondere im Bereich Finanz- und Kontodaten häufig eingesetzt.

Nachteile

- Message-Queues (ob kommerziell oder Open-Source) sind in sich komplexe Systeme mit teilweise hohem Einführungs- und Administrationsaufwand.
- Asynchrone und nachrichtenbasierte Programmierung ist signifikant aufwendiger als einfacher *call-and-return*-Stil. Fehlersuche in asynchronen Systemen kann aufwendig sein.

4.5.6 Interaktionsorientierte Systeme

Viele Informationssysteme, sowohl im Web wie auch auf Desktop- oder Mobilcomputern, basieren auf Interaktion mit ihren (menschlichen) Benutzern. In diesem Abschnitt stelle ich Ihnen zwei der bekanntesten Strukturmuster vor, die solche Mensch-Maschine-Interaktionen adressieren.

4.5.6.1 Model-View-Controller

Teile ein interaktives System mit (grafischer) Bedienoberfläche in Bestandteile auf, die über einen Notifizierungsmechanismus (*change propagation*) ihre gegenseitige Konsistenz sichern:

- Das *Model* enthält die (fachlichen) Daten und Funktionen des Systems – und ist völlig unabhängig von der konkreten UI-Technologie. [Buschmann+07] nennt es den *funktionalen Kern* des Systems.
- Benutzer interagieren über *Controller* mit dem System. *Controller* geben Benutzereingaben zur weiteren Bearbeitung an *Model* oder *View* weiter.
- *View* sorgt für die Anzeige der (Model-)Daten auf der grafischen Oberfläche.

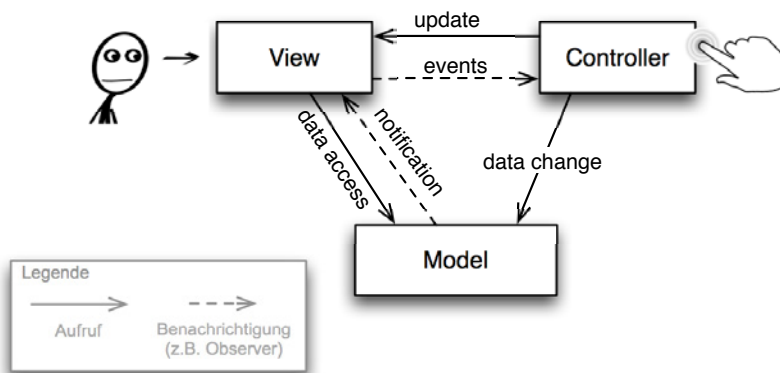


Bild 4.34 Model-View-Controller (Struktur)

Varianten

Bei der Nutzung interaktiver Anwendungen im Web stellt ein Browser die View dar. Aufgrund des eingesetzten HTTP-Protokolls können Model und Controller jedoch die klassischen Notifizierungsmechanismen, beispielsweise über das Observer-Pattern, nicht verwenden. In diesem Fall wird der Controller entweder in einen Client- und Server-Teil zerlegt⁴⁹ bzw. die gesamte Applikation in zwei eigenständige MVC-Triaden. Alternativ bietet sich das Presentation Model an – siehe den folgenden Abschnitt.

Umsetzung in der Praxis

Egal, in welchem technischen Umfeld Sie Systeme implementieren, ob Web oder Desktop: Wahrscheinlich werden Sie die Details des MVC-Musters durch Ihre konkrete UI-Technologie

⁴⁹ Siehe beispielsweise <http://welcome.totheinter.net/2008/08/05/mvc-pattern-for-the-web/>

vorgegeben bekommen. Ob Swing, JavaFX, ASP.NET, iOS, Android oder die vielen Web-UI-Frameworks, praktisch alle⁵⁰ implementieren Varianten von MVC und stellen Ihnen als Entwickler dann eine MVC-Infrastruktur bereit. Sie müssen sich nur noch ins gemachte Nest setzen oder mit den Restriktionen dieser Frameworks leben.

Siehe auch Abschnitt 4.5.3.3 (Ports- und Adapter) bezüglich der strikten Kapselung des (Domänen-)Model.

Vorteile

- Klare Trennung von Verantwortlichkeiten,
- erlaubt mehrere Views pro Model – durchaus auch in unterschiedlichen Technologien,
- reichhaltige Auswahl an passenden Frameworks.

Nachteile

- Implementierung und insbesondere Debugging von Applikationen mit Notifikationen können aufwendig sein (verwenden Sie daher ausgereifte MVC-Frameworks!).
- Notifikationen von Modellen mit mehreren Controller-View-Paaren kann zeitaufwendig sein, so dass Systeme „langsam“ wirken können. Das kann insbesondere geschehen, wenn auch einzelne grafische Elemente (Widgets) als eigenständige MVC-Triaden implementiert sind.

4.5.6.2 Presentation Model

Wie auch das MVC-Muster trennt das *Presentation Model* die reine Ansicht (View) von der Behandlung von Benutzereingaben (Controller) und den dargestellten Daten (Model). Zwischen dem View-Controller-Duo und dem eigentlichen Model entkoppelt das Presentation Model in zwei Richtungen (siehe Bild 4.35):

1. Das Presentation Model repräsentiert Daten und Verhalten einer grafischen View, ohne Bezug auf eine konkrete UI-Implementierungstechnologie zu nehmen. Beispielsweise könnte es die folgenden Zustände einer Checkbox als boolesche Werte enthalten:
 - aktiv oder inaktiv (enabled/disabled),
 - ein- oder ausgeschaltet,
 - sichtbar oder unsichtbar.
2. Das Presentation Model übernimmt die Benachrichtigungen der View über Datenänderungen. Es koordiniert sich mit dem eigentlichen (Domain-)Model.

Das Presentation Model stellt eine Abstraktion einer grafischen Darstellung dar, ohne selbst Abhängigkeiten von konkreten UI-Technologien zu haben. Es enthält sämtliche Zustandsinformationen, die sich durch Benutzeraktionen ändern könnten.

Es trifft sämtliche Entscheidungen, welche Informationen wie angezeigt werden – was die Implementierung der View selbst stark vereinfacht! Das wiederum hat zur Folge, dass automatisierte Tests sich im Wesentlichen auf das Presentation Model konzentrieren können (und damit die unsäglich schwierigen UI-Tests drastisch vereinfacht werden können!).

⁵⁰ Ausnahmen bestätigen diese Regel – beispielsweise setzt das Open-Dolphin-Framework auf das Presentation-Model-Muster (siehe Abschnitt 4.5.6.2).

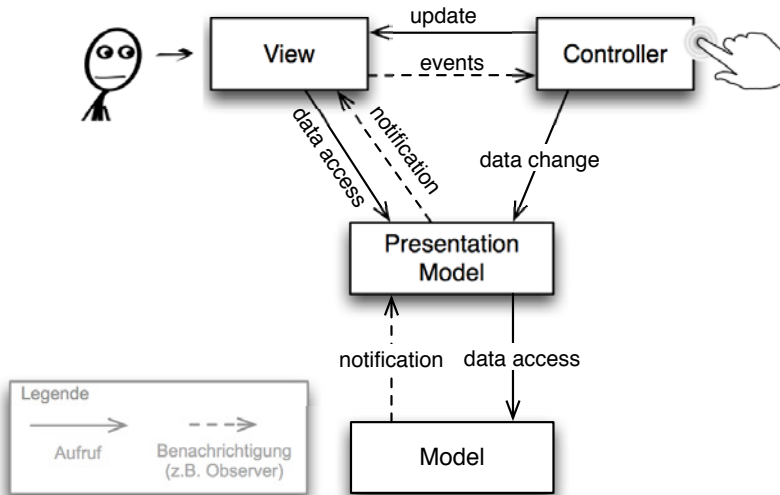


Bild 4.35 Presentation Model (Struktur)

Das Presentation Model kommt primär in verteilten Client/Server-Systemen zum Einsatz und wird dabei zwischen Client und Server geteilt (siehe Bilder 4.35 und 4.36). Auf Client-Seite ist es über Notifikationen mit der View verbunden – diese Abhängigkeit sollte durch ein Framework bereitgestellt werden. Die in den Abbildungen gezeigte Notifikation zwischen Model und Presentation Model ist optional.

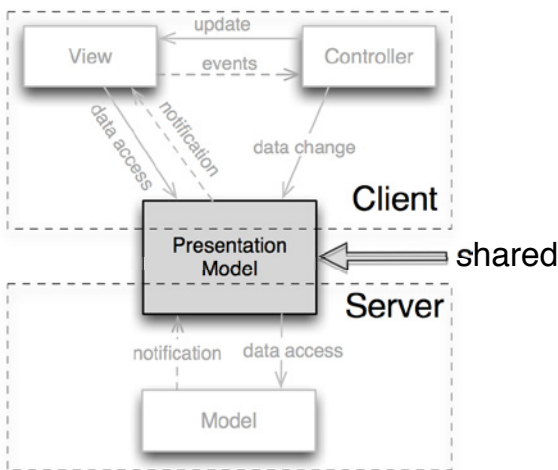


Bild 4.36
Presentation Model geteilt zwischen Client und Server

Kriterien für die Anwendung

- Falls Sie das Verhalten Ihrer grafischen Oberfläche ausgiebig automatisiert testen wollen, so kann Ihnen das Presentation Model helfen: Da es den gesamten Zustand der Oberfläche hält und koordiniert, genügt es oftmals, das Presentation Model zu testen.

- Sie möchten eine „Rich Internet Application“ entwickeln, die aus Masken mit vielfältigen Benutzereingaben sowie möglichst unmittelbaren Validierungen besteht. Statt den View-Code mit diesen Aufgaben zu belasten, können Sie das Presentation Model gut dafür verwenden.

Vorteile

- Das eigentliche Model kann ein reines Domänen-Model bleiben (POJO oder POCO)⁵¹ – ist damit einfacher und i. d. R. leichter testbar.
- Die View, die eigentliche Darstellung, bleibt frei von fachlicher Logik oder Prüfungen, weil diese komplett im *Presentation Model* liegen.

Nachteile

- Sie benötigen einen Synchronisations- oder Notifikationsmechanismus zwischen *Presentation Model* und View (den Sie allerdings bei interaktiven grafischen Systemen praktisch immer brauchen).

Quellen und weitere Informationen

- Das quelloffene Java UI-Framework openDolphin (<http://open-dolphin.org>) basiert auf dem *Presentation-Model*-Muster – es adressiert grafische Benutzerschnittstellen für Unternehmensanwendungen.
- Martin Fowler hat dieses Architekturmuster unter <http://martinfowler.com/eaDev/PresentationModel.html> beschrieben. Er bezieht sich dabei auf Vorlagen von VisualWorks-Smalltalk.

Andere Muster für interaktive Systeme

Martin Fowler erläutert⁵² noch einige weitere Muster für interaktive Systeme:

Supervising Controller (<i>Presenter</i>)	Wie bei MVC erfolgt auch hier eine Dreiteilung in Daten (Model), Anzeige (View) und Eingabeverarbeitung/Koordination (Presenter). Im Gegensatz zu MVC hat die View keine Beziehung zum Presenter. Benutzereingaben werden von UI-Elementen (Widgets) unmittelbar zur Verarbeitung an den Presenter delegiert ⁵³ .
Passive View	Im Gegensatz zu MVC ist der View-Teil hier völlig passiv, d. h., er kümmert sich nicht um die Aktualisierung von angezeigten Daten aus dem Model. Diese Aufgabe obliegt hier dem Controller – der damit zum wesentlichen Gegenstand von Tests wird – weil im View praktisch „nichts Schlimmes“ passieren kann.

⁵¹ Plain Old {Java | C#} Object

⁵² Auf <http://martinfowler.com/eaDev> beziehungsweise [Fowler02]

⁵³ Diese Struktur ist auch als „Model-View-Presenter“ bekannt, siehe z. B. <http://de.slideshare.net/esug/twisting-the-triad>.

4.5.7 Weitere Architekturstile und -muster

Eine Vielzahl weiterer Architekturmuster könnte Ihnen sicherlich in manchen Fällen die Entwurfstätigkeit erleichtern. Die Bücher „Pattern Languages of Program Design“ (PloP) enthalten dazu viele Anregungen, insbesondere die „Pattern-Language for Distributed Computing“ [Buschmann+07]. Weiterhin bietet Martin Fowlers „Patterns of Enterprise Application Architecture“ [Fowler02] einen großen Fundus an Strukturvorschlägen.

REST-Architektur

Von Stefan Tilkov

In seiner berühmten Dissertation hat der Apache-Mitgründer und HTTP-Miterfinder Roy Fielding diesen Stil als die „Architektur des Web“ bezeichnet⁵⁴. Dazu hat er im Nachhinein die Konzepte beschrieben, die Grundlage der Standardisierung der Webprotokollen (vor allem HTTP und URI) waren. Rein formal ist damit das Web als Ganzes die einzig relevante Instanz dieses Stils! In der Praxis interessiert man sich für diese Unterscheidung jedoch herzlich wenig und benutzt die Bezeichnung „RESTful“ für konkrete Architekturen, die Webtechnologien so benutzen, wie es die Prinzipien des Architekturstils vorschreiben.

REST steht für *representational state transfer* und wird durch folgende Regeln definiert:

- REST impliziert eine Client/Server-Architektur. Der Server ist dabei eine Webanwendung; der Client kann entweder ein Webbrowser sein oder eine andere Art *user agent*, sei es ein generischer oder ein von Ihnen entwickeltes Programm.
- Der Server stellt seine Fähigkeiten nach außen nicht als eine Menge von Diensten und/oder Operationen, sondern als *Ressourcen* zur Verfügung, die über einen einheitlichen, standardisierten Mechanismus (eine URI) identifiziert werden. Das bedeutet, dass in einem REST-System nicht nur Ihre Anwendungen oder Ihre Dienste eine URI bekommen, sondern alle fachlichen Konzepte, die Ihnen wichtig sind⁵⁵.
- Die Kommunikation des Clients mit den Ressourcen erfolgt mit Hilfe von Standardmethoden (im Web im Wesentlichen GET, PUT, POST, DELETE) und nicht mit anwendungsspezifischen Operationen. Anders formuliert: Sie benutzen das Standard-Applikationsprotokoll HTTP und erfinden nicht Ihr eigenes.
- Auf Basis der standardisierten Semantik der Operationen können zwischen Client und Server intelligente, generische Zwischenstationen (*intermediaries*) geschaltet werden, die nützliche Dinge tun. Das beste Beispiel dafür sind *Caches*, die anhand von Metadaten, die als Teil von HTTP-Antworten vom Server zurückgeliefert werden, eine Vielzahl von Requests vermeiden können.
- Die Kommunikation erfolgt mit Hilfe von *Repräsentationen* der Ressourcen. Davon kann es für jede Ressource mehr als eine geben, z. B. eine JSON-, eine XML- und eine HTML-Variante.

⁵⁴ Ich ziehe durch diese Fußnote wahrscheinlich die Missgunst aller RESTafarians auf mich – jedoch halte ich REST primär für einen Kommunikationsstil – und eine Untermenge der hierarchischen oder verteilten Systeme. Trotz dieser kleinen begrifflichen Meinungsverschiedenheit: REST ist für moderne, interaktive Webanwendungen zentral – und jeder Webentwickler sollte seine Grundzüge kennen!

⁵⁵ Woher auch rührt, dass Ansätze wie SOAP/WSDL/WS-*–Webservices oder Frameworks wie JSF von REST-Verfechtern nur als Negativbeispiel eingesetzt werden.

- In einer „echten“ REST-Architektur wird darüber hinaus der Kontrollfluss über *Hypermedia* gesteuert. Dahinter verbirgt sich trotz des hochtrabenden Namens nicht mehr als ein aus interaktiven HTML-Anwendungen bekanntes Konzept: Der Server teilt dem Client in der Antwort auf seine Anfragen jeweils über *Hypermedia-Controls* (Links, Formulare usw.) mit, welche Interaktionen als Nächstes möglich sind. Der ideale REST-Client benötigt daher nur eine einzige URI als Einstieg und erfährt von allen anderen Ressourcen nur dynamisch.

Ganz ähnlich wie eine relationale Datenbank dann am besten funktioniert, wenn Sie sie so benutzen, wie es sich die Erfinder der relationalen Algebra gedacht haben, können Webtechnologien ihre Stärken am besten ausspielen, wenn sie REST-Prinzipien folgen. Vor einigen Jahren war REST noch ein relativ exotischer Ansatz einiger Außenseiter, es hat sich in den letzten Jahren aber zu einer ernst zu nehmenden Alternative zu SOAP/WSDL-basierten Architekturen, im Webumfeld gar zur Standardlösung entwickelt.

Varianten

- Wie die Formulierung „echte REST-Architektur“ weiter oben schon andeutet, gibt es Varianten, die nicht alle Regeln befolgen. In den meisten populären REST-Schnittstellen wird der Hypermedia-Aspekt ignoriert. (Es gibt einige Reifegradmodelle, die auf diese Unterscheidung näher eingehen; vom Erfinder des Konzepts kommen jedoch klare Worte: „REST-APIs must be hypertext-driven“.) In solchen Systemen listet die Dokumentation eine Menge von URI-Mustern mit Parametern auf; es ist Aufgabe des Aufrufers, diese mit Werten zu füllen und dann entsprechend der Dokumentation mit den korrekten HTTP-Verben aufzurufen.
- REST wird in der Regel als Architekturstil für Services betrachtet, ist aber mindestens genauso sinnvoll für Webanwendungen, bei denen der Aufrufer ein von einem Menschen bedienter Webbrowser ist. Dazu wird die Oberfläche moderner Webanwendungen entweder vollständig in JavaScript realisiert und HTML auf dem Client erzeugt (sog. Single Page Applications, SPAs) oder ein klassischerer Ansatz gewählt, bei dem HTML auf dem Server erzeugt und JavaScript optional und nur für die Verbesserung der Ergonomie verwendet wird (Resource-oriented Client Architecture, ROCA). In beiden Varianten kann der Server nach REST-Prinzipien realisiert werden.

Kriterien für die Anwendung

- Die Interoperabilität, also die Möglichkeit, aus bekannten und unbekannten Ökosystemen auf ein REST-System zuzugreifen, ist größer als bei praktisch jedem anderen Ansatz, schließlich ist nichts weiter verbreitet als Webtechnologien. Dass Sie ein Web-API nach REST-Prinzipien umsetzen sollten, versteht sich fast von selbst.
- REST-Architekturen sind auch im unternehmensinternen Integrationsumfeld besonders gut geeignet, wenn Sie sich nicht von einzelnen Herstellern oder Produkten abhängig machen wollen.

Vorteile

- Sie können die weit verbreiteten und ausgereiften Werkzeuge und Bibliotheken, die fürs Web zum Einsatz kommen, optimal ausnutzen.
- Durch die statuslose Kommunikation skalieren REST-Systeme besonders gut.

- Die Kopplung zwischen den Kommunikationspartnern in einem REST-Setup ist besonders gering, die Möglichkeiten zu einer Evolution von Schnittstellen sind in besonderem Maße auf Langlebigkeit ausgelegt.

Nachteile

- Wie immer sind lose Kopplung, Standardisierung und Skalierbarkeit nicht umsonst. So müssen Sie sich auch beim Einsatz von RESTful HTTP bewusst sein, dass Sie Nachrichten über das Netz senden und keine lokalen Methoden aufrufen, und auch die Indirektion über Hypermedia kostet etwas.

Quellen und weitere Informationen

Die wesentliche Grundlage für REST legt Roy Fielding in seiner Dissertation⁵⁶. Ansonsten stellt [Tilkov-11] den Zusammenhang zwischen REST, HTTP und deren praktischer Anwendung dar. Falls Sie zu HTTP noch mehr wissen möchten – dessen Spezifikation sagt alles: RFC 2616: Hypertext Transfer Protocol – HTTP/1.1, <http://www.ietf.org/rfc/rfc2616.txt>.

Stefan Tilkov (stefan.tilkov@innoq.com) ist Geschäftsführer und Principal Consultant der innoQ Deutschland GmbH, wo er sich vorwiegend mit der strategischen Beratung im Umfeld von Softwarearchitekturen beschäftigt. Er ist Autor des Buchs „REST und HTTP“ (dpunkt Verlag), Autor zahlreicher Fachartikel und häufiger Sprecher auf internationalen Konferenzen.

■ 4.6 Entwurfsmuster

Gang-of-Four

Entwurfsmuster beschreiben einfache und elegante Lösungen für häufige Entwurfsprobleme. Die in Softwarekreisen mittlerweile berühmte „Gang-of-Four“⁵⁷ (GoF) legte mit [Gamma95] den Grundstein zu diesem interessanten und in der Praxis sehr wichtigen Gebiet des Softwareentwurfs. Ich habe hier einige Muster zusammengestellt, die auch über den Entwurf von Klassenstrukturen hinaus Bedeutung besitzen. Insbesondere können sie Ihnen dabei helfen, Abhängigkeiten in Entwürfen zu reduzieren.

4.6.1 Entwurf mit Mustern

Muster vereinfachen Kommunikation

Entwurfsmuster helfen Ihnen als Softwarearchitekt sowohl bei der eigentlichen Entwurfstätigkeit als auch bei der Kommunikation mit Designern und Entwicklern.

⁵⁶ Fielding, Roy Thomas: Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine, 2000, <http://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>

⁵⁷ Erich Gamma, Rich Helm, Ralph Johnson, John Vlissides

Weil viele wichtige Muster von ihren Autoren sehr präzise dokumentiert wurden, vermeiden sie Missverständnisse bei der Diskussion von Entwürfen. Mit ihrer Hilfe lässt sich ein gemeinsames Verständnis von Entwürfen auf hohen Abstraktionsebenen leichter herstellen. Die prägnanten Bezeichnungen von Mustern erlauben es, auch komplexe Strukturen in vergleichsweise wenigen Worten präzise zu beschreiben. Das erleichtert es Teams, die „Vision“ eines Entwurfs zu entwickeln und prägnant zu beschreiben.

Muster können Ihnen helfen, sich bei der Dokumentation Ihrer Entwürfe auf das Wesentliche zu konzentrieren, nämlich die Besonderheiten Ihrer konkreten Aufgabe. Wenn Sie Muster verwenden, können Sie unter Umständen mit fünf Seiten Entwurfsdokumentation auskommen, wo Sie bislang 50 Seiten benötigten.



Abschließend aber eine Mahnung zur Vorsicht: Setzen Sie Muster bedarfsgerecht ein. Muster machen Ihre Entwürfe oft komplexer, indem sie zusätzliche Abstraktionen oder Komponenten einführen. Schlichtheit und Verständlichkeit können jedoch wichtiger sein als hohe Flexibilität. Wählen Sie im Zweifelsfall die schlichte Alternative (und dokumentieren Sie diese Entwurfsentscheidung).

4.6.2 Adapter

[Ein Adapter] passt die Schnittstelle einer Klasse an eine andere von ihren Klienten erwartete Schnittstelle an. Das Adaptermuster lässt Klassen zusammenarbeiten, die ansonsten dazu nicht in der Lage wären.

[Gamma95]

Wenn Sie DIES brauchen, aber DAS haben, kann Ihnen das Adaptermuster helfen. Genauer gesagt: Ein Adapter soll die Schnittstelle eines existierenden Moduls ändern.



Setzen Sie dieses Muster ein, wenn Sie ein existierendes Modul verwenden wollen, dessen Schnittstelle nicht mit der von Ihnen benötigten Schnittstelle übereinstimmt.

Bild 4.37 zeigt das Adaptermuster. Sie erkennen darin eine vorhandene Klasse, die DAS anbietet, und einen Client, der DIES braucht.

Das Adaptermuster benötigen Sie auch, wenn Sie bestehende Anwendungssysteme kapseln. Somit ist es ein wichtiges Werkzeug für die Integration und das Umwickeln (*Wrapping*) von Altsystemen (siehe Abschnitt 7.3 bzw. 7.4).

[Gamma95] beschreibt das Muster im Detail und schlägt verschiedene Erweiterungen vor, etwa den Zwei-Wege-Adapter oder den steckbaren Adapter.

Adapter unterstützt Kapselung

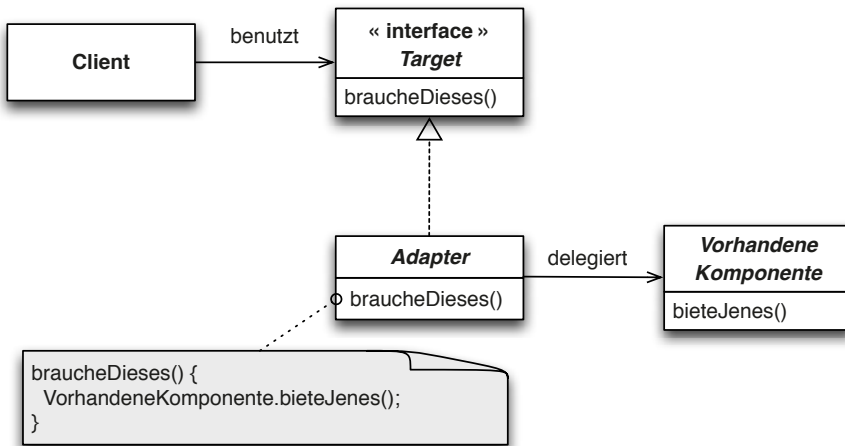


Bild 4.37 Entwurfsmuster „Adapter“

4.6.3 Beobachter (Observer)

Ein Observer definiert eine kontrollierte Abhängigkeit zwischen Objekten, sodass die Änderung eines Objekts die Benachrichtigung und Aktualisierung aller abhängigen Objekte auslöst.



Verwenden Sie Observer, wenn eine Komponente in der Lage sein soll, andere Komponenten zu benachrichtigen, ohne zu wissen,

- wer die anderen Komponenten sind;
- wie viele Komponenten geändert werden müssen (nach [Gamma95]).

Beobachter reagiert auf Zustandsänderung

Es kommt vor, dass eine Komponente (= ein Beobachter) auf eine Zustandsänderung einer anderen Komponente (= das konkrete Subjekt) reagieren soll, ohne dass das Subjekt den Beobachter kennt. Ein

Subjekt kann eine beliebige Zahl von Beobachtern besitzen. Verändert das Subjekt seinen Zustand, so werden alle diese Beobachter benachrichtigt. Als Reaktion darauf synchronisiert sich jeder Beobachter mit dem Subjekt, indem er dessen Zustand erfragt.

In Bild 4.38 verwaltet das Subjekt seine Beobachter. Es bietet ihnen eine Schnittstelle zum dynamischen An- und Abmelden. Konkrete Beobachter kennen ihre beobachteten (konkreten) Subjekte.

Es ist interessant, das dynamische Verhalten des Beobachtermusters zu betrachten (siehe Bild 4.39).

Das Beobachtermuster ist die Grundlage des Model-View-Controller-Musters, das häufig bei Entwurf und Implementierung von Benutzeroberflächen eingesetzt wird (siehe Abschnitt 4.5.6.1).

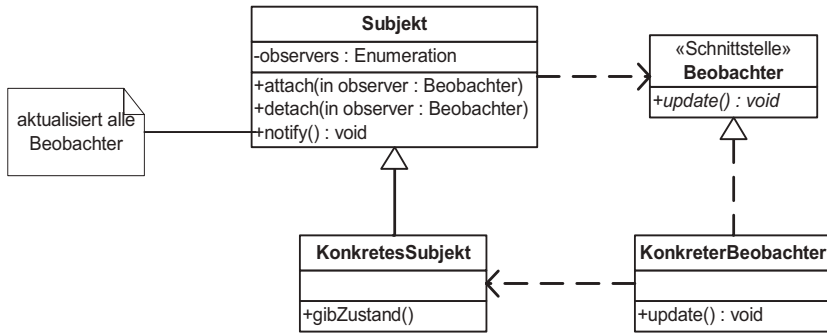
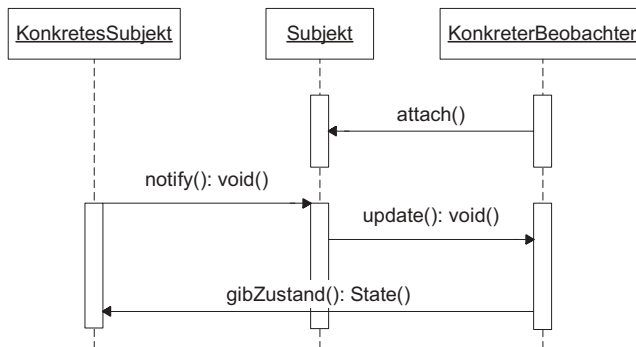


Bild 4.38 Entwurfsmuster „Beobachter“

Bild 4.39
Dynamisches Verhalten des
Beobachtermusters

4.6.4 Dekorierer (Decorator)

Ein Dekorierer erweitert eine Komponente dynamisch um Funktionalität.



Verwenden Sie das Dekorierermuster, um einer Komponente dynamisch und transparent Funktionalität hinzuzufügen, ohne die Komponente selbst zu erweitern (nach [Gamma95]).

Im Gegensatz zum Adapter (Abschnitt 4.6.2) verändert ein Dekorierer Funktionalität, nicht aber die Schnittstelle einer Komponente. Ein Adapter liefert hingegen eine neue Schnittstelle. Wie der Adapter besitzt auch das Dekorierer-Muster Bedeutung für die Integration, Kapselung und Weiterbenutzung bestehender Anwendungssysteme (siehe Bild 4.40).

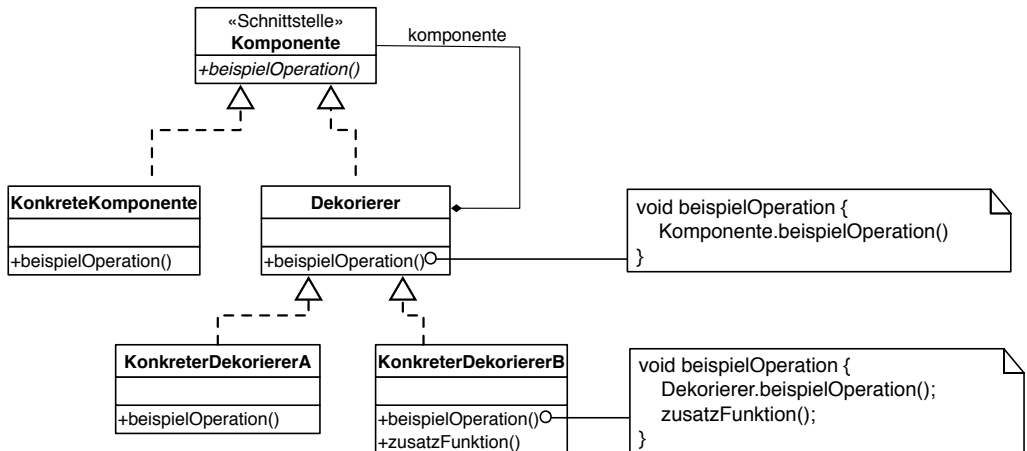


Bild 4.40 Entwurfsmuster „Dekorierer“

4.6.5 Stellvertreter (Proxy)

Ein Stellvertreter (Proxy) stellt einen Platzhalter für eine andere Komponente (Subjekt) dar und kontrolliert den Zugang zum echten Subjekt.

Die Schnittstelle des Stellvertreters ist identisch mit der Schnittstelle des echten Subjekts, sodass der Stellvertreter als Ersatz des Subjekts auftreten kann.



Sie können das Stellvertretermuster einsetzen, wenn Sie den Zugriff auf das echte Subjekt kontrollieren wollen. Der Proxy kann das echte Subjekt erzeugen und löschen.

Darüber hinaus kann er weitere Verantwortlichkeiten besitzen:

- Entfernte Stellvertreter (*remote proxies*) sind verantwortlich für die Abwicklung von Anfragen an das entfernte Subjekt (das möglicherweise auf einem anderen Rechner existiert).
- Virtuelle Stellvertreter (*virtual proxies*) können Informationen des echten Subjekts enthalten, um den Zugriff auf das echte Subjekt zu verzögern. Dies ist nützlich, wenn der Zugriff auf das echte Subjekt langsam oder teuer ist, wenn dazu etwa Datenbankoperationen notwendig werden.
- Schützende Stellvertreter (*protection proxies*) prüfen, ob Zugreifer auf das echte Subjekt über die notwendigen Rechte verfügen.

Bild 4.41 zeigt die Struktur des Stellvertretermusters. Zur Laufzeit leitet ein Proxy Anfragen erst im Bedarfsfall an das echte Subjekt weiter.

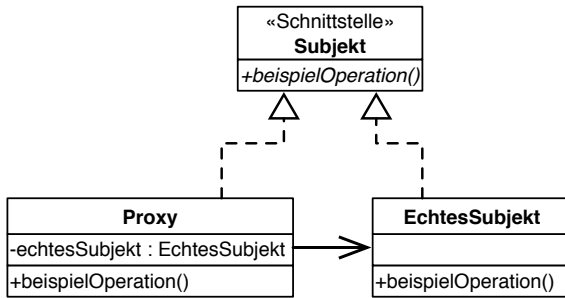


Bild 4.41
Entwurfsmuster „Stellvertreter
(Proxy)“

4.6.6 Fassade

Eine Fassade „bietet eine einheitliche Schnittstelle zu einer Menge von Schnittstellen eines Subsystems“.

[Gamma95]

Das Entwurfsmuster Fassade bietet eine weitere Möglichkeit, Abhängigkeiten zwischen unterschiedlichen Systemkomponenten zu reduzieren. Betrachten Sie dazu Bild 4.42. Es illustriert, wie die Fassade die internen Komponenten des Subsystems nach außen verbirgt.

Fassade reduziert
Abhängigkeiten

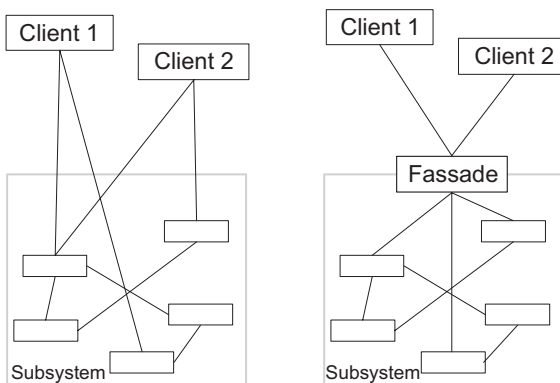


Bild 4.42
Entwurfsmuster „Fassade“



Verwenden Sie das Fassademuster, wenn Sie

- zu einem komplexen Subsystem eine einfache Schnittstelle anbieten wollen;
- Klienten eine vereinfachte Sicht auf das Subsystem anbieten wollen (Klienten brauchen nicht hinter die Fassade zu schauen!);
- ein System in Schichten aufteilen wollen und einen (definierten) Eintrittspunkt in jede Schicht benötigen.

Eine Fassade kennt die internen Details des Subsystems. Sie muss wissen, welche der internen Komponenten für eine bestimmte Anfrage oder einen Aufruf zuständig ist. Sie delegiert Anfragen von Clients an die jeweils zuständigen Subsystemkomponenten (nach [Gamma95]). Eine Spezialisierung des Fassademusters ist die Wrapper-Fassade, die sich auf die Kapselung von Daten und Funktionen nichtobjektorientierter Programmierschnittstellen bezieht. Siehe [Schmidt2000].

4.6.7 Zustand (State)

Das Zustandsmuster „ermöglicht einem Objekt, sein Verhalten zu ändern, wenn sein interner Zustand sich ändert. Es wird so aussehen, als ob das Objekt seine Klasse gewechselt hat“.

[Gamma95]

Es kommt häufig vor, dass Komponenten abhängig von ihrem Zustand ein spezielles Verhalten zeigen sollen. In der UML modellieren Sie das in Zustands- oder Zustandsübergangsdiagrammen. In einem bestimmten Zustand darf eine Komponente dann nur bestimmte Operationen ausführen.

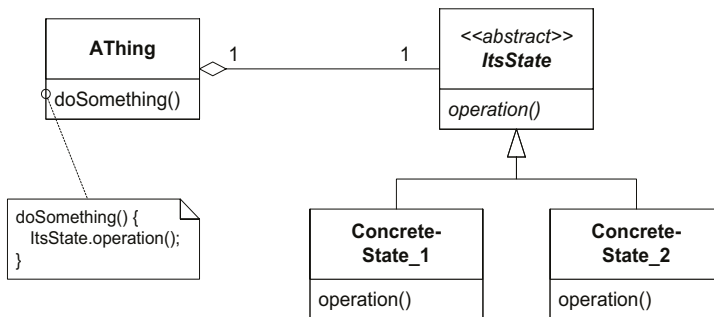


Bild 4.43
Entwurfsmuster
„Zustand“
(nach [Gamma95])

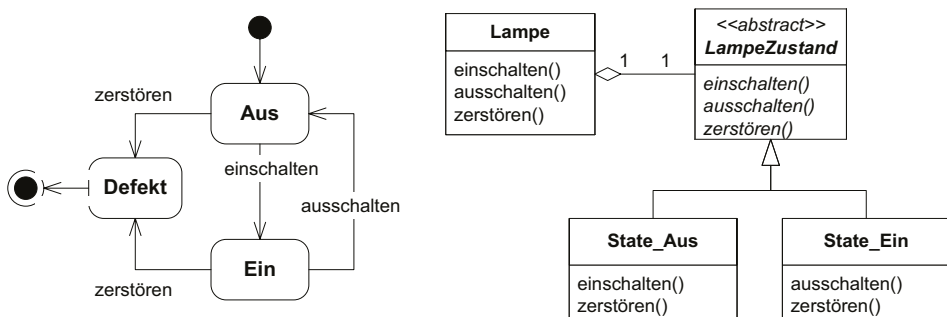


Bild 4.44 Beispiel für Zustandsmuster einer Lampe

Das Zustandsmuster löst dieses Problem über Polymorphismus: Die in einem bestimmten Zustand möglichen Operationen werden durch eine (konkrete) Unterklasse implementiert, die bei Zustandsübergängen bei Bedarf ausgetauscht wird. Zur Laufzeit wird die jeweilige Operation vom (abstrakten) Zustand (in Bild 4.43 heißt er *ItsState*) über Polymorphismus durch den jeweilig instanziierten konkreten Zustand realisiert.

Eine leichte Variation dieses Musters, zusammen mit dem zugehörigen Zustandsdiagramm, finden Sie in Bild 4.44. Die konkreten Zustandsklassen implementieren sämtliche für ihren eigenen Zustand benötigten Operationen

■ 4.7 Weiterführende Literatur

Obwohl das Thema Softwareentwurf zu den wichtigsten Fundamenten der Informationstechnik zählt, gibt es nur relativ wenig Literatur, die systematisch die Grundlagen dieses Gebiets darstellt.



Zu Grundlagen des Entwurfs

[Martin96] beschreibt die Grundlagen des objektorientierten Entwurfs. Seine Beispiele illustriert Robert Martin an der Sprache C++, die Entwurfsprinzipien gelten jedoch für sämtliche objektorientierten Sprachen, teilweise sind sie sogar unabhängig von der Objektorientierung.

[Martin2000] stellt neben einigen der auch hier beschriebenen Entwurfsprinzipien noch weitere vor, etwa einige Prinzipien zur Aufteilung von Klassen auf Pakete.

[Evans04] sowie [Avram+06] (Letzteres auch online verfügbar) stellen *Domain-Driven Design* im Detail vor. Mein Tipp für ernsthafte DDD-Aspiranten: [Vernon-13], allerdings benötigen Sie eine Menge Zeit für dieses monumentale Stück Weisheit.

Domain-Driven Design

[Siedersleben04] erläutert den Entwurf von Komponenten anhand der Trennung in technische und anwendungsbezogene Teile. Seine Darstellung des Themas ist sowohl gründlich als auch praxisbezogen.

Zu speziellen Aspekten des Entwurfs

[Fowler99] erläutert die Methode des Refactoring. Sie stellt einen systematischen Umbau von Programmen dar und ermöglicht es, den Entwurf und die Implementierung bestehender Systeme zu verbessern.

Zu Architekturmustern und Referenzarchitekturen

[Buschmann+96] benutzen einige der grundlegenden Muster von [Gamma95] als Grundlage erweiterter oder umfangreicherer Architekturmuster. So beschreiben sie ausführlich Schichten (*Layer*), Pipes-and-Filter, Model-View-Controller sowie den Befehlsverarbeiter (*Command Processor*).

[Fowler02] geht Muster von der pragmatischen Seite an: Fowler zeigt Lösungsmuster für typische Problemstellungen kommerzieller Informationssysteme auf. Der Titel des Buchs (*Patterns for Enterprise Application Architecture*) ist meiner Meinung nach jedoch irreführend: Im Fokus des Buchs liegen eindeutig Entwurfs- und Programmiermuster – Architekturmuster im Stil von [Buschmann+96] suchen Sie dort vergeblich. Allerdings ist seine Systematik unmittelbar anwendbar.

[Buschmann+07] ist der vierte Band der POSA-Serie (Pattern Oriented Software Architecture). Sehr lesenswerte Zusammenfassung des Vorgehens beim Entwurf, dem systematischen Übergang von *mud to structure*. Wenn Sie nur ein einziges Buch zu Mustern lesen möchten, liegen Sie damit richtig!

Zu Entwurfsmustern

Eine generelle Empfehlung zu Entwurfsmustern sind die Beiträge zu den Konferenzen über Entwurfsmuster (*Pattern Languages of Programs*, PloP, ChiliPloP) und deren europäische Schwesterkonferenz, EuroPloP. Die meisten Beiträge sind online verfügbar, <http://www.hillside.net> ist die Heimat der Pattern-Gemeinde im Internet.

[Gamma95] ist Pflichtlektüre für alle, die (objektorientierten) Softwareentwurf betreiben.

[Eilebrecht+19] gibt einen kompakten Überblick über Muster.

Kommunikation und Dokumentation von Architekturen

*I am more and more convinced every day that communication is, in fact,
what makes or breaks software projects.*

Gojko Adzic (in: Bridging the Communication Gap)



Fragen, die dieses Kapitel beantwortet:

- Was müssen Architekten kommunizieren und dokumentieren?
- Was ist eine Sicht auf eine Architektur?
- Welche nützlichen Architektursichten gibt es?
- Wie entwerfen Sie die Sichten?
- Was gehört neben Sichten noch zur Architekturdokumentation?
- Welche Anforderungen muss eine Architekturdokumentation erfüllen?
- Was sollten Sie bei der Dokumentation von Architekturen beachten?

Softwarearchitekten müssen im Rahmen ihrer Kommunikations- und Dokumentationsaufgabe den übrigen Stakeholdern des Systems die Architektur erklären. Dazu gehören die Strukturen, Entscheidungen und Konzepte, deren Begründungen sowie Vor- und Nachteile. Manchmal müssen Softwarearchitekten all dies gegen Widerstände verteidigen oder die Architektur regelrecht vermarkten. Dazu bedarf es einer ausgewogenen Mischung aus Fingerspitzengefühl und Durchsetzungsvermögen, gepaart mit technischer und fachlicher Kompetenz. Aber Sie wussten ja bereits, dass es Softwarearchitekten manchmal schwer haben, oder?

In diesem Kapitel möchte ich Ihnen die Anforderungen an praxisgerechte technische¹ Dokumentation vorstellen. Im zweiten Schritt präsentiere ich Ihnen einfache Grundregeln, mit deren Hilfe Sie bereits eine ganze Reihe dieser Anforderungen erfüllen können. Im Hauptteil des Kapitels lernen Sie das Konzept von Sichten und Perspektiven kennen. Danach wissen Sie, wie Sie angemessene² Architekturdokumentation für Ihre Systeme aufbauen können.

¹ Benutzer- oder Anwenderdokumentation gehorcht anderen Regeln, auf die ich hier nicht eingehe.

² Sparsam und nutzergerecht

■ 5.1 Architekten müssen kommunizieren und dokumentieren

Architekten müssen kommunizieren

Softwarearchitekten treffen vielfältige Entwurfsentscheidungen, die prägenden Einfluss auf die Arbeit anderer Stakeholder nehmen können. Um ein gemeinsames Verständnis der Architektur sicherzustellen, müssen Architekten die Entscheidungen (Strukturen, Konzepte, Begründungen, Modelle) *kommunizieren*. Bild 5.1 zeigt diese Aufgabe im Zusammenhang mit der Architekturentwicklung.

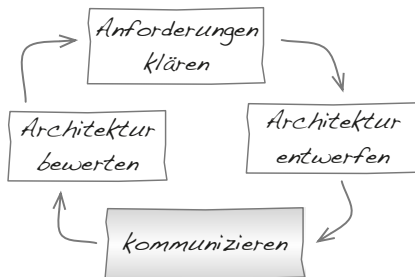


Bild 5.1

Kommunikationsaufgabe von Architekten

Nur durch Kommunizieren können Architekten ihre Entscheidungen motivieren, Entwürfe propagieren sowie die spezifische Architektur dem gesamten Team vermitteln!

Diese Kommunikation erfolgt in der Regel zunächst mündlich (durch gemeinsames Erarbeiten, Erklären, Vorstellen und Überzeugen) und danach schriftlich (durch Dokumente, Modelle oder andere Medien). Für die langfristige Verwendung über die Lebensdauer von IT-Systemen hinweg ist die schriftliche Kommunikation wesentlich – praktisch jedes System profitiert daher von einer methodisch und strukturell fundierten Dokumentation.



In der Praxis scheitern Projekte immer wieder, weil Softwarearchitekten ihrer Kommunikationsaufgabe nicht angemessen nachkommen. Stimmen Sie mit Ihren Stakeholdern den jeweiligen Kommunikations- und Dokumentationsbedarf ab:

- Fragen Sie Stakeholder im Vorfeld nach deren konkreten Bedürfnissen oder Vorstellungen von Dokumentation oder Erläuterungen.
- Holen Sie aktiv Rückmeldungen von Ihren Stakeholdern ein – das hilft Ihnen, Defizite in Ihrer Architekturkommunikation frühzeitig zu erkennen.
- Bleiben Sie offen für Verbesserungsvorschläge –, aber zeigen Sie bei Bedarf auch Durchsetzungs- oder Durchhaltevermögen. Sie tragen die Verantwortung für technische Entscheidungen.
- Verwenden Sie Sichten zur getrennten Beschreibung der unterschiedlichen Strukturen.

- Trennen Sie diese Sichten von den übergreifenden technischen Konzepten (Kapitel 7 hält dazu viele Details für Sie bereit).
- Kommunizieren und dokumentieren Sie Top-down: Beginnen Sie mit *Vogelperspektiven* und fügen schrittweise Details hinzu.
- Benutzen Sie Vorlagen oder Templates für die Gliederung Ihrer Dokumentation (einen bewährten Vorschlag finden Sie in Abschnitt 5.3.1).

Darum sollten Sie Architekturen dokumentieren

Der reine Quellcode kann in realen IT-Systemen aufgrund seines Umfangs sowie seines niedrigen Abstraktionsniveaus die Aufgabe der Dokumentation nicht vollständig übernehmen. Architekturen hingegen ermöglichen eine effektive (d. h. zielgerichtete) gesamthafte Kommunikation über ein IT-System – sie verschaffen Überblick.

Architektur verschafft
Überblick

Die Lebensdauer von IT-Systemen übersteigt in der Regel deutlich die ursprüngliche Erstellungszeit: Viele Systeme entstehen in Projekten von ungefähr einem Jahr Dauer und bleiben für fünf bis fünfzehn Jahre im Einsatz.³ Verständliche, aktuelle und redundanzfreie Dokumentation ermöglicht es in solchen Situationen, auch über einen langen Zeitraum

- den Überblick zu wahren,
- auftretende Fehler und Probleme kurzfristig zu beseitigen,
- geänderte Anforderungen mit angemessenem Aufwand zu erfüllen und
- auf Änderungen im gesamten technischen Umfeld (Hardware, Betriebssysteme, Middleware, Fremdsysteme, Datenbanken etc.) zu reagieren.



- Circa 30–50 % der Wartungs- und Änderungskosten von Software werden benötigt, um bestehenden Code, Konzepte und Strukturen zu verstehen!⁴
- Angemessene Dokumentation kann diesen Aufwand beträchtlich senken!

In IT-Systemen signifikanter Größe korreliert der Aufwand für die oben genannten Aktivitäten direkt mit der Güte der Architekturdokumentation.

³ In manchen Bereichen, wie etwa der Eisenbahntechnik, bleiben Softwaresysteme teilweise länger als 30 Jahre in Betrieb (bzw. die Hersteller müssen für diese Zeit die Wartung und Weiterentwicklung gewährleisten).

⁴ Quelle: <http://blog.optimyth.com/2012/10/6-best-practices-to-save-in-software-maintenance>

■ 5.2 Effektive Architekturdokumentation

Für alle Arten von Architekturdokumentation gelten einige übergreifende Anforderungen und Regeln, die Sie bei der Erstellung solcher Dokumente berücksichtigen sollten. Ich stelle Ihnen zuerst die Anforderungen an Architekturdokumentation vor und leite daraus einige Grundlagen angemessener (= *guter*) Dokumentation ab.

5.2.1 Anforderungen an Architekturdokumentation

In den folgenden Abschnitten stelle ich Ihnen die wesentlichen Anforderungen an Architekturdokumentation vor.

- Die inhaltlichen Anforderungen an Dokumentation beschreiben, was die Dokumentation für Ihre Leserschaft leisten muss. Konkret stelle ich Ihnen einige typische Fragen vor, die Ihre Architekturdokumentation beantworten muss.
- Darüber hinaus sollte Dokumentation einige weitere Anforderungen erfüllen, die ich als „Qualitätsmerkmale von Dokumentation“ bezeichne.

Im Anschluss daran stelle ich Ihnen vor, wie Sie diese Anforderungen erfüllen können. Dazu lernen Sie eine praxistaugliche Struktur von Architekturdokumentation kennen.

Inhaltliche Anforderungen an Architekturdokumentation

Die Softwarearchitektur-Dokumentation (SWAD) muss den Stakeholdern eines IT-Systems eine Reihe wichtiger Auskünfte geben und ihre spezifischen Fragen zur Architektur und allen weiteren technischen Aspekten des Systems beantworten.



Folgende universelle Fragen muss die SWAD beantworten:

- Wie ist der Quellcode des Systems organisiert und strukturiert? Welche „größeren“ Implementierungseinheiten (Bausteine, Strukturelemente) gibt es und welche Beziehungen gibt es dazwischen?
- Wie verhalten sich die Bausteine zur Laufzeit und wie arbeiten sie zusammen?
- Wie fügt sich das System in seine Umgebung ein, insbesondere in seine technische Infrastruktur sowie die Projektorganisation?
- Welche grundlegenden Entscheidungen, Technologien oder Konzepte prägen die Lösung, deren Implementierung und Betrieb?

Die meisten Stakeholder haben daneben noch einige spezifische Fragen. Tabelle 5.1 zeigt einige wichtige im Überblick.

Tabelle 5.1 Typische Fragen an die Architekturdokumentation

Stakeholder	Fragen
Kunden und Auftraggeber	▪ Hat das Entwicklungsteam das Problem verstanden? ▪ Was ist der Kontext des Systems? Mit welchen Nachbarn arbeitet das System zusammen? ▪ Was sind die wesentlichen Entwurfsentscheidungen bzw. Lösungsstrategien? ▪ Passen die Strukturen und Konzepte der (geplanten) Lösung zum Problem? ▪ Ist diese Architektur für den geplanten Einsatz tragfähig hinsichtlich nichtfunktionaler Anforderungen wie Performance und Wartbarkeit?
Manager	▪ Was muss der Auftraggeber zuliefern, welche Teile müssen wir selbst entwickeln, welche Teile können wir zukaufen?
Fachexperten, Entwickler und Qualitätssicherer	▪ Welchen Teil des Systems entwerfe/realisiere/prüfe ich gerade und wie ordnet sich dieser in das Gesamtsystem ein? ▪ Aus welchen Gründen wurden Entwurfsentscheidungen getroffen? ▪ An welchen Stellen muss das System flexibel und variabel sein?
Neue Mitarbeiter	▪ Was hat das Team bisher entworfen und entwickelt? ▪ Wie sieht das System aus der Vogelperspektive aus? Was ist die „Lösungsstrategie“ des Systems? ▪ Welche technischen Grundlagen (Implementierungstechnologien, Frameworks, Konzepte) sind relevant? ▪ Warum sieht der Code so aus? Welche Gründe haben dazu geführt? ▪ Wie sehen die technischen Konzepte (etwa: für Persistenz, Logging, Sicherheit o. Ä.) aus?
Administrator, Betreiber	▪ Wie müssen wir die Bausteine des Systems zur Laufzeit verteilen, damit alles korrekt funktioniert? ▪ Welche Regeln und Randbedingungen müssen wir bei der Installation des Systems, im laufenden Betrieb sowie beim Austausch von Systemteilen beachten? ▪ Wie muss die zugrunde liegende Hardware dimensioniert sein? ▪ Welche Abhängigkeiten von anderen Softwaresystemen gibt es?

Sie entnehmen dieser (unvollständigen) Übersicht,⁵ dass unterschiedliche Stakeholder völlig unterschiedliche Fragestellungen hinsichtlich der Softwarearchitektur eines Systems haben.

Ihre Dokumentation sollte möglichst viele dieser Fragen beantworten können. Die weiter unten vorgestellten Architektursichten helfen dabei, die unterschiedlichen Informationsbedürfnisse verschiedener Lesergruppen angemessen zu erfüllen.

⁵ Stellen Sie bitte in Ihren Projekten sicher, dass Sie sämtliche Fragen, die Ihre Stakeholder an die Architekturdokumentation haben, im Vorfeld kennen!

Qualitätsanforderungen an Architekturdokumentation

Alle Stakeholder stellen an die SWAD hohe Qualitätsansprüche.

- **Aktuell und korrekt:** Falls die Dokumentation Fehler enthält oder einen veralteten Stand widerspiegelt, wird sie niemand lesen wollen. Fehler in technischer Dokumentation sind nicht tolerierbar!
- **Zielgruppenadäquat:** Die Dokumentation muss auf die Bedürfnisse ihrer jeweiligen Leserschaft eingehen und deren jeweilige Fragen beantworten.
- **Verständlich:** Ihre Leser möchten „abgeholt“ werden. Verwenden Sie geeignete Ausdrucksmittel und bitten Sie Ihre Stakeholder, die Verständlichkeit zu bewerten, etwa in Schulnoten. Zur Verständlichkeit gehört auch Konsistenz, also die durchgängige und einheitliche Verwendung von Begriffen, Modellen und Strukturen zur Darstellung bestimmter Aspekte.
- **Wartbar:** Damit die Forderung nach Aktualität erfüllt werden kann, muss die SWAD leicht angepasst werden können. Dabei helfen Modularität und Kürze erheblich. Verständlichkeit steigt durch kontrollierte Redundanzen, die Wartbarkeit nimmt dadurch allerdings ab.
- **Kompakt:** Konzentrieren Sie sich auf die wesentlichen Fakten, die Ihre Leser kennen müssen. Arbeiten Sie aktiv und bewusst daran, Texte und Dokumente möglichst kurz und kompakt zu gestalten, Ihre Leser werden es Ihnen danken. Leider sind Kürze und Prägnanz in Dokumenten erheblich schwerer zu erreichen als übermäßige Länge. Kompakte Dokumentation ist in der Regel leichter wartbar.
- **Top-down⁶ oder hierarchisch organisiert:** Sie sollten einen kurzen Überblick über das System bereitstellen sowie davon abgeleitete Verfeinerungen. In den folgenden Abschnitten finden Sie einige Vorschläge für geeignete Dokumentarten und Strukturen. Achten Sie immer darauf, trotz vieler Bäume den Wald noch erkennen zu können. Dieser Ratschlag bedeutet übrigens NICHT, dass Sie Entwurf und Entwicklung Top-down arbeiten sollen!
- **Kostengünstig:** Zumindest Ihre Manager werden lautstark danach rufen. Als Architekt sollten Sie auf diesen berechtigten Ruf hören. Dokumentieren Sie angemessen: so viel wie nötig, aber so wenig wie möglich.



- Kurze, kompakte Dokumentation ist meistens besser als (übermäßig) ausführliche.
- Verwenden Sie in Ihrer Dokumentation Abstraktionen und reduzieren Sie systematisch Details. Verweisen Sie für Details auf Quellcode.

5.2.2 Regeln für gute Architekturdokumentation

Nun kennen Sie die (Mindest-)Anforderungen an die Architekturdokumentation, Sie benötigen aber noch geeignete Mittel und Wege, sie zu erfüllen. Leider ist es nicht damit getan, einfach „drauflos“ zu schreiben – Dokumentation benötigt Struktur! Ich habe genügend Systeme erlebt, zu denen Hunderte, ja Tausende Seiten an (Detail-)Dokumentation existierten, ohne dass die Verständlichkeit oder die Wartbarkeit des Gesamtsystems davon profitiert hätte.

⁶ Top-down ist lediglich die Richtung der Kommunikation – beim Konstruieren und Entwickeln können Sie ganz beliebig vorgehen.

Häufig fehlte solchen Dokumentationen (die ihren Namen nicht verdienten) jegliche übergeordnete Struktur, sie blieben unverständlich und letztlich nutzlos.

Schaffen Sie Klarheit (um Missverständnisse zu vermeiden)

Sie müssen Syntax und Semantik einer Sprache kennen, um Sätze dieser Sprache zu verstehen. Andernfalls besteht das Risiko von Fehlinterpretationen und Missverständnissen.

Genauso verhält es sich mit Architekturbeschreibungen. Sie als Architekt und Ihre Leser, also die Stakeholder, müssen von Syntax und Semantik der Architekturbeschreibung eine identische Vorstellung besitzen. Die in Abschnitt 5.3.1 beschriebene Strukturvorlage für Architekturbeschreibungen hilft Ihnen, Missverständnisse und Unklarheiten zu vermeiden. Sie gibt Ihnen eine Hilfestellung bezüglich Zweck, Form und Adressaten für die verschiedenen Sichten von Architekturbeschreibungen.

Ich habe in meiner Projektpraxis mehrfach Architekturbeschreibungen gesehen, die von Architekten, Auftraggebern, Managern und Entwicklungsteams völlig unterschiedlich interpretiert wurden, zum Teil mit fatalen Folgen für die Projektziele.

Einige Tipps für mehr Klarheit und weniger Missverständnisse:



- Schaffen Sie eine Gesamtstruktur Ihrer Dokumentation und legen Sie diese Struktur in Form einer Navigationshilfe oder eines *Documentation Guide* offen (die Architektur-der-Architektur-Dokumentation).
- Kommunizieren Sie diese Struktur an alle Betroffenen, insbesondere die Leser und die Ersteller der Dokumentation.
- Stellen Sie sicher, dass alle Stakeholder die Syntax und Semantik der verwendeten Sprache oder Modellierungsmethoden kennen.
- Eine feste und in Ihrer Organisation etablierte Gliederung der Architekturdokumentation erleichtert deren Verständnis, sowohl für Leser als auch Autoren.
- Im Idealfall verfügt ein System über eine *Architektur* der Gesamtdokumentation. Leider ist dieser Idealtypus in der Praxis nahezu ausgestorben.

Beherzigen Sie die Grundprinzipien von Dokumentation

Aufgrund ihrer zentralen Bedeutung fasse ich in diesem Abschnitt wesentliche Grundprinzipien für eine nützliche Dokumentation zusammen.

Sie sollten diese Prinzipien bei der Erstellung von (Architektur-)Dokumenten beherzigen. Falls Sie als Reviewer solcher Dokumente tätig sind, können Sie die folgenden Ratschläge als Checkliste verwenden.



- Schreiben Sie aus der Sicht der Leser: Dokumente werden häufiger gelesen als geschrieben. Achten Sie darauf, dass Ihre Leser sie auch verstehen können. Sie sind als Autor Dienstleister am Leser. Verwenden Sie das Vokabular Ihrer jeweiligen Leserschaft. Erklären Sie zentrale und für das Verständnis wesentliche Begriffe in einem Glossar.
- Dokumentieren Sie das Warum: Beschreiben Sie die Gründe, die zu einem Entwurf geführt haben. Zu dieser Begründung gehört idealerweise auch die Beschreibung der verworfenen oder zurückgestellten Alternativen.
- Dokumentieren Sie Ihre Annahmen, Rahmenbedingungen und Voraussetzungen: Ihre Entscheidungen werden verständlicher, wenn Sie Annahmen und Voraussetzungen klar beschreiben. Bitte denken Sie daran: Für Sie als Autor sind möglicherweise ganz andere Dinge selbstverständlich als für Ihre Leser.
- Vermeiden Sie zu viele Wiederholungen (Don't repeat yourself, DRY-Prinzip), auch bekannt als Prinzip des Single-Point-of-Truth (SPOT-Prinzip). In der Praxis ist es manchmal nützlich, zur Verbesserung der Verständlichkeit kontrollierte Redundanz zuzulassen, um beispielsweise die Anzahl der Querverweise zu verringern. Beachten Sie, dass Redundanzen den Aufwand für die Pflege von Dokumenten stark erhöhen.
- Vermeiden Sie Überraschungen (Principle of least astonishment, POLA-Prinzip). Lösungen sollen möglichst geringe Verwunderung auslösen. In Dokumentationen sollten Sie alle möglichen Ursachen für Verwunderung zumindest ansprechen und erklären.
- Erklären Sie Ihre Notation (oder verwenden Sie Standards, die erklärt sind): Solange Sie die Bedeutung von Symbolen offen lassen, bleibt Spielraum für Interpretation, Mehrdeutigkeit und Missverständnis. Sie sollten es Ihren Lesern möglichst einfach machen, die Bedeutung von Symbolen und Diagrammen so zu verstehen, wie Sie es gemeint haben.
- Geben Sie jedem Diagramm eine Legende – eine Spezialisierung von „Erklären Sie Ihre Notation“.
- Erklären Sie jedes Element in Diagrammen. Erklären Sie sämtliche Elemente (Bausteine und Beziehungen) innerhalb von (Architektur-)Diagrammen. (Siehe auch „Vermeiden Sie Mehrdeutigkeit“.)
- (Möglichst) 7 +/- 2 Elemente pro Diagramm.⁷ Eine Regel der kognitiven Psychologie besagt, dass der Mensch im Kurzzeitgedächtnis nur zwischen fünf und neun Elementen speichern kann. Als Ausnahme gilt die „Architekturtapete“ in Abschnitt 5.3.
- Verwenden Sie standardisierte Strukturen. Etablieren Sie eine standardisierte Gliederung für die Architekturdokumentation. Stellen Sie sicher, dass Ihre Leser diese Struktur kennen. Das hilft Ihnen und Ihren Lesern ungemein, Informationen schnell zu finden.

⁷ Hier lasse ich Ausnahmen zu – wenn es 20 sinnvolle Bausteine ohne Struktur gibt, dann sollten Sie auch keine künstliche Struktur schaffen. In solchen Fällen hilft es gelegentlich, sich in manchen Diagrammen auf einen Teil der gesamten Bausteine zu beschränken (und zu erwähnen, dass es noch weitere gibt!).

- Kennzeichnen Sie offene Punkte. Falls Informationen noch nicht verfügbar sind, sollten Sie, anstatt Abschnitte oder Überschriften wegzulassen, solche Teile mit kurzen Kommentaren (etwa: <TBD> für to-be-done) kennzeichnen. (Siehe auch „Keine degenerierten Kapitel“.)
- Prüfen Sie Dokumente auf Zweckdienlichkeit. Führen Sie inhaltliche Reviews der Dokumente durch, die mehr als nur Formalien prüfen. Fragen Sie verschiedene Stakeholder in unterschiedlichen Rollen nach deren inhaltlichem Urteil und Meinung. Lassen Sie Reviewer Schulnoten für die Verständlichkeit vergeben (und beseitigen Sie die Ursachen möglicher schlechter Noten!).⁸
- Keine degenerierten Kapitel.⁹ Lassen Sie keine Abschnitte oder Kapitel leer – kennzeichnen Sie absichtlich oder vorläufig leere Teile zumindest mit Stereotypen oder Formatierungen wie <<entfällt>>, <<to-be-done>> oder <<unzutreffend>>.

Bitte beachten Sie auch die Ratschläge zur Modellierung in Abschnitt 6.3.

■ 5.3 Typische Architekturdokumente

In diesem Abschnitt stelle ich Ihnen einige häufig benötigte Arten von Dokumenten und deren mögliche Gliederung vor. Sie können diese als Vorlagen für Ihre eigenen Architekturdokumentationen verwenden.



Erstellen und pflegen Sie NUR Dokumente, die Sie und die Stakeholder des Systems wirklich benötigen.

Seien Sie bewusst sparsam!

Die folgende Liste stellt eine Obermenge dar, keinesfalls eine „Empfehlung-für-alle-Fälle“!!

Dafür benötigten Sie natürlich Werkzeuge – auf die gehe ich in Abschnitt 5.11 noch ein.

Zu jedem System sollte es eine zentrale Architekturbeschreibung geben, die (möglichst) alle architekturrelevanten Informationen enthält. Als Gliederung empfehle ich [arc42], erläutert in der nachfolgenden Tabelle 5.2 in Abschnitt 5.3.1. Dazu gehören wesentliche Qualitätsanforderungen, Kontextabgrenzung Lösungsansätze, querschnittliche Konzepte,

⁸ Nein, Sie sollen NICHT Ihre Reviewer beseitigen, sondern die schlecht verstandenen Teile der Dokumentation verständlich machen!

⁹ Der Begriff „degenerierte Kapitel“ spielt auf ein Anti-Pattern des objektorientierten Entwurfs an, bei dem Methoden einer Oberklasse in einer Unterklasse durch leere Implementierungen überschrieben werden. Im OO-Falle stellt dies eine krasse Verletzung des Liskov’schen Substitutionsprinzips dar.

Sichten und einige zentrale Entwurfsentscheidungen. Beispiele für solche Dokumentation finden Sie in Kapitel 12.

Parallel dazu empfehle ich jedem Entwicklungsprojekt ein *Programmers' Daily Reference Guide*, ein sehr kurzes Dokument mit den wichtigsten (technischen) Informationen für Entwickler und Tester hinsichtlich Entwicklungsmethoden, Programmierung, Übersetzung/Bau sowie Start und Test des Systems.

Falls Sie Systeme mit hohen Anforderungen an Dokumentation entwickeln und pflegen (oder Sie zu den fleißigen Menschen gehören), könnten weitere Arten von Dokumenten langfristig nützlich sein:

- Dokumentationsübersicht: ein Verzeichnis sämtlicher architekturrelevanter Dokumente, sozusagen die Architektur der Architekturdokumentation.
- Übersichtspräsentation: eine Sammlung von Folien, mit der Sie in maximal einer Stunde die Architektur Ihres Systems fachlich und technisch motivieren und die Strukturen in den Grundzügen erläutern können.
- Architekturtapete: eine Mischung aus Baustein- und Verteilungssicht, die viele Architektur Aspekte zusammen zeigt. Eine solche Tapete widerspricht dem „7 +/- 2“-Prinzip, kann allerdings manchmal eine gute Diskussionsbasis für Architekten und Entwickler sein.
- Handbuch zur Architekturdokumentation: Hier beschreiben Sie, wie die Architekturdokumentation für Ihr System funktioniert. Sie benennen Ihre Sichten mit zugehöriger Notation und erläutern die Struktur Ihrer Sichtenbeschreibungen. In vielen Fällen genügen die Beispiele aus Kapitel 12 und die Informationen der Website [arc42].

5.3.1 Zentrale Architekturbeschreibung

Die zentrale Architekturbeschreibung stellt den umfangreichsten und auch detailliertesten Teil der Architekturdokumentation dar. Sie heißt manchmal auch SWAD¹⁰, Architektur-Gesamtdokument, Architektur-Referenz oder auch *Architecture-State-of-the-Art*. In Tabelle 5.2 finden Sie einen Gliederungsvorschlag, der auf dem (frei verfügbaren) Template [arc42] basiert.

Tabelle 5.2 Gliederungsvorschlag für die zentrale Architekturbeschreibung

Überschrift	Erläuterung
1. Einführung und Ziele	Die maßgeblichen Forderungen der Auftraggeber. In diesem Kapitel fassen Sie (kurz!) wichtige Punkte der Anforderungsdokumentation zusammen.
1.1 (Fachliche) Aufgabenstellung	Eine kompakte Zusammenfassung des fachlichen Umfelds. Erklärt den „Grund“ für das System. Stellen Sie die wichtigsten vom System bearbeiteten Geschäftsprozesse als Use-Cases oder User-Stories (Text und Diagramm) dar. Verweisen Sie für Details auf die (hoffentlich vorhandene) Anforderungsdokumentation.

¹⁰ SWAD = Software Architektur Dokumentation

Tabelle 5.2 (Fortsetzung) Gliederungsvorschlag für die zentrale Architekturbeschreibung

Überschrift	Erläuterung
1.2 Qualitätsziele	Beschreiben Sie hier Qualitätsanforderungen, deren Erfüllung oder Einhaltung den maßgeblichen Stakeholdern besonders wichtig ist. Sprechen Sie hier insbesondere architekturelevante Themen wie Performance, Sicherheit, Änderbarkeit, Bedienbarkeit und Ähnliches an. Hierzu gehören auch Mengengerüste: Benennen und quantifizieren Sie wichtige Größen des Systems, etwa Datenaufkommen und Datenmengen, Dateigrößen, Anzahl Benutzer, Anzahl Geschäftsprozesse, Transfervolumen und andere. Beschränken Sie sich auf die wichtigsten (fünf bis zehn) Ziele.
1.3 Stakeholder	Eine Liste der wichtigsten Personen oder Organisationen im Umfeld des Systems, zusammen mit deren wesentlichen Zielen und Erwartungen an das System (beziehungsweise dessen Konstruktion und Entwicklung)
2. Einflussfaktoren und Randbedingungen	Führen Sie organisatorische und technische Randbedingungen auf, die Auswirkungen auf Architekturentscheidungen besitzen können. Führen Sie die wichtigsten Einschränkungen der Entwurfsfreiheit auf. Insbesondere gehören hier technische, organisatorische und juristische Randbedingungen sowie einzuhaltende Standards hinein.
3. Kontextabgrenzung	Sicht aus der „Vogelperspektive“, zeigt das Gesamtsystem als Blackbox und den Zusammenhang mit Nachbarsystemen, wichtigen Stakeholdern sowie der technischen Infrastruktur. Sie sollten sowohl den fachlichen als auch den technischen Kontext darstellen. Siehe Abschnitt 5.5.
4. Lösungsstrategie	Stellen Sie hier die Kernidee Ihrer Lösung dar: Was sind Ihre zentralen Lösungsansätze, Gestaltungskriterien oder Herangehensweisen? Dieses Kapitel sollten Sie kompakt halten – die ausführliche Darstellung können Sie auf die Bausteine beziehungsweise technischen Konzepte verschieben.
5. Bausteinsichten	Statische Zerlegung des Systems in Bausteine (Subsysteme, Module, Komponenten, Pakete, Klassen, Funktionen) und deren Zusammenhänge und Abhängigkeiten. Beginnt mit der Whitebox-Darstellung des Gesamtsystems, wird über abwechselnde Black- und Whitebox-Sichten schrittweise weiter verfeinert. Siehe Abschnitt 5.6 für die Strukturen der Bausteine und Abschnitt 5.7 für deren Schnittstellen.
5.1, 5.2, 5.3 ...	Verfeinerungsebenen 1, 2, 3 der Bausteinsicht.

Tabelle 5.2 (Fortsetzung) Gliederungsvorschlag für die zentrale Architekturbeschreibung

Überschrift	Erläuterung
6. Laufzeitsicht	Zeigen Sie das Zusammenspiel der Architekturbausteine in Laufzeit-szenarien. Sie sollten hier mindestens die Erfolgsszenarien der zentralen Use-Cases beschreiben sowie weitere aus Ihrer Sicht wichtige Abläufe. Siehe Abschnitt 5.7.
7. Verteilungs- oder Infra-struktursicht	Diese Sichten zeigen, in welcher Umgebung das System abläuft, sowohl Hardware (Rechner, Netze etc.) als auch Software (Betriebssysteme, Datenbanken, Middleware etc.). Siehe Abschnitt 5.8.
8. (Querschnitt-liche) Konzepte	Zu übergreifenden oder querschnittlichen Konzepten zählen beispielsweise Persistenz, Ausnahme- und Fehlerbehandlung, Logging und Protokollierung, Transaktions- und Session-Behandlung, der Aufbau der grafischen Oberfläche, Ergonomie sowie Integration oder Verteilung des Systems. In Abschnitt 5.10 erfahren Sie, wie Sie Konzepte dokumentieren. In Kapitel 7 stelle ich Ihnen einige dieser Konzepte näher vor.
9. Entwurfs-entscheidungen	Dokumentieren Sie hier die wichtigen <i>übergreifenden</i> Architektur-entscheidungen und deren Gründe. Beschreiben Sie auch verworfene Alternativen. Entscheidungen bezüglich der Struktur einzelner Bausteine sollten Sie in den jeweiligen Whitebox-Darstellungen der Bausteinsicht beschreiben.
10. Qualitäts-szenarien	Szenarien konkretisieren Qualitätsanforderungen und bilden eine wichtige Grundlage für Architekturentwicklung und -bewertung. Sie besitzen langfristigen Wert und bleiben meistens über längere Zeit stabil. Daher lohnt es sich, sie in die Architekturdokumentation aufzunehmen. Siehe Abschnitt 3.2.3 für Details dazu.
11. Risiken und technische Schulden	Risikomanagement ist im Allgemeinen die Aufgabe vom Management, Risiken sollten auch von diesem dokumentiert werden. Falls Ihr Management das jedoch vernachlässigt, haben Sie hier einen passenden Platz, um technische Risiken mit ihren möglichen Auswirkungen und Abhilfe-maßnahmen zu dokumentieren. Legen Sie hier auch die technischen Schulden des Systems offen – das kann Ihnen bei der späteren Änderung helfen.
12. Glossar	Die wesentlichen (Fach-)Begriffe. Im Domain-Driven Design auch bekannt als <i>Ubiquitous Language</i>

5.3.2 Architekturüberblick

Der Architekturüberblick ist ein Textdokument, das die Inhalte der Übersichtspräsentation enthält, doch ohne weitere Erläuterung verständlich sein muss.

Im Architekturüberblick ist Kürze gefragt, nicht Vollständigkeit. Meine persönliche Zielvorstellung liegt bei diesen Überblicksdokumenten bei maximal 20 bis 30 Seiten – mehr mögen gestresste Stakeholder oder termingeplagtes Management nicht lesen.

In jedem Fall enthält dieses Dokument die zentrale Lösungsstrategie, einige Sichten und erläutert daran die Funktionsweise der Architektur hinsichtlich der wichtigsten funktionalen und Qualitätsanforderungen.

Bei der Gliederung können Sie sich an der Übersichtspräsentation orientieren.

5.3.3 Dokumentationsübersicht

In der Regel werden Architekturdokumentationen aus mehreren Einzeldokumenten bestehen. Der Zusammenhang zwischen diesen Teilen wird in einer eigenständigen Dokumentationsübersicht (*Documentation Roadmap*) beschrieben.

- Welche Dokumente/Dokumentation gibt es (mit Namen und Ablageort)?
- Was ist deren Ziel und Inhalt (Abstract)?
- Welche Stakeholder sollten was in welcher Reihenfolge lesen?
- Welche Abhängigkeiten bestehen zwischen den Dokumenten?



Ich habe in Entwicklungsprojekten gute Erfahrungen damit gemacht, Dokumentationsübersichten auf der Einstiegsseite im Wiki zu beschreiben.

5.3.4 Übersichtspräsentation der Architektur

Im Laufe eines Projekts ergibt sich immer wieder die Notwendigkeit, die Architektur des Systems in Form einer Präsentation zu erläutern. Dazu sollten Sie eine Übersichtspräsentation erstellen, die Sie (möglicherweise in leichten Variationen) dann wiederverwenden können. Dafür schlage ich Ihnen folgende Gliederung vor:

- *Einführung*: Welches Problem löst das System? Verwenden Sie zwei bis drei Folien, nehmen Sie die allerwichtigsten Anforderungen an Architektur und technische Infrastruktur mit auf.
- *(Architektonischer) Lösungsansatz*: Beschreiben Sie auf ca. zwei Folien die Strategie sowie die Besonderheiten der Architektur. Benennen Sie grundlegende technische oder querschnittliche Konzepte, Architekturmuster oder -stile.
- *Kontext des Systems*: Zeigen Sie die Kontextabgrenzung mit den Außenschnittstellen sowie seinen Nachbarsystemen. Sie dürfen abstrahieren, d. h. Daten- oder Kontrollflüsse zu „größeren“ Einheiten zusammenfassen.

- *Strategische/grundlegende Lösungsentscheidungen:* Funktionsweise der Architektur hinsichtlich der wichtigsten Qualitätsmerkmale: Welche Maßnahmen haben Sie als Architekt ergriffen, um die geforderten Qualitätsmerkmale (etwa: Effizienz/Performance, Flexibilität, Skalierbarkeit, Sicherheit etc.) zu erreichen (zwei bis drei Folien)?
- *Architektursichten:* Fünf bis 15 Folien zeigen jeweils eine Grafik mit Erläuterungen einiger wichtiger Architekturbausteine. Verweisen Sie auf die detaillierten Sichtenbeschreibungen. Hierin sollten mindestens ein bis zwei Bausteinsichten, eine Laufzeitsicht (für die wichtigsten Use-Cases des Systems) sowie eine Infrastruktursicht enthalten sein.
- *Stakeholder-Übersicht:* Ein bis zwei Folien stellen möglichst alle Stakeholder sowie deren Ziele oder Intentionen und spezifische Informationsbedürfnisse dar.

5.3.5 Architekturtapete

In manchen Fällen kann eine Gesamtansicht vieler Architekturelemente auf einem einzigen Diagramm die Diskussion zwischen Architekten und Entwicklern vereinfachen. Eine solche Darstellung eignet sich in der Regel nicht als Einstieg oder Referenz, ermöglicht jedoch schnelle Sprünge von abstrakten zu detaillierten Aspekten der Architektur.

In meiner Berufspraxis habe ich so manches Mal die Nützlichkeit dieser „Tapeten“ erlebt:

- Sie beeindrucken damit in der Regel Ihre Manager.
- Sie können Architekturelemente (Bausteine, Knoten oder Schnittstellen) mit Statusinformationen überlagern, etwa hinsichtlich Fertigstellungsgrad, Risiko, Performance oder ähnlich wichtigen Managementinformationen.

Beachten Sie, dass die Erstellung riesiger Diagramme auch einen ungeheuren Aufwand bedeutet, ebenso wie deren Pflege. Erstellen Sie die Architekturtapete erst dann, wenn Stakeholder danach fragen.

Papier/Whiteboard und Stifte sind prima Medien für eine „große“ Übersicht dieser Art.

■ 5.4 Sichten

Ich möchte das Konzept von Sichten mit einer Analogie aus dem Immobilienbau motivieren: Beim Entwurf und Bau von Immobilien entstehen für unterschiedliche Stakeholder völlig unterschiedliche Pläne. Tabelle 5.3 zeigt eine (unvollständige) Liste.

Diese unterschiedlichen Pläne sind allesamt Modelle, d. h. Abstraktionen der Realität. Kein einzelner Plan gibt die gesamte Komplexität eines Hauses wieder, jeder Plan (jede Sicht) vernachlässigt gewisse Details: Jede Zielgruppe erhält genau die Pläne, die sie für ihre jeweilige Aufgabe benötigt!

Dieses Prinzip heißt Sichtenbildung. Viele Ingenieursdisziplinen arbeiten danach und verwenden dazu Normen, die die Struktur, Syntax und Semantik ihrer Pläne (Sichten) detailliert beschreiben.

Tabelle 5.3 Sichten auf Immobilien

Plan/Sicht	Bedeutung	Format	Nutzer
Grund- und Aufriss	Lage und Beschaffenheit von Mauern, Maueröffnungen (Türen, Fenstern, Durchgängen), Böden, Decken	Normiert nach DIN	Architekt, Maurer, Käufer
Elektroplan	Lage von spannungsführenden Leitungen, Schaltern, Steckdosen, Verteilern, Sicherungen sowie sonstiger Elektroinstallation	Normiert nach DIN	Architekt, Käufer, Elektriker, Küchenbauer, Verwaltung (wegen Stromversorgung)
Heizungs-, Wasser- und Sanitärplan	Lage von Wasser- und Abwasserleitungen, Heizungsrohren sowie Gasleitungen	Normiert nach DIN	Architekt, Heizungs- und Sanitärinstallateur, Käufer, Küchenbauer, Verwaltung (wegen Abwasseranschluss)
3D-Modell	Dreidimensionale Darstellung des Gebäudes im Ganzen oder in Teilen	Beliebig, Bilder oder Filme („virtuelle Begehung“)	Käufer, Verkäufer
Raumplan	Zweidimensionale Darstellung von Zimmern und Einrichtung	Beliebig, angelehnt an DIN	Käufer, Innenarchitekt, Küchenbauer

Interpretieren Sie eine Sicht als eine spezifische Perspektive, aus der heraus Sie ein beliebiges System betrachten können.

5.4.1 Sichten in der Softwarearchitektur

- Eine einzelne Darstellung vermag die Vielschichtigkeit und Komplexität einer Architektur nicht auszudrücken. Betrachten Sie dazu Bild 5.2. Daher sollten Sie Architekturen grundsätzlich aus mehreren Sichten oder Perspektiven beschreiben.
- Sichten ermöglichen die Konzentration auf einzelne Aspekte des Gesamtsystems. Damit erleichtern sie Entscheidungen innerhalb dieser Aspekte (beispielsweise für die Zerlegung in statische Bausteine) und reduzieren zusätzlich die Darstellungskomplexität.
- Die Beschreibung von Architekturen ist für viele Stakeholder mit ganz unterschiedlichen Informationsbedürfnissen wichtig. Zum Beispiel benötigt Management andere Informationen aus der Architekturbeschreibung als Entwicklungsteam, Qualitätssicherer und Betreiber.

Stakeholder, Anforderungen und Dokumente

Stakeholder benutzen Dokumente, um Architekturen zu verstehen und darüber zu kommunizieren. Verschiedene Stakeholder benötigen meistens auch unterschiedliche Arten von Dokumenten. In Abschnitt 5.3 finden Sie Beispiele einiger typischer Architekturdokumente.

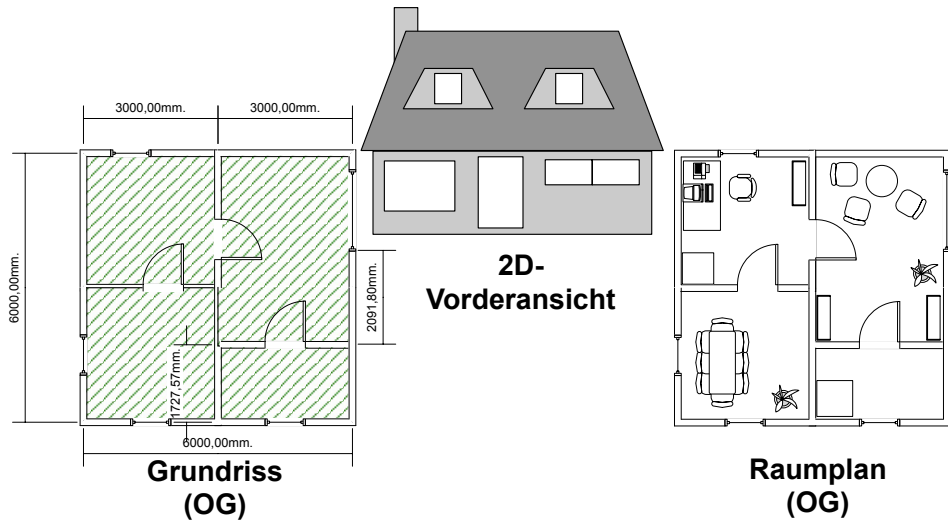


Bild 5.2 Drei Sichten auf eine Gebäudearchitektur



Eine Architekturdokumentation sollte sämtliche architekturrelevanten Belange der Stakeholder adressieren. Sie als Architekt stehen in der Pflicht, diese Stakeholder und deren Belange zu identifizieren und in der Architektur sowie ihrer Dokumentation zu berücksichtigen.

Sichten, Dokumente und Architekturdokumentation

Eine Sicht zeigt das System aus einer spezifischen Perspektive. Sie abstrahiert von Details, die für diese Perspektive nicht von Bedeutung sind. Sichten erlauben die Konzentration auf bestimmte Details oder bestimmte Aspekte.

Die Analogie aus der Gebäudearchitektur aus Bild 5.2 zeigt das Konzept von Sichten: Grundriss, Raumplan und 2D-Darstellung eines Gebäudes sind Sichten auf dessen Architektur, die jeweils einen bestimmten Aspekt hervorheben und dafür von anderen Details abstrahieren.

Dokumentation von Software- und Systemarchitekturen besteht ebenso aus mehreren Sichten, (meist) verteilt auf mehr als ein Dokument.

Die Diagramme oder textlichen Beschreibungen einer Sicht können auch unterschiedliche Abstraktionsebenen oder Detaillierungsstufen beschreiben. Damit können Sie verschiedene Teile einer einzigen Sicht beispielsweise für unterschiedliche Adressaten nutzen. Hinzu kommen noch die querschnittlichen oder technischen Konzepte, denen das gesamte Kapitel 7 gewidmet ist.

Sichten und Konzepte als Grundlage agiler Softwarearchitekturen

Architektursichten und querschnittliche Konzepte (siehe Kapitel 7 sowie Abschnitt 5.10) ermöglichen die praxisrelevante Beschreibung von Softwarearchitekturen. Die Sichten bleiben so flexibel, dass sie auch unterschiedlichen und heterogenen Anforderungen genügen.

Damit bilden sie, neben den agilen Prozessen, eine wichtige Grundlage flexibler und anwendungsorientierter Softwareentwicklung.

Bei der Definition der Sichten standen *Agilität* und *Angemessenheit* im Vordergrund:



- Legen Sie Wert auf Flexibilität statt auf starre Muster. Setzen Sie Sichten bedarfsgerecht und effektiv ein, indem Sie die konkreten Informationsbedürfnisse der Stakeholder adressieren. Fragen Sie Ihre Stakeholder ausdrücklich, welche Aspekte des Systems sie für ihre jeweiligen Aufgaben benötigen.
- Und noch eine schwierige Regel: Verwenden Sie so wenig Formalismus wie möglich, aber so viel wie nötig.
- Orientieren Sie den Umfang der Dokumentation (von Sichten) am jeweiligen Risiko: Dokumentieren Sie Komponenten oder Systeme mit vielen Risikofaktoren ausführlich und detailliert. Bei Komponenten mit überschaubarem Risiko können Sie pragmatisch vorgehen und auf manche Details verzichten.
- Querschnittliche Konzepte können helfen, die Sichten zu vereinfachen, indem sie wiederkehrende Themen „konzeptionell“ beschreiben.

5.4.2 Vier Arten von Sichten

Bild 5.3 zeigt die vier wichtigsten Arten von Sichten, um Softwarearchitekturen effektiv und bedarfsgerecht zu beschreiben. Je nach Art Ihres Systems können Sie diese Sichten verschieden detailliert darstellen oder unterschiedlich gewichten.

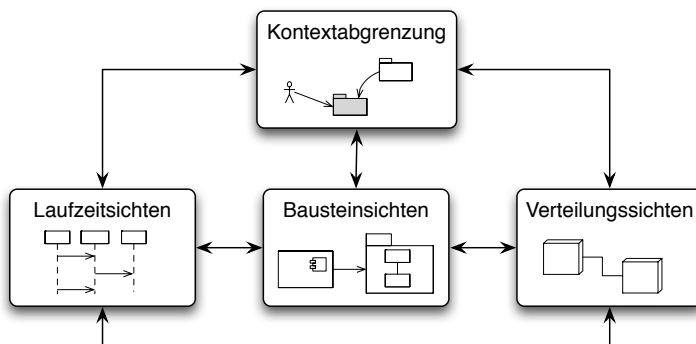


Bild 5.3
Vier Arten
von Sichten

Die Pfeile zwischen den Sichten symbolisieren dabei mögliche Abhängigkeiten oder Wechselwirkungen.

Der Plural bei der Bezeichnung der Sichten drückt aus, dass Sie in der Regel mehr als eine Sicht dieser Art entwickeln werden:

- Sie verfeinern einzelne Teile der Sichten: Die interne Struktur von Elementen, die in einer (abstrakteren) Sicht als Blackboxes stehen, wird in Verfeinerungen offengelegt und erläutert.

- Sie beschreiben unterschiedliche Architekturbausteine des gleichen Abstraktionsniveaus in verschiedenen Sichten der gleichen Art. Wenn Ihr System beispielsweise aus drei Subsystemen besteht, würden Sie zu jedem dieser Subsysteme eine eigene Baustein- und Laufzeitsicht entwickeln.

Die Sichten – ein Überblick

- *Kontextabgrenzungen* – Wie ist das System in seine Umgebung eingebettet? Kontextabgrenzungen zeigen das System als Blackbox in seinem Kontext aus einer Vogelperspektive. Hier zeigen Sie die Schnittstellen zu Nachbarsystemen, die Interaktionen mit wichtigen Stakeholdern sowie die wesentlichen Teile der umgebenden Infrastruktur.
- *Bausteinsichten* – Wie ist das System intern aufgebaut? Wie ist der Quellcode organisiert? Bausteinsichten zeigen die statischen Strukturen der Architekturbausteine des Systems, Subsysteme, Komponenten und deren Schnittstellen. Sie können (und sollten!) die Bausteinsichten Top-down beschreiben,¹¹ ausgehend von einer Kontextsicht. Die letzte mögliche Verfeinerungsstufe der (Baustein-)Zerlegung bildet der Quellcode. Bausteinsichten dienen als Landkarte für den Quellcode und die oberste Ebene hilft dabei, die Gesamtstruktur des Systems zu verstehen.
- *Laufzeitsichten* – Wie läuft das System ab? Die Laufzeitsichten beschreiben, welche Bausteine des Systems zur Laufzeit existieren und wie sie zusammenwirken. Im Gegensatz zur statischen Betrachtungsweise bei den Bausteinsichten beschreiben Sie hier dynamische Strukturen.
- *Verteilungssichten (Infrastruktursichten)* – In welcher Umgebung läuft das System ab? Diese Sichten beschreiben die Hardwarekomponenten, auf denen das System abläuft. Sie dokumentieren Rechner, Prozessoren, Netztopologien und -protokolle sowie sonstige Bestandteile der physischen Systemumgebung. Die Infrastruktursicht zeigt das System aus Betreibersicht.

Gibt es noch weitere Sichten?

Neben den oben genannten Arten von Sichten benötigen manche Stakeholder möglicherweise weitere Darstellungen.

Es spricht vieles dafür, Stakeholdern auch spezifische (oder exotische?!) Wünsche zu erfüllen, da zufriedene Stakeholder die Zusammenarbeit erheblich erleichtern, aber:



Mein Tipp:

Verzichten Sie möglichst auf weitere Sichten. Jede Sicht kostet Erstellungs- und Wartungsaufwand, der Sie eventuell von (wichtigeren) Architekturaufgaben abhält. Die grundlegenden Aspekte der Architektur- und Systementwicklung decken Sie in den meisten Fällen mit den vier Sichten (Kontextsicht, Bausteinsicht, Laufzeitsicht, Verteilungssicht) bereits ab.

¹¹ Entwickeln und bearbeiten sollten Sie die Bausteinsichten in einer für Sie passenden Reihenfolge – nicht unbedingt Top-down!



Beispiel (nach Peter Hruschka):

Beim Häuserbau könnten Kakteen- und Orchideenzüchter nach der Sonneneinstrahlung in einzelnen Räumen fragen und zum Wohle ihrer pflanzlichen Lieblinge einen gesonderten Plan wünschen. Wie groß ist Ihrer Erfahrung nach die Zahl derer, die beim Bau oder beim Kauf einer Immobilie diese „pflanzliche“ Sicht als Entscheidungskriterium verwenden?

Falls Sie unbedingt eine neue Sicht beschreiben wollen (oder müssen): Definieren Sie vorher, welche Art von Elementen diese neuen Sichten enthalten und mit welcher Notation sie beschrieben werden sollen.

Wo sind Daten beschrieben?

Bei hochgradig datengetriebenen Anwendungen kann es nützlich sein, Daten- oder Informationsflüsse innerhalb des Systems explizit zu beschreiben. Wenn Sie (statische) Datenstrukturen benötigen, gehören die in die Bausteinsicht. Datentransformationen sind Abläufe und gehören in die Laufzeitsicht.

Wo sind Schnittstellen dokumentiert?

Schnittstellen beschreiben Sie im Normalfall in den Bausteinsichten. Wenn Schnittstellen in Ihren Systemen eine herausragende Rolle spielen, dann extrahieren Sie Schnittstellenbeschreibungen in eigenständige Dokumente (und referenzieren in den Bausteinsichten darauf!). In Abschnitt 5.9 lernen Sie, wie Sie Schnittstellen effektiv und verständlich beschreiben.

5.4.3 Entwurf der Sichten

Der Entwurfsprozess der Sichten wird von deren starken Wechselwirkungen und Abhängigkeiten geprägt. Softwarearchitekturen sollten daher iterativ entstehen, weil die Auswirkungen mancher Entwurfsentscheidungen erst über die Grenzen von Sichten hinweg spürbar werden.

Wechselwirkungen der Architektursichten

Der Entwurf einer bestimmten Architektursicht hat oftmals prägenden Einfluss auf andere Sichten. Änderungen einer Sicht ziehen Anpassungen anderer Sichten nach sich.

Betrachten Sie dazu nochmals Bild 5.3 Die Linien zeigen, wie sich die einzelnen Sichten gegenseitig beeinflussen. Nebenbedingungen oder Einschränkungen in einer der Sichten wirken sich auf die übrigen aus.

In welcher Reihenfolge entwerfen Sie die Sichten?

Letztlich spielt es kaum eine Rolle, mit welcher Architektursicht Sie beginnen. Im Laufe des Entwurfs der Softwarearchitektur werden Sie an allen Sichten nahezu parallel arbeiten oder häufig zwischen den Sichten wechseln. Oft werden Sie auch querschnittliche Themen (siehe Kapitel 7 und Abschnitt 5.10) parallel zu den Sichten entwerfen oder implementieren.



Einige Tipps zum Vorgehen: Beginnen Sie den Entwurf der Architektur mit einer

- Bausteinsicht, wenn Sie:
 - bereits ähnliche Systeme entwickelt haben und eine genaue Vorstellung von benötigten Implementierungskomponenten oder technischen Konzepten besitzen;
 - ein bereits teilweise bestehendes System verändern müssen und damit Teile der Bausteinsicht vorgegeben sind.
- Laufzeitsicht, wenn Sie bereits eine erste Vorstellung wesentlicher Architekturbausteine besitzen und deren Verantwortlichkeit und Zusammenspiel klären wollen.
- Verteilungssicht, wenn Sie viele Randbedingungen und Vorgaben durch die technische Infrastruktur, das Rechenzentrum oder den Administrator des Systems bekommen.

Meiner Erfahrung nach beginnt Entwicklung oftmals mit Teilen der Bausteinsicht und dazu passenden Laufzeitszenarien. Parallel dazu entstehen erste querschnittliche (oftmals technische) Konzepte.

Pragmatismus und Effektivität

Sie erhalten mit den Sichten ein methodisches Werkzeug, das Sie in ihrer Entwicklungsarbeit vielseitig nutzen können. Setzen Sie dieses Werkzeug angemessen und pragmatisch ein.



- Dokumentieren Sie nur so viel wie nötig. Identifizieren Sie konkrete Informationsbedürfnisse der Stakeholder, dann können Sie Ihre Dokumentation darauf ausrichten.
- Anders ausgedrückt: Dokumentieren Sie sparsam. Abstrahieren Sie, beschränken Sie sich auf relevante und/oder überraschende Teile.
- Machen Sie die Architekturdokumentation allen Stakeholdern zugänglich.
- Stellen Sie sicher, dass alle Stakeholder die Architektur verstehen. Kommunizieren Sie die Architektur und die zugehörigen Entwurfsentscheidungen. Denn: „Sie glauben nur so lange, dass Ihr Entwurf perfekt ist, bis Sie ihn jemand anderem gezeigt haben“ (nach [Rechtin2000]).
- Beachten Sie Rückmeldungen der Stakeholder. Auch ein Entwurfs- oder Programmierfehler kann eine solche Rückmeldung sein – vielleicht hätten Sie anders dokumentieren sollen.
- Sie können Details in der Bausteinsicht oder in Laufzeitszenarien oftmals durch querschnittliche Konzepte ersetzen (und damit möglicherweise viel Dokumentationsaufwand sparen!).

Wie viel Aufwand für welche Sicht?

Rechnen Sie damit, dass Sie 60 bis 80 % der Zeit, die Sie für den Entwurf der Architektur insgesamt benötigen, alleine für die Ausgestaltung der Bausteinsicht sowie der querschnittlichen Konzepte aufwenden.

Laufzeitszenarien benötigen Sie oftmals zum „Finden“ geeigneter Bausteine oder zur Diskussion von Abläufen mit dem Entwicklungsteam. In der Dokumentation des Systems bleiben (hoffentlich) nur sehr wenige dieser Szenarien übrig.

Spezielle Wechselwirkungen dokumentieren



Sie sollten in Ihrer Architekturdokumentation die Entwurfsentscheidungen dokumentieren, die besonderen Einfluss auf das System, dessen Erstellung und Wartung haben.

■ 5.5 Kontextabgrenzung

Was zeigt die Kontextabgrenzung?

Kontext bedeutet Zusammenhang oder Umfeld. Die Kontextabgrenzung zeigt das Umfeld eines Systems sowie dessen Zusammenhang mit seiner Umwelt. In diesem Sinne ist die Kontextabgrenzung eine Vogelperspektive oder gesamthafte Übersicht.¹² Sie zeigt das System als Blackbox sowie dessen Verbindungen und Schnittstellen zur Umwelt. Es ist eine Sicht auf hoher Abstraktionsebene.

Fast alle Stakeholder benutzen diese Vogelperspektive als Überblick oder Systemlandkarte. Sie erleichtert das Verständnis der übrigen Architektursichten.

5.5.1 Elemente der Kontextabgrenzung

Kontextabgrenzungen zeigen

- das System als Blackbox, d. h. in einer Sicht von außen;
- die Schnittstellen zur Außenwelt, zu Anwendern, Betreibern und Fremdsystemen, inklusive der über diese Schnittstellen transportierten Daten oder Ressourcen;
- die technische Systemumgebung, Prozessoren, Kommunikationskanäle.

Die Kontextabgrenzungen stellen also eine Abstraktion der übrigen Sichten dar, jeweils mit dem Fokus auf den Zusammenhang oder das Umfeld des Systems.

¹² In der Literatur finden Sie auch folgende Bezeichnungen: „Kontextsicht“ [ISAQB] oder „Conceptual Architecture View“ [Hofmeister2000].

5.5.2 Notation der Kontextabgrenzung

Da Sie in der Kontextabgrenzung abstrakte Darstellungen der Baustein- und/oder Verteilungs-/Infrastruktursichten zeigen, können Sie auch deren jeweilige Notation verwenden:

- Schnittstellen zur Außenwelt zeigen Sie in Klassendiagrammen über Assoziationen zu anderen Systemen oder Akteuren.
- Die technische Systemumgebung zeigen Sie in Verteilungsdiagrammen.

5.5.3 Entwurf der Kontextabgrenzung

Im Idealfall erhalten Sie die Kontextabgrenzung als ein Ergebnis der Anforderungsanalyse – zumindest den fachlichen Teil davon.



Ich empfehle Ihnen, beim Entwurf der Kontextabgrenzung von einer fachlichen Sicht (Facharchitektur) auszugehen und auf dieser Basis eine Vogelperspektive des Systems zu entwerfen. Das minimiert den logischen Unterschied (*representational gap*) zwischen fachlichen und technischen Architekturmodellen.

Zeigen Sie in jedem Fall sämtliche¹³ (!) Nachbarsysteme, ohne Ausnahme. Alle ein- und ausgehenden Daten und Ereignisse müssen in der Kontextabgrenzung zu erkennen sein.

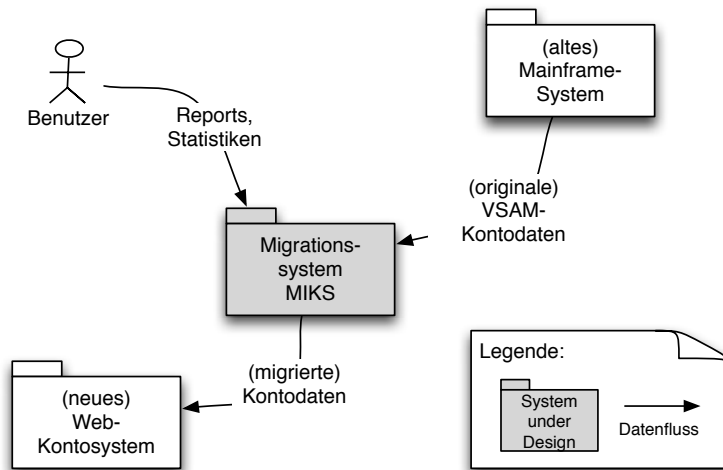


Bild 5.4 Beispiel: Diagramm einer Kontextabgrenzung

¹³ Wobei Sie natürlich abstrahieren dürfen, d. h. beispielsweise mehrere ähnliche externe Schnittstellen zusammenfassen.

Verwenden Sie Kontextabgrenzungen zur Kommunikation

Die Kontextabgrenzungen des Systems sowie die ersten Baustein- und Laufzeitsichten bieten Ihnen eine hervorragende Möglichkeit, mit anderen Stakeholder über das System zu diskutieren. Nutzen Sie diese Gelegenheit!



- Erläutern Sie Auftraggebern oder Kunden, wie das System die Anforderungen erfüllen soll (und hören Sie auf deren Rückmeldungen).
- Zeigen Sie (möglichst allen) Stakeholdern die Kontextabgrenzung und weisen Sie auf mögliche Risikofaktoren hin.
- Fragen Sie Entwickler nach deren Meinung. Ihr Entwurf befindet sich wahrscheinlich noch „weit weg von Technik“. Entwickler kennen häufig die Risiken und Tücken, von denen die Kontextabgrenzung abstrahiert!
- Sprechen Sie mit den künftigen Betreibern des Systems.
- Lassen Sie die Ergebnisse dieser Gespräche in die Entwürfe einfließen.

5.6 Bausteinsicht

Was sind Bausteine von Softwarearchitekturen?

Unter dem Begriff „Bausteine“ fasse ich sämtliche Software- oder Implementierungskomponenten zusammen. Bausteine repräsentieren (existierenden oder geplanten) Quellcode in verschiedenen Detaillierungsgraden. Dazu gehören Klassen, Prozeduren, Programme, Pakete, Komponenten oder Subsysteme. Bild 5.5 illustriert das in Form eines einfachen Metamodells.

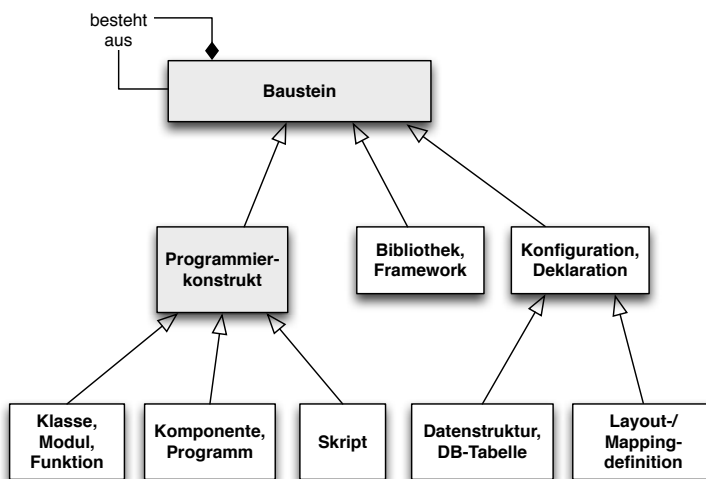


Bild 5.5 Metamodell von Bausteinen

Was zeigt die Bausteinsicht?

Die Bausteinsicht bildet die Aufgaben¹⁴ des Systems auf Softwarebausteine oder -komponenten ab. Diese Sicht macht Struktur und Zusammenhänge zwischen den Bausteinen der Architektur explizit. Bausteinsichten zeigen die Struktur des Quellcodes. In dieser Hinsicht entsprechen sie den konventionellen Implementierungsmodellen.

In der Bausteinsicht müssen Sie Funktionalitäten (Anwendungsfunktionalität, Kontrollfunktionalität sowie Kommunikationsaufgaben) und nichtfunktionale Anforderungen auf Architekturbausteine abbilden.¹⁵

Die Bausteinsicht beantwortet folgende Fragen:

- Aus welchen Komponenten, Paketen, Klassen, Subsystemen oder Partitionen besteht das System?
- Welche Abhängigkeiten bestehen zwischen diesen Bausteinen? Welcher Baustein muss welche Schnittstelle(n) implementieren?
- Welche Bausteine müssen Sie implementieren, konfigurieren oder kaufen, um die gewünschten Anforderungen zu erfüllen?

Ihre Adressaten sind alle an Entwurf, Erstellung und Test von Software beteiligten Personen, bei Bedarf Auditoren oder externe Bewerter.

Bausteinsichten zeigen zwei verschiedene Arten von Abstraktion, nämlich Black- und Whiteboxes:

Blackboxes sind ausschließlich durch ihre externen Schnittstellen und ihre Funktionalität beschrieben. Sie folgen dem Geheimnisprinzip: Ihr gesamtes Innenleben und somit ihre innere Komplexität bleiben verborgen.

- Whiteboxes sind *geöffnete* Blackboxes: Sie zeigen deren innere Struktur und Arbeitsweise. Whiteboxes bestehen ihrerseits wiederum aus einer Anzahl von Blackboxes.

Den Zusammenhang zwischen Black- und Whitebox zeigt Bild 5.6.

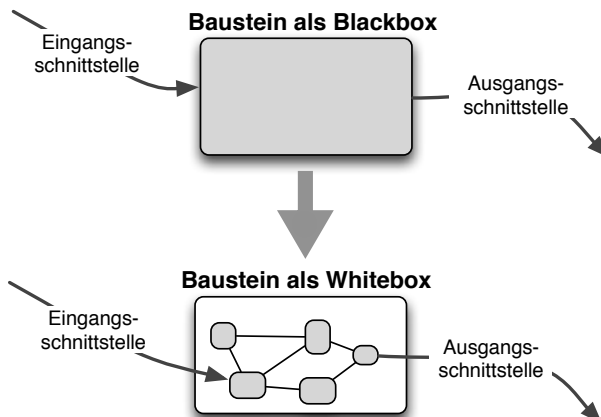


Bild 5.6
Zusammenhang
Black- und Whitebox

¹⁴ Während der Entwicklung zeigt die Bausteinsicht die gewünschte Struktur der Bausteine, beim fertigen System die vorhandene.

¹⁵ Daneben gibt es eine Reihe von Anforderungen, wie etwa Hochverfügbarkeit oder Clusterfähigkeit, die sich nicht ausschließlich durch systemeigene Bausteine umsetzen lassen, sondern Unterstützung durch die jeweilige Infrastruktur benötigen.

Verfeinern Sie Bausteinsichten Top-down

Sie können Blackboxes weiter verfeinern, indem Sie ihren „Inhalt“ als Whitebox zeigen. In Bausteinsichten zeigen Sie abwechselnd Blackbox- und Whitebox-Darstellungen. Hier stellen Sie Bausteine in Hierarchien oder Architekturebenen („Levels“) dar, wobei Sie den Detaillierungsgrad der Sichten schrittweise verfeinern.

Bild 5.7 zeigt diese Hierarchie von Verfeinerungen. Sie beginnt grundsätzlich mit der Kontextsicht, die das gesamte System als Blackbox darstellt. Die erste Verfeinerungsebene („Level 1“) stellt dann das Gesamtsystem als Whitebox dar.

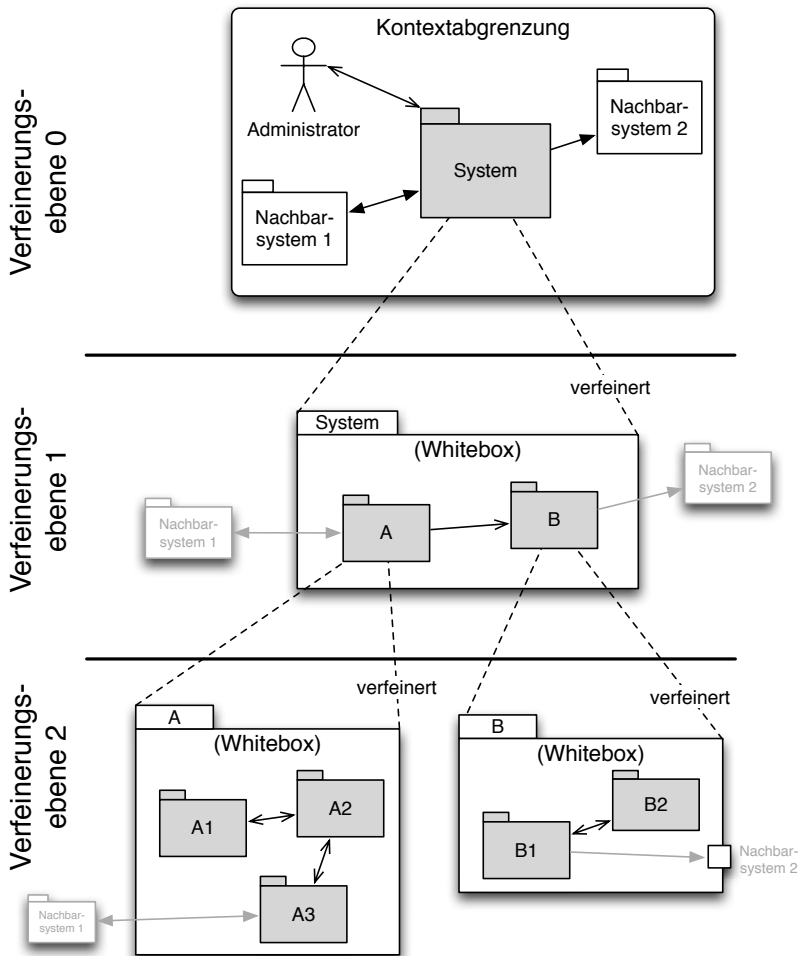


Bild 5.7 Hierarchie und Verfeinerung von Bausteinsichten

Nutzen Sie die Verfeinerung pragmatisch

Manchmal treten bei der schrittweisen Verfeinerung von Systemen Fälle auf, bei denen die oben gezeigte Art der Hierarchie schwierig oder umständlich ist. Wägen Sie in diesen Fällen Verständlichkeit, Wartbarkeit und Einfachheit ab. Im Zweifelsfall fragen Sie Ihre Leser!

Einige Beispiele solcher Situationen:

- Die Detaillierung einer Blackbox lässt sich durch Quellcode oder anderen Text leichter oder präziser beschreiben als durch Diagramme.
- Sie müssen mehrere Blackboxes gemeinsam verfeinern. Nutzen Sie in diesem Fall die Hierarchie im Sinne Ihrer Leser: Verfeinern Sie, was zusammengehört. In der folgenden Bild 5.8 sehen Sie die Blackboxes A und B aus Bild 5.7 gemeinsam in einer Whitebox.

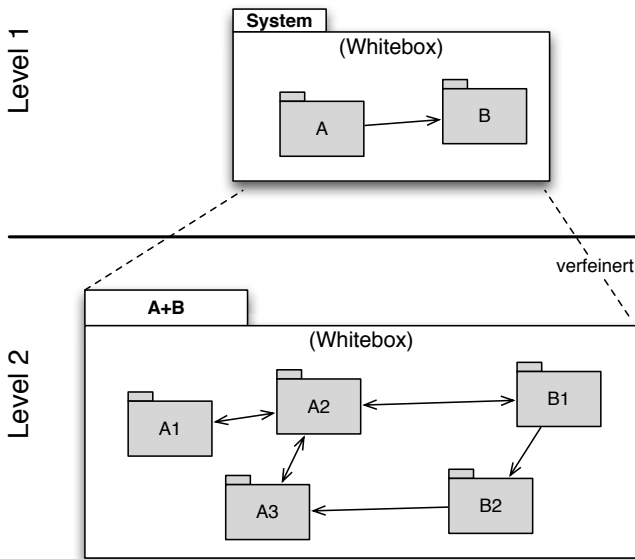


Bild 5.8

Gemeinsame Verfeinerung von Bausteinen

Verfeinern Sie redundanzfrei

Sie sollten bei der zunehmenden Detaillierung der Bausteinsicht auf wiederkehrende Muster oder Strukturen achten und die dadurch drohenden Redundanzen vermeiden: Dokumentieren Sie solche Muster nur einmal und verweisen Sie bei weiterem Auftreten jeweils auf diese Musterdokumentation.

Ein bekanntes Beispiel dafür ist der Zusammenhang von Domänenobjekten mit ihrer Benutzeroberfläche: Innerhalb eines GUI-Frameworks wird diese Struktur immer gleich aussehen – daher sollte sie auch nur einmal beschrieben werden.

In diesem konkreten Fall bietet sich meiner Meinung nach eine Kombination aus Diagramm und Quellcode zur Dokumentation an. Alternativ beschreiben Sie solche Sachverhalte als technisches Konzept (siehe Abschnitt 5.10).

5.6.1 Elemente der Bausteinsicht

Verwenden Sie für Bausteine (Implementierungskomponenten) die folgenden Elemente (Symbole) der UML – unabhängig davon, ob Sie Black- oder Whiteboxes darstellen:

- Klassensymbole bezeichnen einzelne Bausteine. Das können Klassen einer objektorientierten Programmiersprache sein, aber auch alle anderen ausführbaren Softwareeinheiten (Funktionen, Prozeduren, Programme).
- Komponenten bezeichnen ebenfalls einzelne Bausteine, bieten jedoch die Möglichkeit, die ein- und ausgehenden Schnittstellen genau zu beschreiben.
- Pakete stehen für Gruppen, Mengen oder Strukturen von Bausteinen. Diese Symbole zeigen, dass es sich um Abstraktionen handelt (die in der Architektur oder im detaillierten Entwurf weiter verfeinert werden).

Beschreiben Sie Verantwortlichkeiten und Schnittstellen von Blackboxes



Sie sollten zu jedem Blackbox-Baustein dessen spezifische Verantwortlichkeit oder Aufgabe in Kurzform dokumentieren

Zusätzlich beschreiben Sie die Ein- und Ausgabeschnittstellen der Blackboxes. Schnittstellen können verschiedene Ausprägungen annehmen, etwa: Aufruf/Rückgabe (*call-return*), synchron-asynchron, *push/pull* oder Benachrichtigung (*event notification*). Sie sollten die Semantik dieser Beziehungen angemessen dokumentieren:



- Falls Sie eine bestimmte benötigte Semantik mit den Symbolen der UML nicht direkt ausdrücken können, benutzen Sie Notizen oder verfeinern Sie die Darstellung durch zusätzliche Diagramme.
- Beschreiben Sie kritische Schnittstellen, etwa zu Nachbarsystemen, möglichst früh im Projekt und mit der gebotenen Genauigkeit für die Lesergruppe, so dass Überraschungen in der Zusammenarbeit mit Außenstehenden rechtzeitig entdeckt werden können.

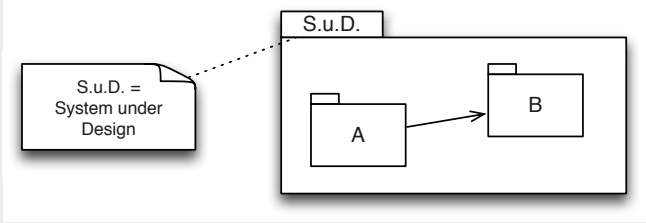
Weitere Hinweise und Vorschläge zur Dokumentation von Schnittstellen finden Sie in Abschnitt 5.9.

Ich empfehle Ihnen, zu Blackboxes den Zusammenhang zu den wichtigsten zugehörigen Codeartefakten ausdrücklich zu beschreiben – ein Verweis auf die entsprechenden Dateien oder Pakete genügt. Damit dokumentieren Sie, welche Teile des Quellcodes diese Blackbox implementieren. Beschränken Sie sich dabei auf die *wesentlichen* Teile und hüten Sie sich vor zu vielen Details – sonst wird die Pflege der Dokumentation zu teuer.

Beschreiben Sie die Struktur von Whiteboxes

Jede Whitebox-Beschreibung muss die innere Struktur eines Bausteins darstellen. Dabei helfen Diagramme und Modelle – ergänzt durch textuelle Erläuterung. Jede Whitebox-Beschreibung sollte die in Tabelle 5.4 beschriebenen Elemente umfassen.

Tabelle 5.4 Elemente von Whitebox-Beschreibungen

Überschrift	Inhalt
Übersichtsdiagramm	Ein Diagramm (UML-Paket- oder Klassendiagramm), das die innere Struktur dieser Whitebox zeigt. Ein Beispiel (Auszug aus Bild 5.7):  Bild 5.9 Beispiel: Whitebox-Darstellung
Lokale Bausteine	Tabelle oder Liste der lokalen Blackbox-Bausteine. Deren interne Struktur können Sie in einer weiteren Verfeinerungsebene darstellen.
Lokale Beziehungen	Tabelle oder Liste der Abhängigkeiten und Beziehungen zwischen den lokalen Bausteinen.
Entwurfs- entscheidungen	Gründe, die zu dieser Struktur geführt haben (oder zur Ablehnung von Alternativen).

Gemeinsamkeiten mit anderen Systemen ausnutzen



Bevor Sie sich an den Entwurf der Bausteinsicht begeben, prüfen Sie, ob Sie Teile dieses Systems aus anderen Quellen wiederverwenden können. Suchen Sie in anderen Projekten, in der Literatur und bei Anbietern von Softwarekomponenten nach Teilen, die Sie für Ihr aktuelles Projekt (wieder) verwenden können!

Suchen Sie dabei auch an den Stellen, die nicht unbedingt zum Kern Ihres konkreten Systems gehören, sondern allgemeine (und daher unauffällige) Basisdienste darstellen!

Selbst wenn diese Wiederverwendung nur kleine Teile des Systems betrifft, die Taktik des „Nicht-mehr-entwerfen-Müssens“ wirkt sich positiv aus!

5.6.2 Notation der Bausteinsicht

UML-Diagramme:
siehe Kapitel 6

Zur Notation der Bausteinsicht stehen Ihnen verschiedene Diagrammarten der UML zur Verfügung. Meine Empfehlung: Zeigen Sie diese Sicht in jedem Fall durch Komponentendiagramme, bei Bedarf durch Paket- oder Klassendiagramme unterstützt. UML-Klassen in diesen Diagrammen repräsentieren beliebige Bausteine oder Implementierungskomponenten.

Pakete symbolisieren Bausteine, die eine (komplexe) innere Struktur aufweisen, deren exakte Schnittstelle(n) Sie nicht genauer beschreiben wollen oder die keine physische Repräsentation zur Laufzeit besitzen. Sie stellen eine logische Kapselung oder Abstraktion dar.

UML-Komponenten sind Bausteine, für die Schnittstellenbildung und Kapselung wichtig sind. Zur Laufzeit wird es immer mindestens eine Instanz von Komponenten geben. Komponenten sind (mittlerweile) mein favorisiertes Konstrukt der Bausteinsicht, weil ich bei Bedarf beliebig Details – etwa zu Schnittstellen – hinzufügen kann.

5.6.3 Entwurf der Bausteinsicht

Beim Entwurf der Bausteinsicht sind Sie als Architekt sozusagen beim Kern des eigentlichen Problems angelangt. Jetzt müssen Sie exakt beschreiben, wie das System (strukturell) aufgebaut ist und aus welchen Bausteinen es besteht.

Beginnen Sie¹⁶ den Entwurf mit der Kontextabgrenzung, der *Vogelperspektive* der Implementierungsbausteine. Jetzt gilt es, kleinere Architekturelemente wie Sub- oder Teilsysteme, Komponenten oder sogar atomare Bestandteile zu finden. Mittlerweile sind Sie mit allerlei Rüstzeug gewappnet, diese Herausforderung zu meistern!



- Sie haben auf der Basis einer „Systemidee“ einen „ersten Wurf“ des Systems vor Augen. Wenn nicht: Unternehmen Sie einen (literarischen) Ausflug in Kapitel 3 (Vorgehen bei der Architekturentwicklung) und kehren Sie anschließend hierher zurück.
- Sie berücksichtigen Einflussfaktoren und Randbedingungen bei der Definition von Bausteinen. Insbesondere kennen Sie:
 - die technische Infrastruktur: Wo und wie muss das System ablaufen (Rechner, Betriebssysteme, Netzwerke)?
 - die technischen Einflussfaktoren (Programmiersprachen, Middleware, Datenbank, Transaktionsmonitor, Entwicklungsumgebung)
 - die organisatorischen Einflussfaktoren: Über welche Erfahrung verfügt das Team?
 - die Qualitätsanforderungen: Welche Performance und Zuverlässigkeit muss das System erreichen, welche Ansprüche bestehen hinsichtlich Änderbarkeit und so weiter? (Siehe dazu auch Kapitel 3, Vorgehen bei der Architekturentwicklung.)
- Sie können beim Entwurf der obersten Abstraktionsebene der Bausteinsichten von Elementen oder Begriffen der Fachdomäne starten.
- Sie können Architekturmuster oder Referenzarchitekturen einsetzen. Einige Hinweise dazu erhalten Sie in Kapitel 4 (Entwurf).

Folgende Zerlegungsstrategien könnten Ihnen nützlich sein:

- Zerlegung nach fachlichen oder funktionalen Gesichtspunkten (= Domain-Driven Design, Zerlegung nach Anwendungsfällen)

¹⁶ Ich finde es ideal, Entwürfe im Team zu diskutieren, um frühzeitig Feedback zu Entscheidungen zu bekommen.

- Zerlegung gemäß der Verteilungssicht, Hardware oder anderer technischer Infrastruktur (etwa: Bausteine für mobile Clients, Browser-Clients und/oder Server)
- Zerlegung gemäß organisatorischer Verantwortungen (z. B. analog der Struktur der Entwicklungsorganisation)
- Zerlegung gemäß vorhandenen Technologien, Frameworks, Programmiersprachen oder eingesetzter Produkte
- Zerlegung gemäß wichtiger Qualitätsanforderungen (etwa: Ein Baustein muss im Cluster laufen, während andere Bausteine das nicht müssen)
- Manche querschnittlichen oder technischen Konzepte (Regeln, Prinzipien) können bestimmte Bausteine oder Bausteinstrukturen notwendig machen.

■ 5.7 Laufzeitsicht

Alias: Ausführungssicht

Die Laufzeitsicht beschreibt, welche Bestandteile des Systems zur Laufzeit existieren und wie sie zusammenwirken. Dabei kommen wichtige Aspekte des Systembetriebs ins Spiel, die beispielsweise den Systemstart, die Laufzeitkonfiguration oder die Administration des Systems betreffen.

Primär dokumentiert die Laufzeitsicht, wie das System zur Laufzeit seine wesentlichen Aufgaben (= Funktionen, Use-Cases, Abläufe) ausführt.

Dokumentieren Sie das System zur Laufzeit

Neben der internen Struktur von Softwaresystemen sollten Sie beschreiben, welche Komponenten zur Laufzeit existieren, wie diese Komponenten zusammengesetzt sind und wie sie ablaufen.



Beispiel: Die Fachlogik eines Systems im industriellen Bereich besteht aus einem Paket von nur drei Klassen. Zur Laufzeit wird das System durch einen komplexen Konfigurationsprozess für jeden angeschlossenen Client als eigenständiger Betriebssystemprozess gestartet und durch einen Monitorprozess kontinuierlich überwacht. Die Konfigurations- und Administrationskomponenten stellen einen großen Teil der Laufzeitsicht dar. Sie müssen auch in Bausteinsichten erscheinen. Möglicherweise (da es sich um vorgefertigte Komponenten handelt) werden in der Bausteinsicht nur ihre Schnittstellen beschrieben.

Dokumentieren Sie spezielle Aspekte der Laufzeit:

- Wie arbeiten die Systemkomponenten zur Laufzeit zusammen?
- Wie werden die wichtigsten Use-Cases durch die Architekturbausteine bearbeitet?

- Welche Instanzen von Architekturbausteinen gibt es zur Laufzeit und wie werden diese gestartet, überwacht und beendet?
- Wie arbeiten Systemkomponenten mit externen und vorhandenen Komponenten zusammen?
- Wie startet das System (etwa: notwendige Startskripte, Abhängigkeiten von externen Subsystemen, Datenbanken, Kommunikationssystemen etc.)?

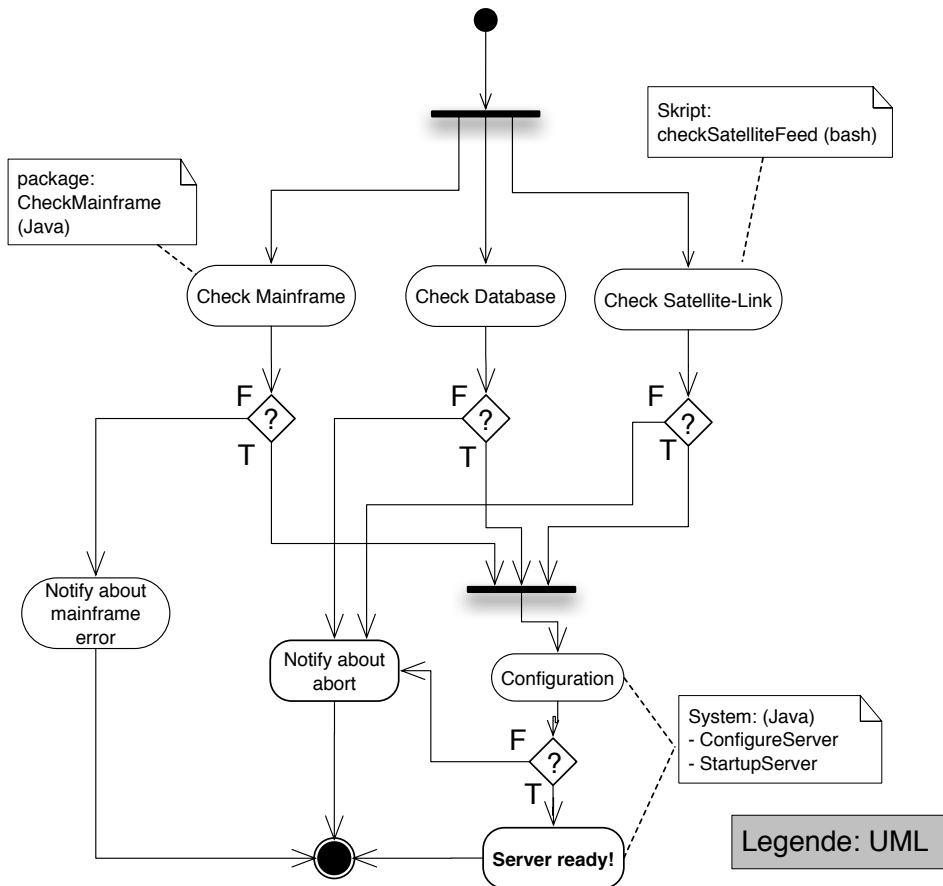


Bild 5.10 Beispiel einer Laufzeitsicht (Systemstart)

5.7.1 Elemente der Laufzeitsicht

Elemente der Laufzeitsicht sind ausführbare Einheiten sowie deren Beziehungen zueinander:

- Instanzen von Implementierungsbausteinen – alle diejenigen Architekturbausteine, von denen zur Laufzeit Exemplare erzeugt werden. Bitte beachten Sie, dass dies nicht unbedingt für alle Architekturbausteine gilt. Denken Sie an abstrakte Klassen oder logische Interfaces. Laufzeitelemente sind nur diejenigen, die als Einheit ausgeführt oder aufgerufen werden.
- Beziehungen zwischen den Laufzeitelementen – das können sowohl Daten- als auch Kontrollflüsse sein.

5.7.2 Notation der Laufzeitsicht

Dokumentieren Sie die Laufzeitsicht mit UML-Sequenz-, Aktivitäts- oder Kollaborations-/Kommunikationsdiagrammen. In Bild 5.11 zeigt ein Sequenzdiagramm das Zusammenwirken einiger Bausteine der Kundendaten-Migration MIKS.

Gerade bei den Sequenzdiagrammen sollten Sie die UML pragmatisch einsetzen: Anstelle der üblichen Objektsymbole können Sie bei Bedarf auch Komponenten-, Paket- oder andere Symbole verwenden. Dazu finden Sie ein Beispiel in Bild 5.11 (Komponente MigrationDatabase) und Bild 5.12.

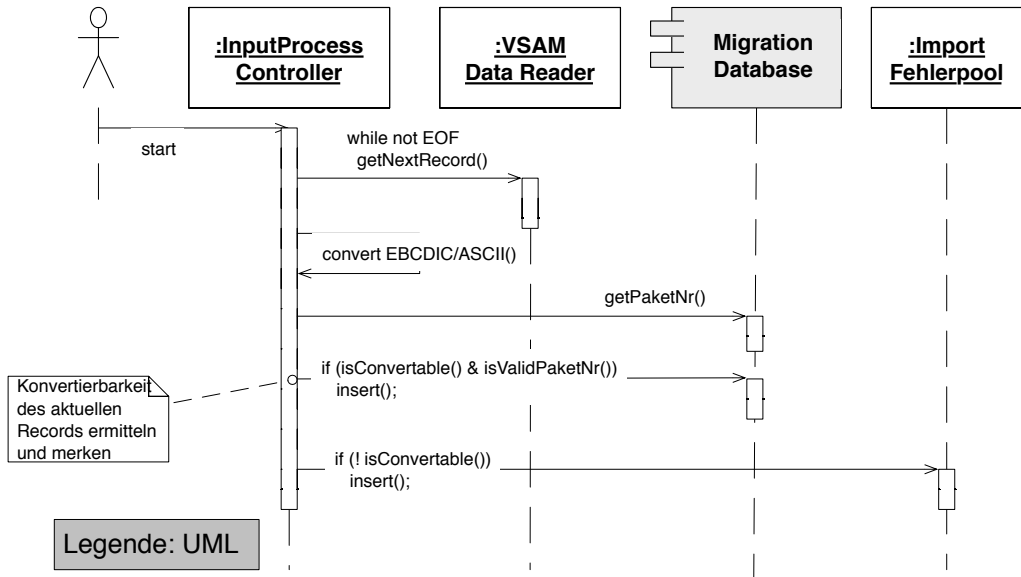


Bild 5.11 Beispiel: Laufzeitsicht MIKS Input-processing

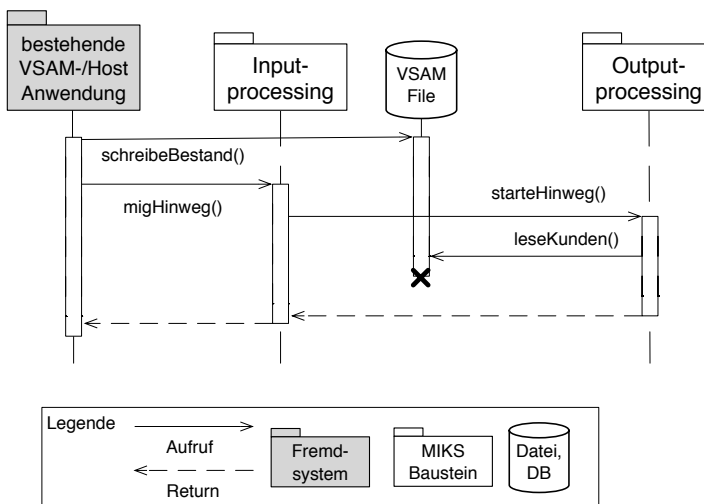


Bild 5.12
Beispiel:
Laufzeitsicht
Top-Level Use-Case
MIKS

Eine Laufzeitsicht als Aktivitätsdiagramm zeigt Bild 5.10. Dort sehen Sie den Startvorgang eines Systems, bei dem zur Laufzeit vor der Konfiguration und dem Start des eigentlichen Systems die Verfügbarkeit einiger externer Subsysteme geprüft wird. Als UML-Notiz sind außerdem die Namen einiger beteiligter Skripte und Pakete genannt, die nicht Bestandteil des eigentlichen Systems sind.

**Tipp:**

Zur Beschreibung von Abläufen können Sie auch nummerierte Listen oder Quellcode verwenden, sofern er für Ihre Leser gut verständlich ist und – meine persönliche Regel – der entsprechende Text auf eine einzige Seite passt.

5.7.3 Entwurf der Laufzeitsicht



Beachten Sie bei der Dokumentation der Laufzeitsicht auch die Informationsbedürfnisse der Betreiber und Administratoren des Systems, nicht nur die von Managern und Entwicklern.

Elemente der Laufzeitsichten sind Instanzen der (statischen) Architekturbausteine, die Sie in den Bausteinsichten dokumentieren. Ein möglicher Weg zur Laufzeitsicht führt daher über die Bausteinsichten: Beschreiben Sie die Dynamik der statischen Bausteine, beginnend bei den wichtigsten Use-Cases des Gesamtsystems. Eventuell hilft Ihnen dabei auch die Kontextsicht, die Sie in Abschnitt 5.5 kennengelernt haben.

Ein anderer Weg besteht darin, die Laufzeitsicht auf Basis der Verteilungs-/Infrastruktursicht (siehe folgender Abschnitt) zu entwickeln.

5.8 Verteilungssicht

Alias: Infrastruktursicht

Die Verteilungs- oder Infrastruktursicht verfolgt zwei Intentionen:

1. Abbildung der technischen Ablaufumgebung des Systems in Form von Hardwarekomponenten (Prozessoren, Speicher, Netzwerke, Router und Firewalls);
2. Abbildung (= Deployment, Verteilung, Installation) Ihrer Softwarebausteine auf die Hardware.

Technische Infrastruktur

Sie können in der Infrastruktursicht die Leistungsdaten und Parameter der beteiligten Elemente darstellen, wie etwa Speichergrößen, Kapazitäten oder Mengengerüste. Außerdem können Sie zusätzlich die Betriebssysteme oder andere externe Systeme aufnehmen.

Die Verteilungssicht ist von großer Bedeutung für die Betreiber des Systems, die Hardwarearchitekten, das Entwicklungsteam sowie Management und Projektleitung.

5.8.1 Elemente der Verteilungssicht

In der Verteilungssicht beschreiben Sie folgende Elemente:

- Bestandteile der technischen Infrastruktur, sogenannte Knoten. Dies sind entweder echte oder virtuelle Rechner oder Prozessoren, aber auch sonstige Hardware (Firewalls, Router, Speicher oder anderes).
- Laufzeitelemente, auch Laufzeitartefakte genannt (d. h. Instanzen von Bausteinen, siehe Abschnitt 5.6), die auf Knoten ablaufen (und dort installiert, neudeutsch *deployed*, werden).
- Kanäle sind Verbindungen zwischen Knoten (physische Kanäle) beziehungsweise zwischen Laufzeitelementen (logische Kanäle). Logische Kanäle müssen immer über physische Kanäle realisiert werden.

5.8.2 Notation der Verteilungssicht

Deployment-Diagramme

Nutzen Sie für die Verteilungs-/Infrastruktursicht UML-Einsatzdiagramme (*deployment diagrams*). Sie können Knoten für beliebige technische Elemente verwenden, Komponenten- und Paketsymbole für die Laufzeitelemente (Softwaresysteme) sowie UML-Beziehungen für die physischen Verbindungen zwischen den Knoten oder Laufzeitelementen.

Ein Beispiel finden Sie in Bild 5.13.

Das Mapping „Baustein-auf-Knoten“ ist tabellarisch eventuell einfacher als grafisch – möglicherweise haben Sie das ohnehin in Ihrem Build-Skript beschrieben.

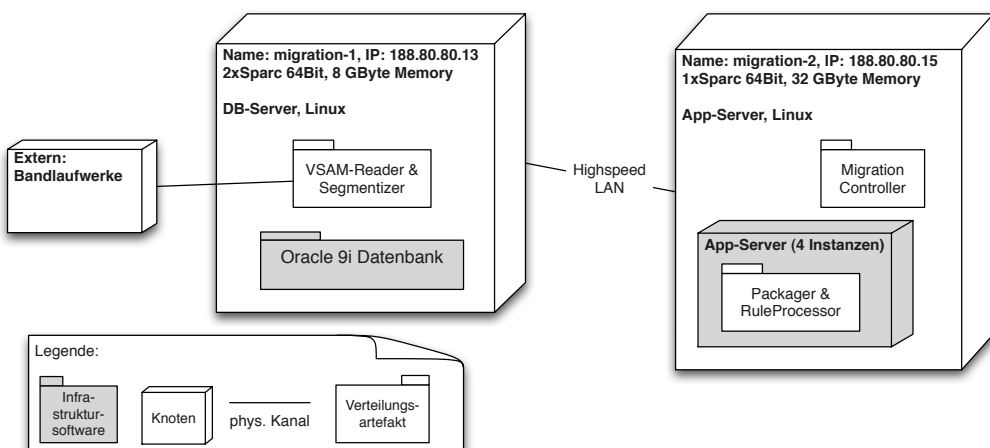


Bild 5.13 Beispiel für Verteilungssicht

Noch ein ergänzendes Wort zu den Kanälen: Für manche Stakeholder sind die logischen Kanäle wichtiger als die physischen oder umgekehrt. Erfragen Sie, welche Bedürfnisse Ihre Stakeholder für die Verteilungssicht genau haben. Für Entwickler sind oftmals die logischen interessanter, für Betreiber und Manager die physischen.

5.8.3 Entwurf der Verteilungssicht



- Ein Modell der Infrastruktursicht sollte eine Landkarte der beteiligten Hardware und der externen Systeme sein. Diese Information sollte sämtlichen Stakeholdern zur Verfügung stehen.
- Gleichen Sie die Infrastruktursicht mit den Mengengerüsten der beteiligten Daten ab. Genügen die verfügbare Hardware und die Kommunikationskanäle oder gibt es potenzielle Engpässe?
- Sie können Mengengerüste und Datenvolumina mit der Infrastruktursicht überlagern, um potenzielle Engpässe zu erkennen.
- Falls Ihre Systeme in verteilten oder heterogenen Umgebungen ablaufen, sollten Sie vorhandene Kommunikationsmechanismen, Protokolle und Middleware in die Infrastruktursicht aufnehmen. Das ermöglicht einen Abgleich mit der Bau-sicht.
- In heterogenen und verteilten Systemlandschaften sollten Sie eine detaillierte Sicht der technischen Infrastruktur erstellen. Diese sollte mindestens folgende Aspekte berücksichtigen: Netztopologie und Protokolle, real verfügbare Netzkapazitäten (maximale und mittlere Werte), Anzahl und Leistung der vorhandenen Prozessoren sowie die Speicherkapazitäten der beteiligten Rechner.
- Berücksichtigen Sie die realen Verfügbarkeiten der technischen Komponenten. Teilweise sind Komponenten durch nächtliche Datensicherungen, Batch-Läufe oder Wartungsarbeiten regelmäßig nicht verfügbar. Dokumentieren Sie diese Einschränkungen.
- Oftmals ist die „Verpackung“ von Implementierungsbausteinen in Deployment-Artefakte eine aufwendige oder komplexe Aufgabe, die hoffentlich Ihr Build-Prozess automatisiert. Bei Bedarf übernehmen Sie die Zuordnung von Bausteinen zu Deployment-Artefakten in Ihre Architekturdokumentation oder Sie verweisen auf Ihre Build-Skripte.

■ 5.9 Dokumentation von Schnittstellen

Schnittstellen besitzen als Abgrenzungen zwischen den verschiedenen Bausteinen von Systemen besondere Bedeutung: Über Schnittstellen arbeiten diese zusammen und erzeugen letztlich den Mehrwert des Gesamtsystems.

Schnittstellen beinhalten in der Regel den Transfer von Daten oder Steuerungsinformationen. Manche Schnittstellen basieren auf Standardkonstrukten der eingesetzten Programmiersprachen (Methoden- oder Prozeduraufrufe, Remote Procedure Calls, Shared Memory etc.). Andere Schnittstellen sind organisatorischer Art.

Viele Schnittstellen werden bereits als Bestandteile von Sichten dokumentiert, insbesondere in der Bausteinsicht. Häufig benötigen manche Stakeholder eigenständige Schnittstellendokumentationen – insbesondere für externe Schnittstellen.

Struktur von Schnittstellenbeschreibungen

Wie auch bei der Beschreibung von Sichten sollten Sie bei der Beschreibung von Schnittstellen eine einheitliche Gliederung verwenden.

Nachfolgend bezeichnet der Begriff *Ressource*¹⁷ jeglichen Baustein einer Softwarearchitektur, der eine Interaktion über die betroffene Schnittstelle anbietet.



Ressourcen – ein überladener Begriff

Allgemein gesprochen ist eine Ressource ein Arbeitsmittel. In der Softwarearchitektur verwenden wir den Begriff in zwei leicht unterschiedlichen Ausprägungen:

- Die unspezifische Verwendung (die ich im Folgenden auch für die Schnittstellenbeschreibung benutze) subsumiert einen beliebigen Datenstrom, ohne weitere Einschränkung.
- Der REST-Architekturstil (representational state transfer) verwendet Ressourcen als Bestandteile von (Online-)Interaktionen und differenziert strikt zwischen Ressourcen und deren Repräsentationen.

Siehe <http://www.w3.org/TR/webarch>

¹⁷ Das Konzept der ressourcenbasierten Schnittstellenbeschreibungen stammt aus [Clements+03].

Tabelle 5.5 Struktur von Schnittstellenbeschreibungen

Überschrift	Inhalt
Identifikation	Genaue Bezeichnung und Version der Schnittstelle
Beispiele	Beispiele zur Benutzung dieser Schnittstelle; Quellcode, etwa in Form von Unit-Tests ¹⁸
Bereitgestellte Ressourcen	Welche Ressourcen stellt dieses Element für Ihre Akteure (Benutzer, Aufrufer) bereit? Sie müssen hier mehrere Aspekte dokumentieren: ▪ Syntax der Ressource, etwa die Signatur von Methoden, das API ▪ Semantik der Ressource: Welche Auswirkungen hat ein Aufruf dieser Ressource? Hier gibt es mehrere Möglichkeiten: ▪ Welche Events werden ausgelöst? ▪ Welche Daten werden (wie) geändert? ▪ Welche Zustände ändern sich? ▪ sonstige von Menschen oder anderen Systemen wahrnehmbare Nebenwirkungen ▪ Restriktionen bei der Benutzung der Ressource
Fehlerszenarien	Beschreiben Sie sowohl mögliche Fehlersituationen als auch deren Behandlung.
Qualitätseigenschaften	Welche Qualitätseigenschaften wie Verfügbarkeit, Performance, Sicherheit gelten für diese Schnittstelle? Die neudeutsche Bezeichnung dieses Teils der Schnittstellenbeschreibung lautet Quality-of-Service (QoS) Requirements.
Entwurfsentscheidungen	Welche Gründe haben zum Entwurf dieser Schnittstelle geführt? Welche Alternativen gibt es und warum wurden diese verworfen?

Schnittstellen in UML beschreiben

Bei der UML-Darstellung von Bausteinen haben Sie eine Reihe von Optionen (siehe Bild 5.14) für Schnittstellen – nachfolgend in der Reihenfolge der „Gründlichkeit“ und des damit verbundenen Aufwands erklärt:

- Eine benannte und/oder gerichtete Verbindung zweier Bausteine sagt lediglich aus, dass es überhaupt eine Schnittstelle gibt.
- Ein Port-Symbol (kleines Viereck am Rande einer Komponente) gibt an, dass ein Baustein eine Schnittstelle besitzt – differenziert nicht zwischen Anbieten und Benötigen.
- Ein Baustein referenziert einen Port: Dieser Baustein (intern in Bezug auf die umgebende Komponente) realisiert oder benötigt diese Schnittstelle.
- Explizite Angabe von angebotener („Ball“) und benötigter („Socket“) Schnittstelle – mit (optional benannter) Beziehung dazwischen. Hierbei ist die Schnittstelle ein eigenständiges Modellierungselement mit (beliebig genauer) Beschreibung.
- Explizite Ball- und Socket-Notation in Kombination mit Ports: Hier können Sie bei Bedarf sogar die Schnittstellen noch Kommunikationswegen oder Technologien zuordnen.

¹⁸ Ein großartiges Vorbild ist die JOptSimple-Library, deren API hervorragend durch Unit-Tests beschrieben ist.

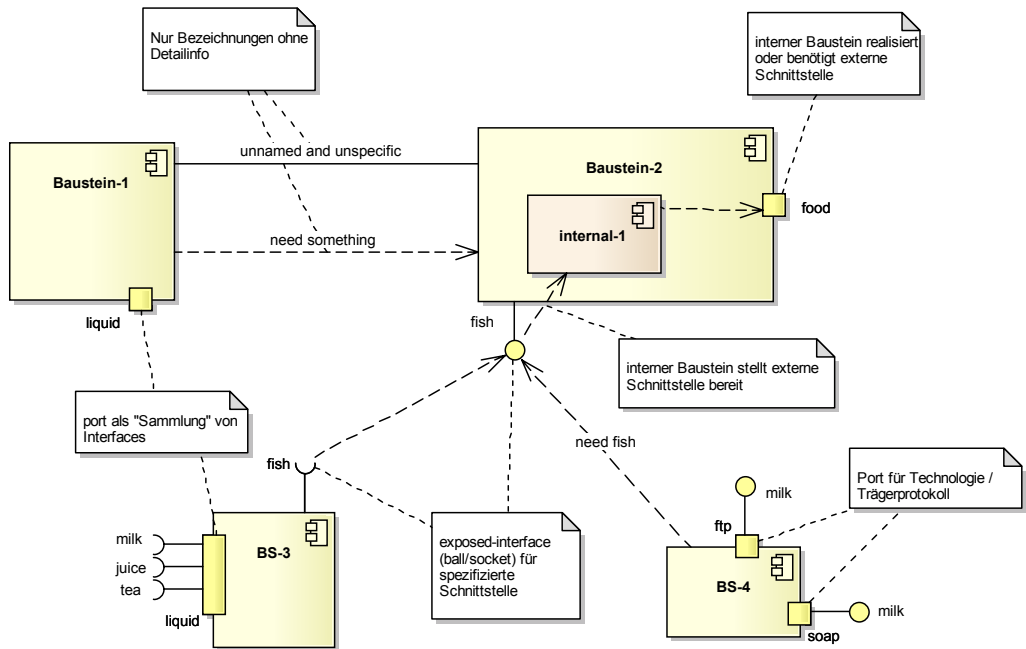


Bild 5.14 Optionen für Schnittstellen in UML



Einige Tipps zur Modellierung von Schnittstellen mit UML:

- Beginnen Sie sparsam mit der gerichteten und benannten Verbindung von Bausteinen. Nur wenn Sie mehr Details benötigen, führen Sie Ball-/Socket-Elemente ein.
- Falls mehrere Teams unabhängig voneinander an Bausteinen arbeiten, die eine gemeinsame Schnittstelle besitzen: Machen Sie diese Schnittstelle zu einem „first-order-citizen“ und investieren Sie in „exakte“ Darstellung. Verwenden Sie (mindestens) Ball-/Socket-Notation.

Ergänzen Sie UML-Dokumentation an komplexen oder aufwendigen Schnittstellen immer um Beispielcode, der die Initialisierung und Benutzung der Schnittstelle genau aufzeigt.

■ 5.10 Dokumentation technischer Konzepte

Technische Konzepte bilden einen Grundpfeiler der konzeptionellen Integrität: Sie zeigen die Lösung wiederkehrender Aufgaben und sorgen dafür, dass „gleiche Dinge gleich gelöst“¹⁹ werden.

An dieser Stelle geht es um die Dokumentation, Beschreibung oder Darstellung solcher Konzepte. Dafür empfehle ich Ihnen folgende Gliederung (die Sie bei Bedarf auch leicht kürzen dürfen):

- **Aufgabenstellung, Anforderungen:** Skizzieren Sie die (wiederkehrende) Aufgabe möglichst genau, beschreiben Sie die Anforderungen an das Konzept (= die Lösung).
- Welche **Randbedingungen** und **Einschränkungen** muss das Konzept berücksichtigen?
- Lösungsansatz: Zeigen Sie Beispiele der Lösung, am besten mit Original-Quellcode („Referenzimplementierung“). Erklären Sie, falls nötig, zugehörige Strukturen und Abläufe.
- Geben Sie **Referenzen** an – denn gute Quellen zitieren ist weniger Arbeit, als dieselben Sachverhalte selbst zu dokumentieren.
- Weisen Sie ausdrücklich auf mögliche **Risiken** der Lösung hin, auf Kompromisse oder mögliche Nebenwirkungen.
- Schließlich können Sie **Alternativen** benennen und begründen, warum diese Alternativen nicht verwendet werden.



Tipp:

Beispiele erweisen sich in der Praxis als kompakte und hilfreiche Dokumentation technischer Konzepte. Insbesondere Unit- oder Integrationstests können Entwicklern schnell und präzise Auskünfte geben.

■ 5.11 Werkzeuge zur Dokumentation

Egal, ob Sie visuell (= mit Diagrammen oder Bildern) oder verbal (= mit Worten) dokumentieren – für Erstellung, Pflege und Organisation müssen Sie Werkzeuge einsetzen.

Ich möchte Ihnen einige typische Kategorien vorstellen, anstatt für konkrete Vertreter (Schleich-)Werbung zu machen.

¹⁹ Diese Formulierung stammt von Stefan Zörner aus [Zörner-15].

(UML) Modellierungswerkzeuge	
+ effiziente Erstellung und Pflege von Diagrammen und Modellen	▪ schlecht geeignet für Text und Tabellen
+ gute Navigierbarkeit in Modellen	▪ schlechte Integration in Code-Repositories
	▪ oft unbrauchbare Reportgeneratoren
Textwerkzeuge („Office-Werkzeuge“)	
+ universelle Verfügbarkeit	▪ schlechte Mehrbenutzerfähigkeit
+ gute Integration von Text, Tabellen und Diagrammen	▪ schwierig für große Dokumente
	▪ keine Erstellung von Diagrammen
Plaintext-Werkzeuge (z. B. AsciiDoc oder Markdown)	
+ Docs-as-Code	▪ nur für Entwickler – andere Stakeholder möchten üblicherweise keinen Plaintext bearbeiten
+ Entwicklerfreundlich	▪ Kein WYSIWYG
+ gute Basis für Generierung diverser Ausgabeformate	
+ modularisierbar, sehr flexibel	
Wiki	
+ sehr gute Mehrbenutzerfähigkeit	▪ (kaum/keine) Offline-Fähigkeit
+ gute Suche/Navigierbarkeit	▪ Integration von Diagrammen (teilweise) aufwendig
+ (oft) Integration mit Issue-Tracker	
Zeichenwerkzeuge	
+ optisch attraktive Resultate	▪ zeichnen statt modellieren
+ hohe Flexibilität in Darstellung	▪ (formal) exakte Modellierung nicht möglich

Mögliche Auswahlkriterien für Werkzeuge

Hier möchte ich Sie lediglich auf einige Ideen bringen, welche Kriterien für die Auswahl von Werkzeugen relevant sein könnten (bzw. in meinen Projekten einmal waren):

- Einfachheit der Benutzung: Erstellen, Ändern, Finden und Lesen, Volltextsuche, intuitive Benutzbarkeit ohne Handbuch
- Verfügbarkeit im Team: Nutzung ohne großen Aufwand, parallele Benutzung durch mehrere Personen gleichzeitig, für mehrere Betriebssysteme verfügbar
- Integration mit Code-Repository, Merge-Fähigkeit von Inhalten, Versionierung sowie Vergabe von Tags/Labels analog zum Quellcode
- Robustheit und Ausfallsicherheit, sichere Ablage sämtlicher Daten (und Metadaten)
- Rechte- und Rollenkonzept, Zugriffsschutz, Differenzierung von Benutzergruppen mit unterschiedlichen Schreib- und Leserechten auf bestimmte Teile der Dokumentation
- Automatisierbarkeit, Generierung benötigter Artefakte (etwa: pdf-Reports), Integration in Daily-Build, offene Datenformate
- Lizenz- und Betriebskosten, Lizenzmodell (shared- oder floating-Lizenz, Einzelplatz-Lizenz, Open-Source-Lizenz, Wartungs- oder Update-Kosten)
- Offline-Fähigkeit: Nutzung auch ohne robuste Netzverbindung

- Review- oder Kommentierungsfähigkeit, Änderungs- oder Vorschlagsmodus
- Compliance zu Metamodell (etwa: UML-Metamodell)

Sie sehen schon – viele Faktoren könnten Ihre Werkzeugauswahl beeinflussen. Beginnen Sie „klein“ und lassen Sie niemals eine Dokumentation an Werkzeugen scheitern: Einfache Werkzeuge (Papier, Plain-Text) funktionieren auch!

Welches Werkzeug verwenden?

Für kleine und mittlere Systeme eher geringer Kritikalität (d. h. insbesondere ohne Safety-Anforderungen) nutzen Sie ein Wiki oder Plaintext-Ansätze wie Markdown/AsciiDoc. Diagramme zeichnen Sie mit einfachen Grafikwerkzeugen oder entsprechenden Wiki-Plug-ins. Siehe beispielsweise [doctoolchain] oder [htmlsc].

Bei etwas größeren oder kritischen Systemen nutzen Sie Modellierungswerkzeuge für Diagramme, und Wiki oder Textwerkzeuge für den Rest. Sequenzdiagramme können Sie z. B. mit PlantUML (<http://plantuml.com>) in einer textuellen Notation beschreiben und die Grafik automatisch erzeugen lassen.

Ich persönlich mag die Kombination aus Wiki und Modellierungswerkzeug: Die textlastigen Teile (Erklärungen, Begründungen, Konzepte und jegliche Tabellen) stehen im Wiki, lediglich Diagramme pflege ich in einem UML-Werkzeug. Diese übernehme ich als jpg-Kopie ins Wiki (ja – das ist manueller Aufwand!)²⁰.

Mit jedem Release der Software generiere ich aus dem Wiki ein PDF, das ich zusammen mit dem Sourcecode des Systems unter Versionsverwaltung halte. Damit ist der Stand der Architekturdokumentation mit jeder „offiziellen“ Version der Software dauerhaft zugreifbar.

■ 5.12 TOGAF zur Architekturdokumentation

TOGAF („The Open Group Architecture Framework“) ist ein generisches und sehr flexibles Rahmenwerk (Framework) für die Entwicklung und das Management von Unternehmensarchitekturen. Es enthält eine Methode zur Architekturentwicklung („ADM“), die Aktivitäten und Prinzipien zur Entwicklung von Unternehmensarchitekturen umfasst. Außerdem definiert TOGAF folgende vier Typen von Architekturen:

- Business Architecture: definiert Geschäftsstrategie, IT-Governance und wesentliche Geschäftsprozesse.
- Information (oder Data) Architecture: beschreibt die Struktur logischer und physischer Unternehmensdaten sowie die Ressourcen zu deren Management.
- Application Architecture: enthält das Mapping zwischen Anwendungen und den in der Business Architecture definierten Geschäftsprozessen.
- Technical Architecture: beschreibt die zum Betrieb der Anwendungen nötige Hard- und Softwareausstattung.

²⁰ Oftmals komme ich mit einem Dutzend Diagramme aus – und die sind ziemlich grobgranular, d. h. brauchen nur selten geändert zu werden.

Der Entwicklungsprozess ADM adressiert grundsätzlich diese Architekturtypen, bedarf jedoch der organisationsspezifischen Anpassung. In Bild 5.15 sehen Sie einen Überblick (nach [OpenGroup 09]).

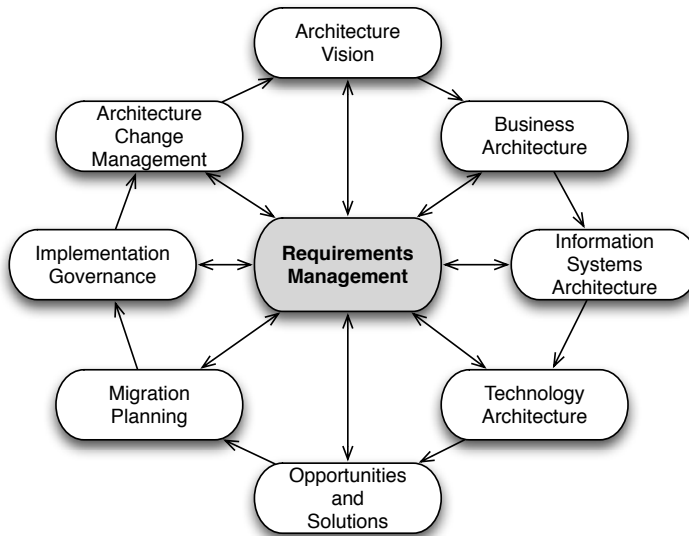


Bild 5.15
TOGAF Architecture
Development Method

Wenn Sie sich jetzt immer noch nicht vorstellen können, wie Sie ein Stück Software mit TOGAF modellieren, stehen Sie nicht alleine da: Meiner Meinung nach eignet sich TOGAF aufgrund seiner größtenteils sehr abstrakten Konzepte kaum zur Modellierung konkreter Softwaresysteme, sondern spielt seine Stärken vielmehr bei der Entwicklung und Modellierung der Unternehmens-IT-Architekturen aus – von denen Sie in Kapitel 11 noch einiges lesen können. Als eindeutigen Pluspunkt sehe ich bei TOGAF eine generische, aber klare Begriffsbildung von Architekturen und (Enterprise-IT-)Architekturaufgaben.

Falls Sie jedoch für praktische Softwareentwicklung methodische Unterstützung suchen, rate ich von TOGAF ab: Die spezifische Konfiguration von dessen Metamodell sowie die Umsetzung der zugehörigen Prozesse erfordern meines Erachtens mehr Zeit, als die meisten Entwicklungsprojekte aufbringen können – überlassen Sie das Ihrem IT-Vorstand oder Enterprise-IT-Architekten.

■ 5.13 Weiterführende Literatur



[arc42] ist eine Sammlung praktischer Hilfsmittel für Softwarearchitekturen. Unter anderem finden Sie auf der Website ein Template für pragmatische Architekturdokumentation. Ausführliche Info und zahlreiche Tipps zur Anwendung finden Sie unter <https://docs.arc42.org> (siehe Tipp unten).



Bei [arc42ByExample] ist der Name Programm: Dieses eBook zeigt die Architekturdokumentation von sechs realen Systemen mittels arc42. Online: <https://leanpub.com/arc42byexample> [Hargis+04] erläutern Grundprinzipien technischer Dokumentation. Geschrieben von erfahrenen technischen Redakteuren. Lesenswert, falls Sie öfter dokumentieren.

[Kruchten95] stellt ein Modell von fünf Sichten vor: Logische Sicht, Prozess-Sicht, Physische Sicht, Entwicklungssicht und Use-Case-Sicht. Leider hat Phillipe die technischen Konzepte und Entwurfsentscheidungen in seiner Darstellung weggelassen.

[Zörner15] geht auf alle praktischen Belange der Dokumentation von Softwarearchitekturen ein. Wer Softwaresysteme dokumentieren muss, sollte dieses Buch kennen!



Tipp:

Insbesondere die Website <http://docs.arc42.org> bietet einen großen Fundus praktischer Tipps und Hinweise für Dokumentation von Softwarearchitekturen. Die mehr als 130 Tipps sind nach Schlüsselworten gruppiert:

- lean: Tipps getreu dem Motto „*travel light*“.
- thorough: Tipps für eher formale Systeme, die strengen Regularien unterworfen sind oder auditiert werden.
- essential: Kennzeichnet Dinge, die Sie auf jeden Fall über Ihr System kommunizieren oder dokumentieren sollten.

6

Modellierung für Softwarearchitekten

Mit Unterstützung von Peter Hruschka

*Wenn die Sprache nicht stimmt,
ist das, was gesagt wird, nicht das, was gemeint ist.*

Konfuzius



Fragen, die dieses Kapitel beantwortet:

- Was bedeutet „Modellierung“?
- Welche Möglichkeiten und Ausdrucksmittel gibt es für die Modellierung?
- Wie hilft UML bei der Modellierung?
- Welche Ausdrucksmittel stellt die UML für Softwarearchitekturen zur Verfügung?
- Welche Diagrammarten der UML sollten Sie für welche Architektursicht einsetzen?

6.1 Modelle als Arbeitsmittel

Modelle sind vereinfachte Darstellungen (Abstraktionen) der Realität. In komplexen Situationen oder Sachverhalten helfen Modelle, weil wir darin gezielt bestimmte Details der Realität aus- oder einblenden, um die Menge gezeigter Informationen zu kontrollieren.

In der Softwarearchitektur verwenden wir Modelle typischerweise in folgenden Situationen:

- Modellierung fachlicher Situationen (fachliche Klassen-, Prozess- oder Datenmodelle), um die inhaltliche Komplexität fachlicher Sachverhalte zu reduzieren und auf die für eine IT-Umsetzung relevanten Teile zu fokussieren.
- Modellierung statischer oder dynamischer Codestrukturen (Paket-, Klassen- oder sonstiger Bausteindiagramme), um die Mengenkomplexität von Quellcode zu reduzieren und auch in großen Codebasen den Überblick zu behalten. Hierunter fällt auch die Modellierung von Interaktionen oder technischen Abläufen.

- Modellierung von Systemzusammenhängen (Komponenten-, Kontext- oder Verteilungsdiagramme), um Zusammenarbeit, Bezüge oder Abhängigkeiten zwischen Systemen (Hardware oder Software) zu zeigen.
- Modellierung als Basis für Codegenerierung: fachliche oder technische Situationen werden modellhaft dargestellt, um auf dieser Basis beispielsweise Quellcode, Datenstrukturen oder andere Artefakte per Generator zu erzeugen.
- Modellierung zur Vereinfachung, um anderen Personen das Verständnis einer Situation zu erleichtern (= *Abholen von Stakeholdern*).



Bild 6.1 Realität und (ein mögliches) grafisches Modell¹

Modelle unterstützen
gezielte Sparsamkeit

Ich persönlich schätze Modelle, weil sie mir eine gezielte Sparsamkeit erlauben. Ich kann mich auf wesentliche Details einer aktuellen Situation konzentrieren und dabei genau diejenigen Informationen ver- und bearbeiten, die ich aktuell benötige.

Wenn Sie Modelle verwenden, müssen Sie zwei wesentliche Entscheidungen treffen:

- Welche Details lasse ich weg (= welche Teile der Realität sind Bestandteil meiner Modelle)?
- Wie stelle ich meine Modelle dar (grafisch oder textuell)?

Die erste dieser Fragen müssen Sie ganz allein beantworten (beziehungsweise Sie erhalten in Kapitel 4 und Abschnitt 5.2 einige Hinweise dazu) – für die zweite Frage habe ich im folgenden Abschnitt einige Anregungen für Sie gesammelt.

¹ Fotos lizenziert von iStockPhoto.com

6.1.1 Grafische oder textuelle Modellierung

Lange Zeit galten in der Informatik grafische Notationen als das Mittel der Wahl, etwa UML (siehe Abschnitt 6.2), Entity-Relationship-Diagramme, Fluss- oder Aktivitätsdiagramme oder Petri-Netze. In den letzten Jahren haben textbasierte Modelle unter dem Stichwort „Domänenspezifische Sprachen“ (DSL, siehe [Völter-2013]) vermehrt an Bedeutung gewonnen.

Textuelle DSL als Mikrosprachen

Textbasierte Modelle finden sich beispielsweise in funktionalen Programmiersprachen oder Build-, Konfigurations- oder Installationsframeworks:

- Funktionale Sprachen wie Lisp, Haskell oder Clojure erleichtern es, ein Vokabular für eine bestimmte fachliche oder technische Situation als Funktionen (= funktionale Abstraktion) zu implementieren.
- Build-Systeme (wie ant² oder gradle³) stellen sprachliche Ausdrucksmittel für das Übersetzen, Bauen, Testen oder Verteilen von Sourcecode bereit.
- Installationsframeworks (wie homebrew⁴ oder sdkman⁵) besitzen „Vokabeln“ für das Installieren und Deinstallieren bestimmter Software, deren Upgrade oder sogar für die Verwaltung mehrerer paralleler Versionen.

Textuelle Modelle von Software

Sie können beliebige Strukturen von Software (Komponente, Zusammenhänge, Abläufe) als Text beschreiben. Die Idee dieser textbasierten Beschreibung perfektioniert beispielsweise xtext⁶, ein auf Basis des Eclipse-Ökosystems entstandenes Framework zur Entwicklung beliebiger Programmier- oder Domänensprachen.

Dieser Ansatz besitzt maximale Flexibilität. Allerdings müssen Sie vor dem eigentlichen Modellieren Ihres Sachverhalts erst einmal Ihre Modellierungssprache definieren (oder Modellierung und Sprachentwicklung iterativ und agil verzahnen). In jedem Fall ist vor der eigentlichen Modellierung eine Vorarbeit notwendig, die Sie bei universellen Modellierungsmethoden oder -notationen nicht benötigen.

Textuelle Ansätze haben einige bestechende Vorteile:

- Text ist entwicklernah und wird mit denselben Werkzeugen (Editor, Versionsmanagement, diff-Tools) bearbeitet wie „normaler“ Quellcode.
- Spezifische DSLs sind maximal flexibel – und können sehr exakt auf die jeweiligen Bedürfnisse abgestimmt und entwickelt werden.

Als großen Nachteil empfinde ich den oftmals hohen Initialaufwand und die technische Komplexität der DSL-Erzeugung. Insbesondere bei größeren Systemen fallen diese Aspekte auf lange Sicht kaum ins Gewicht.

² Ant: Java-basiertes Buildsystem: <http://ant.apache.org>

³ gradle: Groovy-basiertes Buildsystem: <http://gradle.org>

⁴ Homebrew: Package-Installationsmanager für Mac-OS, <http://brew.sh>

⁵ sdkman: Installationsmanager für Java- und Groovy-Frameworks, <http://sdkman.io/>

⁶ <http://www.xtext.org>



Sowohl textbasierte wie auch grafische Ansätze habe ich in meiner Berufspraxis positiv wie negativ erlebt – daher möchte ich für keinen der beiden eine pauschale Empfehlung (oder Ablehnung) aussprechen.

- Klären Sie mit den beteiligten Personen die Anforderungen und Voraussetzungen, bevor Sie (oder Ihr Team) sich für eine Art der Modellierung entscheiden.
- Arbeiten Sie iterativ: Zeigen Sie anhand inhaltlich relevanter (Teil-)Aufgaben, dass und wie sich ein bestimmter Modellierungsansatz bei Ihnen verhält und „anfühlt“.

Am Ende wird ein System niemals an der Wahl einer Modellierungsmethode scheitern, sondern höchstens daran, wie das Team Methoden und Werkzeuge in der Praxis einsetzt.

■ 6.2 UML 2 für Softwarearchitekten

In den folgenden Abschnitten möchten wir Ihnen pragmatische Vorschläge unterbreiten, wie Sie die Ausdrucksmittel von UML 2 zur Dokumentation von Softwarearchitekturen effektiv nutzen können. Das Kapitel baut auf dem Sichtenmodell aus dem vorigen Kapitel auf.

Warum sollten Sie UML nutzen?

Viele Ingenieurdisziplinen haben Standards zur besseren Kommunikation ihrer Ergebnisse geschaffen. Denken Sie nur an Gebäudearchitekten und ihre Baupläne oder an Elektrotechniker und deren Schaltpläne. Auch Chemiker haben eine Formelsprache, um sich über Verbindungen auszutauschen. In der Softwareindustrie gab es lange Zeit keine Standards zur Notation von Modellen.

Seit die Object Management Group (OMG) 1997 die Unified Modeling Language (UML) standardisiert hat, haben auch die Softwareingenieure ein weltweit anerkanntes Mittel zur Dokumentation von Analyse- und Architekturmodellen. Seither wurde der Standard immer wieder in Details angepasst. Die praktisch relevanten Ausdrucksmittel sind stabil geblieben.

Die UML 2 stellt mehr als ein Dutzend unterschiedlicher Diagramm- und Darstellungsarten bereit, alle durch ein klares Metamodell präzise definiert. Das wirkt auf den ersten Blick sehr kompliziert und teilweise abschreckend – wie bei vielen mächtigen Werkzeugen.

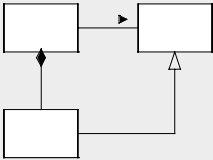
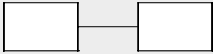
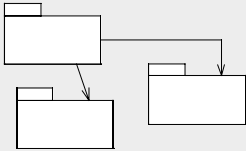
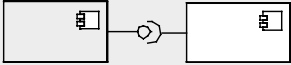
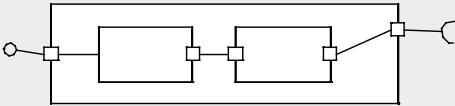
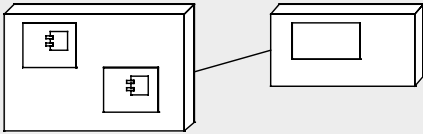
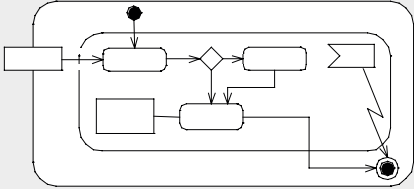
Hier möchten wir Ihnen eine pragmatische Auswahl vorstellen, mit deren Hilfe Sie wesentliche Architektursichten effektiv modellieren und kommunizieren können.

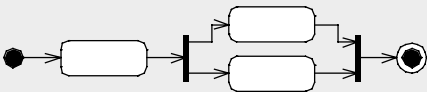
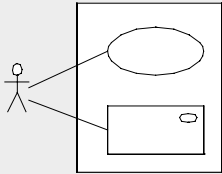
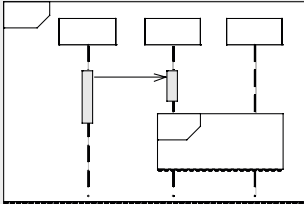
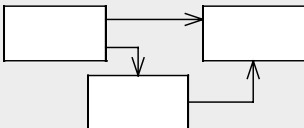
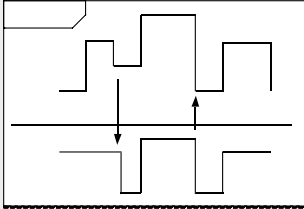
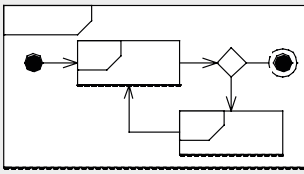
UML ist als Standard für grafische Modelle industrieweit akzeptiert. Die Unterstützung durch eine Vielzahl praxiserprobter Modellierungswerkzeuge sichert diesen Status. Sie gehen daher für die nächsten Jahre kein Risiko ein, wenn Sie sich heute für UML als Notation entscheiden.

6.2.1 Die Diagrammarten der UML 2

Bevor wir Ihnen die Anwendung der UML 2 zur Darstellung von Softwarearchitekturen erläutern, möchten wir in Tabelle 6.1 einen Überblick über die verschiedenen Diagrammarten der UML 2 geben.

Tabelle 6.1 Diagrammarten der UML 2

Diagrammart	Verwendung
Strukturdiagramme	
	Klassendiagramm: Zeigt die statische Struktur von Klassen und deren Beziehungen (Assoziationen, Aggregationen sowie Spezialisierungen/Generalisierungen).
	Objektdiagramm: Zeigt eine Struktur von Instanzen sowie deren Verbindungen; stellt insofern einen Schnappschuss des Klassendiagramms dar.
	Paketdiagramm: Gibt einen Überblick über die Zerlegung des Gesamtsystems in Pakete oder Teilsysteme. Enthält logische Zusammenfassungen von Systemteilen.
	Komponentendiagramm: Zeigt, wie Systembausteine zu Komponenten mit definierten Ein- und Ausgabeschnittstellen zusammengefasst werden.
	Kompositionsstrukturdiagramm: Zeigt den inneren Aufbau eines Systembausteins (Komponente, Klasse, Paket) sowie die Abbildung äußerer Schnittstellen auf innere.
	Verteilungsdiagramm (<i>Deployment Diagram</i>): Zeigt die Infrastruktur und ihre Abhängigkeiten sowie die Verteilung der Laufzeitelemente auf die technische Infrastruktur (Hardware).
Verhaltensdiagramme	
	Aktivitätsdiagramm: Zeigt mögliche Abläufe innerhalb von Systembestandteilen (etwa: Klassen, Komponenten oder Use-Cases). Kann Algorithmen, Daten- und Kontrollflüsse sehr detailliert beschreiben.

Diagrammart	Verwendung
	Zustandsdiagramm (State Diagram): Zeigt die möglichen Zustände eines Systembestandteils sowie die erlaubten Zustandsübergänge an und verknüpft Aktivitäten mit diesen Zuständen und Übergängen.
	Anwendungsfalldiagramm (Use-Case-Diagram): Zeigt eine Übersicht über alle Prozesse, mit denen das System auf Wünsche von Akteuren (oder Nachbarsystemen) reagiert.
Interaktionsdiagramme	
	Sequenzdiagramm: Zeigt den Nachrichtenaustausch zwischen Instanzen von Systembestandteilen für einzelne Szenarien. Seit UML 2 schachtelbar.
	Kommunikationsdiagramm: In früheren UML-Versionen Kollaborationsdiagramm genannt, zeigt die Zusammenarbeit zwischen Instanzen von Systembestandteilen.
	Zeitverhaltensdiagramm (Timing Diagram): Zeigt Zeitabhängigkeiten zwischen Ereignissen und beschreibt Zustandsänderungen in Abhängigkeit vom Zeitverlauf.
	Interaktionsübersichtsdiagramm: Zeigt das Zusammenspiel verschiedener Interaktionen und besteht in der Regel aus Referenzen auf andere Interaktionsdiagramme. Eine Art Aktivitätsdiagramm auf hoher Abstraktionsebene.

Einen Überblick über die UML-2-Notation finden Sie in [arc42] und [Rupp+12].

6.2.2 Die Bausteine von Architekturen

Die kleinsten Bausteine objektorientierter Softwarearchitekturen sind Klassen bzw. zur Laufzeit daraus instanziierte Objekte. UML stellt diese als Rechtecke dar, in die Sie neben dem Namen Attribute und Operationen eintragen können. Generell gilt: Ein einzelner Name deutet auf eine Klasse hin. Wann immer Sie im Namensfeld eine Unterstreichung und einen Doppelpunkt entdecken, handelt es sich um konkrete Objekte (Instanzen).

Sobald Ihre Systeme etwas umfangreicher werden, benötigen Sie neben Klassen noch Möglichkeiten zur Modellierung größerer Einheiten. Sie müssen aus kleineren Bausteinen größere Bausteine zusammenstellen. Zwei Beweggründe treiben Sie dazu: Sie wollen abstrahieren und Sie wollen Sichtbarkeit und Zugriff regeln.

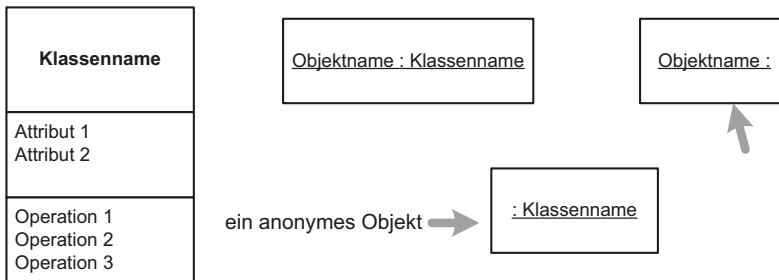


Bild 6.2 Klassen und Objekte in der UML

Dafür stellt die UML Pakete und Komponenten zur Verfügung. Etwas vereinfacht gelten folgende Regeln zur Unterscheidung:

1. Wenn Sie nur einen Namen für eine Menge von (kleineren und größeren) Bausteinen brauchen – also nur einen abstrakteren Begriff für eine Gruppe von z. B. acht kleineren Bausteinen – dann sollten Sie Pakete verwenden.
2. Wenn Sie nicht nur abstrahieren wollen, sondern gleichzeitig auch Zugriffsschutz über sauber definierte Schnittstellen gewährleisten wollen, dann sind Komponenten das Ausdrucksmittel Ihrer Wahl.

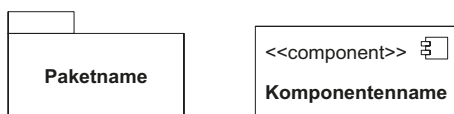


Bild 6.3

Größere Bausteine: Pakete und Komponenten

Vereinfacht ausgedrückt, sind Pakete nur Sammeltöpfe für kleine Bausteine. Komponenten hingegen sind eine Art von großen Klassen: Sie können über eigene Attribute verfügen (aber wer will schon globale Variablen?), können Beziehungen eingehen, haben nach außen sichtbare und nach innen versteckte Teile.

Pakete und Komponenten dürfen beliebig geschachtelt werden. Somit lassen sich auch sehr komplexe Systeme mit Tausenden von Klassen durch drei bis vier Abstraktionsebenen von Paketen oder Komponenten überschaubar gestalten.

6.2.3 Schnittstellen

Die Art der Schnittstellendarstellung unterscheidet sich für die unterschiedlichen Bausteine von Architekturen.

Die Schnittstellen der kleinsten funktionalen Bausteine, der Operationen, sind deren Ein- und Ausgabeparameter. Dafür lässt die UML die Syntax offen. Üblicherweise nutzt man ähnliche Schreibweisen wie in der Programmiersprache Ihrer Wahl.

Die Schnittstellen einer Klasse bilden im einfachsten Fall die Operationen, die die Klasse öffentlich („public“) macht (im Gegensatz zu den privaten Operationen, die von außen nicht aufgerufen werden können). Die UML verwendet das Zeichen „+“ vor dem Operationsnamen für public und „-“ für private (vgl. Bild 6.4).

Interessant wird die Darstellung der Schnittstellen von Komponenten. Bei diesen sind für uns als Architekt zwei Aspekte wichtig:

- Welche Services bietet die Komponente anderen Komponenten an (die Exportschnittstelle, „provided interface“)?
- Was benötigt die Komponente von anderen (die Importschnittstelle, „required interface“)?

Die UML nutzt dafür entweder die Ball- und abgekürzte Socketnotation (vgl. Bild 6.4) oder verbindet die Komponente explizit mit Interface-Klassen. Betrachten Sie eine Interface-Klasse als eine degenerierte Klasse, die neben dem Namen (und der Kennzeichnung mit dem Stereotyp <<interface>>) nur die Signaturen von Operationen enthält, jedoch keine „Bodies“ und keine Attribute. Der Pfeil zu Schnittstellen, die der Baustein anbietet (d. h. implementiert), wird – nicht ganz zufällig – ebenfalls mit dem hohlen Dreieck notiert, wie die Generalisierung/Spezialisierung von Klassen (jedoch mit gestrichelter Linie). Die Komponente wird als Spezialisierung der Schnittstelle verstanden. Schnittstellen, die benötigt werden, sind über den gestrichelten Abhängigkeitspfeil mit dem Stereotyp <<use>> verbunden.

Die UML 2 hat noch ein weiteres Ausdrucksmittel für Schnittstellen eingeführt: Ports, dargestellt als kleines Rechteck im Rahmen der Komponente. Ein Port ist nur eine abkürzende Schreibweise für eine benannte Gruppe von beliebig vielen Export- und Importschnittstellen – und bringt somit keine konzeptionellen Neuigkeiten. Um zu wissen, was an dem Port

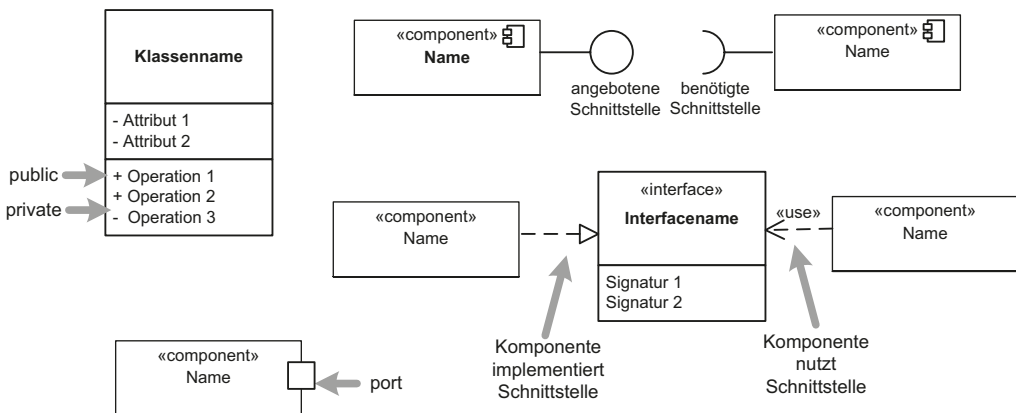


Bild 6.4 Verschiedene Arten der Schnittstellendarstellung

genau angeboten oder erwartet wird, müssen Sie diesen ausführlich beschreiben oder mit Ball- und Socketnotation explizit modellieren.

6.2.4 Die Bausteinsicht

Nach diesen Grundbegriffen zu Bausteinen und deren Schnittstellen können wir nun die UML-Modelle betrachten, mit denen Sie die Bausteinsicht darstellen können. Bild 6.5 zeigt ein UML-Klassendiagramm. Darin werden die Klassen statisch mit einfachen Assoziationen, Teile-Ganzen-Beziehungen („besteht aus“) oder Generalisierung/Spezialisierung („ist ein“) in Zusammenhang gebracht.

Der Zusammenhang zwischen den größeren Bausteinen wird in Form von Paket- oder Komponentendiagrammen hergestellt. Die wesentliche Beziehung zwischen diesen ist die einfache Abhängigkeit („dependency“), die durch Stereotypes wie <<access>>, <<import>> oder andere präzisiert werden kann. Je nach Reifegrad Ihrer Architekturfestlegungen können dabei direkte Abhängigkeiten oder Abhängigkeiten über Schnittstellen eingezeichnet werden.

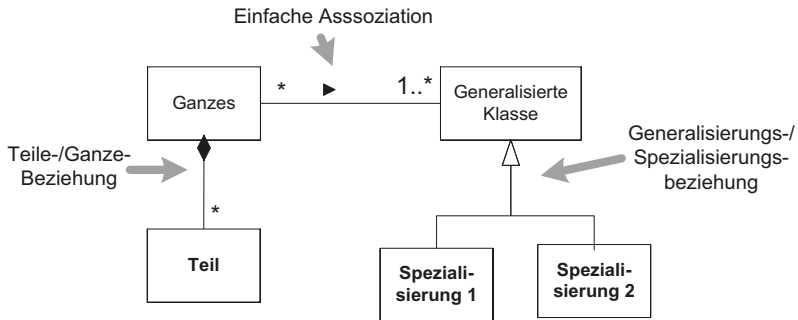


Bild 6.5 Klassendiagramm

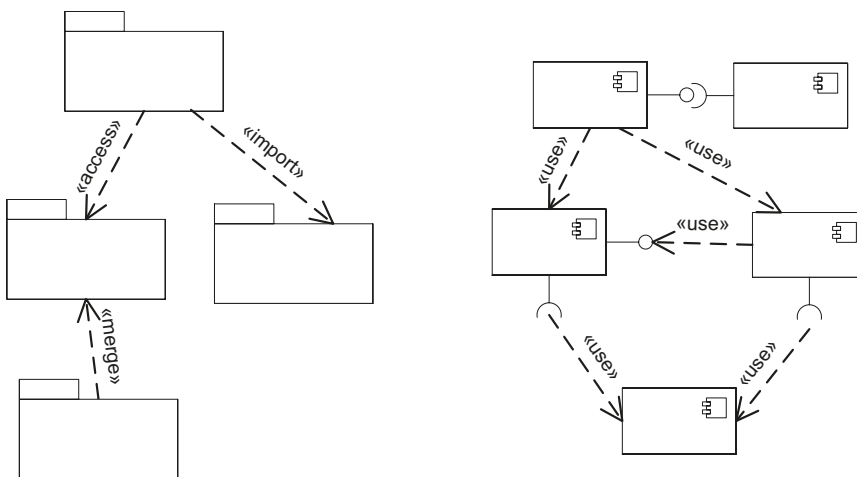


Bild 6.6 Paket- und Komponentendiagramm

Hier noch einige Tipps zum Erstellen von Klassen-, Paket- und Komponentendiagrammen.



- Denken Sie sorgfältig über gute Namen für Klassen und Komponenten nach.
- Nutzen Sie in Klassendiagrammen die Stärke der UML, Assoziationen mit Namen versehen zu können, Rollennamen anzugeben, die mögliche Navigationsrichtung vorzuschreiben und Multiplizitäten an beiden Enden festzulegen. Gepaart mit guten Namen werden die Diagramme semantisch aussagekräftiger und sparen das Nachschlagen in Hintergrunddokumenten.
- Nutzen Sie Stereotypes für verschiedene Arten von fachlichen und technischen Klassen und Komponenten, z. B. <<entity>> für datenorientierte, <<control>> für steuerungsorientierte Bausteine und <<view>> für Sichtenbildung.

Die bisher erwähnten Diagramme der Bausteinsicht stellen statische Aspekte in den Vordergrund. Zwei Diagramme für dynamische Aspekte ergänzen die Bausteinsicht: Aktivitätsdiagramme und Zustandsdiagramme. In beiden wird unabhängig von Laufzeitaspekten das allgemein gültige Verhalten von Bausteinen beschrieben.



- Beim Reengineering von Architekturen können Sie Aktivitätsdiagramme mit Bahnen benutzen, um aufzuzeigen, welche Funktionen derzeit welchem Baustein zugeordnet sind. Zeigen Sie so z. B. die Aufteilung zwischen Client und Server, zwischen verschiedenen Standorten oder Rechnern oder zwischen verschiedenen Abteilungen.
- Nutzen Sie Bahnen in Aktivitätsdiagrammen beim Neuentwurf gezielt, um Funktionen in die Verantwortung größerer Bausteine zu übergeben.
- Wenn es Ihnen schwerfällt, Funktionsreihenfolgen in Aktivitätsdiagrammen linear zu modellieren, dann verbergen sich dahinter oft Zustände. Nutzen Sie in diesem Fall Zustandsmodelle, um Funktionen in Abhängigkeit von externen, internen oder Zeitereignissen zu modellieren.
- Zustandsmodelle eignen sich hervorragend, um Lebenszyklen komplexer Klassen zu modellieren (von der Geburt nach Ausführung des Konstruktors über alle Ereignisse, die Objekten der Klassen danach zustoßen können, bis hin zur Zerstörung im Destruktor).
- Verwenden Sie Zustandsmodelle auch zur Modellierung des Verhaltens komplexer, asynchron operierender Subsysteme. Ein Beispiel dafür sind Echtzeitsysteme, die auf unabhängige äußere Ereignisse reagieren müssen; ein anderes Beispiel sind Benutzungsoberflächen, die kontextabhängig bei Benutzereingaben Menüs wechseln und Tastenbelegungen aktualisieren müssen.

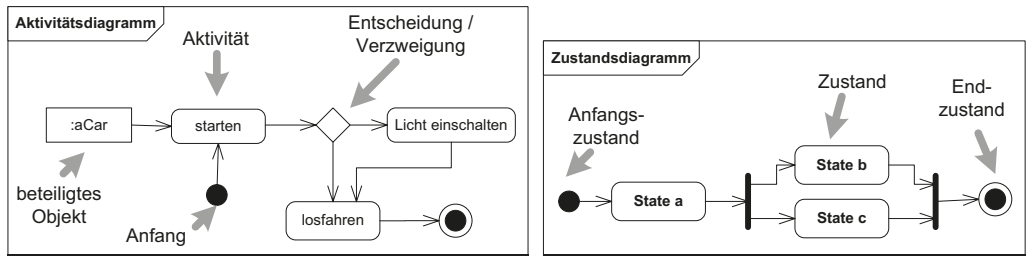


Bild 6.7 Aktivitäts- und Zustandsdiagramme

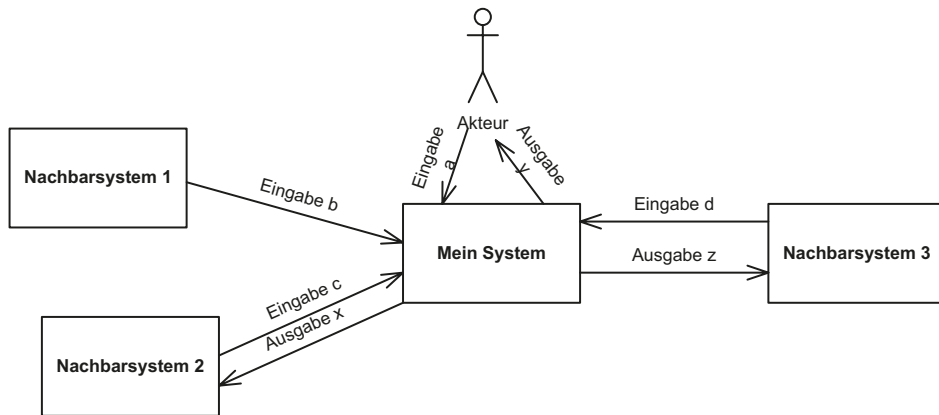


Bild 6.8 Klassendiagramm zur Darstellung des statischen Kontexts

Nutzen Sie Klassen- oder Komponentendiagramme auch für die Modellierung der statischen Kontextsicht. Zeichnen Sie ein Diagramm mit Ihrem System in der Mitte als Blackbox und ordnen Sie rundherum alle Quellen an, von denen Ihr System Informationen erhält, und alle Senken, an die Ihr System Informationen liefert. Die UML hat dieses bewährte Kontextdiagramm nicht als eigenständiges Diagramm aufgenommen, doch erfüllt das Klassendiagramm diesen Zweck, wenn Sie sich die Freiheit nehmen, die Assoziationen zwischen System und Systemumgebung gerichtet zu zeichnen und sie mit den Ein- und Ausgabennamen zu beschriften (vgl. Bild 6.8).

6.2.5 Die Verteilungssicht

UML stellt eine Diagrammart zur Verfügung, um Infrastruktur unserer Softwarearchitektur zu modellieren und deren Bausteine dieser Infrastruktur zuzuordnen: das Verteilungsdiagramm (engl. deployment diagram).

Die Hauptelemente dieser Verteilungsdiagramme sind Knoten und Kanäle zwischen den Knoten. Stellen Sie sich Knoten als Standorte, als Cluster, als Rechner, als Chips oder als Devices vor – kurz: alles, was sich etwas merken (d. h. Daten persistent festhalten) und etwas tun (d. h. Operationen ausführen) kann. Die Kanäle, die die Knoten verbinden, repräsentieren die

physikalischen Übertragungswege, auf denen Informationen zwischen Knoten ausgetauscht werden können – also alles von Kabeln über Rohrpost, Rauchzeichen bis hin zu Bluetooth und anderen drahtlosen Verbindungen. Die UML behandelt diese Infrastruktur etwas stiefmütterlich. Für verteilte Systeme sind diese Diagramme jedoch unabdingbar.

Knoten sind das einzige dreidimensionale Symbol in der UML, um die Prozessoren und Devices besser ausdrücken zu können. Die Kanäle sind als normale Assoziationen (wie im Klassendiagramm) dargestellt. Knoten sind schachtelbar, um z. B. die Prozessoren eines Clusters oder die verschiedenen Platinen in einem großen Gehäuse darzustellen.

Eigenschaften von Knoten wie der Rechnertyp oder die Speichergröße können über Stereotypen (<<Security Server>>) oder Tagged Values ({memory = 40 Gbyte}) in den Diagrammen festgehalten werden.

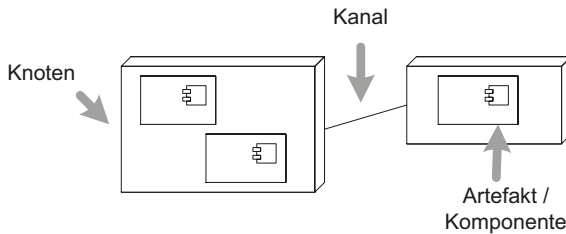


Bild 6.9

Knoten, Kanäle und verteilte Artefakte



- Modellieren Sie die Infrastruktur nur so weit, wie Sie es für die Diskussion der Verteilung der Software benötigen. Beschriften Sie auch die Kanäle sparsam, falls Sie z. B. Datenwege erläutern möchten.
- Ordnen Sie die Knoten im Bild so an, wie sie auch in der realen Welt angeordnet sind: bei geografisch verteilten Systemen z. B. Hamburg eher oben im Bild und Freiburg eher links unten; bei Prozessoren und Speichereinheiten in einem Gehäuse so, wie Sie diese Teile nach dem Aufschrauben des Gehäuses auch sehen können. Dies erleichtert die Wiedererkennung und verbessert die Lesbarkeit der Diagramme.

Der Hauptzweck der Verteilungsdiagramme ist die Zuordnung von Bausteinen zu Knoten.

Zeichnen Sie auch für die Verteilungssicht ein physikalisches Kontextdiagramm. Dazu setzen Sie den Knoten, auf dem Ihr System läuft, in die Mitte und ordnen – ähnlich wie beim logischen Kontextdiagramm – alle menschlichen Akteure und Nachbarprozessoren rundherum an. Legen Sie jetzt jedoch die physikalischen Schnittstellen fest, die zu den Nachbarsystemen vorhanden sind, z. B. Thin-Wire-Ethernet als Verbindung, das Druckerkabel, Bus-Systeme, Bluetooth etc. (vgl. Bild 6.10). Dieses Bild hilft Ihnen, die Abbildung von logischen Ein- und Ausgaben auf entsprechende Medien herzustellen.

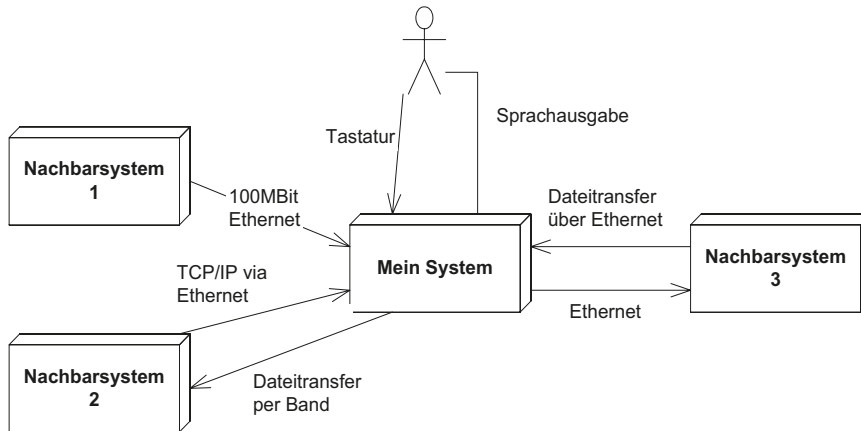


Bild 6.10 Physikalisches Kontextdiagramm: die Kanäle zu Ihrem System

6.2.6 Die Laufzeitsicht

Oftmals interessiert Sie nicht nur die prinzipielle Struktur der Bausteine Ihrer Architektur, sondern gezielte Aussagen, was zur Laufzeit Ihres Systems passieren soll. Auch dafür stellt die UML eine Menge von Diagrammen zur Verfügung. Betrachten wir zunächst die Elemente der Laufzeitsicht. Dabei handelt es sich immer um Instanzen von Bausteinen, die in der Bauzeitsicht enthalten sind, also um Objekte zu den Klassen oder um instanziierte Komponenten.

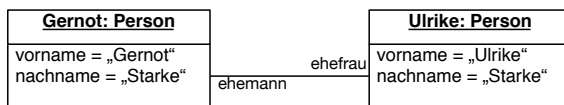


Bild 6.11
Objektdiagramm

Mit Objektdiagrammen können Sie Schnappschüsse der existierenden Laufzeitobjekte darstellen und auch instanziierte Beziehungen. Die UML bietet dabei die Möglichkeit, zwischen aktiven und passiven Objekten zu unterscheiden. Letztere werden, wie der Name schon ausdrückt, nicht selbst aktiv, sondern warten darauf, benutzt zu werden. Alle aktiven Bausteine (gekennzeichnet mit doppelter Seitenlinie in der Grafik) sind unabhängig voneinander selbstständig aktiv und führen Prozesse gemäß ihrer Spezifikation aus.



Nutzen Sie Objektdiagramme, wenn Sie gezielt die Konstellation spezieller Objekte zur Laufzeit zeigen wollen, wie z. B. die wirklich instanziierten Empfangsplätze in einem Hotel zu den Zeiten größeren Andrangs morgens und am frühen Abend im Gegensatz zur Minimalbesetzung zwischendurch oder die Zuordnung einzelner Fluglotsen zu bestimmten Flugzeugen statt der allgemeinen Beziehung „Lotse überwacht Flugzeug“.

Primär werden Sie zur Darstellung der Laufzeitsicht dynamische Modelle nutzen. Die UML verwendet für diese Diagramme den Sammelbegriff „Interaktionsdiagramme“, weil alle davon die Zusammenarbeit von Objekten zur Laufzeit beschreiben, d. h. den Austausch von Nachrichten, die gegenseitigen Aufrufe bzw. das Senden und Empfangen von Ereignissen.

Das populärste Diagramm ist das Sequenzdiagramm. Es stellt beispielhafte Abläufe (Szenarien) durch das System dar. Wie Bild 6.12 zeigt, sind die am Szenario beteiligten Objekte durch ein Rechteck mit dranhängender Lebenslinie gekennzeichnet. Von oben nach unten sehen Sie die Ereignisse und Nachrichten, die von einem Objekt zu einem anderen geschickt werden.

UML 2 hat die Sequenzdiagramme um zahllose Konstrukte erweitert, wie Abfragen, Schleifen, Parallelität, Schachtelung etc. Sie können auch die Art der Kommunikation durch unterschiedliche Pfeile grafisch darstellen (synchron mit und ohne Antwort, asynchrone Nachrichten etc.). Tasten Sie sich langsam an die Notationselemente heran, die Ihnen für die Kommunikation zwischen Systemteilen wichtig sind.

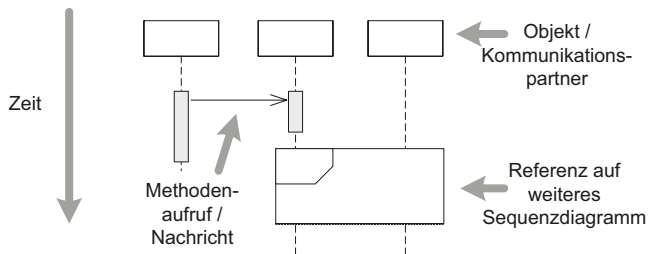


Bild 6.12
Sequenzdiagramm



- Nutzen Sie Sequenzdiagramme, um „Kandidatenobjekte“ zu finden, d. h. Bausteine, die für eine bestimmte Aufgabe die Verantwortung übernehmen können. So helfen Diagramme der Laufzeitsicht, die Elemente der Bausteinsicht zu identifizieren.
- Nutzen Sie Szenarien auch, um Ihre Architekturen zu überprüfen. Zeichnen Sie typische, offensichtliche Abläufe zwischen den Instanzen und
- prüfen Sie, ob ihre Klassen und Komponenten entsprechende Operationen bereitstellen.
- Nutzen Sie Sequenzdiagramme auf verschiedenen Abstraktionsebenen. Die größte Abstraktion ist ein Laufzeitkontextdiagramm (siehe Bild 6.13). Sie können sich auch ausschließlich auf die Kommunikation zwischen Rechnern konzentrieren, indem Sie je ein Stellvertreterobjekt für jeden Rechner zeichnen und die Kommunikation zwischen diesen. Unterdrücken Sie dabei alle Nachrichten, die innerhalb eines Rechners ausgetauscht werden. Modellieren Sie dann auf einer nächstfeineren Ebene z. B. die Kommunikation zwischen Anwendungen, die auf einem Rechner laufen, und erst auf der feinsten Ebene die Kommunikation zwischen den primitiven Objekten innerhalb einer Komponente.

Auch in der Laufzeitsicht lohnt es sich, ein Kontextdiagramm zu zeichnen. Wiederum ist Ihr System dabei ein einziges Blackbox-Objekt. Sie zeigen in diesem Laufzeitkontextdiagramm beispielhafte Kommunikation des Gesamtsystems mit Akteuren und anderen Nachbarsystemen.

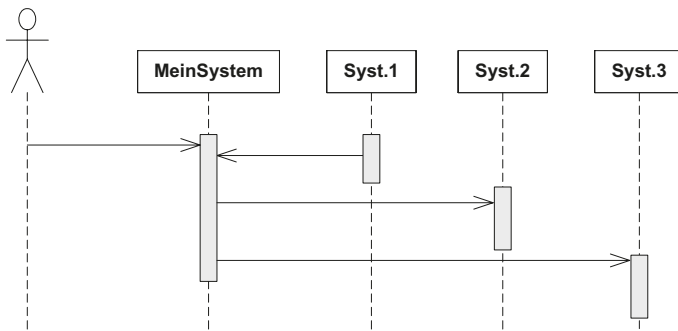


Bild 6.13
Laufzeitkontext-
diagramm



Nutzen Sie diese Kontextsicht für die frühzeitige Klärung der heikelsten Schnittstellen Ihres Systems: der Schnittstellen nach außen!

Während Sequenzdiagramme die Abläufe „sequenziell“ von oben nach unten lesbar darstellen, können Sie mit Kommunikationsdiagrammen eher die Topologie Ihrer Klassen- und Komponentendiagramme aufgreifen. Stellen Sie sich ein Kommunikationsdiagramm als ein instanziiertes Klassendiagramm dar, in das Sie jetzt (natürlich nummeriert) die Folge von Nachrichten einzeichnen, die zwischen den Beteiligten ausgetauscht werden.

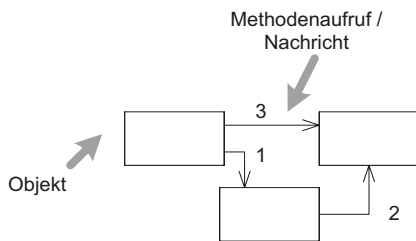


Bild 6.14
Kommunikationsdiagramm



- Nutzen Sie Kommunikationsdiagramme, wenn Sie schon Klassen und Komponenten gefunden haben und nun über deren beispielhafte Zusammenarbeit diskutieren wollen.
- Modellieren Sie ein Szenario für den Normalfall und dann z. B. Szenarien für heikle Sonderfälle. Testen Sie somit, ob die Klassen schon über geeignete Operationen verfügen, um den Gesamtablauf zu bewerkstelligen.

Da Interaktionen bei großen Systemen manchmal sehr komplex werden können, hat die UML 2 noch ein weiteres, sehr nützliches Diagramm bereitgestellt: das Interaktionsüberblicksdiagramm. Es bringt keine wesentlichen neuen Modellierungselemente ins Spiel. Stellen Sie sich das Diagramm als eine Art grafisches Inhaltsverzeichnis oder einen Übersichtsplan für viele einzelne Interaktionsdiagramme vor. Auf sehr hoher Ebene zeichnen Sie die Abhängigkeiten zwischen Interaktionsdiagrammen. Für genaueres Wissen zoomen Sie dann jeweils in eines der Diagramme hinein.

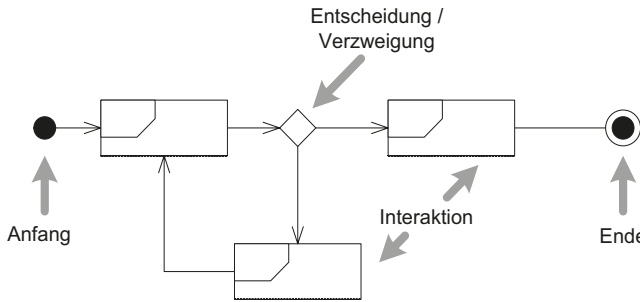


Bild 6.15
Interaktionsüberblicks-
diagramm

Mit den Ausdrucksmitteln aus Aktivitätsdiagrammen (Verzweigung, Parallelität, Verfeinerung, ...) können Sie in diesem Überblicksdiagramm alle möglichen Fragmente von Interaktionen kombinieren, die dann jeweils in eigenständigen Sequenz- oder Kommunikationsdiagrammen weiter ausgeführt sind.



- Nutzen Sie die Stärke von Szenarien. Während die Methoden der 1980er-Jahre hauptsächlich auf die Bausteinsicht mit Strukturen und generalisierten Verhaltensmodellen Wert gelegt haben, bietet die UML jetzt zusätzlich die Aussagekraft der Beispiele an. Interaktionsdiagramme sind grundsätzlich beispielhafte Abläufe. Ein gutes Beispiel ist instruktiver als eine schlechte Abstraktion!
- Wählen Sie die Beispielabläufe gezielt aus (so wie Sie Testfälle auswählen): ein Beispiel für den Normalfall, einige Beispiele für Grenzfälle und seltene Ausnahmen, vielleicht auch ein Gegenbeispiel, um die Robustheit des Systems zu testen.

6.2.7 Darum UML

So hilfreich es auch ist, gemeinsam an einer Wandtafel zu stehen und Ideen über die Gestaltung von Architekturen durch informelle Diagramme auszutauschen – die Zusammenarbeit wird noch effektiver, wenn jeder bezüglich der dargestellten Kästen und Verbindungen das Gleiche versteht.

Die UML hat Kästchen und Striche für uns standardisiert. In den statischen Modellen lassen sich die Bausteine der Architektur auf verschiedenen Abstraktionsebenen (vom Gesamtsystem als Blackbox über Subsysteme, Komponenten und Pakete bis zu den kleinsten Klassen) modellieren und miteinander in Beziehung setzen.

In den dynamischen Modellen zeigen Sie, wie Instanzen dieser Bausteine, d. h. Objekte, aktive Klassen, Prozesse und Threads, zur Laufzeit zusammenarbeiten und miteinander kommunizieren.

Verteilungsdiagramme erlauben das Mapping (von Artefakten) der Bausteine auf die vorhandene Infrastruktur.

Somit stellt Ihnen UML genügend grafische Hilfsmittel zur Verfügung, um alle Aspekte einer Softwarearchitektur transparent zu machen – und zwar in einer Weise, die weltweit verstanden wird.

6.2.8 Darum nicht UML

So hilfreich eine standardisierte Notation ist – die Mächtigkeit von UML kann auch abschreckend wirken. In Entwicklungsteams habe ich bezüglich UML oft heftige Akzeptanzprobleme erlebt, ausgelöst durch überzogenen Formalismus oder komplexe UML-Werkzeuge. Fokussieren Sie auf Inhalten, bevor Sie viel Zeit in UML-Missionierung investieren. Auch in eher formlosen Notationen können Sie signifikante und relevante Inhalte vermitteln (= visuell kommunizieren). Zum Diskutieren und gemeinsamen Brainstormen eignen sich Skizzen am Whiteboard oft besser als formale Modelle. Bei Bedarf können Sie solche Skizzen zu einem späteren Zeitpunkt in (UML-)Werkzeuge überführen.

■ 6.3 Tipps zur Modellierung

Let's keep the modeling baby but throw out the bureaucracy bathwater.

Scott Ambler, in [Ambler-12]

- Modellieren-im-Team: Skizzieren Sie erste Versionen im Team am Whiteboard. Verzichten Sie (vorläufig) auf komplexe Grafik- oder UML-Werkzeuge. Solche Zusammenarbeit verbessert das gemeinsame Verständnis der jeweiligen Systemteile erheblich. Nennen Sie diese Teamarbeit bewusst „skizzieren“, nicht „modellieren“!
- gerade-gut-genug: Vermeiden Sie das Streben nach Vollständigkeit Ihrer Modelle – meistens genügen Ausschnitte oder Teile des Systems für das Verständnis! [Ambler-12] nennt dies „Just Barely Good Enough“.
- gerade-rechtzeitig: Verzögern Sie die Erstellung der ausgelieferten Dokumentation – es könnten sich ja noch Dinge ändern! [Ambler-12] nennt das „Document Late“.
- Dokumentieren Sie kontinuierlich. Am Ende der letzten Iteration oder des Projekts können Sie sich nicht mehr an alle relevanten Dinge erinnern. In jeder Iteration oder Phase sollten Sie die dabei wichtigen Dinge dokumentieren. [Ambler-12] nennt das „Document Continuously“.
- Diagramm-plus-Text: Ergänzen Sie grafische Modelle um kurze textuelle Erläuterungen. Gute technische Dokumentation kombiniert Bild mit Text!
- Manche Modellierungswerkzeuge bieten einen *Napkin-Style* an, bei dem Diagramme ähnlich wie handgemalt aussehen. Damit können Sie optisch sehr schön ausdrücken, dass das betreffende Modell einen Zwischenstand repräsentiert.
- Halten Sie Modelle möglichst redundanzfrei. Versuchen Sie, benötigte Dokumente aus einer einheitlichen Informations- oder Modellbasis zu generieren.

■ 6.4 Weiterführende Literatur



[Rupp+12] ist das derzeit ausführlichste UML-2-Buch – nicht nur im deutschsprachigen Markt. Anhand vieler Beispiele aus alltäglichen Situationen präsentieren die Autoren die Diagrammarten.

[OMG_UML] ist die Originalquelle des UML-2-Standards: vier Dokumente mit mehreren Hundert Seiten für Liebhaber staubrockener Literatur (böse Zungen empfehlen diese Werke gegen Schlaflosigkeit).

[Ambler-12] Sehr pragmatisch finde ich die Hinweise von Scott Ambler zu gutem Modellierungsstil – zu finden unter <http://www.agilemodeling.com/style/>. Scott gibt sehr nützliche Hinweise, wie Sie die Mittel der UML besser, lesbarer und einfacher anwenden können.

Technische Konzepte und typische Architekturaspekte

We shape our buildings; thereafter they shape us.

Winston Churchill



Fragen, die dieses Kapitel beantwortet:

- Was ist Entwurf-durch-Routine und was bedeutet Entwurf-durch-Innovation?
- Was sind Strukturen und technische Konzepte?
- Wie behandelt man folgende Themen in Softwareentwürfen:
 - Persistenz
 - Geschäftsregeln
 - Integration, Verteilung und Kommunikation
 - Grafische Oberflächen
 - Workflow-Management
 - Sicherheit
 - Protokollierung, Logging
 - Ausnahme- und Fehlerbehandlung

Routine und Innovation im Software Engineering

Der Ausdruck „Software Engineering“ entstand 1969 im Zuge eines von der NATO veranstalteten Workshops ([NATO69]). Seither schmückt sich unsere Disziplin mit den Lorbeeren des „*ingenieurmäßigen* Vorgehens“, wobei die Praxis sich doch erheblich von anderen Ingenieurdisziplinen unterscheidet. So ist es im Maschinenbau, der Elektrotechnik und der Gebäudearchitektur durchaus üblich, bei Neuentwicklungen auf bereits vorhandene Lösungen zurückzugreifen. Bei der Konstruktion einer Flugzeugbremse werden die beteiligten Ingenieure auf dokumentierte Erkenntnisse und Lösungen zurückgreifen und diese gegebenenfalls modifizieren. Kaum ein Maschinenbauingenieur käme beispielsweise auf die Idee, die für eine Bremse notwendigen Hydraulikpumpen, die Bremscheiben und die Steuerelektronik von Grund auf neu zu entwerfen.

In Softwareprojekten sieht die Realität gänzlich anders aus: Dort entwickeln Softwarearchitekten oftmals sämtliche für ein System benötigten Bestandteile neu, obwohl große Teile davon in anderen, ähnlichen Systemen bereits vorhanden sind.

Entwurf-durch-Routine

Die klassischen Ingenieurdisziplinen verlassen sich im Gegensatz zur Informatik darauf, dass ihr Wissen in einer für Praktiker anwendbaren Form kodifiziert wird. Sie praktizieren damit eine Form des Entwurfs, den [Shaw+96] als *Entwurf-durch-Routine* bezeichnet. Diese Art des Entwurfs ist charakterisiert durch die Wiederverwendung großer Teile früherer Lösungen. Mit dieser Art des Entwurfs können Ingenieure die weitaus meisten praktischen Probleme lösen!

Entwurf-durch-Innovation

Im Gegensatz dazu erfordert der *Entwurf-durch-Innovation* die Entwicklung neuartiger Lösungen für bislang unbekannte Aufgabenstellungen. Diese innovative Arbeit mag spektakulärer oder auch interessanter sein, in der Praxis tritt sie jedoch höchst selten auf. Im Bereich der praktischen Informatik entstehen die meisten Softwaresysteme in bekanntem Kontext. Dennoch arbeiten Informatiker häufig nach dem Verfahren *Entwurf-durch-Innovation*. Sie verzichten dadurch auf Wiederverwendung und erhöhen gleichzeitig das Projektrisiko.



- Entwerfen Sie möglichst „durch-Routine“. Verwenden Sie erprobte Konzepte und Ansätze weiter. Pflegen Sie in Ihrer Organisation ein Wertesystem, das Wiederverwendung höher bewertet als Neuerfindung.
- Zahlreiche solche Muster finden Sie in der Literatur (etwa: [Gamma95], [Buschmann96], [Larman2001], [Fowler02]).
- Muster zu vielen Aspekten der Softwareentwicklung finden Sie im Internet unter den Tagungsberichten der Pattern-Konferenzen. Ein guter Startpunkt ist die Website <http://www.hillside.net/patterns>

Der nachfolgende Katalog von Architektur Aspekten unterstützt den *Entwurf-durch-Routine* und hilft Ihnen, Software ingenieurmäßig zu entwerfen, indem er Lösungsansätze für häufig wiederkehrende Architekturfragen aufführt.

Darum technische Konzepte und Architektur Aspekte

In diesem Buch fasse ich unter der Überschrift *Technische Konzepte und Architektur Aspekte* einige der übergreifenden und querschnittlichen *technischen* Themen zusammen, die Softwarearchitekten bei der Entwicklung von IT-Systemen häufig und intensiv beschäftigen. Zu solchen Konzepten gehören übergreifende oder querschnittliche Themen, die meistens mehrere Bausteine des Systems betreffen. Diese Konzepte prägen die Bausteinstrukturen oder deren Implementierung nachhaltig. Sie stellen oftmals zentrale technische Entscheidungen dar. Ziel dieses Kapitels ist es, Ihnen den Kontext dieser Konzepte oder Architektur Aspekte und erste Lösungsansätze aufzuzeigen und Sie dann auf die einschlägige und vertiefende Literatur zu verweisen. Diese Lösungsansätze geben lediglich Hinweise, die Sie für Ihren konkreten Einsatz konkretisieren müssen. Dennoch profitieren Sie von diesen Konzepten:

Zeit sparen**Unsicherheit mindern**

- Sie sparen Zeit, weil die Nutzung wiederkehrender Konzepte Ihnen Teile Ihrer Entwurfsarbeit erleichtern.
- Sie vermindern Unsicherheit, weil die Beschreibung der Konzepte Sie auf mögliche Risiken oder Problemzonen der jeweiligen Bereiche hinweist.

Das Buch deckt aus Platzgründen nur einige solcher technischen Konzepte ab.

Technische Konzepte beinhalten konkrete Technologien

Oft manifestieren sich in technischen Konzepten die Details der jeweiligen Implementierungstechnologie oder entsprechender Frameworks. Die technischen Konzepte beeinflussen die Bauelementstrukturen von Systemen – und umgekehrt.

In diesen Konzepten steckt technisches Wissen und meistens können Sie solche Konzepte nur auf Basis fundierter technischer Erfahrung erstellen. Hier ist es besonders nützlich, auf die Erfahrung der Entwicklungsteams zurückzugreifen.

Welche Arten technischer und querschnittlicher Konzepte gibt es?

Zunächst möchte ich Ihnen eine Vorstellung von der thematischen Breite möglicher technischer Konzepte vermitteln. In den weiteren Abschnitten des Kapitels finden Sie zu einigen von ihnen Details.¹



Diese Liste ist für Sie möglicherweise unvollständig! Stellen Sie für Ihre Projekte oder Domäne eine ähnliche Tabelle als Checkliste bereit, damit Sie beim Entwurf Ihrer Architekturen an alle für Sie wesentlichen Aspekte denken.

Tabelle 7.1 Mögliche Konzepte (nach einer Zusammenstellung aus [arc42])

Architektur Aspekt	Bedeutung und Herausforderung
Persistenz (siehe Abschnitt 7.1)	Datenspeicherung, insbesondere von Objekt- und Klassenstrukturen auf Tabellen, Entkopplung von Fachdomäne und (Datenbank-)Infrastruktur
Geschäftsregeln (siehe Abschnitt 7.2)	Behandlung domänenspezifischer Zusammenhänge und Regeln als eigenständige („externalisierte“) Einheiten
Integration (siehe Abschnitt 7.3)	Einbindung bestehender Systeme in einen neuen oder veränderten Kontext
Verteilung (siehe Abschnitt 7.4)	Verteilung von Systembestandteilen auf getrennte Ablaufumgebungen
Kommunikation (siehe Abschnitt 7.5)	Übertragung von Daten zwischen Systemkomponenten, innerhalb und außerhalb von Prozess- oder Adressräumen
Grafische Oberflächen (siehe Abschnitt 7.6)	Steuerung von Navigation und Zustandswechseln in grafischen Benutzeroberflächen, Verarbeitung von Benutzereingaben und Ereignissen
Ablaufsteuerung, Geschäftsprozessmanagement (siehe Abschnitt 7.7)	Steuerung systemübergreifender Abläufe, Koordination verschiedener Softwaresysteme, Workflowmanagement, Steuerung von Geschäftsprozessen
Sicherheit (siehe Abschnitt 7.8)	Methoden zur Gewährleistung von Datenschutz und -sicherheit, Verhinderung von Datenmissbrauch

¹ Wobei ich rein subjektiv diejenigen Aspekte ausgewählt habe, die mir in den letzten Jahren in Projekten als besonders wichtig erschienen.

Tabelle 7.1 (Fortsetzung) Mögliche Konzepte (nach einer Zusammenstellung aus [arc42])

Architektur Aspekt	Bedeutung und Herausforderung
Logging, Protokollierung, Tracing (siehe Abschnitt 7.9)	Sammeln von Informationen über den Programmzustand während der Laufzeit
Ausnahme- und Fehlerbehandlung (siehe Abschnitt 7.10)	Abweichungen von „erwarteten“ Situationen, Fehlverhalten oder Defekte von Systemteilen, unerwartetes Verhalten von Benutzern oder anderen Systemen
Skalierbarkeit (siehe Abschnitt 7.11)	Anpassbarkeit eines Systems an steigende Datenmengen, Nutzeranfragen oder Transaktionen pro Zeiteinheit
Blockchain (siehe Abschnitt 7.12)	Eine Blockchain kann Dokumente und vor allem Transaktionen aller Art speichern. Das Besondere an der Blockchain ist die Art und Weise, wie diese Daten gesichert werden.
Batchverarbeitung	(Stapelverarbeitung): Datensatzweise Verarbeitung großer Datenbestände ohne Interaktion mit Benutzern
Transaktions- behandlung	Transaktionen sind nicht teilbare Arbeitsabläufe, die in sich komplett abgeschlossen sind. Transaktionen müssen immer komplett und vollständig abgearbeitet oder aber „zurückgerollt“ werden. Solche Transaktionen müssen die ACID-Forderungen erfüllen: Atomicity, Consistency, Isolation und Durability.
Konfigurierbarkeit	Anpassung des Systems an besondere Umwelt- oder Umfeldbedingungen durch Änderungen von Installations-, Start- oder Laufzeitparametern
Zustands-, Session- oder Sitzungsbehandlung	Verwaltung von Client- und Server-Zuständen, Zuordnung von Clients zu spezifischen Sessions („Sitzungen“), insbesondere im verteilten Client/Server- und Webumfeld wichtig
Plausibilisierung und Validierung von Eingabe- oder Eingangsdaten	Wo und wann sollen Datenprüfungen stattfinden, um einen angemessenen Kompromiss zwischen Wartbarkeit („zentralisierte Prüfung“) und Laufzeit („Prüfung bei Dateneingang oder -eingabe“) zu finden?

■ 7.1 Persistenz

Persistenz: Beständigkeit, Dauerhaftigkeit. Daten aus dem (flüchtigen) Hauptspeicher auf ein beständiges Medium (und wieder zurück) bringen.

7.1.1 Motivation

Daten müssen dauerhaft gespeichert werden

Viele Daten, die Softwaresysteme bearbeiten, müssen dauerhaft auf einem Speichermedium gespeichert oder von solchen Medien gelesen werden:

- Flüchtige Speichermedien (Hauptspeicher oder Cache) sind technisch nicht für dauerhafte Speicherung ausgelegt. Daten gehen verloren, wenn die entsprechende Hardware ausgeschaltet oder heruntergefahren wird.
- Die Menge der von kommerziellen Softwaresystemen bearbeiteten Daten übersteigt üblicherweise die Kapazität des Hauptspeichers.
- Auf Festplatten, optischen Speichermedien oder Bändern sind oftmals große Mengen von Unternehmensdaten vorhanden, die eine beträchtliche Investition darstellen.
- Einfache Speicherung bzw. Serialisierung von Daten in Dateien genügt in der Regel nur für sehr kleine Datenmengen und *single-user* Anwendungen.²

Persistenz ist ein technisch bedingtes Thema und trägt nichts zur eigentlichen Fachlichkeit eines Systems bei. Dennoch müssen Sie sich als Architekt mit dem Thema auseinandersetzen, denn ein erheblicher Teil aller Softwaresysteme benötigt einen effizienten Zugriff auf persistent gespeicherte Daten. Hierzu gehören praktisch sämtliche kommerziellen und viele technischen Systeme. Eingebettete Systeme und Software für mobile Geräte gehorchen jedoch oft anderen Regeln hinsichtlich ihrer Datenverwaltung – was ich hier nicht weiter vertiefe.

Verschiedene Speichermodelle

Aufgrund der vielseitigen Anforderungen an performante, zuverlässige und plattformübergreifende Speicherung von Daten entstanden neben dem einfachen „Speichern in Dateien“ verschiedene Arten von Datenbanksystemen³ als Speichermodelle (siehe Bild 7.1).

² Hier zeigt das pickle-Modul aus der Python-Standardbibliothek ein vorbildlich einfaches und verständliches Vorgehen auf. Falls Python wider Erwarten für Ihr Problem nicht ausreichen sollte, könnten Sie mit dem mächtigen Google-Protocol-Buffer an fast jedes Serialisierungsziel gelangen: „Protocol Buffers are a way of encoding structured data in an efficient yet extensible format. Google uses Protocol Buffers for almost all of its internal RPC protocols and file formats.“ (<http://code.google.com/p/protobuf/>)

³ Der korrekte Terminus lautet Datenbank-Management-Systeme (DBMS) – aber so genau nehme ich das meist nicht.

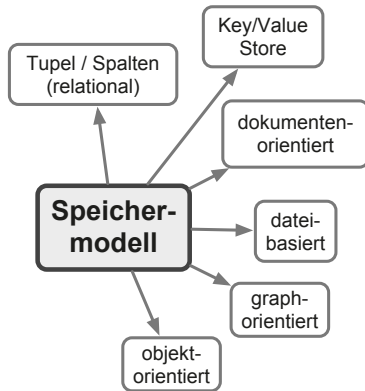


Bild 7.1
Speichermodelle

Typen von DBMS

Eine Datenbank bietet Mittel, um Daten zu persistieren und wieder in den Hauptspeicher zu laden. Es gibt verschiedene Paradigmen von Datenbanken (chronologisch sortiert):

- Netzwerkdatenbanken
- Hierarchische Datenbanken
- Relationale Datenbanken (Speichermodell Tupel/Spalten). Hierin werden Daten in Tabellen gespeichert, die über Schlüsselbeziehungen aufeinander referenzieren können. Relationale Datenbanken stoßen bei Clusterung, Replikation beziehungsweise automatischer Partitionierung von Datenbeständen an ihre Grenzen.⁴ Viele Vertreter dieser Zunft glänzen durch höchste Zuverlässigkeit.
- Objektorientierte Datenbanken. Sie können Objekte und Objektgraphen direkt speichern und wiederherstellen. Trotz einfacher Programmiermodelle, hoher Robustheit und Performance sind diese Systeme in der Praxis nur wenig verbreitet.
- NoSQL-Datenbanken⁵ (alternative Speichermodelle beispielsweise Key/Value, Column-Stores, Graphen oder dokumentenorientiert). Hinter diesem Begriff verbirgt sich eine nicht klar abgegrenzte Menge von Datenbanksystemen, die meistens auf die Nutzung von SQL als Abfragesprache verzichten. Der Markt für diese Systeme ist sehr heterogen, viele sind relativ jung, viele Open Source oder zumindest frei verfügbar.

Bisher größte Bedeutung: Relationale DBMS

Die weitaus größte Bedeutung am Markt haben bisher relationale DBMS gewonnen. Netzwerk- und hierarchische Datenbanken kommen in neu entwickelten Systemen praktisch nicht mehr zum Einsatz, doch liegen immer noch große Mengen Unternehmensdaten auf Mainframe-Systemen in dieser Art von Datenbanken. Objektorientierte oder Graph-Datenbanken besitzen aktuell (Stand 2013) nur geringen Marktanteil.

NoSQL auf dem Vormarsch

Die sogenannten NoSQL-Systeme gewinnen im Webumfeld, insbesondere bei Anwendungen mit hohen Last-, Skalierungs- oder Performance-Anforderungen, immer mehr an Bedeutung.

⁴ Eine Ausnahme bilden RDBMS, die auf spezialisierter Hardware laufen. Dabei sind Hard- und Software auf Skalierung, Datenpartitionierung und Verteilung von Grund auf ausgelegt. Ein Nachteil sind die interessanten Listenpreise solcher Systeme ...

⁵ NoSQL steht für Not-Only-SQL. Siehe [Edlich+11].

Allerdings können einfache Anwendungen auch mit dem Speichermodell „Datei“ auskommen, bei dem Daten durch Serialisierung in Dateien direkt auf dem Dateisystem abgelegt werden.



Verwenden Sie **relationale DBMS** für Anwendungen aus folgenden Bereichen:

- Anwendungen mit vielen Datensätzen gleicher (tabellarischer) Struktur (festes Schema). Änderungen an dieser Struktur sind selten.
- Betriebliche Informationssysteme, in denen zahlreiche, aber relativ einfache Operationen auf Daten ausgeführt werden.
- Anwendungen, die mit Fremdsystemen auf Basis relationaler DBMS arbeiten.
- Anwendungen, die vorhandene Mainframe-Legacy-Systeme integrieren. RDBMS passen oft recht gut zu den vorhandenen hierarchischen oder Netzwerkdatenbanken solcher Systeme.
- Anwendungen in konservativem, organisatorischem Umfeld. RDBMS sind ausgereift und weit verbreitet, praktisches Wissen und Erfahrung problemlos verfügbar. Ihr Einsatzrisiko ist gering.
- Anwendungen, die auf „kurzen“ Transaktionen basieren. Relationale Datenbanken eignen sich nicht für lange Transaktionen, bei denen Objekte über Stunden oder Tage gesperrt werden müssen.



Verwenden Sie **objekt- oder graphenorientierte oder andere nichtrelationale DBMS** für Anwendungen aus folgenden Bereichen:

- Multimedia-Systeme, die komplexe Datenstrukturen wie Grafiken, Audio oder Video bearbeiten.
- Anwendungen aus den Bereichen Computer-Aided-Design (CAD), Computer-Aided-Manufacturing (CAM) oder Computer-Aided Software Engineering (CASE).
- Anwendungen, die in hohem Maße von Vererbungshierarchien und benutzerdefinierten Datentypen abhängen (objektorientierte DB).
- Anwendungen, die komplexe Graphen speichern oder verarbeiten müssen (graphorientierte DB).



Sie sollten **NoSQL Datenbanken** für Anwendungen aus folgenden Bereichen in Erwägung ziehen:

- Anwendungen mit sehr hohen Benutzerzahlen und/oder sehr großem Datenvolumen. Beispiel: Große Webanwendungen, Börsensysteme mit hohen Transaktionsvolumina.
- Anwendungen mit sehr hohen Anforderungen an Skalierbarkeit beziehungsweise Verteilung auf sehr viele Knoten.
- Anwendungen, bei denen Sie durch Prototypen den Einsatz neuartiger Technologien wie NoSQL verifizieren können (d. h., bei denen Sie ausreichend Zeit für solche Prototypen zur Verfügung haben!).

Mischformen der Speicherung (= polyglott persistence) bilden eine mögliche Option – wobei durch zusätzliche Komponenten (hier: verschiedene DBMS) die Komplexität eines Systems steigt.

Einfache Ansätze führen zu Problemen

Im einfachsten Fall, dargestellt in der linken Hälfte von Bild 7.2, kümmert sich eine fachliche Komponente eines Systems selbst um die notwendige Speicherung ihrer Objekte. Dabei enthalten Klassen aus der Fachdomäne den Programmcode für die jeweiligen Datenbankoperationen, etwa als SQL-Befehle.

Dies führt zu sehr starker (und manchmal fataler) Abhängigkeit fachlicher Komponenten vom Persistenzmechanismus. Änderungen an der Datenbank erfordern dann meistens Änderungen am Quelltext der fachlichen Komponenten. Beispielsweise führt die Anpassung des Datenmodells (etwa aus Performance-Gründen) zu notwendigen Änderungen am Fachmodell. Testen des Systems funktioniert nur gegen die echte Datenbank, eine Mock-Datenbank ist ohne Änderung im Quellcode nicht möglich.

Selbst eine einfache Änderung, wie die Umbenennung einer Datenbanktabelle, lässt sich nicht ohne Änderung am Quellcode der Fachklasse durchführen.

Deutlich flexibler ist die Variante rechts im Bild 7.2: Eine Schnittstelle entkoppelt die fachlichen Bausteine von der Persistenz. Repository-Bausteine können diese Schnittstelle implementieren. Diese Struktur entkoppelt die fachlichen Bausteine komplett von der zugrunde liegenden Technologie (siehe Abschnitt 6.2.3.3).

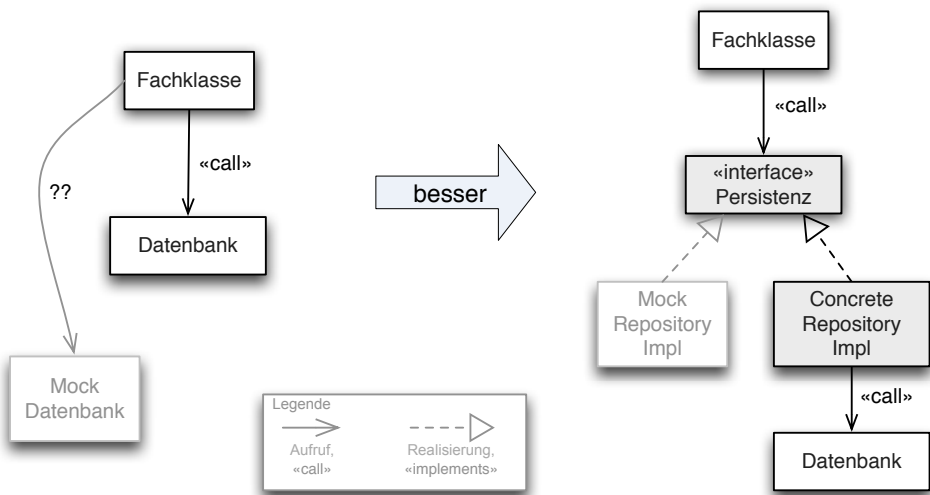


Bild 7.2 Einfache Ansätze für Persistenz

7.1.2 Einflussfaktoren und Entscheidungskriterien

Es gibt eine ganze Reihe von Einflussfaktoren und Entscheidungskriterien, die Sie beim Entwurf und der Implementierung von Persistenzkomponenten beachten sollten (siehe Bild 7.3).

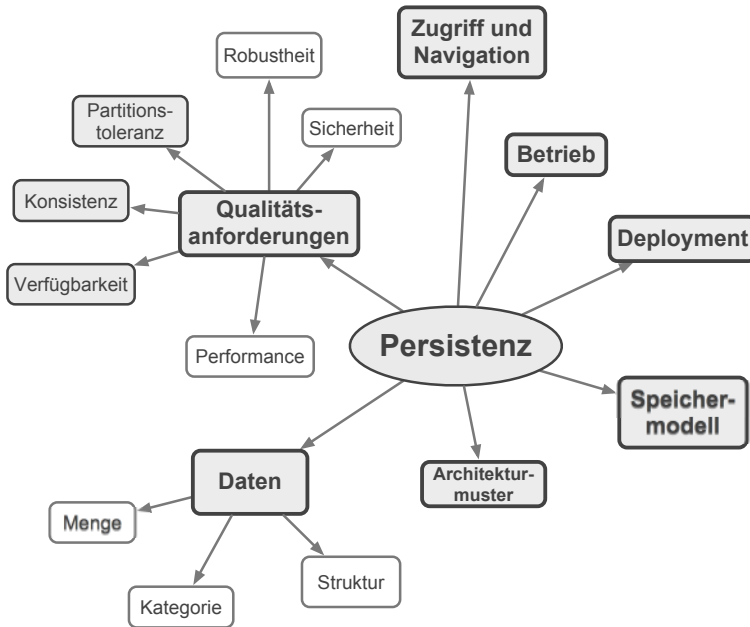


Bild 7.3 Überblick über Faktoren, die Einfluss auf die Persistenz haben

Neben Struktur, Kategorie und Menge der zu speichernden Daten spielen Qualitätsanforderungen, notwendige Zugriffe und Navigation, Betrieb, Deployment sowie die oben bereits erwähnten unterschiedlichen Speichermodelle eine große Rolle beim Entwurf der Persistenz. Auch aufgrund der aktuellen Entwicklungen im NoSQL-Bereich (siehe [Edlich+11]) sollte die Antwort auf die Frage „Worin speichern wir unsere Daten?“ heutzutage nicht mehr automatisch „in einer relationalen Datenbank“ lauten, sondern differenzierter untersucht werden.⁶

7.1.2.1 Art der zu speichernden Daten

Die Art der zu speichernden Daten prägt den Entwurf ihrer Persistenz maßgeblich. Hier sollten Sie drei wesentliche Fragenkomplexe klären:

- Besitzen die Daten eine feste Struktur („Schema“) oder sind sie unstrukturiert? Kann die Struktur über die Zeit variieren oder bleibt sie lange Zeit fest? Relationale Datenbanken sind für feste tabellarische (relationale) Strukturen optimiert, können jedoch mit schwach strukturierten Daten variabler Struktur relativ schlecht umgehen bzw. Sie müssten variable Datenstrukturen bereits im (festen!) Tabellenschema vorsehen.
- Welche Mengen oder Volumina von Daten müssen Sie speichern? Wenn diese Menge den verfügbaren Speicherplatz einzelner Rechnersysteme überschreitet, müssen Sie über Partitionierung Ihrer Daten nachdenken – wobei manche NoSQL-Systeme Sie hervorragend unterstützen und die meisten relationalen Datenbanken Ihnen eher Kopfschmerzen verursachen werden.

⁶ Für unternehmensintern genutzte Anwendungen wird sicherlich die Antwort auch heute noch meistens „relationale Speicherung“ lauten; allerdings entstehen viele aktuelle Softwaresysteme im Kontext „Internet“ – und dabei existieren weitere Optionen, die in puncto Verfügbarkeit, Skalierbarkeit und Betriebskosten hervorragend abschneiden.

- Müssen Sie Daten und deren Strukturinformation versionieren? Müssen Sie eventuell sogar mit unterschiedlichen Versionen von Daten zur gleichen Zeit umgehen können? Manche NoSQL-Datenbanken bringen dafür Features wie *Multi-Version-Concurrency-Control* mit (etwa Apache's CouchDB). Das erlaubt Ihnen, auftretende Konflikte zwischen unterschiedlichen Versionen zum großen Teil automatisiert zu bearbeiten. In relationalen Strukturen müssen Sie das in der Regel selbst modellieren und implementieren.

7.1.2.2 Konsistenz und Verfügbarkeit (ACID, BASE oder CAP)

Die Verarbeitung von Daten in relationalen Datenbanken erfüllt in der Regel die ACID⁷-Eigenschaften, d. h. sämtliche Verarbeitungsschritte sind atomar, konsistent, isoliert und dauerhaft. Dies stellt die Verlässlichkeit von Transaktionen sicher.

Gerade im Webumfeld benötigen Anwendungen diese strikten Garantien nicht unbedingt. Blog-Kommentare oder Ihre Friends-Liste bei Facebook dürfen, zumindest für kurze Zeit, leicht inkonsistent sein. Diese gegenüber ACID etwas gelockerten Anforderungen werden in den BASE-Kriterien zusammengefasst: *basically available, soft state, eventually consistent*.

Hierzu schreiben [Edlich+11]:

„Bei BASE dreht sich im Gegensatz zu ACID alles um Verfügbarkeit. Konsistenz wird der Verfügbarkeit bei BASE untergeordnet. Wo ACID einen pessimistischen Ansatz bei der Konsistenz verfolgt, ist BASE ein optimistischer Ansatz, bei dem Konsistenz als ein Übergangsprozess zu sehen ist und kein fester Zustand nach einer Transaktion. Daraus entsteht ein völlig neuartiges Verständnis von Konsistenz: Eventually Consistency. Systeme, die nach BASE arbeiten, erreichen auch irgendwann den Status der Konsistenz, die Betonung liegt dabei aber auf irgendwann. Infolgedessen wird Konsistenz erst nach einem Zeitfenster der Inkonsistenz erreicht und nicht unmittelbar nach jeder Transaktion.“

CAP-Theorem: Neue Qualitätsanforderungen an Datenbanksysteme

In diesem Zusammenhang steht auch die Erkenntnis, dass in verteilten Systemen die Merkmale Konsistenz, Verfügbarkeit und Partitionstoleranz nicht vollständig vereinbar sind.

CAP-Theorem

Unter dem Titel CAP-Theorem (consistency, availability, partition-tolerance) hat Eric Brewer ([Brewer00]) diese Erkenntnis publiziert.

Sie können jeweils zwei dieser drei Eigenschaften sicher erreichen, müssen dann jedoch Abstriche bei der dritten Eigenschaft hinnehmen. Siehe auch Bild 7.5.

Nun sind Konsistenz und Verfügbarkeit verständliche Begriffe – was hingegen bedeutet Partitionstoleranz? Dieser Begriff wurde erst durch die horizontale Skalierung von (NoSQL)-Datenbanken bedeutsam.

Stellen Sie sich vor, Ihr Datenbestand ist auf mehrere Knoten (= Partitionen) verteilt, die sich gegenseitig selbstständig aktualisieren (d. h. über Änderungen (insert, update) informieren). Diese Situation zeigt Bild 7.4. Wann meldet die Datenbank eine erfolgreiche Schreiboperation: Genügt es, die Daten in einer der Partitionen zu speichern (Fall 1), müssen es mehrere sein (Fall 2) oder müssen *alle* Partitionen erfolgreich speichern?

⁷ Ausführliche Diskussionen von ACID finden Sie in der Standardliteratur zu Datenbanksystemen. [Edlich+11] enthält eine Gegenüberstellung von ACID und BASE.

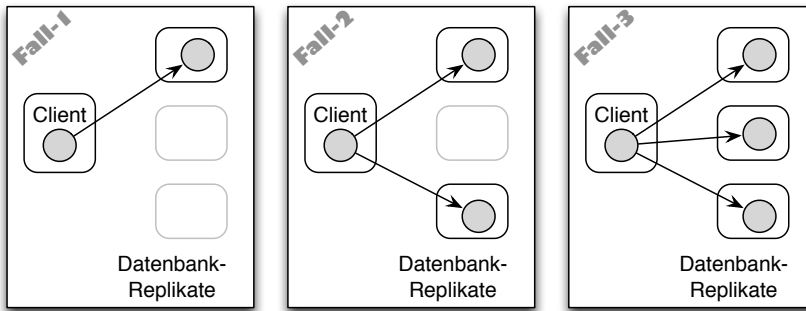


Bild 7.4 Partitionierung, oder: Wann ist eine Schreiboperation erfolgreich?

Partitionstoleranz gibt an, wie tolerant ein System gegenüber dem Ausfall einzelner Partitionen (bzw. der Verbindung zwischen einzelnen Partitionen) ist.

In Fall 1 könnte es zu kurzfristiger Inkonsistenz kommen: Aktualisiert ein Client Daten auf dem Knoten K1, während die Verbindung zwischen K1 und dem Rest des Systems unterbrochen ist, werden andere Clients bei Anfragen nach diesen Daten gegen die anderen Knoten des Systems andere (d. h. inkonsistente) Antworten erhalten.

Die ausführliche Diskussion dieses Thema füllt Bände⁸ – der Einfachheit halber empfehle ich Ihnen den anschaulichen grafischen Überblick von Nathan Hurst („Visual Guide to NoSQL“)⁹ – in Kurzform in Bild 7.5:

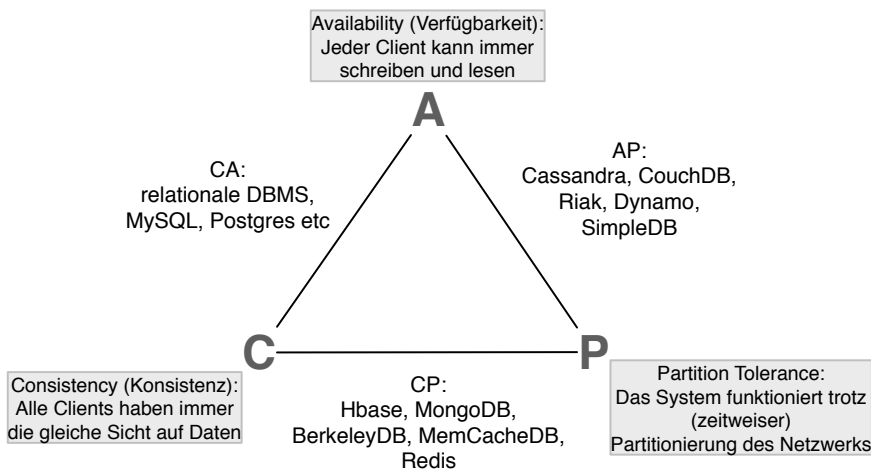


Bild 7.5 CAP-Eigenschaften einiger NoSQL-Datenbanken⁸

⁸ Ich empfehle den Originalautor Eric Brewer, der eine aktualisierte Einschätzung des CAP-Theorems („12 Years later“) gibt: <http://www.infoq.com/articles/cap-twelve-years-later-how-the-rules-have-changed>

⁹ Online in seinem Blog: <http://blog.nahurst.com/visual-guide-to-nosql-systems>

7.1.2.3 Zugriff und Navigation

Bezüglich der Zugriffe auf Ihre Daten und Navigation zwischen diesen sollten Sie prüfen, ob Sie charakteristische Muster für diese Zugriffe erkennen können. Beispielsweise müssen Sie bei den meisten Speichermodellen parallele Schreibzugriffe auf Daten über Sperren verhindern.

Sollten Sie allerdings fast ausschließlich lesende Zugriffe auf Ihre Daten erwarten, so können Sie diese durch Replikation, Parallelisierung, Caching und Indizierung signifikant beschleunigen – was allerdings möglicherweise Ihre Schreibperformance deutlich einschränkt.

Benötigen Sie die Möglichkeit, deklarativ und flexibel ständig neue Abfragen auf Ihre Daten zu formulieren? Dafür wurde SQL geschaffen, das Sie ans relationale Modell bindet. Feste Abfragen können Sie mittels Map-Reduce-Verfahren a priori ausrechnen lassen, was erhebliche Geschwindigkeitsvorteile mit sich bringen kann, allerdings zurzeit primär von NoSQL-Systemen unterstützt wird.

7.1.2.4 Deployment und Betrieb

Ob Ihr System auf einem leistungsfähigen Server-Cluster abläuft, einem Desktop oder Notebook, auf kleinen Smartphones oder sogar auf embedded-Devices – für die Ausgestaltung Ihrer Persistenz machen diese unterschiedlichen Ablaufumgebungen riesige Unterschiede. Ich empfehle Ihnen realitätsnahe Last- und Performance-Tests durch Prototypen, um die Leistungsfähigkeit Ihrer Ziel-Hardware möglichst frühzeitig zu prüfen.

Bei kommerziellen DBMS sollten Sie die Lizenzkosten für die konkrete Zielhardware, insbesondere bezüglich der Anzahl der Prozessoren, Prozessorkerne, Server-Instanzen oder Betriebsarten ins Kalkül ziehen.

Betriebliche Aspekte

Zur Laufzeit Ihrer Systeme kümmern sich in vielen Fällen dedizierte Personen (in der Rolle von Administratoren oder Betreibern) um geregelte, zuverlässige Abläufe. Diese Stakeholder fordern von Datenbanken folgende Eigenschaften:

- *Configuration*: Lässt sich die Datenbank an die Laufzeitumgebung oder an geänderte Anforderungen per Konfiguration anpassen?
- *Monitoring*: Lassen sich Laufzeitparameter und -verhalten automatisch überwachen?
- *Backup*: Über welche Möglichkeiten der Datensicherung verfügt die Datenbank? Ist eine Sicherung im laufenden Betrieb möglich? Lassen sich Zugriffsrechte auf Datensicherungen vergeben und Sicherungen oder bestimmte Teile davon sich zuverlässig wieder einspielen?
- *Portabilität*: Lässt sich die Datenbank mit anderen Betriebssystemen, virtuellen Maschinen oder Netzwerkkonfigurationen zuverlässig betreiben?

In der Realität könnten Ihre Betreiber auch nach Möglichkeiten zum Tuning fragen, zur Anbindung an vorhandene Rechte-Management-Systeme oder gar Migration. Ich gehe auf diese (schweren) Fragen hier nicht weiter ein.

7.1.3 Lösungsmuster

7.1.3.1 Persistenzschicht

Entkoppeln Sie in Ihren Architekturen den fachlichen Kern Ihrer Systeme von der gewählten (technischen!) Persistenz. Dazu stelle ich Ihnen hier die *Persistenzschicht* (siehe Bild 7.6) vor, die gut zum Modell der Anwendungsschichten (siehe Abschnitt 4.5.3) passt.

Die hier vorgestellte Persistenzschicht gehört zur Infrastrukturschicht. Ob Sie die Persistenz als getrennte Anwendungsschicht modellieren oder als integralen Bestandteil der Infrastruktur, bleibt Ihnen überlassen.

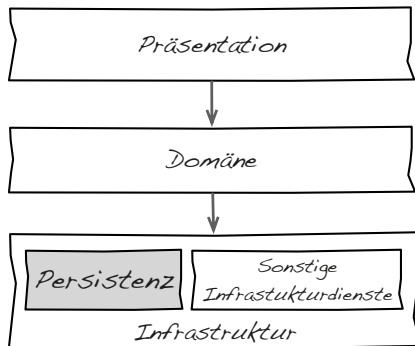


Bild 7.6

Persistenzschicht und Infrastrukturschicht

Wie funktionieren Persistenzschichten?

Eine Persistenzschicht kapselt die Details der Datenspeicherung gegenüber den übrigen Anwendungsteilen. Sie können Persistenzschichten unterschiedlich flexibel und aufwendig realisieren:

- Sie implementieren eine eigene Klasse, um die Persistenzaufgaben zu kapseln. Der folgende Abschnitt („Datenklassen“) diskutiert einige Vor- und Nachteile dieses Ansatzes.
- Sie verwenden ein Framework, wie beispielsweise Hibernate. Das bringt Ihnen etwas höhere Komplexität, allerdings auch ein erhebliches Maß an Flexibilität und Robustheit. Für relationale Datenbanken stellt dies eine weit verbreitete Standardlösung dar, die viele Risiken von Persistenz mindert. Allerdings orientieren sich viele Persistenz-Frameworks heute noch an relationalen Datenbanken. Für NoSQL-Speichermodelle müssen Sie unter Umständen selbst Lösungen entwickeln.

Datenklassen: Die einfachste Persistenzschicht

Wenn Sie die Persistenz Ihrer Fachklassen in dedizierten Datenklassen kapseln, erreichen Sie eine bessere Kapselung von Fachdomäne und Infrastruktur. Diesen Ansatz illustriert Bild 7.7. Er wird unter anderem in dem von Microsoft favorisierten Ansatz der „ActiveX Data Objects“ (ADO) verfolgt.

Änderungen an der Datenbank oder dem zugrunde liegenden Datenmodell bewirken dabei „nur“ Änderungen an den Datenklassen, nicht aber an den Fachklassen. Eine vollständige Trennung von Anwendungslogik und Infrastruktur erreicht dieser Ansatz nicht – Ihre Entwickler müssen immer noch die technischen Grundlagen der Datenbank beherrschen.

Datenklassen
Active Data Objects (ADO)

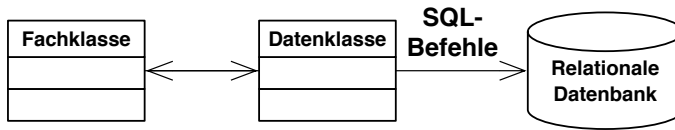


Bild 7.7
Persistenzschicht
mit Datenklassen

Schwierig ist dieser Ansatz auch dann, wenn Sie vernetzte Objektstrukturen speichern wollen, und nicht nur „einzelne“ Objekte. Die Abhängigkeit zwischen Klassenstrukturen und Tabellenstrukturen ist sehr hoch, was Änderung und Optimierung erschwert.



Verwenden Sie diesen Ansatz in folgenden Fällen:

- Sie benötigen keine flexible und hochgradig wartbare Persistenzschicht, und Ihre Anwendung verfügt nur über wenige Fachklassen.
- Sie persistieren nur einzelne Objekte, keine vernetzten Objektstrukturen. Die Vererbungstiefe ist klein.
- Endtermin und Entwicklungskosten werden in Ihrem Projekt deutlich höher bewertet als Flexibilität und Wartbarkeit. Höhere Änderungs- und Anpassungsaufwände werden von Kunden und Auftraggebern akzeptiert.
- Die zu speichernden Klassen des fachlichen Modells sind Datencontainer, haben viele Attribute und wenige Methoden. Ihre Abbildung auf relationale Strukturen ist sehr einfach und keinen Änderungen unterworfen.
- Es gibt in Ihrer Organisation keine wiederverwendbare Persistenzschicht und kein (kommerzielles) Persistenz-Framework.

Anforderungen an Persistenzschichten

Wenn Sie eine Persistenzschicht oder eine Persistenzkomponente entwerfen, sollten Sie folgende Aspekte als mögliche Anforderungen beachten:¹⁰

- Einfache Kapselung des Persistenzmechanismus. Fachliche Klassen benutzen lediglich Methoden wie `save()`, `read()` oder `delete()`, alle weiteren Dinge verbirgt die Persistenzschicht.
- Vollständige Kapselung der Persistenz. Dabei werden auch die oben erwähnten Methoden vor den Fachobjekten versteckt: `read()` verbirgt sich in einer Factory, `save()` erledigt ein Transaktionsmechanismus (alles, was „dirty“ (geändert) ist, wird gespeichert), `delete()` bearbeitet ein Destruktor.
- Unterstützung unterschiedlicher Persistenzmechanismen oder Speichermodelle: Datenbanksysteme, Dateien oder auch Legacy-Systeme.
- Unterstützung von Vererbungshierarchien und Polymorphismus.
- Unterstützung von Transaktionen.

¹⁰ Nach [Ambler2000] und [Keller98]. In der Realität habe ich noch viele weitere Anforderungen erlebt: Locking- und Caching-Strategien, spezielle Anfragesprachen, Connection-Pooling, spezielle Assoziationen zwischen persistenten Objekten, kaskadierende Operationen, Replikation, Backup, Benutzer- und Rollenkonzepte, Verschlüsselung ... Bei dieser Menge an Anforderungen denken Sie hoffentlich noch mal über das Selbst-Entwickeln nach ...

- Unterstützung von Lazy-Loading: Es werden immer nur diejenigen Teile von Objektgraphen aus der Datenbank in den Speicher geladen, die man aktuell benötigt. Hilfreich wäre auch das Pendant – Eager-Loading: Laden auf Vorrat.
- Bearbeitung mehrerer Objekte auf einmal, analog dem Cursor-Konzept von SQL.
- Automatische Erzeugung eindeutiger Object-Identifizier (OIDs).
- Unterstützung mehrerer Verbindungen (connections) parallel. Dies ist bereits für das Kopieren von Objekten von einer Datenbank in eine andere notwendig.
- Eine programmatische Schnittstelle (= API) für die Erstellung und Ausführung von Abfragen (Queries).
- Generierung von Migrationsskripten bei Änderungen des Datenmodells.
- Toleranz gegenüber Maßnahmen zum Performance-Tuning von Datenbanken. Solche Optimierungen kommen recht häufig vor und sollten ohne Änderungen am Quellcode von Anwendungssystemen funktionieren.



Einige Fragen und Antworten zu Persistenz

von Wolfgang Keller

- *Zu welchem Zeitpunkt in einem (objektorientierten) Projekt sollte man die relationale Datenbank entwerfen?*
Entwerfen Sie die Tabellen auf Basis des Objektmodells, nachdem Sie einen Architekturprototyp implementiert haben.
- *Was ist eine gute Struktur für ein Persistenzsubsystem?*
Sie sollten zwei Subsysteme für die Persistenz vorsehen, die eine Schichtenstruktur bilden. Die obere Schicht (Objektschicht, *object layer*) kapselt die Konzepte der Objektorientierung, während die untere Schicht (*storage layer*) eine Schnittstelle zu Ihren physikalischen Speichermedien, Datenbanken oder Dateien bildet.
- *Wie verbindet man eine Persistenzschicht mit einem transaktionsbasierten Mainframe-Datenserver?*
Benutzen Sie einen Agenten oder Vermittler, der Aufrufe an den Mainframe bündelt (*request bundling*) und sie als Block unter der Kontrolle des (Mainframe) Transaktionsmonitors ausführt.
- *Wie repräsentiert man die Individualität eines Objekts in einer relationalen Datenbank?*
Erzeugen Sie synthetische Schlüssel (*object identifier*), die jedes Objekt von seiner Geburt bis zu seiner Zerstörung begleiten. Begraben Sie diesen Schlüssel mit dem Objekt.
- *Wie stellt man die Identität eines Objekts sicher (und behält sie)?*
Erzeugen Sie einen Objekt-Cache für jeden Client-Prozess der Datenbank. Dieser Cache basiert auf einem Container (Array, Vektor, Set oder Ähnliche), der Object Identifier auf Objekte (repräsentiert durch Pointer oder Proxies) abbildet. Der Objekt-Cache enthält Zeiger auf die instantiierten Objekte, die aus der Datenbank gelesen wurden.

- *Wie verhindert man, dass alle „verwandten“ (assoziierten) Objekte geladen werden, wenn man ein Objekt bearbeitet, das viele Beziehungen zu anderen Objekten hat?*
Benutzen Sie Proxy-Objekte, die nur den Object Identifier (OID) enthalten, sowie einen Zeiger. Dieser Zeiger kann auch NULL sein – das Objekt wird erst dann geladen, wenn es tatsächlich benötigt wird.
- *Wie handhabt man Transaktionen?*
Machen Sie aus einer Transaktion ein Objekt mit Methoden begin, commit, rollback, retry (TransactionObject, siehe [Keller97]).
- *Wie füllt man Auswahllisten, List-Boxen oder Combo-Boxen?*
Erzeugen Sie passende Datenbank-Views. Diese sollten nur die notwendigen Attribute plus einen Primärschlüssel enthalten (*Narrow Views*).
- *Wie kann man das Füllen von Auswahllisten oder Combo-Boxen noch weiter beschleunigen (und den Transfer nicht benötigter Daten vermeiden)?*
Laden Sie diese Daten in Blöcken mit vertretbarer Zugriffs- oder Ladezeit. Als Daumenregel empfehlen wir etwa 30–50 Datensätze oder das Doppelte der Zeilenzahl der Auswahlliste (*Short Views*).
- *Wie kann man den performanten Zugriff auf große Datenmengen durch eine objektrelationale Persistenzschicht gewährleisten?*
Benutzen Sie „stored procedures“ oder eine spezialisierte Zugriffskomponente ausschließlich für diesen Zweck. Diese sollte exakt die benötigten Daten auf einmal laden (*Cluster Read*).
- *Wie verkürzt man die Wartezeit, wenn große Datenmengen in eine Datenbank geschrieben werden sollen?*
 - Sammeln Sie die Schreiboperationen innerhalb der Persistenzschicht und schicken Sie diese en bloc an die Datenbank (*bundled write*).
 - Speichern Sie Daten in einem lokalen Puffer und geben Sie die Kontrolle an das System zurück. Starten Sie einen parallelen Thread, der diesen Puffer an die Datenbank übermittelt (*store for forward*).
 - Schreiben Sie die Daten in eine (eventuell transaktionsgesicherte) Datei und laden Sie diese Datei zu einem späteren Zeitpunkt in die Datenbank (*flat file write*).
 - Arbeiten Sie asynchron, etwa durch eine Warteschlange (*message queue*).
Kommerzielle Message-Queue-Produkte unterstützen dabei auch Transaktionen.

Diese Fragen und Antworten repräsentieren einen kleinen Teil der Entwurfsmuster für objektrelationale Persistenzschichten, die [Keller98] und [Keller97] ausführlich vorstellen. Bevor Sie selbst eine Persistenzschicht implementieren, sollten Sie diese Entwurfsmuster gründlich lesen.

Wolfgang Keller (Email: wk@objectarchitects.de) arbeitet als selbstständiger Projektmanager und Berater für Unternehmensarchitekturen. Davor war er u. a. Entwicklungsmanager bei der Generali Vienna Group, an der Entwicklung mehrerer großer OO-Systeme beteiligt, hat Zugriffsschichten selbst implementiert und zugekaufte verwendet. Als Hobby betreibt er eine Website, unter anderem mit Patterns zum Thema Persistenz (www.objectarchitects.de).

7.1.3.2 DAO: Eine Miniatur-Persistenzschicht

Es gibt ein bedeutendes Entwurfsmuster, das Sie im Zusammenhang mit Persistenz kennen sollten, das Data Access Object (DAO). Es stellt eine Art Miniatur-Persistenzschicht dar und entkoppelt die eigentliche Datenspeicherung von der jeweiligen Anwendung. DAOs arbeiten meist mit sogenannten Data Transfer Objects (DTO) oder Transferobjekten zusammen. Sie erhöhen die Flexibilität von Systemen hinsichtlich Details der Persistenzschicht und vereinfachen dadurch beispielsweise Versionswechsel zugrunde liegender Datenbanken oder ähnliche Änderungen. In Bild 7.8 und Bild 7.9 sehen Sie die statische Struktur sowie den schematischen Ablauf.

In Java-Systemen, insbesondere ab JEE6, machen die Annotationen für Persistenz den Einsatz des DAO-Patterns in den meisten Fällen obsolet.¹¹

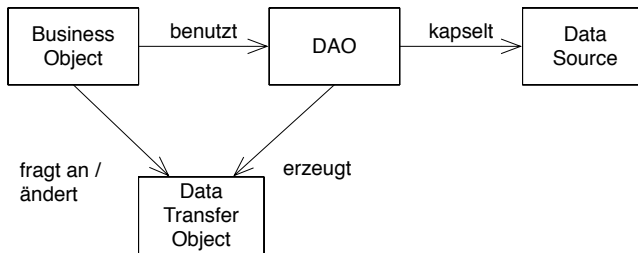


Bild 7.8
Struktur
des DAO-Musters

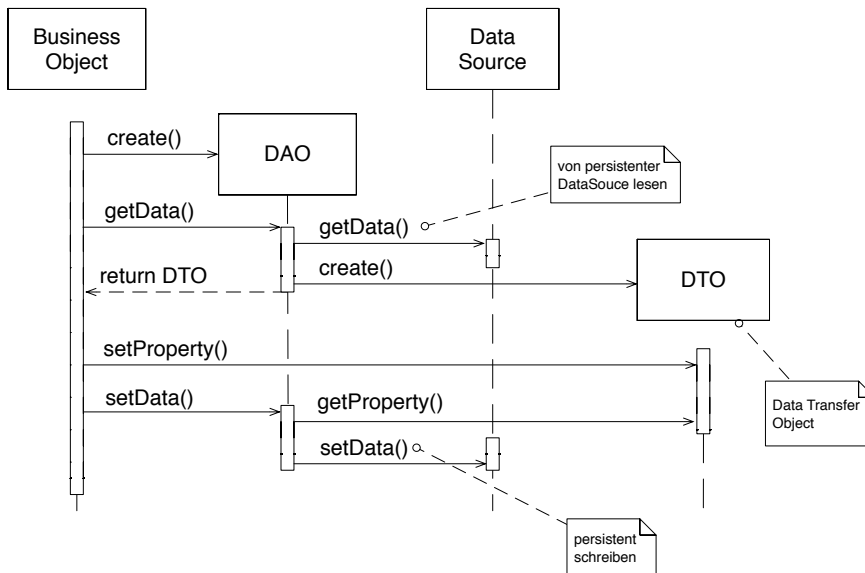


Bild 7.9 Schematischer Ablauf bei Nutzung des DAO-Musters

¹¹ Der Java- und Tomcat-Guru Peter Roßbach nennt das „POJO schlägt DAO“.

7.1.4 Bekannte Risiken und Probleme

Risiko durch Abhängigkeit von technischen Details des DBMS

Praktisch alle Anbieter von Datenbanksystemen (relationale wie auch nichtrelationale) entwickeln ihre Produkte kontinuierlich weiter. Als Architekt müssen Sie daher bei Versionswechsel Ihrer Datenbank mit Änderungen am Quellcode Ihrer Systeme rechnen, es sei denn, eine Persistenzschicht (siehe folgender Abschnitt) kapselt solche Änderungen.

Es ist grundsätzlich möglich, während der Lebens- und Nutzungsdauer eines Systems den zugrunde liegenden Persistenzmechanismus auszutauschen, etwa zwecks Performance-Steigerung oder aufgrund geänderter Einsatzszenarien der betroffenen Systeme. Falls Ihre Systeme modular aufgebaut sind und die Persistenz sauber kapseln, sollten solche Operationen mit überschaubarem Risiko möglich sein. In allen anderen Fällen sollten Sie solche Änderungen als „Operation am offenen Herzen“ betrachten und entsprechende Risikovorsorge treffen.

Seiteneffekte, wenn Geschäftslogik in der Datenbank abläuft

Insbesondere relationale Datenbanken verfügen meist über sogenannte *Trigger*: Operationen, die beim Eintreten bestimmter Bedingungen direkt von der Datenbank ausgelöst werden.

Solche Trigger können beispielsweise Aufräumarbeiten wie „kaskadierendes Löschen“ implementieren, aber auch fachliche Logik („Wenn das Attribut Rechnungssumme > 100, dann erhöhe die Rabattstufe um eins“).

Sie ahnen sicherlich die Risiken dieser Trigger: Sie sind nicht Bestandteil des eigentlichen Sourcecodes Ihres Systems, operieren jedoch auf dessen Daten. Aus Sicht Ihres Systems (sei es Cobol, Java oder C#) realisieren Trigger *Seiteneffekte* – ein riskantes Unterfangen.

Ähnliches könnte Ihnen mit Stored-Procedures oder sogar (modifizierenden) Views passieren.



Ich habe Situationen erlebt, in denen Dutzende von Triggern auf der Datenbank eine wilde Mischung technischer und fachlicher Aufgaben realisierten, zwar hochperformant, aber weitgehend undokumentiert – parallel zur eigentlichen Anwendung.

Die Wartbarkeit und Erweiterbarkeit dieses Systems wurden durch diese Trigger erheblich negativ beeinflusst. Über die langjährige Lebensdauer dieses Systems bekamen die Trigger nach und nach den Status von „Einflussfaktoren und Randbedingungen“ anstelle von Lösungsmechanismen.

Setzen Sie Datenbank-Trigger daher mit Vorsicht ein.

- Vermeiden Sie es, fachliche Logik oder Regeln durch Trigger zu implementieren.
- Verwenden Sie Trigger nur, wenn Ihre normale Programmiersprache keine angemessene Möglichkeit für die betreffenden Operationen bietet.
- Dokumentieren Sie Trigger – nehmen Sie insbesondere in Ihren „normalen“ Quellcode Verweise auf die Trigger auf!

Kooperation mit Mainframe-Systemen

Manche Organisationen speichern große Teile ihrer Unternehmensdaten auf Mainframes (Großrechnern, Host-Systeme). Zugriffe und Speicherung dieser Daten erfolgen dann über Mainframe-Programme und deren häufig Cobol-basierte Schnittstellen.

Viele dieser Mainframe-Programme enthalten fachliche Logik (Plausibilitätsprüfungen), die das Speichern und Lesen von Daten beeinflussen. Manchmal ist es schwierig, die bestehende Logik mit der Fachlichkeit des neuen Systems zu koordinieren. Insgesamt bedeutet die Interaktion mit solchen Systemen eine zusätzliche Komplexität durch technische und fachliche Abhängigkeiten.

Plausibilitäten

Strukturbrüche

In vielen Fällen kommt es bei der Entwicklung von Softwaresystemen zu einem Strukturbruch (dem sogenannten impedance mismatch (siehe Bild 7.10)) zwischen Programmiersprache und Datenbanksystem. Der typische Fall dieses Problems: Objektorientierte Programme speichern Daten in relationalen Datenbanken. Objektstrukturen im Hauptspeicher müssen in Tabellenstrukturen im Datenbanksystem übersetzt werden.¹² Weder syntaktisch noch semantisch ist diese Übersetzung eindeutig – ein weites und schwieriges Feld für Architekten!

Impedance mismatch

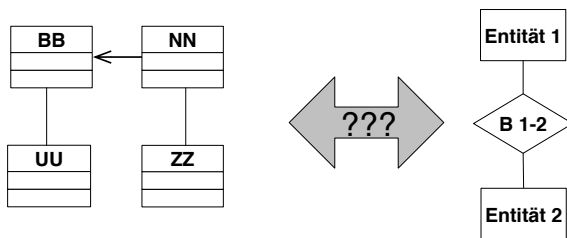


Bild 7.10

Strukturbruch zwischen objektorientierten und relationalen Modellen

7.1.5 Weitere Themen zu Persistenz

Zur Behandlung der Persistenz von Systemen gehören nach meiner Erfahrung neben dem Entwurf der Persistenzschicht noch folgende Aspekte:

Vererbung Aggregation
Assoziation

- Abbildung von Objektstrukturen auf Relationen (*OR-Mapping*)
- Entscheidung zwischen Eigenentwicklung oder Kauf (*Make-or-Buy*)

Abbildung von Objektstrukturen auf Relationen

Die Abbildung von Objekt- und Klassenstrukturen (Objekt-relationales Mapping, OR-Mapping) auf Relationen löst den Strukturbruch zwischen diesen beiden Paradigmen auf (siehe Bild 7.11). Für die Basiskonstrukte objektorientierter Sprachen (Vererbung, Aggregation, komplexe Typen und Assoziationen) legt ein solches Mapping fest, wie die Übersetzung in relationale Strukturen erfolgt.

OR-Mapping

¹² Leider lösen moderne NoSQL-Datenbanken dieses Mapping-Problem nicht automatisch; vielmehr gibt es inzwischen diverse neue Kategorien von Mappings: Objekte-auf-Dokumente, Objekte-auf-Key-Value-Repräsentation sowie beliebige Mischungen davon. Trotz NoSQL ist die Persistenzwelt also immer noch schwierig. ☺

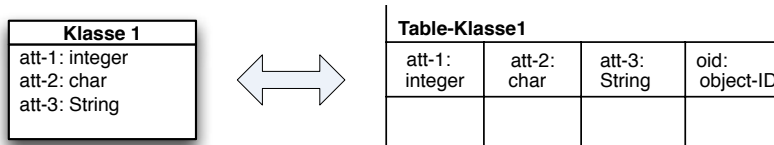
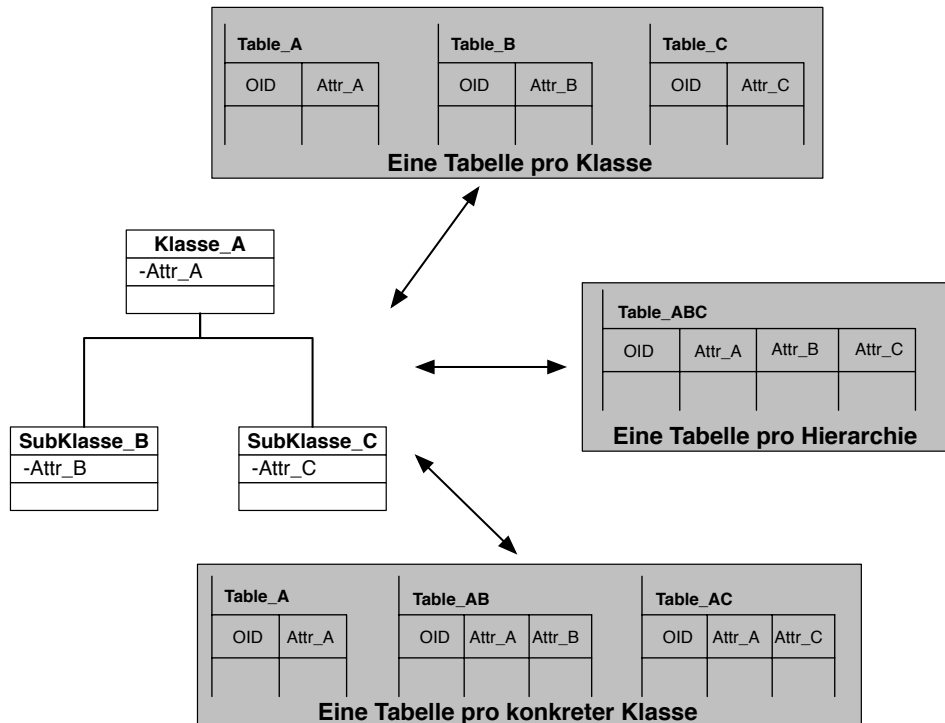


Bild 7.11 OR-Mapping für einfache Attribute



- Aggregation über Fremdschlüsselbeziehung: Zwischen den beteiligten Tabellen wird über den OID eine Fremdschlüsselbeziehung aufgebaut.
- Benutzung einer Überlauftabelle: Die ersten k Objekte einer 1:n-Assoziation oder einer Aggregation werden mit in die Tabelle zu einer Klasse aufgenommen, alle weiteren in der Überlauftabelle gespeichert.
- Assoziationen werden zu Fremdschlüsselbeziehungen (*foreign key relations*), wobei der Objektschlüssel (OID) des referenzierten Objektes zum Fremdschlüsselattribut wird.
- Zur Abbildung von Vererbung gibt es unterschiedliche Strategien (siehe Bild 7.12):
 - Eine Tabelle pro Klasse der Vererbungshierarchie: Die Attributwerte eines Objektes aus einer abgeleiteten Klasse befinden sich dabei verteilt auf alle Tabellen der Klassen, die vor ihm in der Hierarchie stehen. Objekte werden durch Join-Operationen über alle betroffenen Tabellen gelesen.
 - Eine Tabelle pro Hierarchie. Diese Tabelle (in Bild 7.12 „Table_ABC“) vereinigt die Attribute aller Klassen einer Klassenhierarchie in einer Tabelle. Hierbei benötigt man kein Join mehr zum Zusammensetzen eines Objektes aus einer abgeleiteten Klasse.
 - Eine Tabelle pro konkreter Klasse. Es gibt keine Tabelle für abstrakte Basisklassen. Für jede konkrete Klasse existiert eine Tabelle, in der ihre Attribute und die Attribute aller in der Hierarchie vor ihr stehenden Klassen aufgenommen werden.

Zur Entscheidung für eine konkrete Strategie des Mappings müssen Sie einige Aspekte berücksichtigen:

- Die Struktur Ihrer Objekte zur Laufzeit. Wie häufig sind polymorphe Objekte? Wie umfangreich werden Aggregationen im Durchschnitt? Wie tief sind die Vererbungshierarchien?
- Benötigen Sie den Zugriff auf Objekte über ihre jeweiligen Oberklassen? Dann brauchen Sie Unterstützung für Polymorphismus. Bevor Sie den mühevoll implementieren, erwägen Sie, auf Implementierungsvererbung ganz zu verzichten und stattdessen nur Protokollvererbung zu benutzen.
- Die Struktur der Anfragen (queries). Wie häufig resultieren Anfragen in (zeitaufwendigen) Joins auf der Datenbank? In welchem Zahlenverhältnis stehen die CRUD-Operationen zueinander (gibt es etwa mehr lesende oder mehr schreibende Operationen)?
- Welche Anfragen lassen sich durch ein alternatives Mapping optimieren?
- Wie unterscheiden sich Schreib- und Leseverhalten des Mappings?
- Wie speicheraufwendig ist ein bestimmtes Mapping?
- Wie gewichten Kunden und Auftraggeber Speicherplatz und Performance gegeneinander? Manche Mapping-Strategien nutzen Speicherplatz optimal aus und sind dafür bei vielen Operationen langsamer.

In der Realität werden Sie für Ihr konkretes Objektmodell oft eine Mischung aus den unterschiedlichen Strategien wählen, die sich an den konkreten Bedürfnissen orientiert. Tabelle 7.2 (in Anlehnung an [Keller97]) zeigt einige Vor- und Nachteile verschiedener Mapping-Strategien auf.

Eine flexible Persistenzschicht sollte in jedem Fall unterschiedliche Mapping-Strategien unterstützen.

Mapping-Strategie

Tabelle 7.2 Vergleich der Mapping-Strategien von Vererbungsbeziehungen

	Performance			Platz- bedarf	Änderbar- keit
	Schreiben	Lesen	Polymorphes Lesen		
Eine Tabelle pro Klasse	–	–	–o	+	+
Eine Tabelle pro Vererbungspfad	+	+	–	+	–
Eine Tabelle pro konkreter Klasse	+o	+o	+	–	+

Legende: + = Gut, – = Schlecht, * = irrelevant, o = neutral

Weitere Patterns zur Persistenz

In seinem aus meiner Sicht sehr hilfreichen Buch „Patterns of Enterprise Application Architecture“ ([Fowler]) beschreibt Martin Fowler eine Reihe weiterer Lösungsmuster, die für den Entwurf von Persistenzkonzepten nützlich sein können. Eine ausführliche Darstellung würde den Rahmen dieses Buches sprengen, doch sind zumindest Kurzbeschreibungen seiner Patterns online¹³ verfügbar.

Beachten Sie insbesondere die folgenden Themen:

- Data Source Architectural Patterns, etwa: Table und Row Data Gateways, Active Record sowie Data Mapper
- objektrelationale Verhaltensmuster, etwa: Lazy Load
- objektrelationale Strukturmuster, etwa: Identity Field, Foreign Key Mapping, Single, Class oder Concrete Table Inheritance

Eigenentwicklung oder Kauf (Make-or-Buy)?

Eine leistungsfähige und flexible Persistenzschicht zu entwickeln ist schwierig und aufwendig. Sie ist abhängig von den zugrunde liegenden Datenbanken. Versions- oder Herstellerwechsel ziehen Änderungen der Persistenz-Komponente nach sich. Diese Aspekte erschweren die Erstellung und Pflege einer eigenen Persistenzschicht. Für relationale Datenspeicherung rate ich Ihnen vom Eigenbau fast grundsätzlich ab. Für Datenspeicherung in NoSQL-Systemen verbreiten sich zurzeit (Stand 2013) erste Systeme.



Sie sollten in einem funktionalen Prototyp die Leistungsmerkmale verfügbarer Persistenzschichten mit den Anforderungen Ihres Systems abgleichen (und dabei alle beteiligten Legacy-Systeme berücksichtigen). Erst auf dieser Grundlage sollten Sie eine Make-or-Buy-Entscheidung treffen.

Da im Bereich „Persistenz“ viele Frameworks quelloffen sind, können Sie durchaus diese Frage erweitern auf „make, modify or buy?“

¹³ <http://martinfowler.com/eaCatalog/>

7.1.6 Zusammenhang zu anderen Aspekten

Sicherheit

Sie müssen gewährleisten, dass die Sicherheit der bearbeiteten Daten auch über die Persistenzschicht hinweg gewährleistet bleibt. Außerdem dürfen auch Protokolle oder Logs keinerlei sicherheitsrelevante Informationen enthalten.



Beispiel: In einem System wurden sensible Personendaten bearbeitet und gespeichert. Das System selbst wurde durch ausgeklügelte technische und organisatorische Mechanismen (Smartcards, 4-Augen-Prinzip) vor Missbrauch geschützt. Die Persistenzschicht erlaubte durch ein Benutzer-/Rollenkonzept nur autorisierten Benutzern den Zugriff auf diese sensiblen Daten. Die zugrunde liegende Datenbank verfügte ebenfalls über einen adäquaten Zugriffsschutz.

Leider wurde die tägliche Sicherung des gesamten Datenbestandes als „Dump“ auf ein Band geschrieben. Mit einfachen Kommandos des Betriebssystems ließen sich sämtliche Daten von diesem Band auch außerhalb des Systems wieder lesen. Alle Sicherheitsmechanismen des Systems, der Persistenzschicht und der Datenbank wurden dadurch wirkungslos.

Das Problem entdeckte übrigens ein externer Review; es wurde durch eine Anpassung des Sicherungskonzepts wirksam gelöst.



Wenn Sie mit sicherheitskritischen Daten arbeiten, prüfen Sie sämtliche Kopien dieser Daten auf ihre Sicherheit. Eine hochgradig gesicherte Persistenzschicht genügt nicht, wenn gleichzeitig Daten im (ungesicherten) Dateisystem abgelegt werden.

Transaktionen

Softwaresysteme lösen oftmals mehrere logisch zusammengehörige (Datenbank-) Operationen aus. Diese müssen oft dem „Alles-oder-Nichts“-Prinzip genügen: Entweder sämtliche Operationen sind erfolgreich, oder keine einzige darf ausgeführt werden. Solche Transaktionen dienen dazu, die Konsistenz und fachliche Korrektheit eines Datenbestands zu sichern. Persistenz und Transaktionen hängen von daher eng miteinander zusammen.

Alles-oder-Nichts

Protokollierung und Monitoring

Die Abbildung von Objekten auf Tabellen (OR-Mapping) können Sie durch entsprechende Protokollierung optimieren. Lassen Sie beispielsweise die SQL-Anweisungen der Persistenzschicht über einige Zeit protokollieren. Dann können Sie prüfen, ob sich etwaige Join-Operationen durch kontrollierte Einführung von Redundanzen (Stichwort: Denormalisierung) vermeiden lassen.

Protokolle helfen bei Optimierung

Caching

Caching optimiert
Zugriffszeiten

Der Zugriff auf persistente Speichermedien ist hinsichtlich Zeit- und Ressourcenaufwand meistens mehrere Größenordnungen aufwendiger (d. h. teurer) als Zugriffe auf Objekte im Speicher. Durch

Zwischenlagern oder Vorhalten von Objekten im Speicher können Sie Zugriffszeit sparen, allerdings auf Kosten eines signifikant höheren Verwaltungsaufwands. Kritische Fragestellungen beim Caching sind beispielsweise:

- Nach welcher Strategie nehmen Sie Daten in den Cache-Speicher auf und wie/wann wieder heraus?
- Halten Sie die Daten zum Lesezugriff im Cache, oder wollen Sie auch schreibende Zugriffe beschleunigen? Bei Systemabstürzen besteht dann das Risiko, dass Inkonsistenzen zwischen dem persistenten Speichermedium und dem (vor einem Absturz noch nicht gesicherten) Cache auftreten.
- Wie groß dimensionieren Sie den Cache für welche Art von Objekten?

Generieren der Persistenz mit MDA oder MDSD

MDSD kann Persistenz
vereinfachen

MDA oder MDSD-Generatoren können Ihnen eine Menge Aufwand bei der Entwicklung Ihrer Persistenz abnehmen. Beispielsweise können Sie Entitäten in einem fachlichen Modell über Stereotypen („Entity“)

kennzeichnen und den Generator und dessen Templates die notwendigen Codeartefakte (Klassen, SQL-Skripte, Konfigurationsdateien etc.) erzeugen lassen.

Schwieriger gestalten sich bei diesem Vorgehen *Releasewechsel* Ihrer Systeme, weil die Generatoren in der Regel die bestehenden Datenbestände nicht berücksichtigen, d. h. keine *Migrationsskripte* generieren. Diese müssen Sie auf Basis der bestehenden und neuen Tabellen- oder anderer Speicherstrukturen selbst entwickeln.

7.1.7 Praktische Vertiefung

Damit Sie sich selbst einen praktischen Überblick verfügbarer Werkzeuge zur Speicherung von Daten verschaffen können, empfehle ich Ihnen die folgenden Einstiegspunkte (mit jeweils steigender Komplexität beziehungsweise Leistungsfähigkeit). Experimentieren Sie selbst¹⁴ – das macht Spaß und hat einen großen Lerneffekt:

- Speichern Sie beliebige Datenstrukturen und Objekte aus Python (oder Jython) mit Hilfe des Standardmoduls „pickle“. Dabei lernen Sie eine einfache, aber effektive Art der Serialisierung ins Dateisystem kennen. Für Fehler- und Ausnahmebehandlung müssen Sie selbst sorgen – dafür gibt's praktisch keinen technischen Overhead.
- Experimentieren Sie mit einer embedded-Datenbank. Das könnte beispielsweise die (relationale) H2¹⁵ sein, oder auch die objektorientierte db40¹⁶. Hier können Sie schon mit wenigen Codezeilen Objekte in einer *echten* Datenbank speichern.

¹⁴ bzw.: Lassen Sie sich von dem vielseitigen Buch „Seven Databases in Seven Weeks“ [Redmond+12] inspirieren.

¹⁵ Siehe www.h2database.com, die kleine aber feine Open-Source-Datenbank.

¹⁶ www.db40.com (wird auf die Versant Website umgeleitet; Download-Archive dort verfügbar).

- Nun vertiefen Sie sich in die Java Persistence API (JPA). Darin hat (zumindest für die Java-Welt) die Speicherung von Daten eine immense Vereinfachung erfahren.
- Eine weitere interessante Möglichkeit der Speicherung und Suche von Daten bietet das GORM (grails object relational mapping) des Groovy-basierten Web-Framework Grails. Natürlich können Sie auch das Ruby-Pendant dazu, Rails, für diesen Versuch nutzen.
- Das CoreData Modul aus Apples Betriebssystem MacOS® und iOS® leistet bei der Verwaltung und Speicherung von Daten (zumindest auf diesen Plattformen) gute Dienste.¹⁷
- Als Nächstes steht eine der *ausgewachsenen* relationalen Datenbanksysteme an. Ob MySQL oder PostgreSQL, spielt dabei kaum eine Rolle – selbst von den bekannten kommerziellen Datenbanksystemen¹⁸ bekommen Sie in der Regel kostenfreie Testversionen. Auf diese Systeme können Sie aus praktisch allen gängigen Programmiersprachen zugreifen, SQL lernen und die Probleme des objektrelationalen Mappings am eigenen Code erfahren. In jedem Fall sollten Sie zugehörige Managementwerkzeuge in Ihre Experimente einbeziehen.
- Nun erarbeiten Sie sich Kenntnisse (mindestens) in einer der modernen NoSQL-Datenbanken, vielleicht CouchDB, MongoDB oder RIAK¹⁹. Alle drei sind Open Source und ermöglichen hochgradig skalierbare Speicherstrukturen. Sie formulieren Ihre Abfragen mit Hilfe von map/reduce-Funktionen und können in typischer Webmanier mit REST und http auf Ihre Daten zugreifen. Beide Vertreter geben Ihnen einen groben Einblick, wie wirklich große Systeme speichertechnisch *ticken* können. Zum tieferen Verständnis konsultieren Sie [Edlich+11], der viele Aspekte rund um NoSQL anschaulich und vertieft erklärt.
- Als Nächstes lernen Sie Speicherkonzepte der *Cloud* kennen. Dafür empfehle ich Ihnen den S3-Service von Amazon®. Eine erste Übersicht gibt <http://www.hongkiat.com/blog/amazon-s3-the-beginners-guide/>.
- Falls Sie noch mehr Interesse an Datenspeicherung haben, könnten Sie mit den Konzepten von DataWarehouse- oder Business-Intelligence-Systemen fortfahren. Diese OLAP (online analytical processing)-Systeme dienen primär der flexiblen Analyse und Auswertung von Daten.

7.1.8 Weiterführende Literatur

[Edlich+11] führen in die Welt der NoSQL-Datenbanken ein. Sie beschreiben deren konzeptionelle Grundlagen (wie map/reduce oder das CAP-Theorem) sowie einige typische Vertreter. Leicht zu lesen – jedoch entwickelt sich die NoSQL-Welt schneller weiter, als Verlage Papier bedrucken können. Aus diesem Grund betreibt der Hauptautor die lesenswerte NoSQL-Website (<http://nosql-database.org/>).



[Fowler02] erläutert diverse praxisgerechte Muster zur Persistenz, unterlegt mit kurzen Codebeispielen. Ein wichtiger Lese- und Nachschlagetipp für Persistenz, diese Patterns sind wirklich brauchbar!

¹⁷ Sofern Sie sich auf ObjectiveC einlassen wollen und einen Mac zum Ausprobieren griffbereit haben. Letzteres empfehle ich Ihnen ohnehin.

¹⁸ Beispielsweise die Express-Edition (XE) von Oracle®.

¹⁹ Unter <http://vimeo.com/11240885> sehen Sie, wie Sie in weniger als 60 Minuten einen 3-Node RIAK-Cluster aufsetzen können – das hat einen sehr hohen Geek-Faktor.

[Keller97] beschreibt Strategien zur Abbildung von Objekten auf Tabellen und diskutiert ausführlich deren Vor- und Nachteile.

[Redmond+12] stellt sieben unterschiedliche Datenbanksysteme vor – und erklärt sehr praxis- und codenah deren Programmierung. Mir hat's außerordentlich gut gefallen.

Herzlichen Dank an Jürgen Krey, Lothar Piepmeyer, Peter Roßbach, Khalid Dermoumi, Michael Perlin, Robert Reimann, Till Schulte-Coerne und Stefan Tilkov für die hilfreichen Beiträge und Anregungen zur Persistenz. Dieses Thema hätte ein eigenes Buch verdient.

■ 7.2 Geschäftsregeln

Geschäftsregeln: Domänenspezifische Kausalzusammenhänge oder bedingte Handlungsanweisungen.

7.2.1 Motivation

In der Praxis existieren viele Zusammenhänge der Form „Wenn eine Bedingung X erfüllt ist, dann handle auf folgende Art und Weise: ...“.

Geschäftsregeln

Ich nenne Zusammenhänge dieser Art nachfolgend „Geschäftsregeln“ oder auch Fachlogik – obwohl sie manchmal sehr technischer Art

sein können.²⁰

Statt einer förmlichen Definition möchte ich Ihnen einige Beispiele solcher Regeln aufzeigen:

- (Versicherungsbranche) Schadenfälle mit einem geschätzten Gesamtschaden von über 100 000 Euro werden nur in der Zentrale reguliert.
- (Versicherungsbranche) Gehört ein an einem Schadenfall direkt Beteiligter zur oberen Managementebene des Konzerns, so darf der Schaden nur von Sachbearbeitern der Vertraulichkeitsstufe V1 bearbeitet werden.
- (Finanzbehörde) Hat der Steuerpflichtige seine Termine zur Vorauszahlung mehr als zwei Mal erst nach Erinnerung wahrgenommen, so wird ihm 24 Monate lang kein Aufschub gewährt.

In vielen Branchen gelten Regeln dieser oder ähnlicher Art. Viele Aktionen oder Abläufe hängen von bestimmten Bedingungen oder Zuständen ab – und genau diese legen die unternehmens- oder domänenspezifische Fachlichkeit fest. Eine Entität „Kunde“ allein mit ihren Attributen macht eben noch keine Fachlichkeit aus – erst die Kombination aus Daten und (durch Regeln gesteuertem) Verhalten macht die „Musik“.

²⁰ Denken Sie etwa an Regeln für das Routing von Nachrichten innerhalb eines Systems – das manchmal von Auslastungs- oder Verfügbarkeitskriterien abhängt und nicht von fachlichen Bedingungen.

Oft stecken zu viele Geschäftsregeln im Code

Meine These zu Geschäftsregeln: (Zu) viele solcher Regeln stecken verborgen und fest zementiert in Form verschachtelter if-then-else-Konstrukte tief in schwer änderbarem Quellcode. Nur wenige fachliche Abläufe und Regeln können wir in Anwendungssystemen so einfach lesen und verstehen wie die oben genannten Beispiele. Nur mit großen Schwierigkeiten können wir in bestehenden Systemen neue Geschäftsregeln implementieren oder an aktuelle Gegebenheiten anpassen.

Leichte Anpassbarkeit stellt für viele Softwarearchitekturen jedoch ein wesentliches Entwurfsziel dar: Wir benötigen Entwurfs- oder Architekturmittel, um Geschäftsregeln einfach ändern zu können.

Verteilte Geschäftsregeln erschweren Änderungen

Woraus resultiert diese schwere Änderbarkeit? Wir haben doch Fachlichkeit von Technik getrennt, Entwurfsprinzipien angewendet, brav dokumentiert und sind methodisch vorgegangen. Unsere Architektur enthält einen *business layer* mit fachlich motivierten Domänenklassen.

Grundsätzlich gut – die Trennung von Fachlichkeit und Technik schafft Überblick und führt zu verständlichen Strukturen. Leider schleichen sich in die geschäftlichen Bausteine (Klassen, Methoden, Funktionen) jedoch immer wieder Teile ein, die Änderungen erschweren:

Unabhängig von der jeweiligen Implementierungstechnologie stehen fachliche Objekte in solchen Architekturen dabei in gegenseitigen Abhängigkeitsbeziehungen. Fachliche Methoden referenzieren jeweils andere (fachliche) Objekte, um (fachliche) Abläufe zu zementieren, pardon, zu implementieren. Noch einmal: Fachliche Abläufe, ein Bestandteil der Fachlogik, entstehen aus dem aktiven Zusammenspiel einzelner Fachobjekte.

Beim Entwurf solcher Zusammenarbeit müssen fachliche Abhängigkeiten vorab feststehen. Zwischen den beteiligten Klassen entstehen statische Abhängigkeiten. Änderungen an der Fachlogik bedürfen hierbei der Anpassung von Quellcode, immer verbunden mit seinem erneuten Build- und Deploy-Zyklus der gesamten Applikation.²¹ Zu diesen Abhängigkeiten kommt das Navigationsproblem: Ein Fachobjekt muss möglicherweise vor der eigentlichen fachlichen Tätigkeit durch komplexe Objektgraphen zu anderen Objekten navigieren. So entstehen aus vermeintlich einfachen fachlichen Operationen häufig umfangreiche (und gar nicht mehr einfache) Codefragmente. Die Navigation zwischen Objekten hat nichts mit Fachlichkeit zu tun – sie basiert auf der Architektur und dem Design des jeweiligen Objekt- und Klassengeflechts. Damit bewerte ich sie als reine Infrastruktur- oder Technikaufgabe, die Wartbarkeit und Verständlichkeit von Quellcode negativ beeinflusst.

Das Sequenzdiagramm in Bild 7.13 zeigt diese Situation. Die Fachlogik (d. h. die Geschäftsregeln) ist auf drei Objekte verteilt. Beachten Sie insbesondere die Abfragen „if x“ und „if y“ im Sequenzdiagramm: Sollten sich diese Bedingungen ändern, folgt daraus zwangsläufig eine Änderung von Quellcode sowie ein erneutes Deployment der beteiligten Komponenten.

²¹ Daran ändert auch Dependency Injection kaum etwas: Sie können damit Abhängigkeiten reduzieren, aber praktisch nicht aus Ihren fachlichen Abläufen eliminieren.

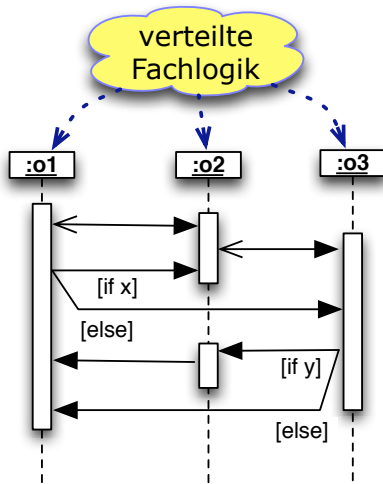


Bild 7.13
Fachobjekte mit verteilter Ablauflogik

Externalisierte Regeln zur Rettung

Alternativ können Sie einem Regelinterpretier (auch Regelmaschine, *rule-engine* genannt) zur Laufzeit die Ausführung der Fachlogik überlassen. Dazu beschreiben Sie Geschäftsregeln als eigenständige Einheiten (*first order citizens*) in einer Regelsprache.

Die grundsätzliche Vorgehensweise zeigt Bild 7.14. Ihr eigenes System übergibt die verfügbaren Geschäftsobjekte (o1, o2, o3) an die Regelmaschine. Die wiederum führt auf den Geschäftsobjekten die benötigten Methoden aus.

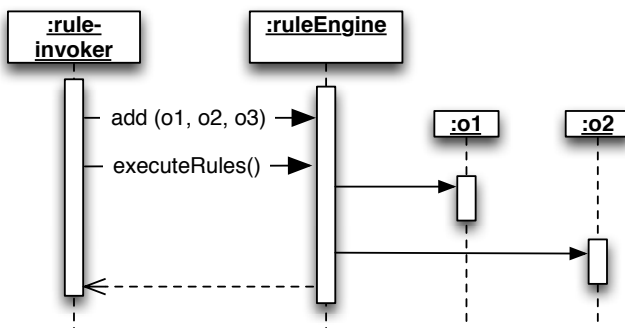


Bild 7.14
Regelmaschine steuert
Ablauflogik

Regeln in diesem Kontext bestehen aus Wenn-dann-Sätzen. Im Wenn-Teil (auch condition part oder „Left-Hand-Side“ bzw. LHS genannt) stehen eine Menge von Bedingungen. Der Dann-Teil der Regeln (auch „Right-Hand-Side“ bzw. RHS) beschreibt, was zu tun ist, falls diese Bedingungen erfüllt sind:

```

Rule "MySimpleRule"
  when <conditions>      //Left-Hand-Side, LHS
  then <consequence>    //Right-Hand-Side, RHS
end
  
```

Durch Regeln können wir Fachlogik an zentraler Stelle (beispielsweise in eigenen Dateien) beschreiben. Sie drücken Fachlichkeit deklarativ aus, im Gegensatz zum imperativen Programmcode. Regeln beschreiben, was wir erreichen möchten. Das zugehörige Wie erledigt die Regelmaschine für uns. Dieses Prinzip steckt übrigens auch hinter SQL und regulären Ausdrücken, zwei erfolgreichen Beispielen deklarativer Problemlösung.

7.2.2 Funktionsweise von Regelmaschinen

Regelbasierte Ansätze dieser Art reichen bis in die achtziger Jahre des letzten Jahrhunderts zurück – in Informatik-Zeitrechnung also fast bis in die Steinzeit. Damals setzten hauptsächlich Wissenschaftler solche Regelsysteme ein, weil sie sich nur schwer in normale Systeme integrieren ließen. Im Rahmen der künstlichen Intelligenz begannen sie seinerzeit mit der Entwicklung der sogenannten Expertensysteme, meist auf Basis von Regelsystemen.

Die Regelmaschine arbeitet auf einem sogenannten Working-Memory, das die sogenannten Fakten sowie den aktuellen Bearbeitungszustand der Regeln enthält. Bei der Auswertung von Regeln geht die Regelmaschine in der Abfolge „Match“, „Select“, „Act“ vor:

- In der **Match**-Phase versucht die Regelmaschine, die Bedingungssteile sämtlicher Regeln mit den Fakten aus dem Working-Memory zu erfüllen. Sie testet dabei sämtliche möglichen Kombinationen von Fakten. Eine erfüllbare Regel mitsamt den dafür notwendigen Fakten wird in den Conflict-Set aufgenommen.
- Während der **Select**-Phase ermittelt die Regelmaschine für sämtliche erfüllbaren Regeln (also den gesamten Conflict-Set) eine Ausführungsreihenfolge, den Schedule. Bei den meisten Regelmaschinen erhöht beispielsweise die Komplexität der LHS ihre Priorität, teilweise spielen sogar Zeitstempel der beteiligten Fakten eine Rolle.
- In der **Act**-Phase schließlich führt die Regelmaschine die rechten Regelseiten der Regeln aus dem Conflict-Set in der vorher bestimmten Reihenfolge aus. Nun kann's spannend werden: Die RHS einer Regel kann dazu führen, dass die Regelmaschine mit einer neuen Match-Phase wieder starten muss: RHS können nämlich die Fakten im Working-Memory ändern.

Diesen Zyklus implementieren moderne Regelmaschinen durch hochgradig optimierte Algorithmen (wie RETE²², LEAPS), deren Details ich Ihnen hier erspare.

Bild 7.15 zeigt die Grundkonstrukte eines regelbasierten Systems: Ihr eigenes System verwendet, wie bisher auch, Fach- oder Geschäftsobjekte. Als sogenannte Fakten (*Facts*) stellt es dem Working-Memory der Regelmaschine Geschäftsobjekte zur Verfügung. Die Regelmaschine wertet eine Menge von Regeln aus, die zur Laufzeit aus Dateien oder Datenbanken gelesen werden.

Diesen Ablauf finden Sie schematisch in Bild 7.16 als Sequenzdiagramm dargestellt.²³ Ihr System initialisiert die Regelmaschine mit der Regeldatei „rules.txt“, erzeugt Fachobjekte und übergibt diese an die Regelmaschine zur Verwendung im Working-Memory. Anschließend wird die Regelauswertung (select-match-act-Zyklus) gestartet, die möglicherweise zur Änderung der Fachobjekte (hier: Objekt f) führt.

²² Eine gute Erklärung finden Sie auf Wikipedia: http://en.wikipedia.org/wiki/Rete_algorithm

²³ In den Details unterscheiden sich die verfügbaren Regelmaschinen. Meine Darstellung orientiert sich an JBoss-Drools, einer in der Java-Welt häufig eingesetzten Open-Source-Regelmaschine. Siehe [Drools].

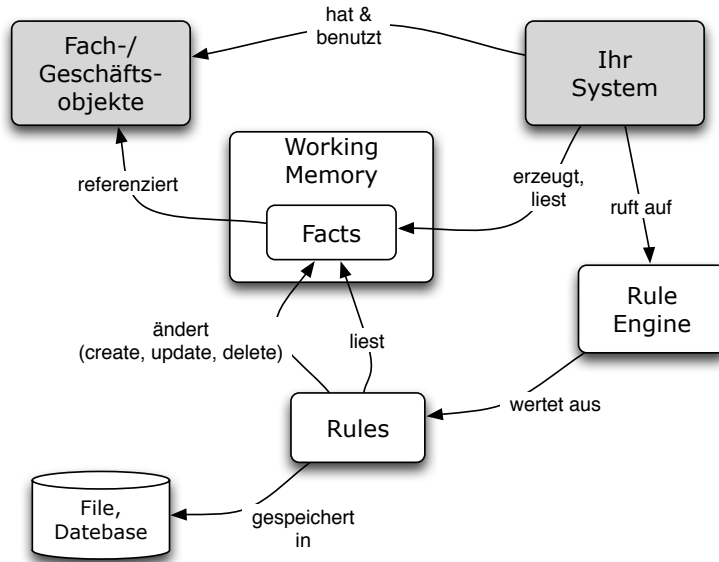


Bild 7.15 Grundkonstrukte eines regelbasierten Systems

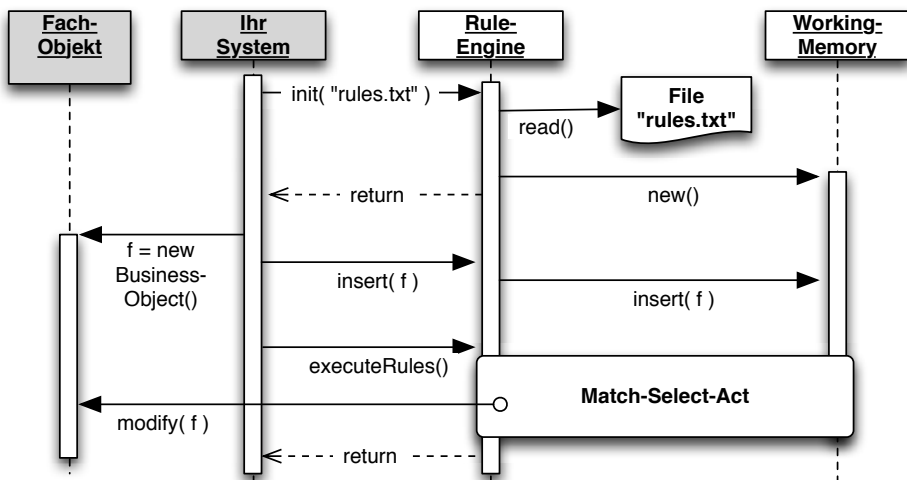


Bild 7.16 Schematischer Ablauf der Regelverarbeitung

Änderungen an fachlichen Regeln werden in diesem Szenario in der Regeldatei „rules.txt“ gepflegt.

Bei dieser Variante der Zusammenarbeit haben die Fachobjekte untereinander deutlich weniger Abhängigkeiten. Das kann zu besserer Änderbarkeit führen. Obendrein lassen sich Regeln häufig in (fast) natürlicher Sprache formulieren:

```

When (Person.Age < 18)
Then Person.setCreditAllowed( false );

```

Falls Sie sich jetzt fragen, ob so etwas in der Praxis wirklich funktioniert, habe ich einige Beispiele für (erfolgreiche) Anwendungen von Regelsystemen parat, was so viel bedeutet wie „Ja, es funktioniert an vielen Stellen“:

- Content-based Routing von Nachrichten in serviceorientierten Architekturen
- Tarif- oder Preisberechnung in Versicherungen
- Steuerung von Callcenter-Abläufen
- Steuerung der Datensatzverarbeitung von Batch-Prozessen

7.2.3 Kriterien pro & kontra Regelmaschinen

Ich möchte Ihnen einige Kriterien vorstellen, die für oder gegen den Einsatz von Regelsystemen sprechen:

- Müssen Sie innerhalb Ihrer Anwendungen Entscheidungen auf der Basis vieler bedingter Verzweigungen (z. B. verschachtelte if-then-else-Konstrukte) treffen? Dann passt eine Regelmaschine sicherlich gut zu Ihrem Problem. Können Sie Ihre Entscheidungen jedoch durch Datenbankabfragen oder rein algorithmische Berechnungen treffen, verwenden Sie besser keine.
- Wenn sich Ihre Geschäftsregeln nur selten ändern, können Sie statt Regeln ebenso konventionellen Programmcode schreiben und sich das Risiko und die Aufwände von Regelmaschinen ersparen.
- Versuchen Sie mal, einige Ihrer Geschäftsregeln in „Wenn-dann“-Form aufzuschreiben. Lassen Sie sich ruhig von einem Nicht-ITler dabei unterstützen. Wenn Ihnen nach einer halben Stunde immer noch keine fachlich sinnvollen Regeln eingefallen sind, spricht das gegen eine Regelmaschine. Lassen Sie sich aber vorher mal von Beispielregeln einer Regelmaschine inspirieren. Manchmal braucht das Umdenken von klassischen Programmiersprachen zu Regelsprachen einige Zeit.
- Wenn Ihre Entscheidungsprozesse einfach sind, d. h. Sie immer nur ein oder zwei if-Statements zur endgültigen Entscheidung benötigen, ist eine Regelmaschine wahrscheinlich nicht angemessen.
- Falls es bei Ihnen auf Millisekunden ankommt, weil Sie höchste Anforderungen an Performance stellen, vergessen Sie Regelmaschinen (obwohl die in den letzten Jahren wirklich flott geworden sind). Gleiches gilt für Echtzeitanforderungen (also hard-real-time constraints). Zumindest sind mir keine Regelmaschinen bekannt, die Laufzeiten im Echtzeitsinne garantieren können.
- Haben Sie Druck im Projekt? Stehen Sie kurz vor Fertigstellung? Neue Technologien stellen ein zusätzliches Risiko dar.
- Bedenken Sie, dass Einarbeitung in die Methode und Technik sowie Erstellung der Regeln hohen Aufwand benötigen. Falls Sie den Return-on-Investment für eine Regelmaschine in weniger als zwölf Monaten erreichen müssen, wird's riskant.
- Wenn Sie eine System- oder Softwarearchitektur entwerfen, bei der Fachlogik klar von der Technik getrennt sein soll, dann können Regelsysteme Sie dabei unterstützen.

- Regelmaschinen sind serverseitige Komponenten. Falls Sie Teile der Geschäftsregeln auf Clients (Browser, Smartphone, grafische Oberfläche) implementieren müssen: Erstens Vorsicht vor der entstehenden Redundanz, zweitens sind Regelmaschinen hierfür nicht gut geeignet.
- Wenn Sie architektonisch sauber arbeiten können, ihr System beispielsweise genau gemäß der Ports-und-Adapter-Architektur (= Clean-Architecture, siehe Abschnitt 6.2.3.3) implementieren – können Sie auf eine Regelmaschine verzichten.

7.2.4 Mögliche Probleme

Wechsel des Programmierparadigmas

Vielen Entwicklern imperativer Programmiersprachen (Java, C++, C#, VB und Co.) fällt es sehr schwer, auf das deklarative Paradigma regelbasierter Sprachen umzuschwenken. Sie sollten Regelsprachen niemals dazu missbrauchen, „normale“ imperative Programme zu schreiben.

Dieser Unterschied kann zu Widerständen seitens der Entwickler führen, zu Problemen bei der Fehlersuche oder zu schlechten Regelwerken.

Sie sollten ausreichend Zeit für die Einführung eines neuen Programmierparadigmas einplanen und möglichst mit kleineren Regelwerken beginnen. Hat Ihr Team erst mal „Lunte gerochen“, lösen sich diese Probleme leicht auf.

Hohe Flexibilität birgt Risiken

Regeln in regelbasierten Systemen besitzen den gleichen Stellenwert wie sonstiger Quellcode. Sie müssen getestet werden und beeinflussen den Ablauf der Programme. Aus rein fachlicher Sicht bringen Regeln jedoch eine Menge zusätzliche Flexibilität. Regeln können Sie ohne die Umstände starrer Releasezyklen ändern. Diese Flexibilität hat ihren Preis in Form von höherem Risiko: Die einfache Änderbarkeit birgt das Risiko, dass Änderungen *einfach so* geschehen: Mit dem Texteditor im Produktivsystem flugs einige Regeln geändert – und schon ist es um die Stabilität des Systems geschehen (weil nichts mehr läuft).

Regeln wollen verwaltet werden

Geschäftsregeln einer Regelmaschine sollten Sie genauso behandeln wie anderen Quellcode: Modularisiert beschreiben, in Versionsverwaltung halten und bei allen Änderungen automatisiert testen.

Falls Sie sehr viele Regeln beschreiben und pflegen müssen, benötigen Sie Unterstützung für Modularisierung und Refactoring der Regeln.

Falls Ihre Regeln zur Laufzeit (!) geändert werden können, wird Versionsmanagement schwierig bis unmöglich. Bedenken Sie: Wenn Anwender oder Administratoren die fachlichen Regeln ändern dürfen, können diese Stakeholder auf diese Weise gravierende Fehler ihres Systems auslösen.

7.2.5 Weiterführende Literatur

Das Open-Source-Regelsystem JBoss-Drools kann mit einer ausgezeichneten Dokumentation aufwarten (siehe <http://jboss.org/drools>) – auch zu grundlegenden Regel-Themen. Seine Regelsprache können Sie sogar anwendungsspezifisch anpassen. Für Java-Projekte empfehle ich dieses System als Einstieg.



Ansonsten tummeln sich auf dem Markt sehr viele kommerzielle Regelsysteme, teilweise mit umfangreicher Unterstützung für Entwicklung und Betrieb (sogenannte *business rules management systems*).

7.3 Integration

Integration: Einbindung bestehender Systeme (in einen neuen Kontext). Auch bekannt als (Legacy) Wrapper, Gateway, Enterprise Application Integration (EAI).

7.3.1 Motivation

Integration von Legacy-Systemen

Viele Softwaresysteme entstehen in einem Umfeld bestehender Anwendungen und Daten. Insbesondere kommerzielle Informationssysteme werden in der Regel in einem solchen Kontext von Legacy²⁴-Systemen entwickelt. Der Entwurf von Software in diesem Umfeld hat dann häufig einen der folgenden Schwerpunkte:²⁵

Legacy

- Erweiterung oder Ergänzung bestehender Software (zur Anpassung existierender Geschäftsprozesse).
- Ablösung von Altsystemen (weil diese nicht mehr wirtschaftlich wartbar oder anpassbar sind).
- Entwicklung neuer Software mit Schnittstellen zu Altsystemen (Eintritt in neue Geschäftsfelder mit Nutzung von Teilen der existierenden Software).
- Bestehende Daten müssen beim Entwurf neuer Systeme berücksichtigt werden.
- Notwendige Kooperation von Altsystemen durch Reorganisation oder Fusion von Unternehmen. In diesem Fall müssen unterschiedliche Systeme kooperieren, die ähnlichen Geschäftsprozessen dienen.

²⁴ Legacy (englisch): Altlast, Hinterlassenschaft. Hier in der Bedeutung von „Altsystem“ zu verstehen.

²⁵ Achten Sie auch auf Betrieb und Wartung der neuen Systeme. Sie können diese Aufgaben als Architekt zwar zunächst an das Projektmanagement delegieren, aber die technischen Aspekte werden garantiert wieder bei Ihnen landen!

Integration kommerzieller Komponenten

COTS

Das Problem der Integration tritt auch beim Entwurf von Systemen auf, die in hohem Maße auf kommerzielle Komponenten oder Frameworks zurückgreifen. Diese „Fertigprodukte“ (*Commercial-Off-The-Shelf*-Komponenten, COTS) können aus unterschiedlichen Bereichen stammen:

- Technische Komponenten, beispielsweise Bausteine grafischer Oberflächen, Persistenz-Frameworks oder Komponenten zur Datenverschlüsselung;
- Fachliche Komponenten, beispielsweise Rechnungswesen, Lager- oder Personalwirtschaft.

Frontend-Integration

Integration kann sich natürlich auch auf rein visuelle Darstellung innerhalb einer (grafischen) Oberfläche beziehen:

- Sie integrieren über Links/Hypermedia die Ausgabe anderer Programme. Beispiele hierfür sind Portalserver oder die sogenannten Mashups (= Kombination bestehender Inhalte). Typisches Beispiel: Integration von Werbung innerhalb von Webseiten.
- Sie integrieren Teile der Benutzerschnittstelle anderer Programme. Was dann für Endbenutzer wie ein einziges Programm aussieht, ist in Wirklichkeit eine Sammlung (Frontend-integrierter) Einzelteile. In Webtechnologien ist diese (grafische) Integration verbreitet, etwa durch Portalserver.
- Ein systematischer Ansatz sind die Micro-Frontends²⁶, etwa mit Hilfe von Web-Components.

Integration bedeutet Wiederverwendung

Die Einbindung bestehender Systeme oder Komponenten bedeutet Wiederverwendung. Wiederverwendung kann in folgenden Aspekten helfen:

- Kosten und Zeit sparen: Wiederverwendete (d. h. zu integrierende) Komponenten müssen nicht neu entworfen, entwickelt und getestet werden.
- Risiko mindern: Neuentwicklung von Komponenten ist grundsätzlich mit Risiken behaftet. Durch den Einsatz bereits bestehender Software können Sie das gesamte Projektrisiko reduzieren.

Schnittstellen und Verantwortlichkeiten sind wichtig

Für die Legacy-Integration, besteht eine wesentliche Herausforderung in der Zuordnung klarer Verantwortlichkeiten und dem Entwurf effektiver Schnittstellen. Bei der Integration von COTS-Komponenten hingegen muss man die darin vorhandenen Verantwortlichkeiten und Schnittstellen verstehen und angemessen einsetzen (weil es in COTS-Komponenten meist nichts mehr zu entwerfen gibt).

Minimal invasiv

Dabei ist es ein wichtiges Entwurfsziel, in bestehende Systeme möglichst *minimal invasiv*²⁷ einzugreifen. Die Integration sollte aus Gründen der Risiko- und Kostenminimierung möglichst ohne Veränderung oder Anpassung der bestehenden Systeme vorgenommen werden.

²⁶ Siehe Cam Jackson, Micro-Frontends: <https://martinfowler.com/articles/micro-frontends.html>

²⁷ Der Begriff stammt ursprünglich aus der Medizin.



Beachten Sie bei der Integration kommerzieller Frameworks, dass diese häufig signifikante Teile der Softwarearchitektur fest vorgeben. Das kann in den Fällen hinderlich sein, in denen die Kontrolle des Gesamtsystems nur über das Framework und nicht mehr unter der Verantwortung des „eigentlichen“ Systems erfolgen kann (*Don't call us, we call you!*).²⁸

7.3.2 Typische Probleme

Folgende Probleme könnten Ihnen beim Entwurf und bei der Implementierung integrierter Systeme das Leben erschweren:

- Monolithisch implementierte Legacy-Systeme: Es gibt keine (erkennbare) Trennung zwischen Applikationslogik, Datenhaltung und Benutzeroberfläche. Dies tritt oftmals bei sehr alten Mainframe-Batch-Programmen auf.
- Technisch optimierte Legacy-Systeme: In längst vergangenen Zeiten wurden Softwaresysteme oftmals unter Speicherplatz- oder Performance-Gesichtspunkten optimiert. Solche Optimierungen gingen (und gehen) meist zu Lasten der Wartbarkeit²⁹ oder Integrierbarkeit.
- Der Quellcode der Systeme liegt nicht mehr vor und ist nicht dokumentiert. Dieses Phänomen geht, streng nach Murphys Regel, häufig mit den beiden vorigen Fällen einher.
- Gerade auf Mainframe-Systemen enthalten die Datenzugriffsprogramme häufig „Plausibilitätsprüfungen“³⁰. Diese Prüfungen sind eine Form impliziter Fachlogik. Oft entstehen durch diese Plausibilitäten starke Abhängigkeiten von Komponenten untereinander, die eine Integration der betroffenen Legacy-Systeme erschweren.
- Die Fehler- und Ausnahmebehandlung der Legacy-Systeme basiert häufig auf numerischen Rückgabewerten. Teilweise werden technische und fachliche Fehlersituationen gemeinsam behandelt, was eine aussagekräftige Fehlerbehandlung erschwert.
- Technische Aspekte der Kommunikation zwischen dem neuen System und den Altsystemen, etwa unterschiedliche Protokolle, aufwendige Datenkonvertierung, fehlerträchtiger Datentransport, Fehlerbehandlung oder programmiersprachabhängige Aspekte. Siehe Abschnitt 7.5 (Kommunikation).
- Durch die Integration bestehender Systeme entstehen oftmals (und teilweise ungewollt!) verteilte Systeme. Wenn etwa das neue System als Client/Server-System auf neuen Hardware- und Betriebssystemen realisiert wird und auf bestehende Mainframe-Anwendungen zugreift, so ist das Gesamtsystem verteilt! Bitte beachten Sie daher den Abschnitt 7.4 (Verteilung).

²⁸ Dies ist in der Framework-Gemeinde als „Hollywood-Prinzip“ bekannt.

²⁹ Können Sie sich noch an die Jahr-2000-Probleme erinnern? Die wurden wesentlich durch technische Optimierungen (2-stellige Jahreszahl) ausgelöst.

³⁰ Im typischen Mainframe-Jargon oft „Plausis“ genannt.

- Performance: Wenn die Performance bereits existierender Systeme schlecht ist, stellt ihre Integration in ein neues System ein technisches Projektrisiko dar.³¹ Siehe dazu auch Abschnitt 7.4 (Verteilung).
- Sicherheit der integrierten Systeme. Insbesondere Mainframe-basierte Legacy-Anwendungen arbeiten mit völlig anderen Sicherheitsmechanismen als moderne Client/Server-Systeme. Hierdurch entstehen Risiken.
- Softwareentwickler haben manchmal die negative Eigenschaft, eigenen Code gegenüber wiederverwendetem zu bevorzugen (und damit auf mögliche Vorteile integrierter Systeme zu verzichten).
- Durch unterschiedlich geprägte Entwicklungsteams (beispielsweise Cobol-Mainframe- und Java-Client-Entwickler) kann es zu gravierenden Kommunikationsproblemen kommen, weil den Teams eine gemeinsame „Sprache“ fehlt.

7.3.3 Lösungskonzepte

Integration kann auf ganz verschiedenen Wegen erfolgen (nach [Hohpe+03]):

- Dateitransfer: Eine Applikation schreibt in eine Datei, die eine andere Applikation zu einem späteren Zeitpunkt lesen kann.
- Gemeinsame Datenbank (*Shared Database*): Mehrere Systeme verwenden eine gemeinsame Datenbank.
- Funktions- oder Methodenaufrufe (Synchrone Applikationsintegration, *Remote Procedure Call*, RPC): Ein System stellt nach außen aufrufbare Funktionen oder Methoden bereit, die über sogenannte Remote Procedure Calls synchron aufgerufen werden können.
- Asynchrone Kopplung (Messaging): Ein System schickt eine Nachricht an einen gemeinsamen oder öffentlichen Nachrichtenkanal. Andere Anwendungen können diese Nachricht von diesem Kanal zu einem späteren Zeitpunkt lesen.

Dateitransfer

Dateitransfer ist die wohl einfachste Art der Integration, unabhängig von Architekturen, Programmiersprachen oder Betriebssystemen der beteiligten Systeme. Beide Systeme müssen sich auf ein gemeinsames Dateiformat, Dateipfad und -namen sowie organisatorische Details (wann und wohin wird die Datei geschrieben, wann wird sie von wem wieder gelöscht?) einigen.

Dem Vorteil der Einfachheit und universellen Verfügbarkeit von Dateien stehen einige Nachteile gegenüber:

- Oftmals hoher Aufwand zur Konvertierung des gemeinsamen Dateiformats in das jeweils benötigte Format. XML-basierte Formate bieten sich heute als Austauschformat an, können allerdings von vielen Mainframe-Anwendungen nicht problemlos geschrieben oder gelesen werden.

³¹ Oftmals können bestehende Systeme nicht beeinflusst oder optimiert werden, weil etwa der Quellcode nicht vorliegt, die Änderungen mit hohem Risiko verbunden wären, die eingesetzten Rechner voll ausgelastet sind oder Ähnliches.

- Geringe Performance von Dateioperationen.
- Sicherheitsmechanismen gelten lediglich für die gesamte Datei, es gibt keine feingranulare Zugriffskontrolle auf einzelne Teile. Das Zielsystem kann die Authentizität der Daten nicht ohne Weiteres sicherstellen.
- Das Quellsystem hat keine Information darüber, ob, wann und mit welchem Ergebnis das Zielsystem die Datei verarbeitet hat.

Gemeinsame Datenbank

Aufgrund der engen Kopplung (siehe unten) eher ein Anti-Muster:

Vorsicht: Eher ein Anti-Muster

Alle beteiligten Systeme müssen sich auf eine gemeinsame Datenbank mit einem einheitlichen Schema einigen. Hinsichtlich Performance, Zeitnähe von Updates, Konsistenz und Sicherheit ist diese Lösung deutlich leistungsfähiger als der Dateitransfer. Durch leistungsfähige Datenbanksysteme ist die Nutzung dieses Konzeptes aus unterschiedlichen Programmiersprachen und Betriebssystemen (fast) problemlos möglich. Doch auch dieser Ansatz kann einige Nachteile besitzen:

- In der Praxis ist es manchmal (überraschend) schwierig, ein für mehrere beteiligte Systeme geeignetes Datenbankschema zu entwerfen.
- Martin Fowler gibt in [Hohpe+03] zu bedenken, dass insbesondere die Integration von Standardprodukten in dieses Lösungskonzept schwierig ist, weil solche Produkte in der Regel mit unveränderlichen Datenbankschemata geliefert werden.
- Falls viele beteiligte Systeme parallel die gemeinsame Datenbank lesen und schreiben, kann diese zum Engpass hinsichtlich Performance werden. Die Ansätze der verteilten-und-replizierten Datenbanken birgt organisatorische und technische Risiken.
- Enge Kopplung zwischen den beteiligten Systemen: Änderungen an den grundlegenden Datenstrukturen bedingen Änderungen an allen beteiligten Anwendungen. Falls Flexibilität und continuous-delivery für Sie bedeutsam sind – dann stellt diese enge Kopplung ein K.o.-Kriterium dar.

Remote Procedure Call

Falls Sie Systeme über Funktionen anstatt über deren Daten integrieren möchten, stehen Ihnen die *Remote Procedure Calls* als Lösungsansatz zur Verfügung. Konzeptionell bedeutet RPC den Aufruf einer Funktion auf einem (möglicherweise entfernt, das heißt in einem anderen Prozessraum) ablaufenden Programm. Bild 7.17 zeigt das Vorgehen schematisch. Dort finden Sie zwei zentrale Begriffe, Stubs und Skeletons: Diese Komponenten wickeln für die beteiligten Systeme die Kommunikation mit dem entfernten Partnersystem ab.

Jedes beteiligte System verantwortet seine Daten selbst, was eine gute Kapselung bedeutet.

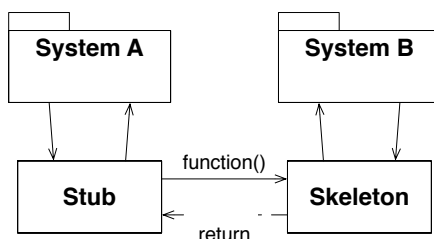


Bild 7.17
Schematische Darstellung
von Remote Procedure Calls

Dieses Konzept wird durch Technologien wie CORBA (siehe unten), COM, .NET-Remoting, Java-RMI und SOAP umgesetzt. Es besticht durch hohe Flexibilität und direkte Wiederverwendbarkeit bestehender Systeme. RPCs sind die Grundlage von Client/Server-Systemen. In den folgenden Abschnitten finden Sie weitere Muster und Hinweise zur Integration über RPC.

Zu den möglichen Nachteilen von *Remote Procedure Calls* gehören folgende:

- Entfernte Funktionsaufrufe sind in Bezug auf Zeit- und Ressourcenbedarf um mehrere Größenordnungen teurer als lokale Aufrufe. In der Praxis führt das teilweise zu gravierenden Performance-Problemen.
- Es bedarf einer komplexen technischen Infrastruktur (Middleware), um entfernte Funktionsaufrufe zu ermöglichen. Technologien wie CORBA und andere sind zwar stabil, performant und in der Praxis bewährt, erhöhen jedoch die Komplexität von Systemen (und Projekten) ganz erheblich.
- Durch die komplexe Hardware- und Softwareinfrastruktur von gekoppelten Systemen gefährden viele potenzielle Fehlerquellen den ordnungsgemäßen Gesamtablauf.
- Enge Kopplung zwischen den beteiligten Systemen:
 - Mögliche Blockierung: Wenn das aufgerufene System nicht oder nur sehr langsam antwortet, kann es zu Blockierung im Aufrufer kommen, weil der im Regelfall wartet, bis die Antwort vorliegt.
 - Eine Schnittstellenänderung im aufgerufenen System hat möglicherweise Konsequenzen im Aufrufer. Dadurch besteht das Risiko, dass Änderungen nichtlokale Auswirkungen nach sich ziehen. In der schematischen Abbildung müssen Sie bei Änderungen am System B stets auch System A testen!

Messaging

Bei der nachrichtenbasierten Integration, kurz Messaging, senden sich die beteiligten Systeme Datenpakete, die in sogenannte Nachrichten verpackt werden. Eine von den Applikationen getrennte Kommunikationssoftware (*Message Oriented Middleware*, MOM) übermittelt die Nachrichten über Kanäle (Channels oder Queues). Kanäle speichern Nachrichten so lange, bis sie von den Empfängern verarbeitet wurden.

Nachrichten im Messaging-Konzept bestehen meist aus zwei Teilen:

- Im *Message-Header* befinden sich Meta-Informationen über die Nachricht, etwa der Name von Sender und Empfänger. Message-Header werden meist von der Messaging-Middleware genutzt.
- Im *Message-Body* befinden sich die eigentlichen Nutzdaten. Diese können vom Empfänger beliebig interpretiert werden, als Daten oder auch als Aufforderung, einen bestimmten Funktionsaufruf auszuführen.

Bild 7.18 zeigt den schematischen Verlauf dieser Kommunikation (Erläuterung in Anlehnung an [Hohpe+03]):

1. *create*: Der Sender erzeugt die Nachricht und füllt sie mit Daten. Wenn nötig, wird der Nachricht adressiert, d. h. mit einer Empfängeradresse versehen.
2. *send*: Der Sender fügt die Nachricht einem Kanal hinzu und übergibt sie damit der Kontrolle des Messaging-Systems.

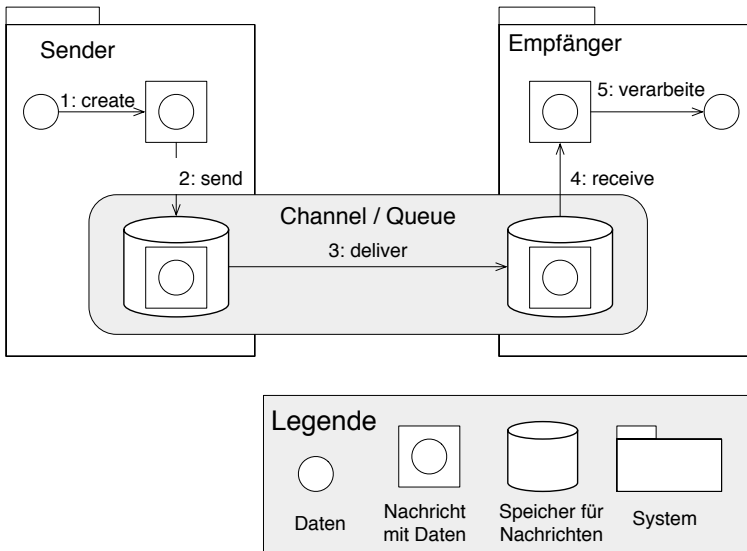


Bild 7.18 Schematische Darstellung von Messaging (nach [Hohpe+03])

3. *deliver*: Das Messaging-System leitet die Nachricht vom Computer des Senders an den des Empfängers weiter und stellt sie dort dem Empfänger zur Verfügung.
4. *receive*: Der Empfänger liest die Nachricht vom Kanal und übernimmt sie damit vom Messaging-System.
5. *verarbeite*: Der Empfänger extrahiert die Daten aus der Nachricht und verarbeitet sie.

In diesem Ablauf erkennen Sie zwei wichtige Konzepte:

- **Senden-und-vergessen** (*send-and-forget, fire-and-forget*): Dem Sender wird die Zustellung der Nachricht vom Messaging-System garantiert. Er kann die Nachricht nach dem Senden getrost vergessen. Dies ist der wesentliche Unterschied zu synchroner Kommunikation: Der Sender muss nicht auf die Verarbeitung der Nachricht warten!
- **Speichern-und-weiterleiten** (*store-and-forward*): Wenn der Sender die Nachricht dem Kanal hinzufügt, speichert das Messaging-System diese Nachricht auf dem Computer des Senders. Zu einem geeigneten Zeitpunkt wird sie dann auf den Computer des Empfängers weitergeleitet und dort ebenfalls gespeichert.

Bei Integration durch Messaging müssen sich die Anwendungen auf den Kanal sowie auf das Format der Nachricht einigen. Das Messaging-System übernimmt sämtliche technischen Details der Kommunikation. Es garantiert unverfälschte Übertragung sowie Zustellung der Nachricht. Kommerzielle MOM-Produkte bieten zusätzliche Funktionen, wie etwa Rückruf und Neuordnen von Nachrichten, Vergabe von Prioritäten, Versenden an mehrere Empfänger und weitere.

Die asynchrone Integration durch Messaging besitzt unter anderem folgende Vorteile:

- Kommunikation über Systemgrenzen hinweg, unabhängig von Programmiersprachen und Betriebssystemen (solange auf den beteiligten Systemen das Messaging-System verfügbar ist!).

- Zuverlässigkeit (*guaranteed delivery*) durch das zwischenzeitliche Speichern von Nachrichten.
- Vermeidet Blockierung beim Sender und Überlastung der Empfänger.
- Auch Anwendungen ohne permanente Netzverbindung können durch Messaging integriert werden: Sie übermitteln oder empfangen ihre Nachrichten, wenn sie gerade online sind – in der übrigen Zeit können sie unabhängig von Netzverbindungen arbeiten.

Nun, nach so vielen Vorteilen dürfen Sie mit Recht auch einige Nachteile erwarten:

- Komplexes Programmiermodell. Das konventionelle Modell von Funktions- oder Methodenaufrufen funktioniert für Messaging nicht mehr.
- Das Konzept garantiert keine spezifische Reihenfolge von Nachrichten.
- Falls geschäftliche Anforderungen synchrone oder zumindest kurzfristige Antworten erfordern, können Sie das Konzept nicht ohne Weiteres anwenden. Ihre Kunden wollen beispielsweise eine Preis Auskunft sofort haben, und nicht nach einer beliebigen Zeitspanne.
- Abhängigkeit von Messaging-Produkten, neudeutsch *vendor lock-in*.

Viele große Informationssysteme, bei denen Zuverlässigkeit und Entkopplung wichtige Architekturziele sind, basieren auf Messaging. Hierzu gehören beispielsweise die zentralen Buchungssysteme von Banken, Abrechnungssysteme der Telekommunikation sowie große Buchungs- und E-Commerce-Systeme.

Ich werde hier nicht weiter auf die Aspekte asynchroner Integrationslösungen eingehen. Eine umfassende und praxisnahe Abhandlung des Themas finden Sie in [Hohpe+03].

7.3.4 Entwurfsmuster zur Integration

Wenn Sie als Architekt häufig mit Integration zu tun haben, sollten Sie den Abschnitt über Entwurfsmuster in Abschnitt 4.6 gründlich lesen. Folgende Entwurfsmuster gehören bei der Integration zum wichtigen Entwurfswerkzeug:

- *Adapter*: Passt die Schnittstelle einer Komponente an eine von ihren Klienten erwartete Schnittstelle an. Lässt ansonsten inkompatible Komponenten zusammenarbeiten.
- *Brücke*: Entkoppelt eine Abstraktion von ihrer Implementierung.
- *Fassade*: Eine einheitliche Schnittstelle zu einer Menge von Schnittstellen eines Subsystems.
- *Proxy*: Ein vorgelagertes Stellvertreterobjekt.
- *Vermittler (Mediator)*: Definiert ein Objekt, welches das Zusammenspiel mehrerer anderer Objekte in sich kapselt.

Manchmal finden Sie auch die Begriffe *Wrapper* oder *Gateway* für Integrationsbausteine.

7.3.5 Zusammenhang mit anderen Aspekten

Kommunikation und Verteilung

Kommunikation und Verteilung sind mit der Integration eng verwandt. Kommunikation stellt die technischen Grundlagen zur Integration bereit. Verteilung ist das Gegenstück zur Integration. Gerade die (möglichen) Probleme bei der Verteilung von Komponenten gelten auch für die Integration. Siehe Abschnitt 7.4 (Verteilung) und Abschnitt 7.5 (Kommunikation).

Sicherheit

Bei Legacy-Systemen auf Basis von Mainframes kommen häufig Sicherheitsmechanismen und entsprechende Produkte zum Einsatz, die auf anderen Plattformen nicht verfügbar sind. Beachten Sie beispielsweise folgenden Aspekt:

Die Benutzerverwaltung auf Mainframe-Systemen unterscheidet sich manchmal von der im Client/Server-Bereich. Wenn Sie unterschiedliche Mainframe-Systeme integrieren müssen, kann es an dieser Stelle zu Problemen kommen. Nehmen Sie diesen Aspekt als Einfluss- oder Risikofaktor in Ihre Planung auf!

Es kann hilfreich sein, sogenannte „technische Benutzer“ einzuführen. Das sind Benutzerkennungen, denen keine reale Person entspricht und deren Kennwörter nicht ablaufen. Diese technischen Benutzer können nützlich sein, wenn Teile Ihrer Anwendungen auch ohne angemeldete Benutzer aktiv sein müssen.

Technische Benutzer



Beispiel: Für das Callcenter einer Versicherung wurde eine dezentrale Anwendung entworfen, die eine Mainframe-basierte Adressverwaltung integrierte. Einige Basisdienste, beispielsweise die Prüfung von Postleitzahlen, sollten als Systemdienste unabhängig von einzelnen Benutzern betrieben werden. Der Mainframe beharrte jedoch darauf, die Liste der gültigen Postleitzahlen nur nach erfolgreicher Benutzerautorisierung „herauszugeben“.

Zur Abhilfe wurde ein technischer Benutzer eingeführt, der den Start sämtlicher Basisdienste ermöglichte.

Administration

Die Administration integrierter Systeme wirft eine Reihe von Problemen auf:

- **Überwachung:** Die Überwachung hinsichtlich Betriebsbereitschaft kann auf verteilten Systemen komplex sein. Es gibt entsprechende Werkzeuge zur Systemverwaltung, die hier unterstützen.
- **Systemstart:** Der Start („das Hochfahren“) eines verteilten Systems kann bestimmte Konstellationen der beteiligten Komponenten erfordern. Klären (und dokumentieren!) Sie exakt, welche Schritte zum Start des Gesamtsystems erforderlich sind (und in welchen Zuständen sich alle beteiligten Subsysteme befinden müssen).
- **Einhaltung von Service Level Agreements:** Wie bereits oben erwähnt, stellen „garantierte“ Eigenschaften des Gesamtsystems (etwa: Verfügbarkeit und Datenschutz) Probleme dar.

Für die Administration integrierter Systeme ergibt sich die zusätzliche Schwierigkeit, die Einhaltung allgemeiner Service Level Agreements *nachzuweisen*. Stellen Sie sich beispielsweise vor, mehrere beteiligte Subsysteme schreiben jeweils ihre eigenen (und proprietären) Protokolldateien (Logfiles).

7.3.6 Weiterführende Literatur



[Hohpe+03] enthält über 60 verschiedene Muster zu Entwurf und Implementierung (asynchroner) Integrationslösungen, sogenannter Messaging Solutions. Die Autoren führen sehr verständlich in die vielfältigen Aspekte von Integrationslösungen ein und behandeln sowohl Grundlagen als auch (sehr) fortgeschrittene Themen.

[Keller02] diskutiert Enterprise Application Integration (EAI) von einem sehr praktischen Standpunkt aus. Falls EAI in Ihrer Organisation ein Thema ist, sollten Sie dieses Buch einerseits selbst lesen und andererseits den verantwortlichen Managern schenken.

■ 7.4 Verteilung

Verteilung: Entwurf von Softwaresystemen, deren Bestandteile auf unterschiedlichen und eventuell physikalisch getrennten Rechnersystemen ablaufen.

7.4.1 Motivation

- Bei Client/Server-Systemen arbeiten Clients und Server in Form eines verteilten Systems zusammen.
- Verteilung von Komponenten kann nützlich sein, wenn diese Komponenten spezielle Eigenschaften der zugrunde liegenden entfernten Hard- oder Softwareplattform ausnutzen.
- Verteilte Systeme können durch Integration bestehender (Legacy-)Systeme in neue Software entstehen. Befinden sich das neue System und das bestehende System auf unterschiedlichen Hardwareplattformen, so liegt eine Form der Verteilung vor.
- Der Wunsch nach hoher Ausfallsicherheit oder dynamische Lastverteilung auf mehrere getrennte Systeme kann die Verteilung von Softwarekomponenten bedeuten.
- Die Verbreitung von Smartphones und Tablet-Computern als intelligente Clients erlaubt, verteilte Systeme auch in Massenmärkten zu nutzen.

7.4.2 Typische Probleme

Folgende typische Probleme treten bei verteilten Systemen auf:

- Entfernte Methoden- oder Funktionsaufrufe erfordern einen erheblich höheren Overhead als lokale Aufrufe.
- Datenkonvertierung und -transport: Ein verteiltes System muss die Integrität von Daten über Komponentengrenzen hinweg sicherstellen. Das Marshalling verpackt Daten zum Transport über „den Draht“, ein korrespondierendes Unmarshalling entpackt die übertragenen Daten wieder. Hierzu gehören Konvertierungen von Zeichensätzen (EBCDIC, ASCII, Unicode), Datentypen und Protokollen.
- Fehlerbehandlung über Systemgrenzen hinweg sowie die Differenzierung zwischen fachlichen Fehlern und (technischen) Kommunikationsfehlern.

7.4.3 Lösungskonzept

Techniken für Verteilung

Bekannte technische Ansätze zur Implementierung verteilter Systeme sind folgende:

- Kommunikation über http(s) als Applikationsprotokoll mit REST (siehe Kapitel 6); heute (Stand 2020) gerade in Web-basierten Systemen das Mittel der Wahl.
- Web-Services auf Basis des „*Simple Object Access Protocol*“ (SOAP)³².
- CICS und MQ-Series,³³ beide im Mainframe-Umfeld.
- CORBA, entwickelt und standardisiert durch die Object Management Group³⁴, sowohl kommerzielle als auch freie Implementierungen verfügbar. In der Praxis kaum noch von Bedeutung.
- Message-Broker (wie etwa Kafka) – als Integrations- und Verteilungsplattform (für eine Übersicht siehe <https://medium.com/@dasun.15/distributed-systems-messaging-eee8e70c4a77>)

Ich rechne auch andere kommerzielle Middleware-Produkte zu den Techniken, die Verteilung ermöglichen. Hierzu gehören beispielsweise Transaktionsmonitore, die verteilte Verarbeitung ermöglichen sowie das weite Feld der EAI-Werkzeuge (*Enterprise Application Integration*).

³² Details zu SOAP und Web-Services finden Sie unter www.w3.org/TR/SOAP.

³³ CICS und MQ-Series sind Produkte (und Warenzeichen) der Firma IBM.

³⁴ www.omg.org

Beschreibung von Schnittstellen

Ein wichtiges Hilfsmittel für alle Aspekte der Verteilung ist die exakte Beschreibung der Schnittstellen zwischen den Komponenten.



Tipps zur Verteilung (Verteilen – aber richtig!)

- Können Sie Verteilung vermeiden, indem Sie sich auf eine einzelne Plattform beschränken? Das senkt das Entwicklungsrisiko und vereinfacht den Betrieb des Systems. Verteilung heißt immer höhere Komplexität und mehr Fehlerquellen.
- Verteilen Sie so, dass Verarbeitungsprozesse „nahe“ bei den betroffenen Daten ablaufen. Das kann Netzverkehr reduzieren und die Performance verbessern.
- Entwerfen Sie möglichst stabile Schnittstellen. Aufgrund der Abhängigkeiten zwischen Client und Server ist dieser Aspekt wichtiger als bei nichtverteilten Systemen.
- Reduzieren Sie die Menge der zu übertragenden Daten. Versuchen Sie auf die Übertragung komplexer Objektstrukturen zu verzichten. Führen Sie stattdessen möglichst schlanke „*transfer objects*“ ein. Solche Transfer-Objects repräsentieren einen Extrakt aus einem Objekt (oder einer Objektstruktur). Mit ihrer Hilfe reduzieren Sie die Anzahl entfernter Funktions- oder Methodenaufrufe.
- Fassen Sie zusammengehörige Funktions- oder Methodenaufrufe in Prozessobjekten zusammen. Das ist eine Ausprägung des Fassade-Entwurfsmusters (siehe Abschnitt 4.6.6).
- Erzeugen Sie Objekte so spät wie möglich, nämlich erst dann, wenn sie wirklich benötigt werden. Lassen Sie verteilte Komponenten nicht „auf Vorrat“ arbeiten. Weichen Sie von dieser Regel nur bei Objekten ab, die das System auf jeden Fall benötigt (etwa: DB-Verbindungen oder ähnliche Hilfsobjekte; Objekte, deren Erzeugung viel Zeit in Anspruch nimmt).

7.4.4 Konsequenzen und Risiken

Die Verteilung von Komponenten besitzt viele Vorteile hinsichtlich Flexibilität, Skalierbarkeit und Lastbalancierung. Jedoch stehen diesen Vorteilen auch eine Reihe potenzieller Risiken gegenüber:

- Der Betrieb verteilter Systeme umfasst mehrere potenzielle Fehlerquellen. Sämtliche Kommunikationskanäle (Netzwerke) müssen für den fehlerfreien Betrieb eines verteilten Systems zur Verfügung stehen. Auch kleine Probleme mit Netzwerken (Aus- oder Überlastung, kurze Aussetzer) können eine Fehlfunktion des Gesamtsystems bewirken.
- Entwicklung und Test verteilter Systeme sind schwieriger als bei zentralen Systemen. Stellen Sie sicher, dass Ihre Entwicklungsumgebung beispielsweise über Mechanismen zum Remote-Debugging verfügen. Fügen Sie Komponenten zur Fehler- und Ausnahmebehandlung ein, die auch Kommunikations- oder Verfügbarkeitsprobleme behandeln.
- Ungünstige Verteilung von Komponenten kann aufgrund extensiver Kommunikation der Komponenten untereinander zu Performance-Problemen führen.
- Verteilte Client/Server-Systeme benötigen manchmal einen komplexen Prozess der Softwareverteilung, -installation und Versionsverwaltung. Hier hilft der Architekturansatz „Microservices“ mit dem getrennten und unabhängigen Deployment einzelner Services.

7.4.5 Zusammenhang mit anderen Aspekten

Verteilung, Integration und Kommunikation hängen eng miteinander zusammen. Siehe daher auch Abschnitt 7.3 und Abschnitt 7.5.

7.4.6 Weiterführende Literatur

[Tilkov-15] ist ein guter Wegweiser durch Konzepte, Herausforderungen und Lösungen Web-basierter Architekturen mit REST und HTTP. Leicht lesbar und praxisorientiert.

[vanSteen+17] führt in (fast) alle Aspekte verteilter Systeme ein – und ist obendrein noch kostenfrei als eBook verfügbar. Mitautor ist der Netzwerk- und Betriebssystem-Guru Andrew Tanenbaum.



■ 7.5 Kommunikation

Kommunikation: Übertragung von Daten zwischen Systemkomponenten. Bezieht sich auf Kommunikation innerhalb eines Prozesses oder Adressraumes, zwischen unterschiedlichen Prozessen oder auch zwischen unterschiedlichen Rechnersystemen.

Ziel dieses Abschnitts ist es, einige für den Entwurf von Systemen wichtige Grundbegriffe der Kommunikation vorzustellen. Ich habe mögliche Lösungskonzepte in die Vorstellung der Begriffe und Konzepte eingearbeitet.

Die Aspekte der physikalischen Übertragung kommen hier nicht zur Sprache. Sie finden viele Grundlagen der Datenkommunikation beispielsweise in [Tanenbaum+12].

7.5.1 Motivation

Beim Entwurf verteilter oder integrierter Systeme (siehe Abschnitte 7.3 Integration oder 7.4 Verteilung) müssen Komponenten miteinander kommunizieren. Eine solche Kommunikation kann im einfachen Fall aus dem Austausch von Daten oder Dateien bestehen (*file transfer*), es kann sich aber auch um den Aufruf von entfernten Funktionen, Methoden oder Programmen handeln (*remote procedure call*).

Eine physikalische Verbindung zweier Rechnersysteme allein genügt noch nicht, um diese Aufgaben zu erfüllen.

7.5.2 Entscheidungsalternativen

Es gibt eine ganze Reihe von Entscheidungen, die Sie hinsichtlich der Kommunikation zwischen Komponenten treffen können (und müssen):

- Findet Kommunikation synchron oder asynchron statt?
- In welchem Stil kommunizieren Komponenten miteinander? Mögliche Varianten sind: Remote Procedure Call, Publish/Subscribe oder Broadcast.
- Kommunizieren die beteiligten Komponenten direkt oder indirekt (über Gateways, Broker oder andere Middleware) miteinander?
- Welche Protokolle (etwa: TCP/IP, HTTP, IIOP, SOAP) kommen zum Einsatz?
- Wie werden unterschiedliche technische Gegebenheiten der beteiligten Kommunikationspartner (etwa: Zeichensatz, Zahlendarstellung, Wortgröße) behandelt?

7.5.3 Grundbegriffe der Kommunikation

Ich beschreibe Ihnen in diesem Abschnitt die architekturelevanten Fragen und Aspekte der Kommunikation. Der Fokus liegt dabei auf den Anforderungen kommerzieller Systeme.

Synchron und asynchron

Bei der synchronen Kommunikation wartet (blockiert) der Aufrufer, bis die aufgerufene Komponente ein Ergebnis (oder eine Statusinformation) zurückliefert. Synchrone Kommunikation ist der Normalfall bei der Softwareentwicklung. Alle gängigen Programmiersprachen nutzen fast ausschließlich synchrone Mechanismen zur Kommunikation von Programmkonstrukten untereinander.

Synchron

Asynchrone Kommunikation hingegen basiert auf dem Prinzip „versenden und vergessen“. Eine Nachricht wird verschickt, aber der Aufrufer arbeitet sofort weiter, ohne auf ein Ergebnis oder eine Zustandsinformation zu warten (Einweg-Kommunikation³⁵). Ein bekanntes Beispiel asynchroner Kommunikation ist die elektronische Post.

Asynchron

Asynchrone Kommunikation besitzt für Softwarearchitekturen hohe Bedeutung, weil über Schnittstellen zu externen Systemen häufig asynchron kommuniziert wird.

Erwägen Sie in folgenden Fällen den Einsatz asynchroner Kommunikation:



- Die Verbindung zwischen Client und Server ist nicht permanent. Falls Clients „offline“ sind, bleibt nur die asynchrone Kommunikation.
- Die beteiligten Systeme sind nur teilweise oder zu unterschiedlichen Zeiten verfügbar.
- Sie erwarten spezielle Anfragen oder eine hohe Transaktionslast, die Ihre Server aus Performance- oder Verfügbarkeitsgründen nicht synchron verarbeiten können. Beispiel: Datenbankoperationen, deren Laufzeit für Benutzer nicht akzeptabel ist (und deren Resultate sie nicht unbedingt sofort benötigen, etwa Änderungen einer großen Zahl von Datensätzen).
- Das System löst Aktionen aus, die manueller Nacharbeit bedürfen (etwa: Freigaben, Bestätigungen, Korrekturen oder Klassifikationen). Dieser Fall tritt häufig bei den Workflow-Systemen auf, siehe Abschnitt 7.9 (Workflow). Konkrete Beispiele aus unseren Projekten sind: Bestätigung von Bestellungen, Kreditfreigaben, Rechnungsprüfung oder Freigaben von Lieferungen.
- Sie wollen Nachrichten über einen Zeitraum sammeln und erst später verarbeiten. Dieses Vorgehen ist nützlich, wenn das System beispielsweise die Reihenfolge oder Priorität von Nachrichten erst nach dem eigentlichen Erzeugen der Nachricht festlegen kann.

Möglichkeiten asynchroner Kommunikation

Die Realisierung asynchroner Kommunikation erfordert in jedem Fall einen Speicher für die zu verarbeitenden Nachrichten. Dieser kann elektronisch sein (etwa Message Queues, Dateien oder E-Mail), aber auch aus Papier bestehen.

Asynchrone Kommunikation kann über Nachrichtenpuffer (*Message Queues, MQ*) ablaufen. Diese Variante ist die Domäne der sogenannten „Messaging-Systeme“, die als flexible Verwalter solcher Nachrichtenpuffer auftreten.

Message Queues

³⁵ Das zugehörige Konstrukt der CORBA Interface Definition Language heißt treffend „oneway“.

Sie können etwa die Übermittlung der Nachricht garantieren (*guaranteed delivery*) und Nachrichten persistent zwischenspeichern, um Systemausfälle zu überbrücken.



Beispiel: Großunternehmen benutzen für ihre Transaktionen (Buchungen, Bestellungen oder Ähnliches) oft Messaging-Systeme und asynchrone Kommunikation. Transaktionen werden über eine bestimmte Periode (etwa: einen Werktag) hinweg gesammelt und zu bestimmten Zeiten von der Warteschlange *en bloc* verarbeitet.³⁶

„Richtige“ Messaging-Systeme (kommerziell wie Open Source) haben noch andere Vorteile. Sie können Nachrichten beispielsweise (im Nachhinein) umsortieren, neu priorisieren, gruppieren oder auf unterschiedliche Zielsysteme verteilen (etwa zur Lastbalancierung). Hilfreich ist auch die Möglichkeit der „garantierten“ Zustellung (= *robust* oder *guaranteed delivery*).

Andere Varianten asynchroner Kommunikation, etwa Dateien oder E-Mails, sind technisch einfach und weniger aufwendig, hinsichtlich Flexibilität allerdings den Messaging-Systemen unterlegen.

Beachten Sie, dass Kommunikation mit Medienbrüchen ebenfalls asynchron ist.

Stile der Kommunikation

Remote Procedure Call (RPC)

Remote Procedure Call (RPC) arbeitet genau wie ein lokaler Prozedur- oder Methodenaufruf, die aufgerufene Komponente kann sich jedoch auf einem entfernten Rechnersystem befinden. Client und Server sind in RPC-Szenarien häufig (über Festlegungen im Quellcode) eng gekoppelt. Grundsätzlich muss der RPC einige Schwierigkeiten bewältigen:

- Wie werden entfernte Prozeduren lokalisiert und gestartet? Was geschieht, wenn mehrere Clients gleichzeitig dieselbe Prozedur benötigen?
- Wie werden Parameter zwischen Client und Server ausgetauscht?
- Wie werden Fehlersituationen behandelt? Da Client und Server unabhängig voneinander fehlschlagen können, muss ein RPC-basiertes System alle möglichen Kombinationen von Fehlersituationen behandeln.

Publish-Subscribe

Publish - Subscribe: In diesem Szenario meldet ein Client durch eine Subscription an einer bestimmten Art von Nachrichten sein Interesse an. Ausführlich erläutert, finden Sie das Publish-Subscribe-Muster in [Buschmann96]. Client und Server sind in diesem Fall nur lose gekoppelt. Clients müssen von Servern nur die Schnittstelle für Subscribe-Operationen kennen. Der Mechanismus wird im Observer-Entwurfsmuster sowie im Model-View-Controller-Paradigma ausgenutzt.

Broadcast

Bei *Broadcast*-Kommunikation ist die Kopplung zwischen Client und Server noch lockerer. Server übermitteln Nachrichten ohne Angabe eines konkreten Empfängers oder „an alle“. Verwenden Sie Broadcast in folgenden Fällen:

³⁶ Das kann auch in Batch-Systemen geschehen.



- Wenn das System prüfen soll, welche Services oder Clients gerade für Anfragen zur Verfügung stehen („Hey, wer kann mir gerade mal helfen?“).
- Wenn das System von verschiedenen Services „Angebote“ einholen soll.
- Wenn Sie dynamische und spontane Interaktion zwischen Komponenten benötigen.
- Wenn das System den Namen des Empfängers nicht kennt.

Direkte und indirekte Kommunikation

Bei der direkten Kommunikation kennt der Client seinen Server. Das bedeutet für den Softwareentwurf eine Abhängigkeit dieser beiden Komponenten voneinander. Schon Änderungen der Server-Adresse ziehen Änderungen an allen beteiligten Clients nach sich.

Bei der indirekten Kommunikation sind Clients von ihren Servern entkoppelt. Services werden über Vermittler³⁷ aufgerufen. Dadurch wird es möglich, Services flexibler zu betreiben. Eine Änderung am Service bedeutet höchstens eine Änderung beim Vermittler oder Gateway, die beteiligten Clients bleiben unverändert. Wichtige Elemente der indirekten Kommunikation sind Namensdienst (*naming*) sowie Broker, Message-Systeme oder Gateways.



Sie sollten indirekte Kommunikation einsetzen, wenn:

- Kommunikation über System- oder Rechnergrenzen notwendig wird;
- technische Rahmenbedingungen von Clients oder Servern sich häufig ändern (etwa: Standorte, Netzwerkparameter oder Service-Bezeichner);
- an das Gesamtsystem hohe Anforderungen hinsichtlich Flexibilität gestellt werden;
- Abhängigkeiten zwischen Client und Server nicht erwünscht sind.

In Kapitel 6 (Strukturentwurf) finden Sie weitere Ratschläge, Abhängigkeiten innerhalb Ihrer Entwürfe zu verringern oder zu vermeiden.

Protokolle

- TCP/IP (*Transmission Control Protocol/Internet Protocol*): Das Basisprotokoll im Internet. Es ermöglicht die Kommunikation zwischen jeweils zwei Teilnehmern (*peers*) über die sogenannten Sockets. Diese bestehen aus IP-Adresse und einer Portnummer.
- HTTP (*Hyper-Text Transmission Protocol*), basiert auf TCP/IP. Es ist das Standardprotokoll bei der Kommunikation im Internet (häufig: zwischen Webserver und Browser). Es ist zustandslos, was bei Internet-basierten Client/Server-Systemen das Problem der Verwaltung des Anwendungszustands aufwirft. Siehe [Tilkov-15].
- SOAP: Ein Protokoll, das *Remote Procedure Calls* in XML beschreibt und (meistens via HTTP) überträgt. Heute eher selten in Gebrauch.
- Apache Thrift: Ein Framework für die Entwicklung sprachübergreifender Dienste, nutzt Codegenerierung um die Kommunikation zwischen vielerlei Sprachen zu ermöglichen (u. a. C++, Java, Python, PHP, Ruby, Erlang, Perl, Haskell, C#, JavaScript, Node.js und weitere). Leistet das, was CORBA immer leisten wollte, aber nie so ganz richtig geschafft hat ☺

³⁷ Hierzu gehört jegliche Art von Middleware oder Broker.

- IIOP (*Internet Inter-ORB Protocol*): Protokoll für die Kommunikation in CORBA-Systemen. Heute kaum noch in Gebrauch.

Das OSI-7-Schichten-Modell

Das Referenzmodell für Datenkommunikation ist das OSI-7-Schichten-Modell, dargestellt in Bild 7.19.

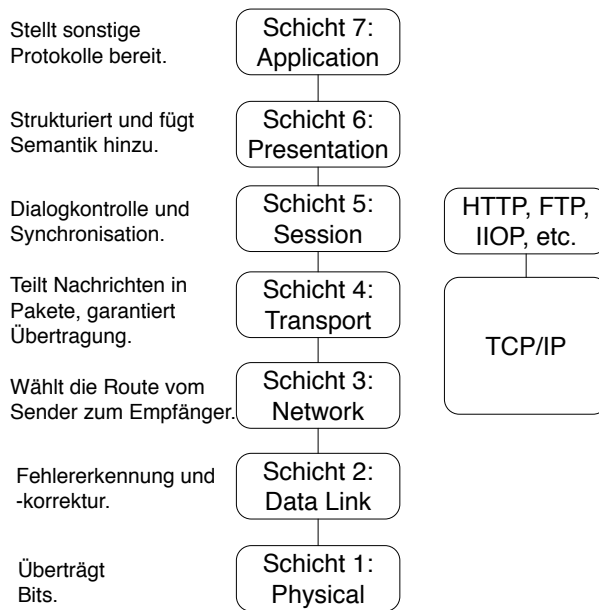


Bild 7.19

OSI-Referenzmodell und TCP/IP

Parallel dazu zeigt diese Abbildung das TCP/IP-Protokoll in Relation zum OSI-Modell. Es umfasst nahezu die Leistungen der Netzwerk- und Transportschichten des OSI-Modells. Oberhalb von TCP/IP befinden sich die bekannten Protokolle HTTP, FTP und IIOP.

Weitere Informationen zum OSI-Modell, zu Protokollstacks und anderen Feinheiten der Kommunikation finden Sie in [Tanenbaum+12].

7.5.4 Weiterführende Literatur

Die Grundlagen sowie ausführliche Details zum Thema Integration und Kommunikation finden Sie im Standardwerk [Hohpe+03] – besonders empfehlenswert!

[Tanenbaum+12] ist (immer noch) ein Standardwerk für Details zur Datenkommunikation.

[Tilkov-15] erklärt Ihnen auf angenehme Art und Weise alles, was Sie über REST wissen sollten.

■ 7.6 Grafische Oberflächen (GUI)

7.6.1 Motivation

Benutzer sind anspruchsvoll, benötigen ergonomische, reaktive und angemessene Bedienoberflächen (je nach Fachlichkeit, Art und Vorkenntnissen der Benutzer, technischen Gegebenheiten)

7.6.2 Einflussfaktoren und Entscheidungskriterien

Aus meiner Sicht prägen im Wesentlichen drei Aspekte die Umsetzung einer Benutzeroberfläche:

- Welche Arten von Benutzern verwenden das System, über welche Kenntnisse und Fähigkeiten verfügen sie?
- Welche Arten von Interaktionen soll das System ermöglichen oder unterstützen?
- Welche Plattformen und Technologien soll das System unterstützen?

Art der Benutzer

Welche Personen sollen das System benutzen: Können Sie technische und/oder fachliche Kenntnisse voraussetzen? Sollen neue und erfahrene Benutzer identisch behandelt werden oder unterschiedliche Benutzerschnittstellen bekommen?

Handelt es sich um eine geschlossene oder offene Benutzergruppe? Geschlossene Gruppen finden sich primär bei Inhouse-Anwendungen, offene Benutzergruppen beispielsweise bei Web-Anwendungen oder COTS³⁸-Systemen.

Soll das System verschiedene Rollen von Benutzern unterstützen, mit jeweils unterschiedlichen Rechten und Interaktionsmöglichkeiten?

Gemeinsam ist menschlichen Benutzern eine Menge typischer Verhaltensmuster ([Tidwell-2011] nennt sie *behavioral patterns*). Ich möchte sie in Form einer Tabelle kurz vorstellen – sie stellen meiner Meinung nach eine gute Grundlage für Anforderungen an Benutzeroberflächen dar.

³⁸ Commercial Off-The-Shelf: Software, die als Standardprodukt ohne Individualisierung verkauft wird. Beispiel: Office-Produkte oder Computerspiele.

Tabelle 7.3 Was sich viele Benutzer von Benutzeroberflächen wünschen (behavioral patterns, nach [Tidwell-2011])

Verhalten	Erklärung
Sicheres Erforschen	„Lass mich das System erkunden, ohne dass ich in Schwierigkeiten gerate.“
Unmittelbare Belohnung	„Ich möchte jetzt sofort etwas schaffen, nicht irgendwann später.“
Gerade gut genug	„Das ist gut genug. Ich möchte nicht mehr Zeit investieren, um zu lernen, wie es noch besser klappen könnte.“
Mittendrin ändern	„Ich habe mir mittendrin etwas anderes überlegt.“
Aufgeschobene Entscheidung	„Ich möchte das jetzt nicht beantworten – lass mich einfach fertig werden.“
Schrittweise Verbesserung	„Lass mich das noch mal ändern, das sieht nicht gut aus. Und noch mal. Jetzt ist es besser.“
Gewöhnung	„Dieser Befehl funktioniert überall – warum nicht hier?“ ³⁹
Räumliches Gedächtnis	„Gerade war der Button noch an dieser Stelle – wo ist er jetzt hin?“
Vorausblickendes Gedächtnis	„Ich lege das hier hin, um mich daran zu erinnern, es später fertig zu machen.“

Art der Interaktionen und GUI-Idiome

Welche Aktionen oder Interaktionen soll die Benutzerschnittstelle unterstützen:

- Reine Darstellung von Informationen, beispielsweise Status- oder Ergebnisanzeige – ohne direkte Interaktion mit Benutzern.
- Darstellung und Benutzung von Datenobjekten, ohne oder stark eingeschränkte Möglichkeiten zu deren Manipulation. Beispielsweise Media-Player, Auswahl und Anzeige von Aktienkursen, Produktauswahl in Web-Shops.
- Erzeugung und Manipulation von Datenobjekten, beispielsweise masken- oder formularbasierte Anwendungen, E-Mail- oder Kalenderprogramme, Texteditoren, Zeichenprogramme.
- Immersive Systeme, die ihren Benutzern durch optische, akustische (und teilweise auch taktile) Effekte „andere Welten“ suggerieren. Beispiele hierfür sind primär Spiele, etwa Autorennen, Flugsimulatoren und Actionspiele.⁴⁰

Jenifer Tidwell stellt in ihrem großartigen Buch [Tidwell-2011] eine Vielzahl solcher Idiome vor – für alle nützlich, die sich mit GUI-Design beschäftigen.

³⁹ Ein prima Beispiel: In Emacs bewirkt die Tastenfolge „Ctrl-A → Ctrl-X → Ctrl-S“ eine Cursorbewegung und danach das Speichern der Datei. In Microsoft-Word bedeutet die gleiche Tastenfolge „gesamten Text markieren, gesamten Text ausschneiden (= löschen), die (jetzt leere) Datei speichern. Oops ☹“

⁴⁰ Wie etwa die berühmten Ego-Shooter und Kampfspiele (z. B. World of Warcraft), aber auch realistische Flugsimulation wie <https://x-plane.com>.

Plattformen und verfügbare Technologien

Benutzeroberflächen müssen sich oftmals an den vorhandenen technischen Möglichkeiten der vorhandenen Zielpattform (Hardware und Software) orientieren. Sie würden persönlich eine ausgefeilte, grafisch anspruchsvolle Oberfläche implementieren, sind aber durch das Laufzeitsystem der Zielpattform an sehr einfache grafische Elemente gebunden ...

Tabelle 7.4 Kurzübersicht einiger Plattformen und UI-Technologien

Web-Clients (primär Browser)	▪ HTML, insbesondere HTML-5 mit CSS und JavaScript (bzw. die vielen Frameworks⁴¹, die HTML als Ausgabeformat generieren) ▪ Proprietäre Formate, etwa Adobe-Flash
Rich-Clients (Arbeitsplatzrechner mit grafischen Displays, Tastatur und Maus/Trackpad o. Ä.)	▪ Microsoft Windows (WPF, WinForms) ▪ Java (JavaFX, Eclipse-RCP) ▪ Plattformneutral: QT⁴², GTK+ (Gimp-Toolkit⁴³)
Mobile Clients (Smartphones, Tablets o. Ä.)	▪ Android ▪ iOS

Im Bereich der Frontend/GUI-Entwicklung haben insbesondere Webentwickler die Qual der Wahl: Eine Vielzahl von Frameworks bringen auch entscheidungsfreudige Zeitgenossen an die Grenzen ihrer Belastbarkeit.

7.6.3 GUI-relevante Architekturmuster

Sie sollten beim Entwurf von interaktiven Systemen sicherlich die klassischen Architekturmuster aus Abschnitt 4.5.6 ff. kennen – allen voran Model-View-Controller oder Presentation-Model. Nach meiner praktischen Erfahrung geben in der Realität allerdings die konkreten technischen (UI-)Frameworks große Teile der diesbezüglichen Architektur- und Implementierungsentscheidungen bereits vor. Viele dieser UI-Frameworks (beispielsweise Windows-WPF oder WinForms, Java-Swing, JavaFX, Java-ServerPages, Ruby-on-Rails, Grails, iOS, Zend, Django oder Griffon) implementieren ihre eigene Interpretation von MVC oder einem ähnlichen Muster. Die Frameworks geben Ihnen vor, wie sie Ihren UI-Code zu strukturieren haben, wie UI-relevante Dinge im Quellcode heißen und wie diese sich verhalten.

Falls Sie also UI-intensive Anwendungen konstruieren, entwickeln oder begleiten müssen, sollten Sie die eingesetzten UI-Frameworks verstehen – das ist wahrscheinlich wichtiger als die konzeptionellen Grundlagen von MVC und Co ...

Im Zweifel implementieren Sie zwei oder drei kleine Prototypen auf Basis unterschiedlicher UI-Frameworks und entscheiden erst danach, welches Framework Sie auf welche Weise in Ihr System aufnehmen.

⁴¹ Rails (Ruby), Grails/Griffon (Groovy), Zend (Php), Django (Python), Ring (Clojure) u. v. a. m.

⁴² Ein Klassiker der UI-Entwicklung: <http://qt-project.org/>

⁴³ <http://www.gtk.org/>

7.6.4 Struktur und Ergonomie von Benutzeroberflächen

Ich möchte erst gar nicht versuchen, Sie in der Kürze dieses Buches zu einem auch nur halbwegs ordentlichen UI-Designer zu machen – dafür ist das Thema zu umfangreich und zu komplex. Ich möchte Sie aber für einige Themen sensibilisieren, die Sie bei der ganzheitlichen Gestaltung von (grafischen) Oberflächen berücksichtigen sollten.

Struktur im Großen (UI-Metapher)

Von einer sehr hohen Warte aus betrachtet, hat fast jedes System eine Vielzahl unterschiedlicher Aufgaben zu erledigen. Auf einer so hohen Ebene entscheiden Sie als Architekt (oder UI-Designer) die UI-Metapher Ihres Systems. Dazu einige Vorschläge:

- Schreibtisch-Metapher: Benutzer bestimmen selbst, welche Objekte in ihrer unmittelbaren Umgebung (Schreibtisch) sichtbar bzw. zu bearbeiten sind.
- Assistenten-Metapher (= Fabrik- oder Fließband-Metapher): Das System leitet Benutzer durch eine feste Folge von Arbeitsschritten. Beispiel: Installations- oder Konfigurationsassistenten.
- Werkbank-Metapher (= Werkzeug-Material-Metapher): Fachliche Aufgabenstellungen und entsprechende Lösungsansätze zur Bearbeitung einzelner Informationsobjekte stehen an der Oberfläche bereit. Beispiel hierfür: Grafische Editoren oder Bildbearbeitungsprogramme.
- Formular-Metapher: Ein Formular (= Maske) enthält meist textuell dargestellte Daten, die teilweise auch erzeugt oder manipuliert werden können.

Struktur im Kleinen

Innerhalb der gesamten Benutzeroberfläche werden Sie meist kleinere Teile bereitstellen, die ihrerseits wieder spezialisierte Aufgaben lösen. Schon wieder möchte ich [Tidwell-2011] zitieren – die vier mögliche Ausprägungen dieser kleineren Teile (= Seiten) vorstellt:

1. Zeige eine einzelne Information oder ein Ding an.
2. Zeige eine Liste von Informationen oder Dingen an.
3. Stelle Werkzeuge zum Erzeugen einer Information oder eines Dings an.
4. Stelle Mittel zur Bearbeitung einer Aufgabe bereit (= Manipulation von Daten).

Navigation innerhalb von Benutzeroberflächen

Haben Sie die Benutzeroberfläche gemäß ihrer Teilaufgaben modular gestaltet, so müssen die Benutzer durch diese einzelnen Teile navigieren. Dafür haben sich unterschiedliche Möglichkeiten etabliert:

- Menüs und Menübäume: Über Menü (meist am oberen Rand des Bildschirms) können Benutzer Aufgaben oder Daten auswählen und werden durch das System dann an die entsprechende Stelle im UI geleitet.
- Navigationsleisten oder -elemente: Hervorgehobene Elemente der Benutzeroberfläche erlauben die Navigation an bestimmte Stellen der Oberfläche.
- Tastaturkürzel (= Hot-Keys), die sofort die gewünschten Dinge darstellen bzw. Aktionen ausführen.

Benutzer sollten grundsätzlich sehen können, an welcher Stelle des Systems sie sich gerade befinden („Schritt 3 von 6“) oder in welchem Kontext sie arbeiten. Hierzu bieten sich etwa benannte Fenstertitel oder die sogenannten breadcrumbs (= Brotkrümelnavigation) an.

Arten von Benutzereingaben

Neben der bekannten Tastatur- und Mausbedienung grafischer Oberflächen sind in den letzten Jahren weitere Technologien zur Entgegennahme von Benutzereingaben entwickelt worden:

- Kombinierte und spezialisierte Eingabegeräte, beispielsweise Game-Controller, Joysticks, Grafiktablets.
- Berührungsempfindliche Displays ermöglichen Interaktionen durch Berührung und Gesten direkt auf dem Bildschirm (= touch-Display). Weit verbreitet haben sich diese Systeme primär über Smartphones und Tablet-Computer.
- Dreidimensionale Sensoren, die Bewegungen im Raum erkennen können. Zuerst haben die Hersteller von Spielekonsolen erste Versionen dieser Sensoren praxisreif gemacht, die Arm- und Handbewegungen im Bereich einiger Zentimeter differenzieren können. Mittlerweile differenzieren optische Sensoren in einem Erkennungsbereich von ca. 50 cm Radius im dreidimensionalen Raum im Bereich von Millimetern. Damit werden feinmotorische Interaktionen möglich.

Natürlich sind analoge Bedienkonzepte (Dreh- und Schieberegler, Taster, Schalter) heute auch durch Softwaresysteme nutzbar. In hohem Maße werden beispielsweise komplexe Maschinen heute durch Kombinationen verschiedener Eingabekanäle bedient.

7.6.5 Bekannte Risiken und Probleme

Verbindung von UI und Daten (Data Binding)

Nehmen wir das Beispiel eines einfachen Informationssystems, das Daten aus einer Datenbank verarbeitet und an einer Benutzeroberfläche anzeigt. Betrachten Sie dazu Bild 7.20. Die Datenbank enthält ein Datum in Zustand (1), das von einem Server gelesen wird – Zustand (2). Zwei Clients stellen dieses Datum nun unabhängig voneinander dar (3) und (4).

Jeder dieser Teile (1) bis (4) könnte dieses Datum modifizieren – durch DB-Trigger, externe Zugriffe auf die Datenbank, fachliche Ereignisse auf Serverseite oder auch durch die verschiedenen Benutzer.

Die Darstellung zeigt einen klassischen Fall verschiedener Repräsentationen desselben Datums. In der Softwarearchitektur, insbesondere bei der Konstruktion grafischer Oberflächen, benötigen wir Mechanismen, um diese unterschiedlichen Repräsentationen miteinander in Beziehung setzen zu können. Die Änderung einer der Repräsentationen sollte von einem Laufzeitsystem, etwa durch Events oder Notifikationen, den übrigen Beteiligten mitgeteilt werden. Diese können dann selbstständig entscheiden, welche Aktionen sie ergreifen (etwa: Daten neu lesen, Daten verwerfen, Daten als ungültig markieren o. Ä.).

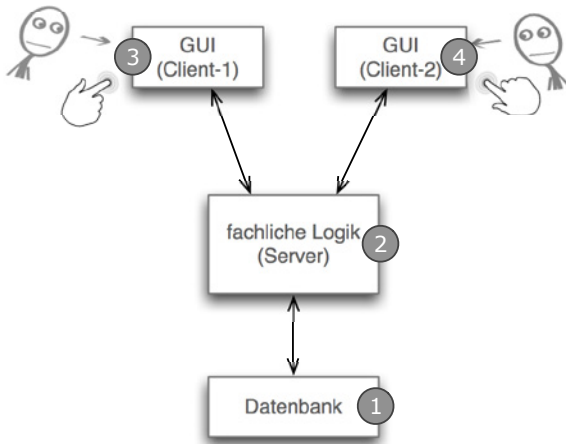


Bild 7.20
Notwendigkeit von Data-Binding

Insbesondere sollte ein Data-Binding-Mechanismus folgende Aufgaben für Sie lösen:

- Zirkuläre Abhängigkeiten von Repräsentationen erkennen und die damit möglichen unendlichen Event-Schleifen auflösen können.
- Transitive Abhängigkeiten von Repräsentationen erkennen und entsprechende Events auslösen – etwa aus dem Ursprungsdatum berechnete Informationen dann ebenfalls als geändert markieren.
- Senden und Verarbeiten von Events an Bedingungen knüpfen oder nur selektiv bestimmte Events „abonnieren“ können.

Client/Server-Verteilung

Oftmals läuft die Benutzeroberfläche eines Systems (= Client) getrennt von der Backend-Verarbeitung (= Server). Mit dieser Trennung gehen eine Reihe möglicher Schwierigkeiten einher:

- Remote-Kommunikation zwischen Client und Server, bedeutet fast immer einen höheren Implementierungsaufwand, beispielsweise durch Serialisierung/Deserialisierung, Remote-Call-Infrastruktur oder die Behandlung möglicher netzwerkbedingter Fehler.
- Netzwerke zwischen Clients und Servern können, wie generell in verteilten Systemen, mit zeitverzögerter Übertragung (Latenz) zu kämpfen haben. Für zeitkritische Aufgaben an Benutzeroberflächen können solche Verzögerungen hinderlich wirken.
- Auf Client und Server können unterschiedliche Betriebssysteme, Laufzeitumgebungen oder Implementierungstechnologien eingesetzt werden. Dies ist etwa bei Web-Anwendungen der Normalfall – wo auf (modernen) Clients oft Browser & JavaScript-Runtimes verwendet werden, auf der Serverseite aber ganz andere Technologien vorherrschen.
- Fachliche Logik auf Client oder Server implementieren: Implementierung auf dem Client macht diese Prüfungen für Benutzer gefühlt performanter, möglicherweise muss diese Logik dann redundant für verschiedene Client-Typen implementiert werden und wäre schwerer wartbar. Die Implementierung im Server wäre universell verwendbar und zentralisiert.
- Synchronisation von Aktionen mehrerer paralleler Benutzer: Falls mehrere Benutzer parallel dieselben Objekte oder Daten bearbeiten, müssen Sie Aspekte wie Konsistenz, Transaktionalität, Rollback oder Kompensation beachten.

Internationalisierung

International eingesetzte Systeme müssen neben verschiedenen Sprachen (= Locales) und Zeichensätzen auch mögliche weitere Aspekte berücksichtigen:

- Formatierung von Daten, etwa: Uhrzeit, Datum, Telefonnummern
- Syntaktische Unterschiede in ähnlichen Daten, etwa: Postleitzahl
- Schreibrichtungen: im westlichen Sprachraum links-nach-rechts, in arabischen Ländern rechts-nach-links, in manchen asiatischen Ländern oben-nach-unten.
- Länderübergreifende Uhrzeitregelungen:⁴⁴ Welche Zeitzone soll eine internationale Anwendung als Basis verwenden? Falls mehrere Zeitzonen möglich sind, gibt es im System keine „Normalzeit“, sondern nur benutzerrelative Zeiten. Ach ja – da waren noch Sommer- und Winterzeit.
- Kulturelle Unterschiede, insbesondere in Bild- und Zeichensprache, sowie Farbwahl. Schwarz gilt in vielen westlichen Kulturen als Farbe der Trauer, andere Länder verwenden dafür Weiß.
- Bezeichnung von Personen: Ein- oder zwei Vornamen, mit oder ohne Suffix („Jr.“), die Vornamen vor den Nachnamen oder dahinter, ist ein Vorname zwingend (in vielen afrikanischen Kulturen gibt es beispielsweise keine Vornamen!), bleibt die Anrede nach einer Hochzeit identisch oder ändert sie sich ... Hier drohen Ihnen eine Menge möglicher „Fettnäpfchen“.

Die Optik: Farben, Fonts, Fläche, Anordnung

Die Festlegung optischer und gestalterischer Details erscheint uns ITlern manchmal als simple Kleinigkeit – in der Realität verbirgt sich dahinter die hohe Kunst des Designs.

Im Interesse Ihrer zukünftigen Benutzer – lassen Sie einen erfahrenen, ausgebildeten (Interaktions-)Designer die optischen Details Ihrer Benutzeroberfläche entscheiden. Ich habe oft und intensiv unter Oberflächen gelitten, die von mir und anderen „IT-Geeks“ entworfen wurden – fast immer mit dem Ergebnis mangelhafter Akzeptanz und entsetzter Benutzer.

Ich empfehle Ihnen einen Blick in das „Non-Designers Design-Book“ [Williams-2008], um die Vielfalt rund um dieses Thema zu erleben und verständliche und hilfreiche Ratschläge aufzugreifen.

7.6.6 Zusammenhang zu anderen Aspekten

In den Architekturmustern und -stilen aus Kapitel 4 finden Sie eine Reihe von Lösungsvorschlägen zur Konstruktion von Systemen mit Benutzeroberflächen – siehe insbesondere Abschnitt 4.5.6 über die interaktionsorientierten Systeme.

⁴⁴ Wer mal versucht hat, in einem der verbreiteten Kalender-Programme (ziemlich egal, in welchem) einen internationalen Termin über mehrere Zeitzonen zu organisieren, weiß, wovon ich hier spreche.

■ 7.7 Geschäftsprozess-Management: Ablaufsteuerung im Großen

von Martin Bartonitz und Tammo van Lessen

Ablaufsteuerung im Sinne von Geschäftsprozess-Management (engl. Business Process Management, BPM) bezieht sich auf systemübergreifende Aspekte. Hier geht es um die Steuerung und Koordination zwischen verschiedenen Softwaresystemen, aber auch um die Integration von menschlichen Tätigkeiten in die Steuerungsketten.

Geschäftsprozess-Management ist für Softwarearchitektur aus verschiedenen Gründen von Bedeutung:

- Es ist beeinflusst die Erfassung der funktionalen Anforderungen durch prozessorientiertes Denken.
- Es betrifft die Schnittstellen zwischen verschiedenen Systemen.
- Es betrifft die Schnittstellen zwischen Systemen und Menschen.
- Es beeinflusst möglicherweise die Dienste oder Schnittstellen, die ein System „nach außen“ bereitstellt.
- Zur Abwicklung komplexer Geschäftsprozesse sind oftmals mehrere Softwaresysteme notwendig.

In feiner Abgrenzung zum *Geschäftsprozess-Management* verwenden wir den Begriff *Workflow-Management* – abgekürzt WfM. Während Workflow-Management sich vordringlich auf die technische Abwicklung von Aktivitätsketten fokussiert, hat BPM den Anspruch einer ganzheitlichen Sicht auf die Geschäftsprozesse. Neben der verwendeten Technik werden also auch Methoden zur kontinuierlichen Prozessoptimierung betrachtet.

Die kontinuierliche Prozessverbesserung (KVP) lässt sich am besten anhand des BPM-Lifecycle erklären. Es gibt in der Literatur verschiedene Ausprägungen dieses Lebenszyklus, die Variante in Bild 7.21 stellt die aus unserer Sicht wichtigsten Merkmale zusammen:

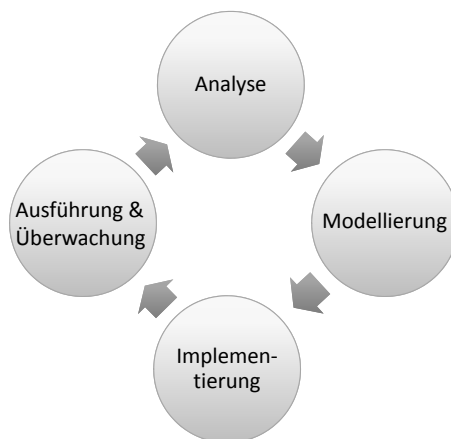


Bild 7.21
BPM-Lifecycle

Als Einstieg in den Kreislauf beginnen wir mit der Analyse. In dieser Phase werden zunächst die Ist-Prozesse erhoben. Dazu werden häufig klassische Requirements-Engineering-Methoden (Workshops, Interviews, Planspiele) angewendet. Hinterlassen die gelebten Prozesse bereits elektronische Spuren, z. B. in Logdateien, kann auch „Process Mining“ angewendet werden, um aus den historischen Daten Prozessmodelle abzuleiten. Im nächsten Schritt, der Modellierungsphase, wird das durch die Analyse gewonnene Wissen dokumentiert. Da sich Freitexte als ungeeignete Dokumentationsmittel herausgestellt haben (insbesondere bei gleichzeitig ablaufenden Teilprozessen), sollte hierfür eine grafische Notation wie die BPMN verwendet werden. Das Ziel dieser Phase ist es, ein Verständnis darüber zu erlangen, welche Prozesse unter Beteiligung welcher Ressourcen und Verantwortlichkeiten gelebt werden, und dieses zu dokumentieren. Es werden also drei Dimensionen berücksichtigt. Die „Was“-Dimension beschreibt, welche Aktivitäten in welcher Reihenfolge ausgeführt werden sollen, damit ein bestimmtes Geschäftsziel erreicht wird. Die „Wer“-Dimension berücksichtigt, welche Personen oder Organisationseinheiten an dem Prozess beteiligt sind. Schließlich dokumentiert die „Womit“-Dimension, welche Hilfsmittel dafür benötigt werden. Oft werden bei der Modellierung schon erste Abweichungen der Ist-Prozesse von der Soll-Vorstellung identifiziert, sodass erste Optimierungen diskutiert und implementiert werden können. Manche BPM-Projekte enden nun hier, z. B. wenn es hauptsächlich um die Dokumentation und Sicherung der Prozesskonformität geht, oder setzen Simulationstechniken ein, um „Nadelöhre“ in der Ressourcenplanung oder den Prozessmodellen zu finden.

Für Softwarearchitekten beginnt aber nun erst der spannende Teil. Denn basierend auf der Analyse und der daraus resultierenden fachlichen Modelle können (Teil-)Prozesse für eine Prozessautomatisierung identifiziert werden. An dieser Stelle lassen sich die ersten Architekturweichen stellen. Die Prozesse können beispielsweise hart auskodiert werden oder aber an eine Process Engine zur Ausführung übergeben werden. Solch eine Engine kann direkt in die Anwendung eingebettet oder aber als zentrale Infrastruktur in einem verteilten System implementiert werden.

Sind (Teil-)Prozesse auf solche Weise automatisiert, ist es, insbesondere unter der Verwendung von Process Engines, leichter die Ausführung der Prozesse zu überwachen und zu analysieren. Dadurch, dass die Prozesse idealerweise Abläufe mit direktem Geschäftswert beschreiben, beschreibt jede Prozessinstanz eine konkrete Wertschöpfung. Mithilfe von Monitoringwerkzeugen kann man in Prozessinstanzen hinschauen und korrigierend eingreifen oder mittels Berichtswerkzeugen konkrete fachliche Fragen beantworten. Zusätzlich schreiben Process Engines typischerweise einen sogenannten Audit Trail, halten also die Historie fest, wann wer welche Aktivität ausgeführt hat. Man kann diese Daten in der nächsten Analysephase gezielt auswerten und die Prozessmodelle so weiter optimieren oder an Marktveränderungen anpassen.

Der Einsatz einer Process Engine hat einen signifikanten Einfluss auf die Architektur, denn sie erzwingt eine Unterscheidung zwischen „Programming in the Large“ und „Programming in the Small“. Ersteres wird durch die technischen Prozessmodelle abgebildet. Sie sind in der Regel durch einen Prozessexperten von den fachlichen Prozessmodellen abgeleitet und um technische Details angereichert. Diese Prozessmodelle sind auch grafisch modelliert und werden von der Engine ausgeführt. Hinter den von der Engine ausgeführten Aktivitäten verbergen sich entweder Aufgaben für Menschen oder sogenannte Geschäftsfunktionen. Letztere sind typischerweise in traditionellen Programmiersprachen implementiert (Programming in the

Small) und können von dem Modellierer als Funktionsbibliothek betrachtet werden, die er im Prozessmodell in die gewünschte Reihenfolge bringt. Durch die geschickte Trennung lässt sich ein Prozessmodell an neue Marktanforderungen anpassen, ohne die Geschäftsfunktionen zu verändern. Dadurch verspricht man sich eine schnelle Reaktionszeit und einen höheren Return on Investment. Eine serviceorientierte Architektur (siehe Kapitel 10) bietet sich dafür an.

7.7.1 Workflow-Sprachen

Einen großen Einfluss auf eine Reihe ergänzender Aspekte des BPM hatten mehrere neue Standards. Anfang des Jahrtausends erblickten die Web-Services das Licht der großen Welt. In großen Prozessumgebungen müssen diverse Web-Services „orchestriert“ werden. Dazu wurden zunächst von IBM und Microsoft zwei konkurrierende Spezifikationen definiert. XLang von Microsoft beschrieb den Kontrollfluss durch die Aktivitäten mithilfe von strukturierenden Blöcken, wie wir sie aus konventionellen Programmiersprachen kennen. WSFL von IBM nutzte stattdessen einen graph-basierten Ansatz, der den fachlichen Modellierern mehr entgegenkommt. Um einen signifikanten Marktanteil zu erlangen, einigten sich die Hersteller auf eine Zusammenführung, aus der der Standard WS-BPEL (= Business Process Execution Language) unter dem Dach der OASIS durch die marktbestimmenden Unternehmen entstand und in deren BPM Suiten umgesetzt wurde.

Aus heutiger Perspektive wurden bei der Entwicklung von WS-BPEL jedoch zwei Dinge außer Acht gelassen: Zum einen hatte man es versäumt, eine grafische Notation zu definieren. Das führte dazu, dass jeder Hersteller eine eigene Repräsentation der Sprache entwarf und sich keine als Quasistandard etablieren konnte. Zum anderen fehlten WS-BPEL die Mittel, um menschliche Aufgaben in die Prozessketten integrieren zu können. Die Sprache war also auf die Orchestrierung von Web-Services beschränkt. Dies versuchte man durch die Standardisierung von BPEL4People und WS-HumanTask zu korrigieren. Während die Konzepte Einfluss auf zahlreiche Produkte hatten, konnten sich die beiden Spezifikationen nie wirklich durchsetzen.

Da die parallel entwickelte BPMN (damals noch Business Process Modelling Notation, seit Version 2.0 Business Process Model and Notation) zunehmend an Bedeutung gewann und sowohl für die Fachseite wie die technischen Modellierer verständlich und attraktiv wurde, galt lange die Devise: Fachliche Modelle werden in BPMN erfasst und dann von einem Prozessexperten, der sich sowohl in der fachlichen als auch der technischen Domäne zuhause fühlt, in BPEL überführt. Die Transformation von BPMN zu BPEL ist leider nicht immer eins zu eins möglich, denn die BPMN ermöglicht eine freiere Modellierung, die auch exotische Workflow Patterns wie „Arbitrary Cycles“ erlaubt, auf diese Weise aber auch Modellierungsfehler nicht algorithmisch ausschließt. BPEL dagegen stellt zu jedem Zeitpunkt der Ausführung sicher, dass keine Deadlocks oder Synchronisationsprobleme im Kontrollfluss auftreten können. Aufgrund dieser unterschiedlichen Entwurfsannahmen ist eine standardisierte Abbildung daher komplex und das Ursprungsmodell ist im Zielmodell nicht immer wiederzuerkennen.

Wegen der oben genannten Unzulänglichkeiten und der wachsenden Verbreitung der BPMN auf den Fachseiten wurde die Standardisierung BPMN 2.0 begonnen. Das Ziel war es, der BPMN mit einem ausführbaren Metamodell sowie einem Austauschformat für Diagramme ein solides Fundament zu geben und damit die Überführung zu BPEL obsolet zu machen.

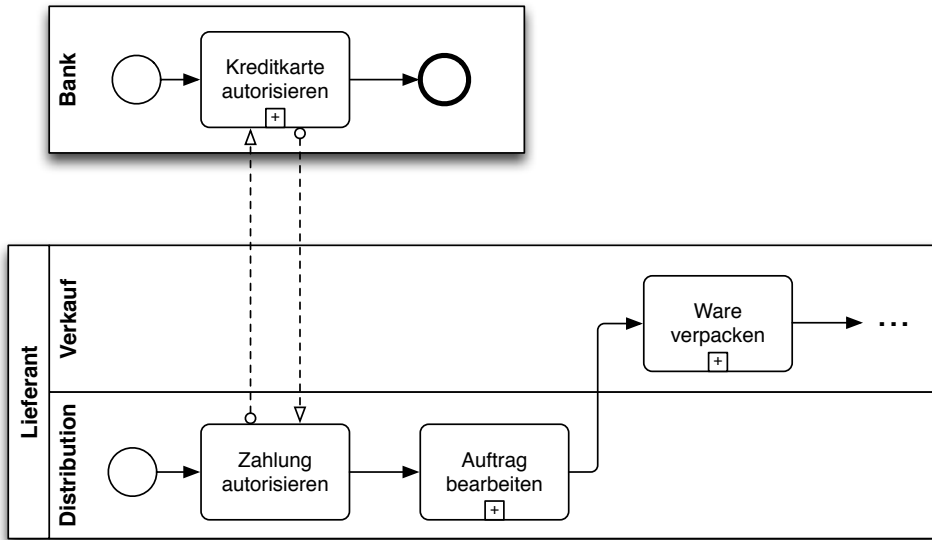
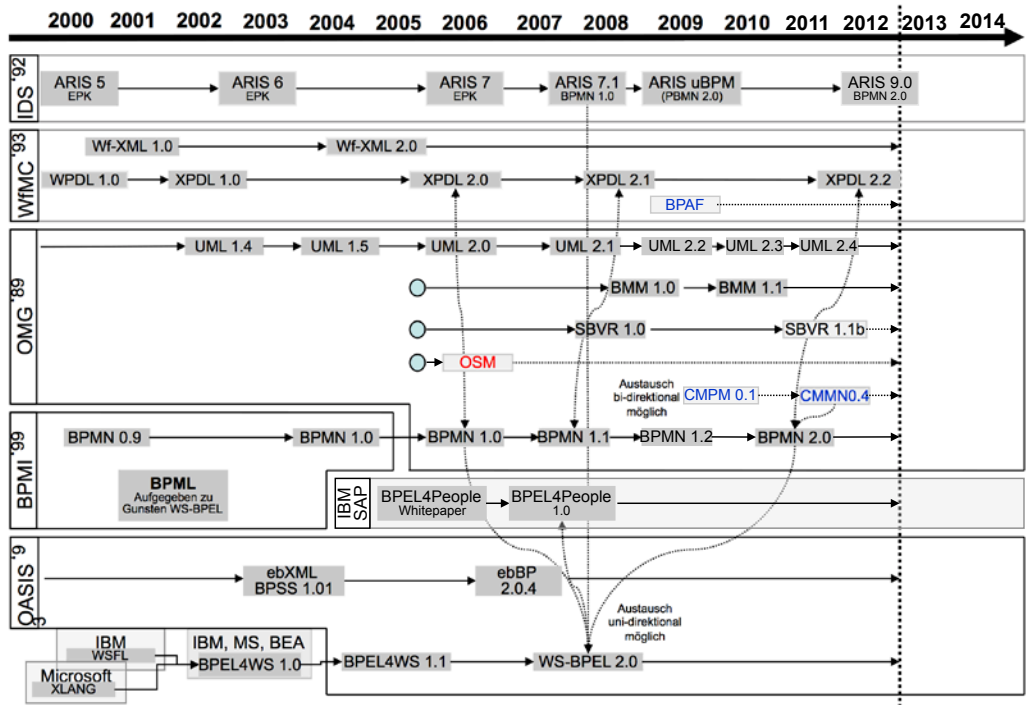


Bild 7.22 BPMN-Beispiel



Dr. Martin Bartonitz, Jan. 2013

Bild 7.23 Historie der Entwicklung von Standards im Geschäftsprozessmanagement seit 2000

So soll eine durchgängige Modellierung von fachlichen und technischen Prozessen in der gleichen Sprache möglich werden. Zusätzlich erhofft man sich dadurch eine bessere Interoperabilität zwischen Werkzeugen. Zumindest was die Austauschbarkeit der Diagramme betrifft, kann man einen Erfolg vermelden: Die OMG BPMN Model Interchange Working Group überprüft regelmäßig die Austauschbarkeit der Diagramme und arbeitet eng mit den Herstellern zusammen, in der Regel gleichen sich die Modelle auf die Pixel. Auf der Seite der Ausführbarkeit sind die Modelle leider meist nicht interoperabel, unter anderem auch, weil der Standard hier keine klaren Vorgaben macht. Die BPMN 2.0 ist seit 2011 ein Standard der OMG und wurde 2013 in minimal modifizierter Form zum internationalen Standard (ISO/IEC) erhoben.

Die Projekterfahrungen mit der Modellierung auf Basis von BPMN haben gezeigt, dass die Dokumentationssicht der Anwender nach der Umsetzung in die Workflow-Umgebung doch deutlich anders aussah. In einigen Fällen ist die Anzahl der Aktivitäten meist auf die Hälfte zusammengeschrumpft. Das passiert dann, wenn Modellierer Geschäftsprozesse in BPMN erfassen, ohne das technische Zielsystem im Detail zu kennen. Process Engines haben ihre Eigenheiten. Sie haben sich über die Zeit in diversen Kundenprojekten entwickelt, sprich, sie folgten den dortigen weiteren Anforderungen. Wird nun ein schon vorhandenes Modell an den Prozessexperten übergeben, so wird er sich doppelt bemühen. Einerseits wird er die Aufgabenketten den Möglichkeiten der Engine anpassen. Aber noch viel wichtiger: Er wird auf Ergonomie achten und die Aufgaben möglichst zusammenfassen. Damit verlagern sich Teile der dokumentierten Aufgaben im BPMN-Modell in die von dem Workflow-Client aufgerufenen Anwendungen und unterliegen damit nicht mehr der Workflow-Engine. In anderen Fällen werden die technischen Modelle deutlich komplexer, da technische Eigenheiten ausmodelliert werden müssen, die fachlich keinen Belang haben. Als Beispiel sei eine fachliche Benachrichtigung genannt, die technisch aber durch Polling realisiert und deshalb anders modelliert werden muss. Zudem stimmen häufig die fachlichen Organisationseinheiten, in BPMN durch Pools und Lanes modelliert, nicht immer mit denen der technischen Modelle überein. Ein einfacher Bestellprozess mit Kunde und Anbieter kann fachlich z. B. mit zwei Pools modelliert werden. Wenn die technische Umsetzung nun aber fordert, die Bestellung des Kunden über eine Weboberfläche abzuwickeln und die Bestätigung per E-Mail zuzustellen, dann benötigt man zwei Pools (die Weboberfläche und den E-Mail-Service), die stellvertretend für den fachlichen Kunden auftreten. Die Modelle lassen sich somit nicht mehr einfach überlagern. Mit diesen Gedanken im Hinterkopf sollte man die Versprechen einiger Hersteller, ein fachliches Modell unmittelbar ausführen zu können, gründlich hinterfragen.

Unsere Empfehlungen dazu:

1. Für fachliche und technische Modelle die gleiche Modellierungssprache zu verwenden, ist erstrebenswert, so denn die Sprache zur Zielarchitektur passt.
2. Ausführbare fachliche Prozessmodelle sind eine Illusion. Auf jeden Fall sollte die Rolle des Prozessexperten besetzt werden, der sowohl die Fachlichkeit als auch die Eigenheiten der verwendeten Process Engine kennt und die Modelle entsprechend überführen kann.
3. Wir teilen die Meinung der großen Hersteller für die Orchestrierung von Services, weiterhin WS-BPEL zu empfehlen. Die BPMN ist dagegen zu favorisieren, wenn menschliche Aktivitäten mit einbezogen werden sollen.

4. Komplexe Entscheidungsregeln sollten nicht im Prozessmodell ausmodelliert werden. Es ist zwar ohne Weiteres möglich, Verzweigungen auf Basis von komplizierten Regeln in BPMN auszudrücken, besser ist es jedoch, diese auszulagern. Auf diese Weise bleiben die Modelle übersichtlich und die Regeln können geändert werden, ohne die Prozessmodelle anpassen zu müssen. Ein geeignetes Mittel könnte die gerade in der Entwicklung befindliche DMN (= Decision Model and Notation) sein, die als ergänzendes Element zur Regelmodellierung im Rahmen der OMG standardisiert wird.

7.7.2 Vorhersagbarkeit

Klassische BPM-Projekte verfolgen in der Regel das Ziel, wiederkehrende Abläufe zu identifizieren, zu strukturieren und im weiteren Verlauf zu standardisieren und möglichst auch zu automatisieren. Das macht Sinn bei Prozessen, die ohnehin „dunkelverarbeitet“ würden, bei denen man sich durch das Two-Level-Programming aber mehr Flexibilität bezüglich der Änderbarkeit verspricht. Anders ist die Situation bei Prozessen, die die kreative Mitarbeit von Menschen benötigen. Seit einigen Jahren gibt es eine Bewegung, die kritisch hinterfragt, ob die Automatisierung und die Reduktion des Menschen auf „Aufgabenarbeiter“ für alle Arten von Geschäftsprozessen anwendbar sind.

Unter diesem Aspekt lohnt es sich, die Art der zu leistenden Arbeit in den Prozessen genauer anzusehen. Hat die Arbeit einen hohen Grad an Wiederholungen oder nicht? Sind die Arbeitsschritte immer im Vorfeld vorhersehbar? Ist es Routinearbeit oder sogenannte Wissensarbeit, die nur von hochqualifizierten und kreativen Menschen durchgeführt werden kann?

Keith Swenson hat in [Swenson2012] ein Spektrum über sieben Kategorien von Vorhersagbarkeit aufgespannt, das Einfluss auf die Architekturentscheidungen hat:

- **Traditionelle Anwendungsentwicklung** – ist die durch den Prozess beschriebene Arbeit gut vorhersehbar und ändert sie sich auch über die Zeit nicht, können die Prozesse direkt in „Code gegossen“ werden. Die zunächst höheren Entwicklungskosten zahlen sich über die Zeit aus. Flexible Änderungen sind aber nicht oder nur sehr aufwendig möglich.
- **Process-driven Server Integration (PDSI)** – ist die Arbeit wiederkehrend und vorhersehbar, aber möchte man flexibler auf Veränderungen reagieren können, lassen sich die Prozesse deklarativ modellieren und von einer Process Engine ausführen. Man spricht hier auch von „Dunkelverarbeitung“. Menschen werden nur in Fehlerfällen involviert. Die Engine kümmert sich um die Persistenz des Zustands und das Korrelieren von zusammengehörenden Daten und Ereignissen.
- **Human Process Management (HPM)** – ist die Arbeit wie bei PDSI hoch vorhersagbar, sie involviert aber auch Menschen, fällt sie in diese Kategorie. Der Prozess ist dafür verantwortlich, Daten zwischen Personen zu routen. Da Menschen mehr Flexibilität als Server von Geschäftsprozessen einfordern, müssen hier zusätzliche Dinge wie Urlaubsvertretungen, Feiertagskalender, Eskalationsregeln und Usability betrachtet werden. Die Prozesse erstellen Aufgaben, die den Menschen zugewiesen werden. Als Schnittstelle dient eine sogenannte Task- oder Worklist. Die Menschen melden sich an und sehen die ihnen zugewiesenen Aufgaben.

- **Production Case Management (PCM)** – diese Kategorie verlässt den Bereich des BPM und betritt das sogenannte Case Management. Die zu erledigende Arbeit erfordert ein solches Maß an Flexibilität, dass sie sich nicht mehr durch einen einzelnen Prozess beschreiben lässt. Dennoch steht die Menge der möglichen Aktionen im Vorfeld fest und wird durch die IT unterstützt. Der Wissensarbeiter kann fallweise entscheiden, welche der Aktivitäten ausgeführt werden müssen und welche nicht.
- **Adaptive Case Management (ACM)** – ist die Arbeit so unvorhersehbar, dass ein Wissensarbeiter für jeden Fall neue, kreative Lösungswege erfinden muss und nicht mit vordefinierten Aktionen arbeiten kann, fällt sie in diese Kategorie. Es gibt keinen Unterschied mehr zwischen Design-time und Runtime. Die durchgeführten Aktionen werden aber innerhalb des Falls dokumentiert, so dass sich die Wissensarbeiter darüber austauschen können. Die Arbeit ist immer noch so vorhersagbar, dass alte Fälle auch zum Lösen neuer herangezogen werden können. Die IT unterstützt die Arbeit mit solchen Fällen.
- **Social Business Software** – gemeint ist kollaborative Software, die keine Arbeitsstruktur im Sinne von Prozessen oder Fällen vorschreibt. Sie stellt aber die Mittel zur kollaborativen Aufgabenverwaltung zur Verfügung.
- **E-Mail, Telefon, ...** – die schließlich loseste Form der Kommunikation ohne vorgegebene Struktur und IT-Unterstützung.

Zu Beginn jedes BPM-Vorhabens sollte man sich über die Einordnung in diese Kategorien klar werden. Die meisten BPM-Suiten auf dem Markt konzentrieren sich auf PDSI und HPM, immer mehr bieten aber auch Unterstützung für PCM an. Der noch recht neue Standard CMMN (Case Management Model and Notation) bietet dafür eine alternative Notation zur BPMN an. Noch ist allerdings fraglich, ob sich diese Notation als eigener Standard etablieren wird oder ob dessen Elemente nicht besser in die BPMN als Ergänzung für schwachstrukturierte Prozesse einfließen sollten.

Im weiteren Verlauf des Kapitels besprechen wir hauptsächlich die Ausprägungen von PDSI und HPM.

7.7.3 Zweck der Ablaufsteuerung

Geschäftsprozesse repräsentieren Arbeitsabläufe in Unternehmen, durch die unternehmerischer Mehrwert entsteht. Geschäftsprozesse sind häufig Vorgänge, die auf Basis einiger Stammdaten zugehörige Vorgangsdaten bearbeiten. Typische Stammdaten sind Adressen oder Produktdaten, Beispiele für Vorgangsdaten sind Angebote, Aufträge oder Rechnungen.

Geschäftsprozesse werden in vielen Unternehmen auch heute noch zu einem erheblichen Teil auf Papier- oder Aktenbasis abgewickelt. Kommt eine Akte zu einem Sachbearbeiter, führt er einige spezifische Arbeitsschritte aus (ändert einige Daten, verschickt eine Rechnung oder Ähnliches) und leitet die Akte an andere Bearbeiter (etwa zur Genehmigung, zur Ansicht oder zu weiteren Aktionen) weiter. In zunehmendem Maße werden solche Akten (oder Vorgänge) in Unternehmen zwecks Weiterbearbeitung per Mail an andere Sachbearbeiter oder Abteilungen geschickt. Dieses Vorgehen ist umständlich, schwer nachvollziehbar und kann nicht kontrolliert werden. Ziel von Workflow-Systemen ist die (möglichst nahtlose) Einbindung solcher Abläufe in Softwaresysteme.

Die Übergänge zwischen verschiedenen Systemen sollen transparent und ohne Medienbrüche erfolgen. Daher steht das Workflow-Management in enger Beziehung zu Integration und Kommunikation, siehe Abschnitte 7.3 (Integration) und 7.5 (Kommunikation).

Bild 7.24 verdeutlicht, wie die Aufbau- und Ablauforganisation in Unternehmen auf den Entwurf von Softwaresystemen einwirken. Workflow-Systeme sollen dabei koordinierend und steuernd auf den Ablauf anderer Systeme wirken. Sie sollen:

- den richtigen Personen (in der Aufbauorganisation),
- zur richtigen Zeit (innerhalb der Ablauforganisation),
- mit den richtigen Werkzeugen (Softwaresystemen, Applikationen)
- die richtigen Daten bereitstellen.

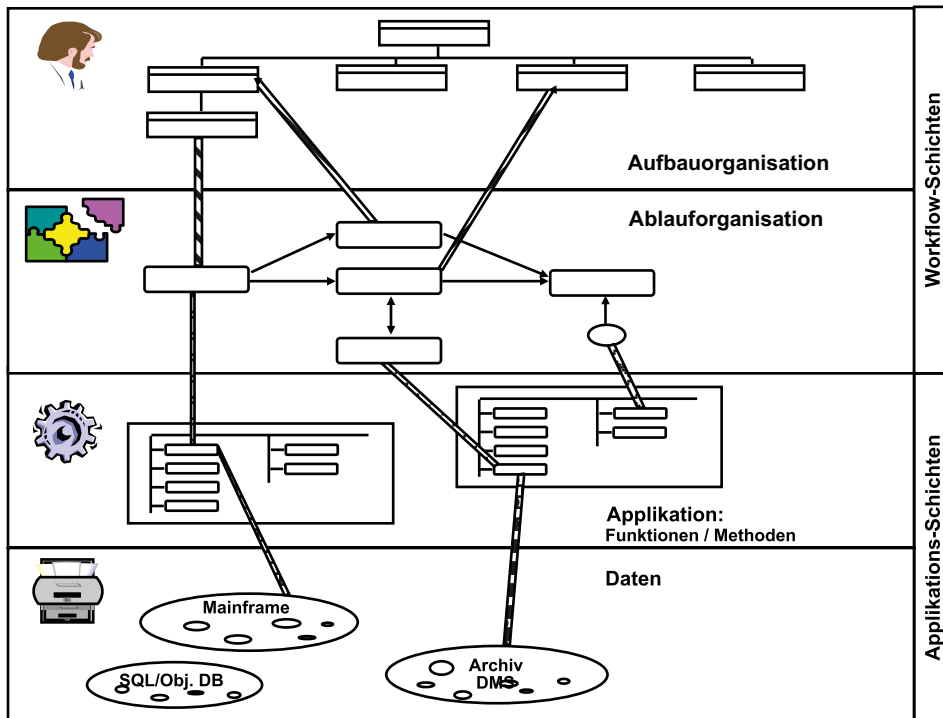


Bild 7.24 Von der Aufbauorganisation zum Softwaresystem

Zusätzlich verwalten sie Versions- und Statusinformation über die betroffenen Geschäftsvorfälle und weisen anhand von Gruppen-, Rollen- oder Verantwortlichkeitsprofilen Vorgänge flexibel zu (unter Berücksichtigung von Benutzerrechten, Vertretungs- oder sonstigen Geschäftsregeln).

Definition von Workflow-Management

Die Workflow Management Coalition⁴⁵ (WfMC) definiert Workflow-Management wie folgt:

⁴⁵ www.wfmc.org



Workflow is concerned with the automation of procedures where documents, information or tasks are passed between participants according to a defined set of rules to achieve, or contribute to, an overall business goal (WfMC Workflow Reference Model 1995, Chapter 2).

Workflow-Management ist die Automatisierung von Geschäftsprozessen oder Vorgängen, in denen Dokumente, Informationen oder Arbeitsaufträge unter Berücksichtigung von Regeln oder definierten Verfahren zur Erreichung allgemeiner Geschäftsziele von einem Teilnehmer zum nächsten gereicht werden.

Diese Definition stellt Anwender in den Mittelpunkt und lässt die beteiligten Systeme unspezifiziert.

7.7.4 Lösungsansätze

Wollen Sie in der Architektur eines Softwaresystems eine flexible Ablaufsteuerung (im Sinne der oben genannten Definition von Workflow-Management) vorsehen, so haben Sie drei Alternativen:

1. Ein Zustandsautomat mit Transitionstabellen steuert das System. Dieser Zustandsautomat ist Bestandteil des Systems selbst. Die Process Engine wird also selbst implementiert.
2. Ein Workflow-Management-System wird direkt in die Anwendung integriert, interagiert also nicht über externe Schnittstellen mit der Anwendung, sondern wird Bestandteil des Systems (*embedded workflow*).
3. Die Steuerung erfolgt durch ein externes Workflow-Management-System (WfMS) oder synonym Business Process Management System (BPMS).
4. Vorgänge werden manuell per E-Mail oder Groupware weitergeleitet.

Im ersten Fall werden die Zustände der Applikation sowie die möglichen Übergänge zwischen ihnen in einer Tabelle hinterlegt und können im Falle neuer Anforderungen darin zentral administriert werden. Zustände beziehen sich hierbei auch auf Bearbeitungszustände von Vorgängen oder Dokumenten. Beispiele: „Kredit Antrag genehmigt“, „Kredit Antrag vorläufig freigegeben“, „Kredit Antrag endgültig freigegeben“.

Im zweiten Fall werden die Prozessmodelle in einer standardisierten Sprache modelliert und von einem in die Anwendung eingebetteten Workflow-Management-System ausgeführt. Die Aufgabe der Navigation durch das Prozessmodell sowie die Persistenz des Zustands übernimmt die eingebettete Process Engine. Die enge Kopplung zwischen Engine und Anwendung hat Vor- und Nachteile. Ein gern genannter Vorteil ist die geringere Zugangshürde zu der Technologie für Entwickler. Sie müssen nicht mit einer externen Blackbox interagieren, sondern integrieren die Aufgabenimplementierung beispielsweise direkt über Java-Klassen. Ein gewichtigerer Vorteil ist aus Sicht der Autoren aber die Möglichkeit, dank entsprechenden Glue-Codes zwischen Anwendungslogik und WfMS die Prozessmodelle mit weniger technischen Details belasten zu müssen. Dadurch bleiben auch die technischen Prozessmodelle näher an den fachlichen. Als nachteilig wirkt sich aber die enge Kopplung z. B. in service-orientierten Architekturen aus, in denen man sich eine zentrale Orchestrierungskomponente mit Fokus auf den Nachrichtenaustausch zwischen Systemen wünscht.

Im dritten Fall wird anhand der Anforderungen des Projekts und der Organisation eine passende Standardsoftware ausgesucht. Streng genommen ist das Workflow-Management-System (WfMS) selbst ein Zustandsautomat, allerdings mit einer Reihe weiterer Aufgaben. Standard-WfMS haben den Vorteil, dass, wie weiter unten beschrieben, eine Vielzahl von fertigen Funktionen mitgeliefert werden. Oft verwenden kommerzielle Systeme eine proprietäre Benutzerverwaltung, was den Betriebs- und Verwaltungsaufwand erhöht. Die meisten Systeme bieten allerdings eine Synchronisation mit Verzeichnisdiensten per LDAP.

Im vierten Fall arbeiten Softwaresysteme lose gekoppelt. Es besteht das Risiko von Medienbrüchen oder Prozessunterbrechungen. Im Falle hochgradig komplexer oder kreativer Prozesse („individuelle Vorgänge“), deren Bearbeitung stark durch Heuristiken geprägt ist, bietet sich diese Alternative an.

Dank der Entwicklung im Bereich Adaptive/Production Case Handling steigt der Druck auf die WfMS-Hersteller, auch dynamischere, unstrukturierte Prozessvarianten zu unterstützen. Deshalb können Anwendungen des vierten Falls zunehmend auch mithilfe von WfMS umgesetzt werden. Dadurch ist die Nachverfolgung/Auditierung der Fälle deutlich einfacher als bei der ungeführten Abwicklung durch E-Mail oder Groupware.

Definition von Workflow-Management-Systemen

Die Definition eines Workflow-Management-Systems (WfMS) durch die WfMC hat einen technischen Fokus:



A Workflow Management System is one which provides procedural automation of a business process by management of the sequence of work activities and the invocation of appropriate human and/or IT resources associated with the various activity Steps. (WfMC – Workflow Reference Model, Chapter 2)

Ein Workflow-Management-System ist ein System, das Prozesse definiert, erzeugt und steuert durch den Einsatz von Software, die auf einer oder mehreren Workflow Engines läuft, die Prozessdefinitionen interpretieren, mit Anwendern interagiert und, wo gefordert, Softwaresysteme aufruft.

Typische Funktionen von WfMS

Workflow-Management-Systeme enthalten einige typische Funktionsgruppen:

- Definitions- oder Modellierungswerkzeuge für die Aufbauorganisation (die Benutzerverwaltung soll sicherstellen, dass rollen- oder personenbezogene Rechte auch systemübergreifend korrekt eingehalten werden) und die Ablauforganisation (Geschäftsprozesse). Die Systeme stellen hierfür (grafische) Werkzeuge zur Verfügung, mit deren Hilfe die entsprechenden Ablaufparameter zu den beteiligten (externen) Softwaresystemen hinterlegt werden können.
- Sowohl der Client als auch der Server müssen in der Lage sein, an vorgesehenen Stellen im Geschäftsprozess beteiligte Fremdsysteme (Applikationen) aufzurufen. In diesem Fall werden notwendige Parameter mit der Applikation ausgetauscht. Siehe hierzu den Abschnitt 7.3 (Integration).
- WfMS benötigen eine Administrationskomponente, mit der notwendige Systemparameter gepflegt werden können (etwa der Arbeitstagekalender).

- Auf Basis einer Protokollierungskomponente können Berichte über den Verlauf der Prozesse erstellt werden. Hier sei der Hinweis erlaubt, dass in Deutschland der Betriebsrat oder Personalrat großen Einfluss auf die Ausgestaltung der entsprechenden Prozesse und auf die zu protokollierenden Daten besitzt. Involvierem Sie diese Organisationseinheiten deshalb frühzeitig.

Praxisrelevante Zusatzfunktionen

Workflow-Management-Systeme müssen mehrere andere Systeme steuern. Diese Steuerung umfasst eine Reihe von Zusatzaufgaben, die spezifisch für die Bearbeitung von Geschäftsprozessen ist:

- Integration mit einem Postkorb: Ziel ist es, die Nutzung von Workflow- und E-Mail-Systemen möglichst einheitlich im Sinne der Benutzerakzeptanz zu gestalten. Daher bieten einige WfMS Plug-ins für die wichtigsten Mail-Systeme an, über die sie Worklist-Daten direkt in diesen Applikationen präsentieren und Workflow-Funktionen zur Ausführung anbieten. Mit WS-HumanTask steht inzwischen auch eine mächtige, standardisierte API zur Worklist-Integration zur Verfügung. Auch wenn nur wenige Hersteller die API voll unterstützen, sind zumindest die meisten Worklist-Implementierungen sehr davon inspiriert.
- Wiedervorlage: Ein Auftrag wird mit einem Termin versehen und zum definierten Datum wieder im eigenen Postkorb oder in dem einer Gruppe vorgelegt.
- Termin: Termine können für jede einzelne Aufgabe absolut oder relativ berechnet werden. Terminfestlegungen sollen eine Kontrolle des Durchflusses ermöglichen. Wird der Termin überschritten, werden diverse Mahnungen fällig oder andere Eskalationswege automatisch initiiert.
- Monitor: Diese Funktion führt Buch über den Status sowie die Historie der Prozesse. Sie ist damit in der Lage, Terminüberschreitungen, Laufzeitüberschreitungen und Ähnliches zu erkennen und zu dokumentieren. Neue Begriffe am Markt sind hier Business Activity Monitoring, Complex Event Processing und Business Performance Management. Das System soll mehr und mehr sich anbahnende Probleme, die durch Ressourcenengpässe verursacht werden, frühzeitig erkennen, Warnungen erhalten und damit proaktiv gegensteuern können. Eine Herausforderung liegt in der Abbildung der Zustände im technischen Modell auf Zustände im fachlichen Modell, denn obwohl das WfMS das technische Modell ausführt, sind die Fachabteilungen am Monitoring der fachlichen Aspekte interessiert.
- Ablage: Ein Prozess ohne elektronische Dokumente ist kaum noch denkbar. Diese Dokumente müssen abgelegt werden oder die Referenz auf die Ablage muss mitgeführt werden.

Aufbau von Workflow-Management-Systemen

Wenn sie nicht in die Anwendung eingebettet sind, werden WfMS im Normalfall als Client/Server-Systeme implementiert. Bild 7.25 verdeutlicht, dass sowohl der Workflow Client als auch der Server (die sogenannte *Engine*) mit vorhandenen Applikationen zwecks Steuerung kommunizieren.

Der Workflow-Client (Worklist Handler) bietet dem Benutzer eine Liste der Arbeitsaufträge (Tasks), die durch ihn zu erledigen sind. Initiiert der Benutzer eine der Aufgaben, so steuert der Client anhand der hinterlegten Parameter die jeweils nutzbare Anwendung an. Abschließend wird der Geschäftsfall in seinem Status weitergeschaltet, d. h. zur nächsten Aktivität geleitet. Können mehr als eine Aktivität erreicht werden, kann die Entscheidung entweder durch den Benutzer oder vordefiniert durch die Workflow Engine erfolgen.

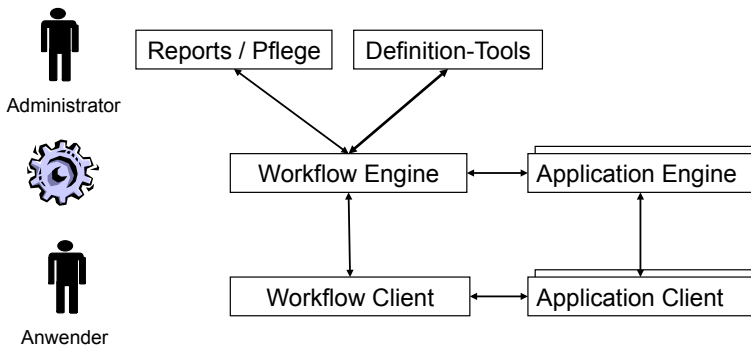


Bild 7.25 (Reduziertes) WfMC-Workflow-Referenz-Modell

7.7.5 Integration von Workflow-Systemen

Die Integration eines Workflow-Management-Systems mit anderen Systemen kann auf zwei Arten erfolgen:

- Ein Workflow-Management-System integriert die externen Systeme (siehe oben).
- Externe Systeme integrieren die Workflow-Funktionen: Arbeiten Benutzer immer nur mit einer Applikation, kann die Integration so gestaltet werden, dass über eine Programmierschnittstelle des Workflow-Management-Systems die Worklist von der Engine gelesen und in der Applikation angezeigt wird. Hierbei verschmelzen die beiden in Bild 7.25 dargestellten Clients zu einer gemeinsamen Anwendung.



Falls Sie ein System entwerfen, das mit einem WfMS kooperieren soll, sollten Sie diese Aspekte mit Ihren Auftraggebern klären. Diese Entscheidung besitzt großen Einfluss auf die Gestaltung der Ablaufsteuerung innerhalb Ihres Systems, obwohl das WfMS primär die übergreifende Steuerung behandelt.

Ein anderer Aspekt der Integration, der zunehmend wichtiger wird, ist der Austausch mit anderen Prozessmodellierungs- oder Simulationstools. Viele große Firmen setzen zur Dokumentation ihrer Geschäftsprozesse grafische Tools⁴⁶ ein und möchten diese Modelle im WfMS wiederverwenden. Anschließend müssen die Modelle den Eigenschaften der Workflow Engine angepasst werden. Dabei erfahren die Modelle meist eine Metamorphose: Einige der dokumentierten Aktivitäten (z. B. manuelle Aufgaben) werden in der technischen Umsetzung fehlen, dafür werden andere Aktivitäten (z. B. bei asynchronen Interaktionen oder für Datentransformationen) hinzukommen. Zudem werden die Aktivitäten mit weiteren spezifischen Steuerungsinformationen angereichert. Damit weichen die beiden Modelle stark ab und ein erneutes Importieren macht keinen Sinn mehr.

Der Autor geht davon aus, dass ein manuelles Übertragen sinnvoller ist, weil dieser Prozess der Metamorphose so viel bewusster und mit besserer Qualität durch den Prozessexperten im Zusammenspiel mit dem Anwender/Prozessberater erfolgen wird.

⁴⁶ Visio™, ARIS™ oder andere Zeichenwerkzeuge.

7.7.6 Mächtigkeit von WfMS

WfMS und BPMS müssen sich seit einigen Jahren daran messen lassen, in welchem Umfang sie die Workflow Patterns von Wil van der Aalst [VDAalst2002] umsetzen. Diese Muster beschreiben verschiedene Kombinationsmöglichkeiten bei der Abarbeitung von Aufgabenketten sowie Gestaltungsmöglichkeiten für den Datenfluss, der durchaus unterschiedliche Wege einschlagen kann.

Auf der Website⁴⁷ zu [VDAalst2002] sind die Patterns zum besseren Verständnis grafisch animiert. Eine tabellarische Aufstellung beschreibt eine Reihe von Systemen bezüglich ihrer Umsetzung der Patterns. Ebenso sind eine Reihe von Prozessbeschreibungssprachen, darunter auch die schon erwähnten BPMN und BPEL, hinsichtlich ihrer Pattern-Vollständigkeit beurteilt worden.

7.7.7 Weiterführende Literatur



[VDAalst2002] ist eine Übersicht über Workflow-Pattern.

[Bartonitz2005] gibt einen Überblick über die wichtigen Standards inklusive der historischen Entwicklung rund um Workflow-Management, fortgesetzt in [Bartonitz2006, 2010].

[Bartonitz2010] → BPM, GPM, BAM, BPMN, BPEL, XPD, EABPM, CMPM

[http://www.bpm-netzwerk.de/content/articles/viewArticle.do?id=d915570e265ad62201266f515ff902d0 ...](http://www.bpm-netzwerk.de/content/articles/viewArticle.do?id=d915570e265ad62201266f515ff902d0...)

[Freund2014] *Freund, J., Rücker, B.*: Praxishandbuch BPMN 2.0. Hanser. 2014.

Gibt eine Einführung in die BPMN 2.0 anhand von vielen praktischen Beispielen.

[Lessen2011] *van Lessen, T., Lübke, D., Nitzsche, J.*: Geschäftsprozesse automatisieren mit BPEL. dpunkt. 2011.

Gibt eine Einführung in die Modellierung von Prozessen zur Orchestrierung von Services mit WS-BPEL.

[Slama2011] *Slama, D., Nelius, R.*: Enterprise BPM: Erfolgsrezepte für unternehmensweites Prozessmanagement. dpunkt. 2011.

Stellt eine Methode zur strategischen und strukturierten Planung und Implementierung von BPM in Projekten (IBPM) und Unternehmen (EBPM) vor.

[Swenson2010] *Swenson, K. D.*: Mastering the Unpredictable: How Adaptive Case Management Will Revolutionize the Way That Knowledge Workers Get Things Done. Meghan-Kiffer Press. 2010.

Stellt die Grundlagen und Konzepte von unstrukturierten Prozessen im Kontext ACM vor.

[Swenson2012] *Swenson, K. D., Palmer, N., Pucher, M. J.*: How Knowledge Workers Get Things Done: Real-World Adaptive Case Management. Future Strategies Inc. 2012.

⁴⁷ www.workflowpatterns.com, eine Initiative der Universitäten Eindhoven und Queensland.

Martin Bartonitz (martin@bartonitz.net) wandte sich 1992 von der Prozesssteuerung in der Welt der Physik der Vorgangssteuerung in der Bürowelt zu. Erste Erfahrungen damit machte er beim Hersteller des Workflow-Management-Systems COSA Workflow als Projektleiter, Trainer und Produktmanager. Zwischen 1998 und 2003 stellt er sein Wissen auf den Gebieten Methoden und Werkzeuge im Workflow-Umfeld als Berater zur Verfügung. Seit 2004 arbeitet er als Produktmanager in den Bereichen Workflow- und Dokumenten-Management, zuerst bei der Firma SAPERION AG, aktuell bei der OPTIMAL SYSTEMS GmbH.

Tammo van Lessen beschäftigt sich seit knapp zehn Jahren intensiv mit der Automatisierung von Geschäftsprozessen. Zurzeit arbeitet er als Senior Consultant bei der innoQ im Bereich SOA/BPM. Er ist PMC Chair Apache ODE (Open Source BPEL Engine) und Member der Apache Software Foundation. Außerdem hat er aktiv an der Standardisierung von BPMN 2.0 mitgewirkt. Er ist Mitautor des Buchs „Geschäftsprozesse automatisieren mit BPEL“ und hat zahlreiche Artikel über BPM veröffentlicht.

■ 7.8 Sicherheit

von Wolfgang Korn und Tobias Hahn

„There's no such thing as perfect security.“

– Bruce Schneier

In der modernen Gesellschaft finden sich Computer überall. Egal ob im Auto, an der Supermarktkasse oder im eigenen Smartphone – die Informationstechnologie hat inzwischen alle Bereiche des täglichen Lebens erreicht. Daten werden kontinuierlich erfasst, verarbeitet und zwischen verschiedenen Systemen übertragen. Die Sicherheit der Daten und Systeme zu gewährleisten, ist eine Herausforderung, die bereits in der Konzeptionsphase beginnt und in allen Phasen der Softwareentwicklung beachtet werden muss.

In diesem Abschnitt werden Sie mit den grundlegenden Konzepten der IT-Sicherheit vertraut gemacht. Anhand von ausgewählten Beispielen wird verdeutlicht, wie sich einzelne Herausforderungen in der Praxis lösen lassen.

7.8.1 Motivation – Was ist IT-Sicherheit?

Das BSI definiert IT-Sicherheit als den „Schutz von Informationen“. Die persönlichen Daten von Privatpersonen, die Kundendaten einer Bank oder die Firmengeheimnisse eines Unternehmens sind schützenswerte Daten und sollten nur für berechtigte Personen oder Systeme zugänglich sein. Erhöht man allerdings den Schutz der Daten, z. B. indem man die Zugriffsmöglichkeiten einschränkt, hat das normalerweise Auswirkungen auf andere Eigenschaften des Systems, wie zum Beispiel auf die Benutzbarkeit und somit auch auf die Produktivität.

Professor Eugene Spafford, bekannt geworden unter anderem durch die Analyse des ersten Computerwurms:

„The only true secure system is one that is powered off, cast in a block of concrete and sealed in a lead-lined room with armed guards – and even then I have my doubts.“

Besonders wichtig wird dieser Punkt durch die Reaktion eines normalen Nutzers auf Sicherheitsmechanismen, die ihn bei der Arbeit einschränken. Muss der Nutzer selbst einen zusätzlichen Aufwand leisten, um die Sicherheit zu erhöhen, z. B., indem er verschlüsselte Mails verschickt, wird er in den meisten Fällen darauf verzichten und den für ihn einfacheren, jedoch unsichereren Weg wählen. Selbst in den Zeiten von PRISM werden die meisten E-Mails immer noch unverschlüsselt verschickt, obwohl es mit PGP bereits seit Langem eine inzwischen auch relativ einfach zu bedienende Möglichkeit zur Verschlüsselung von E-Mails gibt. Eine von Telepolis durchgeführte Umfrage unter deutschen Journalisten zeigte, dass die meisten nicht in der Lage sind oder kein Interesse daran haben verschlüsselte E-Mails zu verschicken oder zu empfangen. Eine hohe Akzeptanz von IT-Sicherheit durch den Nutzer wird nur erreicht, indem die Schutzmechanismen möglichst transparent umgesetzt werden.

7.8.2 Sicherheitsziele

Die grundlegenden Konzepte der IT-Sicherheit sind Vertraulichkeit, Integrität und Verfügbarkeit, die je nach Anwendungsgebiet unterschiedlich wichtig bewertet werden.

Vertraulichkeit

Vertraulichkeit =
lesbar nur für Zielgruppe

Vertraulichkeit bedeutet, dass Daten lediglich von denjenigen Personen oder Systemen gelesen werden können, für die diese Daten bestimmt sind. Insbesondere bei Systemen, die über Rechnernetze oder das Internet miteinander kommunizieren, ist bei der Klartextkommunikation keine Vertraulichkeit gegeben. Daten passieren auf ihrem Weg durch solche Netze verschiedene Stationen, die in der Regel von Fremden administriert werden. Da die Sicherheit dieser Stationen weder eingeschätzt noch beeinflusst werden kann, sollte hier immer vom schlimmsten Fall ausgegangen werden.



Denken Sie pessimistisch: Sehen Sie Klartextkommunikation über Rechnernetze grundsätzlich als unsicher an! Firmeneigene interne Netze stellen hier keine Ausnahme dar!

Integrität

Integrität =
Manipulationen entdecken

Integrität bedeutet, dass Daten nicht unberechtigt in ihrem Inhalt manipuliert werden.⁴⁸ Diese Forderung gilt während der Kommunikation, aber auch wenn sich Daten gerade in der Datenbank ausruhen.⁴⁹

⁴⁸ Das Format der Daten kann sich durch die technischen Gegebenheiten der Kommunikation durchaus ändern, etwa durch Protokoll- oder Zeichensatzkonvertierungen.

⁴⁹ Oder sich mit Freunden im Container des Application-Servers treffen.

Vor der Veränderung der Daten während der Kommunikation kann ein Softwaresystem grundsätzlich nicht geschützt werden. Es ist jedoch möglich, solche Veränderungen oder Manipulationen systematisch aufzudecken. Man spricht dabei von der Gewährleistung der Datenintegrität. Verfahren, die hierfür eingesetzt werden können, sind Hash-Funktionen und digitale Signaturen.

Authentifizierung und Autorisierung

Um Informationen und Daten kontrolliert verfügbar zu machen, werden die Konzepte der Authentifizierung und Autorisierung verwendet. Bei der Authentifizierung wird nachgewiesen, dass der Nutzer, der auf Daten zugreifen will, auch wirklich der ist, der er vorgibt zu sein. Dieser Nachweis kann auf verschiedenen Wegen erfolgen, z. B. durch Passwörter (etwas, das der Nutzer kennt), durch Smartcards oder kryptografische Tokens (etwas, das der Nutzer besitzt) oder durch biometrische Merkmale, wie einen Fingerabdruck (etwas, das der Nutzer ist). Zur Erhöhung des Sicherheits-Niveaus können auch verschiedene Methoden kombiniert werden, man spricht dann von Zwei- oder generell Mehr-Faktor-Authentifizierung. Ein bekanntes Beispiel hierfür ist Online-Banking: Nach der Anmeldung mit Benutzername und Passwort werden zusätzlich TANs verwendet, die sich im Besitz des Nutzers befinden. Dieses Verfahren ist weit verbreitet und wird von großen Teilen der Nutzer akzeptiert.

Authentizität =
Nachweis der Identität

Der Vorgang der Authentifizierung selbst kann sich sowohl auf Personen (etwa bei der Anmeldung an ein System) als auch auf Softwaresysteme beziehen. In letzterem Fall weist beispielsweise ein Webserver gegenüber dem Benutzer seine Identität nach.

Unmittelbar verknüpft mit der Authentifizierung ist die Autorisierung. Im Rahmen der Autorisierung werden Benutzern oder Systemen nach erfolgreicher Authentifizierung ihrer Identität entsprechende Berechtigungen eingeräumt. Anhand der zugewiesenen Berechtigungen wird dann überprüft, ob auf die angeforderten Ressourcen oder Daten zugegriffen werden darf. Die zugewiesenen Rechte sollten dabei möglichst dem „*Principle of least privilege*“ folgen: Der Nutzer darf nur auf Ressourcen zugreifen, die er unbedingt für die Erfüllung seiner Aufgabe benötigt.

Autorisierung = Berechtigung

Verfügbarkeit

Neben den genannten Zielen der Sicherheit von Datenkommunikation gibt es nach dem Modell I des Bundesamtes für Sicherheit in der Informationstechnik noch das Ziel der „Verfügbarkeit“. Dies bedeutet, dass der Zugriff auf Daten und Programme für berechtigte Nutzer jederzeit möglich sein muss. Ein Beispiel für den zumindest temporären Verlust von Verfügbarkeit sind die sogenannten „*Denial-of-Service*“-Angriffe, bei denen z. B. ein Webserver durch eine sehr große Anzahl von gefälschten Anfragen überlastet wird. Der Webserver kann dann keine weiteren Anfragen mehr bearbeiten, die Verfügbarkeit ist nicht mehr gegeben.

Denial-of-Service

Elektronische Kommunikation muss gesichert werden

Ungesicherte elektronische Kommunikation besitzt nicht einmal das Sicherheitsniveau einer Postkarte: Während bei einer Postkarte in der Regel lediglich die Vertraulichkeit verletzt ist, erreicht die ungesicherte elektronische Kommunikation keines der drei Sicherheitsziele.

Veränderungen und falsche Absender können bei einer (handschriftlichen) Postkarte leicht erkannt werden, während dies bei elektronischen Daten nicht ohne weitere Hilfsmittel möglich ist.

Anhand einer kurzen Geschichte soll hier verdeutlicht werden, welche Auswirkungen die Verletzung der einzelnen Sicherheitsziele haben kann:



... Wie schon in den vergangenen Jahren verliefen auch die diesjährigen Gehaltsgespräche nicht zu Mallory's Zufriedenheit. „Das muss sich jetzt ändern!“, dachte sich Mallory, seines Zeichens Systemverantwortlicher der Firma Insecure Communications Inc. Bereits im vergangenen Jahr hatte Mallory beim routinemäßigen Schnüffeln in den Mails seiner Kollegen festgestellt, dass sein Chef Bob nach Abschluss der Gehaltsverhandlung die Ergebnisse per Mail der zuständigen Personalreferentin Alice mitteilte. „Gut, dass Bob ein so ausgeprägtes Sicherheitsbewusstsein besitzt. Leichter kann er es mir nicht machen“, sagte sich Mallory und machte sich auch gleich ans Werk. Er selbst betreute den bei Insecure Communications eingesetzten Mail-Server, was ihm sein Vorhaben deutlich erleichterte. Das „Korrigieren“ seiner Gehaltserhöhung war insofern eine Kleinigkeit für ihn. Die kleine Änderung würde Alice sicher nicht bemerken – wie auch. Nebenbei verschaffte sich Mallory auch gleich einen Überblick über die Ergebnisse seiner Kollegen. Dabei stieß er auf Cora Victim, seine Kollegin in der Systemadministration. Ihr gutes Abschneiden in der diesjährigen Gehaltsrunde erhöhte seinen Neid auf die begabte Kollegin. „Wenn ich schon mal dabei bin, könnte ich auch gleich etwas gegen die ständige Bevorzugung von Cora tun. Eine kurze Mail von Cora an Alice, in der sie sich über ihre nicht angemessene Gehaltserhöhung beschwert, sollte erst mal reichen, um sie ein wenig in Misskredit zu bringen.“ ...

7.8.3 Lösungskonzepte

Verschlüsselung
Plaintext
Ciphertext

Vertraulichkeit, Integrität und Authentizität lassen sich durch die Verwendung von kryptografischen Verfahren und Algorithmen auf einem hohen Sicherheitsniveau erreichen.

Eine der elementaren Anwendungen von Kryptografie ist die Verschlüsselung (siehe Bild 7.26): Der zu übermittelnde Klartext (*Plaintext*) wird vom Absender durch einen Algorithmus in eine verschlüsselte Form (*Ciphertext*, von engl. cipher:

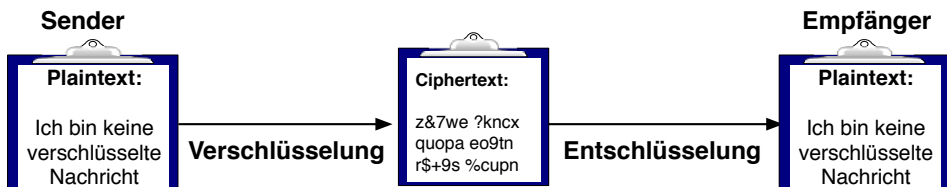


Bild 7.26 Prinzip der Verschlüsselung

Verschlüsselung) überführt. Dieser Ciphertext wird zum Empfänger übertragen und dort wieder in den Klartext entschlüsselt. Wird bei gleichem Klartext ein unterschiedlicher Schlüssel verwendet, ergibt sich ein unterschiedlicher Geheimtext.

Grundlagen kryptografischer Verfahren

Die Sicherheit von modernen kryptografischen Verfahren, die zur Gewährleistung der oben genannten Ziele eingesetzt werden, hängt von der Geheimhaltung der dabei verwendeten Schlüssel ab. Der jeweils eingesetzte Algorithmus sollte allgemein bekannt sein.⁵⁰

In die Transformation von Plaintext in Ciphertext geht eine Variable ein, der sogenannte Schlüssel. Der Schlüssel bestimmt, wie die Transformation der Eingabedaten erfolgt. Die Sicherheit dieser Verfahren beruht darauf, dass die benötigten Schlüssel nur den berechtigten Teilnehmern zugänglich sind und nicht erraten oder ausprobiert werden können. Mit Kenntnis des Schlüssels kann jedermann Nachrichten verschlüsseln und verschlüsselte Nachrichten lesen. Ohne Kenntnis des Schlüssels sollte es nicht möglich sein, vom Klar- auf den Geheimtext (und umgekehrt) zu schließen.

Für die Auswahl der Verfahren sollte man folgende Regeln berücksichtigen:



Verwenden Sie **ausschließlich** Verfahren, deren Sicherheit nur auf der Geheimhaltung der verwendeten Schlüssel basieren. Dieser wichtige Grundsatz wurde bereits 1883 formuliert und ist als *Kerckhoffs'sches Prinzip* bekannt. Algorithmen, deren „Sicherheit“ in der Unkenntnis des Verfahrens besteht (*Security by Obscurity*), haben zwangsläufig das Problem, dass keine zuverlässigen Erkenntnisse über ihre Sicherheit vorliegen.

Eigene Implementierungen bekannter Algorithmen sind selten sinnvoll. Für nahezu alle Verfahren existieren kommerzielle oder freie (teilweise Open-Source-)Produkte. Auf Grund der breiteren Anwendung der Produkte werden Sicherheitsmängel schneller bekannt und behoben, als dies bei Eigenentwicklungen der Fall ist.

Verschlüsselungsverfahren

Für die Verschlüsselung gibt es symmetrische und asymmetrische Verfahren:

- Bei symmetrischen Verfahren kommt für Verschlüsselung und Entschlüsselung der gleiche Schlüssel zum Einsatz. Um verschlüsselt miteinander kommunizieren zu können, müssen alle Teilnehmer den geheimen Schlüssel kennen. Da die Sicherheit des Verfahrens von der Geheimhaltung des Schlüssels abhängt, muss ein sicherer Austausch der Schlüssel zwischen allen beteiligten Partnern stattfinden. Bei Kommunikation mit mehr als zwei Beteiligten ist auch die Anzahl der benötigten Schlüssel ($n \cdot (n-1)/2$) kaum handhabbar. Ein symmetrisches Verfahren kann als sicher angesehen werden, wenn es keinen Angriff gibt, der wesentlich schneller als das komplette Durchsuchen des Schlüsselraums ist (*Brute-Force Angriff*). Der bekannteste Vertreter dieser Verfahren ist AES.

⁵⁰ Die Nutzung bekannter Algorithmen bietet den Vorteil, dass diese von unabhängigen Personen geprüft und auf ihre Sicherheit hin getestet werden können. Erkannte Sicherheitsmängel werden publiziert und die Algorithmen oder Implementierungen entsprechend angepasst.

- Bei asymmetrischen Verfahren wird ein Schlüsselpaar verwendet. Einer der beiden Schlüssel wird nur zur Verschlüsselung verwendet und ist öffentlich zugänglich. Die Entschlüsselung ist nur mit dem zweiten, geheimen Schlüssel möglich. Diese Verfahren werden als *Public-Key-Verfahren* bezeichnet. Wichtigster Vertreter der asymmetrischen Verfahren ist der RSA-Algorithmus. Ein Verfahren, das ausschließlich zur Signatur genutzt werden kann, ist DSA. Der Nachteil dieser Verfahren besteht in ihrer Ausführungsgeschwindigkeit: Sie sind etwa 1000-fach langsamer als ihre symmetrischen Kollegen. Aus diesem Grund kommen in der Praxis normalerweise Kombinationen beider Verfahren, sogenannte *Hybrid-Verfahren*, zum Einsatz. Die Sicherheit der asymmetrischen Verfahren hängt bei den meisten Verfahren von einem zugrunde liegenden mathematischen Problem ab, welches ohne die Kenntnis von Zusatzinformationen (in Form des geheimen Schlüssels) nicht gelöst werden kann.

Hybride Verfahren

Hybride Verfahren (siehe Bild 7.27) machen sich die Vorteile der symmetrischen (Geschwindigkeit) und der asymmetrischen Verfahren (Schlüsselverteilung) zunutze. Zunächst wird ein zufälliger symmetrischer Sitzungsschlüssel generiert. Mit diesem werden die Nutzdaten verschlüsselt (Punkt 1 in Bild 7.27). Anschließend wird der Sitzungsschlüssel mit dem öffentlichen Schlüssel des Empfängers asymmetrisch verschlüsselt (Punkt 2). Jetzt können die verschlüsselten Nutzdaten und der verschlüsselte Sitzungsschlüssel gefahrlos transportiert werden. Auf Seiten des Empfängers wird zunächst der Sitzungsschlüssel wieder (asymmetrisch) entschlüsselt (Punkt 3). Mit dem Sitzungsschlüssel können die Nutzdaten wieder symmetrisch entschlüsselt werden (Punkt 4).

Dieses Verfahren wird auch im Internet beim Aufruf verschlüsselter Webseiten über das https-Protokoll verwendet. Das vom Server übertragene Zertifikat enthält dessen öffentlichen Schlüssel, mit dem der Client dann den von ihm gewählten symmetrischen Schlüssel verschlüsseln und an den Server schicken kann.

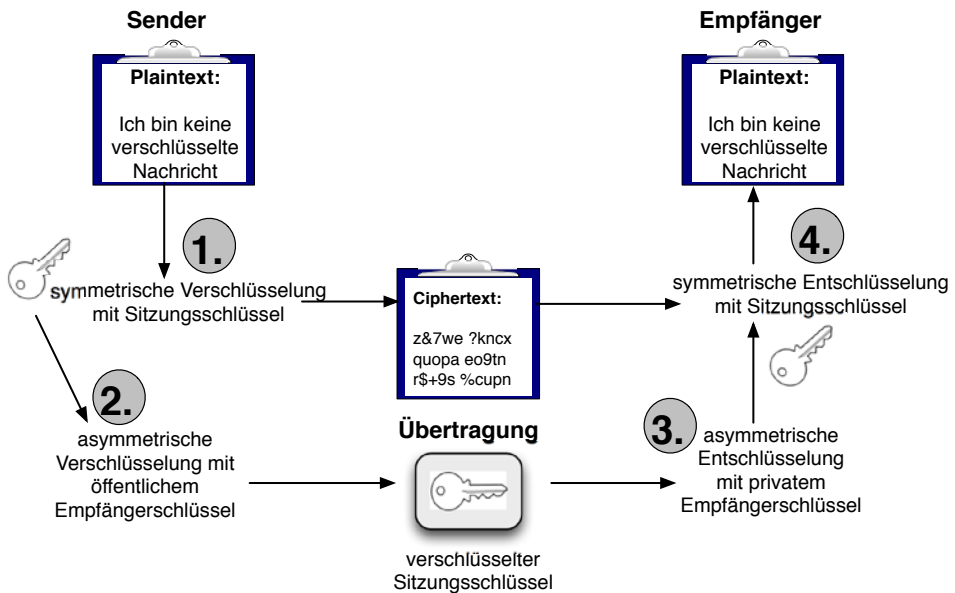


Bild 7.27 Hybride Verschlüsselungsverfahren

Integrität und Authentizität

Integrität und Authentizität werden mit Hilfe digitaler Signaturen gewährleistet, die auf asymmetrischen Verschlüsselungsverfahren basieren. Dabei ist die Bedeutung von öffentlichem und geheimem Schlüssel vertauscht. Das Verfahren zur Erzeugung einer digitalen Signatur entnehmen Sie Bild 7.28.

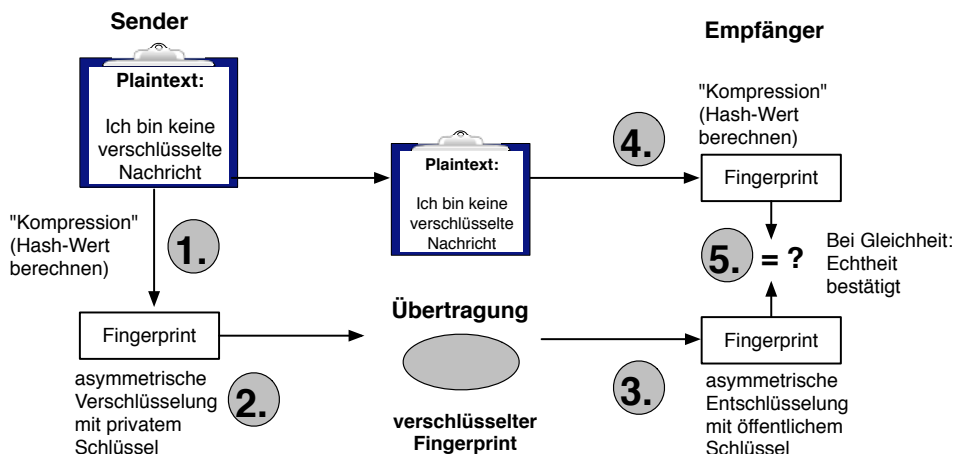


Bild 7.28 Digitale Signatur

Zunächst wird mit Hilfe einer kryptografischen Hashfunktion (etwa: MD5, SHA-1) eine Prüfsumme (*Fingerprint*, *Message Digest*) der zu signierenden Nachricht errechnet. Danach anschließend wird diese Prüfsumme mit dem privaten Schlüssel verschlüsselt. Die verschlüsselte Prüfsumme wird dann zusammen mit der Nachricht an den Empfänger übertragen. Der Empfänger entschlüsselt die Prüfsumme mit dem öffentlichen Schlüssel des Absenders. Gelingt dies, ist die Herkunft der Nachricht gesichert. Im nächsten Schritt wird die Prüfsumme der Nachricht errechnet und mit der der Nachricht beigelegten Prüfsumme verglichen. Ergeben sich Abweichungen, wurde die Nachricht während des Transports verändert.

Authentisierungen auf Basis kryptografischer Verfahren funktionieren ähnlich wie die digitale Signatur. Im ersten Schritt stellt das System, an dem die Authentisierung erfolgen soll, eine zufällige Datenfolge bereit. Diese wird von dem System, das sich authentisieren will, mit seinem privaten Schlüssel verschlüsselt und zurück übertragen. Durch Entschlüsselung dieser Daten mit dem öffentlichen Schlüssel und Vergleich mit den Originaldaten kann überprüft werden, ob die Authentisierung erfolgreich war oder nicht. Dieses Verfahren wird auch als *Challenge-Response-Verfahren* bezeichnet.

Die beschriebenen Verfahren setzen die nachprüfbare Zuordnung von öffentlichen Schlüsseln zu Personen oder Systemen voraus.⁵¹

⁵¹ Falls diese Zuordnung nicht sauber gelöst ist, kann es zu den gefährlichen „man in the middle“-Angriffen kommen, bei denen ein Angreifer beiden Kommunikationspartnern eine falsche Identität vortäuscht und Nachrichten verfälscht oder missbraucht.

Ein Beispiel aus der Praxis: OAuth

OAuth steht für Open Authorization und ist ein offenes, auf Standards der IETF aufbauendes Protokoll zur sicheren API-Autorisierung verschiedener Applikationen. Durch die Verwendung von OAuth können Nutzer einer dritten Partei Zugriff auf ihre Web-Ressourcen erlauben, ohne ihre Credentials (Benutzername und Passwort) weitergeben zu müssen. Die Zugangsdaten und die Nutzeridentität bleiben der Client-Anwendung gegenüber verborgen. Die hierbei verwendeten Access Tokens ermöglichen (nach Autorisierung durch den Nutzer) einer Applikation im Namen eines Nutzers den Zugriff auf eine Web-Ressource. In den Tokens sind die vom Nutzer erlaubten Berechtigungen enthalten, die entsprechend dem „*Principle of least privilege*“ vergeben werden sollten.

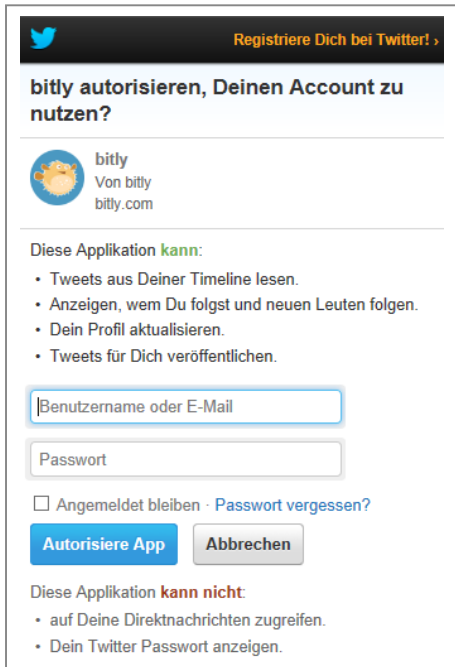


Bild 7.29
Autorisierungsdialog für bitly und twitter

OAuth bietet Unterstützung für mehrere Anwendungsfälle, die in der englischen Literatur oft als „flows“ bezeichnet werden. Beispielhaft wird hier das Webserver-Szenario dargestellt. Die daran beteiligten Parteien werden in drei unterschiedliche Rollen eingeteilt:

- Der Nutzer des Dienstes.
- Ein Client, z. B. eine Web- oder eine Mobil-Anwendung. Als Beispiel wird hier der Kurz-URL-Dienst *bitly* verwendet.
- Ein Server, auf dem die Ressourcen des Nutzers liegen. Unser Beispiel: *twitter* (siehe Bild 7.29).

Will jetzt der Nutzer dem Client Zugriff auf den Server geben, werden die folgenden Schritte durchlaufen:

1. Der Nutzer öffnet die bitly-Webseite und möchte bitly erlauben, auf seinen twitter-Account zuzugreifen.

2. bitly startet den OAuth-Protokollablauf, der Nutzer wird zu twitter umgeleitet, dort werden ihm die von bitly angeforderten Berechtigungen angezeigt. Stimmt der Nutzer zu, kann er, indem er sein twitter-Passwort eingibt, den Zugriff für bitly erlauben.
3. Nach erfolgreicher Autorisierung erhält bitly ein Access Token für den twitter Account des Nutzers.
4. bitly kann das Token jetzt verwenden, um im Namen des Nutzers auf dessen twitter-Account zuzugreifen.

Der bei diesem Beispiel verdeutlichte Anwendungsfall ist nur einer von vielen die mit OAuth realisiert werden können. Durch die Verwendung der anderen Szenarios lässt sich das OAuth Protokoll auch in vielen weiteren Situationen sinnvoll einsetzen. Es wird inzwischen von vielen großen Seiten verwendet, um den Zugriff per API zu kontrollieren. Hierzu gehören unter anderem twitter, Facebook, LinkedIn und XING. Google bietet Entwicklern mit dem „OAuth 2.0 Playground“ eine Möglichkeit, das bei ihren Diensten verwendete Protokoll kennenzulernen.

Zertifikate: Zuordnung von Schlüsseln an Personen oder Systeme

Die Bindung von Schlüsseln an Personen oder Systeme wird durch *Zertifikate* erreicht. Diese enthalten Informationen zum Inhaber sowie dessen öffentlichen Schlüssel. Um ein Fälschen dieser Zertifikate zu verhindern, werden sie vom Herausgeber digital unterzeichnet. Voraussetzung für die Vertrauenswürdigkeit von Zertifikaten ist daher die Vertrauenswürdigkeit des Herausgebers. Die Rolle dieser Herausgeber wird durch Zertifizierungsstellen (*trust center*) übernommen. In Deutschland unterliegen diese Zertifizierungsstellen als Voraussetzung für die Rechtsverbindlichkeit der Zertifikate einer strengen behördlichen Kontrolle.

Zertifikate = Bindung von Schlüsseln an Personen

Werden Zertifikate lediglich zur Sicherung der internen Kommunikation Ihres Unternehmens benötigt, können Sie eine eigene Zertifizierungsstelle aufbauen. Dazu einige kritische Aspekte als Entscheidungshilfe:



- Ist der Betrieb einer eigenen Zertifizierungsstelle und die Verwendung eigener Zertifikate juristisch abgesichert? Wenn Sie durch die Verwendung der Zertifikate Rechtsverbindlichkeit erreichen wollen, unterliegen Sie dem Signaturgesetz.
- Zum Betrieb einer eigenen Zertifizierungsstelle benötigen Sie qualifiziertes Personal.
- Die technische und organisatorische Infrastruktur eines Trust-Centers muss sicher und verfügbar betrieben werden. Wenn Sie beispielsweise die sichere Verwahrung Ihrer Zertifikate nicht gewährleisten können, sollten Sie auf den Betrieb eines eigenen Trust-Centers verzichten.
- Der Betrieb eines eigenen Trust-Centers verursacht durch administrative und organisatorische Prozesse erhebliche Kosten. Stellen Sie diese den Kosten für eingekaufte Dienstleistung gegenüber.
- Sowohl fachliche als auch technische Gründe können für den Betrieb einer eigenen Zertifizierungsstelle sprechen. Ist beispielsweise die Hinterlegung von Schlüsseln gefordert, lässt sich dies mit einer akkreditierten Zertifizierungsstelle nicht realisieren.

Eine Frage der Ebene

Bei der Implementierung von Sicherheitsmaßnahmen sollten Sie zuerst festlegen, auf welcher Ebene des OSI-Schichtenmodells die Realisierung erfolgt:

- Netzwerkebene (OSI-Schicht 3). Eine Implementierung auf dieser Ebene hat keinen Einfluss auf ein zu entwickelndes System. Die Sicherheitsprotokolle dieser Ebene werden häufig für die Realisierung sogenannter *Virtual Private Networks (VPN)* genutzt. Innerhalb dieser Protokolle (etwa *IPSEC*) erfolgen Verschlüsselung, Sicherstellung der Datenintegrität und Authentisierung der Endsysteme. Die Mechanismen werden in diesem Fall auf den gesamten Datenverkehr zwischen den beteiligten Endsystemen angewandt. Für Softwaresysteme ist die Verwendung der Sicherheitsmechanismen auf der Netzwerkebene damit völlig transparent. Eine selektive Anwendung auf einzelne Anwendungen oberhalb der Netzwerkebene ist nicht möglich. Den Zeitpunkt für Entschlüsselung und Prüfung der Datenintegrität können Anwendungen ebenfalls nicht bestimmen. Übertragene Daten werden unmittelbar nach dem Eingang beim Empfänger entschlüsselt und geprüft.
- Transportebene (OSI-Schicht 4). Auch für diese Variante existieren standardisierte Protokolle wie *Secure Socket Layer (SSL)* oder *Transport Layer Security (TLS)*. Bei Einsatz dieser Protokolle kann jede Anwendung eigenständig über deren Verwendung entscheiden. Die Mechanismen beziehen nicht mehr das gesamte Endsystem ein, wie bei der Sicherung auf Netzwerkebene. Generell werden aber alle über eine solche Transportstrecke übertragenen Daten gesichert. Die Anwendung hat keinen Einfluss auf den Zeitpunkt von Entschlüsselung und Integritätsprüfung. Beides erfolgt unmittelbar nachdem die Daten den Transportkanal verlassen haben. Für eine zu realisierende Anwendung bedeutet die Verwendung solcher Mechanismen nur einen geringen Zusatzaufwand. Lediglich beim Aufbau neuer Verbindungen ergeben sich Änderungen, beispielsweise durch die Behandlung von Zertifikaten. Die Abwicklung der eigentlichen Kommunikation unterscheidet sich nicht von der ungesicherten Kommunikation.
- Anwendungsebene (OSI-Schicht 7). Diese Art der Implementierung gibt dem System maximale Kontrolle über die eingesetzten Sicherheitsmechanismen. Es bedeutet allerdings auch, dass die Verwendung der Mechanismen in einem System explizit programmiert werden muss. Transparenz oder Austauschbarkeit bezüglich der Verwendung von Sicherheitsmechanismen geht somit verloren. Klassisches Beispiel für dieses Verfahren ist die Verschlüsselung von E-Mails gemäß *S/MIME*.

In der Praxis hängt die Entscheidung für ein spezifisches Verfahren von den funktionalen und organisatorischen Anforderungen an das zu realisierende Softwaresystem ab. Einige Faustregeln:



- Für Anwendungen im Kontext des Signaturgesetzes ist eine Sicherung auf Transportebene nicht ausreichend, da einige gesetzliche Forderungen (etwa die Nachvollziehbarkeit einer Signaturprüfung) nicht erfüllt sind.
- Wenn Sie Daten auch nach dem Transport sichern wollen, müssen Sie Sicherungsmaßnahmen auf Applikationsebene einsetzen.
- Genügt Ihnen die Sicherung der Daten während der Übertragung, sollten Sie Sicherheitsmaßnahmen auf Transportebene wählen.

- Müssen Sie Sicherheitsmaßnahmen in bestehende Anwendungen integrieren, können Sie dies auf der Transportebene mit geringerem Aufwand erreichen. ■

7.8.4 Security Engineering mit Patterns

Die hier vorgestellten kryptografischen Verfahren können, wenn sie richtig eingesetzt werden, einen wichtigen Teil zur Entwicklung von sicheren Systemen beitragen. Es sind allerdings keine „Zaubermittel“, die einzusetzenden Verfahren müssen sorgfältig ausgewählt und umgesetzt werden. Selbst bei dem weltweit eingesetzten AES-Verfahren gibt es Betriebsmodi, die als unsicher angesehen werden. Da sich die meisten Softwareentwickler oder -architekten nur selten mit Sicherheit beschäftigen, fehlt ihnen das nötige Spezialwissen.

Eine Lösung hierfür bieten die 1997 von Yoder und Baralow vorgestellten und von Schumacher und Rödiger weiterentwickelten „Security Patterns“ ([Schumacher06]). Analog zu den aus anderen Bereichen bekannten Mustern wird auch hier das Wissen von Experten gesammelt und strukturiert wiedergegeben. Ein Muster beschreibt dabei ein in einem bestimmten Kontext immer wieder auftretendes Problem und bietet eine Lösung unter Einbeziehung aller zu beachtenden Aspekte. Die von Schumacher et al. eingeführten Security Patterns bestehen aus den folgenden Elementen:

- **Kontext:** Anhand eines Beispielszenarios werden die Umstände beschrieben, unter denen das zu lösende Problem auftreten kann. Zusätzlich wird, sofern relevant, auf bereits angewendete Security Patterns verwiesen.
- **Problem:** Unter Einbeziehung aller funktionalen und nichtfunktionalen Anforderungen an das System wird hier das Problem beschrieben, welches durch die Anwendung des Patterns gelöst werden soll.
- **Lösung:** Die Lösung des vorgestellten Problems im Rahmen des Kontexts unter Einbeziehung aller Anforderungen. Dabei kann die Lösung auf unterschiedlichen Ebenen umgesetzt werden und beinhaltet gegebenenfalls Gegenmaßnahmen, um die Auswirkungen bestimmter Angriffe oder Bedrohungen zu minimieren.
- **Auswirkungen:** Hier werden die Vor- und Nachteile der vorgestellten Lösung diskutiert.

Im Folgenden wird anhand eines Beispiels versucht, dem Leser die relevanten Punkte zu verdeutlichen. Das hier ausgewählte Muster ist nur eines von vielen, die in [Schumacher06] ausführlich beschrieben werden.

Security Patterns in der Praxis: Das „Security Session“-Pattern

- **Kontext:** In einem Mehrbenutzersystem müssen die einzelnen Komponenten in der Lage sein, die mit einzelnen Benutzern assoziierten Berechtigungen zu überprüfen.
- **Problem:** In komplexen Systemen kann der Nutzer an vielen verschiedenen Stellen unterschiedliche Aktionen durchführen. Die Berechtigungsprüfung für jede einzelne Aktion durch Eingabe von Benutzername und Passwort ist nicht praktikabel, eine Speicherung und Weitergabe der Credentials durch die einzelnen Systeme darf nicht erfolgen.

- Lösung: Es wird ein Session-Objekt eingeführt, das alle für die Berechtigungsprüfungen relevanten Benutzerdaten enthält. Jede vom Benutzer durchgeführte Aktion wird anhand des in der Anfrage enthaltenen Session-Objekts überprüft. Ein bekanntes Beispiel aus dem Bereich der Webanwendungen sind Cookies.
- Auswirkungen:
 - Die Bindung von Session-Objekten an Benutzer ermöglicht es, die gleichzeitige Nutzung von Credentials zu erkennen, was auf eine Kompromittierung des Benutzeraccounts hindeuten kann.
 - Die Session-Objekte sind ein attraktives Ziel für Angreifer und müssen daher auch geschützt werden, z. B. durch Verschlüsselung oder Signaturen.

7.8.5 Weiterführende Literatur



Microsoft (Online: <https://www.microsoft.com/security/sdl/resources/publications.aspx>) beschreibt seinen eigenen „Security Development Lifecycle“: Für die einzelnen Phasen der Softwareentwicklung werden Security Practices vorgestellt, mit denen Sicherheitslücken verhindert, entdeckt und behoben werden können:

Das *Open Web Application Security Project* (<https://www.owasp.org>) bietet einen aktuellen Überblick über die relevanten Sicherheitsaspekte im Bereich von Webanwendungen.

[Schneier96] ist das Standardwerk zum Thema Kryptografie.

[Schumacher06] erläutert die Konzepte der Security Patterns und enthält eine ausführliche Liste von in unterschiedlichen Situationen anwendbaren Mustern.

Wolfgang Korn (wolfgang.korn@gmx.de) hat mehrere Jahre als Berater für IT-Sicherheit und Softwarearchitekturen gearbeitet. Zurzeit arbeitet er als Softwarearchitekt für die blueCarat AG in Köln. Seine Mission besteht darin, seine Familie von den Vorzügen mechanischer Klaviere zu überzeugen und für seine Kunden sichere Architekturen zu entwickeln.

Tobias Hahn arbeitet als IT-Security Spezialist bei Lufthansa. Vorher hat er sich einige Jahre im Rahmen von Forschungsprojekten des Fraunhofer-Instituts SIT mit verschiedenen Aspekten der IT-Sicherheit beschäftigt.

■ 7.9 Protokollierung

Protokollierung: Sammeln von Informationen über den Programmzustand während des Programmablaufs.

Es gibt zwei Ausprägungen der Protokollierung, das „Logging“ und das „Tracing“. Bei beiden werden Funktions- oder Methodenaufrufe in das Programm aufgenommen, die zur Laufzeit über den Status des Programms Auskunft geben.

In der Praxis gibt es zwischen Logging und Tracing allerdings sehr wohl Unterschiede:

- Logging kann fachliche oder technische Protokollierung sein, oder eine beliebige Kombination von beidem.
 - Fachliche Protokolle werden gewöhnlich anwenderspezifisch aufbereitet und übersetzt. Sie dienen Endbenutzern, Administratoren oder Betreibern von Softwaresystemen und liefern Informationen über die vom Programm abgewickelten Geschäftsprozesse.
 - Technische Protokolle sind Informationen für Betreiber oder Entwickler. Sie dienen der Fehlersuche sowie der Systemoptimierung.
- Tracing soll *Debugging*-Information für Entwickler oder Supportmitarbeiter liefern. Es dient primär zur Fehlersuche und -analyse.

7.9.1 Typische Probleme

Hohe Abhängigkeit von der Protokollierung

Ein Problem einfacher *ad-hoc*-Lösungen der Protokollierung ist die starke Abhängigkeit nahezu sämtlicher Komponenten von der Protokollierung. In fast allen anderen Komponenten wird von der Protokollierung explizit Gebrauch gemacht, indem Aufrufe der Form

```
writeLogEntry( message, priority, category );
```

enthalten sind. Der Empfänger solcher Nachrichten (oder Funktionsaufrufe) muss daher in jeder Komponente bekannt sein. Spätere Erweiterungen der Protokollierung, insbesondere ihrer Schnittstelle, ist mit Quellcode-Anpassungen an vielen Stellen verbunden.

Von daher ist es im Rahmen der präventiven Qualitätssicherung angebracht, vorhandene Bibliotheken zur Protokollierung heranzuziehen, die flexible Anpassung an erweiterte Nutzungsbedürfnisse ohne Codeänderungen erlauben. Hierzu gehören die Konfiguration zur Laufzeit (Ein- und Ausschalten der Protokollierung, Umlenkung auf andere Ausgabekanäle) und die Definition neuer Kategorien, Prioritäten oder Ausgabekanäle.

Hohe Flexibilität gefordert

Die für die Protokollierung verantwortliche Softwarekomponente, nachfolgend kurz *Logger* genannt, benötigt Flexibilität in folgenden Aspekten:

- Welche Ereignisse sollen protokolliert werden? Hierzu werden Ereignisse in Kategorien eingeteilt. Der Logger entscheidet zur Laufzeit darüber, ob ein auftretendes Ereignis protokolliert wird oder nicht.
- Wie wichtig oder gravierend sind Ereignisse einer bestimmten Kategorie? Dazu können Kategorien mit Prioritäten versehen werden, etwa DEBUG, INFO, WARNING, ERROR und FATAL.
- Wohin sollen Ereignisse protokolliert werden? Je nach Anforderung können dazu beliebige Ausgabekanäle sinnvoll oder notwendig sein: Neben den konventionellen Logdateien können Ereignisse an Betriebssystemdämonen⁵² oder Warteschlangen geschickt werden, oder sogar über Mobilfunk (SMS) oder E-Mail direkt an die verantwortlichen Personen.
- Wie sollen Ereignisse einer bestimmten Kategorie formatiert werden? Für bestimmte Ereigniskategorien mag eine Übersetzung in die Landessprache der Endanwender sinnvoll sein. Benötigt das System die Nachrichten als Text, HTML, XML oder in anderen Formaten?

Protokollierung in heterogenen Systemen

Protokollierung über Programmiersprachen- und Betriebssystemgrenzen ist komplex. Es gibt einige technische Probleme (etwa: Netzbelastung, Parameterübergabe zwischen unterschiedlichen Programmiersprachen).

Bei getrennter Protokollierung auf Clients und Server ist die Konsolidierung von Client- und Serverprotokollen schwierig.

7.9.2 Lösungskonzept

Die Apache Foundation hat den *Logging Framework for Java* (log4j) entwickelt, eine freie Bibliothek zur Protokollierung in Java.⁵³ Die Konzepte dieser Bibliothek erfüllen die oben genannten Anforderungen an Logging und Tracing. Die Konzepte dieses Ansatzes sind leicht auf andere Sprachen übertragbar.

⁵² Etwa den Unix `syslogd` oder entsprechende Mechanismen unter anderen Betriebssystemen.

⁵³ Mittlerweile gibt es auch Portierungen für C++ und Python.

Einige Tipps zur Protokollierung:



- Nutzen Sie möglichst ein bestehendes Framework. Dieses Rad müssen Sie nicht neu erfinden!
- Protokollieren Sie sämtliche Interaktionen der Benutzeroberfläche, weil damit der „Gang“ des Benutzers durch die Applikation nachvollziehbar wird.
- Protokollieren Sie zusätzlich alle Interaktionen zwischen Subsystemen oder Komponenten.
- Nutzen Sie die Protokollierung zur Fehlersuche und -analyse. Durch Protokollaufrufe instrumentierter Code erleichtert die Fehlersuche.
- Beachten Sie die entstehende Netzbelastung, wenn Sie in verteilten Systemen Protokolle schreiben. Lösungsansatz: Getrennte Protokolle auf den einzelnen Plattformen. Nachteilig hierbei ist, dass der Zugriff auf die Client-Protokolle eventuell (rein organisatorisch) schwer sein kann, etwa wenn die Clients im Internet verteilt sind.
- Wenn die kontinuierliche Fortschreibung einer Protokolldatei aus Platz- oder Performance-Gründen nicht möglich ist, können Sie Ringpuffer als Alternative einsetzen. Dabei steht eine feste Anzahl von Einträgen im Protokoll zur Verfügung, die zyklisch überschrieben werden. Siehe dazu auch [Marick2000].

7.9.3 Zusammenhang mit anderen Aspekten

- Sicherheit: Die korrekte Behandlung sensibler Daten in Protokolldateien muss gewährleistet werden.
- Verteilung: Auf welchem physischen Knoten läuft der Logger? Gibt es eine oder mehrere Instanzen?
- Kommunikation: Logging in verteilten Systemen erzeugt Netzlast.

7.9.4 Weiterführende Literatur

Einige Open-Source-Projekte erstellen Bibliotheken für die Protokollierung. Zu den bekannten gehört der log4j Framework des Apache-Projekts, zu finden unter www.apache.org. Einen Wrapper zur Kapselung unterschiedlicher Logging-Frameworks bietet Apache im Commons-Projekt. Ab Version 1.4 enthält Java eine Standard-API für Logging.

[Gupta2003] diskutiert Logging mit Java und log4j ausführlich, mit vielen Benutzungshinweisen.

[Marick2000] beschreibt einige Muster von Logging mit Ringpuffern, die sich zum dezentralen Einsatz eignen, also etwa auf Arbeitsplatzrechnern mit eingeschränktem Speicherplatz.



■ 7.10 Ausnahme- und Fehlerbehandlung

Errors are serious problems.

Tim McCune, aus „Exception Handling Antipatterns“

7.10.1 Motivation

Kennen Sie Murphys Regel? Es geht immer etwas schief. Und das grundsätzlich im unpassenden Augenblick. Reifen platzen meistens im strömenden Regen und Sie bekommen genau dann Zahnschmerzen, wenn Ihre Lieblingszahnärztin gerade ihren wohlverdienten Urlaub genießt.

Murphys Regel

Software stürzt bevorzugt während wichtiger Präsentationen ab – und dann begräbt sie gleich noch diverse andere Programme mit im virtuellen Nirwana.⁵⁴

Demgegenüber erwarten Anwender und Administratoren zu Recht, dass Software in Fehler- und Ausnahmesituationen:

- vorhersehbar und angemessen reagiert und Anwender nicht im „Regen“ stehen lässt;
- möglichst keine Auswirkungen auf andere Programme hat, insbesondere BSOD⁵⁵ und ähnliche Effekte vermeidet;
- die Diagnose des Fehlers erleichtert.

Ausnahme- und Fehlerbehandlung dreht sich um folgende Teilaufgaben:

- Ausnahmen und Fehler erkennen. In diesem Fall sollten Sie die folgenden drei Fragen⁵⁶ beantworten können:
 1. Was ging schief?
 2. Wo ist es passiert?
 3. Warum ging es schief?
- Auf Fehler angemessen reagieren. Bei Bedarf Benutzer oder Administratoren benachrichtigen und passende korrektive Maßnahmen ergreifen.
- Fehlersituationen vorzeitig erkennen und präventiv reagieren. Für *fehlertolerante* Systeme wichtig – beispielsweise den freien Speicherplatz kontinuierlich messen und bei Speicherknappheit gründlich aufräumen.

⁵⁴ Kenner von Wahrscheinlichkeitsrechnung und Stochastik mögen schmunzeln. Ich zumindest kenne Murphy zur Genüge.

⁵⁵ Blue Screen Of Death. Frühere Versionen eines bekannten (und mittlerweile sehr robusten) Betriebssystems zeigten BSOD im Falle fataler Fehler.

⁵⁶ Nach Jim Cushing, <http://today.java.net/pub/a/today/2003/12/04/exceptions.html>

Fehlerbehandlung ist kein (reines) Programmierproblem

Immer wieder stoße ich in Projekten auf die Meinung „Fehlerbehandlung übernimmt doch schon die Programmiersprache“ – zumindest bei solchen Entwicklern, deren favorisierte Programmiersprache ein Exception-Konzept mitbringt.

Aus meiner Sicht weit gefehlt: Vor der Realisierung irgendeiner Ausnahme- oder Fehlerbehandlung steht der grundlegende und strukturelle Entwurf eines entsprechenden Konzeptes, das folgende Fragen klärt:

- Welche Ausnahmen und Fehlerkategorien gibt es überhaupt?
 - Welche externen Ereignisse können zu Ausnahmen oder Fehlern im Programm führen? Beispiele: Falsche oder fehlende Eingabedaten, fehlerhaft bediente Eingabeschnittstellen, Fehler in der genutzten Infrastruktur. Wie erkennen Sie solche Situationen?
 - Welche internen Zustände gelten als Ausnahme oder Fehler? Wie erkennen Sie diese?
- Auf welche dieser Kategorien soll Ihr System in welcher Form reagieren?
 - Gibt es Ausnahmen oder Fehler, die das System selbstständig oder mit Nachfragen korrigieren muss?
 - Welche Fehlersituationen muss das Programm an welche Adressaten melden? Diese Frage steht im Zusammenhang mit dem Aspekt „Protokollierung und Logging“ (siehe Abschnitt 7.11).
- Wie können Sie die Behandlung von Ausnahmen und Fehlern angemessen (d. h. mit einem zur Domäne passenden Verhältnis aus Testaufwand und Testabdeckung) testen? Dazu müssen Sie Ausnahmen und Fehler reproduzierbar herstellen, was oft ein erhebliches Problem darstellt.

Allerdings läuft es nach diesen konzeptionellen Aufgaben auf eine gründliche und einheitliche Programmierung hinaus – was Programmiersprachen mit Exception-Konstrukten erheblich erleichtern.

Fehler sind Abweichungen von Annahmen

Ein *Fehler* liegt vor, wenn ein Unterschied zwischen einem erwarteten und einem konkret vorliegenden Ergebnis oder Zustand entsteht. Beispiele:

- Ihr System erwartet an einer Eingabeschnittstelle XML, erhält aber pdf.
- Klainer Fähler. Sie erwarten (berechtigterweise) die korrekte Schreibweise.
- Ihr System erwartet Eingabedaten über eine Web-Service-Schnittstelle, erhält dort aber keine (sondern Sie erhalten per Post eine CD).
- Ihr System sucht aus einer Datenbank Bestelldaten anhand von Kundennummern heraus. Während der Suche fährt der Operator die Datenbank herunter.
- Ihr Webserver erhält zu viele Anfragen und kann nicht mehr schnell genug antworten.
- Ein Baustein Ihres Systems empfängt ein Ereignis, auf das er nicht vorbereitet ist (etwa: ein Interrupt-Signal).
- Ein Baustein Ihres Systems kommt während der Verarbeitung in einen inkonsistenten Zustand, führt eine Division-durch-Null aus, arbeitet auf einem nicht initialisierten Objekt, überschreitet die maximale Puffergröße, erzeugt einen Speicherüberlauf oder benimmt sich auf andere Weise daneben.

Sie sehen – die Fantasie des Fehlerteufels kennt kaum Grenzen.

Als *Ausnahme* bezeichne ich eine Situation, in der ein Programmbaustein aufgrund eines *Fehlers* seinen „normalen“ Ablauf nicht fortführen kann.⁵⁷

Architekten kümmern sich um Laufzeitfehler

Uns Softwarearchitekten interessieren hauptsächlich Fehler, die zur Laufzeit von Programmen auftreten. Compiler tragen zur Vermeidung mancher Fehler bei. Leider verschwimmt jedoch in manchen Fällen die Grenze zwischen Compile- und Laufzeit:

- In interpretierten Sprachen findet eventuell keine Syntaxprüfung zur Compilezeit statt (etwa in Shellskripten, Ruby oder Perl).
- Manche (compilierten) Systeme enthalten Bestandteile, die erst zur Laufzeit interpretiert werden, beispielsweise Metadaten, Konfigurationsdaten oder interpretierte Geschäftsregeln. Das sind beliebte Mittel, um die Flexibilität von Systemen zu erhöhen und gleichzeitig die vom Compiler gewährte Sicherheit zu reduzieren!

Sie erkennen jetzt, dass wir als Softwarearchitekten auch *falsch geschriebene* Worte ins Kalkül unserer Fehlerbehandlung aufnehmen müssen, zumindest dann, wenn uns die Compiler mal wieder im Stich lassen. Die Theorie „Der Compiler wird’s schon finden“ ist eben nur eine Theorie.

7.10.2 Fehlerkategorien schaffen Klarheit

Meine vordringliche Empfehlung bezüglich Ausnahme- und Fehlerbehandlung lautet, die Arten oder Kategorien der möglichen Fehler möglichst umfassend zu ermitteln. Im Idealfall können Sie dazu auf die Vorarbeit Ihrer Systemanalytiker zurückgreifen, in der Realität bleibt diese Arbeit komplett an Ihnen hängen. Anschließend definieren Sie, wie Ihr System mit diesen Kategorien umgehen soll, und erstellen daraus konkrete Vorgaben zur Implementierung der Fehlerbehandlung.

Ich schlage Ihnen mehrere Kategorisierungen vor, ohne Anspruch auf Vollständigkeit. Mithilfe dieser Kategorien können Sie einfacher beschreiben, ob und wie Ihr System in solchen Fällen reagieren soll.

Sie werden merken, dass ein einzelner Fehler durchaus zu mehreren Kategorien gehören kann.

Fachliche und technische Fehler

Fachliche Fehler resultieren aus Problemen in den fachlichen Abläufen oder entsprechenden Daten von Systemen. Beispiel: „Kein Vertrag zu diesem Kunden vorhanden.“ Solche Fehler können Sie mit genügend Verständnis der Fachdomäne recht gut charakterisieren. Insbesondere sollte Ihr System zu *erwarteten* fachlichen Fehlern eine für Anwender plausible Abhilfe anbieten.

⁵⁷ In manchen (Java-)Programmen habe ich Exceptions gefunden, die völlig normale („fachliche“) Ereignisse abbildeten. Ich persönlich halte diese Interpretation von Ausnahmen für schlecht, weil damit der „Fehlercharakter“ von Ausnahmen verschwimmt.

Demgegenüber resultieren technische Fehler oft aus Problemen der zugrunde liegenden Infrastruktur („Datenbankverbindung verloren“) oder reinen Programmierfehlern („Division durch null“ oder NPE, Null Point Exception). Sie können jedoch auch in fachlichen Abläufen oder Bausteinen auftreten.



Technische Fehler sollten Sie nicht in fachlichen Bausteinen behandeln – das bläht deren Code unnötig auf. Dennoch müssen Sie auf technische Fehler in fachlichen Bausteinen reagieren!

Syntax-, Format- oder Protokollfehler

Daten oder übermittelte Dokumente können syntaktische Fehler aufweisen und für Ihr System dann unbrauchbar sein (etwa wenn ein lieferndes System die Syntax der übertragenen Auftragsnummern ohne Absprache geändert hat ...). Syntaxfehler kann Ihr System durch *Parsen* feststellen.

Syntaxfehler: parsen

Trotz korrekter Syntax können Daten für Ihr System unverständlich sein, beispielsweise den vereinbarten Wertebereich überschreiten oder ungültige Werte für Datenbankschlüssel darstellen. Solche Formatfehler (oder semantische Fehler) kann Ihr System nur durch *Interpretieren* diagnostizieren. Sie führen oft zu semantischen Problemen, sofern sie nicht frühzeitig erkannt werden. In diese Kategorie fallen Verletzungen von Kann/Muss-Bedingungen oder die sogenannten „Plausibilisierungsfehler“ an Benutzeroberflächen.

Formatfehler: interpretieren



Gegen Syntax- und Formatfehler können Sie sich durch klar definierte Schnittstellenverträge und entsprechende Prüfungen (assertions) an Ihren Eingangsschnittstellen wappnen. Zusätzlich können Unit-Tests überprüfen, ob Ihre Bausteine selbst solche Fehler verursachen.

Nehmen wir jetzt an, das datenliefernde Fremdsystem hat syntaktisch korrekte Daten in den richtigen Formaten geliefert, leider jedoch zum falschen Zeitpunkt an den falschen Ort oder über einen falschen Kanal. Dann haben wir es mit einem *Protokollfehler* zu tun. Einer der Partner hat das vereinbarte *Protokoll* einer Schnittstelle verletzt und dadurch einen Fehler verursacht. Siehe dazu auch „Exakte Verträge“ in Abschnitt 7.10.3.



Meiner Erfahrung nach wird die Kategorie „Protokollfehler“ häufig vergessen! Sie fallen häufig erst im operativen Betrieb auf und lassen sich oftmals nur manuell erkennen und reparieren.

Korrigierbare und unkorrigierbare Fehler

Eine fehlerhafte manuelle Eingabe kann Ihr System mit einer entsprechenden Meldung beim Benutzer reklamieren und eine Korrektur einfordern. Diese Kategorie von Fehlern nenne ich *korrigierbar*.

Bei Schnittstellen von und zu Fremdsystemen ist eine Korrektur oftmals schwieriger, weil sie nicht interaktiv eingefordert werden kann (es sei denn, die entsprechende Schnittstelle sieht eine solche Korrektur vor!). Viele technische Fehler sind unkorrigierbar: Ist eine notwendige Datenbankverbindung nicht vorhanden, kann Ihr System nicht speichern – eben unkorrigierbar.⁵⁸

Solche unkorrigierbaren Fehler kann Ihr System nicht selbstständig beheben. Sie werden zu (fachlichen) *Clearing*-Fällen oder aber sorgen für den Abbruch der jeweiligen Aktion oder eines Prozesses. Möglicherweise muss Ihr System im Falle unkorrigierbarer Fehler jedoch noch „Aufräumarbeiten“ in Form von Rollbacks oder Kompensationen durchführen, um das Gesamtsystem wieder in einen konsistenten Zustand zu bringen.

Tipps zur Ermittlung von Fehlerkategorien



- Stimmen Sie Ihre Ausnahme- und Fehlerkategorien mit erfahrenen Testern oder Ihrer Qualitätssicherung ab. Den QS-Experten fallen garantiert noch diverse Fehler ein, an die Sie (oder Ihre Entwickler) bisher nicht gedacht haben.
- Achten Sie insbesondere auf Schnittstellen: An Stellen, wo Daten Systemgrenzen überschreiten, entstehen oftmals *semantische Lücken*. Sie dürfen auch *Problemzonen* dazu sagen.
- Besonders kritisch sind die Schnittstellen zu Fremdsystemen, also die externen Schnittstellen Ihres Systems.

7.10.3 Muster zur Fehlerbehandlung

Exakte Verträge mit externen Systemen

Exakte Absprachen

Treffen Sie mit externen Systemen möglichst exakte Absprachen hinsichtlich der auszutauschenden Daten. Beachten Sie dabei neben der reinen Syntax und Semantik der ausgetauschten Daten auch Formate (Zeichen- und Zahlencodierung, Schlüsselwerte etc.), Muss-/Kann-Belegungen, Wertebereiche, Ausnahme- oder Sonderfälle, Grenzfälle (Nullwerte, leere Dateien, überlange Zeichenketten etc.), Übertragungskanäle und -protokolle, Häufigkeit des Datenaustausches, eventuelle Berechtigungs- oder Sicherheitsaspekte, Transaktions- und Zustandsbehandlung, eventuell notwendige Transformation, Synchronität, Statusverfolgung und andere. Kann das liefernde System bereits gelieferte Daten nachträglich für ungültig erklären und durch eine neue Version ersetzen?

Sie sehen – solche Details können Schnittstellenverträge deutlich aufblähen, meiner Erfahrung nach ist jedoch nur eine solche systemübergreifende Fehlerbehandlung realistisch.

⁵⁸ Aber: Mit entsprechendem Aufwand könnten Sie auch solche Fehler korrigieren – sofern das für Ihre Auftraggeber und Stakeholder wichtig genug ist.



Mein persönlicher Tipp: Skizzieren Sie jede (wirklich jede einzelne) externe Schnittstelle aus Laufzeitsicht mit ihrem kompletten *Protokoll*: Wer stößt wie eine Übertragung vom/ins System an? Gibt es nur Nettodaten oder auch eine Quittung? Zeichnen Sie ein paar Sequenzdiagramme für den Sonnenschein- und einige Fehlerfälle.

Versteckte technische Fehler vor Anwendern

Meldungen zu technischen Fehlern sind meist nur für Administratoren oder Betreiber relevant und fast nie für Endanwender von Softwaresystemen.⁵⁹

Stellen Sie zur Behandlung technischer Fehler eine einheitliche Schnittstelle bereit, die folgende Dinge leistet:

- Technische Fehler gehören praktisch immer in das zentrale Logfile, wo sie den Administratoren oder Entwicklern bei der Fehlerdiagnose helfen. Hier sollten Sie sämtliche relevanten Details protokollieren.
- Parallel dazu zeigen Sie Endanwendern eine benutzerfreundliche und allgemein verständliche Meldung. Erläutern Sie, dass etwas Gravierendes geschehen ist, das aber nichts mit den Aktionen der Anwender zu tun hat. Fügen Sie dieser Meldung eine eindeutige Fehlerkennung hinzu, auf die sich Anwender gegenüber Administratoren berufen können.
- Sofern angemessen, sollte eine automatisierte Meldung an den Support (Administrator, Helpdesk, Callcenter oder Ähnliches) versandt werden.

Optimiere Diagnostizierbarkeit

Sie haben sicherlich schon erlebt, dass die Suche nach einem Fehler tagelang dauerte und dessen Behebung nur wenige Minuten. Daher sollte ein übergreifendes Entwurfs- und Implementierungsziel sein, Fehler und deren Ursachen möglichst einfach und schnell diagnostizieren zu können.

Dafür benötigen Sie einerseits ein einheitliches Konzept zur Ausnahme- und Fehlerbehandlung, andererseits eine Menge Disziplin und Energie bei dessen Umsetzung und Überprüfung! Prüfen Sie möglichst viele dieser Vorgaben automatisiert durch Codeanalyse.

Siehe auch die Hinweise zu „Original- und Folgefehler“ im nachfolgenden Abschnitt.

7.10.4 Mögliche Probleme

Verteilte Verarbeitung

Falls ein Fehler in einem entfernten („remote“) Baustein auftritt, müssen Sie weitere Informationen zu diesem Fehler eventuell erst über ein Netzwerk übermitteln. Das kann in manchen Fällen zu lange dauern (oder aufgrund technischer Fehler nicht funktionieren). Dann werden die Protokollierung oder das Logging solcher Fehler problematisch. [Longshaw+04] gibt den (guten!) Rat: „Log at Distribution Boundary“ (Protokolliere den Fehler dort, wo er auftritt).

⁵⁹ Schlimmer noch: Da Endanwender solche Meldungen häufig nicht verstehen, sinkt möglicherweise ihr Vertrauen in das System.

Leite über Verteilungsgrenzen lediglich allgemeine Fehlermeldungen weiter, und vermeide es, umfangreiche Fehlerprotokolle quer durch verteilte Systeme zu senden.

Dieses Problem taucht bereits bei einfachen Client/Server-Systemen auf. Dort müssen manche Meldungen den Anwender an der Benutzeroberfläche (also im Client) erreichen, andere das zentrale Logfile und manche beide Stellen.

Original- und Folgefehler schwer unterscheidbar

Klingt trivial: Unterscheiden Sie zwischen der Ursache (dem Originalfehler) und seinen Folgefehlern. Nur so behalten Sie den Überblick darüber, in welchen Bausteinen wirkliche Fehler auftreten (und können dann auch das wahre Problem nachhaltig beheben).

Leider spielt uns die Praxis in diesem Falle schon wieder übel mit: Oftmals versteckt sich das ursprüngliche Problem, insbesondere im Falle *unerwarteter* Fehler. Es kann sehr schwierig sein, Originalfehler automatisch zu ermitteln – da hilft oft nur Erfahrung, Geduld und Forensik („Spurensuche“).

In Sprachen mit Exception-Handling tritt dieses Problem oft auf, wenn Exceptions nach ihrem Auftreten „verschluckt“ oder „umgewandelt“ werden. Vermeiden Sie leere Catch-Klauseln oder häufige Umwandlung von Exception-Typen innerhalb des Quellcodes.⁶⁰

Ein Tipp aus [Longshaw+04]: Geben Sie jedem Fehler ein eindeutiges Fehlerkennzeichen (*unique ID*). Das kann in verteilten Logfiles die Ursachenforschung stark vereinfachen.

7.10.5 Zusammenhang mit anderen Aspekten

Protokollierung und Logging

- Geben Sie als Softwarearchitekt dem Entwicklungsteam klare Vorgaben hinsichtlich des Loggings von Ausnahmen und Fehlern: Alle Fehler sollten in einem Logfile landen, möglicherweise sogar in mehreren (eine ausführliche Meldung in einem lokalen Logfile, eine verkürzte Version in einem zentralen). Es muss nach einem Fehler immer möglich sein, Ursache, Ort und Auswirkung von Fehlern exakt zu bestimmen! *Log at distribution boundaries*.
- Ihre Fehlerkategorien können Entwicklern als Anhaltspunkte für Logging dienen.
- Falls Ihre bevorzugte oder vorgeschriebene Programmiersprache keine Exceptions kennt oder Laufzeitfehler nur schwer erkennen kann: Eventuell können Sie zumindest Teile Ihrer Systeme in Sprachen wie C# oder Java erstellen, die mit technischen und Laufzeitfehlern gut umzugehen wissen (sprich: ausgereifte Exception-Konzepte bereitstellen!).
- Sowohl Logging wie auch Exception-Handling können Sie mit aspektorientierter Entwicklung (*aspect-oriented programming*, AOP) zentralisieren.

⁶⁰ Java-Projekten empfehle ich dafür Tools wie SonarQube (<http://www.sonarqube.org/>), PMD (pmd.sourceforge.net), CheckStyle (checkstyle.sourceforge.net) oder ähnliche Codeanalyse-Werkzeuge, um solche typischen Probleme bereits im Build zu identifizieren!

Internationalisierung

Falls Ihre Software in mehreren Ländern zum Einsatz kommt, müssen Sie Meldungen über Ausnahmen und Fehler internationalisieren, d. h. die entsprechenden Meldungstexte in den adäquaten Ländereinstellungen des jeweiligen Benutzers ausgeben. Siehe Abschnitt 7.8.

7.10.6 Weiterführende Literatur

Es sieht ziemlich düster aus mit Literatur zu diesem (wichtigen!) Thema. Sie finden mit etwas Mühe diverse wissenschaftliche Traktate teilweise mit einem mathematisch-statistischen oder auch historischen Fokus (etwa Sammlungen bekannter Softwarefehler, wie z. B. <http://www5.in.tum.de/~huckle/bugse.html>).

Sehr nützlich fand ich die Pattern-Sammlung von Andy Longshaw und Eoin Woods ([Longshaw+04]). Darin zeigen die Autoren sieben zentrale Muster zur Behandlung von Fehlern auf. Zum Thema Ausnahmebehandlung auf Programmiererebene (Exception-Handling) hat Tim McCune einige lesenswerte Antipatterns zusammengefasst, online unter: <http://today.java.net/pub/a/today/2006/04/06/exception-handling-antipatterns.html>

Einige Hinweise zum „ordentlichen“ Exception-Handling (nicht nur) für Java zeigt <http://www.onjava.com/pub/a/onjava/2003/11/19/exceptions.html>. Gunjan Doshi erläutert Kriterien, nach denen Sie eine Schnittstelle mit Exceptions entwerfen sollten.

■ 7.11 Skalierbarkeit

Von Phillip Ghadir

7.11.1 Was bedeutet Skalierbarkeit?

Unter Skalierbarkeit versteht man die Eigenschaft eines Systems, auf Änderungen an Datenmengen, Anzahl an Nutzeranfragen oder Anzahl an Transaktionen pro Zeiteinheit angepasst werden zu können.

Mit linearer Skalierbarkeit wird die Fähigkeit bezeichnet, mit n -fachem Aufwand (für Hardware, Infrastruktur, ...) die n -fache Kapazität erbringen zu können.

Man spricht vom „Hochskalieren“, wenn bei steigendem Bedarf die Leistungsfähigkeit gesteigert werden soll, und vom „Herunterskalieren“, wenn bei sinkendem Bedarf die Leistungsfähigkeit – und damit die Kosten – reduziert werden soll.



7.11.2 Skalierungsstrategien

Viele unterscheiden zwei grundsätzliche Dimensionen der Skalierung: horizontal und vertikal. Dabei bevorzuge ich eine etwas andere Unterteilung mit zwei primären Strategien:

Scale-Up (= Aufrüsten von Verarbeitungseinheiten)

Scale-Out (= Hinzufügen von Verarbeitungseinheiten)

Dabei kann die Scale-Out-Strategie in die bereits genannten Dimensionen horizontal und vertikal unterteilt werden.

Je nach Kontext bietet jede Strategie ihre Vorzüge. In verteilten Systemen ist häufig ein geeigneter Strategiemix sinnvoll, bei dem für die Systembestandteile eine jeweils passende Strategie gewählt wird, um ein Gesamtsystem skalierbar zu machen.

7.11.3 Elastizität

In Zeiten des Cloud-Computing werden Elastizitätsanforderungen bezahlbar, die noch vor vielen Jahren nicht praktikabel umsetzbar gewesen wären. Mit Elastizität verbindet man im Allgemeinen die Fähigkeit, auf wechselnden Bedarf hoch- oder herunterzuskalieren.

Während früher die notwendige Infrastruktur dediziert für eine IT-Gruppe aufgebaut werden musste, gibt es heute – auch dank Breitbandinternet – Infrastrukturdienste, die es innerhalb von Sekunden oder Minuten ermöglichen, neue Infrastrukturkomponenten hinzuzuschalten oder wegzunehmen.

Während man früher beispielsweise gezwungen war, die IT-Kapazität für das Hochsaisongeschäft aufzubauen – und über das gesamte Jahr vorhalten zu müssen – können heute vielfach die nötigen Kapazitäten kurzfristig zu flexiblen Abrechnungsmodellen hinzugebucht werden.

7.11.4 Scale-Up-Strategie

Unter Scale-Up versteht man das Aufrüsten einer Maschine. Das kann mehr RAM, mehr Plattenplatz, schnellere Peripherie, mehr CPU-/GPU-Power oder Ähnliches bedeuten. Der IT-Leiter eines Kunden sagte gern: „Wenn ich das Problem mit Eisen erschlagen kann, tue ich das.“

Die Vorzüge sind häufig klar: Das Risiko ist tendenziell überschaubar, der Leistungszuwachs kann in Grenzen bei bekannter Softwarearchitektur gut prognostiziert werden. Es gibt Kataloge und Hersteller, die bezüglich der Optionen für diese Art der Problemlösung die Auswirkungen direkt vergleichbar machen. Eine Kosten-Nutzen-Abwägung fällt somit leicht.

Scale-Up kann man sich auch durch Optimierung in der Betriebsinfrastruktur vorstellen: Beispielsweise kann durch einen Betriebssystemwechsel die Anzahl der offenen File-Handles oder die maximale Anzahl von Threads erhöht werden, was unter Umständen ebenfalls die Leistungsfähigkeit eines verarbeitenden Knotens steigern kann.

7.11.5 Vertikale Scale-Out-Strategie

Manchmal erreicht man beim Skalieren über das Aufrüsten einer Maschine (Scale-Up) eine technische oder wirtschaftliche Grenze, so dass wir über sinnvolle Alternativen nachdenken müssen.

Eine Alternative ist das physische Aufteilen des Systems auf mehrere Schichten und deren Zuordnung zu verschiedenen Knoten. Anfragen werden dann über mehrere Schichten hinweg verarbeitet. Beispielsweise könnte man die Interaktionsschicht herausziehen, um die Masse der Anfragen anders zu bedienen, als es für die tatsächliche Verarbeitung komplexerer Transaktionen im Backend nötig ist.

Durch das Verteilen der Verarbeitung einzelner Anfragen auf mehrere Verarbeitungseinheiten erhält man für jede einzelne nun wiederum die Möglichkeit, gesondert über die konkrete Skalierungsstrategie nachzudenken.

7.11.6 Horizontale Scale-Out-Strategie

Beim horizontalen Skalieren stellt man verschiedene – üblicherweise identische – Verarbeitungseinheiten parallel nebeneinander, um diese mit den gleichen Aufgaben zu betrauen. Anfragen werden in dem Fall nach Verteilungsstrategien wie z. B. Round-Robin, nach geografischen Aspekten oder aufgrund von Berechtigungen auf die beteiligten Knoten verteilt.

Dieser Ansatz erlaubt es im Vergleich zu den anderen Ansätzen tendenziell grenzenlos im wirtschaftlichen und geografischen Sinne zu skalieren. Die sprungfixen Kosten für diesen Ansatz sind erfahrungsgemäß überschaubarer als in den beiden zuerst genannten Varianten.

Insbesondere für das Erreichen einer gewissen Elastizität ist die horizontale Scale-Out-Strategie vielversprechend.

Abhängig von der konkreten Ausprägung der horizontalen Scale-Out-Strategie kann eine Investition in zusätzliche vorher vernachlässigbare Aspekte erforderlich werden. Sei dies durch eine stärkere Abhängigkeit zum DNS (= Domain Name System), zum Loadbalancer (der dann vermutlich irgendwann mitskaliert werden muss) oder zu anderen Infrastrukturkomponenten.

Die beinahe grenzenlose Skalierbarkeit ist an all den Stellen begrenzt, an denen synchronisiert werden muss. In Anlehnung an das Amdahlsche Gesetz (Amdahl's Law) sollten wir beachten, dass Bestandteile in unserem System, die sich nicht parallelisieren lassen bzw. nicht von uns parallelisiert werden, beim Hochskalieren zu einem Flaschenhals werden können.

7.11.7 Der Strategiemix

Häufig sind Problemstellungen, Algorithmen und Konsistenzanforderungen so unterschiedlich, dass eine homogene Strategie – über alle (Software-)Bausteine hinweg – nicht die wirtschaftlichste oder effektivste Lösung darstellt. In dem Fall sollten die verschiedenen Strategien zielgerichtet kombiniert werden.

Stellen Sie sich eine Implementierung vor, die nicht von mehr Plattenplatz oder CPU-Kernen profitiert, weil sie single-threaded iteriert und rechnet. Wenn wir den Algorithmus nicht so ändern können, dass er sich parallelisieren lässt, ist die beste Option, für so eine Komponente – sofern sie den Flaschenhals darstellt – eine maximal schnelle CPU (für die Ausführung eines Threads) zu beschaffen und alles dafür zu tun, dass dieser Thread schneller arbeiten kann.

Das kann sich sowohl auf die Prozessorarchitektur, die Größe des 1st-, 2nd- und 3rd-Level-Cache als auch auf die Auswahl der RAM-Bausteine beziehen und erfordert üblicherweise Hardwarewissen, über das wir Softwerker tendenziell nur sehr eingeschränkt verfügen.

Das vertikale Skalieren erhöht – zumindest für einen Teil der Anfragen – die Latenzzeit, da die Verarbeitung der Anfrage nun auf Knoten verteilt ist. Dennoch kann sich das Erhöhen der Latenz amortisieren, wenn zum Beispiel dadurch die Speicherverwaltung effizienter wird oder auch unterschiedliche Kapazitäten für Ein- und Ausgabe sowie für rechenintensive Teile bereitgestellt werden können. So kann beispielsweise im Frontend auf Netzwerk-I/O, in der Geschäftslogik auf CPU-Power und im Backend auf Platten-Cache und -I/O optimiert werden.

Anfragen, die sich leicht parallel verarbeiten lassen, ohne dass dabei ein gemeinsamer Zustand synchronisiert werden müsste, lassen sich hervorragend horizontal skalieren. Ein Beispiel für diese Skalierungsstrategie ist der Einsatz sogenannter Content Delivery Networks (CDNs), die es abhängig vom eingekauften Service-Level erlauben, Daten von geografisch nächstgelegenen Datenzentren zu beziehen.

7.11.8 Allgemeine Daumenregeln

Sie wissen, dass Daumenregeln stets auf ihre Eignung im eigenen Kontext betrachtet werden müssen.

Tendenziell würde ich meinen, dass das Kaufen von leistungsfähiger Hardware sehr kosteneffizient funktioniert. Sie brauchen statt 4 oder 8 GB eher 64, 128 oder 512 GB RAM? – Das ist üblicherweise schnell amortisiert.

Sie brauchen etliche Terabyte oder Petabyte? – Vermutlich sprechen wir dann über Investitionen, die Sie nicht selbst stemmen können und für die Sie einen Sponsor im Business finden müssen.

Sie verarbeiten im Weihnachtsgeschäft etwa das 20-Fache dessen, was Sie im Jahresdurchschnitt verarbeiten? Dafür wollen wir eher keine einzelne Maschine dimensionieren, falls die Kosten für Netzwerk, RAM und CPU überproportional anstiegen. Wenn wir einen kostengünstigeren Weg finden, die Last in den wenigen Wochen „Weihnachten“ zu verarbeiten, würde uns das Business vermutlich danken.

Sie wollen sich den Schmerz antun, alles horizontal zu skalieren und die Verarbeitung auf gleichberechtigte Knoten zu verteilen? Wenn sich das organisatorisch regeln lässt, ist dieser Ansatz sehr leicht beherrschbar. Aber wenn das nicht gelingt, dann müssen Sie sich um Lastverteilung, Überwachung, Protokollierung in einem dynamischen Umfeld mit sehr vielen Fehlerquellen kümmern.

Häufig gibt es Grenzen der organisatorischen Regelungen. Operative Fehler zwingen uns dann dazu, im Betrieb des Gesamtsystems festzustellen, an welcher Stelle ein Fehler aufgetreten ist. Das kann insbesondere im Umfeld von elastischer horizontaler Skalierung schwierig werden,

wenn ein Teil der Anfragen auf einem Knoten verarbeitet wurde, der einen Hardwaredefekt hatte, aber nur temporär genutzt wurde. Hier steigt also die Anforderung an die Überwachung des Systems deutlich gegenüber der vertikalen Scale-Out- oder der Scale-Up-Strategie.

Bevor wir uns die Stellschrauben anschauen, die uns bei Scale-Up zur Verfügung stehen, ist eine Warnung angebracht, die John Gall in „The Systems Bible“ sinngemäß formuliert hat: „Ein System, das durch einfaches Vergrößern der Dimensionen eines kleineren funktionierenden Systems abgeleitet wird, verhält sich überhaupt nicht wie das kleinere.“

7.11.9 CPU-Power

Zwar überschätzen manche die Leistungsfähigkeit einer CPU gern, allerdings sollten wir nicht vergessen, dass sie für rechenintensive Aufgaben einen guten Hebel bietet. Beispielsweise ist der LMax Disruptor ein Muster, das von der Taktung eines CPU-Kerns profitiert. Kann es sein, dass der Prozessor der Flaschenhals ist, hilft nicht die Investition in weitere Kerne: Damit der Disruptor schneller wird, muss der Prozessor schneller werden.

7.11.10 GPU-Power

Üblicherweise sind Standard-Rechnerarchitekturen sehr effizient darin, Daten aus dem RAM zu holen und dahin zu transportieren. Das Transportieren von Daten zur GPU ist da manchmal einfach teurer. Wenn allerdings eine Menge gleichförmiger Daten einheitlich – und unabhängig voneinander – verarbeitet werden müssen, zahlt es sich unter Umständen aus, die Daten zur GPU zu schaufeln, dort mit Hunderten oder Tausenden von einfachen Prozessoren gleichförmig zu verarbeiten und dann wieder zurückzuschaukeln.

Hier bieten einheitliche Programmierschnittstellen wie CUDA oder OpenCL plattformneutrale Programmiermodelle.

7.11.11 RAIDs, SANs und andere Speichersysteme

Bei Speichersystemen muss man stets zwischen Kapazität, Latenz, Durchsatz, Lebenserwartung, Datensicherheit und Transferraten abwägen. Normale Plattensysteme sind tendenziell sehr schnell für sequenzielle Zugriffe, sobald sie Daten laden, die zusammenhängend abgelegt sind. Fragmentierte Daten sind für ein Plattensystem tendenziell teuer. Der parallele Zugriff auf zerstreute Daten ist für Plattensysteme auch nicht effizient verarbeitbar. Daher haben die Systeme unterschiedliche Strategien, zugegriffene Daten zu cachen oder das physikalische Speichern so zu verzögern, dass das Gesamtsystem durch Zugriffe auf das Speichersystem nur minimal ausgebremst wird.

Neben Plattensystemen sind heutzutage SSDs oder Flash-basierte Systeme verfügbar, mit denen Sie die Latenz im Vergleich zu plattenbasierten Systemen erheblich reduzieren können.

Platten- oder speicherbausteinbasierte Systeme (wie zum Beispiel SSDs) können parallel geschaltet werden, um gewisse Qualitätseigenschaften wie Durchsatz oder Datensicherheit

zu erhhen. Einer der dafr typischen Standards ist RAID (Redundant Array of Independent Disks), den es in verschiedenen Ausprgungen (RAID 0, 1, 10 o. .) gibt.

Auf eine Abhandlung der verschiedenen RAID-Arten wird hier verzichtet. Fr uns wichtig ist das Wissen, dass wir hier auch hardwareseitig verschiedene Optionen haben, die Qualitt zu steuern. Wenn es um Durchsatz geht, gibt es zwei entscheidende Faktoren: die Produktqualitt der konkreten Hardware an sich sowie die rein rechnerische Kapazitt, die die Hardware zu leisten vermag.

7.11.12 Bussysteme fr die Speichieranbindung

blicherweise werden Speichersysteme ber Bussysteme (PCIe, SATA, Fibre Channel, Thunderbolt o. .) angebunden, deren Transferraten weit ber dem liegen, was in blichen Rechnernetztopologien mit den dort blichen Netzwerkkarten mglich ist.

Nr.	Bus	Datenrate
1	PCIe 4.0 x16	~ 32 GB/s
2	SATA 3	6 GB/s
3	SATA express 16	16 GB/s
4	Fibre Channel	16 GB/s
5	Thunderbolt 2	20 GB/s
6	InfiniBand	40+ GB/s

7.11.13 Geringere Bandbreite im Netz

Die Verteilung der Verarbeitungslogik auf mehrere Knoten fhrt automatisch die Abhngigkeit zu dem zwischen dem Knoten liegenden Netzwerk ein. Es gibt unterschiedliche Netzwerktechnologien, die sich auch in Bezug auf Signallaufzeit, Bandbreite, Latenz, Durchsatz und Zuverlssigkeit unterscheiden.

In lokalen Verteilungsszenarien, in denen alle Knoten innerhalb eines LAN liegen, knnen wir von einem zuverlssigeren Netz ausgehen als im Fall der geografischen Verteilung der Verarbeitungsknoten.

Die beste Skalierbarkeit lsst sich erreichen, indem man die Asymmetrie in den unterschiedlichen Qualittseigenschaften zwischen WAN, LAN und internen Bussystemen bercksichtigt. Je mehr Zeit und Hardwareressourcen beim Zugriff und der Verarbeitung involviert sind, desto schlechter lsst sich ein System skalieren.

Phillip Ghadir (<http://innoq.com>) ist Mitglied der Geschftsleitung und CTO der innoQ und bert Unternehmen in allen Fragen rund um Softwarearchitektur, -technik und -entwicklung. Phillip spricht verhandlungssicheres Java, Ruby, Clojure, C++ und einige andere Sprachen, hat mittlere und groe unternehmenskritische IT-Systeme entworfen und erfolgreich in Produktion gebracht. Als aktives Mitglied des iSAQB e. V. gibt er regelmig Trainings zu Softwarearchitektur.

■ 7.12 Blockchain und dezentralisierte Architektur

Von Dr. Lars Hupel

Im Umfeld der verteilten Systeme hat sich in der jüngeren Vergangenheit eine neue Klasse von Architekturmustern etabliert: sogenannte Blockchains beziehungsweise Distributed Ledgers. Die Grundannahme ist, dass ein dezentrales System – im Gegensatz zum verteilten System – aus einer Menge von Parteien besteht, zwischen denen wenig bis kein Vertrauen herrscht. Gängige verteilte Systeme, zum Beispiel verteilte Datenbanken, benötigen jedoch dieses Vertrauen, um einen reibungslosen Betrieb zu garantieren. Blockchains verkraften aber nicht nur zufällige Ausfälle durch Netzwerkfehler, sondern auch potenziell böswillige Akteure. In diesem Abschnitt stelle ich die Wirkungsweise und Historie der Blockchain-Technologien vor und zeige, welche Einflüsse diese auf die Softwarearchitektur von größeren, netzwerk-basierten Systemen hat.

7.12.1 Definition und Abgrenzung

Blockchain-Technologien treten in vielen verschiedenen Ausprägungen auf, deren Entwicklung rasch fortschreitet. Deswegen fällt eine präzise Definition der Geschmacksrichtungen schwer. Den bisherigen Konsens um die zentralen Begriffe stelle ich Ihnen nachfolgend vor.

Distributed Ledger: Dieser Basisbegriff bezeichnet eine Ablage von Transaktionen, die auf mehreren physikalischen Maschinen repliziert ist und bei der Transaktionen ausschließlich (additiv) angefügt werden dürfen. Es ist nicht zulässig, die Reihenfolge zu ändern oder Transaktionen zu löschen. Ein Distributed Ledger ist damit die verteilte Form eines buchhalterischen Kassenbuchs, welches ähnliche gesetzliche Vorgaben bezüglich Manipulationssicherheit zu erfüllen hat.

Diese Definition lässt vollständig offen, welche äußere Form die Transaktionen haben müssen und ob Anforderungen an die Gültigkeit gestellt werden. In einem traditionellen Kassenbuch sind Transaktionen z. B. immer Buchungszeilen, die eine bestimmte Geldmenge von einem Konto auf ein anderes Konto bewegen, wobei das Quellkonto im Regelfall gedeckt sein muss. Soll eine Buchung rückgängig gemacht werden, so muss eine neue Gegenbuchung erzeugt werden, die den Effekt der alten Buchung aufhebt, aber sie nicht gänzlich löscht.

Ähnlich wie beim Architekturmuster des *Event Sourcing* lässt sich der Zustand des Gesamtsystems bei einem Distributed Ledger auf Grund der gespeicherten Transaktionen nachvollziehen. Die Motivation hierfür ist aber eine andere: während beim Event Sourcing oftmals die flexible Interpretation der gespeicherten Events und andere technische Gesichtspunkte im Vordergrund stehen, ist bei einem Distributed Ledger die Revisionssicherheit das ausschlaggebende Designkriterium.

Blockchain: Aufbauend auf einem Distributed Ledger gruppiert eine Blockchain die gespeicherten Transaktionen in eine linear verkettete Liste von Blöcken. Üblicherweise werden die Verknüpfungen zwischen den Blöcken mit kryptografischen Mitteln abgesichert, um nachträgliche Änderungen zu verhindern.

Während ein Distributed Ledger nicht zwingend eine lineare Folge von Transaktionen sein muss, ist dies bei Blockchains anders. Jeder Block hat genau einen Vorgänger (außer der initiale Block) und höchstens einen direkten Nachfolger. Auf technischer Ebene wird das dadurch realisiert, dass man jedem Block auf Basis der enthaltenen Transaktionen und Metadaten einen Hash-Wert zuordnet. Die Metadaten enthalten neben Zeitstempel aber auch den Hash-Wert des Vorgängerblocks. Würde nun ein bestimmter Block in einer Kette verändert, so würden sich auch die Hash-Werte des unmittelbaren und aller weiteren Nachfolger ändern.

Kryptowährung: Bitcoin hat verschiedene bereits existierende Technologien geschickt kombiniert, um auf Basis einer Blockchain eine digitale Währung zu etablieren. Konkret muss eine Kryptowährung – ähnlich wie die staatlichen Währungen – *fungibel* und teilbar sein.

- Fungibel heißt, dass der Wert einer Münze austauschbar ist; so ist z. B. jede 1-Euro-Münze gleich viel wert und zwei 50-Cent-Münzen lassen sich auch problemlos durch eine 1-Euro-Münze „repräsentieren“. Wenn man auf ein Sparguthaben einen 10-Euro-Schein einzahlt und später wieder auszahlt, erhält man einen anderen, gleichwertigen Schein zurück.
- Teilbarkeit ist zumindest bei Bargeld gegeben, so dass ein Konto rechnerisch einen Stand von 3,1415 € aufweisen kann. Dem sind natürlich Grenzen gesetzt: eine Stückelung von Millionsteln von Cent wäre wenig sinnvoll.

Eine Kryptowährung muss nicht zwingend auf einer Blockchain laufen; manche benutzen auch einen reinen Distributed Ledger ohne Blockstruktur. Wichtig ist allerdings, dass die Infrastruktur das sogenannte *Double Spending* ausschließen muss: derselbe Euro darf nicht mehrfach ausgegeben werden. Um ferner Betrug zu verhindern, werden kryptografische Maßnahmen ergriffen, dass nur die rechtmäßige Eigentümerin ihr Geld ausgeben darf.

Es gibt mehrere Möglichkeiten, wie Wertschöpfung in Kryptowährungen funktioniert. Bitcoin und viele andere nutzen den Prozess des *Minings*, bei der Rechenzeit gegen Belohnung aufgewendet wird. In gewerblichen Kontexten ist das oft nicht sinnvoll, weswegen auch alternativ einer bestimmten Menge von Autoritäten die Wertschöpfung übertragen werden kann; vergleichbar mit dem Zentralbanksystem.

Dezentralisierte Applikation: Solche Applikationen grenzen sich gegenüber verteilten Systemen derart ab, dass es keine zentrale Autorität gibt, die sämtliche Dienste kontrolliert. Mehrere Parteien einigen sich lediglich auf ein Protokoll und Spielregeln, dürfen aber ansonsten eigene Implementierungen benutzen und müssen gegebenenfalls Mehrheitsentscheide treffen. Das Web ist ein Paradebeispiel für ein solches System: Browser und Webserver werden von zahlreichen Organisationen benutzt und betrieben und sprechen HTTP und HTML miteinander. Ein Standardisierungsgremium legt die Spielregeln fest, wird aber nur durch die Teilnehmer und deren Akzeptanz legitimiert.



Nicht alle Blockchains und Distributed Ledgers werden für Kryptowährungen benutzt. Tatsächlich sind die Designentscheidungen, die bei Kryptowährungen getroffen werden, im Regelfall ganz andere als solche, die für die Softwareentwicklung in Unternehmenskontexten sinnvoll wären. Zum Beispiel ist es nicht zwingend, dass Blockchain-Transaktionen wie bei Bitcoin öffentlich verteilt werden. Auch eine auf private Netzwerke beschränkte Verteilung ist denkbar.

Mit diesen Grundbegriffen gewappnet können wir die technischen Grundlagen von Blockchains betrachten.

7.12.2 Technische Grundlagen von Blockchains

Als erste und immer noch bedeutendste Kryptowährung eignet sich Bitcoin, um die technische Funktionsweise von Blockchains zu skizzieren. Grundsätzlich handelt es sich bei Bitcoin um ein Netzwerk aus gleichberechtigten Parteien, ein sogenanntes *Peer-to-Peer-Netzwerk* (kurz: *P2P-Netz*). Jeder Mensch mit einem Computer kann daran teilnehmen, ganz gleich, ob er sich am Währungstransfer beteiligen möchte oder nicht. In einem P2P-Netz gibt es Mechanismen, zum Beispiel eine Liste mit IP-Adressen von bereits bekannten Knoten, mit denen ein neuer Knoten beitreten kann.

Ist man Teil des Netzes, so muss man zunächst die existierende Blockchain in Form von mehreren Gigabyte von Blöcken mit Transaktionen herunterladen. Dazu kann ein Knoten praktisch beliebige andere Knoten fragen, da jeder Knoten diese Daten vorhält. Neue Blöcke werden über ein *Gossip-Protokoll* im Netz verteilt.

Die Grundidee, wie neue Blöcke erzeugt werden, ist schnell beschrieben: jeder darf es zu jedem Zeitpunkt probieren; aber es ist eine Art „digitales Glücksspiel“, bei dem kollektiv alle Knoten ein Rechenpuzzle lösen und durchschnittlich nur etwa alle 10 Minuten jemand Glück hat und einen neuen Block erzeugen darf.



Bitcoin nutzt ein rechenaufwendiges mathematisches Problem, um neue Blöcke künstlich zu „verknappen“. Das verhindert, dass man *einfach* (d. h. ohne hohen Einsatz elektrischer Energie) beliebige Mengen von Blöcken erzeugen kann. Im Vergleich dazu: eine Zentralbank kann auch nur so viel neues Geld drucken, wie das politisch und juristisch möglich ist. Bitcoin braucht aber mathematische Verfahren, denn bisher hat es noch keine Regierung es geschafft, eine weltweit verteilte, anonyme Währung zu regulieren.

Die Transaktionen in den Blöcken sind eine abstrakte Datenstruktur, deren Details den Rahmen sprengen würde). Sie durchlaufen der Reihe nach mehrere Zustände (vereinfacht: Erzeugung, Validierung, Bestätigung). In der Validierungsphase prüfen potenziell alle Knoten im Netzwerk, ob die Transaktion gültige kryptografische Signaturen enthält. Bevor eine validierte Transaktion bestätigt wird, warten die Knoten auf eine bestimmte Menge weiterer Transaktionen, bis die Blockgröße erreicht worden ist. Anschließend wird ein Block erzeugt und im Netzwerk propagiert.

Übrig bleibt noch zu klären, wie im Netzwerk Konsens über die neuen Transaktionen und Blöcke hergestellt wird. Bei Bitcoin heißt dieses Verfahren *Nakamoto-Konsens*, benannt nach dem Pseudonym Satoshi Nakamoto (dem sagenumwobenen Erfinder von Bitcoin). Dieses Verfahren ließe sich auch – etwas plump – als Mehrheitsentscheid beschreiben, wobei sich das Stimmengewicht anhand der eingesetzten Rechenkraft ermittelt. In anderen Worten: Die Mehrheit der CPUs bestimmt die Spielregeln.

7.12.3 Smart Contracts

Bereits 1996, lange bevor Blockchain-Technologien in Sicht waren, definiert Nick Szabo einen *Smart Contract* als Menge von digital spezifizierten Versprechen sowie Protokolle, mit denen die beteiligten Parteien diese Versprechen erfüllen. Mit Hinblick auf die neueren Entwicklungen lassen sich Smart Contracts konkret dadurch definieren, dass es sich um ausführbaren Code handelt, der als Datenpaket auf einem Distributed Ledger gespeichert ist und innerhalb von Transaktionen ausgeführt wird.

Im Vergleich zu traditionellen Verträgen, die in einer natürlichen Sprache formuliert sind und gegebenenfalls von einer juristischen Instanz (z. B. Notar oder Gericht) durchgesetzt werden, sind Smart Contracts vollständig in ihrer Ablaufumgebung eingebettet. Das heißt, dass Smart Contracts nur digitale Güter verwalten können, die in einer bestimmten Form auf dem Ledger existieren. Es gibt zwar Ansätze, die die Verknüpfung von realen Gütern und digitalen Werten herstellen, doch streng genommen erfordern diese stets die vertrauenswürdige Kooperation mit Menschen oder Organisationen.

Die erste Blockchain-Plattform, die Smart Contracts popularisiert hat, ist *Ethereum*. Ähnlich wie Bitcoin handelt es sich dabei um eine öffentliche Kryptowährung; bietet aber zusätzlich eine Programmiersprache für die Implementierung von Code, der auf allen teilnehmenden Knoten im Netzwerk ausgeführt wird. So lassen sich beispielsweise virtuelle Konten erzeugen, auf die man ganz gewöhnlich Geld überweisen kann, aber deren Verfügungsgewalt aus einem „2 aus 3“-Schlüssel ergibt: Es müssen also mindestens zwei der drei Eigentümerinnen zusammenkommen, um eine ausgehende Transaktion zu tätigen.

Da Transaktionen, in denen Smart Contracts ausgeführt werden, in Ethereum grundsätzlich von allen Knoten nachvollzogen werden müssen, herrscht eine tausendfache Redundanz, was sich auch in den Ausführungskosten niederschlägt. Jeder Operation ist ein Kostenwert zugeordnet. Besonders teuer dabei sind Speicheroperationen, da auch der Wert jeder Speicherzelle auf jedem Knoten repliziert wird. Folglich muss man deutlich sparsamer sein als in klassischen Architekturen.

Die virtuelle Maschine, die Ethereum zu Grunde liegt, ist Turing-vollständig, das heißt, dass sich quasi beliebig komplexe Programme darauf abbilden lassen. Andererseits können einmal ausgerollte Smart Contracts nachträglich nicht mehr geändert werden, so dass Programmierfehler fatale Auswirkungen – bis hin zum Totalverlust der verwalteten Güter – haben können.

7.12.4 Blockchains in nichtöffentlichen Kontexten

Ein Pferdefuß von Bitcoin und anderen Kryptowährungen ist, dass sämtliche Transaktionen im öffentlichen Internet verteilt werden. Obwohl Identitäten keine Namen haben – Bitcoin daher pseudonym ist – ist es aus mehrerlei Gründen nicht ratsam, Daten, die aus einem Geschäftsablauf im Unternehmen entstehen, zu publizieren. Das kann den Datenschutz betreffen (z. B. eine Transaktion für einen Versicherungsfall), aber auch Geschäftsgeheimnisse (z. B. ein Smart Contract zur Berechnung von Versicherungsprämien).

Umgekehrt ist es auch nur selten sinnvoll, dass beliebige Personen neue Blöcke erzeugen können. Für eine Versicherungsgesellschaft ist es schlicht irrelevant, was andere Menschen mit ihren Bitcoins machen, für sie sind lediglich die betriebsrelevanten Daten interessant.

Aus diesen Überlegungen heraus sind *Permissioned Blockchains* und *Private Blockchains* entstanden, die oftmals in Kombination auftreten. Permissioned Blockchains schränken zunächst den Kreis an Personen ein, die berechtigt sind, neue Blöcke zu erzeugen. Damit nehmen diese Personen die Rolle einer Autorität ein. Es gibt einige Kryptowährungen, die diesen Ansatz benutzen, wobei hier natürlich ein erhöhtes Vertrauen gegenüber der Autorität existieren muss.

Private Blockchains schränken weiterhin den Teilnehmerkreis ein. Dafür wird eine Kombination von technischen Barrieren genutzt, z. B. VPNs und spezielle Zertifikate.

Beides zusammen wird oftmals als *Konsortiums-Blockchain* bezeichnet. Das Szenario tritt häufig bei Unternehmen oder Unternehmensgruppen auf, die in einem regulierten Markt im gegenseitigen Wettbewerb stehen, aber auch kooperieren müssen. Beispielsweise gibt es eine Vielzahl von Fahrzeugversicherungen, die aber aus juristischen Gründen einen nahtlosen Versicherungswechsel von Kunden sicherstellen müssen. Zur effizienten Gestaltung dieses Prozesses bietet sich eine übergreifende Blockchain-Infrastruktur an, bei der ausschließlich die beteiligten Versicherer befugt sind, Transaktionen einzubringen und bestehende Transaktionen zu lesen.

Smart Contracts können in diesen Szenarien viel flexibler ausgestaltet sein. Während Ethereum eine eigene Programmiersprache mitbringt, bieten Konsortiumslösungen wie z. B. *Hyperledger Fabric* und *Corda* SDKs für gängige Programmiersprachen wie Java und JavaScript mit, so dass sich existierende Geschäftslogik nahtlos einbinden lässt. Diese Flexibilität birgt aber auch einen Nachteil: Während Ethereum's VM auf deterministische und reproduzierbare Operationen festgezurr ist, müssen Fabric und Corda spezielle Tricks anwenden, denn beispielsweise Java-Code kann sich bei jeder neuen Ausführung auch anders verhalten. Man löst das Problem dadurch, dass ein Smart Contract nur zu einem bestimmten Zeitpunkt ausgeführt wird und dessen Effekte (Übertragung von Gütern, Schreiben von Variablen usw.) am Ende zertifiziert werden.

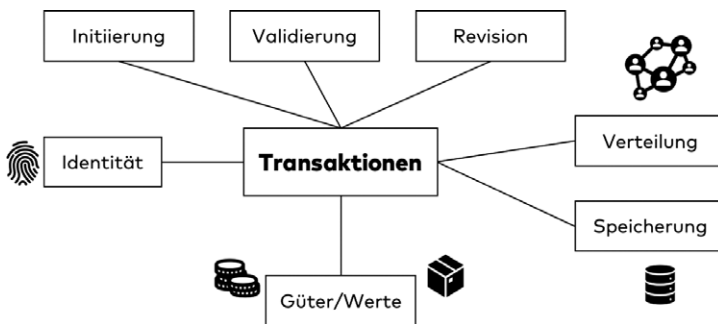


Bild 7.30
Transaktionen
bei Blockchains

7.12.5 Architekturabwägungen

Die Entscheidung für und wider Blockchain und die konkrete Technologie und Architektur bewegt sich im Spannungsfeld verschiedener technischer, organisatorischer und juristischer Belange.

Technische Hürden: In verteilten Systemen – dabei bilden auch Blockchains keine Ausnahme – gelten viele traditionelle Annahmen nicht: Es gibt keinen einheitlichen Zeitfluss, es kann bösartige Netzwerkteilnehmer geben und Kommunikationsfehler sind die Regel. Bevor man sich also für eine Blockchain entscheidet, sollte man ganz genau abwägen, ob ein zentralistisches System die bestehenden Geschäftsprobleme auch lösen kann. Schlüsselkriterien sind hierfür:

- Existiert eine zentrale, vertrauenswürdige Partei? Ist sie dauerhaft, technisch kompetent und neutral?
- Ist eine unveränderliche Geschichte von Transaktionen notwendig? Können regulatorische Bedingungen (Datenschutz) weiterhin eingehalten werden?
- Müssen Daten in einer öffentlichen Blockchain vorliegen?
- Welcher Durchsatz von Transaktionen bzw. welche Menge von Daten muss redundant gespeichert werden?
- Welche Schnittstellen haben die digitalen Güter und Verträge mit ihren realen Pendants?

Organisatorische Hürden: Hat man die technische Machbarkeit einer Blockchain-Lösung geklärt, dann muss der Rahmen geschaffen werden, in der die Knoten betrieben werden können. Je nach gewählter Technologie und Architektur sind mehrere betrieblich getrennte Organisationseinheiten dafür verantwortlich, Knoten lokal oder in einem Rechenzentrum zu betreiben. Üblicherweise einigt man sich auf ein Rahmenwerk, in dem Zertifikate oder Zugriffsdaten ausgetauscht werden, sowie für die Neuaufnahme und das Verlassen von Einheiten im Netzwerk. Dies steht im Gegensatz zu von einem Operationsteam zentral betriebenen Infrastruktur.

Juristische Hürden: Aus rechtlicher Sicht gibt es zwei maßgebliche Fallstricke beim Betrieb einer Blockchain-Infrastruktur. Zum einen wird die Einhaltung rechtlicher Auflagen, z. B. der DSGVO, wird unter Umständen erschwert. Für Betroffene, deren Daten auf einem Ledger gespeichert sind, muss jederzeit die Möglichkeit bestehen, auch in den redundanten Kopien gesperrt zu werden.

Zum anderen muss durch externe Verträge die Wirksamkeit der Smart Contracts sichergestellt sein. Sobald reale Güter betroffen sind, ist deren Gültigkeit nicht zwingend gegeben, denn die meisten Rechtssysteme stellen hohe Anforderungen an die Form von Verträgen. Insbesondere zählt ein Stück Code grundsätzlich nicht als Schriftform. In Konsortien kann dies z. B. durch Partnerschaftsverträge entsprechend ausgestaltet werden.

7.12.6 Architekturmuster

Zu guter Letzt wollen wir einen Blick auf einige Architekturmuster werfen, die sich bei Blockchains bewährt haben.



Ähnlich wie in klassischen Architekturen gibt es oftmals nur eine sehr schwammige Definition der Muster. Bei Blockchains wird dies dadurch erschwert, dass sich die einzelnen Implementierungen teilweise stark unterscheiden und manche Muster entweder fest eingebaut, implementierbar oder gar vollständig nutzlos sind.

Muster	Problem	Lösungsansatz
Orakel	Ein Smart Contract muss auf Daten der „echten Welt“ zugreifen.	Man designiert einen speziellen Knoten, der nach Bedarf Informationen z. B. aus einer API abrufen und diese im Ledger persistiert. Diesem Knoten wird eine besondere Autorität beigemessen.
Tokenization	Güter müssen auf der Blockchain verwaltet werden.	Man zerlegt Güter in die kleinsten sinnvollen Einheiten, z. B. „Produkt A“. Jedem Produkt wird eine Identität in Form eines kryptografischen Schlüssels zugewiesen. Dann erhält man Token, die in Transaktionen verwendet werden können.
Off-Chain Storage	Es fallen bei Prozessen große Datenmengen an.	Das Speichern von Daten ist wegen der Replikation von Blockchains teuer. Stattdessen kann man die Daten klassisch mit niedriger Replikation auf gewöhnlichem Wege ablegen und nur die Hashes auf der Blockchain verwalten. Manipulationen der Daten können dann trotzdem noch erkannt werden.
Incentive Execution	Es gibt mehrere Parteien, die sich an die Regeln halten sollen.	Man versucht die Architektur so auszugestalten, dass die Parteien einen finanziellen Anreiz haben, die Verträge korrekt auszuführen. Das kann z. B. mit Pfandgeldern erfolgen, die durch Verstöße verwirkt werden.
Legal & Smart Contract Pair	Ein Smart Contract soll reale Güter verwalten.	Bevor ein Smart Contract ausgerollt wird, definiert man einen juristischen Vertrag, der die Parteien an die Ergebnisse des Smart Contracts bindet.

Analyse und Bewertung von Softwarearchitekturen

To measure is to know.

James Clerk Maxwell¹



Fragen, die dieses Kapitel beantwortet:

- Warum sollten Sie Architekturen und Code analysieren oder bewerten²?
- Was können Sie in der IT überhaupt bewerten?
- Was unterscheidet qualitative von quantitativer Analyse?
- Wie gehen Sie bei der Analyse oder Bewertung von Architekturen vor?
- Welche Stakeholder sollten bei Analyse und Bewertung mitwirken?
- Wie setzen Sie Softwaremetriken sinnvoll ein?

Bewerten Sie schon oder raten Sie noch?

„*You cannot control what you cannot measure*“, schreibt Tom DeMarco treffend. Positiv formuliert, bedeutet diese Aussage für die Softwareentwicklung: Wenn Sie kontrolliert auf Ihre Ziele zusteuern möchten, müssen Sie regelmäßig Ihre Zielerreichung (möglichst objektiv) kontrollieren, um bei Bedarf steuernd eingreifen und korrigierende Maßnahmen einleiten zu können.

Während der Entwicklung sollten Sie regelmäßig die Frage „Gut genug?“ stellen und beantworten. In Kapitel 3 haben Sie die wesentlichen Aufgaben in der Architekturentwicklung kennengelernt – darin kommen die Analyse und Bewertung als eine der zentralen Tätigkeiten vor.

Bild 8.1 zeigt im Überblick, dass Bewertung auf alle übrigen Architekturaufgaben starken Einfluss hat. Auf Basis systematischer Analyse und Bewertung können Entwicklungsteams schlichtweg Ihre eigene Arbeit besser planen und Ihre Ergebnisse verbessern!

In der Praxis finden Bewertungen (unter dem Titel „Audit“ oder „Review“) oftmals in späten Entwicklungsphasen statt, wenn das betroffene System bereits im produktiven Einsatz läuft. Viel zu selten investieren Teams während der Entwicklungsarbeit in Analyse – obwohl das den Arbeitsergebnissen guttäte.

Bewertung unterstützt Steuerbarkeit

¹ http://de.wikipedia.org/wiki/James_Clerk_Maxwell: Formulierte die Maxwellgleichungen, Grundlagen der Berechnung elektromagnetischer Wellen.

² In früheren Auflagen und in anderen Literaturquellen finden Sie den Begriff „bewerten“ – das impliziert allerdings aufgrund des Suffixes „werten“ eine zahlenmäßige Einstufung. In diesem Kapitel geht es aber über die quantifizierbaren Untersuchungen hinaus.

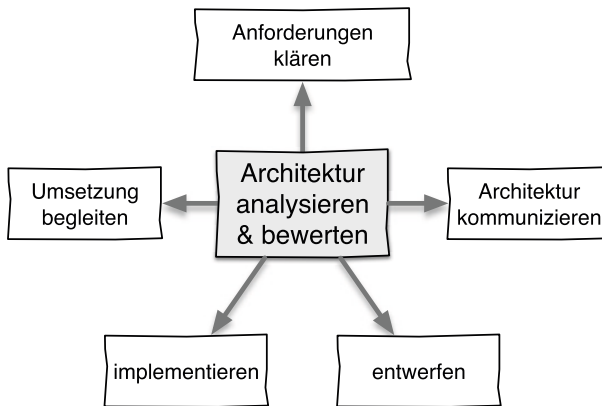


Bild 8.1
Architekturbewertung
als relevante Aufgabe

Was Sie in der IT bewerten können – und wie

Sie können in der Softwareentwicklung zwei Arten von „Dingen“ analysieren und bewerten:

- Prozesse, wie etwa Entwicklungs- oder Betriebsprozesse: Hierbei können Sie organisatorische Aspekte betrachten oder aber den Einsatz von Ressourcen bewerten. Leider können Sie auf Basis der Prozesse kaum Aussagen über die Qualität der entwickelten Systeme treffen.³ Dieses Thema werde ich hier nicht weiter vertiefen.
- Artefakte, wie beispielsweise Anforderungen, Architekturen, Quellcode und andere Dokumente. Einige Arten dieser Artefakte (wie beispielsweise Quellcode) können Sie quantitativ, das heißt in Zahlen, bewerten. Andere (wie etwa die Softwarearchitektur) entzieht sich der rein zahlenmäßigen Bewertung. Diese Gruppe von Artefakten analysieren Sie qualitativ, also ihrer Beschaffenheit oder Güte nach. Zur letzten Gruppe gehören Softwarearchitekturen, deren Bewertung *qualitativ* erfolgt.

Zunächst möchte ich Ihnen erläutern, welche Ergebnisse Sie bei Bewertungen grundsätzlich erreichen können. Anschließend lernen Sie (in Abschnitt 8.1) qualitative Architekturbewertung kennen, angelehnt an die verbreitete ATAM-Methode (Architecture Tradeoff Analysis Method).

Schließlich finden Sie in Abschnitt 8.2 einige Grundlagen quantitativer Bewertung und Metriken.

Bewertung als Soll-Ist-Vergleich

Betrachten Sie qualitative Analyse und Bewertung als einen Soll-Ist-Vergleich:

- Das Soll beschreibt die Qualitätsanforderungen beziehungsweise Architekturziele. Es charakterisiert die gewünschten Eigenschaften eines Systems.
- Sie vergleichen gegen das (aktuelle oder geplante) Ist-System, möglicherweise auf Basis von Dokumentation, Plänen oder Modellen. Dabei gehen Sie feingranular vor, d. h., Sie vergleichen einzelne Anforderungen des „Soll“ mit ihren Entsprechungen im „Ist“.

³ Sie können jedoch durch gezielte Reflexionen oder Projekt-Retrospektiven signifikante Verbesserungen erreichen, die zumindest mit hoher Wahrscheinlichkeit positive Auswirkungen auf die Qualität von Systemen haben.

Lassen Sie mich ein fachfremdes Beispiel heranziehen – siehe die nachfolgende Tabelle: Ich möchte qualitativ meinen morgendlichen Grüntee bewerten:

Tabelle 8.1 Beispiel für Soll-Ist-Vergleich

Soll	Ist	Ergebnis des Soll-Ist-Vergleichs
Hellgrüne Farbe	Hellbraune Farbe	Anforderung nicht erfüllt
Typischer Sencha-Geschmack	Geschmack nach Kirsche und Mandel	Gewünschter Geschmack nicht gegeben, dafür aber Preisanforderung besonders gut erfüllt
Grasiger Geruch	Geruchlos	Problem: Tee erfüllt Geruchsanforderung nicht
Ungezuckert		Wegen des Kirscharomas nicht prüfbar
Preis < 9 €/100 g	5 €/100 g	Anforderung (über-)erfüllt
Serviert in dünnwandiger Tasse	Wandstärke der Tasse < 1,5 mm	Anforderung erfüllt, ok

Bei dieser Art Vergleich können Sie grundsätzlich drei verschiedene Ergebnisse erzielen (siehe auch Bild 8.2):

1. Soll stimmt mit Ist überein – die Architektur bzw. deren Umsetzung ist bezüglich dieser spezifischen Eigenschaft ok.
2. Soll stimmt in mehreren Qualitätsanforderungen nur teilweise mit Ist überein – es wurden Kompromisse geschlossen (einzelne Merkmale wurden verbessert auf Kosten anderer).
3. Einzelne Qualitätsanforderungen können im Ist nicht erreicht werden (Risiko) oder werden nicht erreicht (Problem, eingetretenes Risiko).

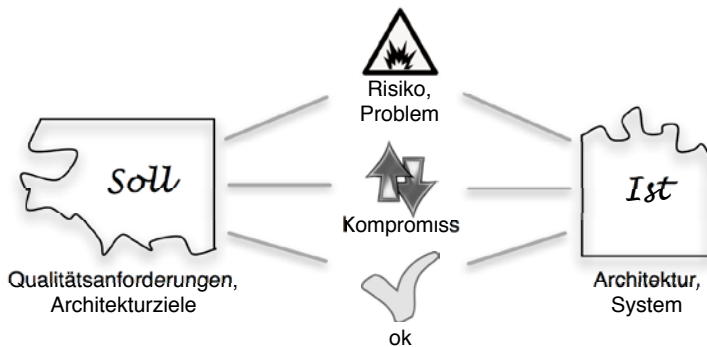


Bild 8.2
Bewertung als
Soll-Ist-Vergleich



Für qualitative Bewertung ist die Granularität der Anforderungen entscheidend: Sie benötigen einen detaillierten und möglichst konkreten Katalog an Qualitätsanforderungen und Architekturzielen!

Als Vorbild könnten Sie die bekannten Produktbewertungen der „Stiftung Warentest“ heranziehen, die oftmals sehr detaillierte Kriterienkataloge (= Anforderungen) als Basis ihrer Bewertungen verwenden.

■ 8.1 Qualitative Architekturbewertung

Bewertung von Qualitätsmerkmalen

Bewertung von Softwarearchitekturen liefert qualitative Aussagen: Sie bewerten Architekturen danach, inwieweit sie die Erreichung bestimmter Qualitätsanforderungen ermöglichen oder behindern.

Beispielsweise treffen Sie Aussagen darüber, in welchem Maß das jeweilige System den Anforderungen an Wartbarkeit, Flexibilität, Performance oder Sicherheit genügt.

Dabei liefert eine Architekturbewertung kein absolutes Maß, d. h., sie bewertet nicht die „Güte-über-alles“, sondern nur im Hinblick auf spezifische Kriterien. Architekturbewertung hilft, Risiken zu identifizieren, die sich möglicherweise aus problematischen Entwurfsentscheidungen ergeben.

Bewertungsgrundlagen für Softwarearchitekturen

In Kapitel 3 (Vorgehen bei der Architekturentwicklung) haben Sie wichtige Qualitätsmerkmale von Softwaresystemen kennengelernt. Ich habe Ihnen gezeigt, wie Sie mit Hilfe von Szenarien diese Qualitätsmerkmale beschreiben und operationalisieren können.

Mit Hilfe von Szenarien können Sie spezifische Messgrößen für Ihre Systeme definieren oder, noch besser, von den maßgeblichen Stakeholdern definieren lassen.

Qualität ist spezifisch für Stakeholder

Dabei gilt es zu ermitteln, welche spezifischen Kriterien die Architektur zu erfüllen hat. Anhand dieser Kriterien können Sie dann die „Güte“ oder „Eignung“ der Architektur ermitteln.

Eine Architektur ist in diesem Sinne geeignet, wenn sie folgende Kriterien erfüllt:

- Das System (das mit dieser Architektur entwickelt wird oder wurde) erfüllt sowohl seine funktionalen als auch nichtfunktionalen Anforderungen (Qualitätskriterien).
- Insbesondere erfüllt das System die spezifischen Anforderungen an Flexibilität, Änderbarkeit und Performance-Verhalten.
- Das System kann mit den zur Verfügung stehenden Ressourcen realisiert werden. Sowohl das Budget wie auch der Zeitplan, das Team, die beteiligten Fremd- und Legacy-Systeme, die vorhandene Basistechnologie und Infrastruktur sowie die gesamte Organisationsstruktur beeinflussen die Umsetzbarkeit einer Architektur.

Zur Bewertung einer Architektur ist es daher von entscheidender Bedeutung, die spezifischen Qualitätskriterien des betroffenen Systems zu kennen. In der Praxis werden diese Kriterien oder Qualitätsziele oft erst bei einer Architekturbewertung expliziert.



Bereits in Kapitel 3 habe ich Ihnen empfohlen, die Qualitätsanforderungen an Ihre Systeme explizit durch Szenarien zu dokumentieren. Diese Szenarien bilden eine wertvolle Orientierungshilfe für weitere Entwicklungsschritte und eine ausgezeichnete Basis für die Architekturbewertung!

Qualitative Bewertung nach ATAM

Ich möchte Ihnen im Folgenden ein methodisches Vorgehen zur Architekturbewertung vorstellen, orientiert an der ATAM⁴-Methode von [Bass+12] und [Clements+03]. Bild 8.3 gibt einen Überblick.

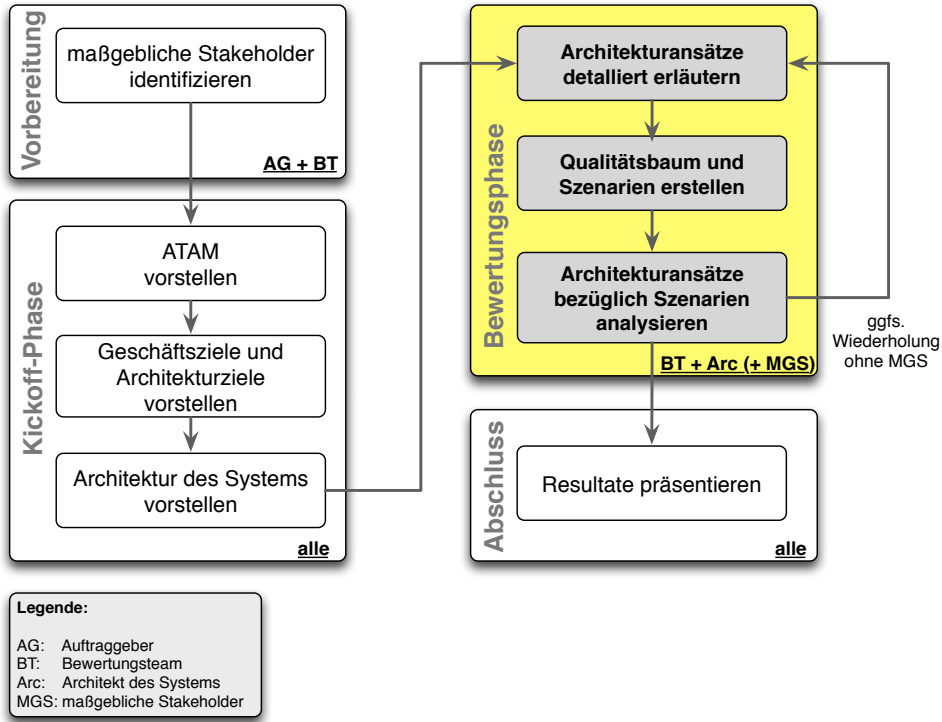


Bild 8.3 Vorgehen bei der Architekturbewertung nach ATAM

Der fundamentale Schritt bei der Architekturbewertung ist die Definition der von den maßgeblichen Stakeholdern geforderten Qualitätsmerkmale in möglichst konkreter Form. Als Hilfsmittel dazu verwenden Sie Qualitätsbäume und Szenarien (siehe auch Abschnitt 4.3.2). Ein Beispiel sehen Sie in Bild 8.4.

Maßgebliche Stakeholder identifizieren

Da die konkreten Ziele Ihrer Architekturen ausschließlich durch die spezifischen Ziele Ihrer Stakeholder vorgegeben werden, muss der Auftraggeber oder Kunde im ersten Schritt jeder Architekturbewertung die hierfür entscheidenden („maßgeblichen“) Stakeholder identifizieren. In der Regel geht eine Architekturbewertung von wenigen Stakeholdern aus, häufig aus dem Management.

⁴ Architecture Tradeoff Analysis Method



Stimmen Sie mit Ihren Auftraggebern gemeinsam ab, welche Stakeholder bei der Festlegung der Bewertungsmaße mitarbeiten sollen.⁵ Denken Sie – neben Ihren Auftraggebern oder Kunden – in jedem Fall an Endanwender und Betreiber.

Bewertungsmethode (ATAM) vorstellen

Bevor Sie mit der Definition der Bewertungsziele beginnen, sollten Sie den maßgeblichen Stakeholdern die Bewertungsmethode vorstellen. Insbesondere müssen Sie die Bedeutung nachvollziehbarer und konkreter Architektur- und Qualitätsziele klarstellen.

Verdeutlichen Sie den Beteiligten, dass es bei der qualitativen Architekturbewertung um die Identifikation von Risiken und Nicht-Risiken sowie um mögliche Maßnahmen geht, nicht um die Ermittlung (quantitativer) *Noten*.

Geschäftsziele vorstellen

Im Idealfall stellt der Auftraggeber die geschäftlichen (Qualitäts-)Ziele des Systems vor, das zur Bewertung ansteht. Dabei sollte der gesamte geschäftliche Kontext zur Sprache kommen, die Gründe für die Entwicklung des Systems sowie dessen Einordnung in die fachlichen Unternehmensprozesse.

Falls diese Ziele in Ihrer Situation bereits in der Anforderungsdokumentation aufgeführt sind, lassen Sie dennoch die *maßgeblichen Stakeholder* zu Wort kommen: Bei der Bewertung geht es um deren aktuelle Sichtweise. Außerdem sind die Zielformulierungen aus den Anforderungsdokumenten von Systemanalytikern gründlich gefiltert worden.



Ich teile die Erfahrung von [Clements+03], die von *Aha-Erlebnissen* bei der Vorstellung der aktuellen Ziele berichten: Die unterschiedlichen Teilnehmer von Bewertungsworkshops sitzen häufig das erste Mal gemeinsam an einem Tisch und lernen aus erster Hand die Zielvorstellungen anderer Stakeholder kennen. Dabei kommt es zu wertvollen „Ach sooo war das gemeint“-Erlebnissen.

Architektur vorstellen

Anschließend sollte der Architekt des Systems die Architektur vorstellen. Dazu eignet sich die Gliederung der Architekturpräsentation, die Sie in Kapitel 5 kennengelernt haben.

Es sollte in jedem Fall der komplette Kontext des Systems ersichtlich werden, das heißt sämtliche Nachbarsysteme. Außerdem gehören zu einer solchen Vorstellung die Bausteine der oberen Abstraktionsebenen sowie Laufzeitsichten einiger wichtiger Use-Cases oder Änderungsszenarien.

Architekturansätze erläutern

Welche Maßnahmen innerhalb der Architektur oder der Implementierung wurden zur Erreichung der wesentlichen (Qualitäts-)Anforderungen ergriffen? Wie wurden, strukturell oder konzeptionell, die wesentlichen Probleme oder Herausforderungen gelöst? Was sind die wesentlichen, prägenden Architekturentscheidungen?

⁵ Ich nenne diese spezielle Gruppe von Stakeholdern gerne die *maßgeblichen Stakeholder*.

Architekt oder Entwickler des Systems sollten diese Fragen beantworten, als Ergänzung zur vorangegangenen Überblicksvorstellung.

Einen spezifischen Qualitätsbaum erstellen

Lassen Sie Ihre Stakeholder im Anschluss an die Vorstellung der Geschäftsziele sowie der Architektur im Rahmen eines kreativen Brainstormings die wesentlichen geforderten Qualitätsziele erarbeiten. Ordnen Sie diese anschließend in (informeller) Baum-Form an: Die allgemeineren Merkmale stehen weiter links (oder oben im Baum), die spezielleren Anforderungen weiter rechts (oder unten).



Mindmaps können bei der Erstellung und Dokumentation von Qualitätsbäumen gute Dienste leisten. Sie sind leicht verständlich und insbesondere für ein *Brainstorming* von Qualitätsmerkmalen geeignet.

Ein Beispiel eines solchen Qualitätsbaums finden Sie in Bild 8.4 und in Bild 8.5. Dort sind als globale Qualitätsziele Performance, Erweiterbarkeit und Verfügbarkeit eingetragen und durch Latenz, Durchsatz etc. verfeinert.

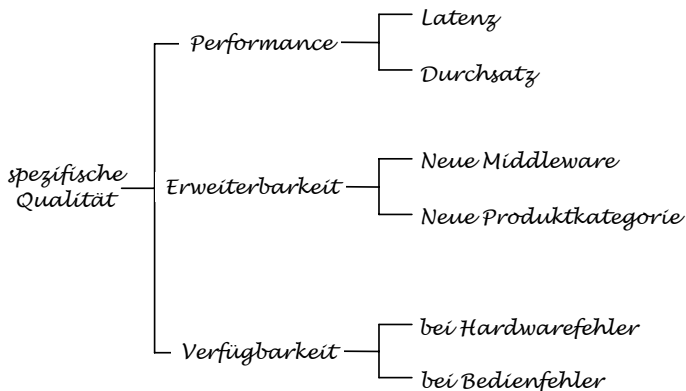


Bild 8.4
Beispiel eines
Qualitätsbaums

Entwickeln Sie Ihre Qualitätsbäume in kleinen und streng fokussierten Workshops, im Stile von Brainstorming-Sitzungen. Lassen Sie die Teilnehmer zu Beginn Architektur- und Qualitätsziele in ihren eigenen Worten beschreiben, ohne sie unmittelbar einzuordnen oder zu bewerten.

Falls Ihre Stakeholder etwas Starthilfe benötigen, können Sie die Architektur- und Qualitätsziele sowie Mengengerüste aus Anforderungsdokumenten in diese Diskussion einbringen. Ich finde es jedoch produktiver, die Teilnehmer einer solchen Diskussion möglichst wenig durch bestehende Dokumente zu belasten.

Anschließend strukturieren Sie gemeinsam mit den Teilnehmern die Beiträge in Form des Qualitätsbaums. Dabei können Sie die Wurzel des Baums auch anders benennen, etwa: „Nützlichkeit“ oder „wichtigste Architekturziele“.

Damit haben Sie den ersten Schritt, die Beschreibung der Bewertungsziele, bereits erledigt.

Qualitätsmerkmale durch Szenarien verfeinern

Szenarien: Abschnitt 3.6.2

Jetzt verfeinern Sie gemeinsam mit den maßgeblichen Stakeholdern die Qualitätsanforderungen und Architekturziele des Systems durch Szenarien. Wichtig ist dabei, die Formulierung der Szenarien möglichst konkret und operationalisiert zu halten. In Bild 8.5 sehen Sie einen um zwei Szenarien angereicherten Qualitätsbaum.

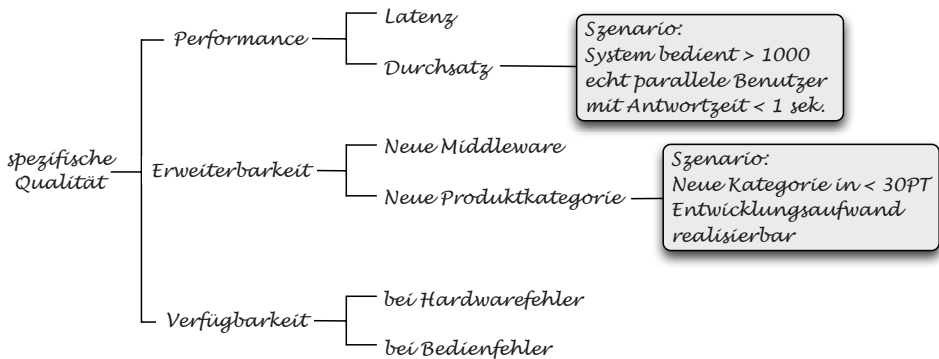


Bild 8.5 Qualitätsbaum mit Szenarien

Oftmals fällt es den Beteiligten leichter, in konkreten Szenarien zu denken als in abstrakten Qualitätsmerkmalen. Beginnen Sie in solchen Fällen ruhig mit einem Brainstorming von Szenarien und ordnen Sie die zugehörigen Qualitätsanforderungen erst im Nachgang als Baum oder Mindmap.



Mehr als 50 ausführlich formulierte praktische Beispiele zu Szenarien finden Sie im freien arc42-Subprojekt „quality-requirements“ unter:
<https://github.com/arc42/quality-requirements>

Bereits für mittelgroße IT-Systeme können Sie in solchen Brainstorming-Sitzungen durchaus 30 bis 50 verschiedene Szenarien finden. Im Normalfall wird Ihnen für die detaillierte Bewertung derart vieler Szenarien jedoch keine Zeit bleiben, sodass Sie sich auf einige wenige beschränken müssen. Dabei helfen Prioritäten.



Lassen Sie Ihre maßgeblichen Stakeholder die gefundenen Szenarien nach ihrem jeweiligen geschäftlichen Nutzen oder Wert priorisieren. Ich bevorzuge dabei eine kleine Skala, etwa A = wichtig, B = mittel, C = weniger wichtig.

Machen Sie deutlich, dass es bei den Prioritäten lediglich um die Reihenfolge bei der Bewertung geht! Auch ein für die Bewertung mit C priorisiertes Szenario wird das Entwicklungsteam liefern – sofern irgendwie möglich!

Architektur hinsichtlich der Szenarien analysieren

Mit den (priorisierten) Szenarien für die aktuellen Qualitätsanforderungen und Architekturziele besitzen Sie nun einen Bewertungsmaßstab für Ihre konkrete Architektur. Jetzt folgt die eigentliche Bewertung, die Sie in der Regel in einer kleinen Gruppe gemeinsam mit den Architekten des Systems durchführen. Weitere Stakeholder sind dabei im Normalfall nicht anwesend.⁶

Gehen Sie bei dieser Kernaufgabe gemäß den Prioritäten der Szenarien vor, beginnend mit den wichtigen und schwierigen! Lassen Sie sich in Form eines *Walkthrough* von Architekten oder Entwicklern erläutern, wie die Bausteine des Systems zur Erreichung dieses Szenarios zusammenspielen oder welche Entwurfsentscheidungen das jeweilige Szenario unterstützen.



Als Bewerter oder Auditor einer Architektur untersuchen Sie die zentralen Architekturziele und Qualitätsanforderungen auf Basis der jeweils definierten Szenarien gemeinsam mit dem Architekten des Systems. Lassen Sie sich die zugehörigen Architekturentscheidungen und -ansätze erläutern und beantworten Sie die folgenden Fragen:

- Welche Architekturentscheidungen wurden zur Erreichung eines Szenarios getroffen?
- Welcher Architekturansatz unterstützt die Erreichung des Szenarios?
- Welche Kompromisse ging man mit dieser Entscheidung ein?
- Welche anderen Qualitätsmerkmale oder Architekturziele beeinflusst diese Entscheidung darüber hinaus?
- Welche Risiken erwachsen aus dieser Entscheidung oder aus diesem Ansatz?
- Welche Risiken gibt es für die Erreichung des jeweiligen Szenarios und der damit verbundenen Qualitätsanforderungen?
- Welche Analysen, Untersuchungen oder Prototypen stützen diese Entscheidung?

Leider muss ich Ihnen an dieser Stelle eine schlechte Nachricht überbringen: Es gibt keinen Algorithmus, der für Sie die Zielerreichung einer Architektur hinsichtlich der Szenarien bestimmt. Ihre Erfahrung (oder die des Bewertungsteams) ist gefragt – auch Ihre subjektive Einschätzung.

Als Resultate erhalten Sie einen Überblick über die Güte der Architektur hinsichtlich konkreter Szenarien, und damit auch hinsichtlich der spezifischen Architektur- und Qualitätsziele. Sie finden heraus,

Ergebnisse:
Risiken + Kompromisse

- welche **Risiken** bezüglich der Erreichung oder Umsetzung wesentlicher Qualitätsanforderungen oder Architekturziele existieren;
- welche Szenarien auf jeden Fall (d. h. risikolos) erreicht werden; ([Clements+03] spricht hier von *Non-Risks*);
- welche **Kompromisse** bei Entwurf und Entwicklung eingegangen wurden (beispielsweise wenn zugunsten hoher Performance die Komplexität gesteigert und die Änderbarkeit reduziert wurden).

⁶ Ziehen Sie andere Beteiligte hinzu, wenn es während der Bewertung Rückfragen zu den Szenarien (= Qualitätsanforderungen) gibt!

Krönender Abschluss: Maßnahmen definieren

Offiziell gehört es nicht mehr zu ATAM – aber zu jeder realen Bewertung: die Formulierung von Maßnahmen, Strategien oder Taktiken gegen die gefundenen Risiken.

Das ist ganz normale Architektur- und Entwicklungsarbeit – nur können Sie jetzt auf Basis sehr spezifischer Anforderungen (= Szenarien + erkannte Risiken) konstruktive Entscheidungen treffen.

Positive Nebenwirkungen von Architekturbewertungen

Neben den wertvollen Aussagen hinsichtlich Zielerreichung und Risiken bringen qualitative Architekturbewertungen oftmals eine Reihe positiver Nebeneffekte hervor.

- Die maßgeblichen Stakeholder diskutieren über ihre eigenen (Qualitäts-)Ziele.⁷ Wesentliche Anforderungen werden dadurch präzisiert – sehr zur Freude der Entwicklungsprojekte.
- Zentrale Architekturentscheidungen werden offengelegt und hinsichtlich ihrer Risiken und eingegangenen Kompromisse transparent.
- Dokumentation wird besser, weil die Bewertung einzelner Szenarien ohne Dokumentation häufig kaum durchführbar ist.

Schließlich kommt es vor, dass maßgebliche Stakeholder sich des hohen Risikos bewusst werden, das durch einige ihrer Anforderungen und Qualitätsziele entsteht.



Bewerten Sie die Architektur so früh wie möglich. Die Ergebnisse qualitativer Architekturbewertung besitzen für Systeme und Projekte meistens langfristigen Wert. Investition in Architekturbewertung lohnt sich.⁸

⁷ Diese Diskussion der Stakeholder hätte bereits in der Analysephase stattfinden sollen. Leider besteht in der Realität hinsichtlich Architekturzielen und Qualitätsanforderungen meistens weder Klarheit noch Einigkeit.

⁸ Zumal auch kurze Bewertungsworkshops von wenigen Stunden Dauer schon kritische Risiken in Architekturen aufdecken können – erheblich schneller als umfangreiche Code- oder Schwachstellenanalysen.

■ 8.2 Quantitative Bewertung durch Metriken

Es gibt eine ganze Reihe quantitativer Metriken, die Sie für Systeme und Quellcode ermitteln können. Einige Beispiele:

- Für Anforderungen: Anzahl der geänderten Anforderungen pro Zeiteinheit.
- Für Quellcode: Abhängigkeitsmaße (Kopplung), Anzahl der Codezeilen, Anzahl der Kommentare in Relation zur Anzahl der Programmzeilen, Anzahl statischer Methoden, Komplexität (der möglichen Ablaufpfade, *cyclomatic complexity*), Anzahl der Methoden pro Klasse, Vererbungstiefe und einige mehr.
- Sehr spannend: Änderungshäufigkeit von Codestellen sowie
- für Tests und Testfälle: Anzahl der Testfälle, Anzahl der Testfälle pro Klasse/Paket, Anzahl der Testfälle pro Anforderung, Testabdeckung.⁹
- Für ganz oder teilweise fertige Systeme: Performance-Eigenschaften wie Ressourcenverbrauch oder benötigte Zeit für die Verarbeitung bestimmter Funktionen oder Anwendungsfälle.
- Für Fehler: Mittlere Zeit bis zur Behebung eines Fehlers, Anzahl der gefundenen Fehler pro Paket/Subsystem.

Sie sehen, eine ganze Menge von Eigenschaften können Sie quantitativ charakterisieren.

Dennoch: Metriken als Unterstützung der Entwicklung haben sich nach meiner Erfahrung in der Praxis bisher kaum durchgesetzt. Insbesondere messen viele Unternehmen nur punktuell, anstatt Messungen kontinuierlich mit der Entwicklung zu verzahnen.



Beispiel: In einem mittelgroßen Client/Server-Projekt gab der Kunde einem Forschungsinstitut den Auftrag, die etwa 1000 entwickelten und bereits produktiv (und zur Zufriedenheit des Kunden) eingesetzten Java-Klassen durch Metriken zu analysieren. Ziel dieser Untersuchung sollte sein, spätere Wartungsaufwände besser abschätzen zu können.

Im ersten Bericht der Forscher wurde die Qualität der Software heftig kritisiert, weil verschiedene Metriken starke Abweichungen vom wissenschaftlichen Ideal zeigten.

Kunde, Architekt, Entwickler und Projektleiter waren entsetzt – die Software funktionierte einwandfrei, dennoch wurde die Qualität bemängelt. Damit war die Akzeptanz von Softwaremetriken im Entwicklungsteam auf ein Allzeittief gesunken.

Übrigens stellte sich später heraus, dass ein übereifriger Forscher neben den Klassen der Anwendung den gesamten GUI-Framework sowie die zugekaufte Middleware mit analysiert hatte und die bemängelten Metrikdefizite ausschließlich in diesen Frameworks lagen.

⁹ Testabdeckung ist eine verbreitete Metrik, jedoch auch eine sehr kritische. [Binder2000] und [Marick97] beschreiben einige dieser damit verbundenen Probleme im Detail.

Metriken über Zeit beobachten

Einige Ratschläge zum Einsatz von Metriken, insbesondere Codemetriken:



- Metriken können gute Hinweise für strukturelle Veränderungen von Software geben – über die Funktionsfähigkeit und Qualität zur Laufzeit sagen Codemetriken hingegen nichts aus. Setzen Sie Metriken daher kontinuierlich ein und beobachten Sie deren Veränderungen über die Zeit.
- Metriken benötigen einen fachlichen und technischen Kontext, um vergleichbar zu sein. Sammeln Sie Daten aus vergleichbaren Systemen.
- Metriken können bei unvorsichtiger Anwendung wichtige strukturelle Aussagen über Entwürfe im Zahlendschungel verbergen. Daher: Minimieren Sie die Anzahl der Metriken und konzentrieren Sie Analysen auf überschaubare Ausschnitte. Weniger ist oft mehr!
- Insbesondere sollten Sie Abhängigkeiten im Quellcode durch ein Werkzeug bei jedem Build¹⁰ messen und überwachen.

Systeme nur auf Basis von Quellcodemetriken zu bewerten, ist riskant, weil grundlegende *strukturelle* Schwachstellen dadurch möglicherweise überhaupt nicht aufgedeckt werden. So kann es vorkommen, dass Sie qualitative Defizite erst spät im Entwicklungsprozess entdecken.¹¹ Möglicherweise notwendige Änderungen der Architektur sind dann in der Regel teuer und aufwendig. Daher möchte ich mich auf den folgenden Seiten auf die qualitative Bewertung von Architekturen konzentrieren.¹²

Messungen am laufenden System liefern nur Indikatoren

Metriken können jedoch zur Systemlaufzeit (etwa: Performance-Werte, Ressourcenverbrauch oder auch Fehlerzahlen) wertvolle Indikatoren für Probleme liefern. In jedem Fall müssen Sie auf Basis solcher Messwerte noch *Ursachenforschung* betreiben!

Metriken können Ihnen zeigen, ob Sie Ihren Code gut geschrieben haben. Ob Sie allerdings den richtigen Code haben, können Sie nur durch qualitative Analysen herausfinden.



Beginnen Sie bereits in frühen Entwicklungsphasen mit der Erfassung von Laufzeitmetriken, insbesondere Performance-Daten (etwa: Durchlaufzeiten wichtiger Anwendungsfälle und Ressourcenverbrauch). Ermitteln Sie diese Werte automatisiert und achten Sie auf Trends.

¹⁰ Sie haben doch hoffentlich einen Daily Build etabliert, inklusive täglicher Durchführung sämtlicher Unit-Tests, oder? Solche Werkzeuge (wie Jenkins, Hudson, Gradle oder Maven) können Metriken mit jedem Build berechnen.

¹¹ Iterative Entwicklungsprozesse mit frühzeitigem Feedback vermeiden dieses Problem.

¹² Ich rate Ihnen aber, in Entwicklungsprojekten Codemetriken zur Steigerung der Qualität einzusetzen. Sie sollten in jedem Fall Komplexitäts- und Abhängigkeitsmaße auf Codeebene messen und überwachen – viele Entwicklungsumgebungen unterstützen Sie dabei.

Bewerten Sie insbesondere Architekturen

Von den vielen Artefakten, die innerhalb von Softwareprojekten entstehen, eignen sich Architekturen meiner Meinung nach besonders für die Bewertung: Architekturentscheidungen haben oftmals große Tragweite, sowohl für die zu entwickelnden Systeme als auch für die Projekte und Organisationen, die diese Systeme realisieren, benutzen und betreiben.

Viele dieser Entwurfsentscheidungen müssen Teams oder Architekten unter Unsicherheit oder (vagen) Annahmen treffen – das birgt Risiken! Architekturbewertung hilft, diese Risikofaktoren frühzeitig und spezifisch zu identifizieren.

■ 8.3 Werkzeuge zur Bewertung

Einige Aspekte der Qualität von Systemen können Sie anhand automatisierbarer Untersuchungen des Quellcodes beobachten. Mittlerweile existieren eine Reihe freier (teilweise quelloffener) Werkzeuge, die auch sprachübergreifende Codequalität messen und analysieren können.

Als Beispiel möchte ich Ihnen die SonarQube[®]-Plattform¹³ skizzieren: Frei nutzbar verfügt sie über Plug-ins für alle erdenklichen Programmiersprachen (Java- und C#, C/C++, Groovy, Scala, Erlang, Drools, PL/SQL, Cobol u. v. m.). SonarQube kann:

- Untersuchungsergebnisse in einer Datenbank speichern und die Veränderungen über die Zeit in einer sogenannten „Time Machine“ sichtbar machen. Ein großartiges Instrument, um in Verbesserungs- oder Refactoring-Projekten Erfolge darzustellen!
- doppelten und ungenutzten Code identifizieren;
- Einhaltung von Coding-Styles prüfen;
- Komplexität im Code messen;
- durch seine Client/Server-Architektur in viele Build-Systeme integriert werden;
- relativ einfach über seine offene Plug-in-Schnittstelle um spezielle Prüfungen erweitert werden.

Es gibt alternative Werkzeuge (z. B. PMD, FindBugs, CheckStyle, Squal, Structure101, SotoGraph) – allerdings schützen Sie diese Werkzeuge natürlich nicht davor, den falschen Code richtig zu entwickeln. Grüne Ampeln bei Werkzeugen bedeuten daher nicht unbedingt glückliche Benutzer oder Stakeholder. Dennoch: Meiner Ansicht nach sollten diese Werkzeuge zum Handwerkszeug jeglicher Softwareteams gehören.

Interessant kann auch der CodeMaat-Ansatz von Adam Tornhill sein, der die Historie von Quellcode (aus der Versionsverwaltung) mit dessen Komplexität korreliert und daraus kritische Hotspots von Code ableitet. Siehe <http://www.adamtornhill.com/>.

¹³ <https://sonarqube.org>

■ 8.4 Weiterführende Literatur



[Bass+12] und [Clements+03] beschreiben die ATAM-Methode (*Architecture Tradeoff Analysis Method*) ausführlich. Meiner Erfahrung nach ist diese Methode jedoch durch die wiederholte Bewertung der Architekturansätze in vielen Fällen zu aufwendig in der Anwendung. Trotzdem legt sie die Grundlage für den Ansatz der szenariobasierten Bewertung.

[Kruchten+02] fassen den State of the Art bezüglich der Architekturbewertung zusammen; viele Hinweise, wie eine Bewertung praktisch ablaufen kann.

Systematische Verbesserung und Evolution

Anstatt Neuentwicklung „auf der grünen Wiese“ prägen Pflege, Änderung und Erweiterung bestehender Systeme den Alltag vieler Softwareentwickler und -architekten. Leider beschäftigt sich die Informatikausbildung immer noch zum großen Teil mit der Neuentwicklung – und ignoriert damit das große Thema „langfristige Sicherung der Qualität“ bei Änderungen.

Praktische Erfahrung zeigt: „Systeme degenerieren mit der Zeit“. Anders ausgedrückt: Was gestern noch gut oder einfach war, ist durch Änderungen am System heute möglicherweise mangelhaft oder kompliziert geworden. Was gestern mit geringem Aufwand implementiert werden konnte, braucht heute unangemessen lange und ist teuer.

Systeme degenerieren mit der Zeit

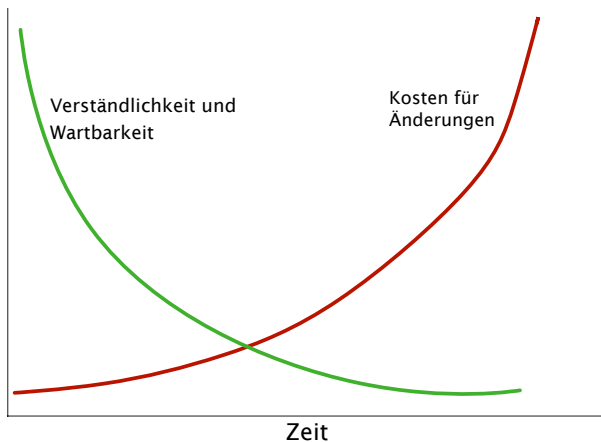


Bild 9.1
Ist-Situation vieler Systeme

Symptome degenerierter Systeme

Ich möchte kurz erklären, was die Symptome oder Konsequenzen solcher Degeneration sein können:

- Entwicklungsteams verlieren immer mehr den Überblick: Sie können Konsequenzen einzelner Änderungen oder Korrekturen nicht mehr abschätzen.
- Die vorhandenen (automatisierten) Tests reichen nicht (mehr) aus, um ungewollte Konsequenzen von Änderungen zu erkennen. Änderungen werden aufgrund ihrer unerkannten Seiteneffekte somit riskant (*ripple effect*).
- Probleme im produktiven Betrieb des Systems (Laufzeitfehler, Abstürze etc.) häufen sich. Ihre Behebung nimmt immer mehr Zeit und Ressourcen in Anspruch.

- Teams können notwendige und angeforderte Qualitätseigenschaften des Systems (Performance, Sicherheit, Robustheit etc.) nicht mehr sicherstellen.
- Änderungs-, Wartungs- und Betriebskosten des Systems steigen immer mehr an. Änderungen und Fehlerbehebungen werden immens teuer.
- Die Zeit bis zur produktiven Einführung (fachlich notwendiger) Features (*time-to-market*) steigt kontinuierlich an. Die Zufriedenheit von Fachbereichen und Auftraggebern sinkt.

Bestimmt möchten Sie (auch!) nicht in solchen Situationen stecken. Ich behaupte, dass die oben genannten Probleme sich in ähnlicher Form in sehr vielen produktiv genutzten IT-Systemen wiederfinden ...

Auf den folgenden Seiten möchte ich Ihnen einen systematischen Weg aus solchen Situationen aufzeigen: Entgegen mancher Gerüchte sind methodische Verbesserung und Modernisierung durchaus möglich!

Vorher möchte ich Ihnen einige mögliche Gründe und Ursachen dieser misslichen Situation aufzeigen.

Gründe für degenerierte Systeme

Ich beschränke mich hier auf Gründe und Ursachen, die mir in realen Projekten bereits begegnet sind:

- Übermäßige und ungewollte strukturelle Komplexität, umständliche Abläufe, Konzepte oder Konventionen innerhalb des Systems.
- Mangelnde konzeptionelle Integrität: Identische Probleme werden innerhalb eines Systems unterschiedlich gelöst, es gibt mehrere unterschiedliche, teils widersprüchliche Lösungsansätze.
- Schlechter Quellcode, der beispielsweise unter zu hoher Kopplung oder niedriger Kohäsion leidet, gegen etablierte Programmierkonventionen verstößt, zu umfangreich oder schlicht unverständlich ist.
- Zum fachlichen Problem unpassende Wahl von Technologien, Frameworks oder sonstiger Infrastruktur. Hiervon kenne ich zwei Ausprägungen:
 - „mit Kanonen auf Spatzen schießen“ und
 - Technologien missbräuchlich außerhalb ihres vorgesehenen Einsatzzwecks verwenden (etwa mit Excel-Makros eine Produktionsanlage steuern ...).
- Hohe Komplexität oder Probleme innerhalb der zum System gehörenden Entwicklungs-, Betriebs- oder auch Managementprozesse: Zu lange oder intransparente Entscheidungswege, fehlende fachliche oder technische Kompetenz, politische Querelen, sparen-am-falschen-Ende ... bitte führen Sie diese Liste selbst weiter ☺



Viele Speditionen und andere Unternehmen betreiben sogenannte Fuhrparks – Flotten von Kraftfahrzeugen, die sie für ihre Wertschöpfung unbedingt benötigen. Viele dieser Unternehmen investieren kontinuierlich in die Erhaltung der Betriebstauglichkeit ihrer Flotte – in die sogenannte *präventive Wartung*.

Beispielsweise finden regelmäßig Inspektionen und Ölwechsel statt – auch wenn für ein bestimmtes Fahrzeug der Ölstand noch ein paar Kilometer länger reichen könnte ...

Rein statistisch hat sich gezeigt, dass die Zuverlässigkeit von Fahrzeugen deutlich höher liegt, wenn z. B. Ölwechsel und andere Wartungsarbeiten zuverlässig regelmäßig stattfinden.

Kurzfristig könnte ein Unternehmen Geld dadurch sparen, in einem Jahr auf sämtliche Ölwechsel zu verzichten (und die gesparten Wartungskosten dann als zusätzlichen Gewinn ausschütten).

Damit steigt allerdings das Risiko, dass Fahrzeuge dieses Unternehmens mit Motorschäden auf der Strecke bleiben und beispielsweise ihre Fracht nicht rechtzeitig bei den Empfängern abliefern können. Das wiederum führt zu unzufriedenen Kunden und langfristig zu weniger Umsatz.

Mein Rat: Investieren Sie auch in die präventive Wartung von Software. Regelmäßiges Refactoring und sonstige „Aufräumarbeiten“ sparen auf lange Sicht Wartungs- und Änderungskosten.

Übrigens versteht auch Ihr Management das Beispiel mit dem Fuhrpark – habe ich selbst ausprobiert 😊

(Die Idee zu diesem Beispiel stammt aus dem fantastischen Buch „The Phoenix Project“ [Kim+13].)

■ 9.1 Wege in den Abgrund

Zu Beginn vieler Entwicklungsprojekte sieht Software doch recht ordentlich aus: Motivierte Teams erarbeiten stimmige Konzepte und setzen diese dann handwerklich sauber um. Konzepte und Technologien passen zum Problem, ebenso die gewählten Datenstrukturen.

Im Laufe der Zeit jedoch beginnt das Drama: Die ursprüngliche Ordnung weicht einem ständig steigenden Chaos. Ausnahmen werden zur Regel, Code wird immer schwerer verständlich, das Entwicklungsteam beginnt von „historisch gewachsen“ zu sprechen – weil kaum jemand nachvollziehen kann, wie es zu dieser kontinuierlichen Verschlechterung kommen konnte.

Durch welche Gründe kommt es dazu? Verlernen Teams über die Zeit das vernünftige und saubere Arbeiten? Nein – vielmehr zwingen oftmals äußere Einflüsse zu Kompromissen, die dann langfristig zu den oben genannten Problemen führen. Neudeutsch nennen viele Beteiligte dieses Phänomen „technische Schulden“ – Softwareentwickler bezeichnen diese als Quick-And-Dirty, Hot-Fix, oder Workaround – und wissen, dass sich dahinter meistens ein weiterer Schritt in Richtung degenerierter Systeme verbirgt.

Innere Werte werden vernachlässigt

Ursächlich resultieren diese Probleme in mangelndem Bewusstsein oder fehlender Zeit für die Bewahrung der *inneren Qualität* von Software. Auftraggeber interessieren sich primär für neue Features, Erweiterungen oder schnellstmögliche Fehlerbehebung, weniger für den Erhalt konzeptioneller Integrität, verständlichen Quellcode oder stimmige Anwendung von Konzepten oder Technologien. Nun stellt aber gerade die *innere Qualität* von Systemen sicher, dass Ziele wie niedrige Änderungskosten, schnelle Fehlerbehebung, hohe Anpassbarkeit zuverlässig erreicht werden.

■ 9.2 Systematisch verbessern

Der Weg zu besserer Software führt in der kommerziellen Realität immer über mehrere, meist kleine, Schritte zum Ziel. Praktisch niemals können wir den Lauf der Welt für längere Zeit anhalten, um in dieser Zeit die Qualität unserer Software zu verbessern. Vielmehr müssen wir jegliche Verbesserungen in reguläre Wartungsarbeiten (etwa: Erweiterungen, Fehlerkorrekturen) integrieren.



Ich möchte Ihnen nachfolgend **aim⁴²** vorstellen¹, eine Methode zur systematischen Verbesserung von Software, die ich gemeinsam mit einem engagierten Team als Open-Source-Projekt ständig weiterentwickle (siehe <http://aim42.org> und <http://aim42.github.io>).

aim⁴² adressiert Business und Technik gleichermaßen

aim⁴² stellt sicher, dass die wirklich relevanten Probleme durch Maßnahmen abgedeckt werden – und nicht vorschnell an Themen optimiert wird, die nur geringen geschäftlichen Nutzen erbringen. Daher finden Sie in den zentralen Begriffen von **aim⁴²** neben „Problem“ und „(Lösungs-)Maßnahme“ auch „Kosten“ und „Risiken“.

Probleme verursachen Kosten

Ein Grundprinzip² von **aim⁴²** ist das explizite Ermitteln der durch Probleme (*issues*) verursachten Kosten.

Um Verbesserungen effektiv an realen Bedürfnissen auszurichten, sollten Sie für alle erkannten Probleme mit Ihrem System oder mit den beteiligten Prozessen explizit deren „Kosten“ ermitteln: Welche Kosten entstehen beispielsweise durch direkt zurechenbaren Mehraufwand bei Entwicklung oder Betrieb, wie viel Geld kostet zusätzlich benötigte Hardware, welcher Umsatzausfall oder welche Opportunitätskosten fallen durch enttäuschte Kunden oder entgangene Verkäufe an? Zu den Kosten ermitteln Sie, in welcher Frequenz sie anfallen – etwa bei jeder Änderung, bei jedem Release des Systems oder bei jedem Aufruf einer bestimmten Systemfunktion.

¹ aim steht für Architecture Improvement Method.

² In der Softwareentwicklung ist es ansonsten üblich, die Schätzung von Kosten und Aufwänden auf „Maßnahmen“ zu beschränken!

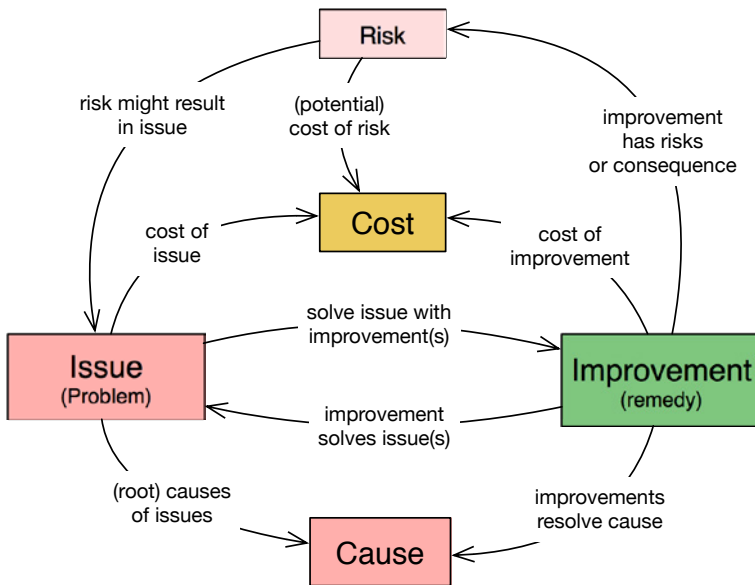


Bild 9.2 „Domänenmodell“ systematischer Verbesserung (nach AIM⁴²)

Bild 9.2 fasst die wesentlichen Kernbegriffe systematischer Verbesserung zusammen. Einige Anmerkungen dazu:

- Probleme (*Issues*) und Lösungsmaßnahmen (*Improvements*) stehen in m:n-Verhältnis zueinander: Mit einer einzelnen Maßnahme könnten Sie möglicherweise gleich mehrere Probleme (teilweise) lösen, andererseits braucht die Lösung eines einzelnen Problems vielleicht mehrere unterschiedliche Maßnahmen.
- Sowohl Probleme wie auch Lösungsmaßnahmen verursachen Kosten.
- Lösungsmaßnahmen enthalten Risiken – beispielsweise kann die Umsetzung der Maßnahme neue Probleme schaffen.
- Teilweise stellen Probleme nur das Symptom einer tiefer liegenden Ursache (*root cause*) dar. Das träge Verhalten der GUI eines Systems hat vielleicht seine Ursache in einer schlecht konfigurierten Datenbank oder einem Netzwerkproblem.

Mit diesem Vokabular ausgestattet kann ich Ihnen nun das *big picture* der systematischen Verbesserung, Modernisierung oder Evolution von Softwaresystemen näherbringen.

Schematisches Vorgehen

In jedem Fall gilt: Verbessern Sie iterativ. Trennen Sie die *Analyse* (d. h. Erkennung von Problemen oder Risiken) von ihrer Bewertung (*Evaluierung*) sowie ihrer Behebung (*Improvement*). Aus der Vogelperspektive teilt AIM⁴² die systematische Verbesserung in drei Phasen ein und ergänzt das Ganze durch einige übergreifende Aufgaben:

1. **Analyze:** Aufdecken und Sammeln von Problemen. Sammeln Sie zuerst die Probleme, die Sie rund um das System und dessen Organisation finden. AIM⁴² nennt das *issue list*³.

³ Sie dürfen auch „Problemliste“ dazu sagen – aber „Issue“ klingt für mich etwas freundlicher ...

In dieser Phase sollten Sie auch Verständnis für die Organisation, Struktur und Lösungskonzepte des Systems gewinnen, weil das für die Erkennung und Entwicklung passender Maßnahmen fundamental ist (siehe Schritt 4 – Crosscutting).

2. **Evaluate:** Schätzen oder ermitteln Sie den (Geld-)Wert von Problemen wie auch Maßnahmen (hinsichtlich einmaliger und/oder wiederholt auftretender Kosten). Erst diese Bewertung ermöglicht zielgerichtete Priorisierung und Planung, weil sie die reinen Probleme und Maßnahmen um relevante Steuerinformationen und KPIs für das IT- und technische Management anreichert.
3. **Improve:** Durchführen von Verbesserungsmaßnahmen. Hier fasst AIM⁴² bewährte Patterns und Praktiken von Evolution, Refactoring, Migration oder stufenweisem Umbau zusammen.
4. **Crosscutting:** Suchen Sie über die drei Phasen hinweg nach Maßnahmen, die konkrete Probleme oder deren Ursachen lösen oder beheben. Sammeln Sie Vorschläge für solche Maßnahmen – AIM⁴² nennt das *improvement backlog*. Planen Sie sowohl Ihre längerfristige Verbesserungsstrategie wie auch Ihre kurzfristigen Maßnahmen.



Bild 9.3
Phasen der Verbesserung

AIM⁴² bettet diese Phasen in ein iteratives Vorgehen ein – weil Verbesserung ein kontinuierlicher Prozess ist. Innerhalb der Iterationen wiederholt sich folgende gedankliche Schleife immer wieder: (1) Sie finden ein Problem oder dessen Ursachen, (2) identifizieren Lösungsmaßnahmen dafür, (3) bewerten Probleme und Maßnahmen hinsichtlich ihrer Kosten. (4) Sie wenden die Lösungsmaßnahme an und (5) analysieren, ob das ursprüngliche Problem damit behoben ist.

Solche Schleifen⁴ laufen in ähnlicher Form⁵ in jeglicher Art von Konstruktions- oder Optimierungsprozessen ab. Nach meiner Erfahrung kranken viele Systeme in der Praxis daran, dass die technisch Verantwortlichen vor der Bewertung der Kosten von Problem und Maßnahmen zurückschrecken. Stattdessen argumentieren sie über „Prinzipien der Softwaretechnik“ oder „technische Konzepte“ – und scheitern damit bei Management oder Auftraggebern.

Kosten von Problemen und Lösungsmaßnahmen gegenüberstellen

Die Gegenüberstellung von Kosten von Maßnahmen sowie Kosten des Problems ergibt wertvolle Entscheidungshilfe für Budget- oder fachlich Verantwortliche. Damit können Sie als Softwarearchitekten endlich in *Business-Sprache* argumentieren und brauchen mit Ihren fachlichen Stakeholder nicht mehr über die schwer vermittelbaren inneren Qualitäten, Kopplung, Kohäsion oder Implementierungsdetails sprechen.

⁴ Möglicherweise kennen Sie ähnliche Schleifen auch als Deming-Kreis oder PDCA-Schleife (plan-do-check-act, siehe <http://de.wikipedia.org/wiki/Demingkreis>).

⁵ Wenn nicht explizit, dann implizit und unbewusst im Kopf.

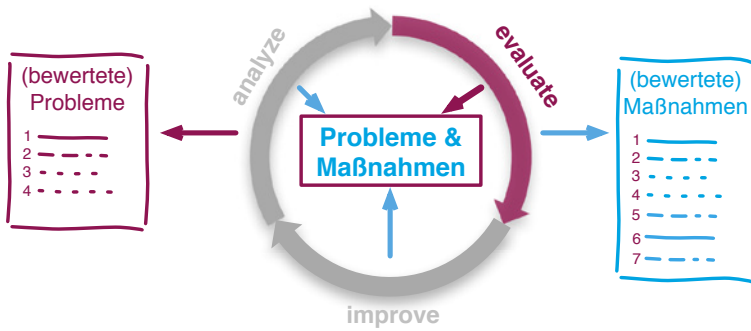


Bild 9.4 Iterativ kontinuierlich verbessern

Bewertungen ändern sich

Bewertungen von Problemen und Maßnahmen können sich über die Zeit ändern, wie sich in modernen Entwicklungsprozessen auch die Prioritäten von beispielsweise Anforderungen oder Zielen über die Zeit ändern können. Nur regelmäßige (iterative) Überprüfung der Probleme (*issues*) und der Maßnahmenvorschläge (*improvements*) stellen deren Aktualität sicher.

Kontinuierlich Verbesserungsmöglichkeiten suchen

Den aus meiner Sicht unbedingt notwendigen *Improvement Backlog* habe ich oben schon erwähnt – die Sammlung von Möglichkeiten, Optionen, Taktiken oder auch konkreten Maßnahmen, mitsamt einer (möglicherweise groben) Bewertung sowie konkretem Bezug auf die adressierten Probleme.

Eine meiner persönlichen Grundregeln für die Suche nach solchen Verbesserungsmöglichkeiten lautet „Verbreite den Suchraum“ (engl. Widen-Your-Options). Als Softwareentwickler tendieren wir manchmal dazu, Verbesserungen primär im Quellcode von Systemen zu suchen, weil wir uns genau dort am besten auskennen. Dabei gibt es oft noch ganz andere Möglichkeiten: Wir könnten den betroffenen Programmteil austauschen, kapseln, die Anforderung wediskutieren, das Problem jemand anderem geben, bessere Hardware kaufen oder oder ... In Abschnitt 9.6.2 zeige ich Ihnen einige Kategorien von Verbesserungsmaßnahmen, die Sie hoffentlich auf weitere Lösungsideen für Ihre konkreten Probleme bringen.

■ 9.3 Bewährte Praktiken und Muster

AIM⁴² nutzt etablierte Vorgehensmuster und Praktiken, anstatt Räder neu zu erfinden. Ein wesentlicher Mehrwert der AIM⁴²-Methode liegt darin, diese Praktiken in ein gesamthaftes Vorgehen einzuordnen.

Das ausführliche Methodenreferenz (method guide) von AIM⁴² katalogisiert zurzeit (April 2015) gute 80 solche Praktiken und Muster – einige Beispiele finden Sie im Textkasten bzw. in den folgenden Abschnitten.

Allerdings garantiert der Einsatz solcher Praktiken keinen Erfolg: Wie andere mächtige Werkzeuge auch gehört zur Anwendung dieser Praktiken Fingerspitzengefühl und etwas Erfahrung. Zwar finden Sie in der Methodenreferenz zu AIM⁴² viele Hinweise über *wenn, wann und wie*, aber das sind eher Heuristiken und keine Algorithmen.



Anhand einiger Beispiele möchte ich die Granularität der AIM⁴²-Praktiken aufzeigen. Für eine ganzheitliche Übersicht verweise ich Sie auf die Online-Methodenreferenz [aim-guide].

ANTICORRUPTION-LAYER: Isoliere Clients von internen Änderungen an Subsystemen.

ATAM: Architecture Tradeoff Analysis Method, findet Risiken bezüglich der Erreichung von Qualitätszielen (siehe Kapitel 8).

CAPTURE-QUALITY-REQUIREMENTS: Qualitätsziele verschiedener Beteiligter offenlegen.

CONTEXT ANALYSIS: Systemkontext und externe Schnittstellen klären und dokumentieren.

EXTRACT REUSABLE COMPONENT: Aus bestehendem System wiederverwendbare Teile extrahieren.

FRONTEND-SWITCH: In der Benutzeroberfläche Anfragen entweder an alte oder veränderte Teile des Systems routen.

HIERARCHICAL-QUALITY-MODEL: Qualitätsziele konkretisieren durch Qualitätsbaum mit Szenarien

IMPROVEMENT-BACKLOG: Pflege einer Übersicht möglicher Verbesserungsmaßnahmen, inklusive ihrer Kosten und Risiken.

ISSUE-TRACKER-ANALYSIS: Bug- oder Issue-Tracker bezüglich Cluster und Häufungen untersuchen.

KEEP-DATA, TOSS-CODE: Systeme durch massive Änderungen im Code, aber unter Beibehaltung der relevanten Daten verbessern.

PRE-INTERVIEW -QUESTIONNAIRE: Interviews durch stakeholder- und situationsspezifische Fragebögen vorbereiten.

PROFILING: Vermesse Speicher- und anderen Ressourcenverbrauch zur Laufzeit.

REFACTORING-PPLAN: teamweite Koordination und Abstimmung von Refactoring-Maßnahmen – zur groß angelegten Codeverbesserung.

ROOT-CAUSE-ANALYSIS: Finde die Ursache von Problemen – differenziere Ursache und Symptom.

SOFTWARE-ARCHEOLOGY: Verstehe Software durch Analyse von Quellcode und seiner Versionshistorie.

STATIC-CODE-ANALYSIS: Analysiere Quellcode, um zusammengehörige Bausteine, Abhängigkeiten, Kohäsion und andere strukturelle Eigenschaften zu finden.

Im Folgenden finden Sie eine Einführung in die großen Bestandteile der methodischen Verbesserung – die drei Phasen sowie die phasenübergreifenden Aufgaben.

■ 9.4 Analyse: Probleme identifizieren

Verschaffen Sie sich einen Überblick über das System, dessen fachlichen Zweck sowie wichtigste Qualitätsziele.

Beschaffen (und bei Bedarf dokumentieren) Sie angemessen tiefgreifendes Verständnis

- der internen Strukturen, Konzepte und Architekturansätze sowie
- aller für Entwicklung, Pflege, Betrieb und Management des Systems notwendigen Prozesse und Abläufe.

Identifizieren Sie möglichst viele Probleme, Symptome, Risiken oder technische Schulden des Systems und der beteiligten Prozesse. Versuchen Sie, zwischen Problemen und Ursachen zu differenzieren, die entsprechende Praktik heißt **Root-Cause-Analyse**.

Zur Analyse gehören eine Vielzahl weiterer Praktiken, Methodiken und Vorgehensmuster – Bild 9.5 gibt einen Überblick.

Meiner Erfahrung nach sollten Sie für die Analyse in jedem Fall folgende Praktiken anwenden (ich verwende die englischen Originalbegriffe aus AIM⁴²):

- Beginnen Sie in jedem Fall mit einer **STAKEHOLDER-ANALYSE**, danach führen Sie **STAKEHOLDER-INTERVIEWS** mit den wichtigsten Beteiligten des Systems oder relevanter Prozesse.
- Verbessern Sie Ihr Verständnis des Systems durch **CONTEXT-ANALYSIS** und **DOCUMENTATION-ANALYSIS**. Lesen Sie insbesondere, falls vorhanden, die Architekturdokumentation.
- Hinterfragen Sie beteiligte Prozesse, u. a. durch **DEVELOPMENT-PROCESS-ANALYSIS**.
- Versuchen Sie, Teile des Quellcodes zu verstehen – dabei hilft Ihnen **STATIC-CODE-ANALYSIS**.
- Legen Sie die Qualitätsanforderungen an das System offen – **CAPTURE-QUALITY-REQUIREMENTS**.
- Mit Hilfe qualitativer Architekturbewertung (etwa nach **ATAM**, siehe Kapitel 8) finden Sie mögliche Qualitätsprobleme oder -risiken.
- Untersuchen Sie das System metrisch, etwa durch **RUNTIME-ANALYSIS**. Damit finden Sie unter anderem Speicher-, Threading- und Performance-Probleme.
- Wenn Sie noch mehr Probleme finden möchten oder müssen – kümmern Sie sich um die Daten, die das System erzeugt, speichert oder verwaltet: Untersuchen Sie mit **DATA-ANALYSIS** die Struktur, Größe, Menge oder inhaltliche/konzeptionelle Besonderheiten dieser Daten.

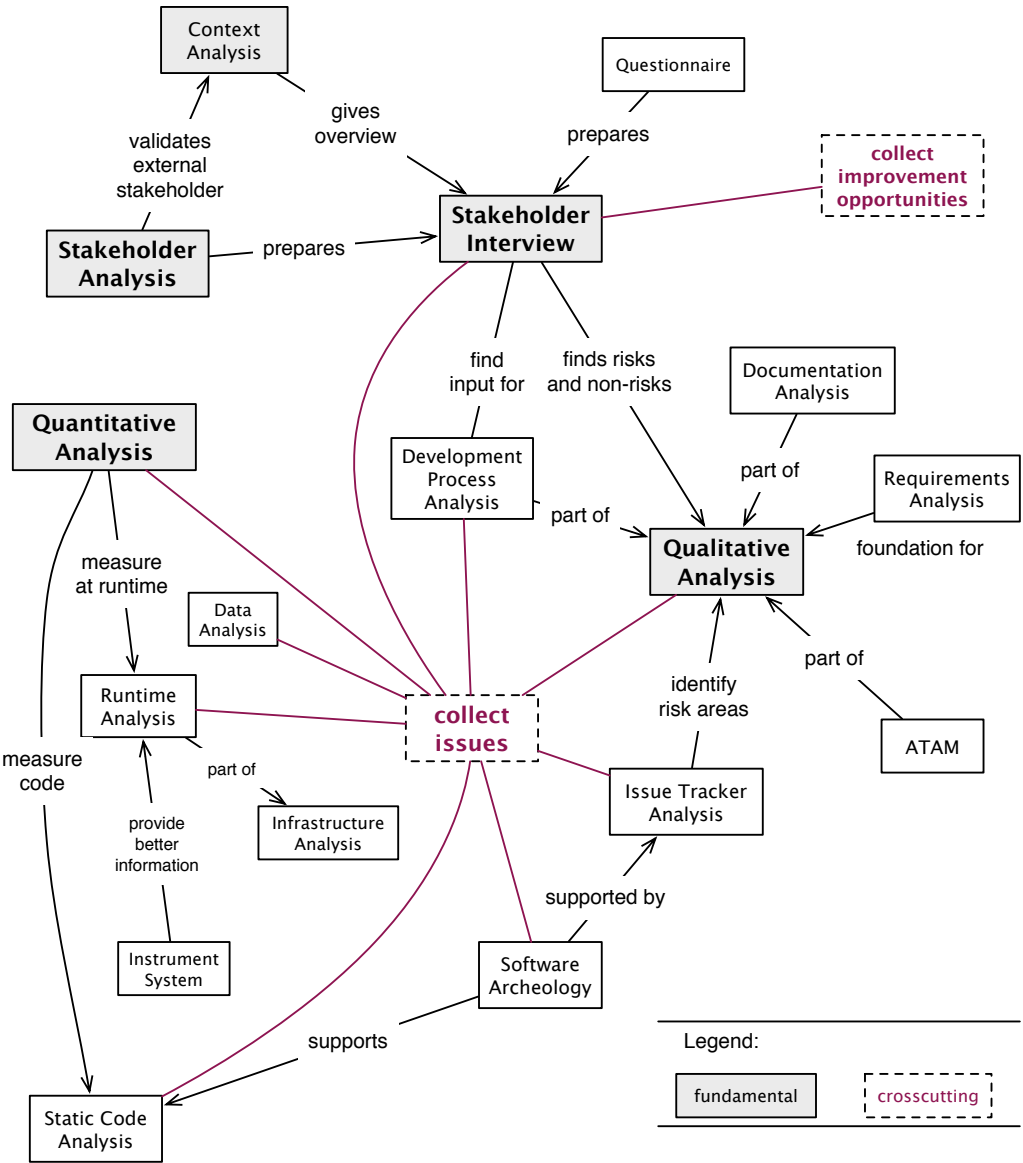


Bild 9.5 Praktiken der Analysephase

■ 9.5 Evaluate: Probleme und Maßnahmen bewerten

Im Wesentlichen bedeutet evaluieren für uns, Kosten oder andere betriebswirtschaftlich relevante Größen von den erkannten Problemen und Maßnahmenvorschlägen zu schätzen. Oftmals höre ich in Projekten „das können wir nicht schätzen ...“ – und dann lange Begründungen, warum eine Schätzung nicht möglich ist: Faule Ausreden – sicherlich *können* Sie das! Damit Probleme und Lösungsvorschläge vergleichbar werden, müssen Sie sich auf das Schätzen einlassen. Der einfache Ausweg heißt: Schätzen Sie grundsätzlich in Intervallen (**Estimate-in-Interval**). Geben Sie untere und obere Grenzen für die betreffenden Werte an, statt sich auf nur eine einzige Zahl festzulegen. Die Breite des Intervalls spiegelt Ihre Unsicherheit wider: Ein kleines Intervall bedeutet „bin mir recht sicher“, ein breites Intervall zeigt hohes Schätzrisiko und unsichere Ergebnisse.

Oftmals ist es schon interessant, die Größenordnung von Kosten oder Aufwänden zu kennen, die durch Probleme oder Maßnahmen verursacht werden – Hauptsache, Sie bekommen beides auf einen neutralen Vergleichsmaßstab (= betriebswirtschaftliche Größe) abgebildet.

Weiterhin sollten Sie die Annahmen, die Ihren Schätzungen zugrunde liegen, grundsätzlich offenlegen. AIM⁴² nennt dies **Explicit-Assumption**.

Schätzen in Intervallen

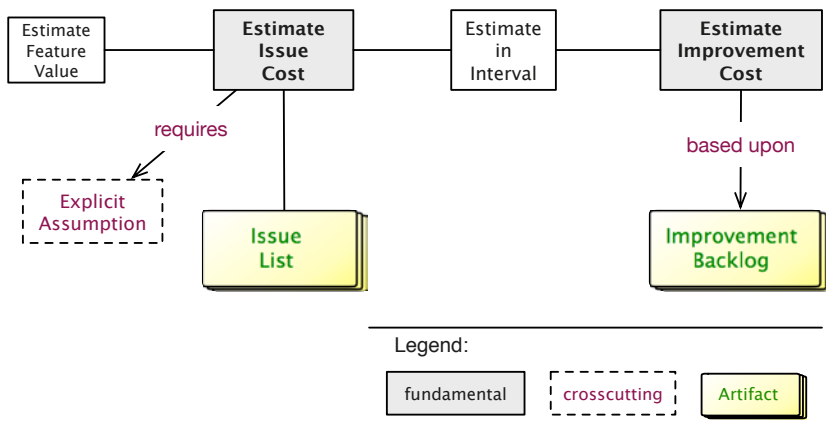


Bild 9.6 Praktiken der Evaluate-Phase

Jetzt besitzen Sie eine solide Übersicht von bekannten Problemen und ersten Lösungsansätzen mitsamt den dadurch verursachten Kosten. Erst eine solche Übersicht ermöglicht es Ihnen und anderen Stakeholdern des Systems, eine angemessene Menge von Maßnahmen für eine Verbesserungsiteration zu wählen und gemäß Ihres üblichen Vorgehens zu planen.

■ 9.6 Improve: Verbesserungsmaßnahmen planen und durchführen

Ich möchte im Folgenden die Vielzahl der möglichen Verbesserungsmaßnahmen durch *Kategorien* strukturieren. Dazu gebe ich Ihnen eine Reihe von Grundregeln (Maxime)⁶ mit auf den Weg, die für praktisch alle Verbesserungs- oder Veränderungsprojekte gelten sollten. Die Kategorien helfen Ihnen dabei, den Suchraum möglicher Maßnahmen zu verbreitern – das hatte ich Ihnen weiter vorne bereits intensiv ans Herz gelegt.

9.6.1 Maxime für Verbesserungsprojekte

Nachfolgend verwende ich die englischen Originalbegriffe aus AIM⁴² als Bezeichner für diese Grundregeln.

Improve Iteratively: Verbessern Sie iterativ. Damit können Sie die Effekte Ihrer Änderungen kurzfristig beobachten – und insbesondere mögliche negative Seiteneffekte leichter kurieren.

Fast Feedback: Beschaffen Sie sich so früh wie möglich Rückmeldungen zu Ihren Änderungen, so dass Sie schon frühzeitig mögliche Kurskorrekturen einleiten können. Dies sollte ein Grundsatz für Ihr gesamtes Verhalten als Softwarearchitekt und -entwickler sein.

Reduce Complexity: Einfachere Lösungen sind leichter verständlich, einfacher zu erweitern, zu ändern und zu betreiben. Auch bekannt als KISS oder das Prinzip der Einfachheit. Entsorgen Sie ungewollte, unnötige (*accidental*) Komplexität in Ihren Systemen.

Prototype Improvement: Verifizieren Sie die Wirkung und Effektivität von Änderungsmaßnahmen im kleinen Maßstab anhand prototypischer Umsetzungen. Dabei können Sie lernen, welche Details Ihnen noch fehlen (*fast feedback*).

Verify Every Change: Überprüfen Sie, ob Ihre Änderungen am System die gewünschten Effekte erzielen und das System in korrektem, funktionsfähigem Zustand hinterlassen. Dies gilt auch für vermeintlich kleine Änderungen, die angeblich „keine Nebenwirkungen besitzen“⁷.

Explicit Assumptions: Legen Sie Ihre Annahmen über unbekannte Dinge (Fakten, Anforderungen, Prozesse, Größen etc.) offen, sofern diese Bedeutung für Ihre Verbesserungsmaßnahmen besitzen.

9.6.2 Kategorien von Verbesserungsmaßnahmen

Setzen Sie einen (iterativen) Prozess zur Verbesserung auf: Entscheiden Sie sich mit den relevanten Beteiligten für grundsätzliche Herangehensweisen: Möchten Sie beispielsweise Teile des Systems schrittweise abschalten (*Legacy Strangulation*), veränderte Teile über *Feature-*

⁶ AIM⁴² nennt die „Fundamentals“, Ralf S. Engelschall hat mir als Alternative den prägnanten Ausdruck „Maxime“ dafür vorgeschlagen.

⁷ Eine großartige Darstellung solcher Fälle anhand konkreter Beispiele hat Gerald Weinberg unter <http://goo.gl/NHJFjl> zusammengefasst.

Toggles oder FRONTEND-SWITCHES ein-/ausschalten oder einzelne Subsysteme im Big-Bang ablösen? Legen Sie mit den Teams eine für alle passende Strategie für die Versionierung von Quellcode fest (etwa: *branch-for-improvement*).

Verbessern Sie Prozesse⁸

Versuchen Sie, in den relevanten Prozessen (etwa: Anforderungserhebung und -management, Test- und Qualitätssicherung, Rollout und Inbetriebnahme, operativer Betrieb) Verbesserungen zu erzielen. Insbesondere klären Sie, warum es überhaupt zu den (aus der Analyse) bekannten Problemen kommen konnte und welche Aktivitäten oder Prozesse diese Probleme verursacht haben.

Um mittel- und langfristig die Architektur von Systemen zu verbessern, benötigen Sie sicherlich kontinuierliche Überprüfung und Management – was ich neudeutsch „Architektur-Governance“ nenne⁹. Dazu zählen sämtliche aus Ihrer Sicht notwendigen Maßnahmen, um die bekannten Probleme sukzessive zu adressieren und eventuelle neue Probleme zeitnah zu erkennen – also relevante Architektur- und Verbesserungsmaßnahmen in Projekten auch durchzusetzen.

Verbessern Sie Codestrukturen

Die Grundthemen hierbei heißen Modularisierung, Restrukturierung sowie Refactoring. Bild 9.7 gibt einen Überblick über einige Praktiken in diesem Bereich. Die Zuordnung ist in manchen Fällen mehrdeutig.

Die eher atomaren Refactoring-Praktiken (**Refactor-Code**) kennen Sie als Entwickler sicherlich zu Genüge: Grundlage sind wirkungsvolle automatisierte Tests, denn ohne ist jegliches Refactoring riskant! Insbesondere in alten Systemen ohne Testautomatisierung bedeutet das hohe Risiken bei Aufräumarbeiten im Quellcode. Sie könnten beispielsweise durch verbesserte Modularisierung diese Risiken lokal halten.

In etwas größerem Maßstab wirken die Praktiken zur Restrukturierung. Hierbei versuchen Sie beispielsweise die Kohäsion von Bausteinen zu verbessern, indem Sie den Schnitt Ihrer Bausteine im Großen verändern, Schichten einführen oder nicht mehr benötigte Teile kennzeichnen oder entfernen.

Verbesserte Modularisierung kann sich positiv auf viele Aspekte von Verbesserungs- oder Veränderungsprojekten auswirken: Änderungen werden lokal(er) durchführ- und testbar und auf mehrere Teams oder Personen parallelisierbar. Sie können Module als Blackbox betrachten und Änderungen am Innenleben damit unabhängiger von anderen Modulen oder Systemen durchführen. Allerdings ist die Einführung klarer Schnittstellen in historisch gewachsenen Quellcode eine schwierige Aufgabe und ohne automatische Unit- oder Integrationstests auch riskant. Klare und inhaltlich sinnvolle Verantwortlichkeiten der einzelnen Bausteine Ihres Systems aus der Kategorie *Restructure-Code* werden hierbei teilweise vorausgesetzt, ansonsten hilft Ihnen **Improve Responsibility** weiter.

Die (frei verfügbare) Sammlung struktureller Verbesserungsmaßnahmen [Demeyer+02] erläutert viele der hier gezeigten Praktiken im Detail.

⁸ Kein reines Architekturthema – aber für Verbesserungsprojekte nach meiner Erfahrung eines der wichtigsten überhaupt!

⁹ Wolfgang Keller hat dazu einen interessanten Vortrag (frei verfügbar) veröffentlicht: <http://goo.gl/yIUL70>.

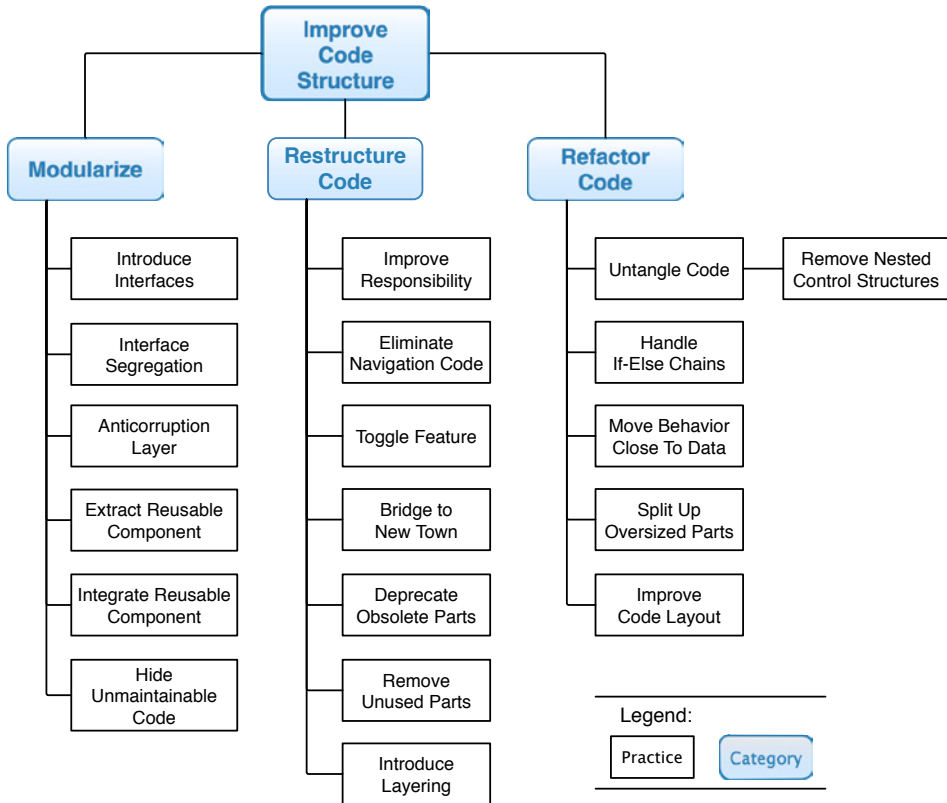


Bild 9.7 Praktiken zur Verbesserung von Codestrukturen

Verbessern Sie querschnittliche und technische Konzepte

Sie könnten damit beginnen, in einem System die Trennung zwischen fachlichen und technischen Belangen zu schärfen, sozusagen die fachliche Kohäsion (siehe Abschnitt 4.1.2) im Nachhinein zu verbessern¹⁰ (**extract business domain**). Als Nebeneffekt ergeben sich meist eine verbesserte Modularisierung und klare Schnittstellen.

Weiterhin können Sie dafür sorgen, dass vorhandene Technologie, wie beispielsweise Frameworks, Bibliotheken oder auch Hardware, effizienter oder sachgerechter eingesetzt wird.

Schließlich könnten Sie nachholen, was ich in Kapitel 3 bereits vorgeschlagen habe – die systematische Ausrichtung Ihrer Entwürfe auf die Erreichung der benötigten und geforderten Qualitätseigenschaften (**quality driven software architecture**). Arbeiten Sie gezielt an einzelnen Qualitätseigenschaften Ihres Systems. Als Grundlage benötigen Sie möglichst konkrete Qualitätsanforderungen, etwa in Form von Szenarien (siehe Abschnitt 8.1 sowie die frei verfügbaren Beispiele aus [arc42-QA]).

Zu dieser Kategorie von Maßnahmen siehe auch Kapitel 7.

¹⁰ Also das, was „Domain-Driven Design“ anstrebt: klare Trennung von fachlichen und technischen Aspekten.

Verbessern Sie die (technische) Infrastruktur

Etwas aufwendiger ist es, gänzlich neue Technologie oder Lösungskonzepte einzuführen – beispielsweise einen selbst geschriebenen OR-Mapper (siehe Abschnitt 7.1) durch ein kommerzielles oder Open-Source-Produkt zu ersetzen, oder auch der Kauf neuer Hardware.

Allgemein könnten Sie hier Teile Ihres Systems durch Fertigteile ersetzen oder frühere *make-or-buy*-Entscheidungen überdenken.

Zu diesem Komplex von Maßnahmen gehört für mich auch die Erhöhung des Automatisierungsgrads innerhalb von Entwicklung und Rollout, also Themen wie Testautomatisierung, *Continuous Integration* oder *Continuous Delivery*.

Verbessern Sie die Analysierbarkeit und Bewertbarkeit

In einigen Reviews und Audits realer Systeme bin ich auf das Problem gestoßen, dass den Beteiligten über das System zu wenige Fakten vorlagen. Weder über Durchsatz, Laufzeiten, Betriebsaufwände, konkrete Wartungsaufwände, *time-to-market*, aufgetretene Fehler und zugehörige Behebungszeiten waren genaue Informationen aufzufinden. Die Systeme und ihre zugehörigen Prozesse waren intransparent und damit nur unzureichend analysier- und bewertbar.

Durch Verbesserung von beispielsweise Logging, Tracing oder auch des betriebswirtschaftlichen Controllings von Entwicklungs- und Betriebsprozessen können Sie mit relativ geringem Aufwand ein gehöriges Maß an Transparenz schaffen.

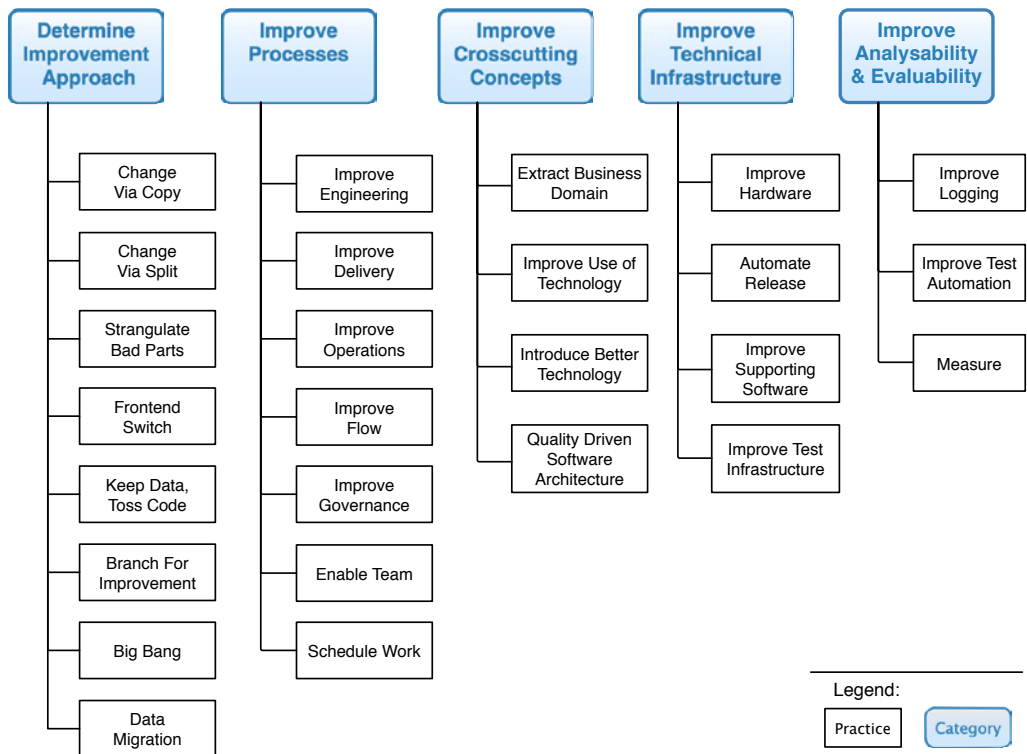


Bild 9.8 Weitere Praktiken zur Verbesserung von Systemen

■ 9.7 Crosscutting: phasenübergreifende Praktiken

Die wichtigste querschnittliche Aufgabe ist es, Maßnahmenvorschläge zu sammeln (und schriftlich festzuhalten). In realen Projekten stoßen Sie möglicherweise dabei gleich noch auf bisher nicht erkannte Probleme oder Risiken – daher gehört auch die Pflege der „issue-list“ zu den querschnittlichen Aufgaben.

Eine kleine Warnung: Verbesserungsprojekte bringen oftmals Veränderungen im System oder in beteiligten Prozessen mit sich. Das stößt erstaunlich oft auf „Gegenwind-aus-Prinzip“ – das entsprechende Muster in AIM⁴² heißt **Expect-Denial**: Rechnen Sie fest damit, dass beteiligte Personen Ihre Maßnahmenvorschläge „aus Prinzip“ ablehnen werden, dass die erkannten Probleme verleugnet werden, dass Risiken als utopisch abgetan werden. Diese manchmal hartnäckige Resistenz gegenüber Veränderungen kann zum Stolperstein von Modernisierungsprojekten werden – wenn Sie nicht proaktiv mit solchen Widerständen umgehen.

Bild 9.9 gibt Ihnen einen Überblick, welche Aufgaben oder Praktiken Sie sonst noch über die bisherigen Phasen hinweg anwenden können.

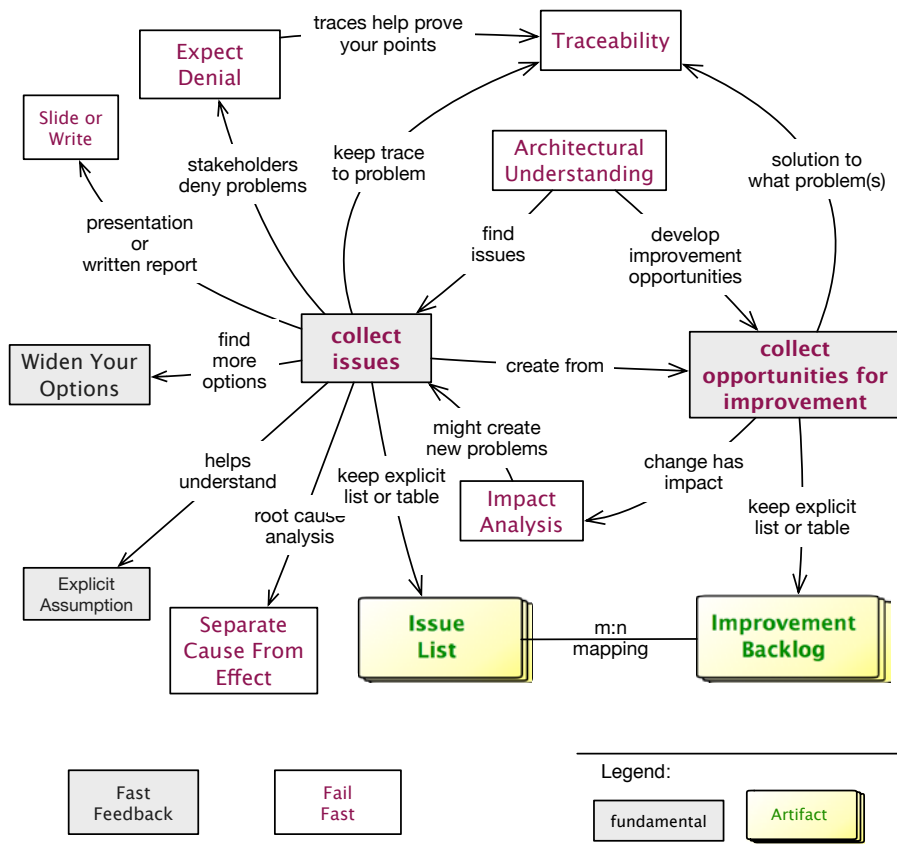


Bild 9.9 Phasenübergreifende Praktiken

■ 9.8 Mehr zu AIM⁴²

Für mehr Details zu den Praktiken empfehle ich Ihnen die ausführliche Methodenreferenz, die AIM⁴² auf der Website <http://aim42.github.io> anbietet.

Die meisten der hier im Buch gezeigten Grafiken finden Sie auch auf dieser Website.

Mitmachen erwünscht

AIM⁴² ist ein junges Open-Source-Projekt, das noch tatkräftige Unterstützung benötigt. Die bisherigen Committer bringen ihre langjährige und reichhaltige Erfahrung ein – und die gesamte Community freut sich über weitere Anregungen und Input.

Für Interessierte: Das Handbuch zur Methodik (der sogenannte method-guide) wird im AsciiDoc¹¹-Format geschrieben, mittels gradle¹² zu html transformiert und automatisiert über einen öffentlichen Build-Server auf <http://aim42.github.io> publiziert. Der gesamte Text liegt unter einer liberalen Creative-Commons-Lizenz auf dem öffentlichen Github-Repository <https://github.com/aim42>.

Werfen Sie einen Blick auf die offenen Punkte, ergänzen Sie Kommentare oder machen Sie Verbesserungsvorschläge.

■ 9.9 Weiterführende Literatur

Auf dem Gebiet der Softwareevolution sieht es leider dürftig aus – die meisten Bücher über Wartung beschränken sich im Wesentlichen auf Refactoring, d. h. bedeutungserhaltende Transformation von Quellcode.

Eine rühmliche (wenngleich ziemlich teure) Ausnahme ist „Managed Evolution“ von Stephan Murer et al. ([Murer+11]): Die drei Autoren beschreiben den strategischen Ansatz für IT-Evolution und kontinuierliche Modernisierung einer Schweizer Großbank – und schreiben dabei sehr offenherzig „aus dem Nähkästchen“. Lesenswert – primär aus Sicht des IT-Managements und wenig technisch.

Weiterhin lege ich Ihnen das „Working Effectively With Legacy Code“ von Michael Feathers [Feathers-07] ans Herz. Der zeigt Ihnen, wie Sie auch schlimmen Quellcode nach und nach retten können.

Anmerkung: Danke an Dr. Carola Lilienthal und Michael Mahlberg für eure gründlichen Reviews dieses Kapitels.



¹¹ Ein einfaches aber leistungsfähiges Markup-Format, um Dokumentation in „plain text“ schreiben zu können: <http://asciidoc.org>

¹² Build-Werkzeug ähnlich make oder ant, nur moderner und auf Groovy-Basis: <http://gradle.org>

Mit tatkräftiger Unterstützung von Alex Heusingfeld



Fragen, die dieses Kapitel beantwortet:

- Was sind Microservices?
- Welche Eigenschaften zeichnen Microservices aus?
- Welche Herausforderungen müssen wir für Microservices lösen?
- Was hat Organisation mit Microservices zu tun?
- Wo finde ich praktische Beispiele von Microservice-Architekturen?

Gestatten Sie mir eine Anmerkung: „Microservices“ sind mittlerweile Thema mehrerer eigenständiger Bücher (u. a. [Newman 20], [Wolff 18]), Konferenzen sowie zahlreicher Artikel¹. Das Thema umfasst neben der reinen Softwarearchitektur auch Aspekte von Deployment (Verteilung), Rollout, Teamorganisation sowie Systembetrieb – die ich in diesem Buch nicht alle ausführlich diskutieren möchte. Zusätzlich leben Microservices in einem (mir persönlich schon fast unheimlich) dynamischen technischen Umfeld: Im Wochenrhythmus gibt es neue Anbieter von Microservice-/Container-/Management-/Monitoring-Infrastruktur – da kommen gedruckte Bücher wie dieses hier von der Geschwindigkeit her nicht mit. Bitte betrachten Sie dieses Kapitel daher als einen (sehr groben) Überblick – und wenden Sie sich für weitere Details einigen der angegebenen Quellen zu. Am Ende dieses Kapitels finden Sie ein paar Hinweise.

Nach meiner Einschätzung funktioniert die Einführung von Microservices in einer Organisation nur, wenn auch die Organisation dementsprechend angepasst wird. Insofern sind Microservices primär ein organisatorisches Muster und nur zum (kleinen) Teil ein Architekturmuster – dazu mehr in Abschnitt 10.4.

Microservices =
organisatorisches Thema

¹ Suchen Sie im Internet nach „Microservice Reading List“ oder „Microservice Conference“ ...

■ 10.1 Was sind Microservices?

Der Begriff „Microservices“ bezeichnet einen Architekturstil, der die Aufteilung größerer Systeme in kleinere, einzeln entwickel-, deploy- und betreibbare Einheiten vorsieht, mit folgenden Zielen:

Microservices: einzeln
betreibbar

- Erhöhung der Wandelbarkeit und Flexibilität von Software
- Verbesserung von Time-to-Market, d. h. neue Funktionen (Features) sowie Fehlerkorrekturen (Fixes) sollen schneller als bisher in produktiven Betrieb kommen.
- Schnellere Entwicklung durch kleinere Einheiten – daher der Name „Microservices“
- Mehr Innovationsfähigkeit sowie Flexibilität bezüglich Technologieauswahl – grundsätzlich können Entwicklungsteams von Microservices ihre Technologieentscheidungen (fast) komplett unabhängig von anderen Microservices treffen.
- Eigene Ablaufumgebung: Jeder Microservice bekommt seine eigene Ablaufumgebung, ist also unabhängig von anderen Microservices, und kommuniziert mit diesen nur über das Netzwerk.

Microservices kombinieren bestehende Konzepte wie beispielsweise *Continuous Delivery* mit neuen Ansätzen von Softwarearchitektur, Systembetrieb und Teamstruktur. Das Konzept adressiert einige häufig auftretende Probleme großer Systeme, bringt jedoch auch neue Herausforderungen mit sich.

■ 10.2 Warum Microservices?

Größere Informationssysteme wurden oftmals als sogenannte „Deployment-Monolithen“ ausgelegt (siehe Bild 10.1), d. h., sie wurden en-bloc als Ganzes aus der Entwicklung (über Test) in den sogenannten Produktivbetrieb übernommen (auch als Staging bekannt). Dabei mussten für jede dieser „Stages“ spezifische Konfigurationseinstellungen vorgenommen werden – insbesondere im Falle von manuellem Deployment ein fehlerträchtiger Prozess.

Große Organisationen haben ihre (großen) Systeme teilweise zwei bis sechs Mal pro Jahr durch diesen Prozess geschleust, d. h., es gab zwei bis sechs neue Releases der Software pro Jahr. Eine neue Funktionalität oder ein Bugfix konnte damit frühestens in zwei Monaten ausgerollt werden. (Alternativ konnten heldenhafter Einsatz und jede Menge Risiko auch „Hotfixes“ in Produktion einspielen.)

Diese langsame Liefergeschwindigkeit (neudeutsch: „time-to-market“) bringt eine Reihe gravierender Probleme mit sich:

- Fachabteilungen oder Auftraggeber möchten nicht so lange warten, bis sie neue Funktionalität im produktiven System bekommen.

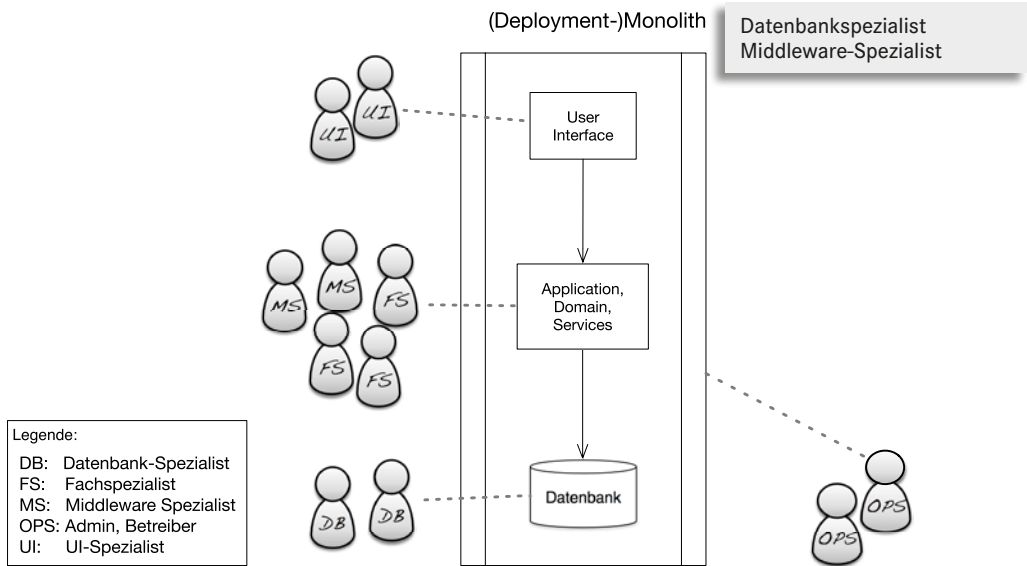


Bild 10.1 Conways Law in der Praxis: Deployment-Monolithen

- Falls im produktiven Betrieb Fehler auftreten, erfahren die betroffenen Entwicklungsteams davon erst mit monatelanger Verzögerung. Das verlängert die Zeit für Bugfixes erheblich, weil Teams sich erneut in die betroffenen Stellen des Systems einarbeiten müssen.

Außerdem sind manuelle Übergänge zwischen Stages, insbesondere bei größeren Systemen, sehr fehleranfällig.

■ 10.3 Eigenschaften von Microservices

Lassen Sie mich Microservices über ihre charakteristischen Eigenschaften erklären, statt eine formale Definition zu versuchen: Im Wesentlichen geht es um Verkleinerung, d. h., statt einzelner großer („monolithischer“) Systeme stellen Microservice-Architekturen fachliche Funktionalität über eine Menge kleiner „Microservices“ bereit, die folgende Eigenschaften besitzen²:

Modularisierung anhand von fachlichen Funktionen/Aufgaben und über Services, die über leichtgewichtige Protokolle (etwa http) interagieren.

² Diese Aufzählung ist, wie auch Bild 10.1, von James Lewis und Martin Fowler inspiriert, aus ihrem lesenswerten Artikel <https://martinfowler.com/articles/microservices.html>.

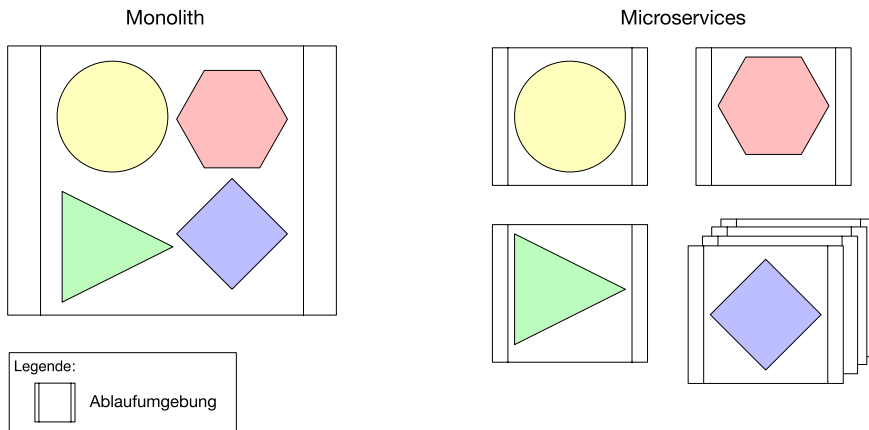


Bild 10.2 Kleinere Einheiten statt monolithische Systeme

Jeder Service läuft in einer eigenen Ablaufumgebung, Deployment und Skalierung geschehen unabhängig von anderen Microservices (siehe Bild 10.2).

Entgegen den schwergewichtigen SOA-Ansätzen mit aufwendigen ESBs (Enterprise Service Bus) sind bei Microservices die Endpunkte der Kommunikation selbstständig agierend („schlau“) und die Kommunikationskanäle simpel.

Management von Microservices geschieht dezentral, d. h., voneinander grundsätzlich unabhängige Teams verantworten jeweils einen (oder wenige) der Microservices (siehe Bild 10.3). Dezentralisierung wird gefördert, zentrales Management minimiert.

Dezentrale Datenverwaltung: Microservices enthalten und verwalten ihre eigenen Daten. Sie können die für sich geeignete Technologie auswählen, unabhängig von anderen Microservices (polyglott persistence).

Die gesamte Infrastruktur von Microservices (Entwicklung, Inbetriebnahme, Deployment/Delivery) ist automatisiert: Beim Übergang von Entwicklung in Test in Betrieb sind keine manuellen Eingriffe mehr nötig.

Kommunikation zwischen Microservices findet in der Regel „remote“ statt. Das benötigt Werkzeuge zum gegenseitigen Auffinden von Service-Instanzen inklusive Failover („Service Discovery“).

Evolutionärer Entwurf: Microservices als kleine Entwicklungseinheiten erlauben kürzere Release-Zyklen und risikoarme Änderungen.

10.4 Microservices und die Organisation

Bevor Sie Microservices produktiv einsetzen können, gilt es, die organisatorischen Weichen zu stellen und den Fokus von technologiegeprägten Teams (Architekten, DBAs, UI-Entwickler, Tester, Administratoren) auf Produkte bzw. Fachlichkeiten zu lenken. Ihre Organisation (Firma, Abteilung, Bereich) sollte dies durch funktionsübergreifende Teams (*cross-functional teams*) unterstützen. Diese Teams sollen sich von Anfang bis Ende, von Konzeption bis Produktion um „ihr Produkt“ kümmern. Amazon hat das mit der Aussage „*You build it, you run it*“ auf den Punkt gebracht. Bild 10.3 zeigt das schematisch: Jedes Team besitzt Expertise für alle notwendigen Arbeitsbereiche, insbesondere den Betrieb ihrer Microservices. Ein Team kann dabei mehrere Microservices verantworten – aber nie umgekehrt: Microservices sollten immer so „klein“ bleiben, dass sie von einem einzigen Team vollständig verantwortet werden können.

Ich halte diese organisatorischen Herausforderungen in vielen Fällen für schwieriger lösbar als die rein technischen: Für Letztere gibt's erstens Beispiele (siehe Abschnitt 10.6), zweitens eine Menge guter Literatur (siehe Abschnitt 10.7).

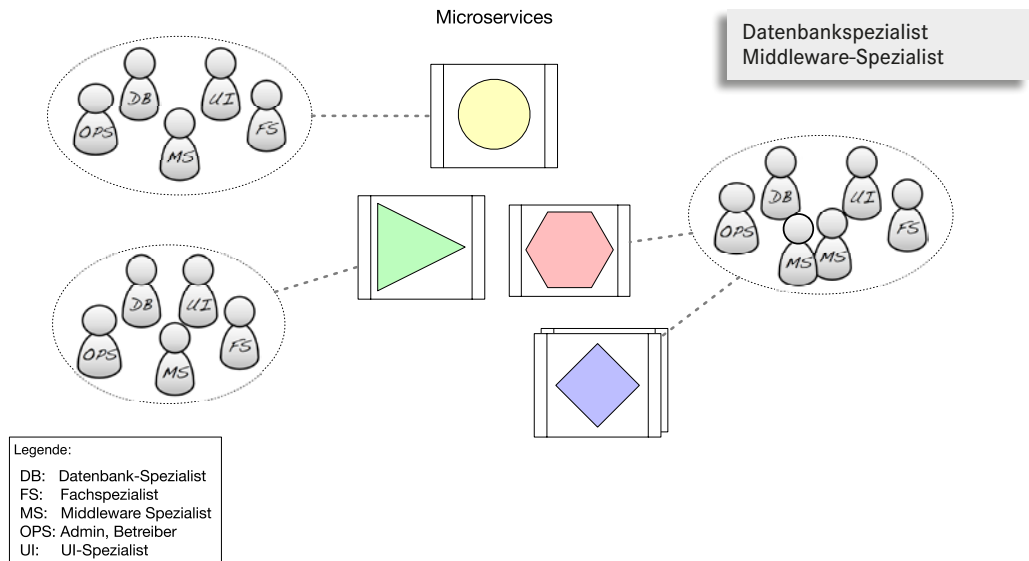


Bild 10.3 Organisation: Crossfunktionale Microservice-Teams

■ 10.5 Für welche Systeme eignen sich Microservices?

Wie alle Architekturansätze passen Microservices nur in manchen Situationen. Microservices eignen sich sehr für internetbasierte Systeme, im weitesten Sinne eBusiness-Anwendungen, mit hohen Anforderungen an Flexibilität (sowohl zur Entwicklungs- wie auch Laufzeit), bei denen Sie die Nachteile verteilter Systeme in Kauf nehmen können.

Klassische Client/Server-Anwendungen können von Microservices profitieren, wenn die serverseitigen Anwendungen stärker modularisiert und voneinander entkoppelt werden. Noch besser, wenn die (oftmals grafischen) Clients in Richtung browserbasierter Oberflächen migriert werden.

Aber auch andere Arten von Anwendungen können von Microservices profitieren: Wann immer unterschiedliche (fachliche) Funktionalitäten voneinander getrennt entwickelt und betrieben werden können, stellen Microservices eine Alternative zu herkömmlichen Deployment-Monolithen dar. Die Notwendigkeit zur Skalierung einzelner Teile getrennt von anderen ist ein Indikator, dass Microservices helfen könnten. [Wolff 15] zeigt als Beispiel ein System zur Suche nach vermissten Schiffen und Flugzeugen („Signaling System“).

Für eingebettete Systeme, oder Systeme mit Echtzeitanforderungen, halte ich Microservices für ungeeignet aufgrund der Notwendigkeit getrennter Laufzeitumgebungen. Ebenso für Systeme, die unter starken Restriktionen von Laufzeitressourcen (etwa: Speicher, Netzkapazität) arbeiten müssen.

In manchen Fällen sollten Sie aus meiner Sicht Microservices lieber bleiben lassen:

- falls Sie Build/Deploy und Release nicht automatisieren können oder
- Ihre Entwicklungs- und Betriebsorganisation strikt voneinander getrennt sind, und sie keine cross-funktionalen Teams aufstellen können oder
- Ihre Betriebsorganisation (etwa: Rechenzentrum) strikt auf vierteljährliche oder noch längere Release-Zyklen besteht und Sie diese Randbedingungen nicht aufheben können.

■ 10.6 Herausforderungen bei Microservices

Sicherlich haben Sie beim Lesen der vorigen Abschnitte schon einige wesentliche Herausforderungen von Microservices erkannt:

Wenn Sie ein System in viele unabhängige Services zerlegen, müssen Sie jetzt „viele“ kleine Dinge verwalten, statt wie bisher ein einzelnes System.

- Entwickeln, testen, bauen und deployen – das sind schwierige Aufgaben, bei denen Teams nicht jedes Mal dieselben grundlegenden Probleme lösen möchten. Wie können wir das vereinfachen?

- Die vielen Services müssen zur Laufzeit zueinander finden – wie kann diese Service-Discovery zuverlässig funktionieren?
- Systeme sollen doch eine einheitliche, konsistente Benutzeroberfläche haben, wie kann das mit vielen eigenständigen Microservices funktionieren?
- Wie sollen Microservices die Konsistenz von Daten sicherstellen, wenn jeder Service doch nur einen kleinen Teil der gesamten Daten verwaltet? Die Antwort lautet „Jeder für sich“, polyglotte Persistenz und/oder Event-basierte Systeme.
- Wie kann es funktionieren, viele Laufzeitumgebungen zu verwalten? Wie können diese Umgebungen zuverlässig aufgesetzt, betrieben und überwacht werden?

Das ist nur ein Auszug aus der Vielzahl von Themen, die Sie bei der praktischen Umsetzung von Microservice-Architekturen berücksichtigen müssen. Hinzu kommen beispielsweise Sicherheit (Vertraulichkeit, Zugriffsschutz, Nichtabstreitbarkeit), Backup/Restore, Lizenz- oder Rechtemanagement, die Schwierigkeiten asynchroner Programmierung oder der Umgang mit Latenzen (die bei remote-Kommunikation eine wichtige Rolle spielen können).

Aber fangen wir mal einfach an:

10.6.1 Überblick über viele Services behalten

Zwar geht es bei Microservices um Dezentralisierung, allerdings werden Sie in jedem Fall einen Gesamtüberblick behalten müssen. Eine minimale Menge zentrale Steuerung hilft dabei weiter, beispielsweise:

Eine einheitliche Regel, wie Schnittstellen von Services dokumentiert und publiziert werden. Verwenden Sie beispielsweise Slate, Swagger³ oder ähnliche Plattformen, um Ihre APIs maschinen- und menschenlesbar zu beschreiben. Verwenden Sie (standardisierte) Guidelines für Ihre Schnittstellen, eine Sammlung finden Sie unter <http://apistylebook.com/design/guidelines/>.

Konzepte und Vorgaben, wie Services technisch miteinander kommunizieren sollen. Legen Sie technische Protokolle (wie http) als Default-Weg fest. Über Ausnahmen können Sie oder Teams ja bei Bedarf immer noch entscheiden.

Regeln für Betrieb und Monitoring: Einerseits sollte jedes Microservice-Team für den Betrieb seiner Services selbst verantwortlich sein. Andererseits möchten Sie einen Gesamtüberblick über eine Vielzahl von Microservice-Instanzen behalten. Ein Tipp: Automatisieren und standardisieren Sie Monitoring und Logging. [Newmann 15] schreibt ein ganzes Kapitel darüber, Eberhard Wolff zählt in [Wolff 15] eine Reihe von Werkzeugen⁴ dazu auf.

10.6.2 Microservices effektiv entwickeln

Am liebsten sollten Entwickler auf der Kommandozeile einen Befehl der folgenden Art ausführen können – und als Ergebnis einen neuen Microservice inklusive sämtlicher benötigter Infrastruktur für Entwicklung, Versionierung, Test, Deployment, Monitoring usw. erhalten:

³ siehe <https://github.com/lord/slate> oder <https://swagger.io/>

⁴ etwa: Graphite, Grafana, ELK-Stack, DynaTrace, Nagios, WeaveWorks

Chris Richardson hat auf <http://microservices.io> dafür den Begriff des „Microservice-Chassis“ geprägt – eine Schablone oder ein Archetyp, der (meist langweilige und aufwendige) querschnittliche Themen erledigt – und Entwicklungsteams damit schneller und produktiver werden lässt.

Beispiele dafür sind Dropwizard, Spring Boot, Vertx, Netflix Karyon, Lagoom – aber bei allen werden Sie meiner Meinung nach individuell anpassen müssen ...

10.6.3 Service Discovery

Ein Microservice C möchte einen Dienst eines anderen Microservice P in Anspruch nehmen. Dafür benötigt A eine Verbindung (Route, Adresse) einer Instanz von P. Bild 10.4 illustriert dieses Problem.

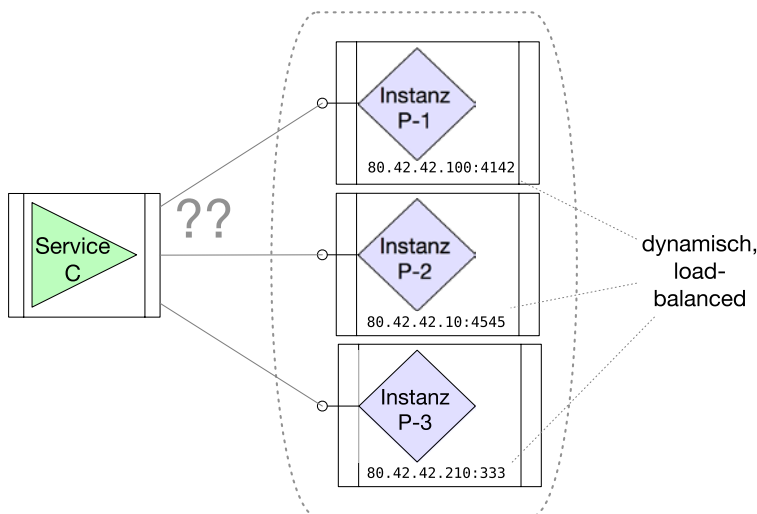


Bild 10.4 Problem: Service Discovery (nach <http://microservices.io>)

Dazu gibt es eine Reihe von Lösungsmöglichkeiten⁵:

Client-Side Discovery: Der Consumer (C) fragt eine Service-Registry nach der aktuellen Adresse einer passenden Instanz von P und stellt seinen Request anschließend dorthin.

Server-Side Discovery: Der Consumer sendet die Anfrage an einen Router (mit fester Adresse), der seinerseits die Service-Registry nach einer Instanz von P fragt und anschließend den ursprünglichen Request weiterleitet.

In beiden Fällen benötigen Sie eine Service-Registry. Im einfachsten Fall könnte das ein DNS-Nameserver sein, in der Regel werden Sie spezielle Tools⁶ verwenden.

⁵ Eine Übersicht gibt <https://dzone.com/articles/service-discovery-in-a-microservices-architecture>

⁶ etwa: Apache Zookeeper, Netflix Eureka, consul.io oder etcd

10.6.4 UI-Integration

Unsere Microservices sind Teil eines großen Ganzen – und das soll eine schicke, optisch ansprechende und grafisch einheitliche Benutzeroberfläche haben. Dafür müssen Sie in der Regel die Ergebnisse mehrerer Microservices in der Benutzeroberfläche integrieren.

Grundsätzlich bieten sich drei verschiedene Möglichkeiten an:

Jeder Microservice bringt seine eigene Benutzeroberfläche mit. Integration zwischen diesen verschiedenen UIs erfolgt beispielsweise im Browser über Hyperlinks.

Microservices stellen jeweils eigene UI-Anteile bereit, die in einer übergeordneten Benutzeroberfläche zusammengeführt werden. Im einfachsten Fall enthält diese nur eine Navigation. Bild 10.5 zeigt das schematisch.

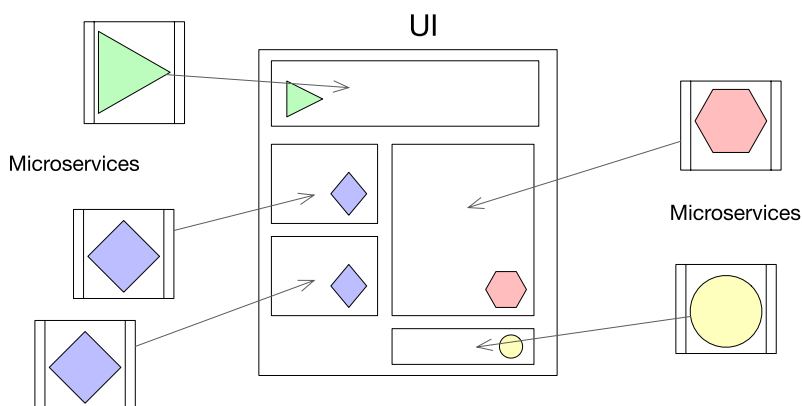


Bild 10.5 Mögliche UI-Integration von Microservices

Eine gemeinsame Benutzeroberfläche nutzt andere Microservices über deren Schnittstellen. Alle übrigen Microservices haben kein UI. Durch diesen Integrationsansatz entstehen möglicherweise ungewünschte Abhängigkeiten, das Deployment einzelner Services ist wahrscheinlich nicht mehr unabhängig von anderen Services (Frontend-Monolith).

10.6.5 Dezentralisierte Daten

Entgegen der großen Datenmodelle klassischer monolithischer Systeme sollte jeder Microservice seine eigenen Daten halten, im Extremfall seine komplett eigene Datenbank besitzen. Daten werden damit dezentral gehalten und im Bedarfsfall auf der Ebene einzelner Microservices zusammengeführt.

Falls Koordination von Datenänderungen über mehrere Services hinweg nötig sein sollte, können Sie das über Event-basierte Systeme abwickeln (Choreografie-Ansatz) – Bild 10.6 zeigt das schematisch am Beispiel einer Änderung von Kundendaten: Drei verschiedene Services benötigen die Information, jeweils für unterschiedliche fachliche Zwecke. Statt einer gemeinsamen Kundendatenbank (die eine Obermenge aller Daten der Kunden enthalten würde), speichert jeder Microservice nur den für ihn jeweils relevanten Teil.

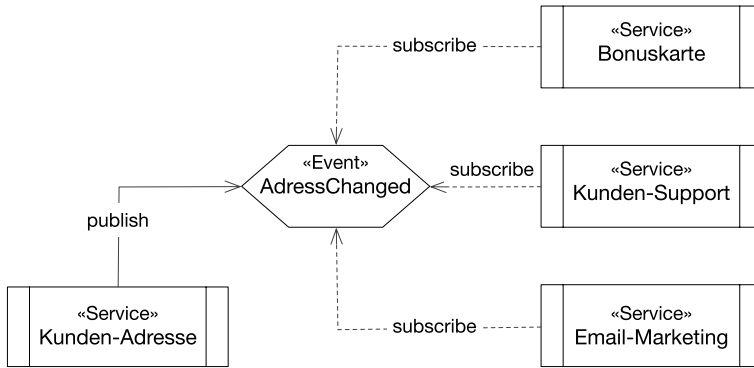


Bild 10.6 Datenintegration und Choreografie über Events

Änderungen oder Aktualisierungen werden über einen Event-Mechanismus publiziert und von den betroffenen Microservices (asynchron) bearbeitet.

Mit Events bekommen Sie keine unmittelbare Datenkonsistenz, wie das ACID-Datenbanken durch Transaktionen anbieten, sondern die sogenannte eventual consistency – also eine möglicherweise zeitlich verzögerte Datenkonsistenz. In der Praxis zeigt sich, dass diese Verzögerung völlig ausreichend für viele Anwendungsfälle ist: Beispielsweise sehen Sie auf dem Kontoauszug Ihrer Bank eine Buchung auch erst mit teilweise tagelanger Verzögerung.

Zur Vertiefung dieses Themas empfehle ich Ihnen ein Video von Chris Richardson⁷, online unter <https://www.infoq.com/presentations/microservices-docker-cqrs>.

10.6.6 Versionierung von Microservices

Kleine, fokussierte Microservices machen die Überarbeitung oder Erneuerung dieser Services sehr einfach. Das wirft die berechtigte Frage nach deren Versionierung auf – denn die Schnittstellen solcher Microservices werden sich über die Zeit fortentwickeln.

Nutzen Sie mindestens semantische Versionierung (siehe <http://semver.org>), eine Spezifikation, die anhand eines dreiteiligen Nummerierungsschemas ersichtlich macht, ob ein Consumer eine bestimmte Provider-Instanz überhaupt benutzen kann: Mit Hilfe der Kategorien MAJOR, MINOR und PATCH können Sie ausdrücken, welche Änderungen eine neue Service-Version enthält:

Die Erhöhung der MAJOR-Versionsnummer drückt aus, dass inkompatible Änderungen in der neuen Version vorliegen. Eine Erhöhung der MINOR-Versionsnummer bedeutet neue Funktionalität, die abwärtskompatibel ist. PATCH bedeutet Fehlerbehebungen, die vollständig kompatibel mit früheren Versionen sind.

⁷ Chris betreibt microservices.io und ist der ursprüngliche Entwickler der heutigen CloudFoundry.

10.6.7 Laufzeitumgebungen und Infrastruktur verwalten

Methoden und Techniken zur Automatisierung von Laufzeitumgebungen und Infrastruktur haben in den letzten Jahren, gefördert durch das rasante Aufkommen von Cloud-Anbietern⁸ enorme Fortschritte erreicht. Werkzeuge zum automatisierten Testen (Unit-, Integrations-, Akzeptanz-, UI- und Lasttests) gehören mittlerweile zu den etablierten good practices, Continuous Integration⁹ und DevOps¹⁰ haben Einzug in die Praxis gehalten.

Cloud- und/oder Container-Technologie bilden gute Grundlagen für Microservice-Architekturen. Noch viel stärker als Virtualisierung prägen Techniken wie Docker und Vagrant die Automatisierung der meisten Infrastrukturaufgaben.

■ 10.7 Beispiele für Microservices

Die rein technischen Aspekte von Microservices, insbesondere Implementierung und Deployment, können Sie an diversen öffentlichen Beispielen nachvollziehen. Eine kleine Sammlung solcher Beispiele habe ich unter <https://github.com/arc42/microservice-examples> veröffentlicht.

■ 10.8 Weiterführende Literatur

Martin Fowler und James Lewis haben unter [Fowler+14] eine kompakte und gut lesbare Übersicht über Microservices veröffentlicht. Einige ihrer Gedanken und Argumente habe ich in diesem Kapitel aufgegriffen. Sehr illustrative Abbildungen – als Einstieg empfehlenswert.



[Newman 15] und [Wolff 15] führen deutlich ausführlicher in die Thematik ein. Zu Letzterem gibt es einen kostenfreien „Primer“ als Einstieg (siehe <http://microservices-book.com/primer.html>).

[Newman 20] beschreibt in Form pragmatischer Muster den (anstrengenden) Weg vom Monolithen bis zu Microservices.

⁸ etwa: Amazon AWS, Microsoft Azure, IBM Bluemix, Google Cloud, RedHat OpenShift, Heroku, CloudFoundry, Pivotal und andere

⁹ siehe https://de.wikipedia.org/wiki/Kontinuierliche_Integration

¹⁰ DevOps (Zusammenziehung aus Development und Operation) hat zum Ziel, Geschwindigkeit der Entwicklung und Auslieferung, Qualität des Systems sowie Kooperation der beteiligten Teams zu verbessern.

Chris Richardson betreibt die Website <http://microservices.io>, auf der Sie diverse Patterns und Praktiken zu Microservices finden. Er hat ebenfalls diverse frei verfügbare Beispiele von Microservices implementiert.

In meiner Sammlung von Microservice-Beispielen <https://github.com/arc42/microservice-examples> habe ich noch einige Listen von Artikeln, Tutorials etc. aufgenommen.

[Tilkov+15] Das REST-Buch, mittlerweile in der 3. Auflage. Hilfreich für alle, die mit der Integration in modernen Websystemen zu tun haben. REST ist eine etablierte Möglichkeit für Microservices, miteinander zu kommunizieren.

*Enterprise architecture is the organizing logic
for business processes and IT infrastructure.*

[Ross+06]



Fragen, die dieses Kapitel beantwortet:

- Auf welchen Ebenen arbeiten Architekten?
- Was bedeutet Enterprise-IT-Architektur?
- Welche Aufgaben sollten Enterprise-IT-Architekten wahrnehmen?

Architekten arbeiten auf verschiedenen Ebenen

Bisher handelte dieses Buch immer von Softwarearchitektur, also den Strukturen einzelner Softwaresysteme. Nun lernen Sie einige weitere Arten von Architekturen kennen, deren Ergebnisse und Entwürfe von prägendem Einfluss auf Softwaresysteme sind.

Die wesentlichen Architekturebenen von Unternehmen finden Sie als Architekturpyramide in Bild 11.1 (in Anlehnung an [Dern09]) dargestellt. Diese Pyramide zeigt nach oben hin zunehmend abstrakte Architekturebenen:

- Die Spitze der Architekturpyramide bildet die Strategie – die Ziele der Organisation – sowie mögliche Wege, um sie zu erreichen, und deren Grenzen. Verfeinert durch die Business-Architektur klärt die Strategie unter anderem die Zusammenarbeit zwischen Business und IT. Auf Strategien gehe ich nicht weiter ein, sondern verweise interessierte Leserinnen auf [Keller17].
- Die Business-Architektur definiert die geschäftlichen Prozesse und betriebswirtschaftlichen Funktionen einer Organisation. Sie verwendet den Begriff „Informationen“ in wirtschaftlichem oder rein fachlichem Sinn, ohne direkten Bezug zu Softwaresystemen. Diese Prozesse haben jeweils einen spezifischen Informationsbedarf.
- Die nächste Ebene der Pyramide, die Informationsarchitektur, definiert die Struktur und Zusammenarbeit aller IT-Systeme einer Organisation (auch IS-Portfolio oder *Anwendungslandschaft*) genannt. Sie stellt die wesentlichen Informationsflüsse und Schnittstellen zwischen sämtlichen Softwaresystemen einer Organisation dar. Die Informationsarchitektur stillt den Informationsbedarf der Geschäftsprozesse. Auf dieser Ebene tummeln sich die Enterprise-IT-Architekten, von denen in diesem Kapitel die Rede sein wird.

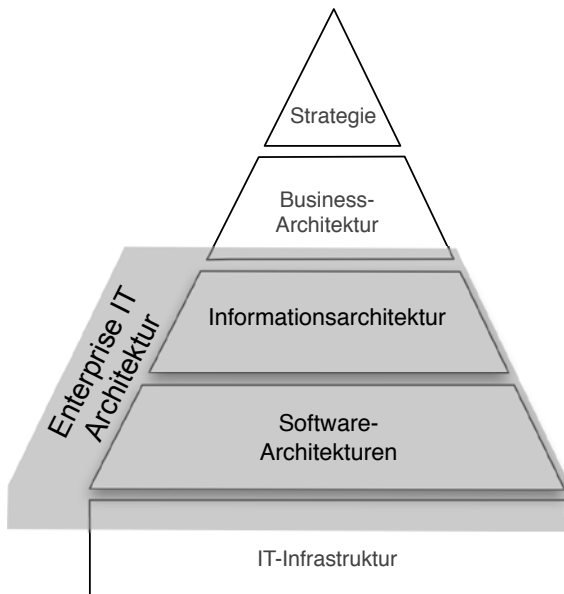


Bild 11.1
Architekturpyramide
(in Anlehnung an [Dern 09])

- Unterhalb der Informationsarchitektur liegt die Ebene der Softwarearchitekturen. Sie bildet das Kerngebiet dieses Buches und beschäftigt sich mit Entwurf und Implementierung einzelner Softwaresysteme. Hierzu gehören für die IT-Systeme einer Organisation jeweils die unterschiedlichen Sichten (Bausteine, Abläufe, Verteilung). Der Gesamtkontext der einzelnen Softwarearchitekturen entstammt den darüberliegenden Ebenen der Informations- oder Business-Architektur.
- Den Fuß der Pyramide bildet die IT-Infrastruktur. Deren *Architektur* dreht sich um Hardware, Netztopologie, Betriebssysteme, Datenbanksysteme und Middleware. Entscheidungen oder Randbedingungen dieser Infrastruktur betreffen oftmals unmittelbar die darüberliegende Ebene der Softwarearchitekturen. Die „Bausteine“ der IT-Infrastruktur sind so konkret, dass Sie sie sogar anfassen können ...

■ 11.1 Wozu Architekturebenen?

Inwiefern hilft die oben dargestellte Pyramide unserem Verständnis von Architekturebenen? Auch wenn einzelne Softwareprojekte komplex und schwierig bleiben, klärt sie aus meiner Sicht einige wesentliche Eckpunkte:

- Im Umfeld mittlerer und großer Organisationen oder Unternehmen geht es heute insbesondere um Komplexitätsmanagement großer Anwendungslandschaften. Dabei hilft insbesondere die Ebene „Informationsarchitektur“ als Vermittler zwischen der Business- und der IT-Sicht.

- Es gibt Strukturen oberhalb einzelner Softwaresysteme. Diese Informations- oder Business-Architekturen geben wesentliche Einflussfaktoren und Randbedingungen (siehe hierzu Abschnitt 3.3) vor, die den Entwürfen einzelner Systeme oft sehr konkrete und enge Grenzen setzen.
 - Durch diese Strukturen im Großen können Unternehmen die finanziellen und personellen Aufwände für ihre Informatikunterstützung beeinflussen.
 - Weiterhin können Unternehmen diese Strukturen kontrolliert vereinfachen und insofern die Gesamtkomplexität ihrer IT reduzieren.
- Einzelne Softwaresysteme bilden die Bausteine der höheren Abstraktionsebene „Informationsarchitektur“. Dort gelten ähnliche Strukturierungs- oder Entwurfsprinzipien wie bei einzelnen Softwaresystemen: Schnittstellen sollten klar definiert und Abhängigkeiten bewusst entschieden werden. Leider mangelt es vielen Organisationen an dieser „Struktur im Großen“ – mit teilweise dramatischen Konsequenzen hinsichtlich der Gesamtkosten der IT. Auch hierauf gehe ich in diesem Buch nicht weiter ein.
- Die Strategie eines Unternehmens steht in einem klar definierten Zusammenhang mit Geschäftsprozessen und somit auch mit der IT – zumindest sollte dies der Fall sein.

Erst in den letzten Jahren haben Unternehmen begonnen, die Strukturen oberhalb einzelner Systeme (Informations- und Business-Architektur) wirklich ernst zu nehmen. Fortschrittliche Organisationen, insbesondere solche mit vielen IT-Systemen, schufen die Positionen oder Teams für Enterprise-IT-Architektur.

■ 11.2 Aufgaben von Enterprise-Architekten

Über Rolle und Aufgaben von Enterprise-Architekten gibt es sowohl in der Praxis als auch in der Literatur sehr unterschiedliche Ansichten. Einen guten Einstieg mit klarer Begriffsbildung finden Sie in den empfehlenswerten Büchern [Dern09] und [Keller12]. Ich möchte Ihnen hier einige der Kernaufgaben vorstellen, ohne dabei detailliert auf die Schnittstellen zur Business-Architektur oder Softwarearchitektur genauer einzugehen.

11.2.1 Management der Infrastrukturkosten

Die IT-Infrastruktur weist ein erhebliches Potenzial zur Reduktion von IT-Ausgaben (sprich: Geld!) auf. Deswegen nenne ich sie als erste Aufgabe, obwohl sie sich methodisch kaum fassen lässt. Von der Server-Konsolidierung über Lizenzmanagement bis hin zum Scorecarding (Vergleich mit den Infrastrukturkosten anderer Unternehmen) reicht die Bandbreite möglicher Tätigkeiten.

11.2.2 Management des IS-Portfolios

Enterprise-IT-Architekten kümmern sich um das Informationssystem-Portfolio ihrer Organisation (auch Anwendungslandschaft oder IS-Portfolio genannt).

Der Begriff IS-Portfolio bezeichnet die Gesamtheit aller Softwaresysteme einer Organisation. In der Realität können das mehrere Hundert bis mehrere Tausend¹ verschiedene Programme sein – mit all ihren Schnittstellen, gegenseitigen Abhängigkeiten, Releasezyklen und Technologien. Bei größeren Unternehmen (Beispiel: Finanzdienstleister, Versicherungen, Automobilhersteller) erfordern Unterhalt und Management dieses IS-Portfolios ein jährliches Budget von mehreren Hundert Millionen Euro, bei einigen sogar bis in den Milliardenbereich. Sie ahnen richtig: Eine anspruchsvolle Aufgabe – denn sie verbindet fachliche mit technischen und organisatorischen Herausforderungen.

Aber eins nach dem anderen – lassen Sie mich mit den *Anforderungen* an das IS-Portfolio beginnen.

Welche Anforderungen bestimmen das IS-Portfolio?

Nach welchen Anforderungen und Kriterien entwickeln und planen Enterprise-IT-Architekten das IS-Portfolio ihrer Organisationen? Woher stammen ihre Anforderungen, was sind ihre Einflussfaktoren und Randbedingungen?²

In erster Linie bestimmen die geschäftlichen Anforderungen der Business-Architektur das IS-Portfolio (werfen Sie einen Blick auf die Pyramide aus Bild 11.1). Dazu gehören neue Dienstleistungen des Unternehmens, neue Geschäftsprozesse oder Veränderungen des geschäftlichen Umfeldes.

Außerdem bestimmen technische oder strategische Überlegungen das IS-Portfolio. Einige Beispiele:

- Die künftige Ausrichtung an Open-Source-Middleware oder Datenbanken.
- Die Konzentration des IT-Betriebs auf genau zwei unterstützte Betriebssysteme statt der bisherigen vier. Hierbei gilt es beispielsweise, passende Migrationswege für die betroffenen Systeme oder Projekte zu finden.
- Die Fusion mit einem anderen Unternehmen und die damit verbundene Konsolidierung der gesamten IT.

In den letzten Jahren haben zunehmend auch gesetzliche (regulatorische) Rahmenbedingungen für Anpassungen im IS-Portfolio gesorgt – so etwa die gesetzlichen Nachweispflichten oder die Vorratsdatenspeicherung.³

Enterprise-IT-Architekten können nicht (wie einige glückliche Softwarearchitekten) auf fertige Anforderungsanalysen zurückgreifen, sondern müssen die Anforderungen an das IS-Portfolio kontinuierlich selbst weiterentwickeln – wobei ihnen Vorstände, Geschäftsführer, Rechenzentrums-Leiter und andere wichtige Personen kräftig „dreinreden“ werden.

¹ Einige deutsche Automobilhersteller verfügen über mehr als 3000 (dreitausend) verschiedene eigenentwickelte (Server- oder Mainframe-)IT-Systeme.

² Diese Begriffe haben Sie im Zusammenhang mit Softwarearchitektur in Abschnitt 3.3 kennengelernt.

³ Beispiele: SOX, Basel-II, Solvency-II, ITIL/Cobit.

Was gehört zur Planung eines IS-Portfolios?

Wie Softwarearchitekten die Abhängigkeiten ihrer „Bausteine“ (Pakete, Komponenten, Klassen, Module) unter Beachtung von Entwurfsprinzipien und konkreten Anforderungen bewusst entwerfen, so sollten Enterprise-IT-Architekten auf ihrer (höheren) Abstraktionsebene einer ähnlichen Systematik folgen:

- Sie entwerfen Schnittstellen für andere Softwaresysteme.
- Sie planen die Reihenfolge und Gruppierung von Entwicklungs- oder Wartungsprojekten. Damit bringen sie Aktivitäten einzelner Entwicklungsprojekte in eine zeitliche Ordnung und setzen Prioritäten für einzelne Entwicklungsaufgaben. Als Ergebnis entstehen oftmals langfristige *Architekturpläne*, ähnlich dem Beispiel aus Bild 11.2.
- Sie geben technische Vorgaben an Entwicklungsprojekte oder -abteilungen; in Bild 11.2 beispielhaft etwa die Migration der Warenwirtschaft (WaWi) auf „AF 4.0“ ab Mitte Q1-2008 oder der Einsatz von MySQL 5 ab ca. Ende Q2-2013.
- Sie unterscheiden zwischen Projekten zur Anwendungs- und Infrastrukturentwicklung.
- Sie bewerten das IS-Portfolio und definieren Optimierungsmaßnahmen.

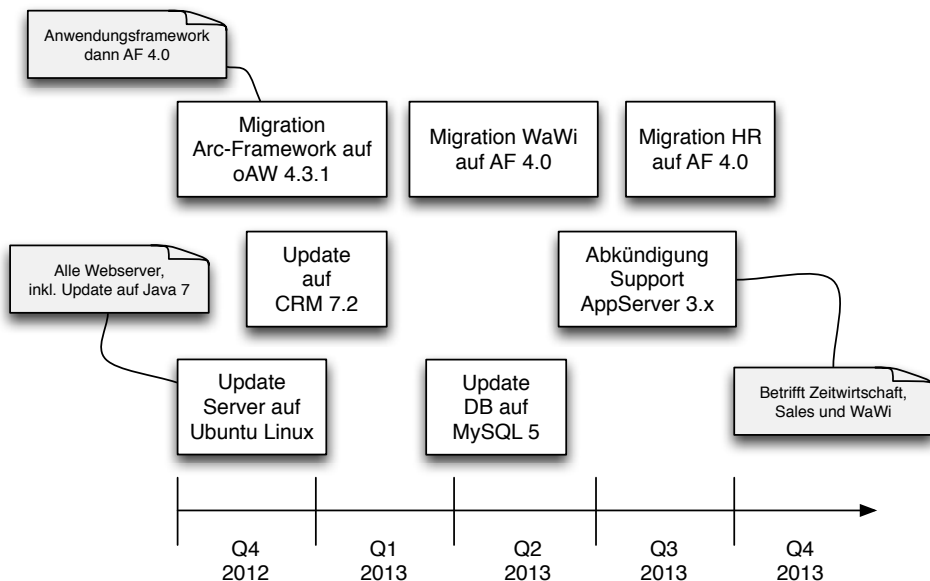


Bild 11.2 Beispiel eines Architekturplans aus dem IS-Portfoliomanagement

11.2.3 Definition von Referenzarchitekturen

Zur Vereinheitlichung und Homogenisierung der gesamten Anwendungslandschaft eines Unternehmens helfen *Referenzarchitekturen* (auch *Blueprints* genannt). Sie geben vor, nach welchen Maßgaben einzelne Systeme zu strukturieren sind, welche Technologien und Produkte eingesetzt werden sollten und wie der Betrieb dieser Anwendungen stattfinden soll.

Projekte können sich an solchen Referenzarchitekturen orientieren und erheblichen Entwurfs- beziehungsweise Evaluierungsaufwand ersparen. Referenzarchitekturen bilden die Grundlage für unternehmensinterne Wiederverwendung von Softwarebausteinen.

Was ist eine Referenzarchitektur?

Lassen Sie mich ein wenig genauer erklären, was eine Referenzarchitektur ausmacht. Dazu verwende ich die Definition aus [Dern09]:

Eine Referenzarchitektur ist eine IT-Architektur, die standardisierend für die IT-Architekturen einer Gruppe von Informationssystemen wirkt.

Dazu gehören folgende Bestandteile:

- Die Definition des Einsatzbereiches dieser Referenzarchitektur: Für welche Art von Systemen gelten ihre Vorgaben – und in welchen Fällen nicht?
- Die (abstrakten) Strukturen, insbesondere Baustein- und Laufzeitstruktur. Sie klären, welche grundsätzlichen Bausteine in diesem Typus von Anwendungen vorhanden sein müssen, welche Schnittstellen sie haben und wie sie zur Laufzeit zusammenwirken.
- Technische Vorgaben, wie etwa die Definition der erlaubten Hardware, Datenbanken, Middleware oder Betriebssysteme.
- Optional können auch Vorgaben zu Entwicklungsprozessen zur Referenzarchitektur gehören, wenn Sie beispielsweise bestimmte Teile Ihrer Webentwicklung grundsätzlich als Outsourcing-Aufträge vergeben.



Einige meiner Kunden haben gute Erfahrung damit gemacht, lediglich *technische* Aspekte wie Persistenz oder Weboberflächen als Referenzarchitektur zu beschreiben. Das hilft insbesondere im Zusammenhang mit generativer Entwicklung weiter, weil dann über einen Generator die entsprechenden Teile der Referenzarchitektur automatisch erzeugt werden können.

Die Referenzarchitekturen der Enterprise-Architekten geben nach [Dern09] in der Regel keine konkreten Komponenten zur Wiederverwendung vor. Sie stellen keine unmittelbar einsetzbaren *Frameworks* oder *Bibliotheken* bereit.⁴

Wie entstehen Referenzarchitekturen?

Eine erste Antwort dazu: Referenzarchitekturen entstehen genau wie gute Entwurfsmuster, niemals nur auf dem Papier. Sie sind aus positiven Erfahrungen abgeleitet und abstrahieren von „guten“ Systemen.

⁴ Hingegen verwenden viele Softwarearchitekten den Begriff *Referenzarchitektur* im Sinne der Vorgabe konkreter Komponenten oder Implementierungen.



- Untersuchen Sie innerhalb Ihrer Organisation systematisch und umfassend, welche Projekte oder Lösungsansätze besonders gut funktioniert haben oder besonders kostengünstig arbeiten.
- Misstrauen Sie „theoretischen“ Referenzarchitekturen oder Blaupausen aus sekundären Quellen. Systeme müssen in Ihrer Organisation funktionieren, und das können Sie nur unter den dortigen spezifischen Bedingungen nachweisen.
- Misstrauen Sie insbesondere den „pauschalen Modetrends“ der Informatik: Mal sollte CORBA alles verbessern, mal half Objektorientierung aus dem Schlamassel, dann EAI, gefolgt von SOA. All diese Ansätze haben ihre Stärken – sind jedoch als „pauschale“ Referenzarchitektur nach meiner Erfahrung kontraproduktiv. Letztlich hilft nur ein systematischer Entwurf spezifischer Strukturen.*

* Wie ich ihn in diesem Buch ja durchgängig propagiere (aber das ist Ihnen sicherlich schon aufgefallen!).

Wählen Sie funktionierende operative Systeme als Vorbilder, und leiten Sie daraus Referenzarchitekturen ab. Möglicherweise gibt es mehrere Referenzarchitekturen, beispielsweise eine für Websysteme, eine zweite für interne Backend-Systeme.

Und nun die zweite (traurige) Antwort: Referenzarchitekturen zu entwickeln, ist schwierig. Der Grund: Die Erstellung wiederverwendbarer Komponenten oder Strukturen ist schwer – und meiner Erfahrung nach mindestens zwei- bis dreimal aufwendiger als die Entwicklung einer Einmal-Lösung.⁵ Lassen Sie sich nicht entmutigen – langfristig lohnt sich dieser Aufwand, sowohl für die Entwicklung als auch für den Betrieb neuer Systeme.

11.2.4 Weitere Aufgaben

Um für Enterprise-IT-Architekten keine Langeweile aufkommen zu lassen, gebe ich Ihnen noch einen kleinen Ausblick auf einige zusätzliche Aufgaben (Details finden Sie in [Dern09] sowie [Keller12]):

- Mitwirkung bei der Formulierung und Weiterentwicklung der IT-Strategie.
- Entwicklung der Infrastruktur und des Systembetriebs.
- Kartografie und Kommunikation der Informationsarchitektur, d. h. modellhafte Darstellung sämtlicher Softwaresysteme, Schnittstellen sowie deren Abbildung auf Geschäftsprozesse.
- Umsetzung von IT-Governance. Falls Sie diesen Begriff hier zum ersten Mal lesen, schauen Sie zur Vertiefung mal auf [Cobit]. Es geht um die Regierung der Informatik im Unternehmen, letztlich den Nachweis, dass IT-Budgets bestmöglich zum Wohle des Unternehmens eingesetzt werden.
- Management von IT-Budgets und Investitionen.

⁵ Andererseits habe ich niemals behauptet, Architektur sei einfach. 😊

Tipps für Enterprise-Architekten



- Homogenisierung und Standardisierung schaffen Übersichtlichkeit und reduzieren Komplexität – das sollten Ihre wesentlichen Ziele sein, zumal Ihre Betriebs- und Entwicklungskosten dadurch mittelfristig deutlich sinken werden. Vorsicht: Rechnen Sie mit starkem Widerstand bei der Einführung und Durchsetzung von Standards – es drohen Glaubenskriege und Philosophenstreit. Daher benötigen Sie neben Überzeugungs-fähigkeit auch die „Insignien der Macht“*, um Standards durchzusetzen.
- Zu viel Standardisierung kostet unverhältnismäßig hohen Planungs- und Kontrollaufwand. Lassen Sie ruhig Abweichungen zu, solange Sie wissen, warum und zu welchem Preis. Kein Mensch möchte in einem Reinraum leben – ein wenig Unordnung ist gemütlich ...
- Kartografieren Sie Ihre gesamte IT-Landschaft. Dazu setzen Sie dieselben Sichten und Diagrammart ein wie für einzelne Softwaresysteme (siehe Kapitel 5 und 6). Der einzige Unterschied liegt im Abstraktionsgrad der einzelnen Bausteine. Hüten Sie sich vor völlig anderen Diagrammart oder Werkzeugen – ansonsten drohen Mehraufwand und Redundanz.
- Das primäre Ziel von Enterprise-IT-Architekten besteht in der Reduktion von Komplexität. Manche Manager wünschen sich von ihnen hauptsächlich Kostensenkung, teilweise durch „Abschalten“ bestehender Systeme. Als Enterprise-IT-Architekt haben Sie aber bereits hervorragende Arbeit geleistet, wenn die Komplexität Ihrer Anwendungslandschaft nicht weiter ausufert.**

* Die Komplexität von Anwendungslandschaften steigt so lange, bis Sie einen signifikanten Aufwand in deren Reduktion investieren (sprich: Enterprise-IT-Architektur betreiben). Komplexität sinkt niemals von alleine – insbesondere nie durch vorschnelle Managemententscheidungen (aber das dürfen Sie Ihren Managern in dieser Form nicht sagen ...).

** Mit diesen Insignien meine ich beispielsweise den Rückhalt der Linienmanager, des IT-Vorstandes, des Rechenzentrumsleiters und der übrigen wesentlichen Manager Ihrer IT-Welt.

Fazit

Unternehmen setzen oft zahlreiche unterschiedliche Softwaresysteme ein – diese „IT-Landschaft“ benötigt Planung und Struktur. Wie sich Softwarearchitekten um Struktur und Qualität einzelner Softwaresysteme kümmern, so Enterprise-IT-Architekten um die Struktur und Qualität der gesamten IT-Landschaft von Unternehmen. Sie sorgen für eine möglichst reibungslose Unterstützung von Geschäftsprozessen durch Softwaresysteme – neudeutsch *Business-IT-Alignment*.

Ihre Rolle verspricht einen hohen Nutzen für Unternehmen, sowohl finanziell als auch bezüglich der gesamten Effizienz.

Viele etablierte Grundsätze des Softwareengineering und der Softwarearchitektur gelten auch auf dieser hohen Abstraktionsebene – allerdings gehört neben einem fundierten Verständnis von Software auch ein ebenso guter Einblick in die fachlichen Prozesse und Unternehmensabläufe zu den Voraussetzungen erfolgreicher Enterprise-IT-Architekten.

■ 11.3 Weiterführende Literatur

[Keller17] stellt die IT-Unternehmensarchitektur aus dem Blickwinkel der verantwortlichen Architekten und Manager dar. Das Buch fokussiert auf methodische Aspekte, mit zahlreichen praktischen Hinweisen zu wichtigen Themen wie IT-Strategie, Governance, Anwendungsportfolios, Budgets sowie der Einführung von IT-Architektur in Unternehmen.



[Ross+06] stammt aus der Feder einiger renommierter Hochschullehrer. Sie erläutern, wie und warum eine IT-Strategie letztlich zu einer verbesserten *Performance* des gesamten Unternehmens führt; richtet sich an IT-Topmanager, enthält Ratschläge zur Verbesserung des ROI für IT-Investitionen sowie zur Senkung der IT-Kosten und der IT-Risiken. Staubtrocken und sehr abstrakt.

Das SEBIS-Projekt der TU München beschäftigt sich mit der Kartografie von Unternehmensarchitekturen. Abschnitt 4.8 aus [Keller17] enthält eine Einführung.

Danke an Gernot Dern und Wolfgang Keller für den gründlichen Review dieses Kapitels.

Dieses Kapitel dokumentiert die Architekturen zweier realer Softwaresysteme gemäß den methodischen und strukturellen Ansätzen aus den Kapiteln 5 und 6. Es orientiert sich dabei an der arc42-Strukturvorlage, die ich Ihnen in Abschnitt 5.3.1 vorgestellt habe.

Anmerkung zur Nummerierung: Hier habe ich die „normale“ Überschriftenzählung dieses Buches in den Unterkapiteln außer Kraft gesetzt und nummeriere die Abschnitte exakt so, wie es in einer *echten* Architekturdokumentation der Fall wäre.

- In Abschnitt 12.1 zeige ich Ihnen ein System zur Datenmigration im Finanzbereich. Sie erleben einen auf Performance optimierten Datenfluss-Architekturstil, ähnlich einer Pipes- und Filter-Architektur.
- Abschnitt 12.2 dokumentiert eine Produktfamilie aus dem Umfeld der *Customer Relationship Management*-(CRM)-Systeme.

Beide Systeme gibt es wirklich. Für die Darstellung hier im Buch habe ich von der Realität stellenweise stark abstrahiert und viele Details aus didaktischen Gründen vereinfacht.

■ 12.1 Beispiel: Datenmigration im Finanzwesen

1 Einführung und Ziele

Zweck des Systems

Dieses Dokument beschreibt die Softwarearchitektur des M&M¹-Systems zur Migration von ca. 20 Millionen Personen- und Kontodaten der Firma Fies Teuer AG, einer Organisation aus der Finanzdienstleistung.²

Der Auftraggeber von M&M betreibt seit etwa 1970 einige Mainframe-Anwendungen (Cobol, VSAM) zur Pflege von Personen-, Konto- und Bankdaten. Diese Systeme werden zurzeit durch eine homogene Java-Anwendung abgelöst, die durch ein anderes, parallel zu M&M laufendes Projekt entwickelt wird.

Fies und Teuer AG hat in Deutschland ca. 20 Millionen Kunden und pflegt für diese Kunden insgesamt mehr als 50 Millionen Konten. Kunden können natürliche oder juristische Personen sein, teilweise auch andere Organisationen (Verbände, Vereine etc.).

Konten enthalten neben reinen Buchungsinformationen auch statistische oder sonstige finanzbezogene Informationen zu den Kunden oder mit ihnen assoziierten Personen (z. B. Ehepartner) oder Organisationen (z. B. von ihnen geführte Unternehmen).

Die inhaltliche Bedeutung dieser Konto- und Buchungsinformationen sowie die darin enthaltenen Attribute haben über die Betriebszeit der bisherigen Anwendung stark gewechselt. Zur Umstellung auf das neue konsolidierte Objektmodell hat ein Team von ca. 20 Fachexperten eine Menge von mehreren Hundert fachlichen Regeln aufgestellt, nach denen die Migration ausgeführt werden muss.

Sämtliche bestehenden Daten aus dem bisherigen Format (VSAM-Dateien, EBCDIC-Codierung) müssen in dieses Java-Objektmodell migriert werden. Diese Migration ist Aufgabe des hier beschriebenen Systems.

Ausgangssituation der bestehenden Daten

- Früher waren die Daten anhand der „Konten“ organisiert.
 - Für jegliche Operationen mussten Sachbearbeiter zuerst die betroffenen Konten identifizieren, erst danach konnten fachliche Operationen (Buchungen, Auskünfte) für die betroffenen Personen ausgeführt werden.
 - Im Zuge der steigenden Kundenorientierung der Fies und Teuer AG sollen die Daten künftig anhand von Personen organisiert werden.
- Die Ausgangsdaten liegen in Form verschiedener Dateien auf Bändern vor, deren Einträge nach fachlichen Kriterien einander zugeordnet werden müssen.

¹ M & M steht für Migration von Massendaten.

² Wie Sie leicht erraten können, habe ich den Namen dieser Organisation bewusst verändert ...

- Früher geltende Schlüssel- und Kennzahlensysteme müssen in neue Schlüssel überführt werden. Beispiel: Die alte Tarifklasse „T13“ wird zur neuen Tarifklasse „Jährliche Zahlung ohne Skonto“.

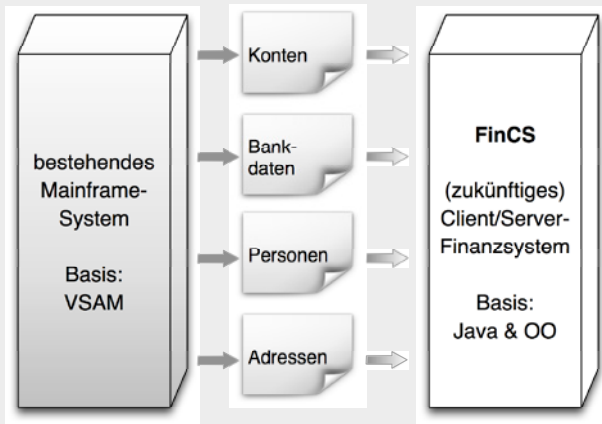


Bild 12.1

Zweck des Systems:
Migration bestehender Daten

Leserkreis

- Alle in Sektion 1.3 dieser Dokumentation genannten Stakeholder von M&M.
- Softwareentwickler, Architekten und technische Projektleiter, die Beispiele für eine Architekturdokumentation (auf Basis des arc42-Templates) suchen.
- Mitarbeiter von IT-Projekten, die sich das Leben in der Softwarearchitektur vereinfachen, indem sie auf ein bewährtes Template zur Dokumentation zurückgreifen.

1.1 Fachliche Aufgabenstellung

Die wichtigsten funktionalen Anforderungen:

- Bestehende Kunden-, Konto- und Adressdaten sollen von den bisherigen VSAM³-basierten Mainframe⁴-Programmen aus dem EBCDIC-Format in das Objektmodell von FINCS, einer in Entwicklung befindlichen Java-Anwendung, migriert werden.
- Die Ausgangsdaten liegen in Form verschiedener Dateien oder „Bänder“ vor, deren Einträge nach fachlichen Kriterien einander zugeordnet werden müssen.
- Früher waren die Daten nach *Konten* organisiert, im neuen System sind *Personen* der Bezugspunkt.
- Teile der früheren Schlüssel- oder Kennzahlensysteme sollen in neue Schlüssel überführt werden.

³ VSAM = Virtual Storage Access Method; eine Zugriffsmethode für von IBM-Großrechnern verwendete Dateien. VSAM-Dateien bestehen aus einem Metadatenkatalog sowie mindestens einer physischen Datei. Mehr zu VSAM in den Literaturhinweisen am Ende dieses Kapitels. Übrigens gibt es das Gerücht „VSAM ist grausam“.

⁴ Eine aktuelle Einführung in OS/390 finden Sie unter <http://www.informatik.uni-leipzig.de/cs/esvorles/index.html>

1.2 Qualitätsziele

Die primären Qualitätsziele von M&M lauten:

- Effizienz (Performance): Migration von ca. 20 Millionen Personen- und Kontodaten innerhalb von maximal 24 Stunden.
- Korrektheit: Die Migration muss revisionssicher und juristisch einwandfrei erfolgen. Hierzu sind geeignete Maßnahmen zur Fehlervermeidung und -erkennung nötig.

Nicht-Ziele

Was M&M nicht leisten soll:

- Änderbarkeit oder Flexibilität der fachlichen Transformationsregeln – die Migration ist einmalig.
- Es bestehen keine besonderen Anforderungen an Sicherheit – M&M wird nur ein einziges Mal produktiv betrieben, und das innerhalb eines gesicherten Rechenzentrums.

1.3 Stakeholder

Rolle	Beschreibung	Ziel
Management der Firma Fies und Teuer AG	Projektmanagement, Lenkungsgremium	Reibungslose und fehlerfreie Migration. Keine negative Presse.
Endkunden der Firma	20 Millionen Kunden. Treten im Projektverlauf nicht in Erscheinung, sind nur indirekt beteiligt.	Korrekte Migration ihrer Daten.
Revision oder Buchprüfung	Interne Revisionsabteilung, die (inhaltliche und juristische) Korrektheit der Migration durch Stichproben validieren wird.	Einfacher Zugang zu Stichproben. Korrekte Migration.
Projekt FINCS	Das Entwicklungs- und Architekturteam des künftigen Client/Server Finanzsystems.	Flexibilität beim geplanten Objekt- und Datenmodell.
Boulevard-presse	Legt besonderes Augenmerk auf Firma Fies und Teuer AG. Fehlverhalten wird gnadenlos publiziert.	Migrationsfehler, „Whistleblower“ im Projekt finden.

2 Einflussfaktoren und Randbedingungen

2.1 Technische Einflussfaktoren und Randbedingungen

Randbedingung	Erläuterung
Hardware- infrastruktur	IBM Mainframe als Plattform des Altsystems, Sun Solaris Cluster für das Zielsystem
Software- infrastruktur	▪ Sun Solaris als Betriebssystem der Zielumgebung ▪ Oracle als neue Datenbank ▪ JEE-kompatibler Applikationsserver als mögliche Betriebsumgebung
Ausgangsdaten in EBCDIC	Ausgangsdaten liegen in EBCDIC-Kodierung auf vier getrennten Bändern vor. Eine Vorsortierung durch die Fies und Teuer AG ist nicht möglich.
Systembetrieb	Batch
Grafische Oberfläche	keine, Bedienung kann per Konsole erfolgen
Programmier- sprachen	Aufgrund des vorhandenen Know-hows des Entwicklungsteams soll in Java programmiert werden.
Analyse- und Ent- wurfsmethoden	objektorientiertes Vorgehen
Datenstrukturen	Objektmodell der Zielumgebung bekannt (Bestandteil eines anderen Projektes)

2.2 Organisatorische Einflussfaktoren

Die Fies und Teuer AG ist als juristisch penibler Auftraggeber bekannt. Das Management versteht rein gar nichts von IT (und gerüchteweise auch nichts von Geld, aber das ist eine andere Geschichte).

Die verworrenen Eigentumsverhältnisse der Fies und Teuer AG lassen komplizierte Entscheidungsstrukturen befürchten.

Fies und Teuer AG hat langfristige Lieferverträge mit diversen IT-Dienstleistern abgeschlossen, die eine marktorientierte und freie Auswahl eventuell benötigter externer Mitarbeiter methodisch verhindern.

Eine hochgradig effektive externe Qualitätssicherung fordert eine umfangreiche Dokumentation (und prüft diese sogar auf inhaltliche Korrektheit – in der IT-Branche eher unüblich, doch ist das ebenfalls eine andere Geschichte).

3 Kontextabgrenzung

Fachlicher Kontext

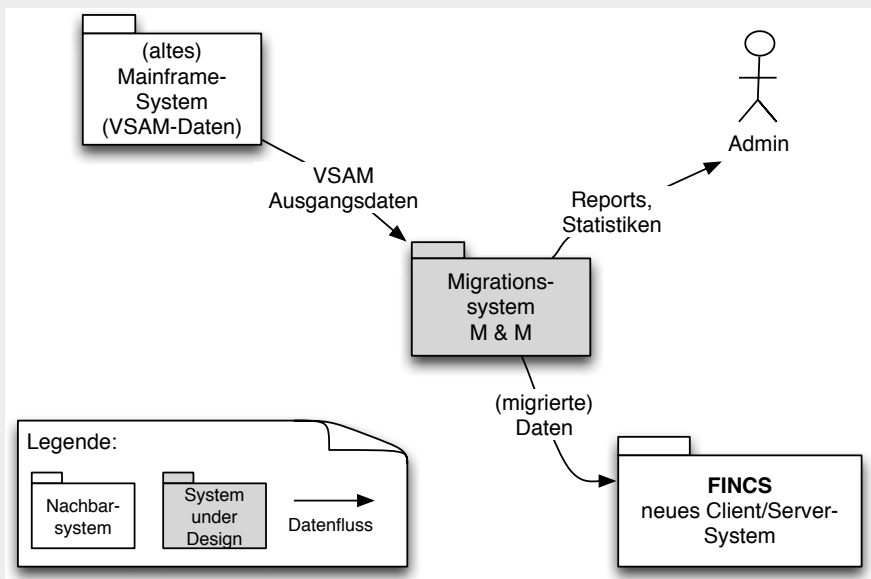


Bild 12.2 Fachlicher Kontext

Kurzbeschreibung der externen Schnittstellen

Schnittstelle	Beschreibung	Technologie
VSAM-Daten	Vier Arten von Daten (Personendaten, Adressdaten, Bankdaten, Kontodaten)	Bänder, LTO-1, siehe Sektion 3.2.1
Migrierte Daten	Gemäß des vom FINCS-Projekt erstellten Objektmodells	Übergabe des Objektgraphen über eine Session-Bean
Reports, Statistiken	Während der Migration: Kontinuierliche Ausgabe der Anzahl migrierter Personen. Nach der Migration: Ausgabe der Anzahl von Fehlersätzen	Konsole

Technischer oder Verteilungskontext

Die Migration wird auf zwei getrennten Servern ablaufen: ein Migrationsserver zur eigentlichen Datenmigration und Ausführung der technischen und fachlichen Transformationen, ein zweiter als Datenbankserver.

Das neue System (FINCS) wird direkt auf die von der Migration erstellte Datenbank zugreifen. Der Migrationsserver muss daher das künftige Klassen- und Tabellenmodell kennen.

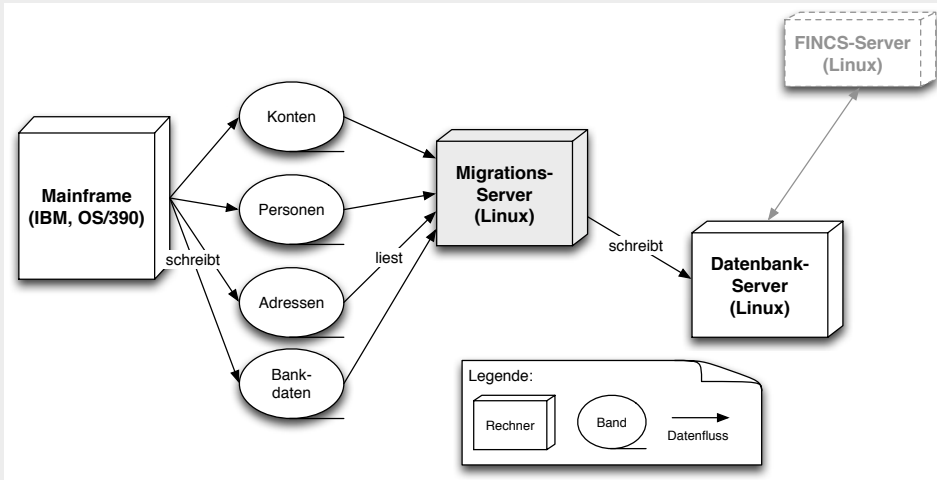


Bild 12.3 Technischer-/Verteilungskontext

Übersicht der Eingangsdaten(-schnittstellen)

Sämtliche Eingangsdaten werden auf jeweils einzelnen LTO-1-Bändern pro Datenart geliefert. Jedes dieser Bänder enthält bis zu 100 Gigabyte an Daten – die genaue Datenmenge kann aufgrund technischer Beschränkungen der Fies und Teuer AG vorab nicht spezifiziert werden.

Eingangsdaten	Beschreibung
Kontodaten	Trotz der Bezeichnung enthält dieses Band die Buchungs-/Bewegungsdaten für sämtliche Konten. Variables Satzformat mit 5–25 Feldern, gemäß FT-Satzart 43.
Personendaten	Stammdaten der von der Fies und Teuer AG betreuten Personen und Organisationen (Unternehmen, Vereine, Stiftungen, Behörden). Variables Satzformat mit 15–50 Feldern, gemäß FT-Satzart 27.
Adressdaten	Adress- und Anschriftsdaten, inklusive Informationen über Empfangsberechtigte, Orts- und Straßenangaben, Angaben über Haupt- und Nebenadressen (Zentralen, Filialen) für Organisationen und Unternehmen. Variables Satzformat mit 5–40 Feldern, gemäß FT-Satzart 33. Mehrere Sätze pro Person möglich.
Bankdaten	Daten externer Banken (Referenz- und Gegenkonten der Personen und Organisationen). Festes Satzformat, aber mehrere Sätze pro Person möglich.

4 Lösungsstrategie

Migration im Batch mit folgenden Kernkonzepten:

- Datenflussarchitekturstil mit Pipe-und-Filter-Elementen, mit relationaler Datenbank als Pipe (Qualitätsziel: Performance).
- Parallelisierung der rechenintensiven fachlichen Migration (Qualitätsziel: Performance).
- Fehlersensoren in sämtlichen Verarbeitungsschritten, Sammlung sämtlicher erkannter Fehler in der Fehlerdatenbank (Qualitätsziel: Korrektheit).

5 Bausteinsicht

Ausgehend von einer Menge von VSAM-Dateien (geliefert als Bänder), konvertiert („migriert“) die M&M-Anwendung sämtliche Datensätze in das neue Objektmodell.

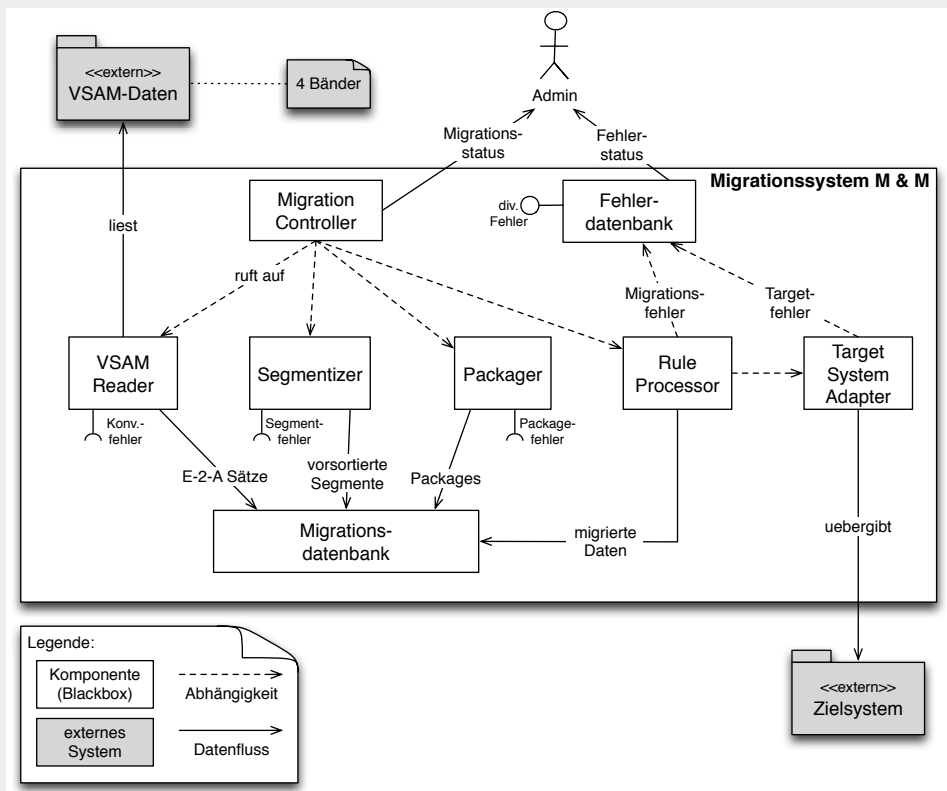


Bild 12.4 Whitebox-Darstellung des Migrationssystems, Level-1

5.1 M&M Bausteinsicht Level 1

Grundlegende Strukturentscheidung: Datenfluss mit Pipes-und-Filters

Zur Lösung des M&M-Migrationsproblems wenden wir ein modifiziertes Pipes-und-Filter-Architekturmuster an, ausführlich beschrieben im Buch „Pattern-Oriented Software Architecture“ von Buschmann et al. ([Buschmann+96]).

Ein wesentliches Entscheidungskriterium für diese Architektur war die Möglichkeit, Teile dieser Bausteine im Bedarfsfall (Performance-Bedarf) zu parallelisieren.

Entgegen dem klassischen Pipe-und-Filter-Muster arbeitet der Eingangsbaustein (VSAM Reader) seine Eingangsdaten erst vollständig ab, bevor der nächste Filter aufgerufen wird.

Baustein	Bedeutung
Migrations-Controller	Koordiniert die übrigen Bausteine und meldet den aktuellen Migrationsstatus an den Admin. Es handelt sich um eine einzelne Java-Klasse mit ausführlicher Ausnahme- und Fehlerbehandlung sowie einer Schnittstelle für Reporting und Auswertungen. Keine weitere Doku.
VSAM-Reader	Führt Formatkonvertierung von EBCDIC nach ASCII/UNICODE durch. Details siehe Sektion 5.1.1 .
Segmentizer	Vorbereitung der Parallelisierung des „Rule Processors“: Zuordnung der einzelnen Datensätze der unterschiedlichen Datenquellen (Personen-, Konto-, Bank- und Adressdaten) zu zusammengehörigen Datengruppen oder Segmenten. Elemente eines Segments können unabhängig von Inhalten anderer Segmente bearbeitet werden. Notwendig ist die Aufteilung der Eingangsdaten auf mindestens drei bis fünf verschiedene Segmente. Es geht um eine möglichst <i>schnelle</i> Einteilung in Segmente, nicht um Optimierung oder Balancierung der Segmente. Daher liegt das Hauptaugenmerk auf Performance. Keine weitere Doku.
Rule-Processor	Erzeugt aus den formatkonvertierten Alt-Daten einen Objektgraphen gemäß den vom Neusystem FINCS erwarteten Klassenstrukturen. Siehe Sektion 5.1.2 .
Migrationsdatenbank	Speichert die konvertierten und segmentierten Migrationsdaten zwischen. Dient im Sinne des Pipe&Filter-Architekturmusters als Pipe, d. h. enthält keinerlei eigene Logik. Die Migrationsdatenbank ist eine Menge von Tabellen einer relationalen Datenbank (Person, Konto, Bank, Adresse plus einige Schlüssel Tabellen).
Target-System-Adapter	Anbindung an das Zielsystem FINCS. Siehe Sektion 5.1.3 .

5.1.1 VSAM Reader

- *Zweck/Verantwortlichkeit:* Konvertiert Datensätze aus VSAM-Format (EBCDIC-Codierung) in ein unter Unix direkt verarbeitbares ASCII- oder UNICODE-Format. Gleichzeitig werden die Bestandteile der VSAM-Daten (teilweise besitzen einzelne Bits- oder Bitgruppen in der EBCDIC-Darstellung besondere Bedeutung) in einzeln identifizierbare Datenelemente aufgelöst. Der VSAM Reader führt keinerlei fachliche Prüfungen durch, sondern lediglich Formatkonvertierungen.
- *Schnittstelle(n):* Eingabe von VSAM-Dateien. Für jede Dateart (Personen, Kontodaten, Adressdaten, Bankdaten) existieren jeweils einzelne Reader-Komponenten, die den Satzaufbau ihrer jeweiligen Eingabedateien kennen.
- *Variabilität:* keine, weil das Gesamtsystem nur einmalig ablaufen soll
- *Offene Punkte:* Für eine spätere Nutzung bei anderen Unternehmen könnten die einzelnen Konverter „pluggable“ gemacht werden. Dies ist zurzeit nicht vorgesehen.

5.1.2 Rule Processor (und Packager)

Zweck/Verantwortlichkeit: erzeugt einen Objektgraphen mit der jeweiligen Person (natürliche oder nichtnatürliche Person) als Wurzelknoten sowie allen zugehörigen fachlichen Objekten als Zweigen (Adressen, Bankverbindungen, Konten, Buchungen).

Der Rule Processor führt damit den eigentlichen fachlichen Migrationsschritt aus der alten (datenartorientierten) Darstellung in die neue (personenorientierte) Darstellung durch. Eventuell sind hierzu mehrere Iterationen über die Regelbasis notwendig.

Beispiele (sind Bestandteil der Anforderungsspezifikation; hier zur Illustration des notwendigen Vorgehens erneut dargestellt):

- Bei der Migration einer verheirateten Frau müssen vorher der Ehepartner sowie gegebenenfalls sämtliche früheren (geschiedenen, verschwundenen oder gestorbenen) Ehepartner migriert werden.
- Bei der Migration von Minderjährigen müssen vorher die natürlichen Eltern, Vormunde und Pflegeeltern migriert werden.
- Bei der Migration von Firmen müssen vorher sämtliche Vorgängerunternehmen, aus denen die aktuelle Firma hervorgegangen ist, migriert werden. Gleiches gilt analog für Vereine oder Verbände.

Damit diese fachlich zusammengehörigen Daten oder Datengruppen durch parallel ablaufende Prozesse migriert werden können, müssen in einem Vorverarbeitungsschritt sämtliche zusammengehörigen Daten als „Paket“ zusammengestellt werden. Diese Aufgabe übernimmt der Packager.

5.1.3 Target-System Adapter

Zweck/Verantwortlichkeit: übernimmt den vom Rule Processor erzeugten Objektgraphen in das Objektmodell des (neuen) Zielsystems. Hierfür sind eventuell kleinere Modifikationen notwendig, je nach endgültig entschiedener Laufzeitumgebung auch nur die abschließende Persistierung.

Nach erfolgreicher Übernahme in das neue System (Zielsystem) liefert der Target-System Adapter eine Meldung zurück – im Fehlerfall wird der gesamte Objektgraph zusammen mit den Fehlermeldungen des Zielsystems in die Fehlerdatenbank geschrieben.

5.2 Bausteinsicht Level 2

Dieser Teil verfeinert die Blackbox-Bausteine von Level 1. Aufgrund des Beispielcharakters dieser Dokumentation sind nur wenige Bausteine detailliert dargestellt.

Anmerkung: Auch in einer echten Dokumentation können Sie sich auf die besonders wichtigen Komponenten konzentrieren. Geben Sie jedoch für sämtliche Komponenten eine Begründung, warum Sie auf eine detaillierte Darstellung verzichten.

5.2.1 VSAM-Reader Whitebox

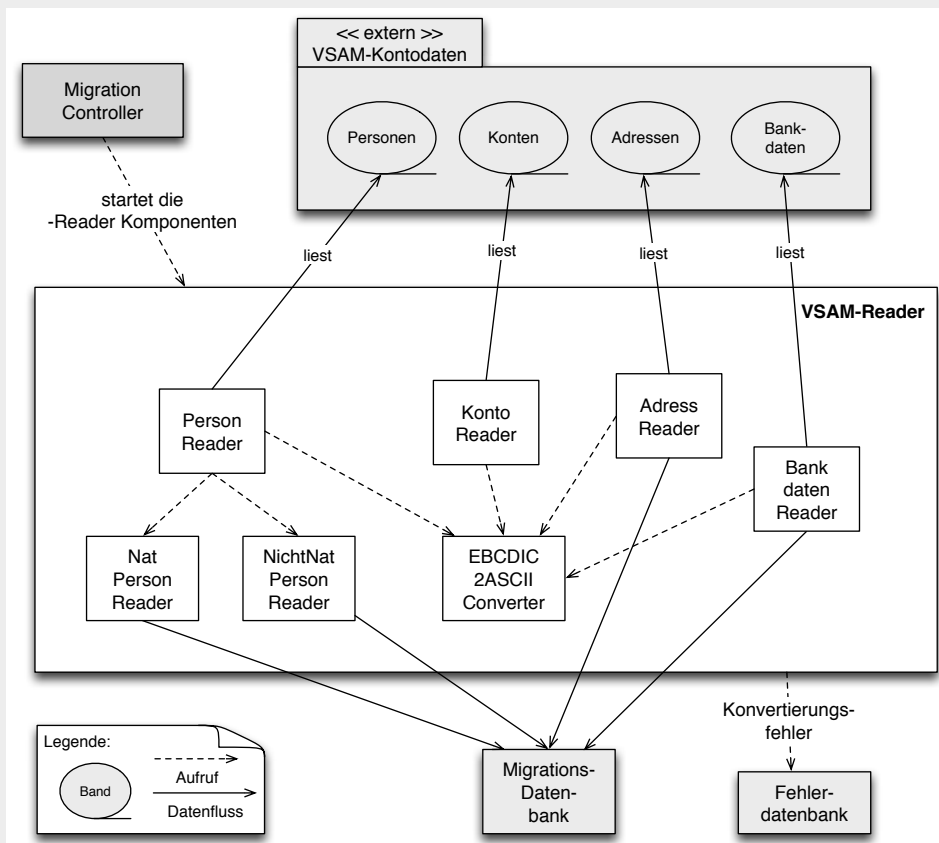


Bild 12.5 Interner Aufbau des VSAM-Reader

Baustein	Bedeutung
EBCDIC2ASCII Converter	Konvertiert eine Zeichenkette von EBCDIC nach ASCII. Als einziger Baustein des Systems in ANSI-C implementiert. Behandelt nur Zeichenketten bis zur Länge von max. 256 Zeichen. Siehe: http://en.wikipedia.org/wiki/EBCDIC
Reader-Komponenten	Die Reader-Komponenten lesen jeweils eine bestimmte Datenart aus „ihrem“ Eingabeband. Sie überführen sämtliche darin enthaltenen Daten in eigenständige Attribute. Die einzelnen Reader kapseln die Syntax der „alten“ VSAM-Repräsentation sowie die technischen Details der Datencodierung. Die einzelnen Reader schreiben ihre Ergebnisse in die Migrationsdatenbank, im Fehlerfall rufen sie die Fehlerdatenbank.

5.2.2 Rule Processor Whitebox

Der Rule Processor führt die eigentliche fachliche Migration von Datensätzen durch. Er kombiniert die unterschiedlichen Kategorien von Eingangsdaten (Personen, Konten und Buchungen, Adressen, Bankverbindungen) miteinander und bearbeitet alle fachlichen Sonderfälle. Insbesondere besorgt der Rule Processor die benötigte Reorganisation der Daten, sodass künftig die Personen als Einstiegspunkte in die Navigation dienen, statt, wie früher, die Konten.

Strukturelevante Entwurfsentscheidungen

- Kein kommerzieller Regelinterpreter: Nach einigen Versuchen mit kommerziellen Regelmaschinen entschied sich das Projektteam gegen den Einsatz einer solchen Komponente. Letztendlich war der Grund, dass die Umsetzung der fachlichen Regeln in eine Regelsprache keine wesentliche Vereinfachung gegenüber der Programmierung in Java gebracht hätte.

Anmerkung: Aus heutiger Sicht (also posthum ...) würde ich diese Entscheidung nochmals überdenken und insbesondere Open-Source-Regelframeworks⁵ auf ihre Tauglichkeit hin überprüfen.

- Keine explizite Regelsprache: Der Ansatz, die Regeln von den Fachexperten formulieren und nur durch einen Interpreter ausführen zu lassen, scheiterte an der Komplexität des gesamten Datenumfelds. Die effiziente und performante Navigation innerhalb des jeweils benötigten Objektraumes war mit einer Regelsprache nicht zu bewerkstelligen. Siehe dazu die Anmerkung zum Regelinterpreter.

⁵ Wie beispielsweise JBoss-Drools.

Beispiele:

- Einpersonengesellschaften als nichtnatürliche Personen können teilweise durch den Personen-Mapper abgebildet werden, abhängig vom Gründungsdatum der Gesellschaft und dem Verlauf der Firmengeschichte (muss immer eine Einpersonengesellschaft gewesen sein und darf nicht aus einer Mehrpersonengesellschaft hervorgegangen sein).
- Natürliche Personen, die in der Vergangenheit Mitinhaber von Mehrpersonengesellschaften gewesen sind und darüber hinaus heute noch als Unternehmer tätig sind, müssen in Abhängigkeit von der Rechtsform der früheren Gesellschaft sowohl als natürliche als auch als nichtnatürliche Person geführt werden. Die Konto- und Buchungsinformationen sind entsprechend aufzuteilen.

<<Weitere Architekturbausteine werden in diesem Beispiel nicht dokumentiert>>

6 Laufzeitsicht

In der ersten Phase lesen die einzelnen Reader des VSAM-Readers alle angelieferten Dateien („Bänder“) und befüllen die Migrationsdatenbank. Danach teilt der Segmentizer die Daten in parallel bearbeitbare Segmente auf.

Der RuleProcessor lässt durch einen Packager (in Bild 12.7 durch den Aufruf „getPackage“ gekapselt) alle für die Migration einer Person oder eines Kontos notwendigen Daten aus der Migrationsdatenbank selektieren und führt anschließend die fachlichen Regeln auf diesem Datenpaket („Package“) aus.

Der Target System Adapter schreibt die migrierten Daten in die neue Datenbank.

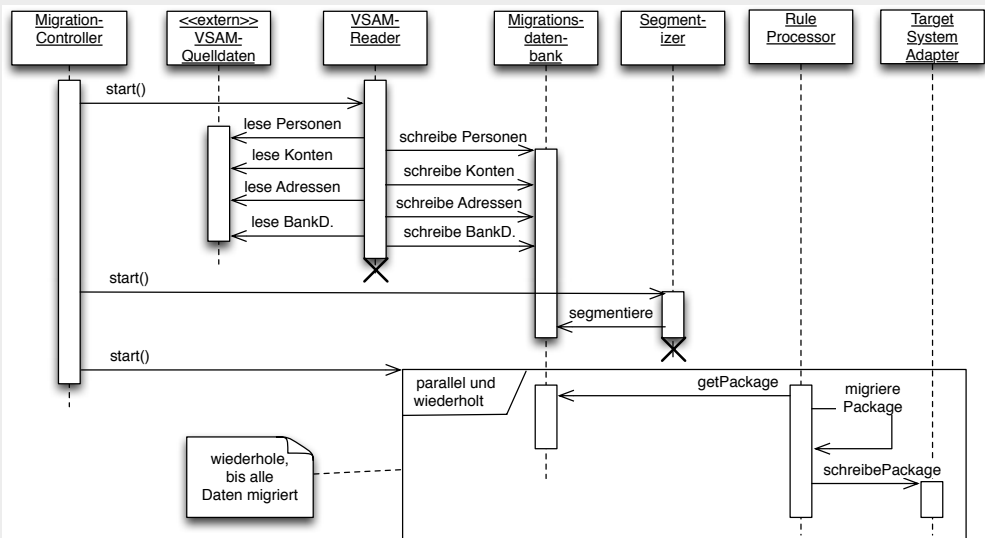


Bild 12.7 Laufzeitsicht der M&M-Migration

7 Verteilungssicht

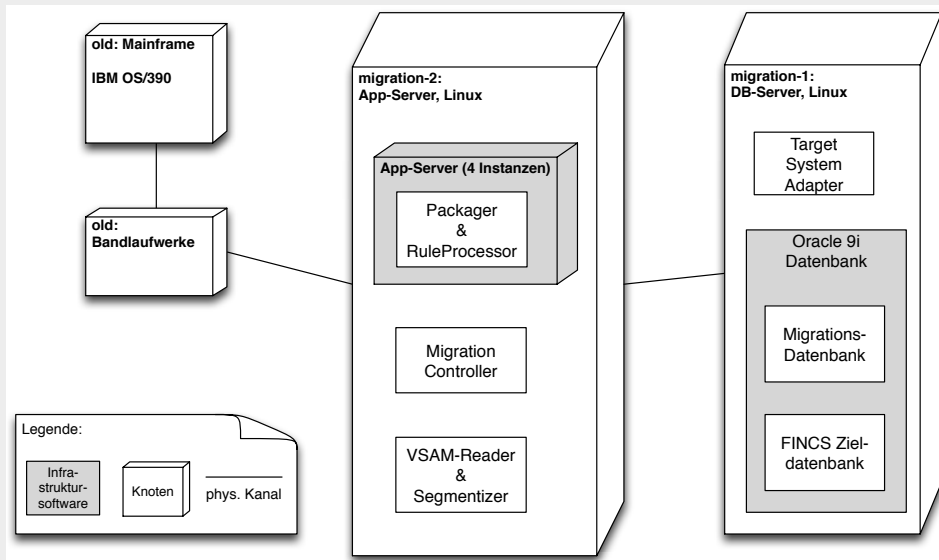


Bild 12.8 Verteilungssicht: Migration auf zwei getrennten Servern

Die gesamte Migration findet auf einem eigenständigen Linux-Server („migration-2“) statt, auf dem in einem EJB-Container mehrere parallele Instanzen der eigentlichen Migrationskomponenten (Packer & RuleProcessor) ablaufen.

Über einen schnellen Bus ist dieser Server mit dem DB-Server („migration-1“) verbunden, auf dem der VSAM-Reader und der Segmentizer die initiale Befüllung der Migrationsdatenbank vornehmen.

Anmerkung: In einer echten Architekturdokumentation sollten Sie hier weitere Leistungsdaten der Hardware (Rechner, Netze, Bus-Systeme etc.) beschreiben.

8 Technische Konzepte

8.1 Persistenz

Die Datenspeicherung im neuen System übernimmt ein Java-Persistenz-Framework (Hibernate). Das Mapping der Objektstruktur auf die relationale Datenbank wird vom Framework komplett übernommen.

Für die eigentlichen Migrationskomponenten wird Persistenz damit nahezu transparent: Ist ein Objektgraph im Speicher aufgebaut, kann er auf einmal gespeichert werden.

Persistenz der Zwischenformate und Fehlerfälle

Die Eingangsdaten in sequenziellem VSAM-Format werden über SQL direkt in die Migrationsdatenbank geschrieben. Ebenso gelangen sämtliche Fehlersätze per SQL in die Fehlerdatenbank.

8.2 Ablaufsteuerung

Die Koordination und Ablaufsteuerung aller Migrationsprozesse wird durch einen zentralen Baustein (MigrationController) durchgeführt.

Die gesamte Verarbeitung erfolgt satzweise. Die Personendatei wird satzweise verarbeitet, sämtliche zugehörigen Kontoinformationen werden diesen Sätzen zugeordnet und die resultierenden Datenpakete anschließend gemeinsam migriert.

Aus Kostengründen hat der Auftraggeber eine kommerzielle Workflow- oder Prozess-Engine abgelehnt.

8.3 Ausnahme- und Fehlerbehandlung

Aufgrund der hohen Korrektheitsanforderungen der Auftraggeber werden sämtliche Ausnahmen beziehungsweise nicht migrierbaren Datensätze in einer Fehlertabelle gespeichert.

Sämtliche Fehlersätze müssen nach der Migration von Sachbearbeitern manuell migriert werden, wofür lediglich ca. 200 Personentage Aufwand zur Verfügung stehen. Jeden Tag kann ein Sachbearbeiter im Mittel 25 Personen migrieren, daher dürfen in der Fehlertabelle höchstens 5000 ($= 200 \cdot 25$) Datensätze enthalten sein.

8.4 Transaktionsbehandlung

Die einzelnen Migrationsschritte speichern ihren Zustand jeweils satzweise ab. Dadurch ist zu jedem Zeitpunkt ein Wiederaufsetzen durch Neustart der gesamten Anwendung möglich.

8.5 Geschäftsregel und Validierung

Aus Kosten- und Lizenzgründen hat der Auftraggeber die Verwendung einer kommerziellen Regelmaschine (wie ILOG oder VisualRules o. Ä.) a priori ausgeschlossen.

Die Geschäftsregeln (hier: Migrationsregeln) wurden vollständig in Java codiert.

8.6 Kommunikation und Integration

Die Integration mit dem Quellsystem erfolgt über den Austausch von VSAM-Bändern.

Integration mit dem Zielsystem über Java Session-Beans und den Java Application Server. Der Objektgraph der migrierten Daten wird als POJO-Parameter übermittelt (*plain old java object*).

9 Entwurfsentscheidungen

Entscheidung	Datum & Entscheider	Begründung, Konsequenz, Alternativen
Beschreibung der Migrationslogik in Java, NICHT in einer vereinfachten natürlichen Sprache	PL	Nach einigen Versuchen, unter anderem mit eigens formulierten Regelsprachen (mit Lex und Yacc) befand das Entwicklungsteam, dass eine Umsetzung fachlicher Regeln in eine solche Regelsprache keine wesentliche Vereinfachung gegenüber der Programmierung in Java gebracht hätte.
Kein kommerzieller Regelinterpreter	PL	Nach einigen Versuchen mit kommerziellen Regelmасhinen entschied sich das Projektteam gegen den Einsatz einer solchen Komponente. Siehe Entscheidung zu Regelsprache. <i>Anmerkung:</i> Aus heutiger Sicht (also posthum ...) würde ich diese Entscheidung überdenken und insbesondere Open-Source-Regelframeworks wie JBoss-Drools auf ihre Tauglichkeit hin überprüfen.
Pipes-und-Filter-Architektur	Architekt, Entwickler	Die Parallelisierbarkeit der eigentlichen Migrationsregeln (im RuleProcessor) wird hierdurch vereinfacht. Es wird keinerlei GUI benötigt.
Dual-Server	Auftraggeber	Aufgrund der hohen Performance-Anforderungen werden Datenbank und Migrationslogik auf zwei getrennten Servermaschinen ausgeführt. Als DB-Server kommt die später vom neuen Kontosystem genutzte Maschine zum Einsatz.
Die Verarbeitung der fachlichen Regeln durch den Rule Processor (siehe Sektion 5.1.6 bzw. 5.2.2) wird durch mehrere parallele Prozesse (innerhalb des J2EE Application Servers) durchgeführt.	PL, Kunde	Anforderung seitens des Kunden. Eine rein sequenzielle Bearbeitung aller vorhandenen Personen wäre in der verfügbaren Laufzeit von 24 Stunden nicht möglich gewesen.

10 Qualitätsszenarien

<Entfällt im Beispiel>

11 Risiken

<Entfällt im Beispiel>

12 Glossar und Referenzen

Das Glossar entfällt im Beispiel.

Mehr zu VSAM im frei verfügbaren IBM-Redbook:

<http://www.redbooks.ibm.com/abstracts/sg246105.html>

Anmerkungen zum System

Einige Worte zur Klärung: Dieses Projekt hat wirklich stattgefunden, und das M&M-System ist wirklich entwickelt worden. Der Auftraggeber, die Fies und Teuer AG, hieß in Wirklichkeit natürlich anders, auch ist der fachliche Inhalt ein wenig vereinfacht worden.

Das echte Migrationsprojekt fand um 2002/2003 irgendwo in Deutschland statt. Über die Laufzeit von gut zwölf Monaten analysierten mehr als zehn Fachexperten die Logik und Geschäftsregeln der bestehenden Mainframe-Systeme und erstellten auf dieser Basis das fachliche Migrationskonzept. Hierbei galt es, die besonderen Anforderungen an Nachweispflichten zu berücksichtigen, ebenso die über die Zeit veränderten gesetzlichen Rahmenbedingungen. Verzahnt damit erstellte ein Team von mehr als zehn Entwicklern und Architekten die passende Software.

Einige der Stolpersteine und Herausforderungen des „echten“ M&M-Projektes:

- Die Datenqualität der Ausgangsdaten schwankte stark: So musste das Projekt die ursprünglich eingeführte Unterscheidung zwischen „Muss“ und „Optional“ für Datenfelder stark einschränken, weil 2–3 % der mehr als 20 Millionen Personendatensätze nicht alle (heute benötigten!) Muss-Attribute enthielten.
- Logische oder fachliche Attribute waren durch extreme Speicherplatzoptimierungen stark verklausuliert und immens schwer identifizierbar. Beispiel: „Wenn es sich um einen Fall des Typs 42 handelt, dann gibt Bit 4 (im EBCDIC-Format) des Datenrecords an, ob es sich um eine Hin- oder Rückbuchung handelt.“
- Die Logik der bestehenden Programme war teilweise älter als 30 Jahre – und für manche der Altprogramme gab es keine Ansprechpartner mehr.

■ 12.2 Beispiel: Kampagnenmanagement im CRM

1 Einführung und Ziele

Zweck des Systems

Dieses Dokument beschreibt die Softwarearchitektur der MaMa⁶-Plattform zur Steuerung von Prozessabläufen zwischen Anbietern, Dienstleistern und Endkunden in Massenmarktsegmenten. MaMa ist technische Basis für Produkte, die für spezifische Anwendungsbereiche angepasst oder konfiguriert werden müssen – sozusagen die Mutter für konkrete Systeme. Auftraggeber und gleichzeitig Betreiber von MaMa ist ein innovatives Rechenzentrum (IRZ), das im Rahmen von Geschäftsprozess-Outsourcing den Betrieb von Software als Service für seine Mandanten anbietet.⁷ IRZ wird konkrete Ausprägungen von MaMa betreiben.

Folgende Beispiele illustrieren die Einsatzmöglichkeiten von MaMa:

- Telekommunikationsunternehmen (= Mandanten) bieten ihren Kunden neue Tarife an. Kunden können diese Angebote über verschiedene Kanäle (Brief, Fax, Telefon) annehmen oder Fragen dazu stellen.
- Handelsunternehmen (= Mandanten) senden spezifische Aktionen (Werbung oder Marketing, etwa: Preisausschreiben oder zielgruppenspezifische Sonderangebote) an ausgewählte Teile ihrer Kunden. Diese können wiederum über unterschiedliche Kanäle auf diese Aktionen reagieren.
- Versicherungsunternehmen (= Mandanten) senden an ihre Versicherten Vorschläge zur Änderung oder Anpassung bestimmter Verträge. Die Versicherten können über verschiedene Kanäle (Brief, E-Mail, Fax, Telefon oder persönliche Gespräche in Geschäftsstellen) auf diese Vorschläge reagieren.

Etwas verallgemeinert, sehen Sie diese Aufgaben in Bild 12.9.

Definitionen der Begriffe Mandant, Partner und Marktteilnehmer finden Sie in Section 12 dieser Beispieldokumentation (Glossar).

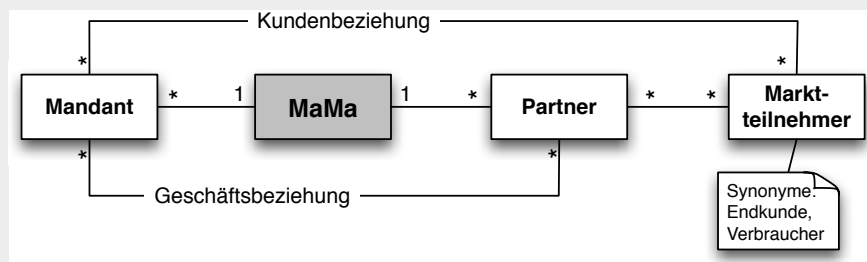


Bild 12.9 Zweck des Systems: Koordination von Prozessabläufen zwischen Mandanten, Partnern und Marktteilnehmern im Massenmarkt

⁶ MaMa steht für **M**assen**M**arkt.

⁷ Die Rahmendaten und den Namen dieses Projektes habe ich erfunden, das hier beschriebene System existiert jedoch wirklich.

1.1 Fachliche Aufgabenstellung

Die wichtigsten funktionalen Anforderungen sind:

- MaMa unterstützt die Ablaufsteuerung bei CRM-Kampagnen, die das Rechenzentrum IRZ für seine Auftraggeber (genannt „Mandanten“ oder „Anbieter“) im Rahmen des Business Process Outsourcing abwickelt. Dazu gehören folgende Funktionen:
 - Tarif- und Vertragsänderungen für Versicherungsgesellschaften, Telekommunikations- und Internetanbieter sowie Energieversorger. Ein Beispiel dazu finden Sie in Abschnitt 1.1.1 weiter unten.
 - Rückfragen zu Stammdaten für Versicherungsgesellschaften oder die Gebühreneinzugszentralen von Rundfunk- und Fernsehgesellschaften.
 - Abfrage zusätzlicher Stamm- oder Personendaten für Versicherungsunternehmen, die im Rahmen von Gesetzesänderungen notwendig werden.
- MaMa ist Basis einer Produktfamilie und muss für unterschiedliche Ein- und Ausgangsschnittstellen konfigurierbar sein. Siehe dazu Abschnitt 1.1.2.

Anmerkung: Aufgrund der hohen fachlichen Komplexität dieses Systems möchte ich die fachlichen Anforderungen anhand eines Beispiels näher erläutern. In einer echten Projektsituation sollte die Anforderungsdokumentation das leisten – die Ihnen in diesem Beispiel leider nicht zur Verfügung steht.

1.1.1 Einsatz von MaMa für Vertrags- und Tarifänderungen bei Telekommunikationsunternehmen

Der Mobilfunkanbieter MoF⁸ schlägt seinen Kunden (synonym: Marktteilnehmern) neue Tarife vor: Manche der bestehenden Tarife oder Tarifkombinationen sind überaltert und technisch nicht mehr aktuell. Andere Tarife sind nicht mehr marktgerecht und können gegen leistungsfähigere oder kostengünstigere ausgetauscht werden. Da solche Änderungen jedoch in die Vertragsverhältnisse zwischen MoF und seinen jeweilig betroffenen Kunden eingreifen, ist zwingend die schriftliche Zustimmung der Kunden erforderlich.

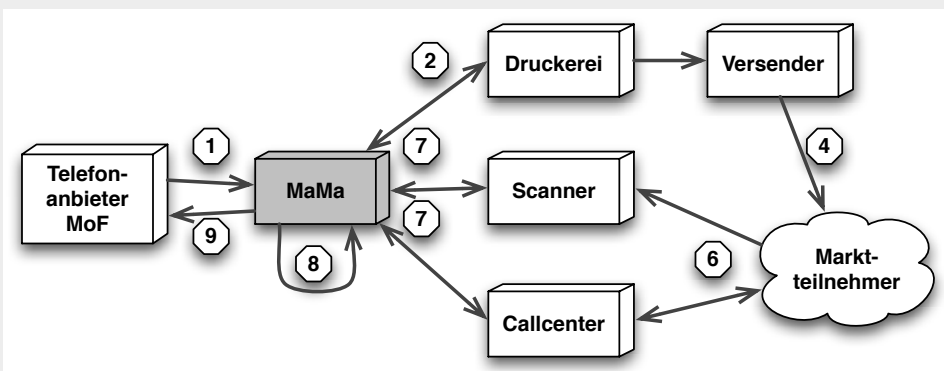


Bild 12.10 Ablauf einer Kampagne für Vertrags- und Tarifänderungen

⁸ Wie Sie richtig vermuten, habe ich den Namen dieses Unternehmens frei erfunden.

MoF möchte im Rahmen einer Kampagne alle betroffenen Kunden schriftlich über die Angebote dieser neuen Tarifmöglichkeiten informieren. Die Kunden können schriftlich, per Fax oder in Filialen eine für sie passende Tarifikombination auswählen und damit formell eine Vertragsänderung beauftragen. Konkret läuft diese Kampagne wie folgt ab:

1. MoF selektiert in seinen internen Systemen die betroffenen Kunden und exportiert deren Stamm-, Tarif- und Vertragsdaten an MaMa.
2. MaMa übergibt die relevanten Teile dieser Daten an eine Druckerei, die personalisierte Anschreiben für sämtliche betroffenen Kunden erstellt und nach Regionen vorsortiert an einen Briefdienstleister liefert.
3. MaMa benachrichtigt die weiteren Partner über diese Kampagne und übergibt die jeweils benötigten Daten. Hierzu gehören:
 - Ein Scan-Dienstleister, der Rücksendungen entgegennehmen und digitalisieren wird.
 - Ein Telefondienstleister⁹, der alle telefonischen Rückmeldungen oder Rückfragen bearbeitet.
 - Sämtliche Filialen beziehungsweise Vertriebspartner von MoF, damit auch persönliche Rückmeldungen der Kunden dort korrekt bearbeitet bzw. weitergeleitet werden.
4. Der Briefdienstleister transportiert die gedruckten Sendungen an die Kunden und übermittelt elektronisch die aufgetretenen Zustellprobleme (verzogen mit Nachsendeantrag, unbekannt verzogen, unbekannt, Annahme verweigert).
5. Die Kunden können auf verschiedene Arten auf dieses Angebot reagieren:
 - Kunde füllt das beigefügte Antwortschreiben vollständig und korrekt aus.
 - Kunde füllt das beigefügte Antwortschreiben unvollständig oder fehlerhaft aus.
 - Kunde hat Rückfragen.
 - Kunde reagiert nicht.
6. In allen Fällen kann der Kunde folgende Arten der Rückmeldung wählen, die letztlich alle durch MaMa koordiniert und bearbeitet werden sollen:
 - Briefsendung an eine (kampagnenspezifische) Adresse.
 - Rückmeldung per Fax.
 - Rückmeldung per E-Mail oder Web-Formular.
 - Persönliche Rücksprache in einer Filiale von MoF.
 - Telefonische Rückmeldung an ein Callcenter.
7. Die jeweiligen Partner übermitteln periodisch oder kontinuierlich alle ihnen vorliegenden Rückmeldungen an MaMa.
8. MaMa ermittelt aus den vorliegenden Antwortdaten der verschiedenen Partner täglich (bei Bedarf auch häufiger) die passenden Folgeaktionen:
 - Es liegen sämtliche Daten inklusive der rechtsgültigen Unterschrift vor, die MoF für eine Tarif- oder Vertragsänderung benötigt. In diesem Fall werden diese Daten an MoF übermittelt.
 - Teile der benötigten Daten fehlen noch und müssen durch passende Rückfragen beim Kunden ermittelt werden.

⁹ („Denglisch“:) Callcenter

- Eine fehlende Unterschrift muss schriftlich über ein erneutes Anschreiben eingeholt werden.
- Fehlende Tarifinformationen können durch ein Callcenter telefonisch erfragt werden. Wenn der Kunde dreimal nicht erreicht werden konnte, wird er erneut angeschrieben.

9. MoF ändert aufgrund der von MaMa gelieferten Ergebnisdaten die Verträge und Tarife der jeweiligen Kunden.

Wichtig für den Mobilfunkanbieter MoF ist, dass er nur mit MaMa Daten austauschen muss, um diese Kampagne durchzuführen. Sämtliche unterstützenden Prozesse erledigt MaMa durch Kooperation mit passenden Partnern!

1.1.2 Konfiguration einer Kampagne

MaMa ist die Basis einer Produktfamilie, d. h. nur in einer konkreten Ausprägung („Konfiguration“) ablauffähig. Dies bedeutet, dass vor der Durchführung einer Kampagne verschiedene Festlegungen getroffen werden müssen. Die folgende Tabelle gibt einen Überblick über diese Konfiguration, orientiert am Beispiel der MoF-Kampagne aus dem vorigen Abschnitt.

Schritt innerhalb der Kampagne	Beispiel (aus Mobilfunk-Domäne)
Mögliche Aktionen der Kampagne definieren (Input- und Output-Aktionen*)	Output-Aktionen: BriefDrucken InfoAnCallCenter InfoAnScandienstleister ErgebnisseAnMandant Input-Aktionen: StammdatenInput ScandatenInput CallCenterInput FilialenInput
Marktteilnehmerdaten sowie zugehörige Kampagnendaten (i. d. R. Vertrags-, Tarif- oder Angebotsdaten) als Erweiterung des MaMa-Basismodells in UML modellieren	Attribute der Marktteilnehmer: Bestehender Tarif Liste möglicher neuer Tarife Vertragsnummer Ablaufdatum und Laufzeit des aktuellen Vertrags
Metadaten der Kampagne definieren und festlegen	z. B. Start- und Endtermine, Stichtage für Aktivitäten, Adresse für Rücksendungen, Telefonnummern für Rückrufe bei Kunden, Zugangsdaten für Auftraggeber und Partner für ftp-Server (zum Up-/Download von Daten)
Kampagne durchführen	Wird für das Beispiel in Abschnitt 1.1.1 genauer beschrieben.

* Die Bedeutung von „Aktionen“ wird in Sektion 8.1 über Ablaufsteuerung näher erklärt.

1.2 Qualitätsziele

Die primären Qualitätsziele von MaMa lauten:

1. Flexibilität: Kampagnen sollen ohne Programmierung und ausschließlich durch Konfiguration aufgesetzt werden können. Das betrifft:
 - die spezifischen Datenstrukturen einer Kampagne müssen innerhalb von 2 h vollständig konfiguriert werden können;
 - die ein- und ausgehenden Schnittstellen zu den Partnern (Übertragungsprotokolle, Aufbau der Datensätze, Text- oder Binärschnittstellen, CSV-, Fix- oder XML-Formate, einmalige oder periodische Übertragung, Push- oder Pull-Aufrufe der Schnittstelle sowie die Art der Authentifizierung, Autorisierung, Datenverschlüsselung und -kompression) sollen innerhalb von 8 Arbeitsstunden vollständig konfiguriert werden können.
 - die notwendigen Abläufe der Kampagne inklusive ihrer Plausibilitätsprüfungen.
 - Die Im- und Exportkonfiguration sowie die individuellen Prozessabläufe sollen innerhalb eines Arbeitstages vollständig konfiguriert werden können.
2. Datenschutz und -sicherheit müssen beispielsweise für Mandanten aus dem Gesundheits- oder Versicherungswesen auditierbar sein. Bei Bedarf müssen daher sämtliche Datenänderungen einer Kampagne revisionssicher protokolliert werden können.
3. Performance: Kampagnen können praktisch beliebig umfangreich werden. Für einige Mandanten sollen beispielsweise mehrere 10 Millionen Endkunden innerhalb einer Kampagne bearbeitet werden. Weitere Mengengerüste:
 - Gesamtgröße aller Attribute einzelner Kundendatensätze innerhalb einer Kampagne zwischen 1 kB und 15 Mbyte (für Kampagnen mit umfangreichen Bild- oder Scandaten).
 - Anzahl möglicher Partnerunternehmen: unter 100
 - Anzahl von Mandanten/Anbietern: unter 100
 - Anzahl gleichzeitiger Kampagnen: unter 100
4. Mandantentrennung: Die Daten der jeweiligen Mandanten einer Kampagne müssen vollständig voneinander getrennt sein. Es darf unter keinen Umständen passieren, dass beispielsweise die Kundendaten mehrerer Versicherungen miteinander verwechselt werden.

1.3 Stakeholder

Name/Rolle	Ziel/Berührungspunkt	Notwendige Beteiligung
Management des Auftraggebers IRZ	Fordert flexible und leistungsfähige Basis für ihre zukünftigen Geschäftsfelder.	Periodische Abstimmung über aktuelle und geänderte Anforderungen, Lösungsansätze
Softwareentwickler, die MaMa mitentwickeln	Müssen den inneren Aufbau von MaMa gestalten und implementieren.	Aktive Beteiligung an technischen Entscheidungen
Partner	Schnittstellen zwischen MaMa und Partnern	Partner müssen Daten liefern bzw. empfangen und haben großes Interesse an flexiblen Schnittstellen.
Normungsgremien	MaMa wird in gesetzlich normierten Bereichen (z. B. Krankenversicherung) eingesetzt.	Normungsgremien (im Beispiel nicht namentlich genannt) geben rechtliche Randbedingungen vor.

Zu MaMa gibt es in der Realität eine Reihe weiterer technischer Dokumente, insbesondere die Schnittstellendokumentation, mit deren Hilfe Partnerunternehmen (wie Druck-, Scan- oder Callcenter-Dienstleister) eine Anbindung an MaMa realisieren können (obwohl in der Praxis deren Schnittstellen meist feststehen und die Betreiber von MaMa die Anbindung konfigurieren!).

2 Einflussfaktoren und Randbedingungen

2.1 Technische Einflussfaktoren

Randbedingung	Erläuterung
Softwareinfrastruktur	Linux als Betriebssystem der Ablaufumgebung MySQL beziehungsweise bei Bedarf Oracle als Datenbank Java 6 oder 7 als Softwareplattform im Betrieb verfügbar
Systembetrieb	Batch, soweit möglich
Grafische Oberfläche	Eclipse RCP zur Konfiguration von Kampagnen und Schnittstellen
Programmiersprachen und -frameworks	Java, Hibernate, MDSD-Generator zur Erzeugung von Teilen des Quellcodes
Modellierungswerkzeuge	UML-2-kompatibles Modellierungswerkzeug zur Modellierung von kampagnenspezifischen Datenstrukturen und -abläufen
Technische Kommunikation	Muss zu Mandanten beziehungsweise Partnern über ftp, sftp, http, https möglich sein, mit und ohne VPN. Anbindung an Messaging-Strukturen (wie Tibco oder MQ-Series) ist nicht vorgesehen.

2.2 Organisatorische Einflussfaktoren

IRZ ist ein agiler Auftraggeber, der innerhalb der Softwareentwicklung iterative Prozesse bevorzugt.

Allerdings legt IRZ großen Wert auf langlebige Systeme und fordert daher aussagekräftige, wartungsfreundliche technische Dokumentation (von der Sie gerade einen Auszug in den Händen halten ...).

3 Kontextabgrenzung

3.1 Allgemeiner fachlicher Kontext

In der logischen Kontextabgrenzung von Bild 12.11 erkennen Sie, dass MaMa nur mit dem Mandanten beziehungsweise mehreren Partnern kommuniziert. Für jede Kampagne werden spezifische Partner ausgewählt – ein Beispiel finden Sie in Section 3.2.

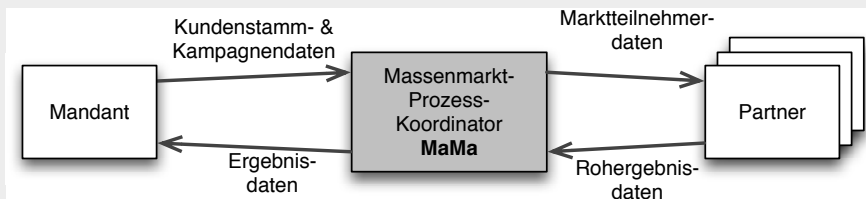


Bild 12.11 Allgemeiner fachlicher (logischer) Kontext

Kurzbeschreibung der externen Schnittstellen

Schnittstelle/Nachbarsystem	Ausgetauschte Daten (Datenformate, Medien)
Kundenstamm- und Kampagnendaten (von Mandanten)	Mandanten übermitteln Stammdaten ihrer Kunden sowie Daten/Metadaten der Kampagnen an MaMa.
Ergebnisdaten (an Mandanten)	MaMa übergibt Ergebnisse, Zwischenergebnisse oder Rückfragen (aufgrund von Fehlern oder Ausnahmen) an Mandant.
Marktteilnehmerdaten (an Partner)	MaMa übergibt Marktteilnehmerdaten an Partner.
Rohergebnisdaten (von Partnern)	Partner übergeben ihre jeweiligen Zwischenergebnisse an MaMa.

Partner sind für MaMa externe Dienstleistungserbringer, beispielsweise:

- Druckereien, zum Druck von Briefsendungen oder ähnlichen Schriftstücken;
- Briefversender oder Distributoren, um Drucksendungen zu Endkunden zu transportieren;

- Scandienstleister, zur Umwandlung von Rücksendungen der Endkunden in Datenformate, inklusive automatische Texterkennung (OCR);
- Callcenter und Telko-Dienstleister, zur telefonischen Ansprache von Endkunden, Faxversand oder Bearbeitung von Anrufen der Endkunden;
- Webhoster und E-Mail-Provider für jegliche Online-Dienste, die im Rahmen von Kampagnen erbracht werden müssen.

3.2 Spezielle Kontextabgrenzung der Mobilfunkkampagne

In Bild 12.10 haben Sie bereits die Kontextabgrenzung einer konkreten Instanz von MaMa für Mobilfunkunternehmen gesehen. Die konkrete Ausprägung für die Telco-Domäne sehen Sie in Bild 12.12.

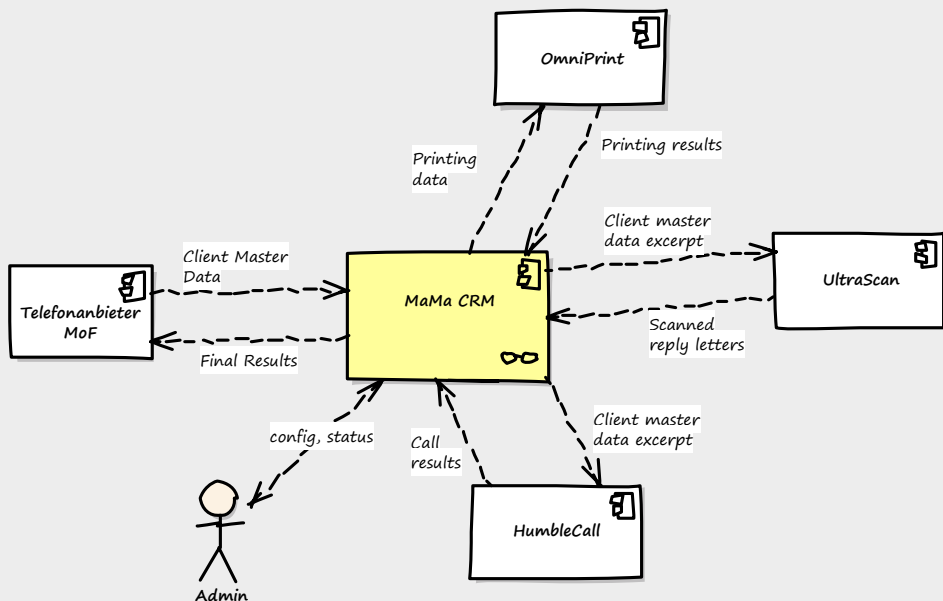


Bild 12.12 Fachlicher Kontext der Telco-Domäne

3.3 Verteilungskontext: MaMa als Basis einer Produktfamilie

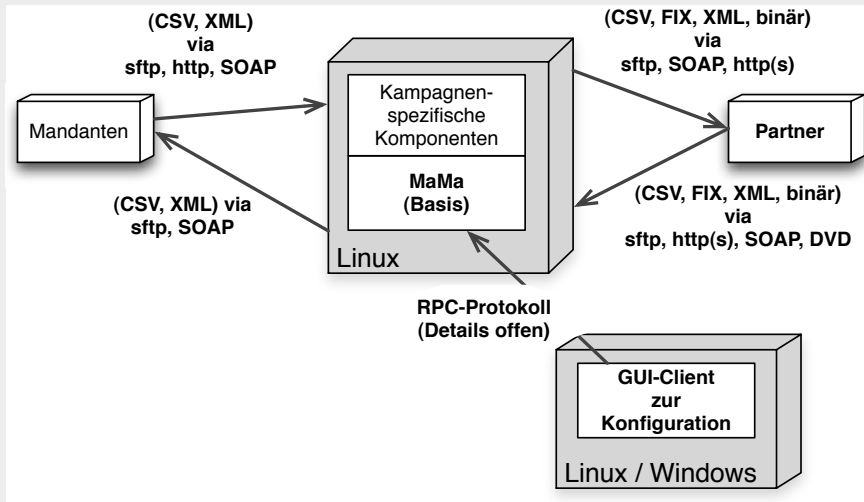


Bild 12.13 Allgemeiner Verteilungskontext von MaMa mit (schematischen) externen Schnittstellen

MaMa stellt die Basis einer Produktfamilie dar. Jede Kampagne wird auf einer (in der Regel virtuell) dedizierten Linux-Installation betrieben. In jeder Kampagne gibt es vier Kategorien externer Schnittstellen von und zu MaMa:

Kurzbeschreibung der externen Schnittstellen

Schnittstelle/Nachbarsystem	Technologie/Protokoll
Kundenstamm- und Kampagnendaten (von Mandanten)	CSV- oder XML-Formate als Dateien per sftp- oder https-Upload. Alternativ: Einlieferung per Web-Service oder DVD.
Ergebnisdaten (an Mandanten)	CSV- oder XML-Formate als Dateien per sftp. Alternativ: Ablieferung per Web-Service.
Marktteilnehmerdaten (an Partner)	Format je nach Partner und Kampagne. MaMa muss CSV, Fix-Length, XML sowie Web-Services unterstützen.
Rohergebnisdaten (von Partnern)	Format je nach Partner, s. o.

Fachliche Abläufe finden Sie in der Laufzeitsicht in [Sektion 6.1](#) dieser Dokumentation.

4 Lösungsstrategie

Die Struktur von MaMa basiert in hohem Maße auf dem technischen Konzept zur konfigurierbaren Ablaufsteuerung, das Sie in Sektion 8.1 dieser Dokumentation finden. Ebenfalls wichtig für das Verständnis ist das Konzept der Produktfamilie, erläutert in Sektion 8.2 (zusammen mit der Persistenz und Generierung). Schließlich erlaubt MaMa die Ein- und Ausgabedaten pro Kampagne und Mandant zu konfigurieren.

Diese Konzepte stellen die Qualitätsziele „Flexibilität“ und „Performance“ sicher.

5 Bausteinsicht

Voraussetzung für das Verständnis der statischen Struktur von MaMa ist das Konzept zur Ablaufsteuerung (siehe Sektion 8.1 dieser Dokumentation).

5.1 MaMa-Bausteinsicht Level 1

Sämtliche Abhängigkeiten innerhalb von Bild 12.14 sind benutzt-Beziehungen innerhalb des MaMa-Quellcodes.

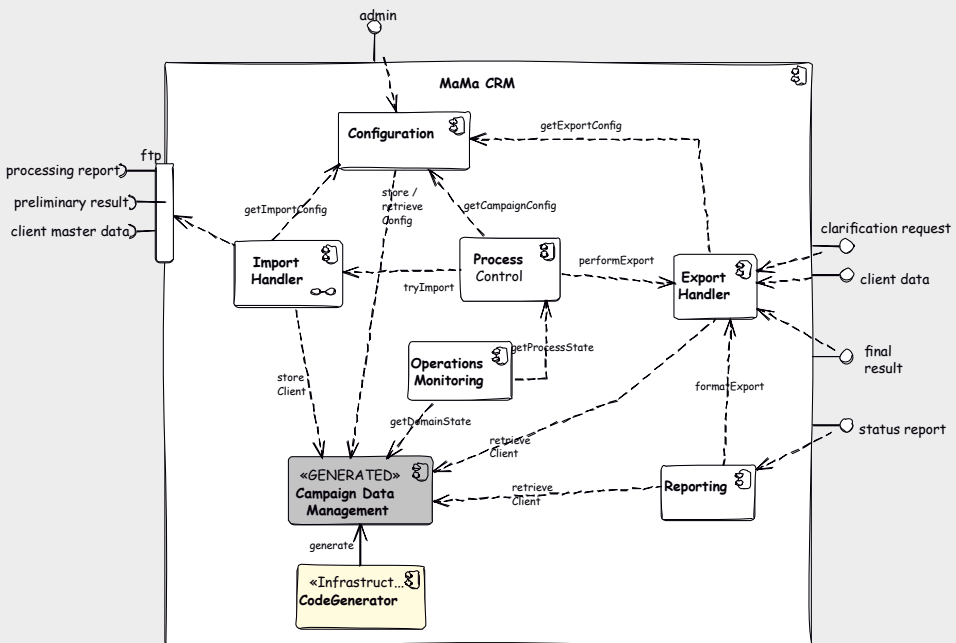


Bild 12.14 Whitebox-Darstellung von MaMa, Level 1

Strukturiert wurde MaMa auf dieser Ebene primär anhand funktionaler Aspekte: Jede der Level-1-Blackboxen verantwortet unmittelbar einen Teil der MaMa-Gesamtfunktionalität. Einzig die Bausteine CampaignDataManagement und OperationsMonitoring haben rein unterstützende Aufgaben (für die Datenspeicherung respektive die Überwachung zur Laufzeit). Das Architekturziel der hohen Flexibilität gegenüber Datenstrukturen und Ein-/Ausgangsschnittstellen wurde an die Bausteine Configuration, InputHandler und ExportHandler delegiert. Das Architekturziel der hohen Performance verantworten primär die Bausteine Input und Output, unterstützt durch CampaignDataManagement. Die folgenden Abschnitte beschreiben die in der Abbildung dargestellten Bausteine, strukturiert nach dem Blackbox-Template.

Tabelle 12.1 Level-1-Bausteine von „MaMa“

Baustein	Beschreibung
Import bzw. Export Handler	Liest/schreibt Daten über die Außenschnittstellen, d. h. an Partner und Mandanten. Hierzu gehören insbesondere Teile von IClient-Instanzen und Clarification Requests.
Campaign Data Management	Speichert sämtliche Kampagnendaten, insbesondere die Domänenentitäten (Instanzen von IClient), zusätzlich Konfigurationsdaten.
ProcessControl	Steuert die Abläufe/Prozesse der Kampagne, verantwortet Ausführung kampagnenspezifischer Geschäftsregeln.
Configuration	Hält Konfiguration aller Import- und Exportformate, Im- und Exportfilter.
Operations Monitoring	Beobachtet und berichtet den Zustand sämtlicher Import- und Exportprozesse sowie relevanter Teile der Datenbank.
Reporting	Fasst den Zustand der Kampagnen für die Mandanten bzw. die Betreiber des Systems zusammen, insbesondere die aufgelaufenen Kosten für die externen Partner (z. B. Druckerei).

5.1.1 ImportHandler

- Zweck/Verantwortlichkeit: Nimmt über Außenschnittstellen alle Arten von Daten von Mandanten und Partnern entgegen.
 - Schnittstelle(n): Eingang: Kundenstamm-, Kampagnen-, Kampagnen-Meta- und Rohergebnisdaten (CSV, XML, binär). Nutzt Methoden/Funktionen von CampaignDataManagement.
 - Quellcode: Package de.arc42.example.mama.importHandler, insbesondere die Klasse InputManager
 - Offene Punkte: Zurzeit unterstützt dieser Baustein keine PGP-verschlüsselten Eingangsdaten, kann bei Web-Service-Einlieferung keine digitalen Signaturen prüfen und besitzt keine Anbindung an MQSeries und TIBCO.
- ► Die Verfeinerung als Whitebox finden Sie in **Sektion 5.2.1**.

5.1.2 Campaign Process Control

- Zweck/Verantwortlichkeit: Steuert die Abläufe innerhalb von Kampagnen, verantwortet die Ausführung und Prüfung der kampagnenspezifischen Geschäftsregeln.
- Schnittstelle(n): Lesezugriff auf CampaignDataManagement, Notify-Mechanismus an Output.
- Quellcode: Package: `de.arc42.example.mama.campaigncontrol`

► ► Die Verfeinerung als Whitebox finden Sie in **Sektion 5.2.2.**

5.1.3 CampaignDataManagement

Dieser Baustein wird vollständig aus dem spezialisierten Domänenmodell generiert. Details siehe Sektion 8.2.

- Zweck/Verantwortlichkeit: Verwaltet sämtliche Kampagnendaten und Metadaten. Stellt Verwaltungsoperationen zum Lesen, Schreiben und Löschen all dieser Daten bereit.
- Schnittstelle(n): Erhält Daten von Input, Configuration sowie CampaignProcessControl. Liefert Daten für praktisch alle anderen Bausteine.
- Quellcode: Package: `de.arc42.example.mama.campaigndata`
- Offene Punkte: Bisher nur wenige Möglichkeiten zum Performance-Tuning der Datenbank.

5.1.4 Configuration

- Zweck/Verantwortlichkeit: Grafische Oberfläche zur Konfiguration von Mandanten, Kampagnen, Schnittstellen und Geschäftsregeln.
- Schnittstelle(n): Speichert Konfigurationsdaten in CampaignDataManagement.
- Quellcode: Package: `de.arc42.example.mama.configuration`
- Offene Punkte: Teile der Konfiguration einer Kampagne erfolgen zur Zeit über ein UML-Modell. Die vom Auftraggeber gewünschte „zentrale Stelle zur Konfiguration sämtlicher Bestandteile einer Kampagne“ ist somit noch nicht gegeben.

5.1.5 Export Handler

- Zweck/Verantwortlichkeit: Liefert über die Außenschnittstellen Ergebnisdaten (an Mandanten) sowie Marktteilnehmerdaten (an Partner).
- Schnittstelle(n): Ergebnisdaten (CSV, XML) sowie Marktteilnehmer-, Kampagnendaten (CSV, FIX, XML, PDF). Nutzt Methoden/Funktionen von CampaignDataManagement.
- Quellcode: Package `de.arc42.example.mama.output`, insbesondere die Klasse `OutputManager`
- Offene Punkte: Zurzeit kann dieser Baustein keine Ausgangsdaten mit PGP verschlüsseln, keine digitalen Signaturen erzeugen und besitzt keine Anbindung an MQSeries und TIBCO.

5.1.6 Reporting sowie Operations Monitoring

<Werden im Beispiel nicht weiter betrachtet.>

5.2 MaMa-Bausteinsicht Level 2

Dieser Abschnitt detailliert die Blackbox-Bausteine der Ebene 1. Aufgrund des Beispielcharakters dieser Dokumentation zeige ich hier nur Auszüge der gesamten Dokumentation.

5.2.1 Whitebox-Sicht Baustein „Import Handler“, Level 2

Die Struktur des ImportHandler wurde aufgrund funktionaler Aspekte gewählt: Die enthaltenen Bausteine leisten jeweils einen Schritt in der grundsätzlichen Verarbeitungskette von Eingabedaten (Empfangen, Archivieren, Filtern, Validieren und In-Java-Objekte-Übersetzen). Die Ablaufsteuerung zwischen diesen Bausteinen obliegt dem Receiver – eine deutliche Schwäche im Entwurf dieser Whitebox. Details dazu in Sektion 5.3.1.

Dieser Baustein verantwortet das Architekturziel der „flexiblen Eingangsformate und Übertragungsprotokolle“ für jede Kampagne.

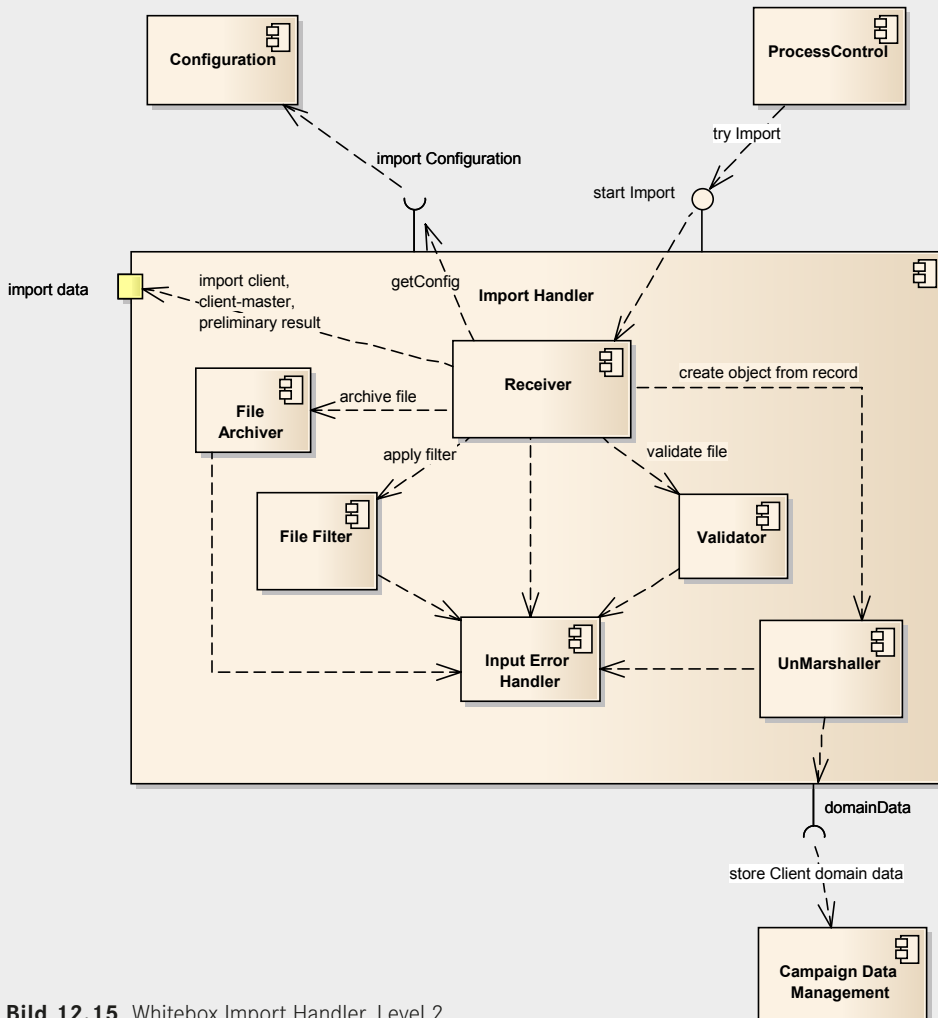


Bild 12.15 Whitebox Import Handler, Level 2

Baustein	Beschreibung
Receiver	Führt den eigentlichen Empfang der Daten durch und koordiniert die übrigen Aufgaben beim Import.
Validator	Prüft eine Datei auf Einhaltung ihrer Formatkonvention (beispielsweise: korrekte Anzahl Felder in CSV-Datei),
File Filter	Führt dateiweite Konvertierungen durch, bislang Entschlüsselung und Dekomprimierung.
File Archiver	Speichert empfangene Dateien und Daten in einem nicht löschbaren Archiv, sofern für die Kampagne notwendig.
Input Error Handler	Berichtet sämtliche beim Import und den nachgelagerten Konvertierungs- und Prüfaufgaben aufgetretene Fehler.
UnMarshaller	Der Magier: wandelt Zeichenketten in gültige IClient-Instanzen um, d. h. in Domain-Entities.

5.2.1.1 Blackbox „Receiver“

- Zweck/Verantwortlichkeit: Führt den physischen Daten- oder Dateieingang durch (wird implementiert durch beispielsweise FileReceiver, FTPReceiver, SOAPReceiver und andere). Koordiniert die übrigen Aufgaben der Input-Verarbeitung.
- Schnittstelle(n): Nimmt über die Außenschnittstellen Kundenstamm- & Kampagnendaten von Mandanten entgegen sowie Rohergebnisdaten von den Partnern. Interne Schnittstellen zu allen anderen Bausteinen von Input.
- Quellcode: Package de.arc42.example.mama.receiver, insbesondere das Interface IReceiver
- Offene Punkte: Zurzeit keine Schnittstellen zu MQ-Series oder TIBCO.

Receiver übernimmt zu viele Aufgaben – neben dem Datenempfang auch noch die Koordination anderer Bausteine. Diese schlechte funktionale Kohäsion führt zu schwer verständlichem Quellcode und zu hohen Wartungsrisiken. Die Koordinationsaufgaben sollten daher in einen eigenen Baustein verlagert werden; siehe Sektion 11 (Risiken).

► ► Die Verfeinerung als Whitebox finden Sie in Sektion 5.3.1.

5.2.1.2 Blackbox „Validator“

- Zweck/Verantwortlichkeit: Überprüft, ob die einzelnen Bestandteile (Datei, Datensätze, Satzteile, Records o. Ä.) der Datei den vereinbarten Kriterien für diese Kampagne/ Partner/Mandanten genügen.
- Schnittstelle(n): Erhält von Receiver die zu validierenden Daten in Blöcken (Dateien oder einzelne Dateieinträge), liest die Validierungsregeln aus CampaignDataManagement.
- Quellcode: Package de.arc42.example.mama.validator
- Offene Punkte: Zurzeit werden Validierungsfehler nur in Logfiles abgelegt und nicht korrekt an InputErrorHandler gemeldet.

Dieser Baustein basiert auf dem [Validator] Framework in Kombination mit einer MaMa-spezifischen Beschreibungssprache. Validiert sowohl Dateien wie einzelne Datensätze – was aus Vereinfachungsgründen künftig auf zwei getrennte Bausteine verteilt werden könnte. Aus heutiger Sicht nicht riskant.

Anmerkung: Diese Information geht über eine reine Blackbox-Beschreibung etwas hinaus. Den Aufwand einer zusätzlichen Whitebox-Beschreibung halte ich jedoch für unangemessen ...

5.2.1.3 Blackbox „FileFilter“

Anmerkung: Hier mal eine alternative Darstellung des Blackbox-Templates, als Tabelle:

Zweck/Verantwortung	Führt sämtliche notwendigen dateiweiten Konvertierungen durch, beispielsweise Dekompression, Entschlüsselung, Auswertung digitaler Signaturen oder Ähnliche.
Schnittstellen	Erhält von <code>Receiver</code> die zu bearbeitenden Dateien und eine Liste der auszuführenden Operationen.
Ablageort	<code>Package de.arc42.example.mama.filefilter</code>
Offene Punkte	Zurzeit kann <code>FileFilter</code> keine PGP-verschlüsselten Dateien lesen, keine digitalen Signaturen auswerten und keine verschlüsselten ZIP-Dateien entschlüsseln.

5.2.1.4 Blackbox „FileArchiver“

- Zweck/Verantwortlichkeit: speichert die vom `Receiver` empfangenen Dateien in einem von MaMa nicht löschbaren Archiv, sofern für die Kampagne notwendig.
 - Schnittstelle(n): enthält vom `Receiver` die zu archivierenden Dateien.
 - Quellcode: `Package de.arc42.example.mama.archiver`, insbesondere die Klasse `FileArchiver`
- Der `FileArchiver` ist eine Fassade zum frei verfügbaren [HoneyComb], einer von Sun Microsystems entwickelten Speicherlösung für unveränderliche Daten.

5.2.1.5 Blackbox „InputErrorHandler“

- Zweck/Verantwortlichkeit: Sammelt sämtliche während des Input-Vorgangs auftretenden Fehler und -meldungen zur späteren Bearbeitung.
- Schnittstelle(n): Erhält Fehlermeldungen von allen Input-Bausteinen, mit Ausnahme von `Validator`.
- Quellcode: `Package de.arc42.example.mama.inputerror`
- Offene Punkte: Zur Zeit keine Integration mit `Validator`.

Die fehlende Integration von `InputErrorHandler` mit dem `Validator` ist eine signifikante Schwäche in Entwurf und Implementierung von MaMa. Siehe Sektion 11 (Risiken).

5.2.1.6 Blackbox „UnMarshaller“

- Zweck/Verantwortlichkeit: Erzeugt aus den Dateibestandteilen (Records o. Ä.) wieder Objekte des Domain-Models und speichert diese ggfs. im `CampaignDataManagement`.
- Schnittstelle(n): Erhält vom `Receiver` Listen von Strings, aus denen `UnMarshaller` echte MaMa-Objekte erstellt.
- Quellcode: `de.arc42.example.mama.unmarshaller`

5.2.2 Whitebox Campaign Process Control, Level 2

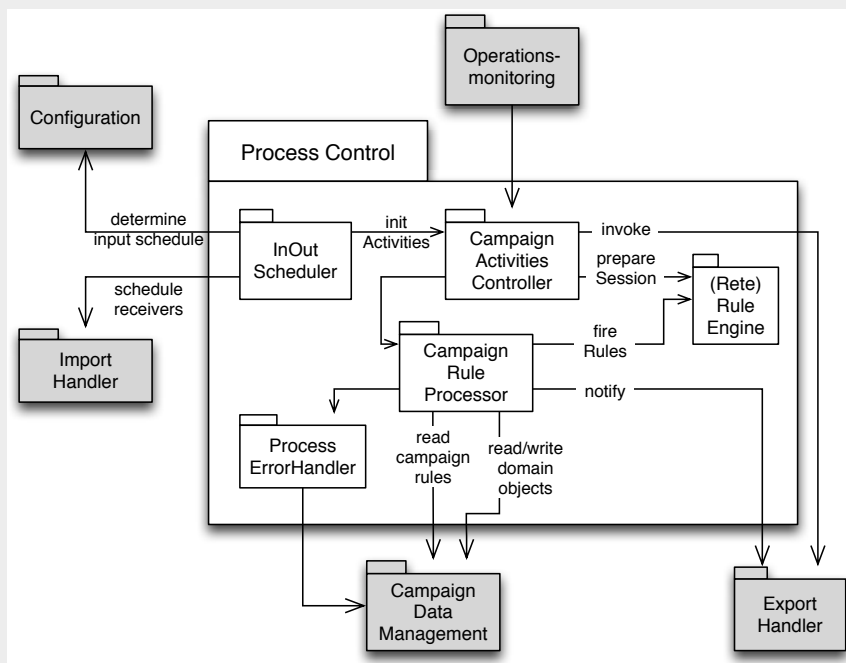


Bild 12.16 CampaignProcessControl, Level 2

Tabelle 12.2 Blackbox-Bausteine von CampaignProcessControl, Level 2

Blackbox	Bedeutung
InOutScheduler	Steuert den Aufruf der für eine Kampagne jeweils benötigten Input- und Output-Aktivitäten (abhängig von Mandanten, Partnern sowie den jeweils vorhandenen Daten).
CampaignActivitiesController	Steuert die Aktivitäten einer Kampagne (genauer: Output- und interne Aktivitäten). Implementiert dazu den in Sektion 8.1 beschriebenen Algorithmus.
CampaignRuleProcessor	Bestimmt aus den für eine Kampagne (bzw. einzelne Marktteilnehmer) vorhandenen Daten die nächsten (Output-)Aktivitäten. Basiert auf kampagnenspezifischer Prozess- und Regeldefinition – nutzt eine Regelmaschine; siehe Sektion 8.3.
ProcessErrorHandler	Behandelt sämtliche Fehler von Kampagnendaten (Stammdaten, Marktteilnehmerdaten, Rohergebnisdaten) und löst eine Output-Aktion mit den jeweils betroffenen Partnern oder Mandanten aus („Clearing“).
(Rete)RuleEngine	Open-Source-Regelmaschine zur Auswertung der kampagnenspezifischen Ablauf- und Plausibilitätsregeln; siehe [Drools].

Für weitere Details siehe Sektion 8.3 (Konzept zu Geschäftsregeln).

5.3 MaMa Bausteinsicht Level 3

Dieser Abschnitt detailliert exemplarisch einen Blackbox-Baustein der Ebene 3.

5.3.1 Whitebox-Sicht Baustein „Receiver“, Level 3

Der Baustein „Receiver“ nimmt die Daten entgegen, die an den Außenschnittstellen von MaMa physisch angeliefert werden. Gleichzeitig koordiniert Receiver die übrigen Aufgaben der Input-Verarbeitung. Diese Vermischung von Verantwortlichkeiten ist riskant und führt in der Entwicklung zu hohen Änderungsaufwänden bei der Einführung neuer Daten- oder Dateiformate. Siehe die Anmerkungen in Sektion 5.2.1.1 (Blackbox Receiver) sowie Sektion 11 (Risiken).

Anmerkung: Die nachfolgende Whiteboxsicht beschreibt die Zielvorstellung, die nach dem Refactoring von Receiver erreicht werden soll.

Funktional werden in Bild 12.17 die Aufgaben von „Datenempfang“ (div. Listener sowie ServiceToFileConverter) getrennt von der Koordination der weiteren Verarbeitung (FileProcessor, FileSplitter, RecordProcessor). Die letztgenannten Bausteine könnten künftig in einen eigenständigen Level-2-Baustein (Vorschlag: InputController) ausgelagert werden.

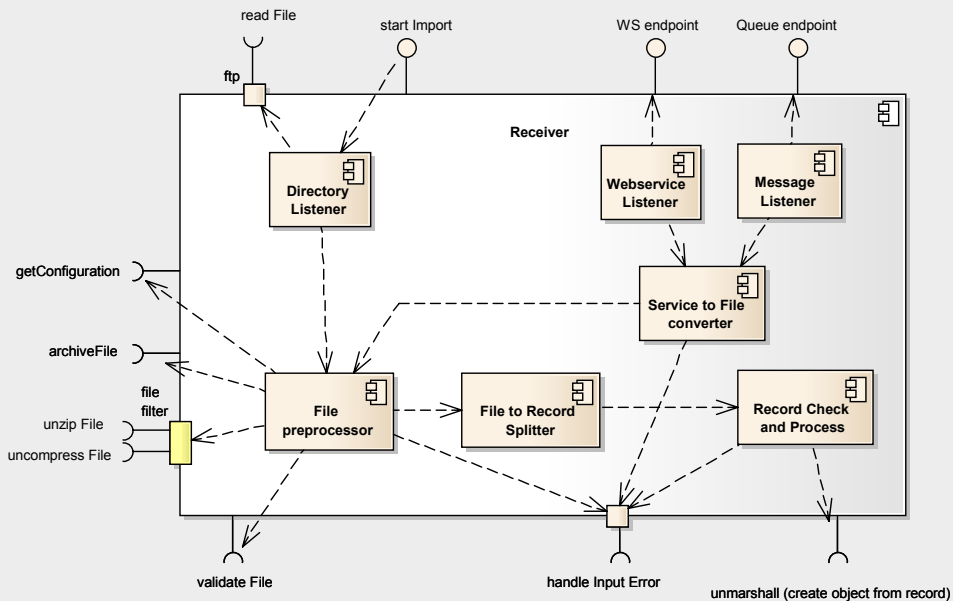


Bild 12.17 Whitebox Receiver, Level 3 (Zielstruktur, zurzeit nicht realisiert)

Tabelle 12.3 Blackbox-Bausteine von Receiver, Level 3

Blackbox	Bedeutung
DirectoryListener	Überprüft periodisch ein vorgegebenes Verzeichnis im Dateibaum auf die Existenz neuer Dateien eines kampagnen- oder aktivitätsspezifischen Namensmusters. Wird eine Datei gefunden, so übergibt der Listener diese an den FileProcessor. Für jede Input-Aktivität der Kampagne wird eine eigene Instanz dieses Bausteins als Thread (respektive Prozess) gestartet.
WebServiceListener	Empfängt Daten über Web-Services. Die zugehörige WSDL-Beschreibung inklusive URL und Service-Namen wird in der Konfiguration der Kampagne festgelegt. Im Falle des fehlerfreien Empfangs einer Nachricht wird diese an ServiceToFileConverter übergeben.
MessageListener	Empfängt Nachrichten von TIBCO beziehungsweise MQ-Series und übergibt diese an ServiceToFileConverter. Zurzeit nicht implementiert.
ServiceToFileConverter	Konvertiert Eingabedatenströme (Messages, Web-Service-Nachrichten) in Dateien, zur Weiterverarbeitung durch FileProcessor.
File PreProcessor	Koordiniert die Verarbeitung von Eingabedateien. Nach Bearbeitung Weitergabe an File-toRecord-Splitter.
File to Record Splitter	Zerlegt eine Datei mit Hilfe eines Parsers in einzelne Datensätze: Bei CSV-Dateien sind das einzelne Zeilen, bei XML-Dateien ist die Zerlegung abhängig vom XML-Schema. Fix-Format-Dateien werden auf Basis ihrer Record-Definition zerlegt. Übergibt eine Liste von Strings an Record-Check-and-Process.
Record Check and Process	Verarbeitet einzelne Datensätze.

6 Laufzeitsicht

Anmerkung: Als Alternative zu Sequenzdiagrammen beschreibe ich einige der folgenden Szenarien in Textform. Sie sollten im Einzelfall entscheiden, ob Ihre Leser besser mit einem Diagramm oder mit Text arbeiten können. In manchen Fällen können Sie bei Laufzeitszenarien übrigens auch Quellcodefragmente in die Architekturdokumentation übernehmen!

6.1 Szenario: Schematischer Input von Daten

Dieses Szenario beschreibt schematisch, wie MaMa die über die Außenschnittstellen (Kundenstamm-, Kampagnen- oder Rohergebnisdaten) (als Datei) gelieferten Daten entgegennimmt. Dieses Szenario gilt nicht für die Datenlieferung mittels Web-Services! Eine Verfeinerung für den Import einer CSV-Datei finden Sie in Sektion 6.2.

1. Mandant oder Partner liefert Datei (per sftp-Upload) in vorab definiertes Verzeichnis.
2. Ein `DirectoryListener` (aus dem Baustein `Receiver`) erkennt die Einlieferung und startet den Baustein `FileProcessor` (der startet `FileFilter`, `Validator` und `FileArchiver`).
3. `FileProcessor` importiert die gelieferten Daten und ruft für jeden (syntaktisch korrekten) Datensatz `CampaignRuleProcessor` auf. Für fehlerhafte Datensätze wird eine Fehlermeldung erzeugt und ein Clearing beim Mandant oder Partner beauftragt.
4. `CampaignRuleProcessor` ermittelt für den aktuellen Datensatz die notwendigen Folgeaktivitäten.

6.2 Szenario: Import einer CSV-Datei

Voraussetzungen:

- Die betroffene Kampagne ist funktionsfähig konfiguriert.
- Mandant und Partner verfügen über entsprechende Rechte, um Dateien bei MaMa einliefern zu können. Anmerkung: Datenlieferung per Web-Service ist nicht Bestandteil dieses Szenarios.

Ablauf:

1. Mandant oder Partner liefert Datei (per sftp-Upload) in vorab definiertes Verzeichnis.
2. MaMa liest aus `CampaignDataManagement` die Metadaten für diesen Mandanten (u. a. Filter- und Prüfregele für Dateien, Syntaxregeln für Datensätze).
3. `Receiver` übergibt Datei mit Statusinfo (Datum, Datei-Hash) an `FileArchiver` – der sie in das Permanent-Archiv kopiert.
4. `FileProcessor` prüft Metadaten der Datei; nicht ok: Clearingfall.
5. `FileFilter` wendet alle konfigurierten Filter (decompress, decrypt etc.) an; nicht ok: Clearingfall.
6. `FileSplitter` zerlegt in Einzelsätze.
7. `RecordProcessor` verarbeitet Datei satzweise und koordiniert `CampaignRuleProcessor` und `UnMarshaller`:
 - 7.1 `UnMarshaller` lässt übergebene Daten durch `Validator` syntaktisch validieren; nicht ok: Clearingfall.
 - 7.2 Falls Daten ok, sucht anhand des Primärschlüssels im Repository nach bereits vorhandener Instanz dieses Datensatzes.

- 7.2.a: Nicht vorhanden: legt neues Objekt (persistent) an.
- 7.2.b: Vorhanden: Update mit gerade gelieferten Daten (offen: Historisierung).
- 7.3 CampaignRuleProcessor wendet Geschäftsregeln der Kampagne auf aktuellen Datensatz an; primär werden dabei die notwendigen Folgeaktivitäten ermittelt (siehe Sektion 8.1 – Ablaufsteuerung).

8. Erzeuge Statusmeldung für Mandanten und übergebe diese via „Output“.

Ergebnisse dieses Szenarios:

- Die in der Datei gelieferten korrekten Datensätze sind in den MaMa-Datenbestand der betroffenen Kampagne übernommen.
- Sämtliche fehlerhaften Daten werden als Clearingfälle zurück an den Lieferanten der Datei gemeldet.

6.3 Szenario: Import einer CSV-Datei (alternative Darstellung)

Zum direkten Vergleich finden Sie nachfolgend das oben geschilderte Szenario als Sequenzdiagramm.

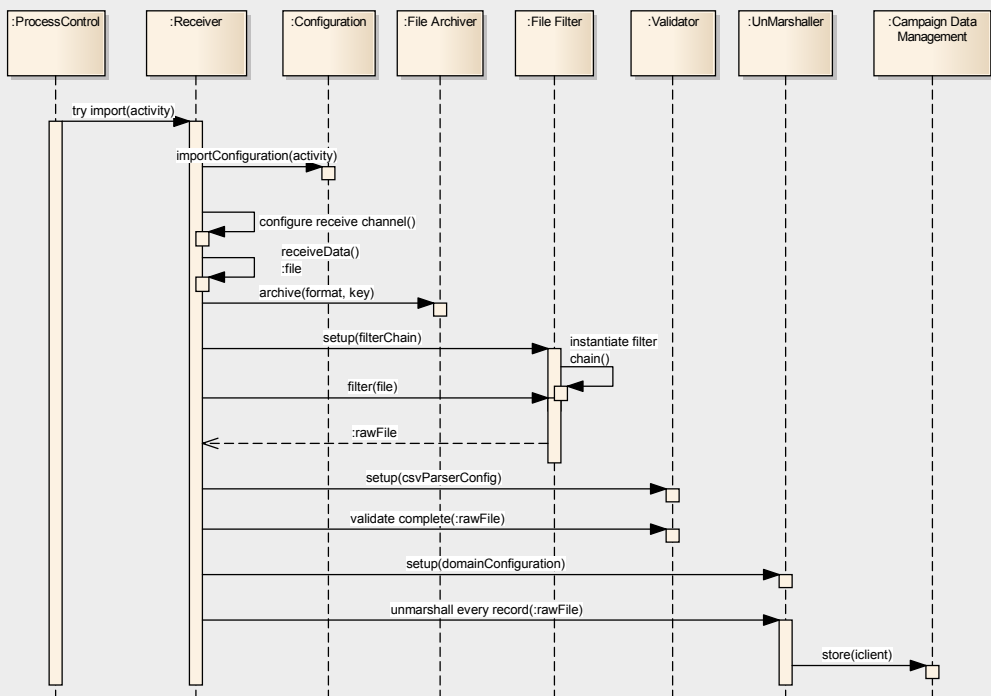


Bild 12.18 Laufzeitszenario: Import einer CSV-Datei mit Filterung

7 Verteilungssicht

Die Verteilungssicht von MaMa ist nahezu trivial, da die meisten Implementierungsbausteine der Bausteinsicht in ein einzelnes Java-Klassenarchiv (jar) zusammengefasst werden.

Die graphischen Teile von MaMa (Configuration, OperationsMonitoring) bilden dabei Ausnahmen.

Tabelle 12.4 Verteilungsartefakte für MaMa

Verteilungsartefakt	Baustein (aus Bausteinsicht)
MaMa<CampaignName>	Alle, bis auf OperationsMonitoring
MaMaOperationsMonitor	OperationsMonitoring, CampaignDataManagement
MaMaConfigurator	Configuration, CampaignDataManagement, CampaignProcessControl

Anmerkung: Auf die technischen Leistungsdaten der zugrunde liegenden Hardware habe ich in diesem Beispiel verzichtet.

8 Querschnittliche Konzepte

8.1 Ablaufsteuerung

Das für MaMa zentrale Konzept der konfigurierbaren Ablaufsteuerung basiert wesentlich auf dem Begriff der Aktivität, jeweils bezogen auf einzelne Marktteilnehmer. MaMa kennt Output-, Input- sowie interne Aktivitäten:

- Output-Aktivitäten übergeben Teile der Daten über Marktteilnehmer (Beispiel: Name, Adresse, Vertrags- oder Tarifdaten) an einen Partner, damit dieser Partner eine für die Kampagne notwendige Aktion durchführen kann. Beispiele für solche Aktionen:
 - Einen Brief an die jeweiligen Marktteilnehmer schreiben (Partner: Druckerei).
 - Die Marktteilnehmer per Telefon kontaktieren (Partner: Callcenter).
 - Auf den Besuch oder Anruf der Marktteilnehmer vorbereitet sein (Partner: Callcenter sowie Filialen).

Output-Aktivitäten haben immer einen definierten Adressaten. Als Konsequenz mancher Output-Aktivitäten liefern die betreffenden Partner zu einem späteren Zeitpunkt Roh-ergebnisdaten an MaMa zurück, die über Input-Aktivitäten eingelesen werden.

- Interne Aktivitäten verändern den Zustand oder die Daten von Marktteilnehmern, ohne dabei einen externen Partner zu benutzen. Hierzu gehören Datentransformationen, Berechnungen, Aggregation mehrerer Datensätze (beispielsweise um Eltern und Kinder zu Familien zusammenzuführen), Löschungen und Statusänderungen.
- Input-Aktivitäten nehmen Daten von den Außenschnittstellen entgegen.

Schematisch verlaufen solche Aktivitäten immer nach folgendem Muster:

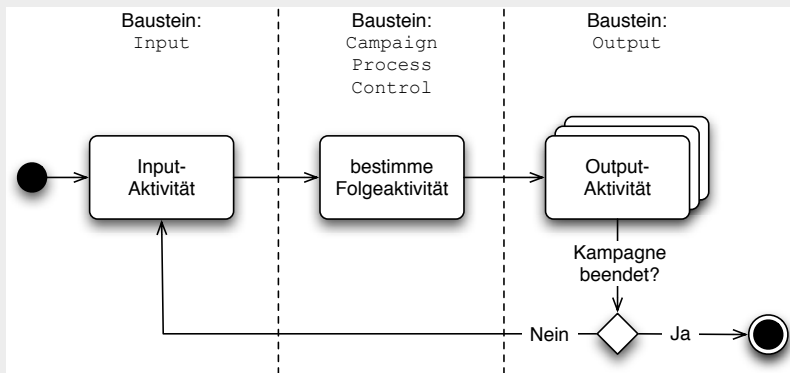


Bild 12.19 Ablaufsteuerung von MaMa – Abfolge von Aktivitäten (schematisch)

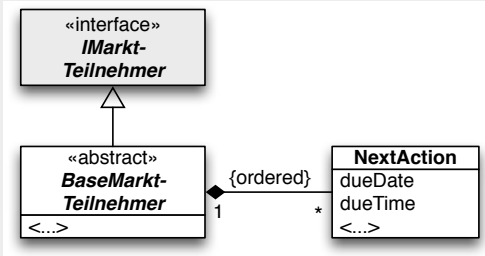
Während jeder Input-Aktivität bestimmt MaMa die notwendigen Folgeaktivitäten auf Basis der Geschäftsregeln der Kampagne sowie der Daten jedes Marktteilnehmers.

Lassen Sie mich diesen Ablauf anhand des bereits bekannten Beispiels „Vertrags-/Tarifänderungen eines Telekommunikationsunternehmens“ kurz erläutern:

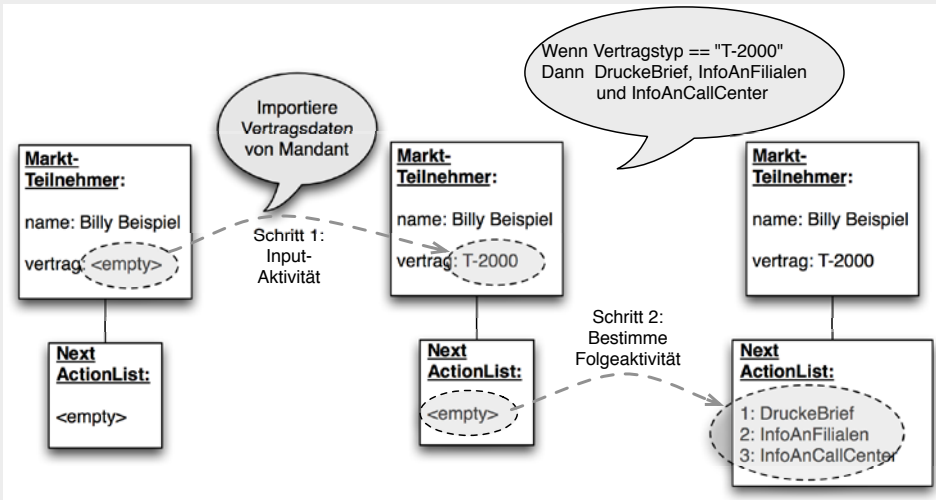
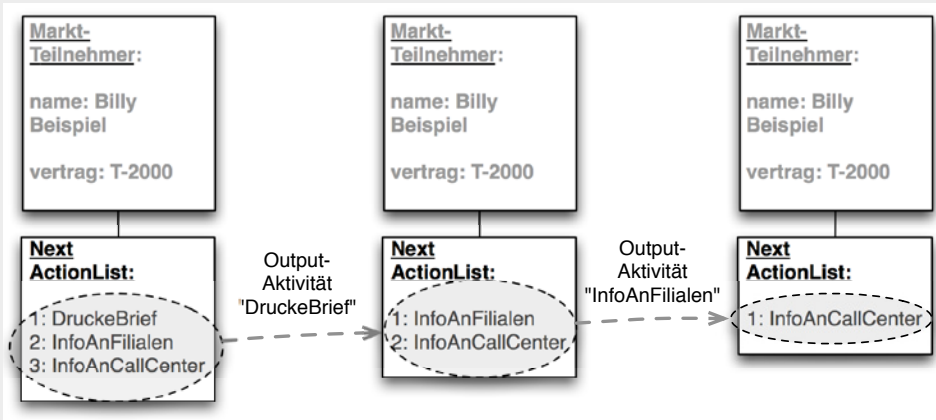
- In einer Input-Aktivität nimmt MaMa vom Mandanten MoF die Vertragsdaten eines (hypothetischen) Kunden „Billy Beispiel“ entgegen (Tarif „T-2000“).
- Als Folgeaktivitäten bestimmt MaMa die Aktivitäten „DruckeBrief“, „InfoAnFilialen“ sowie „InfoAnCallCenter“.
 - „DruckeBrief“ bedeutet, dass für diesen Kunden ein Schreiben zu drucken und zu versenden ist, mit dem er auf einen möglichen Tarifwechsel in den „Tarif Minipreis“ hingewiesen wird. Diesen Tarifwechsel muss der Marktteilnehmer durch seine Unterschrift bestätigen, die Tarifdetails auf einem mitgeschickten Formblatt auswählen.
 - „InfoAnFilialen“ bedeutet, dass eine Kopie des erstellten Briefs an die Filialen von MoF übermittelt wird.
 - „InfoAnCallCenter“ bedeutet, dass dem Callcenter eine Kopie des erstellten Anschreibens übermittelt wird. Falls sich der Marktteilnehmer bei Rückfragen nun telefonisch an das Callcenter wendet, sind dort seine Vertrags- und Tarifdaten bekannt.
- Nun führt MaMa die Output-Aktivität „DruckeBrief“ aus und übermittelt die betroffenen Daten an den Druckdienstleister.

Die technische Lösung von MaMa basiert auf einer Liste von Folgeaktivitäten, die jedem Marktteilnehmer zugeordnet ist – den entsprechenden Auszug aus dem fachlichen Modell entnehmen Sie Bild 12.20.

Im Objektdiagramm aus Bild 12.21 erkennen Sie, wie ein Marktteilnehmer über eine Input-Aktivität seinen „Vertrag“ erhält, aus dem MaMa dann anhand der kampagnenspezifischen Geschäftsregeln die Folgeaktivitäten DruckeBrief, InfoAnFilialen und InfoAnCallCenter ermittelt.

**Bild 12.20**

Fachliches Modell von Marktteilnehmer und Folgeaktivitäten

**Bild 12.21** Beispiel für Ablaufsteuerung von Aktionen**Bild 12.22** Beispiel: Update der NextActionList nach Output-Aktivitäten

MaMa wird nun die für diese Kampagne möglichen Aktionen durchführen und dabei nach jedem Schritt die Liste der möglichen Folgeaktivitäten von „Billy Beispiel“ anpassen: Allgemein folgt MaMa einem einfachen Schema bei der Bearbeitung von (Output- und internen) Aktivitäten:¹⁰

Ermittle, welche Output-Aktivitäten in dieser Kampagne möglich sind.

Für all diese Aktivitäten:

- 2.1 Ermittle diejenigen Marktteilnehmer, deren NextActionList die aktuelle Aktivität enthält.
- 2.2 Führe die Aktivität für diesen Marktteilnehmer aus. Bei Output-Aktivitäten werden dabei Daten (Attribute) dieses Marktteilnehmers zur Ausgabe an einer Außenschnittstelle gesammelt. Bei internen Aktivitäten werden Aggregationen, Berechnungen, Transformationen oder Löschungen von Daten durchgeführt.
- 2.3 Lösche die aktuelle Aktivität aus der NextActionList dieses Marktteilnehmers.
- 2.4 Wende die Geschäfts-/Plausibilitätsregeln der Kampagne auf den aktuellen Marktteilnehmer an (siehe hierzu das Konzept zu Geschäftsregeln in Sektion 9.3).

Dieser Algorithmus wird von CampaignActivitiesController im Paket CampaignProcessControl implementiert. Er wird zeitgesteuert oder periodisch ausgelöst vom InOutScheduler. Siehe dazu Sektion 5.2.2.

8.2 Produktfamilie, Persistenz und Generierung

MaMa bildet die Basis für eine Produktfamilie, von der für konkrete Kampagnen jeweils spezifische Ausprägungen instanziiert werden. Dies geschieht durch folgende Schritte:

1. Erweiterung des MaMa-Domänenmodells (einen Auszug davon finden Sie in Bild 12.23).
2. Generierung des kampagnenspezifischen Quellcodes, bestehend aus den spezialisierten Domänenklassen, Persistenzklassen (Zugriffs-, Transferklassen, DB-Tabellen u. Ä.). In diesem Schritt werden insbesondere der komplette Level-1-Baustein Campaign Data Management (siehe Bild 12.14) sowie die notwendigen DB-Skripte generiert!

¹⁰ Falls Sie das lieber in Pseudocode lesen:

```
// ermittle, welche Aktivitäten in dieser Kampagne möglich sind
Iterator aktionen = getListOfOutputActivities( kampagne );
// führe alle Aktivitäten aus
while (aktionen.hasNext()) {
    currentAction = aktionen.next()
    // selektiere diejenigen Marktteilnehmer, deren
    // NextActionList den Eintrag currentAction enthält
    teilnehmer = getActionableTeilnehmer( currentAction )
    // führe currentAction aus
    performAction( currentAction, teilnehmer)
    applyRules( kampagne, teilnehmer )
}
```

3. Kampagnenspezifische Konfiguration:

- 3.1 Festlegung der möglichen Input- und Output-Aktionen (d. h. Füllen der CampaignActionTypes)
- 3.2 Definition der konkreten Syntax aller ein- und ausgehenden Daten (CSV-Formate, XML-Schemata etc.)
- 3.3 Definition der Ein-/Ausgabeprotokolle, inklusive Metadaten, für Auftraggeber sowie sämtliche Partner
- 3.4 Definition der kampagnenspezifischen Geschäftsregeln

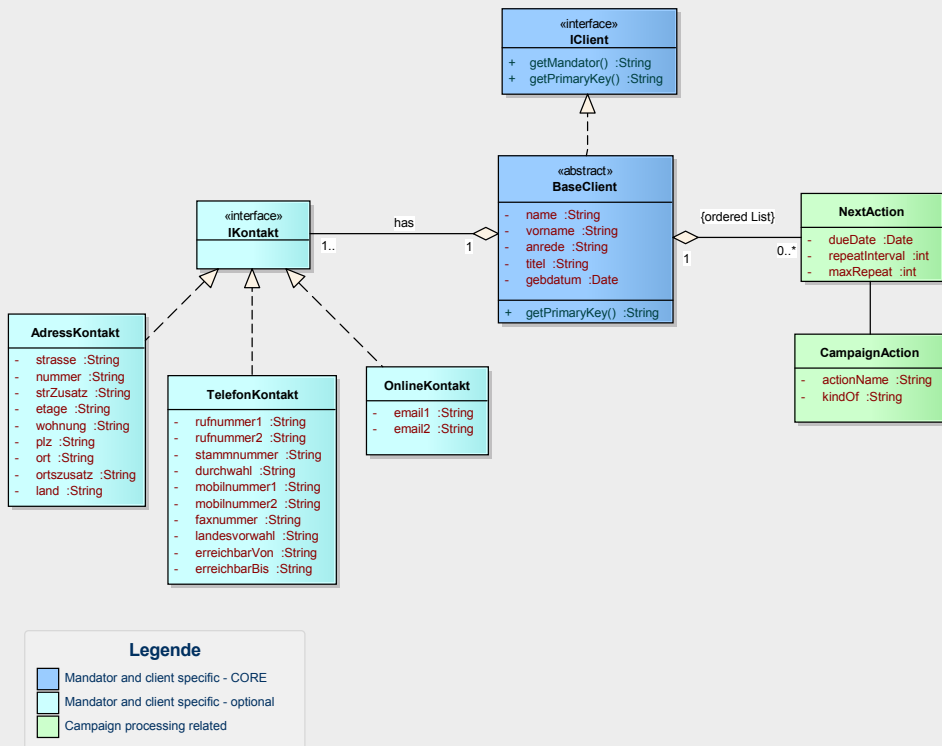


Bild 12.23 Domänenmodell als Basis der MaMa-Produktfamilie (MaMa-Core)

Danach kann die Kampagne gestartet werden.

Die Konfiguration aus Schritt 3 geschieht zum Teil über eine Konfigurationsoberfläche (siehe Baustein „Configuration“ in Sektion 5.1.4).

Beispiel eines angepassten MaMa-Domänenmodells (hier am Beispiel einer hypothetischen Versicherung):

Beachten Sie in Bild 12.24 insbesondere die neu hinzugekommenen Entitäten MorbidInsurance, die von der Core-Entität BaseClient ableitet und die kampagnen- bzw. mandantenspezifischen Attribute hinzufügt.

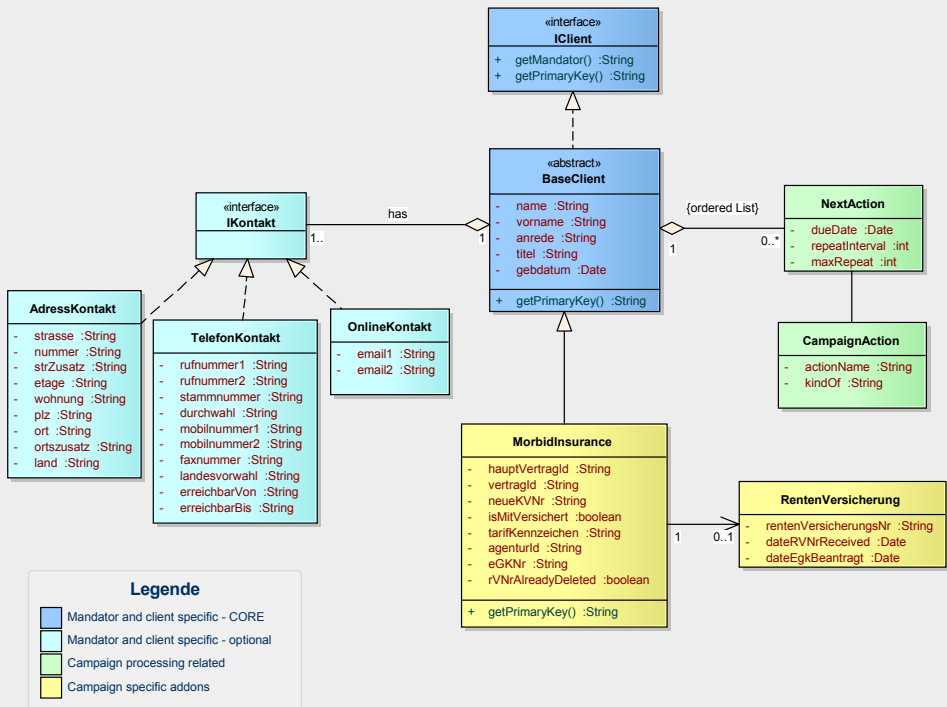


Bild 12.24 Konkretes MaMa-Domänenmodell, abgeleitet aus MaMa-Core

8.3 Geschäftsregeln

Der Auftraggeber hatte ursprünglich eine grafische Modellierung von Geschäftsregeln gefordert, was zurzeit jedoch nur einige kommerzielle Regelmaschinen in praxistauglicher Form leisten. Da seitens des Auftraggebers eine Open-Source-Lösung favorisiert wurde ([Drools]), beschreibt MaMa die Regeln einer Kampagne in textueller Syntax.

Regeln haben die Form „Wenn <X> dann <Y>“, wobei X und Y beliebige Ausdrücke in Java sein können. Mehr zur Regelsprache unter [Drools] bzw. im Java-Paket `de.arc42.example.mama.campaignruleprocessor`.

Die Bausteine `CampaignRuleProcessor` sowie `CampaignActivitiesController` kapseln sämtliche Details der Regelmaschine gemäß dem folgenden Konzept eines Rule Engine Wrappers): Ziel dieses Konstruktes ist einerseits hohe Performance durch Wiederverwendung vorbereiteter „Working Memory Sessions“ (bei denen dann pro bearbeitetem Datensatz nur noch die jeweiligen Fakten über die Methode `addFact()` eingefügt werden müssen und sämtliche Regeln aus der Datei `CampaignRules.drl` bereits geladen sind), andererseits die potenzielle Austauschbarkeit der Regelmaschine (was durch die Wahl der Drools-Regelsprache sehr unwahrscheinlich ist).

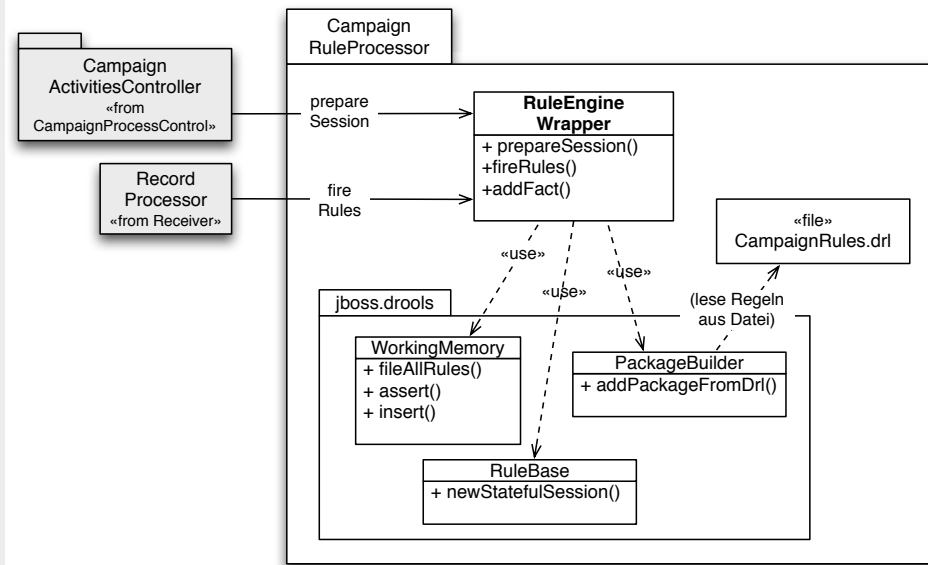


Bild 12.25 Rule Engine Wrapper zur Kapselung der Regelmaschine

8.4 Ausnahme- und Fehlerbehandlung

Für MaMa relevante Ausnahmen und Fehler betreffen hauptsächlich folgende Kategorien von Fehlern, die alle an Eingabeschnittstellen auftreten:

- Fehlerhaft übertragene Daten (z. B. falsche Prüfsummen)
- Syntaktisch falsche Daten (z. B. fehlende Attribute oder falsche Datentypen)
- Nicht zuzuordnende Daten (fehlerhafte Primärschlüssel)
- Verletzungen kampagnenspezifischer Konsistenzbedingungen

MaMa selbst kann diese Fehler nicht korrigieren, sondern lediglich dem jeweiligen Sender von Daten mitteilen.

Die Erkennung dieser Fehler delegiert MaMa an verschiedene Bausteine – die wichtigsten davon sind `ProcessErrorHandler` (aus `CampaignProcessControl`) und `InputErrorHandler` (aus `Input`).

9 Entwurfsentscheidungen

9.1 Kein CRM-Werkzeug

- Entscheidung: MaMa verwendet kein CRM-Werkzeug als technische Grundlage.
- Begründung: Kommerzielle CRM-Werkzeuge erfüllen die funktionalen Anforderungen von MaMa nicht.

9.2 Kein ETL-Werkzeug

- Entscheidung: MaMa verwendet kein ETL-Werkzeug zum Import von Daten oder Dateien.
- Begründung: Zu hohe Lizenz- und Betriebskosten kommerzieller ETL-Werkzeuge.
- Offen: Die Open-Source-Suite [Pentaho] enthält eine leistungsfähige ETL-Komponente namens Kettle. Diese sollte möglichst zeitnah auf ihre Verwendbarkeit in MaMa geprüft werden, bevor zusätzliche Aufwände bei der Weiterentwicklung der Validatoren entstehen.

10 Qualitätsszenarien

<Entfällt im Beispiel>

11 Risiken

ID	Problem/Risiko	Auswirkung	Beschreibung
R1	Input-Kette ist selbst entwickelt.	Überhöhte Aufwände bei Implementierung	Eigenentwicklung der gesamten Input-Kette erscheint zurzeit sehr aufwendig, könnte durch Einsatz eines Open-Source-ETL-Tools [Pentaho] vereinfacht werden. Siehe Sektion 9.2.
R2	Hohe Komplexität beim Reporting von fehlerhaften Input-Dateien	Hohe Entwicklungs- und Betriebsaufwände, schlechte Verständlichkeit des Input-Packages	Schlechte Fehlerbehandlung bei Validierung von Eingabedaten. Der Validator behandelt Fehler (z. B. Syntaxfehler in Eingabedateien) zurzeit völlig anders als sämtliche anderen Input-Bausteine. Das führt zu Mehraufwand und erhöhter Komplexität beim Reporting und Clearing.
R3	Receiver-Baustein funktional überlagert	Hohe Wartungsrisiken bei Änderungen an Receiver	Siehe Sektion 5.2.1.1. Receiver übernimmt zurzeit zu viele Aufgaben und sollte in zwei Funktionsblöcke zerlegt werden (Receiver und InputController).

12 Glossar und Referenzen

Begriff	Bedeutung
CRM	Customer Relationship Management. Siehe de.wikipedia.org/wiki/Customer_Relationship_Management .
Endkunde	Synonym: Kunde, Marktteilnehmer, Verbraucher, Einzelperson, Familie oder sonstige Personengruppe, die in einem Vertrags- oder Kundenverhältnis mit einem Mandanten steht.
ETL	Extract-Transform-Load. Aus dem Data-Warehousing bekannte Familie von Werkzeugen zum Import von Daten. Siehe de.wikipedia.org/wiki/ETL-Prozess .
Mandant	Synonym: Anbieter. Anbieter von Produkten oder Dienstleistungen im Massenmarkt. Für Mandanten wickelt MaMa die (virtuelle) Kundenbeziehung im Rahmen einzelner Kampagnen ab.
Kampagne	Zeitlich befristete Aktion eines Mandanten, in der ein definierter Waren- oder Informationsaustausch zwischen Mandant und ausgewählten Endkunden oder Endkundengruppen stattfindet. Im Sinne von MaMa finden Kampagnen mit dem Ziel statt, Änderungen an bestehenden Vertragsverhältnissen herbeizuführen (z. B. andere Tarife, Vertragslaufzeiten, Produkt- oder Leistungskombinationen oder Ähnliches).
Partner	(Synonym: Dienstleistungspartner) Organisation oder Unternehmen, das Dienstleistungen innerhalb von Kampagnen übernimmt. Partner sind über elektronische Schnittstellen in die Kampagnen eingebunden. Typische Partner sind Druck-, Scan- und Telefondienstleister.
Rohergebnisdaten	Daten, die von Partnern an MaMa geliefert und dort aggregiert, umgerechnet oder angereichert werden, um die Ergebnisdaten einer Kampagne zu ermitteln.

Referenzen

Quelle	Beschreibung
[Drools]	Jboss Drools Rule Engine: Online: www.jboss.org/drools/
[HoneyComb]	Sun Microsystems: OpenSolaris HoneyComb Fixed Content Storage, frei verfügbares Speicher-/Archivsystem für x86-Systeme. Online: www.opensolaris.org/os/project/honeycomb/
[Olivieri]	Ricardo Olivieri: www.ibm.com/developerworks/java/library/j-drools/
[Pentaho]	Pentaho Open Source Business Intelligence. Online: www.pentaho.com/
[Validator]	Apache Commons Validator Framework: Online: commons.apache.org/validator/

Anmerkungen zu diesem System

MaMa durfte ich in der Zeit ab 2004 irgendwo in Deutschland wirklich (mit) entwerfen und seine Umsetzung begleiten. In der Realität waren die umgesetzten Geschäftsprozesse leicht umfangreicher als hier dargestellt. Die Namen des Systems und der Auftraggeber habe ich absichtlich verändert.

Auch in der Realität bildet MaMa eine Familie komplexer Geschäftsprozesse ab – eine so genannte Produktfamilie. Es setzt innerhalb dieser Prozesse gewisse Ähnlichkeiten voraus, die für viele CRM-Kampagnen¹¹ erfahrungsgemäß gegeben sind – beispielsweise die in Sektion 9.1, Bild 12.17 dargestellte Abfolge von Input- und Output-Aktivitäten.

Das System befindet sich bei seinen zufriedenen Kunden und Mandanten seit 2005 im produktiven Einsatz.

¹¹ Die CRM-Experten unter den Lesern mögen mir die starke Vereinfachung des Themas verzeihen. Mir ist bewusst, dass sich hinter „CRM“ noch weit mehr verbirgt, als ich in diesem Beispiel dargestellt habe.

*Wer als Werkzeug nur einen Hammer hat,
sieht in jedem Problem einen Nagel!*

Paul Watzlawik



Fragen, die dieses Kapitel beantwortet:

- Welche Kategorien von Werkzeugen können helfen?
- Nach welchen Kriterien könnte ich solche Werkzeuge auswählen?

Eine kleine Warnung vorweg: Sie werden in diesem Kapitel nichts über konkrete Werkzeuge lernen, keine Bedien- oder Einsatzhinweise finden, keine Tipps, wie Sie mit Tool X Ihr Problem Y lösen.

Vielmehr möchte ich versuchen, etwas Systematik in die Vielfalt der Werkzeuglandschaft zu bringen – und Ihnen das Spektrum aller möglicherweise hilfreichen Werkzeugkategorien vorstellen. Dazu erfahren Sie ein wenig über mögliche Auswahlkriterien.

Kategorien von Werkzeugen

■ 13.1 Kategorien von Werkzeugen

Werkzeuge zum Anforderungsmanagement

Anforderungsmanagement sollte über den gesamten Lebenszyklus von Systemen praktiziert werden – in Form von Lastenheften oder Backlogs, mit Use-Cases, User-Stories oder anderen Formen von funktionalen und Qualitätsanforderungen. Es geht hierbei um Erhebung, Erfassung, Analyse, Dokumentation und Management von Anforderungen.

Diese Werkzeuge sollten textuelle und/oder grafische Darstellung ermöglichen, Redundanzen reduzieren helfen sowie im Bedarfsfall die Rückverfolgbarkeit zwischen Anforderungen und Architektur/Code erlauben.

Herausforderungen sind insbesondere Mehrbenutzerfähigkeit, Zusammenführen/Merge von Modellen sowie die Unterstützung von Versions- und Konfigurationswerkzeugen.

Modellierungswerkzeuge

Modellierungswerkzeuge können fachliche und technische Modelle von Software sowie Anforderungs- und Problemdomänen darstellen. Sie helfen, (meist grafische) Modelle zu erstellen und zu pflegen.

Diese Kategorie von Werkzeugen unterstützt ab der Ebene von Geschäftsprozessen bis hin zur Modellierung von Klassen, Detailabläufen oder Zustandsübergängen beliebig abstrakte oder detaillierte Darstellung statischer und dynamischer Abstraktionen.

Herausforderungen dieser Werkzeugkategorie sind Mehrbenutzerfähigkeit, Vergleich und Analyse von Modellen, Trennung von Modellobjekten und deren grafische Darstellung in Diagrammen, explizite Metamodelle sowie ein Kompromiss aus Standardkompatibilität (etwa zu UML) und Pragmatik (etwa: Freiform-Diagramme). Hilfreich können Reverse-Engineering-Fähigkeiten sein sowie die Generierung von Dokumenten aus Modellen.

Werkzeuge zur Generierung von Code oder sonstigen Artefakten

Generierungswerkzeuge können ausgehend von abstrakten Beschreibungen (grafischen oder textbasierten Modellen in standardisierten oder domänenspezifischen Sprachen) beliebige Artefakte generieren. Primär finden sie Anwendung zur Erzeugung von Quellcode, Datenbanktabellen oder -skripten, Konfigurationsdateien, Lexern oder Parsern, XML-Schemata oder in Form von Web-Service-Unterstützung. Interessant für Entwickler ist die Möglichkeit der programmatischen Nutzung (über Programmierschnittstellen oder die Integration in Build-Prozesse).

Herausforderung dieser Werkzeuge ist die häufig geringe Akzeptanz bei Entwicklern, die aufgrund des hohen Leistungsumfangs lernintensive Bedienung, die Flexibilität der Generierung beziehungsweise Transformationen sowie die Unterstützung frei definierbarer oder veränderbarer Metamodelle.

Werkzeuge zur Generierung von Dokumentation

Als Spezialfall der oben genannten Generierungswerkzeuge können solche Werkzeuge beispielsweise aus Vorlagen im Plaintext (etwa: AsciiDoc oder Markdown) beliebige Artefakte generieren.

Nachfolgende Abbildung zeigt das Beispiel der docToolchain, die als Eingabeformat primär AsciiDoc verwendet, aber auch EA-Modelle, PowerPoint und Grafiken versteht, und daraus dann pdf, html und auch Confluence-Wikis erzeugen kann.

Werkzeuge zum Codemanagement/Implementierungswerkzeuge

Diese Tools unterstützen Sie bei der Erstellung, Verarbeitung, Modifikation, Prüfung, Übersetzung und Ausführung von Quellcode. Primär zähle ich jede Art von Entwicklungsumgebung hierzu, aber auch (syntaxbewusste) Editoren, Refactoring-Tools, Debugger und alle Hilfsmittel rund um die Versionierung (siehe dazu auch das Thema Build- und Konfigurationsmanagement).

Die Unterstützung von Merging, Branching sowie garantiert verlustfreie Speicherung sämtlicher Versionen aller kontrollierten Artefakte für eine Vielzahl von Benutzern zählt zu den Grundforderungen. Für die Akzeptanz sind Verbreitung, Verständlichkeit sowie Anpassbarkeit an spezielle Aufgaben/Anforderungen hilfreich.

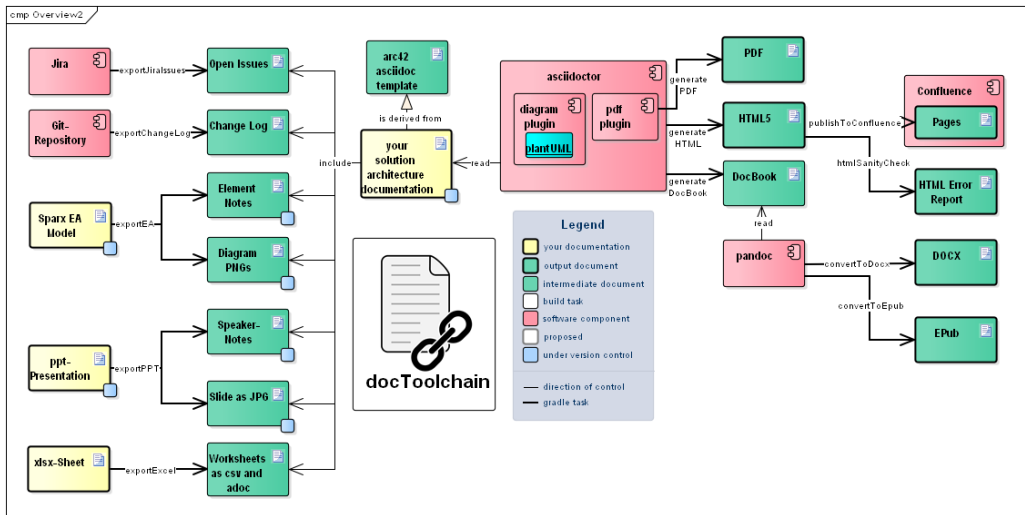


Bild 13.1 Generierung von Dokumentation am Beispiel docToolchain¹

Werkzeuge zur statischen Codeanalyse

Diese Tools prüfen statische Eigenschaften von Quellcode – beispielsweise Abhängigkeiten (Kopplung), Komplexität, Kohäsion, Größe, Testabdeckung oder Einhaltung von Programmierkonventionen.

Eine große Herausforderung dieser Kategorie liegt im heute verbreiteten *polyglott programming*, d. h. der Unterstützung unterschiedlicher Programmiersprachen. Weiterhin hat die starke Verbreitung von Dependency-Injection Frameworks die direkte Analyse von Abhängigkeiten deutlich erschwert – weil Abhängigkeiten in solchen Frameworks dynamisch per Konfiguration aufgebaut werden können.

Werkzeuge zur Laufzeitanalyse von Software

Laufzeit- oder dynamische Analyse betrachtet Softwaresysteme während ihrer Ausführung. Wesentliche Arten von Resultaten sind Laufzeit- und Geschwindigkeitsmessungen, Zeitmessungen bestimmter Systemteile (absolut oder relativ), Messung von Speicher- oder Ressourcennutzung, Kommunikationsverhalten sowie statistische Auswertungen.

Die große Herausforderung besteht darin, durch die Messungen das Laufzeit- oder Speicherverhalten des Systems möglichst wenig zu beeinflussen sowie die Ergebnisse zielgruppengerecht aufzubereiten. In verteilten Systemen gestaltet sich die Korrelation der auf unterschiedlichen Knoten gemessenen Resultate beliebig schwierig. ☹

Werkzeuge zum Build- und Konfigurationsmanagement

Diese Werkzeuge erlauben die teilweise oder vollständige Automatisierung von Übersetzungs-, Paketierungs-, Prüf-, Test- oder Generierungsaufgaben von Software. Zu ihnen zählen

¹ docToolchain ist ein OpenSource-Projekt von Ralf D. Müller zur flexiblen Generierung von (Architektur-)Dokumentation, primär auf Basis von AsciiDoc. Siehe: <https://github.com/rdmueller/docToolchain>

Continuous-Integration-Systeme (die beispielsweise nach Commits im Versionsmanagement die benötigten Teile des Build-Prozesses gezielt anstoßen).

Schwierig für diese Werkzeuge ist der Kompromiss aus deklarativer und imperativer Darstellung von Build-Aufgaben, die Behandlung direkter oder transitiv abhängiger Artefakte und Bibliotheken sowie die Erreichung möglichst hoher Automatisierungsgrade.

Management-Aufgaben wie Inventarisierung sowie Wiederherstellung beliebiger Konfigurationen (inklusive Betriebssystem, benötigte Basissoftware und Datenbestände, Dokumentation sowie der eigentlichen Systeme) sind in manchen Fällen notwendig.

Werkzeuge zum Test von Software

Insbesondere automatisierte Tests geben ein frühes Feedback zu Strukturen, Konzepten, Schnittstellen und Implementierung bereits während der Konstruktion und Entwicklung. Unit-Integrations- und Akzeptanztests erleichtern oft die Kommunikation mit Testern und auch Fachabteilungen. Laufzeit-, Last- und Performance-Tests während der Entwicklung erlauben Prognosen über das Zeit- und Ressourcenverhalten des fertigen Systems. Penetrations- und Sicherheitstests adressieren die Sicherheit der Systeme und beteiligten Daten.

Zu den Herausforderungen zählen die Verwaltung von Testfällen, Testdaten sowie Testergebnissen, die Unterstützung für den Test verteilter Systeme, der Test grafischer Oberflächen sowie lange laufender fachlicher Abläufe. Es hat sich als gute Praxis herausgestellt, Fremdsysteme über sogenannte Mocks zu entkoppeln, die zu Testzwecken das Verhalten dieser Fremdsysteme simulieren.



Ich halte die testorientierte Entwicklung (in Form von Test-Driven oder Acceptance-Test-Driven Development) für ein sehr hilfreiches Entwurfsinstrument, das die Qualität detaillierter Entwurfs- und Implementierungsentscheidungen beträchtlich steigern kann.

Werkzeuge* wie Spockframework, Cucumber, Fitnesse oder andere sollten zur Standardausstattung praktizierender Softwarearchitekten gehören.

* Siehe etwa <http://spockframework.org>, <http://cukes.info>, <http://fitnesse.org> oder [Freeman+10].

Werkzeuge zur Dokumentation

Werkzeuge zur Dokumentation sollen uns bei der langfristigen Kommunikation von Entscheidungen, Strukturen, Konzepten sowie sonstiger relevanter Sachverhalte unterstützen. Zu dieser Kategorie gehören text- und grafikbasierte Werkzeuge zur Erstellung, Pflege oder Generierung von Dokumenten.

Für alle Dokumentationswerkzeuge auf der Basis binärer Formate stellen Teamfähigkeit (parallele Bearbeitung) und Zusammenführung (merge) Herausforderungen dar. Rein textbasierte Tools haben dafür Probleme mit formalen oder gestalterischen Vorgaben (z. B. Corporate Layout, Corporate Design).

■ 13.2 Typische Auswahlkriterien

Im realen Leben müssen Sie selbst die Menge der für Sie und Ihre Stakeholder relevanten Auswahlkriterien für Werkzeuge ermitteln – aber ein paar typische Vertreter beziehungsweise Oberbegriffe möchte ich Ihnen dazu mitgeben.

Technische Kriterien

Arbeitsergebnisse (Artefakte, Dokumente, Modelle) sollten in jedem Fall versioniert gespeichert werden können, d. h. fast alle Werkzeuge benötigen eine Integration mit Subversion, git und Co.

Da wir die meiste Software in Teams entwickeln, müssen oftmals mehrere Benutzer parallel mit Werkzeugen arbeiten – d. h. die sie sollten arbeitsteiliges Vorgehen unterstützen. Mehrere Bearbeiter sollten parallel agieren können, ohne sich gegenseitig zu behindern.

Organisatorische Kriterien

Lizenzmodalitätenkosten spielen bei der Auswahl von Werkzeugen oftmals eine bedeutende Rolle. Beachten Sie, dass eine gesamthafte Kostenbetrachtung von Werkzeugen neben den Anschaffungskosten auch die Kosten der Einführung, Konfiguration sowie eventuelle regelmäßige Wartungs- und Betriebskosten berücksichtigen sollte.

Für manche Produkte benötigen Sie professionelle Unterstützung – dann sollten Sie die Verfügbarkeit dieser Unterstützungsleistung sowie deren Kosten ebenfalls in Ihre Entscheidungskriterien aufnehmen.

Weiterhin spielen gerade im Open-Source-Umfeld die Lizenzbedingungen von Produkten oder Frameworks eine Rolle: Klären Sie, welche Nutzungs- und Vertriebsrechte Ihnen diese Lizenzen einräumen. Manche sind in Dual-Lizenz-Modellen erstellt, bei denen die kommerzielle Nutzung anderen Einschränkungen unterworfen ist als die private Nutzung.

Hüten Sie sich vor juristischen Fallstricken – und ziehen Sie im Zweifel einen in IT- und Lizenzrecht versierten Experten zurate.

Schließlich möchte ich Akzeptanz (im Entwicklungsteam) als ein wesentliches Kriterium für die Auswahl von Werkzeugen nennen. Nicht jedes Werkzeug passt zu jedem Team – und ohne Akzeptanz sind manche Werkzeuge zum Scheitern verurteilt. Akzeptanz können Sie durch gezieltes Coaching und Begleitung steigern – aber nur in seltenen Fällen gegen die Überzeugung eines Teams erzwingen.



Fragen, die dieses Kapitel beantwortet:

- Was ist das International Software Architecture Qualification Board?
- Was nützt ein standardisiertes Curriculum für Softwarearchitekten?
- Welche Gebiete umfasst der iSAQB-Lehrplan?
- Wo können Sie die Lerninhalte des iSAQB-Lehrplans nachlesen?
- Welche Teile des iSAQB-Lehrplans deckt dieses Buch ab?

Viele Missverständnisse um Begriffe der Softwarearchitektur

Softwarearchitektur ist eine relativ junge Disziplin, über deren genauen Umfang und Ausgestaltung in der IT-Branche trotz vieler Publikationen immer noch unterschiedliche Meinungen kursieren. Die Aufgaben und Verantwortungsbereiche von Softwarearchitekten werden sehr unterschiedlich definiert und in Entwicklungsprojekten oftmals neu verhandelt.

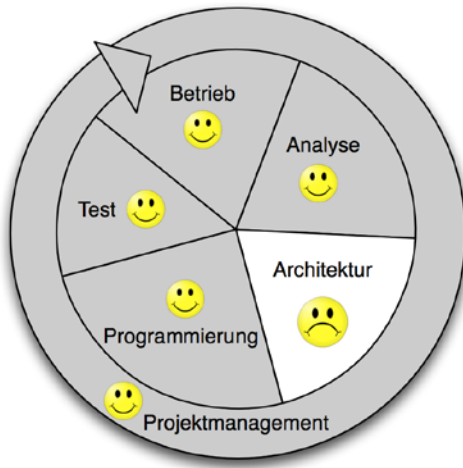
Für andere Disziplinen wie Projektmanagement, Requirements Engineering oder Testen gibt es weitgehenden Konsens, was deren Arbeitsbereiche betrifft. Für sie bieten unabhängige Organisationen¹ Lehrpläne an, die klar beschreiben, welche Kenntnisse und Fähigkeiten eine entsprechende Ausbildung vermitteln muss.

Anfang 2008 haben sich verschiedene Hochschullehrer und Praktiker zum „International Software Architecture Qualification Board“ (ein eingetragener Verein) zusammengeschlossen, um einen standardisierten Rahmen für die Ausbildung von Softwarearchitekten zu definieren:

„Die Grundlagenausbildung² für Softwarearchitekten vermittelt das notwendige Wissen und Fähigkeiten, um für kleine und mittlere Systeme ausgehend von einer hinreichend detailliert beschriebenen Anforderungsspezifikation eine dem Problem angemessene Softwarearchitektur zu entwerfen und zu dokumentieren, die dann als Implementierungsgrundlage bzw. -vorlage genutzt werden kann. Teilnehmer dieser Ausbildung erhalten das Rüstzeug, um problembezogene Entwurfsentscheidungen auf der Basis ihrer vorab erworbenen Praxiserfahrung zu treffen.“ [ISAQB]

¹ Für Tester: ISTQB (www.istqb.org); für Requirements Engineering: www.certified-re.de; für Projektmanagement gibt es mehrere solcher Organisationen mit leicht unterschiedlichem Fokus, etwa das PMI (www.pmi.org).

² Im Sprachgebrauch des iSAQB heißt die Grundlagenausbildung „Foundation Level“ – in Abgrenzung zum „Advanced Level“ der fortgeschrittenen Architekten.

**Bild 14.1**

Rolle und Ausbildung von Softwarearchitekten sind unterspezifiziert.

■ 14.1 Standardisierte Lehrpläne für Softwarearchitekten

Entstanden aus Hochschule und Praxis

Als Grundlage für die Ausbildung von Softwarearchitekten hat der iSAQB in Zusammenarbeit mit Hochschulen und Praktikern umfassende Lehrpläne entwickelt, die das Berufsbild von Softwarearchitekten beschreiben. Seit 2009 liegt ein solcher für die Grundlagenausbildung (*foundation level*) vor und wurde seither mehrfach deutlich aktualisiert. Der *advanced level* definiert eine Reihe vertiefender Module für Fortgeschrittene.

14.1.1 Grundlagenausbildung und Zertifizierung Foundation-Level

Insbesondere vom Foundation-Level-Lehrplan profitieren sowohl Softwarearchitekten wie auch Unternehmen, da er die eingangs geschilderten Missverständnisse bezüglich Rolle und Aufgabe von Softwarearchitekten *unabhängig* klärt (Wer darf sich als Softwarearchitekt bezeichnen? Was leisten Softwarearchitekten für Projekte? Welche Aufgaben sollten sie wahrnehmen?).

Auf Basis dieses Foundation-Level-Lehrplans können Softwarearchitekten die Prüfung und Zertifizierung zum *Certified Professional for Software-Architecture, Foundation Level* (CPSA-F) ablegen.

Gemäß Foundation-Level-Lehrplan gehören folgende Inhalte zur grundlegenden Ausbildung von Softwarearchitekten:

- Begriff und Bedeutung von Softwarearchitektur,
- Aufgaben, Verantwortung und Rolle von Softwarearchitekten sowie
- Methoden und Techniken zur Entwicklung sowie Dokumentation und Kommunikation von Softwarearchitekturen.

Absolventen erlernen die Fähigkeit, mit anderen Beteiligten aus den Bereichen Anforderungs- und Projektmanagement, Entwicklung und Qualitätssicherung wesentliche Softwarearchitekturentscheidungen abzustimmen und Softwarearchitekturen auf der Basis von Sichten, Architekturmustern und technischen Konzepten zu dokumentieren und zu kommunizieren. Sie sollen die wesentlichen Schritte beim Entwurf von Softwarearchitekturen verstehen und nach entsprechender Ausbildung für kleine und mittlere Systeme selbstständig durchführen können.

Der standardisierte Lehrplan definiert die Rolle von Softwarearchitekten und präzisiert deren Aufgaben und Verantwortung in IT-Projekten. Damit hilft er allen Beteiligten bei der Klärung der gegenseitigen Erwartungshaltung.

14.1.2 Fortgeschrittene Aus- und Weiterbildung (Advanced-Level)

Fortgeschrittene Architekten, die Verantwortung für mittlere bis große Systeme übernehmen können, benötigen je nach Branche, Technologie oder System sehr unterschiedliche Schwerpunkte. Der iSAQB e. V. trägt dieser Vielfalt Rechnung, indem die Ausbildung zu Advanced-Level-Architekten individualisierbar angelegt ist. Sie können aus einer Vielzahl von Ausbildungsmodulen die jeweils thematisch passenden zusammenstellen, um das spezifisch benötigte Kompetenzprofil aufzubauen.

Der iSAQB e. V. fordert jedoch, dass Sie dabei folgende drei Kompetenzbereiche abdecken:

- Methodische Kompetenzen: systematisches Vorgehen bei Architekturaufgaben, unabhängig von Technologien
- Technische Kompetenz: Kenntnis und Anwendung von Technologien zur Lösung von Entwurfsaufgaben
- Kommunikative Kompetenz: Fähigkeiten zur produktiven Zusammenarbeit mit unterschiedlichen Stakeholdern, Kommunikation, Präsentation, Argumentation, Moderation

Vor einer Advanced-Level-Zertifizierung müssen Sie in allen diesen Kompetenzbereichen Aus- oder Weiterbildung in Form sogenannter *Credit-Points* nachweisen.

Weitere Voraussetzungen³ für eine *Advanced-Level-Zertifizierung* sind die erfolgreiche CPSA-F-Prüfung, mindestens dreijährige Berufserfahrung an mindestens zwei unterschiedlichen IT-Systemen sowie die erfolgreiche Bearbeitung einer umfangreichen Hausarbeit.

³ Die exakte Liste der Voraussetzungen finden Sie unter [isaqb].

■ 14.2 iSAQB-Foundation-Level-Lehrplan

Bild 14.2 zeigt die Struktur des iSAQB-Lehrplans (Foundation Level) in der Version 5 (Stand Anfang 2020⁴) mit ihrer vorgesehenen zeitlichen Aufteilung.

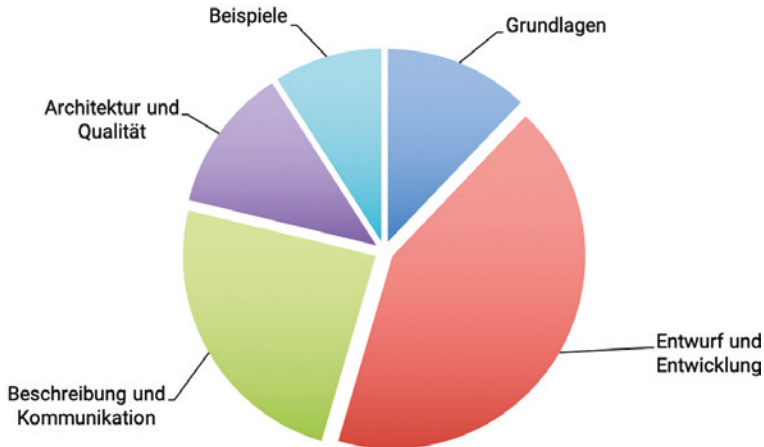


Bild 14.2 Struktur und zeitliche Aufteilung des iSAQB-Lehrplans für Softwarearchitekten

Nach kurzen allgemeinen Hinweisen zu diesem Lehrplan finden Sie in den nachfolgenden Abschnitten die Lernziele der einzelnen Teile. Dazu gebe ich Ihnen Hinweise, wo Sie im Buch Erklärungen dazu finden.

In den Lernzielen gebe ich zu Ihrer Hilfe noch die Prüfungsrelevanz mit an (R1: wird geprüft, R2: kann geprüft werden, R3: wird nicht geprüft).

14.2.1 Können, Wissen und Verstehen

Der iSAQB-Lehrplan differenziert zwischen drei Ebenen von Fähigkeiten:

1. *können*: Diese Aufgaben oder Tätigkeiten müssen Softwarearchitekten selbstständig ausführen können – situativ abhängig von ihrer Praxiserfahrung und den konkreten Projektanforderungen entweder komplett eigenverantwortlich oder auch unterstützt durch weitere Personen. Diese Teile des Lehrplans sind für die iSAQB-Zertifizierung relevant und somit auf jeden Fall Bestandteil entsprechender Prüfungen. Sie sind in den Lernzielen mit **R1** gekennzeichnet.
2. *verstehen*: Diese Lernziele können Bestandteil von Zertifizierungsprüfungen sein; Softwarearchitekten müssen die Aufgaben oder Tätigkeiten nicht unbedingt selbst ausführen können. Diese Teile des Lehrplans können geprüft werden, in den Lernzielen mit **R2** gekennzeichnet.

⁴ Eine Aktualisierung des Lehrplans nimmt der iSAQB ca. alle 24 Monate vor, das nächste Update ist für Ende 2021 geplant.

3. *kennen (wissen)*: Begriffe, Konzepte, Methoden oder Praktiken können das Verständnis für Architekturen oder Architekturentscheidungen unterstützen. Diese Inhalte können in Schulungen bei Bedarf unterrichtet werden, gehören aber nicht zu *Foundation-Level*-Prüfungen. In den Lernzielen mit **R3** gekennzeichnet.

Für die *Können*-Inhalte fordert der iSAQB (ganz in meinem Sinne) ausführliche Übungen in entsprechenden Schulungen. Angehende Softwarearchitekten sollten die Aufgaben selbstständig planen und ausführen können und anschließend in der Lage sein, die erarbeiteten Ergebnisse objektiv zu bewerten.

14.2.2 Voraussetzungen und Abgrenzungen

Entsprechend der oben genannten Zielsetzung setzt der iSAQB-Lehrplan *Erfahrung* in der Softwareentwicklung voraus. Insbesondere gehören folgende Inhalte nicht zum Lehrplan (wohl aber zum notwendigen Können von Softwarearchitekten!):

Voraussetzung:
praktische Erfahrung

- mehr als 18 Monate praktische Erfahrung in der Softwareentwicklung, erworben anhand unterschiedlicher Projekte oder Systeme,
- Kenntnisse und praktische Erfahrung in mindestens einer höheren Programmiersprache,
- Grundlagen von Algorithmen und Datenstrukturen,
- Grundlagen von Modellierung und Abstraktion,
- Grundlagen von UML (Klassen-, Paket-, Komponenten- und Sequenzdiagramme) und deren Abbildung auf Quellcode,
- Praktische Erfahrung in technischer Dokumentation, insbesondere der Dokumentation von Quellcode, von Systementwürfen oder technischen Konzepten.

Da der iSAQB primär auf methodische Fähigkeiten und Wissen fokussiert, gehören konkrete Implementierungstechnologien oder spezielle Werkzeuge explizit *nicht* zum standardisierten Lerninhalt, ebenso wenig wie UML, Systemanalyse oder Projektmanagement.

14.2.3 Lernziele des iSAQB Foundation Level

I. Grundbegriffe von Softwarearchitekturen

LZ-1-1	Definitionen von Softwarearchitektur: Gemeinsamkeiten benennen (Bausteine, Komponenten, Schnittstellen, Beziehungen, Strukturen, Konzepte, Prinzipien)	R1	Abschnitt 2.1
LZ-1-2	Nutzen und Ziele von Softwarearchitektur: Fokus auf Langlebigkeit und Qualitätszielen	R1	Abschnitt 2.1.1
LZ-1-3	Softwarearchitektur im Lebenszyklus	R2	Abschnitte 2.3, 2.4
LZ-1-4	Aufgaben und Verantwortung von Softwarearchitekten	R1	Abschnitt 2.2, Kapitel 3
LZ-1-5	Rolle von Softwarearchitekten in Beziehung zu anderen Stakeholdern	R1	Abschnitt 2.4
LZ-1-6	Zusammenhang zwischen Architektur und Entwicklungsvorgehen	R1	Kapitel 3
LZ-1-7	Kurz- und langfristige Ziele unterscheiden	R1	Abschnitt 2.1.1
LZ-1-8	Explizite und implizite Aussagen	R1	Abschnitt 2.2
LZ-1-9	Softwarearchitektur im organisatorischen Kontext	R3	Kapitel 11
LZ-1-10	Typen von IT-Systemen kennen: Informationssysteme, eingebettete oder Echtzeitsysteme, Decision-Support-Systeme, mobile oder Websysteme	R3	Abschnitt 3.2.2

R1: wird geprüft, R2: kann geprüft werden, R3: wird nicht geprüft

II. Entwurf und Entwicklung von Softwarearchitekturen

In diesem Abschnitt verwenden Sie die Begriffe und Konzepte der voranstehenden Teile, um (unterstützt durch Übungen) selbstständig Architekturentwürfe zu erstellen und zu kommunizieren.

LZ-2-1	Vorgehen zur Architekturentwicklung	R1-3	Kapitel 3
LZ-2-2	Softwarearchitekturen entwerfen	R1	Kapitel 3 und 4
LZ-2-3	Einflussfaktoren erheben und berücksichtigen	R1	Abschnitte 3.1, 3.2, 3.3
LZ-2-4	Querschnittskonzepte entwerfen	R1	Abschnitt 3.6, Kapitel 7
LZ-2-5	Wichtige Architekturmuster und -stile erklären und verwenden	R1-3	Abschnitte 4.5, 4.6
LZ-2-6	Entwurfsprinzipien erläutern und anwenden	R1	Abschnitte 4.1, 4.2
LZ-2-7	Abhängigkeiten von Bausteinen planen	R1	Abschnitte 4.1, 4.2
LZ-2-8	Qualitätsanforderungen erreichen	R1	Kapitel 4
LZ-2-9	Schnittstellen entwerfen	R1-3	Abschnitt 4.4

R1: wird geprüft, R2: kann geprüft werden, R3: wird nicht geprüft

III. Beschreibung und Kommunikation von Softwarearchitekturen

Dieser Abschnitt führt einige Begriffe und Konzepte ein, die Sie zur Kommunikation und Dokumentation von Softwarearchitekturen benötigen. Er hängt eng mit dem vorhergehenden Kapitel über Entwurf und Entwicklung zusammen.

LZ-3-1	Qualitätsmerkmale technischer Dokumentation	R1	Abschnitte 5.2.1, 5.2.2
LZ-3-2	Softwarearchitekturen stakeholdergerecht beschreiben und kommunizieren	R1	Kapitel 5 + 6
LZ-3-3	Notations-/Modellierungsmittel zur Beschreibung von Softwarearchitekturen	R2	Abschnitte 5.3, Kapitel 6
LZ-3-4	Architektursichten erläutern und anwenden	R1	Abschnitte 5.4–5.8, Kapitel 6
LZ-3-5	Kontextabgrenzung erläutern und anwenden	R1	Abschnitt 5.5
LZ-3-6	Querschnittliche Architekturkonzepte dokumentieren und kommunizieren	R1	Abschnitt 5.10, ergänzend: Kapitel 7
LZ-3-7	Schnittstellen beschreiben	R1	Abschnitt 5.9
LZ-3-8	Architekturentscheidungen erläutern und dokumentieren	R2	Kapitel 3, Kapitel 5
LZ-3-9	Dokumentation als schriftliche Kommunikation	R2	Kapitel 5
LZ-3-10	Weitere Hilfsmittel und Werkzeuge zur Dokumentation	R3	Abschnitt 5.11

R1: wird geprüft, R2: kann geprüft werden, R3: wird nicht geprüft

IV. Softwarearchitekturen und Qualität

Das zentrale Thema dieses Kapitels ist Qualität: Qualitätsanforderungen, ausgedrückt durch hierarchische Qualitätsmodelle (beispielsweise DIN/ISO 9126 oder DIN/ISO 25010), mit Qualitätsmerkmalen, Szenarien und Qualitätsbäumen. Da sich Qualitätsmerkmale gegenseitig (meist negativ) beeinflussen, müssen Sie bei Entwurf und Entwicklung häufig Kompromisse bei der Umsetzung von Qualitätsmerkmalen eingehen. Qualitative Architekturbewertung von Risiken hinsichtlich der Erreichung von Qualitätsmerkmalen bildet einen weiteren Schwerpunkt.

LZ-4-1	Qualitätsmodelle und Qualitätsmerkmale	R1	Abschnitte 3.2.3, 8.1
LZ-4-2	Qualitätsanforderungen an Software und Softwarearchitekturen klären	R1	Abschnitt 3.2.3, Abschnitt 8.1
LZ-4-3	Softwarearchitekturen qualitativ analysieren und bewerten	R2-3	Abschnitt 8.1
LZ-4-4	Softwarearchitekturen quantitativ bewerten	R2	Abschnitt 8.2

R1: wird geprüft, R2: kann geprüft werden, R3: wird nicht geprüft

V. Beispiele von Softwarearchitekturen

Sie lernen hier praktische Beispiellarchitekturen sowie den Bezug von Anforderungen zur Lösung kennen sowie (zumindest ansatzweise) die technische Umsetzung der Beispiele. Diese Inhalte sind nicht prüfungsrelevant.

LZ-5-1	Bezug von Anforderungen zur Lösung	R3	Kapitel 12
LZ-5-2	Technische Umsetzung einer Lösung nachvollziehen	R3	Kapitel 12

Zwei Beispiele praktischer Softwarearchitekturen finden Sie in Kapitel 12.

14.2.4 Zertifizierung gemäß iSAQB

Sie haben die Möglichkeit, sich bei verschiedenen unabhängigen Anbietern⁵ durch eine Prüfung gemäß iSAQB-Lehrplan zum *Certified Professional for Software Architecture – Foundation Level* (CPSA-F) zertifizieren zu lassen. Die Prüfungsanbieter müssen dabei standardisierte, vom iSAQB erarbeitete Prüfungsfragen verwenden – die sich strikt am CPSA-F-Lehrplan orientieren.

Die Prüfung selbst findet nach einem Multiple-Choice-Verfahren statt, entweder konventionell auf Papier oder auch in Testcentern online am Computer. So erhalten Sie objektive Prüfungsergebnisse, frei von Interpretation und Ermessen. Ein paar Beispielfragen finden Sie im folgenden Abschnitt

Nach meiner persönlichen Meinung können Sie durch eine erfolgreiche Prüfung zwar *Wissen* nachweisen, Ihre Qualifikation als Softwarearchitekt jedoch nur sehr bedingt unter Beweis stellen. Das gelingt Ihnen erst durch (erfolgreiche) praktische Arbeit an konkreten Systemen und deren Architekturen.

⁵ Auf der Website des iSAQB finden Sie eine aktuelle Übersicht der Prüfungsfirmen (genannt „Zertifizierer“ oder „certifying bodies“).

■ 14.3 Beispielfragen zur Foundation Level Prüfung

Ich kann und darf hier natürlich keine Originalfragen des iSAQB wiedergeben – aber ein paar Beispiele möchte ich Ihnen zeigen.

Es gibt in der Prüfung drei Typen von Prüfungsfragen, wobei oftmals Ihre sachliche Einschätzung gefragt ist („wählen Sie die zwei am ehesten zutreffenden Antworten“):

- A-Frage (A wie „Auswahl“), Einfachauswahl: Auf eine Frage folgen mehrere vorgegebene Antworten, aus denen Sie die einzige richtige oder die beste auswählen sollen. Die Distraktoren müssen nicht völlig falsch sein, die richtige Antwort hebt sich aber deutlich ab. Richtige Antwort gibt volle Punktzahl, keine oder falsche Antwort oder mehrere Kreuze ergeben null Punkte.
- P-Frage (P wie „Pick“ oder Mehrfachauswahl): Mehrfachauswahlfragen (P = »Pick«) – Hier sollen Sie aus einer Liste von Antwortmöglichkeiten die im Aufgabentext vorgegebene Anzahl von korrekten oder am besten zutreffenden Antworten auswählen. Für jede korrekt gewählte Antwort erhalten Sie anteilig Pluspunkte. Falsche Antworten zählen anteilig negativ. Lassen Sie hier Antworten aus, wird das neutral gewertet. Beispiel: Sie sollen zwei am besten passenden Antworten auswählen, die Frage ergibt 2 Punkte:
 - Fall 1: Sie wählen die beiden korrekten Optionen und erhalten zwei Punkte.
 - Fall 2: Sie wählen eine richtige und eine falsche Option und erhalten null Punkte (einmal +1, einmal –1)
 - Fall 3: Sie wählen nur eine (korrekte) Option und erhalten 1 Punkt.
 - Fall 4: Sie wählen gar nichts aus und erhalten null Punkte.
 - Fall 5: Sie wählen zwei falsche Optionen aus und erhalten null Punkte.
- K-Frage (K wie „Klärung“), Zuordnungsfragen: Bei jeder Antwortmöglichkeit müssen Sie hier die Auswahl aus je zwei (gegenseitig ausschließenden) Möglichkeiten treffen.

Sollte es vorkommen, dass Sie einzelne Antwortmöglichkeiten einer P- oder K-Frage nicht beantworten können: Verzichten Sie in diesen Fällen auf Raten, da falsch raten Ihnen Punktabzug einbringen könnte! Weniger Kreuze zu machen als gefordert ist ja neutral (d. h., es gibt dadurch keine Minuspunkte).

Nun aber zu den Beispielen:

K-Frage: Sie dokumentieren ein großes Subsystem (Top-Level-Baustein) Ihrer Bausteinsicht. Welche Informationen sollte die Blackbox-Beschreibung dieser Komponente enthalten und welche nicht?

Ordnen Sie jede Antwortmöglichkeit zu.

soll	soll-nicht	
☒	☐	Schnittstellen
☒	☐	Verantwortlichkeiten
☐	☒	Interne Struktur
☐	☒	Resultate von Performancemessungen

A-Frage: Wie viele Definitionen gibt es für den Begriff "Softwarearchitektur"?

Wählen Sie die eine korrekte Antwort aus.

- ☐ Genau eine für alle Arten von IT-Systemen.
- ☐ Für jeden Typus von Softwaresystem (z. B. Mobile, Embedded, Real-Time, Decision-Support, Web, Batch etc.) eine eigene.
- ☒ Ein Dutzend oder mehr verschiedene.

P-Frage: Der Begriff "Softwarearchitektur" umfasst welche der folgenden Themen?

Wählen Sie die drei am besten zutreffenden.

- ☒ Bausteine/Komponenten
- ☒ Querschnittliche Konzepte („Prinzipien“)
- ☒ (interne und externe) Schnittstellen
- ☐ Datenbank-Trigger
- ☐ Hardware-Sizing

P-Frage: Welche der folgenden Voraussetzungen sollten vor der Erstellung/Entwicklung einer Softwarearchitektur gegeben sein?

Wählen Sie die zwei am besten passenden Antworten aus.

- ☐ Die Anforderungen liegen vollständig, detailliert und konsistent vor.
- ☒ Die wesentlichen Qualitätsanforderungen sind bekannt.
- ☒ Organisatorische Randbedingungen sind bekannt.
- ☐ Continuous-Integration Server ist installiert und betriebsbereit.
- ☐ Hardware-Testsystem ist vorhanden und betriebsbereit.

P-Frage: Welche Faktoren können den Entwurf von Softwarearchitektur beeinflussen?

Wählen Sie die drei am besten passenden Antworten aus.

- ☒ Politische
- ☒ Organisatorische
- ☒ Technische
- ☐ Moralisch-ethische

P-Frage: Identifizieren Sie Beispiele für typische Kategorien von Softwaresystemen?

Wählen Sie die drei am besten passenden Antworten aus.

- ☒ Batch-System
- ☒ Interaktives Online-System
- ☐ Fourier-System
- ☒ Embedded-Echtzeit-System
- ☐ Integrationstest-System

P-Frage: Welche Architektursichten finden in der Softwareentwicklung praktische Anwendung?

Wählen Sie die zwei am besten passenden Antworten aus.

- ☐ Pattern-Sicht
- ☐ Fourier-Sicht
- ☒ Baustein- oder Komponentensicht
- ☒ Deployment-/Verteilungssicht

K-Frage: Sie können Aspekte Ihrer Softwarearchitektur schriftlich und/oder mündlich kommunizieren. Wie hängen diese beiden Varianten zusammen?

Ordnen Sie jede Antwortmöglichkeit zu.

Wahr Falsch

- | | | |
|-------------------------------------|-------------------------------------|--|
| ☒ | ☐ | Mündliche Kommunikation kann und soll schriftliche Dokumentation ergänzen. |
| ☐ | ☒ | Schriftliche Ausarbeitung von Dokumentation muss unbedingt zeitlich vor der mündlichen Kommunikation erfolgen. |
| ☐ | ☒ | Teams sollten sich für eine Variante (schriftlich oder mündlich) entscheiden und während der Entwicklung dann konsistent diesen Weg verfolgen. |

A-Frage: Welches Ziel verfolgt das Prinzip der Umkehrung von Abhängigkeiten (Dependency-Inversion Principle)?

Wählen Sie die die korrekte Antwort aus.

- ☐ Homogenisierung der Code-Größe (Lines-of-Code) zwischen abhängigen Komponenten.
- ☐ Bausteine sollen abhängige Bausteine leichter überprüfen können.
- ☒ Betroffene Bausteine sollen möglichst nur über Schnittstellen voneinander abhängen.

K-Frage: Was sind Kennzeichen enger (hoher) oder loser (geringer) Kopplung?

Ordnen Sie jede Antwortmöglichkeit zu.

Enge Kopplung	Lose Kopplung	
☒	☐	Bausteine rufen direkt ihre abhängigen Bausteine auf, d. h. ohne Umweg über Schnittstellen oder Abstraktionen
☒	☐	Bausteine verwenden gemeinsame Datentypen.
☒	☐	Bausteine verwenden eine gemeinsame Datenbank.
☐	☒	Bei der Implementierung von Bausteinen ist durchgängig das Dependency-Inversion-Prinzip eingehalten.
☐	☒	Bausteine kommunizieren asynchron.

A-Frage: Welche der nachfolgend genannten Eigenschaften können Sie durch Verwendung des Schichten-Musters am ehesten verbessern?

Wählen Sie die eine korrekte Antwort aus.

- ☐ Geringe Latenz (d. h. hohe Performance)
- ☒ Flexibilität bei Weiterentwicklung oder Änderungen am System
- ☐ Flexibilität zur Laufzeit („Konfigurierbarkeit“)
- ☐ Datenkonsistenz

P-Frage: Welche Aussagen über das Prinzip „Don't Repeat Yourself“ treffen zu?

Wählen Sie die zwei am besten zutreffenden Antworten aus.

- ☐ Die strikte Einhaltung von DRY könnte die Modularisierung verschlechtern.
- ☒ Die strikte Einhaltung von DRY könnte zu höherer Kopplung führen.
- ☒ Die Bestandteile, die redundante Teile enthalten, könnten unabhängig voneinander modifiziert und weiterentwickelt werden.
- ☐ Die Einhaltung von DRY führt bei der IT-Sicherheit zu einer Reduktion von Angriffsvektoren.
- ☐ Erst die Umkehrung von Abhängigkeiten (Dependency-Inversion -Principle) ermöglicht die Anwendung des DRY-Prinzips.

P-Frage: Welches sind wesentliche Ziele von Softwarearchitektur?

Wählen Sie die zwei am besten passenden Antworten aus.

- ☐ Muster-Treue in Struktur und Implementierung des Systems verbessern
- ☒ Notwendige Qualitätsanforderungen des Systems nachvollziehbar erreichen
- ☐ Den kostengünstigen Integrations- und Akzeptanztest des Systems ermöglichen
- ☒ Dem Entwicklungsteam und anderen Stakeholdern ein grundsätzliches Verständnis der Strukturen und Konzepte des Systems ermöglichen

In diesem Kapitel finden Sie eine überarbeitete Fassung meines Artikels „Von Analytistan nach Architektonien“, im Original erschienen in der Zeitschrift iX Ausgabe 8/2006. Veröffentlichung mit freundlicher Genehmigung des Heise Zeitschriftenverlages.

In Kürze

Das kleine Architektonien grenzt unmittelbar ans bekannte Analytistan mit seiner prächtigen Hauptstadt Anforderungshausen. Zwischen beiden Lagern herrscht ... ja, wie soll man es nur ausdrücken ... eine Art gespannter Zustand: Während in Anforderungshausen essenzielle Wünsche in den Köpfen der Bewohner wabern, müssen die Architektonier die wirkliche Implementierung vorbereiten und überwachen. Sie stehen unter dem Einfluss herrschsüchtiger Managier und penibler Qualitessen und müssen gleichzeitig die kritischen Stimmen der zahlreichen Prograländer aushalten.

Sich in diesem Spannungsfeld langfristig zu behaupten, erfordert sowohl Vielseitigkeit als auch Kunstfertigkeit. In den langen Jahren ihrer kargen Existenz haben die Architektonier dafür ihr architektonisches Manifest geschaffen, aus dem ich Ihnen einige Auszüge vorstellen möchte.

Folgen Sie mir vorab auf eine kurze Reise durch die IT-Welt, auf der ich Ihnen die wesentlichen Stationen vorstellen möchte.

■ 15.1 In sechs Stationen um die (IT-)Welt

Wie Sie sicher wissen, gehört die IT-Welt ganz und gar den Kundeniern – einer größtenteils unerforschten, unberechenbaren und ziemlich vielseitigen Gattung *homo dubiosis* mit einem Einschlag von *homo semisapiens*. Aus unerfindlichen Gründen beschränken die Kundenier den direkten Kontakt mit der schönen IT-Welt auf drei Länder, nämlich Analytistan, Betriebien und Kostenia. Und werden einmal Bewohner der übrigen IT-Länder nach Kostenia eingeladen (oder entführt!), geschieht das entweder, um sie dort ins Gefängnis zu werfen oder ihnen eine andere Staatsbürgerschaft zu verleihen.

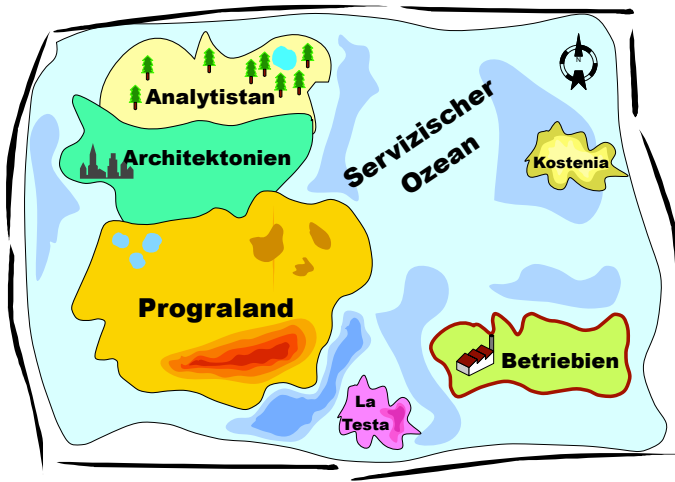


Bild 15.1
Landkarte der IT

Analytistan

Dies ist die Heimat der liebenswürdigen und stets freundlichen Analytistaner, auch genannt Reques (gesprochen: Rekkies). Diese Spezies spricht die gleiche Sprache wie die bereits erwähnten Kundenier. Reques verfügen über kommunikative und hypnotische Fähigkeiten, mit denen sie sogar unbewusste Wünsche der Kundenier erkennen können. Analytistaner fürchten sich vor nichts mehr als vor Entscheidungen – niemals würden sie eigenständig eine Auswahl zwischen möglichen Alternativen treffen. Stets befragen sie in solchen Fällen die ihnen wohlgesonnenen Kundeniern (die allerdings, seien wir ehrlich, häufig die Antwort schuldig bleiben).

Reques gehen zu Beginn von IT-Projekten häufig bei den betroffenen Kundeniern ein und aus. Dies führt jedoch auf beiden Seiten zu gewissen Abnutzungserscheinungen, sodass die intensive Kommunikation oftmals bereits nach kurzer Zeit wieder eingestellt wird.

Reques lieben das geschriebene Wort über alles und pflegen das Landesmotto „*Papier ist geduldig*“. Wo ihre Nachbarn aus Architektonien oder die technophilen Programmisten zu präzisen Werkzeugen wie Java oder Ruby greifen, bleiben Reques grundsätzlich bei redundanzbehafteter, ungenauer, aber dafür ästhetischer, natürlicher Sprache.

Bemerkenswert an den Reques ist auch ihre fast schon legendäre Unbeliebtheit: Außer bei Kundeniern dürfen sie sich praktisch nirgendwo auf dem IT-Globus blicken lassen – obwohl viele von ihnen ziemlich nett sind, ehrlich!

Architektonien

Architektonier lieben die Dunkelheit – zumindest müssen sie dort arbeiten. Und falls es bei der Arbeit doch mal hell wird, herrscht garantiert dichter Nebel. Mit 99%iger Wahrscheinlichkeit hat ein Architektonier bei der Arbeit keine klare Sicht, weder auf die zu lösenden Aufgaben noch auf die vorhandene Technologie.

Macht aber nichts – Architektonier haben im Lauf ihrer fast 25-jährigen Evolution großen Mut entwickelt. Und den brauchen sie auch, nämlich um ihrer wichtigsten Aufgabe nachzukommen: Entscheidungen zu treffen. Ihre zweite Aufgabe lautet, über ihre Arbeit zum

Volk zu sprechen. Architektonier besitzen eine gehörige Portion Sendungsbewusstsein und Kommunikationstalent.

Zwischen Analytistan und Architektonien verläuft eine Grenze, die der ehemaligen innerdeutschen Grenze ähnlich ist: Architektonier können sie dank ihres legendären Mutes problemlos in beide Richtungen passieren, für Requies bleibt sie jedoch in beide Richtungen versperrt.

Prograland

Die Heimat von Geeks, Nerds und weiteren technophilen Vielseitern. Alle Neugeborenen werden in Prograland traditionsgemäß in lauwarmer Bytesuppe gelegt. Das prägt, gründlich und dauerhaft. Prograland strafft sämtliche biologischen Thesen Lügen: Trotz 98,8 % maskuliner Bevölkerung nimmt die Einwohnerzahl von Prograland ständig zu. Irgendwie muss eine Art Raum-Zeit-Kanal frische Geeks in dieses vielseitige, farbenfrohe und wortkarge Land einschleusen. Aber das soll hier keine Rolle spielen.

Prograland besteht aus einer Vielzahl von Provinzen, zwischen denen teilweise offene Konflikte herrschen. So befinden sich seit Jahren die Hochebenen von *IX mit den Niederungen von MS*, oder C* neidet J* seine Erfolge.

Insgesamt sind die Prograländer ziemlich verspielt und bleiben gerne unter sich. Requies sind nicht geduldet, Geld und Zeit spielen keine Rolle (es sei denn, es handelt sich um Millisekunden – die zählen. Personentage aber nicht).

Obwohl ich erklärter Sympathisant von Prograländern bin – sie sind (und bleiben) die Urheber vieler dramatischer IT-Probleme. Wer sonst schreibt `#define private public` in die zentrale Header-Datei und umgeht damit das Geheimnisprinzip? Wer sonst produziert Stack-Overflows, Memory-Leaks und Buffer-Overflows (die dann andere Geeks wiederum für gute oder böse Zwecke ausnutzen können)? Aber eigentlich sind sie sehr nett und umgänglich.

La Testa

Von Prograland durch ein wildes Meer und gewaltige Untiefen getrennt liegt die kleine und wilde Insel La Testa. Weitgehend unberührt und unerforscht, ist sie die Heimat der Testanier (auch „Q“ genannt), vor denen sich die Prograländer sehr fürchten. Das sorgt für gute nachbarschaftliche Beziehungen. Kein echter Testanier begibt sich freiwillig nach Prograland. Weder Geld noch gutes Zureden können einen echten Testanier dazu bewegen, seine elaborierten Worte in den, wie die Q ihn nennen, *technokratischen Sumpf* zu expedieren. Manchen Architektoniern sagt man nach, sie hätten erfolgreich zwischen Q und Geeks vermittelt – aber das können genauso gut Gerüchte sein.

In den letzten Jahren bahnt sich in dieser brisanten Situation übrigens Besserung an – nachdem ein Eidgenosse gemeinsam mit einem eXtremen ein paar Zeilen ultracoolen Java-Code geschrieben und unter der Bezeichnung „JUnit“ veröffentlicht hat.

Architektonier und Kundenier sind auf La Testa gern gesehene Gäste. Angeblich sollen gerade Architektonier auf La Testa einer Extremsportart frönen: dem Bug-Fishing. Aber auch das könnte ein Gerücht sein.

Betriebien

Betriebien ist das einzige komplett industrialisierte Land in der IT-Welt. Es ist umgeben von einem Bauwerk beeindruckender Größe – der betriebischen Mauer. Einem Gerücht nach soll es das einzige Bauwerk sein, das aus dem Weltall mit bloßem Auge sichtbar ist. Falls jemand ins Weltall kommt, sollte er dieses Gerücht in der Blogosphäre bestätigen. Ansonsten bekommt kaum ein Einwohner der übrigen IT-Welt jemals die vermeintliche Schönheit von Betriebien zu Gesicht.

Gelegentlich hört oder liest man, dass Betriebien DAS Land der IT-Welt sei, in dem „Prozesse“ wirklich funktionieren. Übrigens das einzige, garantiert!

Kostenia

Über diese Insel mag ich hier nichts weiter schreiben. Es ist die Heimat der Managissimos – der Regenten der IT-Welt. Sie sorgen für Geld und Zeit in IT-Projekten. Wenn sie genug daran gedacht haben, sinnieren sie über Ressourcen. Und ab und zu organisieren sie auch die Zusammenarbeit von Bewohnern der verschiedenen Länder in der IT-Welt.

■ 15.2 Ratschläge aus dem architektonischen Manifest

Bis hierhin haben Sie einige Details über die seltsame IT-Welt gelesen. Im Folgenden möchte ich Ihnen einige Teile des architektonischen Manifests erläutern. Sie sollen Ihnen als Tipps für erfolgreiche Reisen durch dieses Habitat dienen. Die Ratschläge dieses Manifests richten sich an (Software- und IT-)Architekten, können in vielen Fällen aber auch für andere Mitspieler in IT-Projekten positive Wirkung zeigen. Unter anderem verbessert das Verständnis dieser architektonischen Erfolgsmuster die Völkerverständigung in IT-Land – was die Chance auf erfolgreiche Projekte deutlich verbessern hilft.

Handeln Sie aktiv!

Es gibt nichts geschenkt, und niemand (keiner, niemand, nobody) wird Ihnen freiwillig und ohne Aufforderung Geld, Mitarbeiter, Kompetenz oder Mitspracherechte geben, wenn Sie nicht AKTIV danach fragen. Als Architekt müssen Sie entscheiden, mitreden, vermarkten, argumentieren und möglicherweise investieren – und dazu benötigen Sie Ressourcen und Kompetenzen. Handeln Sie aktiv – und holen Sie sich alles, was Sie für Ihre produktive Architekturarbeit benötigen! Warten Sie nicht darauf, dass Ihnen der Weihnachtsmann oder das Christkind oder der Osterhase solche Dinge schenkt – das passiert nur im Märchen (und die IT-Welt ist ziemlich märchenfrei ...)

Arbeiten Sie iterativ

Iterativ vorgehen bedeutet, Ergebnisse regelmäßig zu bewerten, um schrittweise Verbesserungen einpflegen zu können.

Alle Bewohner der IT-Welt profitieren von Feedback, Rückmeldungen und Ratschlägen. Diskutieren Sie mit Ihren KollegInnen über Ihre Arbeitsergebnisse, Ihre Entwürfe und Strukturvorschläge. Dieser Ratschlag gilt ganz besonders für Architekturentwürfe und deren Dokumentation (obwohl auch alle anderen Arten von Artefakten von iterativer Verbesserung profitieren!).

Durch iteratives Vorgehen können Sie die Qualität Ihrer Arbeitsergebnisse deutlich steigern und gleichzeitig das Risiko von Fehlern oder Unzulänglichkeiten drastisch senken. Diesem Argument öffnen sich auch hartgesottene und kostenzentrierte Managissimos!

Entwickeln Sie eine Null-Rhesus-Negativ-Mentalität

Die Blutgruppe „Null-Rhesus-Negativ“ ist mit sämtlichen anderen Blutgruppen kompatibel. Architektonier müssen mit sämtlichen Beteiligten von IT-Projekten „*kommunikationskompatibel*“ sein. Diese Eigenschaft ermöglicht Ihnen, mit allen anderen Stakeholdern konstruktiv über architekturrelevante Sachverhalte zu kommunizieren.

Sie werden jetzt vielleicht einwenden, dass doch *Managissimos* diese Kommunikation mit anderen Stakeholdern erledigen. Aber das zweifle ich stark an, denn Managissimos können auf *technischem* Niveau oftmals nur unzureichend argumentieren, weil sie beispielsweise der Sprachen von Prograland oder Betriebien kaum mächtig sind.

Üben Sie sich also in Kommunikation. Lernen und üben Sie zu präsentieren, zu argumentieren, zu diskutieren. Trainieren Sie Ihre Kommunikationsfähigkeiten – das steigert neben Ihrer Beliebtheit in Projekten ganz eindeutig Ihre Erfolgsaussichten.

Seien Sie mutig

In Analytistan scheint immer die Sonne, und alle Einwohner sind gut gelaunt. Architektonier hingegen leben die meiste Zeit des Jahres im Nebel und arbeiten praktisch grundsätzlich im Dunkeln.

Requies schreiben auf, was Kundenier ihnen sagen (oder was die Analytiker glauben, was die Kunden gemeint haben könnten). Architektonier müssen (bequeme oder unbequeme) Entscheidungen treffen, die den Erfolg von IT-Projekten stark beeinflussen können. Mut hilft Ihnen dabei als Charaktereigenschaft in vielen Situationen Ihres Architektenlebens weiter. Ich möchte die Empfehlung zu Mut hier in mehrere Richtungen konkretisieren:

- Mut zu suboptimalen Entscheidungen
- Mut zur Lücke
- Mut zum Widerspruch

Als Software- oder IT-Architekt befinden Sie sich ständig in Entscheidungssituationen: Neben Strukturentscheidungen gilt es vielerlei technisch orientierte Aspekte zu entscheiden, die über die Grenzen einzelner Systembausteine hinweg Auswirkungen haben. Ihre Entscheidungen haben also prägenden Einfluss auf das System und seine weitere Entwicklung. Sie beeinflussen das Team, den Entwicklungsprozess und möglicherweise sogar die Anforderungen der Kunden.

Nach Abschluss des Projektes werden Sie auf eine Vielzahl von Entscheidungen zurückblicken und jetzt ganz genau erkennen, welche davon besonders gut und welche suboptimal waren. Zu jedem früheren Zeitpunkt (noch einmal: zu *jedem* früheren Zeitpunkt) bleibt Ihnen diese Bewertung verwehrt – es könnten sich ja noch wichtige Einflussfaktoren, Randbedingungen oder Anforderungen ändern ...

Als Fazit bleibt: Sie können prinzipiell erst am Ende wissen, ob eine bestimmte Entscheidung gut oder schlecht war.

Darum brauchen Sie den Mut, suboptimale, aber notwendige Entscheidungen auch in unklaren Situationen zu treffen. Das Streben nach Perfektion kostet immens viel Zeit und Aufwand – meistens genügen 80 %-Lösungen vollauf.

Structure follows Constraints

Überall lesen Sie den für Designer und Architekten angeblich so wichtigen Leitsatz *Form follows Function*. „Die Form, die Gestaltung von Dingen soll sich aus ihrer Funktion ableiten“ (zitiert nach [wikipedia]). Das mag für Gebrauchsgegenstände und Gebäude eine recht nützliche Herangehensweise sein – für Architektonier jedoch ist dieser Ratschlag, entschuldigen Sie meine überdeutliche Ausdrucksweise, kompletter Unfug. Für sie geht es nämlich primär um Strukturen, Konzepte und Qualitäten – Funktionen werden oftmals sogar zu Nebensächlichkeiten. Der Grund dafür liegt darin, dass die Entscheidungsfreiheit der Architektonier primär von Einschränkungen und Randbedingungen beeinflusst wird, also von leidigen Aspekten wie beispielsweise Performance, Sicherheit, Wartbarkeit, Implementierbarkeit oder Verständlichkeit.

Architekturen sind Strukturen von Bausteinen (und deren Schnittstellen und Interaktionen), zusammen mit übergreifenden Konzepten. In erster Linie prägen Qualitätsanforderungen diese Strukturen und Konzepte, erst in zweiter Linie funktionale Anforderungen.

Schauen Sie als Architekt grundsätzlich zuerst auf die Randbedingungen und Qualitätsanforderungen, denn damit haben Sie in den meisten Fällen schon die wesentlichen Probleme des konkreten Systems im Fokus.

Akzeptieren Sie unvollständige Anforderungen

Streben nach vollständigen und korrekten Anforderungen führt entweder zu Halluzinationen oder ins Paradies. Diese Möglichkeiten sind sehr ungleich verteilt.

Akzeptieren Sie daher ohne Ärger oder Angst unvollständige Anforderungen. Als Architektonier müssen Sie sowieso Requiies-Arbeit leisten, ob Sie wollen oder nicht. Architekten müssen Einflussfaktoren und Randbedingungen klären – egal, ob die Analytiker sehr gute oder gar keine Vorarbeit für Sie geleistet haben. Sie müssen zu den Anforderungen passende Architekturentscheidungen treffen und dafür in den Anforderungen mögliche Widersprüche oder Lücken erkennen, hinterfragen und sie bei Bedarf im Sinne der Kundenier interpretieren. Hierfür benötigen Sie Mut.

Akzeptieren Sie, dass Ihre Kundenier Ihnen ungenaue Anforderungen stellen. Sie meinen es nicht böse. Akzeptieren Sie insbesondere, dass Kundenier Ihnen ungenaue Qualitätsanforderungen stellen – auch das geschieht nicht mit böser Absicht, sondern, weil es sehr schwierig ist, Dinge wie Flexibilität, Wartbarkeit, Verständlichkeit oder Integrationsfähigkeit zu operationalisieren. Als Architektonier müssen Sie das operationalisieren, was die Requiies eigentlich bereits hätten erledigen sollen.

Architektur beeinflusst Anforderungen

Bereits Fragmente von Architekturen beeinflussen in der Regel das Problem oder die Wünsche der Kunden. Der Einfluss ist umso größer, je weniger die Architektur darauf vorbereitet ist.

Sobald ein Kundenier einen Teil eines laufenden Systems zu sehen oder zu spüren bekommt, wird sich seine Meinung zu seinen Anforderungen ändern – sie wird konkreter, erhält andere Prioritäten oder wird um bisher ungekannte Facetten erweitert. Kent Beck (ein sehr berühmter Bürger von Prograland, der zeitweise zum Volksheld in La Testa wurde, dafür aber in Kostenia Einreiseverbot erhielt ...) untertitelte eines seiner lesenswerten Bücher mit „Embrace Change“.

Erfragen Sie aktiv von Ihren Kundeniern aktuelle Anforderungen. Zeigen Sie ihnen regelmäßig nach Iterationsende den Stand der laufenden Software, und hören Sie sorgfältig auf die Rückmeldung. Das ist eine entscheidende Feedbackschleife für erfolgreiche Projekte!

Prüfen Sie Anforderungen

Architektonien kontrolliert den Zugang von Waren und Dienstleistungen ins Land sehr streng, insbesondere solche aus Analytistan. Keine virtuelle oder reale Entität darf ohne Review die Grenze passieren.

Und auf keinen Fall dürfen Anforderungen nach Architektonien gelangen, die nicht vor dem Grenzübergang ein domänenkundiger Architektonier gelesen, verstanden und für gut genug befunden hat.

Manchmal kommen Kundenier auf Ideen, die für Reques wie auch andere Kundenier sehr verlockend klingen, so als müsse die neue Software diese Anforderungen unbedingt erfüllen. Kundige Architektonier sehen solchen Ideen an, dass der Preis für ihre Erfüllung unangemessen hoch liegt – und können die Kundenier darauf hinweisen. (Reques können das nicht, weil sie die Architektur nicht kennen.) Dazu schreiben die Reque-Gurus Suzanne und James Robertson: „Anforderungen, die Sie sich nicht leisten können, sind keine Anforderungen.“

Architektonier sollen also aus ihrer – meist technischen – Sicht die Angemessenheit von Anforderungen prüfen. Falls gewisse Wünsche der Kundenier auffällig schwer zu realisieren sind, extrem lange dauern oder wunderähnlicher Anstrengungen bedürfen, müssen Architektonier sich zu Wort melden: Zuerst argumentieren sie eindringlich und für Kundenier verständlich die Konsequenzen solcher Anforderungen oder Ideen. Sollten die Kundenier auf ihrer Idee beharren, so rechnen Architektonier die Kosten aus – Hilfe leisten hierbei die Managissioms auf Kostenia – und dokumentieren die Entscheidung der Kundenier aufs Genaueste.

Ach ja – wahrscheinlich benötigen Sie all Ihren Mut, um sich der geballten Macht von Kundeniern und Reques in den Weg zu stellen. Und möglicherweise werden diese es Ihnen nicht einmal danken. Als Architektonier können Sie allerdings deutlich ruhiger schlafen, wenn Sie (mal wieder) Ihre Kundenier oder Ihre Projekte vor unangemessenen Anforderungen bewahrt haben.

Ihre Weiterreise

In diesem Nachwort durfte ich Ihnen ansatzweise meine Sicht der IT-Welt präsentieren. Setzen Sie diese Reise nun auf eigene Faust fort. Erinnern Sie sich der Ratschläge des architektonischen Manifestes, falls Sie mal wieder nach Architektonien kommen (oder Bewohner dieses interessanten Landes auf Ihrer Reise treffen). Auf einigen meiner Reisen durfte ich

Manifeste oder Traditionen der übrigen Länder kennen (und schätzen) lernen, und die von mir hoch geschätzten Kollegen [Coplien] haben einen fantastischen Reiseführer geschaffen, in dem sogar das hier unerwähnt gebliebene, sagemumwobene Organisatorien eine Rolle spielen soll. Ich wünsche Ihnen viel Erfolg – und freue mich auf Ihre Reiseberichte!

- [Ambler-12] *Ambler, S.*: www.agilemodeling.com. Ratschläge für schlanke und agile Modellierung.
- [arc42] *Hruschka, P., Starke, G.*: arc42 – Ressourcen für Software-Architekten. <https://arc42.de> beziehungsweise <https://arc42.org>. Praktische Beispiele für den Einsatz von arc42 finden Sie unter <https://leanpub.com/arc42byexample>.
- [arc42-QA] (Freie) Sammlung exemplarischer Qualitätsanforderungen verschiedener Kategorien, <https://github.com/arc42/quality-requirements>.
- [Avram+06] *Avram, A., Marinescu, F.*: Domain Driven Design Quickly. PDF-Version online, <http://infoq.com/books/domain-driven-design-quickly>.
- [Bartonitz06] *Bartonitz, M.*: Wachsen die BPM und Workflow-Lager zusammen? <http://www.bpm-netzwerk.de/articles/66>.
- [Bartonitz10] *Bartonitz, M.*: BPM, GPM, BAM, BPMN, BPEL, XPD, EABPM, CMPM. BPM-Netzwerk.de, online: <http://goo.gl/3YgSI4>.
- [Bass+12] *Bass, L., Clements, P., Kazman, R.*: Software Architecture in Practice. 3. Auflage. Addison-Wesley, 2012.
- [Binder2000] *Binder, R.*: Testing Object-Oriented Systems: Models, Patterns, and Tools. Addison-Wesley, 2000.
- [Brandolini17] *Brandolini, Alberto*: Introducing Event Storming: An act of Deliberate Collective Learning. E-Book: https://leanpub.com/introducing_eventstorming.
- [Brewer00] *Brewer, E.*: CAP-Theorem: <http://www.cs.berkeley.edu/~brewer/cs262b-2004/PODC-keynote.pdf>.
- [Brooks95] *Brooks, F.*: The Mythical Man-Month, Anniversary Edition. Addison-Wesley, 1995.
- [Buschmann+96] *Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P., Stal, M.*: A System of Patterns. Pattern – Oriented Software Architecture. Wiley, 1996.
- [Buschmann+07] *Buschmann, F., Henney, K., Schmidt, D.*: Pattern-Oriented Software Architecture: A Pattern Language for Distributed Computing. Volume 4 der POSA-Serie. Wiley, 2007.
- [Clements+02] *Clements, P., Kazman, R., Klein, M.*: Evaluating Software Architectures – Methods and Case Studies. Addison-Wesley, 2002.
- [Cockburn05] *Cockburn, A.*: Hexagonal Architecture. Online: <http://alistair.cockburn.us/Hexagonal+architecture> (abgerufen Mai 2013).
- [Coplien] *Coplien, J.*: Organisational Patterns. Prentice Hall, 2004.
- [DeMarco98] *DeMarco, T.*: Der Termin – Ein Roman über Projektmanagement. Hanser, 1998.
- [DeMarco+07] *DeMarco, T., Hruschka, P., Lister, T., McMenamin, St., Robertson, J., Robertson, S.*: Adrenalin Junkies & Formular Zombies. Hanser, 2007.
- [Demeyer+02] *Demeyer, S., et al.*: Object-Oriented Reengineering Patterns. Morgan Kaufmann, 2002.

- [Dern09] *Dern, G.*: Management von IT-Architekturen. Informationssysteme im Fokus von Architekturplanung und -entwicklung. 3. Auflage. Vieweg, Edition CIO, 2009.
- [doctoolchain] Eine Open-Source-Werkzeugkette zur effektiven Dokumentation auf Basis AsciiDoc. Online: <https://rdmueller.github.io/docToolchain/>
- [Edlich+11] *Edlich, S., Friedland, A., Hampe, J., Brauer, B.*: NoSQL. Einstieg in die Welt nichtrelationaler Web 2.0 Datenbanken. 2. Auflage. Hanser, 2011.
- [Eilebrecht+19] *Eilebrecht, K., Starke, G.*: Patterns kompakt – Entwurfsmuster für effektive Software-Entwicklung. 5. Auflage. Springer, 2019. Siehe auch <http://patterns-kompakt.de>.
- [Evans04] *Evans, E.*: Domain Driven Design. Addison-Wesley, 2004.
Zugehörige Website: www.domaindrivendesign.org.
- [Feathers-13] *Feathers, Michael*: Working Effectively with Legacy Code. Prentice Hall, 2013.
- [Fowler99] *Fowler, M.*: Refactoring: Improving the Design of Existing Code. Addison-Wesley, 1999.
- [Fowler02] *Fowler, M.*: Patterns for Enterprise Application Architecture. Addison-Wesley, 2002.
- [Fowler+14] *Fowler, M., Lewis, J.*: Microservices – A definition of this new architectural term. Online: <https://martinfowler.com/articles/microservices.html>
- [Freeman+10] *Freeman, St., Pryce, N.*: Growing Object-Oriented Software, Guided by Tests. Addison-Wesley, 2010.
- [Gamma95] [GoF] *Gamma, E., Helm, R., Johnson, R., Vlissides, J.*: Design Patterns. Addison-Wesley, 1995.
- [Gupta2003] *Gupta, S.*: Logging in Java with the JDK 1.4 Logging API and Apache log4j. Apress.
- [Hargis+04] *Hargis, G., Caray, M., Hernandez, A.*: Technical Quality Technical Information: A Handbook for Writers and Editors. Prentice Hall, 2004.
- [Hofmeister2000] *Hofmeister, C., Nord, R., Soni, D.*: Applied Software Architecture. Addison-Wesley, 2000.
- [Hohpe+04] *Hohpe, G., Woolf, B.*: Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. Addison-Wesley, 2004.
- [htmlsc] HTML Sanity Check – Ein (Open-Source-)Beispiel für AsciiDoc-basierte Architekturdokumentation. Online: <https://github.com/aim42/htmlSanityCheck>
- [IEEE2000] *IEEE Architecture Working Group*: IEEE Recommended Practice for Architectural Description, Standard P1471, www.pithecantropus.com/~awg.
- [ISAQB] International Software Architecture Qualification Board e. V., www.isaqb.org.
- [Keller97] *Keller, W.*: Mapping Objects to Tables – A Pattern Language. Beitrag zur EuroPloP Konferenz 1997, <http://www.objectarchitects.de/ObjectArchitects/papers/>.
- [Keller98] *Keller, W.*: Object/Relational Access Layers – A Roadmap, Missing Links and More Patterns. EuroPloP 1998, www.objectarchitects.de/ObjectArchitects/papers/Published/ZippedPapers/or06_proceedings.pdf.
- [Keller2000] *Keller, W.*: The Bridge to the New Town – A Legacy System Migration Pattern. EuroPloP 2000, <http://www.objectarchitects.de/ObjectArchitects/papers/WhitePapers/ZippedPapers/pacman03.pdf>.
- [Keller02] *Keller, W.*: Enterprise Application Integration: Erfahrungen aus der Praxis. dpunkt, 2002.
- [Keller17] *Keller, W.*: IT-Unternehmensarchitektur: Von der Geschäftsstrategie zur optimalen IT-Unterstützung. 3., erweiterte Auflage. dpunkt, 2017.
- [Kim+13] *Kim, G., Behr, K., Spafford, G.*: The Phoenix Project. A Novel About IT, DevOps and Helping Your Business Win. IT-Revolution Press, Portland, Oregon, 2013.
- [Knigge-14] *Hruschka, P., Starke, G.*: Knigge für Softwarearchitekten: Entwickler-Press, 3. Auflage 2018.

- [Kruchten95] *Kruchten, P.*: Architectural Blueprints – The 4-1 View Model of Architecture. IEEE Software November 1995; 12(6), p. 42–50.
- [Kruchten2001] *Kruchten, P.*: The Top Ten Misconceptions about Software Architecture. <http://www.ics.uci.edu/~andre/ics223w2006/kruchten1.pdf>.
- [Kruchten+02] *Kruchten, Ph., et al.*: Software Architecture Review and Assessment (SARA) Report. Online verfügbar unter philippekruchten.com/architecture/SARAv1.pdf.
- [Lilienthal] *Lilienthal, C.*: Langlebige Softwarearchitekturen. dpunkt, 2019.
- [Longshaw+04] *Longshaw, A., Woods, E.*: Patterns for Generation, Handling and Management of Errors. Konferenzbeitrag zur OT2004, <http://www.blueskyline.com/ErrorPatterns/ErrorPatternsPaper.pdf>.
- [Marick97] *Marick, B.*: Classic Testing Mistakes. 1997. Online unter <http://www.exampler.com/testing-com/writings/classic/mistakes.html>.
- [Marick2000] *Marick, B.*: Using Ring Buffer Logging to Help Find Bugs. PloP 2000. <http://www.exampler.com/writing/ring-buffer.pdf>.
- [Martin2000] *Martin, R.*: Design Principles and Design Patterns. http://www.objectmentor.com/resources/articles/Principles_and_Patterns.pdf.
- [Martin08] *Martin, R.*: Clean Code: A Handbook of Agile Software Craftmanship. Addison-Wesley, 2008.
- [Martin02] *Martin, R.*: Agile Software Development: Principles, Patterns and Practices. Addison-Wesley, 2002.
- [Miller09] *Miller, J.*: Design Patterns for Data Persistence. MSDN Magazin, April 2009. Online: <http://msdn.microsoft.com/en-us/magazine/dd569757.aspx>.
- [Möricke04] *Möricke, M. (Hrsg.)*: IT-Sicherheit. Praxis der Wirtschaftsinformatik. dpunkt, 2004.
- [Murer+11] *Murer, S., et al.*: Managed Evolution – A Strategy for Very Large Information Systems. Springer, 2011.
- [NATO069] *NATO Conference Software Engineering*: Bericht einer Konferenz, 1969. Gesponsort vom NATO Science Committee. Quelle nach [Shaw96a].
- [Newman 15] *Newman, Sam*: Microservices: Konzeption und Design. mitp, 2015.
- [Newman 20] *Newman, Sam*: Monolith to Microservices: Evolutionary Patterns to Transform your Monolith. O'Reilly, 2020.
- [OMG_UML] *Object Management Group*: Unified Modeling Language. www.omg.org/uml.
- [Open Group 09] *The Open Group*: TOGAF, Version 9. Online: www.opengroup.org/architecture/togaf9-doc/arch/.
- [Parnas72] *Parnas, D. L.*: On the Criteria to be Used in Decomposing Systems into Modules. Communications of the ACM 1972; 15: p. 1053–1058.
- [Quiang+10] *Kai Quiang, Xiang Fum, et al.*: Software Architecture and Design Illuminated. Jones and Bartlett Publishers, 2010.
- [Rechtin2000] *Rechtin, E., Maier, M.*: The Art of Systems Architecting. CRC Press, 2000.
- [Redmond+12] *Redmond, E., Wilson, J.*: Seven Databases in Seven Weeks. A Practical Guide to Databases and the NoSQL Movement. Pragmatic Programmers, 2012.
- [Robertson 12] *Robertson, S., Robertson, J.*: Mastering the Requirements Process: Getting Requirements Right. 3. Auflage. Addison-Wesley, 2012.
- [Ross+06] *Ross, J. W., Weill, P., Robertson, D.*: Enterprise Architecture as Strategy. Harvard Business School Press, 2006.
- [Rozanski+05] *Rozanski, N., Woods, E.*: Software Systems Architecture: Working with Stakeholders Using Viewpoints and Perspectives. Pearson Education, 2005.

- [Rupp+12] *Rupp, C., Queins, S., die SOPHISTen: UML 2 glasklar. Praxiswissen für die UML-Modellierung.* 4. Auflage. Hanser, 2012.
- [Schmidt2000] *Schmidt, D., Stal, M., Rohnert, H., Buschmann, F.: Pattern-Oriented Software-Architecture – Patterns for Concurrent and Networked Objects.* Vol. 2. Wiley, 2000.
- [Schneier96] *Schneier, B.: Applied Cryptography.* John Wiley, 1996.
- [Schumacher06] *Schumacher, M., et al.: Security Patterns: Integrating Security and Systems Engineering,* Wiley 2006.
- [SEI2001] *Software Engineering Institute (SEI): How Do You Define Software Architecture?* <http://www.sei.cmu.edu/architecture/definitions.html>.
- [Shaw+96] *Shaw, M., Garlan, D.: Software Architecture: Perspectives on an Emerging Discipline.* Prentice Hall, 1996.
- [Tanenbaum+12] *Tanenbaum, A., Wetherall, D.: Computer Networks.* Prentice Hall, 5. Auflage 2012.
- [Tidwell-2011] *Tidwell, J.: Designing Interfaces.* O'Reilly, 2011.
- [Tilkov-15] *Tilkov, S.: REST und HTTP: Einsatz der Architektur des Web für Integrationsszenarien.* 3. Auflage. dpunkt, 2015.
- [Toth-19] *Toth, S.: Vorgehensmuster für Softwarearchitekten.* Hanser, 3. Auflage 2019.
- [vanSteen+17] *van Steen, M., Tanenbaum, A.: Distributed Systems.* Self-published, kostenfrei unter <https://www.distributed-systems.net/>
- [VDAalst-2002] *van der Aalst, W.: Workflow Patterns Website, <http://workflowpatterns.com>.*
- [Vernon16] *Vernon, V.: Domain-Driven Design Distilled.* Addison-Wesley, 2016.
- [Völter-2013] *Völter, M.: DSL Engineering.* Online: <http://dslbook.squarespace.com/>.
- [Volere] *Volere – Schablonen für Requirements Engineering.* Online unter www.volere.co.uk (englisch) oder www.volere.de (deutsch).
- [Williams-2008] *Williams, R.: The Non-Designers Design Book.* PeachPit Press, 2008.
- [Wolff 18] *Wolff, E.: Das Microservices Praxisbuch.* dpunkt, 2018.
- [Zörner15] *Zörner, S.: Softwarearchitekturen dokumentieren und kommunizieren – Entwürfe, Entscheidungen und Lösungen nachvollziehbar und wirksam festhalten.* 2. Auflage. Hanser, 2015.

Stichwortverzeichnis

A

- Abhängigkeiten 59
 - zyklische 70
- Ablaufsteuerung 264
- Abschnitte, UML 186
- Abstraktionen 18, 68
 - Modellierung 183
- ACID 210
- Adapter 131
 - zur Integration 240
- Aggregate 88
- Agilität 6, 155
- aim42 324
- Aktivitätsdiagramm 187
- Änderungsszenarien 43
- Anforderungen
 - an Architekturdokumentation 142
 - an Schnittstellen 99
 - klären 38
- Anforderungsanalyse 29
- Anforderungsmanagement, Werkzeuge 409
- Anpassbarkeit 94
- Anwendungsfalldiagramm 188
- Anwendungslandschaft 351
 - Management der 354
- Anwendungsszenarien 43
- arc42 148, 149, 150
- Architecture Business Cycle 27
- Architecture Tradeoff Analysis Method 311
- Architekten
 - Aufgaben von 26
 - beraten 22
 - bewerten 24
 - entscheiden 21
 - kommunizieren 24
 - konstruieren 21
- Architektenaufgaben 53
- Architektonien 429
- Architektur
 - Bausteine 189
 - Beispiele 361
 - Bewertung 307
 - Business- 351
 - Definition 16
 - Dokumentation 139
 - Enterprise 351
 - hexagonal 113
 - Kommunikation 139
 - mit arc42 148, 149, 150
 - mit UML 2 186
 - nach Qualitätszielen 89
 - qualitativ bewerten 310
 - Unternehmens- 351
 - Werkzeuge 409
- Architekturaspekte 201
- Architekturbeschreibung, zentrale 147
- Architekturbewertung 307
 - als Teil der Architekturentwicklung 308
- ATAM 311
- Auswirkung von 316
- Vorgehen 311
- Architekturdokumentation 139, 147
 - Anforderungen 142
 - Beispiel 361
 - Grundannahmen 153
 - Regeln 144
- Architekturebenen 351
- Architekturentscheidung 16
- Architekturentwicklung, Vorgehen bei der 33
- Architekturkomitee 28
- Architekturmuster 102, 253
 - für GUI 253
- Architektursicht 152
 - Bausteinsicht 161
 - Laufzeitsicht 168
 - Verteilungssicht 171
- Architekturstil
 - Batch-Sequentiell 103

- Broker 119
- datenzentriert 107
- Event-Queue 122
- hexagonal 113
- hierarchisch 109
- interaktionsorientiert 124
- Master-Slave 109
- Messaging 122
- Microservices 340
- Model-View-Controller 124
- Peer-to-Peer 120
- Pipes und Filter 104
- Ports-und-Adapter 113
- REST 128
- Schichten/Layer 110
- Architekturtapete 152
- Architekturüberblick 151
- asynchron 247
- ATAM 311
 - Qualitätsbaum 313
- Aufgaben von Softwarearchitekten 21
- Ausbildung von Softwarearchitekten 415
- Authentifizierung 273
- Authentizität 277
- Autorisierung 273

B

- BASE 210
- Batch-Sequentiell 103
- Bausteine
 - Schnittstellen 190
 - von Architekturen 189
- Bausteinsicht 161, 162, 191
- hierarchische Verfeinerung 163
- UML 2 191
- Beispiele
 - Architekturdokumentation 361
 - von Architekturen 361
- Benutzeroberfläche (GUI) 251
- Beobachter 132
- Beschreibung von Schnittstellen 174
- Betriebbarkeit 41
- Bewertung 310
 - ATAM 311
 - qualitativ 317
 - quantitativ 317
 - Soll-Ist-Vergleich 308
 - von Architekturen 307
 - von Artefakten 308
 - von Prozessen 308

- Werkzeuge 319
- Blackboard 108
- Blackbox 162, 164
- Bottom-up 97
- Broadcast 248
- Broker 119
- Budget 49
- Buildmanagement, Werkzeuge 411
- Business-Architektur 351
- Business-IT-Alignment 358
- Business Process Management 258

C

- Caching 224
- CAP-Eigenschaften 211
- CAP-Theorem 210
- Chaos 5
- Cluster von Fehlern 83
- Codeanalyse, Werkzeuge 411
- Command-Query-Responsibility-Segregation 117
- Conway 27
- CORBA 238
- CPSA-F 416
- CQRS 117
- Curriculum 415
- cyclomatic complexity 317

D

- DAO 217
- Data Access Object 217
- Data-Binding in GUI 256
- Dateitransfer 236
- Datenbanken 206
- Datenklassen 213
- datenzentrierter Architekturstil 107
- Decorator 133
- Definition, Softwarearchitektur 16
- Dekorierer 133
- Denial-of-Service 273
- Dependency Injection 77
- Dependency Inversion Principle 71
- Design 19
 - verfaultes, Symptome 60
- Diagnostizierbarkeit 291
- Diagrammarten, UML 2 187
- Diagramm, Kontext als 46
- digitale Signatur 277
- DIN/ISO 25010 41
- DIP 71

Dokumentation 139

- Grundprinzipien 145
- Konzepte 177
- nach arc42 148, 149, 150
- Qualitätsanforderungen 144
- von Schnittstellen 174
- Werkzeuge 177

Dokumentationsübersicht 151

Dokumente zur Beschreibung von Architekturen 147

Domain-Driven Design 84

Domain-Specific Languages 185

Domänenmodell 84

Don't Repeat Yourself 69

DSL 185

E

Effektiv 8

Effizienz 8, 41

Einfachheit 62

Einflussfaktoren 47

- organisatorische 48
- technische 51

Elastizität 294

Enterprise-Architektur 351

Entwurf 59

- Grundsätze 62
- Maxime 61
- objektorientierter 71
- Prinzipien 65
- QDSA 89

Entwurfsentscheidung 16

Entwurfsmethoden 84

Entwurfsmuster 130

- Adapter 131
- Beobachter 132
- Decorator 133
- Dekorierer 133
- Fassade 135
- Observer 132
- Proxy 134
- State 136
- State (Zustand) 136
- Stellvertreter 134
- Zustand 136

Entwurfsprinzip

- abhängig nur von Abstraktionen 76
- Dependency Injection 77
- Liskov 73
- Schnittstellen abtrennen 74

- Substitutionsprinzip 73

ereignisbasierte Systeme 121

Ergonomie 254

Evaluierung von Architekturen 307

Event-Queue 122

Event Systems 121

Evolution 321

Exceptions 286, 287

explizit 63

F

Fachdomäne 84

Fachlichkeit und Technik trennen 82

Factories 88

Faktoren

- juristische 50
- organisatorische 48
- technische 51

Fassade 135

- zur Integration 240

Fehler 83

- Cluster 83

Fehlerbehandlung 286, 287

Fehlerkategorien 288

Fingerprint 277

Flexibilität 94

Foundation-Level 417

Fragen an Architekturdokumentation 142

Funktionale Eignung 41

G

Geheimnisprinzip 68

Generalisierung 80

Generierung, Werkzeuge 410

Geschäftsprozesse, Informationsbedarf 351

Geschäftsregeln 226

Geschäftsziele bei Architekturbewertung 312

grafische Oberflächen 251

Groovy 54

Grundprinzipien von Dokumentation 145

Grundsätze des Entwurfs 62

GUI 251

GUI-Idiome 252

GUI-Plattformen 253

H

Hardwarearchitektur 31

Hashfunktion 277

Heuristik 79

- Beachte Fehler-Cluster 83

- Fachlichkeit und Technik trennen 82
- Generalisieren 80
- Kenne die Risiken 83
- Perspektive wechseln 81
- Recherche 79
- Schnittstellen beachten 82
- Spezialisieren 81
- Trial-and-Error 80
- Verfeinere schrittweise 79
- hexagonale Architektur 113
- hierarchischer Architekturstil 109
- Homogenisierung 358
- HTTP 249

I

- Idiome für GUI 252
- IEEE-1471 16
- IIO 250
- impedance mismatch 219
- Implementierung 30
- implizit 63
- Information Hiding 68
- Informationsarchitektur 351
- Infrastruktursichten 171
- Innovation 201
- Integration 233
 - Frontend 234
- Integrität 272
 - konzeptionelle 20
 - von Daten 277
- interaktionsorientierte Systeme 124
- Interaktionsübersichtsdiagramm 188
- Interface Segregation Principle 71
- Internationalisierung 257
- iSAQB 415
- iSAQB-Foundation-Level 418
- ISP 71
- Iterationen 27
 - beim Entwurf 35
- IT-Infrastruktur 352
- IT-Sicherheit 271

J

- juristische Faktoren 50

K

- Kai-Zen 54
- Kapselung 68
- Kategorien
 - von Systemen 39

- von Werkzeugen 409
- Keep It Small and Simple 62
- KISS *siehe* Keep It Small and Simple
- Klassendiagramm 187
- Klassen, UML 2 189
- Knoten 172
 - UML 2 194
- Kohäsion 66
- Kommunikation 139, 246
 - asynchron 247
 - sync/async 247
 - von Architekturen 140
- Kommunikationsaufgabe 140
- kommunizieren 53
- Kompatibilität 42
- Komplexität 79
- Komponentendiagramm 187
- Komponenten, UML 2 189
- Konsistenz 20, 69, 210
- Kontext 45
 - als Diagramm 46
- Kontextabgrenzung 159
- Konzepte 201
 - Dokumentation 177
 - technische 201
- konzeptionelle Integrität 20
- Kopplung 65, 66
- Kryptografie 275

L

- Laufzeitsicht 168
 - UML 2 195
- Layer 110
- Lebenszyklus 20
- Legacy 233
- Lehrplan 416
- Liskov Substitution Principle 71
- Lösungsidee 38
- Lösungsstrategien 55
- LSP 71

M

- Master-Slave 109
- Maxime des Entwurfs 61
- Mediator zur Integration 240
- Message Digest 277
- Message Oriented Middleware 238
- Message Queues 247
- Messaging 122, 238
- Messgröße für Softwarearchitekturen 310

Metriken 317
 – für Quellcode 317
 – Software 317
 Microservices 339, 343, 344
 Mindmaps als Hilfsmittel für Qualitätsbäume 313
 Modelle 183
 – fachlich 84, 85, 86
 – textbasiert 185
 Modellierung 183
 – Bausteine 189
 – grafisch 185
 – Klassendiagramm 187
 – Komponentendiagramm 187
 – Laufzeitsicht 195
 – Paketdiagramm 187
 – Schnittstellen 190
 – textuell 185
 – Tipps 199
 – Verteilung (Deployment) 193
 – Verteilungsdiagramm 187
 – Werkzeuge 410
 Model-View-Controller 124
 Modularität 68
 Module, Zerlegung in 68
 Moving Target 27
 Murphys Regel 286
 MVC 124

N

NoSQL 207
 Notationen, grafisch/textuell 185
 Nutzen und die Ziele von Softwarearchitektur 20

O

OAuth 278
 objektorientierter Entwurf 71
 Observer 132
 OCP 71
 Offen-Geschlossen-Prinzip 71
 Open-Closed Principle 71
 organisatorische Faktoren 48
 organisatorische Risiken 55
 organisatorische Standards 50
 OR-Mapping 219
 OSI-7-Schichten-Modell 250

P

Paketdiagramm 187
 Pakete, UML 2 189

Partitionstoleranz 211
 Peer-to-Peer 120
 Performance 93
 Persistenz 205
 – Einflussfaktoren 208
 Persistenzschicht 213
 Perspektive 81
 Pipes und Filter 104, 105
 Ports-und-Adapter 113
 Presentation Model 125
 Prinzipien
 – des Entwurfs 65
 – SOLID 71
 private statement *siehe* Geheimnisprinzip
 Projektplanung 30
 Protokollierung 283
 Proxy 134
 Publish-Subscribe 248

Q

Qualität 39
 qualitative Bewertung 310
 Qualitätsanforderungen 39, 43
 Qualitätsbaum 313
 – ATAM 313
 – Szenarien konkretisieren 314
 Qualitätskriterien als Bewertungsziel 310
 Qualitätsmerkmale 41, 43, 310, 314
 Qualitätssicherung 32
 Qualitätsszenarien 91
 Qualitätsziele 35, 90
 Quality-Driven Software Architecture 89
 quantitative Bewertung 317

R

Randbedingungen 47
 Redundanz 69
 Referenzarchitekturen 355, 356
 Regelmaschine (rule engine) 229
 Regelsysteme 229
 Remote Procedure Calls 237, 248
 Reorganisation 233
 Repositories 88
 Repository 107
 Ressourcen 49, 174
 REST 128
 REST-Architekturstil 174
 Risiken, organisatorische 55
 Risikoanalyse 31
 Risikomanagement 31

Rolle von Softwarearchitekten 15
RPC 248

S

Scale-Up 294
Schalenmodell 115
Schichten 110
Schnittstellen 45, 82, 98, 174
– Anforderungen 99
– Dokumentation von 174
– in UML beschreiben 175
– spezifische 74
– UML 2 190
– von Bausteinen 190
Secure Socket Layer 280
Security 271
Separation of Concern 67
Sequenzdiagramm 188, 196
Sicherheit 42, 271
Sicherheitsziele 272
Sichten 17, 152
– 4 Arten 155
– Arten von 155
– Baustein- 161
– für agile Architekturen 154
– in der Softwarearchitektur 153
– -Kontextabgrenzung 159
– -Laufzeit 168
– neue Arten 156
– -Verteilung 171
Signaturen, digitale 277
Single Responsibility Principle 71
Skalierbarkeit 293
Skalierung bei NoSQL 210
Skalierungsstrategien 294
S/MIME 280
Smoketest 20
SOAP 249
Softwarearchitekten 21
– Aufgaben 21
– Aufgaben von 21, 26
Softwarearchitektur 16
– Ausbildung 415
– Bewertung 307
– Definition 16
– Dokumentation und Kommunikation 140
– Iterationen 27
– Nutzen und Ziele 20
– Rolle 15
– Sichten 18, 152

– Sichten in der 153
– und Qualität 39
SOLID 71
Soll-Ist-Vergleich 308
Speichermodell 205
Speicherung 205
Spezialisierung 81
spezifische Schnittstellen 74
SRP 71
Stakeholder 44
– bei Architekturbewertung 311
– maßgebliche 311
Standardisierung 358
Standards, organisatorische 50
Starrheit 60
State 136
Stellvertreter 134
Strategien 55, 351
Strukturbruch bei Persistenz 219
Strukturen 16
Substitutionsprinzip 73
Symptome von verfaultem Design 60
synchron 247
Systeme
– ereignisbasierte 121
– interaktionsorientierte 124
– Kategorien 39
Szenarien 43
– konkretisieren Qualität 43
– zur Bewertung 314

T

TCP/IP 249
technische Faktoren 51
Test, Werkzeuge 412
Thrift 249
TOGAF 179
Top-down 97, 163
Tracing 283
Transaktionen 223
– ACID 210
Transport Layer Security 280
Trial-and-Error 80

U

Übersichtspräsentation 148
Übertragbarkeit 43
ubiquitous language 84
UML
– für Bausteinsicht 166

- für Laufzeitsicht 169
- für Schnittstellen 175
- UML 2 186
 - Aktivitäten 192
 - Diagrammarten 187
 - für Softwarearchitekten 186
 - Interaktionsübersicht 197
 - Klassen und Objekte 189
 - Knoten 194
 - Kommunikationsdiagramm 197
 - Laufzeitsicht 195
 - Schnittstellen 190
 - Verteilung 193
 - Zustände 192
- Umsetzung begleiten 54
- Unternehmensarchitektur 351

V

- Verallgemeinerungen 80
- Verantwortlichkeit trennen 67
- Verbesserung 321
- Verbesserungsprojekte 332
- verfaultes Design, Symptome 60
- Verfügbarkeit 96, 210, 273
- Verschlüsselung 274
 - symmetrisch/asymmetrisch 275
- Verschlüsselungsverfahren 275
- Verständlichkeit 18
- Verteilung 242
- Verteilungsdiagramm 187
- Verteilungssicht 171
 - UML 2 193
- Vertraulichkeit als Sicherheitsziel 272
- Virtual Private Networks 280

Vorgehen

- bei der Architekturentwicklung 33
- zur Architekturbewertung 311

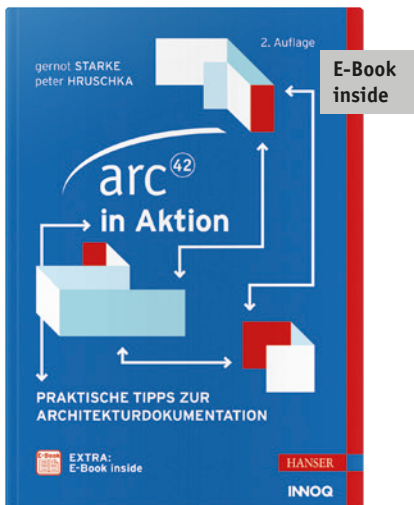
W

- Walkthrough von Szenarien 315
- Wartbarkeit 42
- Wasserfall 2
- Webseite 12
- Website: <http://arc42.de> 12
- Website: <https://esabuch.de> 12
- Werkzeuge 179, 409
 - Auswahlkriterien 413
 - Buildmanagement 411
 - Codemanagement 410
 - für Architekten 409
 - Kategorien 409
 - Test 412
 - zur Bewertung 319
 - zur Dokumentation 177
- Werkzeugkategorien 409
- Whitebox 162, 164
- Workflow 260

Z

- Zerbrechlichkeit 60
- Zerlegung in Module 68
- Zertifikate 279
- Zertifizierung zum CPSA-F 416
- Zugriffe auf Daten 212
- Zustand 136
- Zustandsdiagramm 188
- Zyklen in Abhängigkeiten 70
- zyklische Abhängigkeiten 70

Softwaredokumentation – aber richtig!



Starke, Hruschka

arc42 in Aktion

Praktische Tipps zur Architekturdokumentation

2., überarbeitete Auflage

220 Seiten. Inklusive E-Book

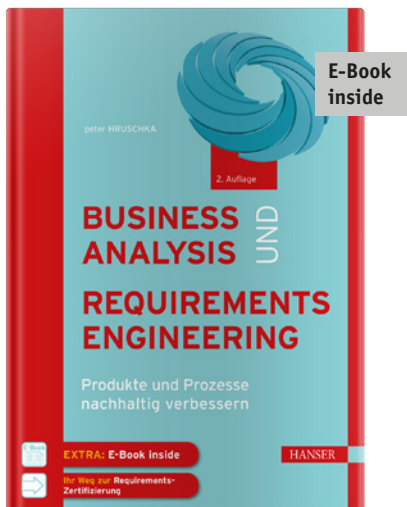
€ 29,99. ISBN 978-3-446-46380-6

Auch einzeln als E-Book erhältlich

Dieses Buch zeigt Ihnen den Praxiseinsatz von arc42 – dem Template für Softwarearchitekturen. Hier finden Sie konkrete Maßnahmen und Praktiken, um arc42 sowohl zur effektiven Kommunikation und Dokumentation wie auch zur Konstruktion und Entwicklung von Systemen anzuwenden.

Softwarearchitekten und -entwickler ziehen daraus unmittelbaren Nutzen für ihre tägliche Arbeit.

Professionell mit Anforderungen umgehen



Hruschka

Business Analysis und Requirements Engineering
Produkte und Prozesse nachhaltig verbessern

2., aktualisierte Auflage

361 Seiten. Inklusive E-Book

€ 36,-. ISBN 978-3-446-45589-4

Auch einzeln als E-Book erhältlich

Wir alle wollen schlanke Geschäftsprozesse und optimale IT-Unterstützung. Wir finden für jedes Problem eine Lösung – wenn wir uns nur genau darauf einigen könnten, was unser Problem ist. Das Verstehen von Problemen und Formulieren von Anforderungen, was wir gerne anders hätten, ist Thema dieses Buches.

Es zeigt einen integrierten Ansatz zum Umgang mit Anforderungen. Es stellt Ihnen Methoden, Notationen und viele pragmatische Tipps (Best Practices) zur Verfügung, mit denen Anforderungen effektiv zwischen Auftraggebern und Auftragnehmern behandelt werden können – von Entdeckungstechniken über Dokumentationstechniken, Prüftechniken bis hin zu Verwaltungstechniken.

Mehr Informationen finden Sie unter www.hanser-fachbuch.de

EFFEKTIVE SOFTWARE-ARCHITEKTUREN //

- Überblick und methodische Einführung
- Direkt umsetzbare Tipps für die Praxis
- Ideal zur Vorbereitung auf die Zertifizierung zum »Certified Professional for Software Architecture®« (Foundation Level) des iSAQB
- Praxisnahe Darstellung von Architekturstilen und -mustern, ausführliche technische Konzepte, NoSQL-Datenbanken, Microservices sowie Beispielarchitekturen
- Detaillierte Beispiele zum Einsatz von arc42

Softwarearchitekt*innen müssen komplexe fachliche und technische Anforderungen an IT-Systeme umsetzen und sie müssen diese Systeme durch nachvollziehbare Strukturen flexibel und erweiterbar gestalten.

Dieser Praxisleitfaden zeigt Ihnen, wie Sie Softwarearchitekturen effektiv und systematisch entwickeln können. Gernot Starke unterstützt Sie mit praktischen Tipps, Architekturmustern und seinen Erfahrungen.

Sie finden Antworten auf zentrale Fragen:

- Welche Aufgaben gehören zur Softwarearchitektur?
- Wie kann ich beim Entwurf vorgehen?
- Wie kommuniziere und dokumentiere ich Softwarearchitekturen?
- Wie helfen Architekturstile und -muster?
- Wie analysiere und bewerte ich Softwarearchitekturen?
- Wie setze ich Persistenz, grafische Benutzeroberflächen, Geschäftsregeln, Integration, Verteilung, Sicherheit, Fehlerbehandlung, Business-Process-Management, Blockchain und andere Konzepte ein?
- Was muss ich über Domain-Driven Design, Microservices und arc42 wissen?
- Wie verbessere ich bestehende Systeme?



Dr. Gernot **STARKE** stellt sich seit vielen Jahren der Herausforderung, die Architektur großer Systeme effektiv zu gestalten. Zu seinen Kunden zählen mittlere und große Unternehmen aus den Branchen Finanzdienstleistung, Logistik, Handel, Telekommunikation und aus dem öffentlichen Bereich. Er ist Mitinitiator und -betreiber von arc42, Mitgründer des iSAQB e.V. sowie Fellow von INNOQ.

AUS DEM INHALT //

- Vorgehen bei der Architekturentwicklung
- Architekturmuster und -stile
- Technische Konzepte
- Microservices
- Blockchain
- Architekturanalyse und -bewertung
- Dokumentation von Architekturen
- Modernisierung bestehender Systeme
- Beispiele realer Softwarearchitekturen
- iSAQB Curriculum

HANSER

